

**UN INSTRUMENT D'AIDE À LA RÉDACTION POUR  
LES DOCUMENTS EXPRIMANT DES MODÈLES  
LOGICIELS**

par

Maxime Beauchesne-Jolin

Mémoire présenté au Département d'informatique  
en vue de l'obtention du grade de maître ès sciences (M.Sc.)

FACULTÉ DES SCIENCES  
UNIVERSITÉ DE SHERBROOKE

Sherbrooke, Québec, Canada, 12 septembre 2026

Le 20 août 2026

Le jury a accepté le mémoire de Maxime Beauchesne-Jolin dans sa version finale

**Membres du jury**

Prof. Luc Lavoie, *Directeur*  
Département d'informatique

Prof. Sylvain Giroux, *Membre interne*  
Département d'informatique

Prof. Christina Khnaisser, *Présidente-rapporteuse*  
Département d'informatique

# Sommaire

Les problèmes en lien avec la documentation sont récurrents en informatique appliquée. Cette situation soulève de nombreuses interrogations qui conduisent à revenir aux fondamentaux. L'informatique appliquée comprend la création et la modification de modèles. Ces modèles, du moins, leurs expressions, sont la plupart du temps disséminés à travers la documentation informatique. Puisque les documents contiennent les modèles, ils sont les piliers des systèmes et des logiciels développés. Cependant, il faut reconnaître que le développement de la documentation informatique ne s'est pas réalisé avec la même rigueur et le même succès que l'algorithmique ou la modélisation de données. Ce mémoire tente d'en donner les raisons et de proposer une solution.

La solution décrite dans ce mémoire répond au souhait d'avoir une documentation partageant certaines des caractéristiques de qualité des logiciels. Cela est possible dès lors que la documentation comme le logiciel concrétisent des modèles. Ainsi, plusieurs approches provenant de la documentation informatique et de la modélisation informatique sont utilisées. Il y a l'utilisation, entre autres, de la programmation littéraire, de langages formels et de méthodes formelles. Ainsi, la solution obtenue prend la forme d'un instrument d'aide à la rédaction. Cet instrument permettra aux utilisateurs de maintenir la documentation cohérente (et donc le modèle) et de l'adapter aux différents changements qui peuvent survenir lors d'un procédé de développement ou après.

**Mots-clés:** documentation, méthodes formelles, définitions de langages formels, compilateurs, modèles de systèmes logiciels, langages de description de systèmes, méthodologies de modélisation, ingénierie logicielle pilotée par les modèles

# Remerciements

Ce document est en partie le résultat d'un long processus dans lequel j'ai eu de nombreuses réflexions sur l'informatique. Si mes nombreuses années d'expérience en informatique n'ont pas entamé ma curiosité, elles ont en revanche attisé mes suspicions envers le domaine. Ainsi, j'espère que ce travail permettra au moins de ne plus tergiverser sur l'importance que peut avoir la documentation pour l'informatique appliquée.

Lors de ces différentes réflexions, j'ai eu l'immense privilège d'être accompagné par le professeur Luc Lavoie. À vrai dire, il est rare de pouvoir côtoyer un érudit et de profiter de son savoir, aussi je ne saurai pas de quelle manière lui témoigner toute ma reconnaissance. Je désire aussi souligner l'effort de Maryse Couture à la révision linguistique de ce document, cela n'a probablement pas été de tout repos.

Je souhaite également exprimer ma gratitude à tous les professionnels avec lesquels j'ai pu échanger sur l'informatique, ainsi qu'aux professeurs et chargés de cours qui ont généreusement partagé avec moi leurs connaissances et leur lucidité. Vous m'avez tous permis de nourrir mes réflexions.

Je ne pourrais conclure ces remerciements sans souligner l'incommensurable patience de toute ma famille (Isabelle, Zachary, Anthony et Mélodie) qui m'a permis paisiblement de m'instruire et de rédiger ce mémoire.



# Table des matières

|                                                           |      |
|-----------------------------------------------------------|------|
| Sommaire                                                  | iii  |
| Remerciements                                             | iv   |
| Table des matières                                        | v    |
| Liste des figures                                         | viii |
| Liste des tableaux                                        | xi   |
| Liste des abréviations                                    | xiii |
| Introduction                                              | 1    |
| 1 Revue de littérature                                    | 4    |
| 1.1 Des documents en informatique appliquée . . . . .     | 5    |
| 1.1.1 Des objectifs . . . . .                             | 6    |
| 1.1.2 Des écueils . . . . .                               | 10   |
| 1.1.3 Des approches . . . . .                             | 18   |
| 1.2 Des modèles en informatique appliquée . . . . .       | 33   |
| 1.2.1 Des objectifs . . . . .                             | 34   |
| 1.2.2 Des écueils . . . . .                               | 35   |
| 1.2.3 Des approches . . . . .                             | 40   |
| 1.3 Des connaissances en informatique appliquée . . . . . | 49   |
| 1.3.1 Des objectifs . . . . .                             | 50   |
| 1.3.2 Des écueils . . . . .                               | 52   |

## TABLE DES MATIÈRES

|          |                                                                |            |
|----------|----------------------------------------------------------------|------------|
| 1.3.3    | Des approches . . . . .                                        | 61         |
| <b>2</b> | <b>Une définition du problème</b>                              | <b>69</b>  |
| 2.1      | Une présentation du problème . . . . .                         | 69         |
| 2.2      | Une modélisation du problème . . . . .                         | 71         |
| 2.3      | Des attentes à considérer pour la solution . . . . .           | 78         |
| 2.4      | Des besoins à considérer pour la solution . . . . .            | 79         |
| <b>3</b> | <b>La réalisation d'une solution</b>                           | <b>82</b>  |
| 3.1      | Des exigences de la modélisation du problème . . . . .         | 82         |
| 3.2      | Une conception d'une solution . . . . .                        | 84         |
| 3.3      | Des exigences de la modélisation d'une solution . . . . .      | 92         |
| 3.4      | Des explications sur l'implémentation d'une solution . . . . . | 92         |
| 3.5      | Des exigences à satisfaire pour la solution . . . . .          | 101        |
| <b>4</b> | <b>L'utilisation de la solution</b>                            | <b>107</b> |
| 4.1      | Applications selon différents contextes . . . . .              | 107        |
| 4.2      | Explications sur le fonctionnement . . . . .                   | 109        |
| <b>5</b> | <b>L'évaluation de la solution</b>                             | <b>114</b> |
| 5.1      | La vérification du respect des exigences . . . . .             | 114        |
| 5.2      | Une guide pour la validation des attentes . . . . .            | 125        |
|          | <b>Conclusion</b>                                              | <b>133</b> |
|          | <b>Bibliographie</b>                                           | <b>187</b> |
|          | <b>Glossaire</b>                                               | <b>188</b> |
| <b>A</b> | <b>Annexe d'un système formel pour le chapitre 1</b>           | <b>193</b> |
| A.1      | Des documents en informatique appliquée . . . . .              | 193        |
| A.2      | Des modèles en informatique appliquée . . . . .                | 205        |
| A.3      | Des connaissances en informatique appliquée . . . . .          | 210        |

## TABLE DES MATIÈRES

|          |                                                                 |            |
|----------|-----------------------------------------------------------------|------------|
| <b>B</b> | <b>Annexe des explications pour le chapitre 1</b>               | <b>218</b> |
| B.1      | Des documents en informatique appliquée . . . . .               | 220        |
| B.2      | Des modèles en informatique appliquée . . . . .                 | 335        |
| B.3      | Des connaissances en informatique appliquée . . . . .           | 401        |
| <b>C</b> | <b>Annexe pour décrire les langages utilisés</b>                | <b>473</b> |
| <b>D</b> | <b>Annexe fournissant des exemples d'intrants et d'extrants</b> | <b>478</b> |
| D.1      | Configuration . . . . .                                         | 478        |
| D.2      | Utilisateur . . . . .                                           | 501        |
| D.3      | Description proposée d'un modèle . . . . .                      | 502        |
| D.4      | Représentation standardisée du modèle . . . . .                 | 504        |
| D.5      | Journal . . . . .                                               | 505        |

# Liste des figures

|     |                                                                                                         |     |
|-----|---------------------------------------------------------------------------------------------------------|-----|
| 1.1 | Diagramme de Venn sur les domaines, techniques et modèle de langage                                     | 27  |
| 2.1 | Diagramme EA du procédé d'obtention d'un instrument . . . . .                                           | 73  |
| 2.2 | Diagramme EA du procédé d'obtention d'un logiciel . . . . .                                             | 76  |
| 2.3 | Diagramme EA du procédé d'obtention d'un modèle . . . . .                                               | 77  |
| 3.1 | Diagramme de contexte de l'IAR . . . . .                                                                | 85  |
| 3.2 | DFD du système IAR . . . . .                                                                            | 86  |
| 3.3 | DFD du processus 1.5 Formalise . . . . .                                                                | 87  |
| 3.4 | Diagramme EA du modèle conceptuel de données de la preuve de concept                                    | 91  |
| 3.5 | DFD pour certains types de compilateurs . . . . .                                                       | 95  |
| 3.6 | Diagramme relationnel du métamodèle de document . . . . .                                               | 100 |
| 4.1 | Une interface utilisateur de la preuve de concept . . . . .                                             | 112 |
| 5.1 | Diagramme BPMN sur des changements aux documents sans l'IAR .                                           | 128 |
| 5.2 | Diagramme BPMN sur des changements aux documents avec l'IAR .                                           | 128 |
| B.1 | Liste des éléments d'une spécification des exigences logicielles . . . . .                              | 231 |
| B.2 | Causes des problèmes de documentation selon des professionnels . . .                                    | 237 |
| B.3 | Standards utilisés pour la documentation selon des professionnels . .                                   | 250 |
| B.4 | Nombre d'articles abordant la qualité de la documentation . . . . .                                     | 252 |
| B.5 | Techniques d'évaluation de la qualité de la documentation utilisées par<br>des professionnels . . . . . | 254 |
| B.6 | Répartition typique des ressources d'un projet logiciel affilié à la NASA                               | 260 |
| B.7 | Phases dans lesquelles des professionnels modifient la documentation                                    | 262 |

## LISTE DES FIGURES

|                                                                                                      |     |
|------------------------------------------------------------------------------------------------------|-----|
| B.8 Utilisation des documents par phase selon des professionnels . . . . .                           | 263 |
| B.9 Fréquence de la répartition documentaire selon des professionnels . .                            | 264 |
| B.10 Délai de la mise à jour de la documentation selon des professionnels .                          | 268 |
| B.11 Causes de la dette technique selon des professionnels . . . . .                                 | 271 |
| B.12 Importance des artefacts selon des professionnels . . . . .                                     | 273 |
| B.13 Procédé de formalisation d'un document . . . . .                                                | 275 |
| B.14 Approches pour augmenter l'adaptabilité . . . . .                                               | 285 |
| B.15 Représentations d'environnements de programmation littéraire . . . .                            | 289 |
| B.16 Technologies sur lesquelles le DITA repose . . . . .                                            | 304 |
| B.17 Fonctionnalités utiles pour les langages d'architecture selon des profes-<br>sionnels . . . . . | 306 |
| B.18 Langages d'architecture utilisés selon des professionnels . . . . .                             | 309 |
| B.19 Diagramme de Venn sur l'IA et le TAL . . . . .                                                  | 313 |
| B.20 Diagramme de Venn sur l'IA . . . . .                                                            | 314 |
| B.21 Procédé typique pour la création d'un modèle de langage . . . . .                               | 317 |
| B.22 Nombre d'articles voulant expliquer l'IA entre 2012 et 2019 . . . . .                           | 319 |
| B.23 Adoption d'outils d'IA pour des tâches de développement . . . . .                               | 326 |
| B.24 Diagramme représentant l'association entre deux modèles . . . . .                               | 345 |
| B.25 Modèle au sujet des instruments de modélisation . . . . .                                       | 371 |
| B.26 Secteurs où des instruments de modélisation sont utilisés par des pro-<br>fessionnels . . . . . | 375 |
| B.27 Caractéristiques générales des instruments de modélisation . . . . .                            | 376 |
| B.28 Fonctionnalités offertes par les instruments concernant les transforma-<br>tions . . . . .      | 377 |
| B.29 Fonctionnalités offertes par les instruments concernant les modèles . .                         | 378 |
| B.30 Nombre d'articles sur les DSL et les ADL entre 1996 et 2011 . . . . .                           | 381 |
| B.31 Distribution des articles sur les DSL et les ADL selon les catégories .                         | 381 |
| B.32 Classes de langages formels (Hiérarchie de Chomsky) . . . . .                                   | 383 |
| B.33 Représentation de la dualité entre le modèle et le code source . . . . .                        | 385 |
| B.34 Procédé réflexif pour faire la rétro-ingénierie d'un modèle formel . . .                        | 389 |
| B.35 Procédé réflexif pour créer un modèle formel . . . . .                                          | 391 |
| B.36 Secteurs où des méthodes formelles sont utilisés par des professionnels                         | 400 |

## LISTE DES FIGURES

|                                                                                                       |     |
|-------------------------------------------------------------------------------------------------------|-----|
| B.37 Diagramme de Venn sur des titres en informatique . . . . .                                       | 412 |
| B.38 Professionnels en informatique par rapport aux diplômés en informatique aux États-Unis . . . . . | 419 |
| B.39 Niveaux de formalisme d'artefacts de représentation des connaissances                            | 422 |
| B.40 Développement de taxinomies par type de recherche et domaine de connaissances . . . . .          | 425 |
| B.41 Types de recherches menées sur les procédés de développement entre 1987 et 2008 . . . . .        | 427 |
| B.42 Manières utilisées par des professionnels pour gérer les exigences obsolètes                     | 431 |
| B.43 Nombre de dépôts et de métamodèles créés sur GitHub entre 2003 et 2020 . . . . .                 | 455 |
| B.44 Types d'usage des ontologies entre 2015 et 2019 . . . . .                                        | 463 |
| B.45 Lacunes importantes chez des professionnels . . . . .                                            | 469 |
| B.46 Obstacles identifiés par des professionnels à l'usage des méthodes formelles . . . . .           | 471 |

# Liste des tableaux

|      |                                                                                                                      |     |
|------|----------------------------------------------------------------------------------------------------------------------|-----|
| 1.1  | Classement des catégories de documents par phase . . . . .                                                           | 9   |
| 1.2  | Problèmes sur la doc. comprenant des caractéristiques de qualité . . .                                               | 13  |
| 1.3  | Sommaire des problèmes et des solutions . . . . .                                                                    | 21  |
| 1.4  | Relevés des catégories d'instruments et leurs fréquences. . . . .                                                    | 22  |
| 1.5  | Pourcentage des tranches d'âge parmi les postes à temps plein occupés<br>en 2021 au Québec . . . . .                 | 54  |
| 1.6  | Nombre de postes occupés en 2021 au Québec selon le métier et le<br>diplôme universitaire de premier cycle . . . . . | 55  |
| 3.1  | Correspondances des besoins et des exigences . . . . .                                                               | 105 |
| 5.1  | Résultats des cas de tests . . . . .                                                                                 | 123 |
| 5.2  | Correspondances des cas de tests et des exigences . . . . .                                                          | 124 |
| 5.3  | Différentes variables pour l'expérimentation . . . . .                                                               | 130 |
| B.1  | Couverture des standards sur des concepts de génie logiciel . . . . .                                                | 229 |
| B.2  | Désaccord des professionnels envers des affirmations . . . . .                                                       | 239 |
| B.3  | Catégorisation formée depuis les articles . . . . .                                                                  | 244 |
| B.4  | Tendances dans l'enseignement du génie logiciel . . . . .                                                            | 245 |
| B.5  | Experts indiquant les trois mesures les plus importantes . . . . .                                                   | 249 |
| B.6  | Problèmes reliés à la documentation . . . . .                                                                        | 252 |
| B.7  | Caractéristiques de qualité de la doc. abordées dans des articles . . .                                              | 253 |
| B.8  | Mesures les plus utilisées en industrie en 1990 . . . . .                                                            | 255 |
| B.9  | Sommaire des problèmes et solutions en anglais . . . . .                                                             | 278 |
| B.10 | Sommaire des étiquettes sur les techniques utilisées . . . . .                                                       | 324 |

## LISTE DES TABLEAUX

|                                                                                                                    |     |
|--------------------------------------------------------------------------------------------------------------------|-----|
| B.11 Risques avec les modèles de langage. . . . .                                                                  | 332 |
| B.12 Avis d’experts en méthodes formelles sur un éventuel remplacement<br>par des méthodes non formelles . . . . . | 334 |
| B.13 Bénéfices attendus de l’utilisation des méthodes formelles . . . . .                                          | 399 |
| B.14 Raisons à la faible utilisation des méthodes formelles. . . . .                                               | 401 |
| B.15 Nombre de postes occupés par des professionnels informatiques au Qué-<br>bec en 2021 . . . . .                | 413 |
| B.16 Nombre de postes occupés à temps plein en 2021 au Québec, par métier<br>et tranche d’âge . . . . .            | 415 |
| B.17 Tentatives d’intégrations . . . . .                                                                           | 444 |
| B.18 Lacunes fréquentes chez des professionnels . . . . .                                                          | 468 |



# Liste des abréviations

- AA** Apprentissage automatique
- ADL** Langage de description d'architecture (*Architecture Description Language*)
- ALM** Gestion du cycle de vie des applications (*Application lifecycle management*)
- ANTLR** Un autre outil pour la reconnaissance du langage (*ANother Tool for Language Recognition*)
- AP** Apprentissage profond
- API** Interface de programmation d'application (*Application Programme Interface*)
- BPMN** Modèle de procédé d'affaire et notation (*Business Process Model and Notation*)
- CASE tool** Outil de génie logiciel assisté par ordinateur (*Computer-Aided Software Engineering tool*)
- CMMI** Modèle intégré de maturité des capacités d'intégration (*Capability Maturity Model Integration*)
- CMOF** Mise en place complète de méta-objets (*Complete Meta-Object Facility*)
- DFD** Diagramme de flux de données
- DITA** Architecture des types d'informations Darwin (*Darwin Information Type Architecture*)
- DSL** Langage spécifique à un domaine (*Domain-Specific Language*)
- DTD** Définition de type de document (*Document Type Definitions*)
- EA** Entité-Association (*Entity-Relationship*)

## Liste des abréviations

**EBNF** Forme étendue de Backus-Naur (*Extended Backus–Naur form*)

**EMF** Cadre de modélisation Eclipse (*Eclipse Modeling Framework*)

**GML** Grand modèle de langage

**I-CASE** Environnement intégré de génie logiciel assisté par ordinateur (*Integrated CASE Environment*)

**IAR** Instrument d’aide à la rédaction

**MBEBOK** Corpus de connaissances en génie logiciel basé sur les modèles (*Model-Based Software Engineering Body of Knowledge*)

**MBSE** Ingénierie système basée sur les modèles (*Model Based Systems Engineering*)

**MDA** Architecture dirigée par les modèles (*Model Driven Architecture*)

**MOF** Mise en place de méta-objets (*Meta-Object Facility*)

**OCL** Langage de contraintes pour objets (*Object Constraint Language*)

**PMBOK** Référentiel des connaissances en gestion de projet (*Project Management Body of Knowledge*)

**RN** Réseau de neurones

**SEL** Spécification des exigences logicielles

**SLEBOK** Corpus de connaissances en génie des langages logiciels (*Software Language Engineering Body of Knowledge*)

**SWEBOK** Guide du corpus de connaissances en génie logiciel (*Software Engineering Body of Knowledge*)

**SysML** Langage de modélisation des systèmes (*SYStems Modeling Language*)

**TAL** Traitement automatique des langues

**UML** Langage de modélisation unifié (*Unified Modeling Language*)

**XML** Langage de balisage extensible (*Extensible Markup Language*)

## Liste des abréviations

**XSD** Définition du schéma XML (*XML Schema Definition*)

**XSLT** Langage de transformation XML (*eXtensible Stylesheet Language Transformations*)

# Introduction

Les documents occupent une place essentielle en informatique appliquée. Pourtant, ils continuent d’être négligés et délaissés. Pour plusieurs, cela peut paraître anodin, mais les répercussions que peut engendrer une absence ou une mauvaise documentation sont réelles. Ainsi, la mauvaise documentation a été pointée comme l’un des facteurs déterminants dans les tragédies causées en 2018 et 2019 par le logiciel **Maneuvering Characteristics Augmentation System** de Boeing [153]. Plus de trente ans auparavant, la mauvaise documentation avait été aussi désignée comme un facteur important dans la tragédie du Therac-25, qui deviendra par la suite un cas d’école [207, 208]. Malgré ces exemples, et de très nombreux autres, les pratiques n’accordent généralement pas l’importance et les moyens qu’elles devraient à la documentation, et ce, même dans les secteurs critiques<sup>1</sup>.

Exposer les mécanismes existants entre les documents et les logiciels ainsi que leurs rôles devrait suffire à convaincre de l’importance de la documentation ceux qui pratiquent l’informatique appliquée, puisqu’il s’agit de l’application d’une science. Pourtant, les années s’écoulent et la vaste majorité des méthodes relatives aux documents en informatique appliquée continuent à être négligées. Par conséquent, la qualité des artefacts informatiques en pâtit et les incidents, bien qu’évitables, se poursuivent. Certaines approches ont été proposées afin d’éviter de reproduire les mêmes erreurs, mais, globalement, le cycle ne se brise pas. Ce mémoire et la preuve de concept associée découlent de la volonté de contribuer à remédier à cette situation. Les chapitres

---

1. Le terme **secteur critique** signifie un secteur d’infrastructure critique ou essentielle à la société (énergie, transport, santé, défense, télécommunication, etc.) [Gouvernement du Canada, site web, « Infrastructures essentielles (IE) du Canada », <https://www.securitepublique.gc.ca/cnt/ntnl-scrt/crtcl-nfrstrctr/cci-iec-fr.aspx> (consulté le 2025-01-24)].

## INTRODUCTION

compris dans ce mémoire sont : la revue de littérature, une définition du problème, la réalisation d'une solution et l'évaluation de la solution.

En premier lieu, le chapitre 1 comprend une revue de littérature qui s'efforce de cerner la nature et l'importance des relations entre les documents, les modèles et les connaissances. Le chapitre est divisé en trois sections et chacune de ses sections en trois sous-sections : les objectifs, les écueils à ces objectifs et les approches pour surmonter ces écueils. Ce chapitre sert de fondation aux autres chapitres. Cette revue de littérature montre, entre autres, que l'atteinte d'un niveau de qualité pour des logiciels est indissociable de la qualité des modèles utilisés. Les documents en sont le véhicule, ils permettent de faire la validation, la réalisation et l'établissement du logiciel. Ainsi, des documents erronés conduisent à des modèles erronés et des modèles erronés conduisent à des logiciels erronés.

Ensuite, le chapitre 2 présente une définition du problème portant sur la formalisation et la compréhension des documents servant aux modèles. Cette définition intègre plusieurs approches qui se sont avérées efficaces sans pour autant être largement utilisées. Après avoir présenté le problème, une description d'un modèle du problème est présentée ainsi que les attentes et les besoins.

Puis, le chapitre 3 présente la réalisation d'une solution à ce problème. Cela débute avec des exigences répondant au problème. Ensuite, une conception est décrite et de nouvelles exigences apparaissent. S'ensuivent des explications sur l'implémentation et sur les instruments utilisés. Le chapitre se termine par un tableau identifiant les besoins comblés par les exigences ainsi qu'un prolongement à la solution développée.

Par la suite, le chapitre 4 présente l'utilisation de la solution. Cela commence en exposant des situations où la solution pourrait être efficace. Ensuite, des explications sont données sur l'utilisation de la solution à travers un exemple.

Après, le chapitre 5 présente l'évaluation de la solution découpée en deux étapes : la première consiste en la vérification des exigences, à l'aide de cas de tests, et la

## INTRODUCTION

deuxième, en une proposition de validation de certaines attentes. Cette proposition de validation comporte la description partielle d'un protocole expérimental. Ce découpage est aussi celui des caractéristiques de qualité. Les caractéristiques de qualité absolues se vérifiant par des cas de tests, tandis que celles relatives se validant par la proposition de validation.

Finalement, la conclusion présente les différentes contributions de ce mémoire ainsi que trois prolongements. Les contributions comprennent une synthèse des thèmes de la documentation, de la modélisation et des connaissances dans le domaine de l'informatique appliquée, ainsi que la démarche ayant mené à une solution prenant la forme d'une preuve de concept. Les prolongements s'inscrivent dans la continuité de cette preuve de concept : les deux premiers portent sur l'intégration et la comparaison des méthodes formelles et de l'intelligence artificielle, et le troisième vise à rendre cette preuve de concept plus générique encore.

# Chapitre 1

## Revue de littérature

Cette revue de littérature démontrera que la documentation logicielle a pour fonction première de transmettre, au moyen de documents, des connaissances organisées en modèles sur lesquels toute opérationnalisation d'un traitement informatique est fondée. Ainsi, l'atteinte d'un niveau de qualité pour des logiciels est indissociable de la qualité des modèles utilisés. En adoptant une perspective d'informatique appliquée, elle analysera les objectifs, les principaux écueils et les approches les plus significatives. Elle montrera enfin que, bien qu'utiles à la production et au maintien des documents, ces approches demeurent insuffisantes pour répondre à une partie des écueils.

- P01. Cette revue de littérature comporte un très grand nombre de citations avec des points de vue qu'il faut concilier. Ces citations proviennent d'opinions éclairées et elles sont rarement démontrables. Dans ce qui explique le phénomène, le comportement humain, sa perspective et son point de vue en sont pour beaucoup. D'ailleurs, lorsque des affirmations sont avancées, elles s'appuient souvent sur des sondages ayant de faibles échantillons et des données corrélées. Il arrive que des expérimentations soient montées, mais trop souvent, dans un contexte très particulier, ce qui les rend très difficilement comparables.

La présente revue propose donc souvent une synthèse de plusieurs informations complémentaires et concordantes. Elle s'organise autour d'affirmations, mises en évidence par une police de caractère distincte, placées au début de chaque paragraphe. Ces af-

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

firmations sont soutenues par plusieurs énoncés (définitions, axiomes, inférences). Les énoncés sont regroupés dans l'annexe A, tandis que leurs explications se trouvent dans l'annexe B. Chaque paragraphe, chaque énoncé et chaque explication est identifié : le paragraphe porte un identifiant, et les énoncés qui soutiennent l'affirmation sont indiqués entre parenthèses au bas du même paragraphe. Des hyperliens permettent de naviguer du paragraphe aux énoncés, puis des énoncés à leurs explications, et inversement. L'objectif étant de conserver le fil conducteur lors d'une argumentation trop longue, tout en rendant accessibles ces informations.

(voir énoncé[s] lié[s] : R01 p.[193](#))

## 1.1 Des documents en informatique appliquée

De cette revue de littérature, il ressort que les objectifs principaux des documents en informatique appliquée sont d'aider à réaliser et à établir des entités informatiques, et plus couramment des logiciels. Pendant que la documentation est créée et utilisée à ces fins, plusieurs écueils se dressent. La majorité de ceux-ci proviennent de la façon d'œuvrer en informatique. Pour pallier ces écueils, il existe des approches constituées de méthodes et de techniques, mais ces dernières ne résolvent pas tout.

- P02. Au préalable, il est nécessaire de donner ces définitions pour éviter des ambiguïtés :
- logiciel : ensemble de suites d'instructions interprétables par un ordinateur ;
  - composant logiciel : logiciel ou partie d'un logiciel<sup>2</sup> ;
  - entité informatique : système, sous-système, composant logiciel ou composant matériel en lien avec l'informatique ;
  - document : écrit servant d'information ;
  - documentation informatique : ensemble de documents associé à un ensemble d'entités informatiques.

(voir énoncé[s] lié[s] : D01, D02, D03, D04, D05 p.[193](#))

---

2. Le logiciel a de particulier qu'il peut comprendre lui-même d'autres logiciels.



## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### 1.1.1 Des objectifs

Les objectifs de la documentation sont déterminés en considérant l’aspect fondamental et l’aspect pratique. L’aspect fondamental est principalement déterminé par l’association entre le document et l’entité informatique en elle-même. L’aspect pratique est déterminé principalement par l’usage des documents et leurs catégorisations. Ce faisant, il est également montré que les documents dans leur ensemble représentent fréquemment le seul véhicule pour objectiver l’entité informatique. Ainsi, ces documents se révèlent nécessaires à la réalisation et à l’établissement du logiciel.

**P03.** L’association fondamentale est que le document sert<sup>3</sup> à une entité informatique. Les documents et les entités informatiques font partie des artefacts du procédé de développement. Voici des définitions :

- procédé : ensemble structuré d’activités visant un but logiquement déterminé par des antécédents et des conséquents où des intrants donnent lieu à des extrants ;
- procédé de développement : procédé dont le but est de créer des entités informatiques ;
- processus : instance d’un procédé ;
- artefact : intrant tangible ou extrant tangible à un processus ;
- phase d’un projet : regroupement d’activités logiquement interreliées découlant de la gestion d’un projet ;
- phase d’un procédé de développement : regroupement d’activités logiquement interreliées découlant d’un procédé de développement.

Le processus va créer un ou plusieurs artefacts. Cependant, le logiciel demeure le but du procédé de développement, les autres artefacts, comme les documents, servent assurément ce but, et il sera décrit la manière. Dans le cadre de ce mémoire, l’utilisation du terme **phase** porte sur les procédés de développement.

(voir énoncé[s] lié[s] : OD01, OD02, OD03, OD04, OD05, OD06 p.194)

---

3. Il peut être utilisé, cité ou requis. Aussi, il est possible qu’un seul document puisse servir à plusieurs entités informatiques.

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**P04. L’usage des catégories de documents, ainsi que le contenu des catégories de documents varie.** D’abord, les catégories de documents utilisés vont varier selon la taille du projet, les lectorats, les procédés utilisés et la réglementation applicable [228, 318]. Ensuite, les catégories de documents elles-mêmes vont varier, puisqu’elles ne sont pas décrites formellement.

- le standard MIL-STD-498 (1994) fournit des gabarits [61, 77] ;
- l’ISO/IEC/IEEE 15289 (2011) fournit une liste de points à couvrir pour certains documents [135] ;
- l’ISO/IEC/IEEE 24765 (2017) fournit des descriptions sommaires [140].

Ce qui demeure invariable est qu’un document est écrit par un autorat et vise un lectorat.

(voir énoncé[s] lié[s] : OD07, ODO8, ODO9, OD10 p.194)

**P05. Les documents ont une utilité selon des organisations de standardisation.** D’abord, plusieurs standards prescrivent des catégories de documents. Certains standards définissent ces catégories, il y a : la MIL-STD-498, le DO-178C, l’IEC 61513 et l’ISO 15289. Les trois premiers standards s’appliquent à des secteurs critiques comprenant des entités informatiques et le dernier concerne les entités informatiques, peu importe le secteur. Ensemble, ils prescrivent au moins 76 catégories de documents, certaines partageant plusieurs éléments.

Ensuite, les organismes de certification peuvent s’assurer du respect d’un standard pour un procédé par les artefacts générés incluant les documents. En effet, les artefacts sont les traces laissées par les processus, ils constituent la preuve de leur exécution. Ils peuvent être examinés pour comprendre les processus et pour s’assurer de la bonne mise en place d’un modèle de procédé, comme le modèle de maturité des capacités d’intégration (CMMI, *Capability Maturity Model Integration* en anglais)<sup>4</sup> [25].

(voir énoncé[s] lié[s] : OD11, OD12 p.195)

**P06. Les documents doivent être présents tout au long du processus de développement.** D’abord, plusieurs experts donnent leurs avis sur l’importance et la place de la documentation :

---

4. Le CMMI est un ensemble de bonnes pratiques pour le développement de systèmes et de logiciels.

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- Frederick P. Brooks Jr. indique que la documentation complète est nécessaire pour terminer le processus [39] ;
- C. A. R. Hoare et David L. Parnas avertissent qu’il s’agit d’une erreur de faire les documents à la fin du processus [119, 296] ;
- Ian F. Sommerville indique que certains documents servent à planifier et se retrouvent donc au début du processus [367].

Cependant, il n’est pas dit que la documentation couvre l’entièreté du processus. Les activités sont regroupées en phases à l’intérieur du processus. Si toutes les phases sont couvertes, alors le processus le sera également. Mais, il n’y a pas de consensus sur les phases d’un processus de développement [139]. En s’inspirant, des phases qui existent dans la littérature, les phases suivantes ont été sélectionnées : l’analyse, la conception, la concrétisation, l’évaluation et la transition<sup>5</sup>. Ainsi, il est possible de distribuer les catégories de documents sur les différentes phases pour observer qu’ils peuvent en effet couvrir l’entièreté du processus (tableau 1.1.).

(voir énoncé[s] lié[s] : OD13, OD14, OD15 p.195)

**P07. Les documents en informatique appliquée ont comme objectifs d’aider à réaliser et à établir des logiciels.** Le premier objectif, réaliser, consiste à rendre tangible ce qui est abstrait. Un logiciel peut être considéré comme une chose immatérielle qui nécessite, pour exister, l’emploi d’un support matériel, comme des disques, des imprimés, etc. Le logiciel peut également être perceptible par les interactions rendues possibles avec l’aide d’un écran, d’un clavier, etc. Dans les deux cas, le logiciel devient tangible. Le second objectif, établir, vise à créer les conditions pour soutenir la pérennité. Pour le logiciel, cela englobe ce qui facilitera la validation, l’exploitation, la maintenance, et ce, pour les différents milieux. Les différentes catégories de documents présentées au tableau 1.1 peuvent servir l’un, l’autre ou même les deux objectifs.

(voir énoncé[s] lié[s] : OD16, OD17 p.195)

---

5. Pour un problème donné, l’analyse est son internalisation. La conception est celle d’au moins une solution. La concrétisation passe par la création d’une machine. L’évaluation est celle de cette machine par rapport au problème et à la solution. La transition vise à transmettre cette machine.

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABLEAU 1.1** – Classement des catégories de documents par phase

| Phase<br>Standard                                      | Analyse                                                                                                                                                                                                                                                                                                                             | Conception                                                                                                                                                                                                                          | Concrétisation                                                                                                                                                                                                                | Évaluation                                                                                                                                                                                                                     | Transition                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>DID title</i><br>MIL-STD-498<br>(1994)<br>[76]      | <ul style="list-style-type: none"> <li>• interface requirements specification</li> <li>• software requirements specification</li> <li>• system/subsystem specification</li> </ul>                                                                                                                                                   | <ul style="list-style-type: none"> <li>• database design description</li> <li>• interface design description</li> <li>• software design description</li> <li>• system/subsystem design description</li> </ul>                       | Non applicable                                                                                                                                                                                                                | <ul style="list-style-type: none"> <li>• software test description</li> <li>• software test plan</li> <li>• software test report</li> </ul>                                                                                    | <ul style="list-style-type: none"> <li>• computer operation manual</li> <li>• computer programming manual</li> <li>• firmware support manual</li> <li>• software center operator manual</li> <li>• software input/output manual</li> <li>• software installation plan</li> <li>• software product specification</li> <li>• software transition plan</li> <li>• software user manual</li> <li>• software version description</li> <li>• operational concept description</li> </ul> |
| <i>Documents</i><br>DO-178C (2012)<br>[104, 116, 315]  | <ul style="list-style-type: none"> <li>• software high-level requirements</li> <li>• specification models</li> <li>• system requirements</li> </ul>                                                                                                                                                                                 | <ul style="list-style-type: none"> <li>• derived requirement</li> <li>• design description</li> <li>• design models</li> <li>• low-level requirements</li> <li>• software architecture</li> </ul>                                   | <ul style="list-style-type: none"> <li>• build instructions</li> <li>• source code</li> </ul>                                                                                                                                 | <ul style="list-style-type: none"> <li>• software verification cases and procedures</li> <li>• software verification plan</li> <li>• software verification result</li> </ul>                                                   | Non applicable                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <i>Documents</i><br>IEC 61513:2013<br>[136, 363]       | <ul style="list-style-type: none"> <li>• application functions requirements specification</li> <li>• functional assignment</li> <li>• requirements specification (functionality, performance)</li> <li>• system requirements specification</li> <li>• system specification (system architecture, software specification)</li> </ul> | <ul style="list-style-type: none"> <li>• architectural design</li> <li>• system detailed design</li> </ul>                                                                                                                          | <ul style="list-style-type: none"> <li>• application software code</li> <li>• system integration</li> <li>• system integration plan</li> <li>• system integration report</li> <li>• system integration test report</li> </ul> | <ul style="list-style-type: none"> <li>• system validation documentation (system validation plan, system validation test report, system validation report)</li> </ul>                                                          | <ul style="list-style-type: none"> <li>• installation report</li> <li>• modification report</li> <li>• qualification plan (functional and environmental qualification, software evaluation and assessment)</li> <li>• system installation plan</li> <li>• system maintenance plan</li> <li>• system operation plan</li> <li>• user documentation</li> </ul>                                                                                                                       |
| <i>Information elements</i><br>ISO 15289:2011<br>[134] | <ul style="list-style-type: none"> <li>• software requirements specification</li> <li>• system requirements specification</li> </ul>                                                                                                                                                                                                | <ul style="list-style-type: none"> <li>• database design description</li> <li>• interface description</li> <li>• system architecture description</li> <li>• system element description</li> <li>• system/software design</li> </ul> | <ul style="list-style-type: none"> <li>• implementation procedure</li> <li>• implementation report</li> <li>• integration procedure</li> <li>• test procedure</li> <li>• test report</li> </ul>                               | <ul style="list-style-type: none"> <li>• verification plan</li> <li>• verification procedure</li> <li>• verification report</li> <li>• validation plan</li> <li>• validation procedure</li> <li>• validation report</li> </ul> | <ul style="list-style-type: none"> <li>• maintenance plan</li> <li>• maintenance procedure</li> <li>• release plan</li> <li>• user documentation</li> </ul>                                                                                                                                                                                                                                                                                                                       |

<sup>a</sup> Le MIL-STD-498 aborde le code source, les tests unitaires, les tests d'intégration sans les considérer comme des **documents connexes**, donc n'ont pas de *Data Item Description* (DID).

<sup>b</sup> Le DO-178C considère la transition à l'extérieur du produit, donc n'a pas à être certifié.

<sup>b</sup> Dans le DO-178C, il n'y a pas de documents de transition, puisqu'ils sont considérés à l'extérieur du produit et qu'ils n'ont pas à être certifiés.

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### 1.1.2 Des écueils

Les écueils aux documents en informatique appliquée arrivent tout au long du processus de développement. Le processus de développement étant rarement linéaire, il devient difficile d'énumérer les écueils dans un ordre totalement logique. Néanmoins, étant donné le nombre important d'écueils, ils sont tout de même divisés entre ceux arrivant pendant un projet de réalisation et ceux arrivant après la réalisation.

#### Pendant la réalisation

Les premiers écueils présentés sont ceux tirés de l'avis des professionnels informatiques et du contexte de développement. Par la suite sont énumérés les écueils touchant à la qualité de la documentation, et plus précisément à ses caractéristiques, à ses mesures et à son contrôle.

P08. Plusieurs professionnels informatiques indiquent que les compétences informatiques et le temps sont essentiels pour surmonter les problèmes de documentations. D'abord, plusieurs professionnels informatiques répondent à un sondage ( $n = 323$ ). Ils nomment comme problèmes de documentation principaux : les ambiguïtés, l'inexactitude et l'incomplétude. L'article conclut que, pour surmonter ces problèmes, une bonne expertise informatique est requise [385].

Pour des professionnels informatiques ayant répondu à un autre sondage ( $n = 146$ ), la principale cause des problèmes entourant la documentation est le manque de temps qui lui est consacré [6]. Lors de la planification, il faudrait allouer plus de temps à la documentation et la commencer au début du processus.

(voir énoncé[s] lié[s] : ED01 p.196)

P09. Plusieurs professionnels informatiques ne désirent pas mettre l'effort dans la documentation. D'abord, parmi les discours encore fréquents à ce jour, il y a :

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- [Traduction libre] « La nature des programmeurs est telle que le travail intéressant est réalisé au détriment du travail ennuyeux, et la documentation est un travail ennuyeux. »<sup>6</sup> [405] ;
- [Traduction libre] « Globalement, le message clé rapporté par certains professionnels est le suivant : « Seul le code explique le code. » »<sup>7</sup> [231].

Ensuite, cette aversion des professionnels informatiques à l'égard de la documentation a conduit à la réalisation d'une documentation minimaliste. L'objectif était de rendre la documentation plus attrayante et efficace. Un article publié en 1990 relève un certain nombre de problèmes au sujet de la couverture et de la compréhension de la documentation minimaliste [38].

(voir énoncé[s] lié[s] : ED02, ED03 p.196)

**P10. La tendance actuelle privilégie les professionnels informatiques qui veulent éviter la documentation à ceux qui la préconisent.** Après les années 2000, les méthodologies agiles se sont imposées comme un courant dominant [76], et ce, jusqu'au secteur critique [344]. Au fondement de ce courant se trouve le manifeste agile qui regroupe des principes. Les quatre premiers principes priorisent certaines valeurs (dont la communication) par rapport à d'autres (les éléments tangibles et vérifiables), tandis que le cinquième (et dernier) principe justifie les quatre premiers sur la base de leurs préférences personnelles<sup>8</sup>. Voici les principes numérotés ;

1. « Les individus et leurs interactions plus que les processus et les outils. »
2. « Des logiciels opérationnels plus qu'une documentation exhaustive. »
3. « La collaboration avec les clients plus que la négociation contractuelle. »
4. « L'adaptation au changement plus que le suivi d'un plan. »
5. « Nous reconnaissons la valeur des seconds éléments, mais privilégions les premiers. »

---

6. « *The nature of programmers is such that interesting work gets done at the expense of dull work, and documentation is dull work.* »

7. « *Overall, the key message reported by some professionals is : « Only the code explains the code. » »*

8. Kent Beck *et al.*, site web, « Manifesto for Agile Software Development », <https://agilemanifesto.org/iso/fr/manifesto.html> (consulté le 2025-05-12).

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

Cela a amené les professionnels informatiques à privilégier la communication à la documentation [314].

(voir énoncé[s] lié[s] : ED04, ED05 p.196)

P11. En général, la qualité de la documentation est considérée secondaire. D’abord, il existe un intérêt sur les modèles de qualité, même si cet intérêt baisse depuis 2010 pour les six caractéristiques les plus notables que sont : [Traduction libre] « fonctionnalité, maintenabilité, fiabilité, efficacité, facilité d’utilisation, portabilité »<sup>9</sup> [230]. Progressivement, le développement de caractéristiques de qualité et de métriques se porte davantage sur la qualité du code source plutôt que sur celle du produit [62, 230]. Pourtant, la qualité du logiciel est intrinsèquement liée à celle de la documentation. Plusieurs experts provenant de secteurs critiques nomment des métriques importantes en lien avec la documentation [222] :

- le nombre de changements dans la spécification des exigences ;
- la densité de défauts à l’intérieur de la conception.

Un sondage de 2014 mené auprès de professionnels informatiques ( $n = 88$ ) révèle que 60 % d’entre eux préfèrent s’abstenir d’utiliser des standards ou des modèles de qualité pour leur documentation, tandis qu’au moins 30 % utilisent d’anciens<sup>10</sup> standards ou modèles de qualité [303]. Ce manque d’intérêt se produit aussi en recherche sur la documentation logicielle. Une citation de David L. Parnas explique ce peu d’intérêt :

[Traduction libre] La plupart des chercheurs en informatique ne considèrent pas la documentation logicielle comme un sujet relevant de l’informatique. Ils ne voient ni mathématiques, ni algorithmes, ni modèles de calcul ; par conséquent, ils ne s’y intéressent pas.<sup>11</sup> [294]

(voir énoncé[s] lié[s] : ED06, ED07, ED08, ED09 p.196)

---

9. « *functionality, maintainability, reliability, efficiency, usability, portability* »

10. Cela peut être pour de bonnes raisons.

11.

*Most Computer Science researchers do not see software documentation as a topic within computer science. They can see no mathematics, no algorithms, and no models of computation ; consequently, they show no interest in it.*

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABEAU 1.2** – Problèmes sur la doc. comprenant des caractéristiques de qualité

| Problèmes provenant d'une analyse de la littérature                                                                                                                                                                                                                                                                                                                                                                                    | Problèmes provenant d'une analyse d'artefacts (catégorie)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• accessibilité</li> <li>• actualité</li> <li>• cohérence</li> <li>• correction</li> <li>• exactitude</li> <li>• exhaustivité</li> <li>• fiabilité</li> <li>• format</li> <li>• informations relatives à l'auteur</li> <li>• lisibilité/compréhension</li> <li>• orthographe et grammaire</li> <li>• structure de l'information</li> <li>• traçabilité</li> <li>• similitude/unicité</li> </ul> | <ul style="list-style-type: none"> <li>• actualité (info)</li> <li>• besoin d'aide (outil)</li> <li>• bogue/problème (outil)</li> <li>• configuration du générateur de documentation (processus)</li> <li>• contribution à la documentation (processus)</li> <li>• exactitude (info)</li> <li>• exhaustivité (info)</li> <li>• internationalisation (processus)</li> <li>• lisibilité (info)</li> <li>• maintenabilité (info)</li> <li>• migration d'outils (outil)</li> <li>• problèmes de développement liés à la documentation (processus)</li> <li>• support/attentes (outil)</li> <li>• traçabilité (processus)</li> <li>• utilisabilité (info)</li> <li>• utilité (info)</li> </ul> |

P12. Plusieurs caractéristiques de qualité portant sur la documentation sont relatives, donc plus difficilement objectivables. Deux analyses fournissent ou dissimulent plusieurs caractéristiques de qualité sur la documentation. La première s'applique sur des articles traitant des problèmes de documentation sélectionnés depuis une revue de littérature ( $n = 69$ ) [412]. La deuxième s'applique sur différents artefacts ( $n = 878$ ) concernant des problèmes avec la documentation [7]. Ces problèmes et les caractéristiques de qualité qu'ils contiennent sont résumés dans le tableau 1.2. Dans ce tableau, il y a des critères relatifs aux autres artefacts, au lectorat ou à l'autorat : la complétude, l'accessibilité, la compréhension, la mise à jour, la maintenabilité. Lorsqu'il est demandé à des professionnels informatiques leurs méthodes d'évaluations de la documentation, la plupart d'entre eux affirment qu'ils n'utilisent que des revues informelles pour éva-



## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

luer ces différentes caractéristiques de qualité [303].

(voir énoncé[s] lié[s] : ED10, ED11 p.196)

P13. Les mesures sur les documents sont de moins en moins mises de l’avant. En 1990, un sondage mené dans l’industrie ( $n = 800$ ) indique que certaines mesures touchent directement la documentation [116] :

- le niveau de complétude et d’exactitude de la documentation (42 %) ;
- la taille et la complexité de la documentation (20 %).

En 2018, un sondage auprès de 129 professionnels informatiques indique qu’au plus deux individus connaissent certaines mesures applicables aux documents, sans pour autant préciser lesquelles [81].

Sans mesure, il devient impossible d’établir des comparatifs et, ainsi, d’effectuer des contrôles. Horst Zuse indique l’importance d’intégrer les mesures dès la première phase du développement, afin d’épargner l’effort et d’éviter des problèmes, du moins lors de la maintenance [413].

(voir énoncé[s] lié[s] : ED12 p.197)

P14. La documentation, comme plusieurs autres artefacts en informatique, ne dispose pas de mesures permettant un contrôle. D’abord, les mesures populaires, comme le nombre de lignes de code, sont utiles une fois le projet terminé. Un article de 2020 analyse 81 articles provenant de conférences sur les langages de programmation et établit que 65 articles utilisent le nombre de lignes ou le nombre de lignes de code source [12]. Alain Abran donne quelques limitations de cette mesure [1]. Elle est disponible que lorsque le projet est terminé et elle est dépendante des technologies utilisées. Cela n’empêche pas Barry W. Boehm et P. N. Papaccio de fournir plusieurs résultats sur le coût relatif de la documentation [32]. Dans l’un de ses ouvrages, Barry W. Boehm indique qu’il utilise une liste de contrôle pour compter précisément le nombre de lignes de code source [33]. Le même principe peut s’appliquer pour le nombre de lignes de texte d’un document.

Ensuite, les mesures ne permettent pas d’établir formellement le coût relatif de la documentation. Selon l’United States Air Force [123] et la National Aeronautics and Space Administration [226], le coût relatif de la documentation se situerait autour

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

de 10 % et 11 % respectivement. Selon Barry W. Boehm et P. N. Papaccio, cela avoisinerait souvent 60 % [32]. D'ailleurs, ces résultats sont fondés sur les projets qui serviront par la suite à Barry W. Boehm dans la création du *Constructive Cost Model* (COCOMO).

Ainsi, ces mesures n'arrivent pas à effectuer un contrôle lors du développement ou à distinguer clairement l'effort exigé entre la documentation et le reste du développement. Cependant, il est possible de mesurer les documents contenant les modèles avec la méthode des points de fonction<sup>12</sup>, ce qui permet d'obtenir des estimations de l'effort requis pour produire d'autres artefacts. Cela confère à ces documents une importance accrue. [129].

(voir énoncé[s] lié[s] : ED13, ED14, ED15 p.197)

**P15. Il faut continuellement maintenir la cohérence à l'intérieur d'un document et entre les documents tout au long du processus.** La documentation accompagne plusieurs autres activités, comme le font la planification et les tests. Donc, ce n'est pas possible de circonscrire la manipulation des documents à une phase [355]. Plusieurs professionnels informatiques indiquent qu'ils doivent modifier au moins partiellement un document, peu importe la phase [303]. Certains types de documents sont utilisés dans différentes activités. Cela pourrait entraîner des modifications supplémentaires. Le guide d'utilisateur et le guide de déploiement font partie des types de documents les plus successibles d'être largement utilisés [6]. S'ajoutent également les duplicata d'informations dispersés à travers les documents, phénomène souvent causé par l'éparpillement des informations à travers des documents, comme c'est le cas dans les documents d'architecture [347]. Ainsi, la maintenance de la cohérence devient d'autant plus difficile à maintenir dans le temps.

(voir énoncé[s] lié[s] : ED16, ED17, ED18 p.197)

Il est usuellement convenu que la documentation n'est pas une priorité pour plusieurs professionnels informatiques. Le contrôle du processus de documentation et le contrôle de la qualité de la documentation sont difficiles à obtenir.

---

12. Mesurer les points de fonction d'un document (spécification des exigences) exprimant un modèle permet d'obtenir des estimations fiables de l'effort nécessaire pour produire les autres artefacts.

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### Après la réalisation

Une fois une première itération du développement terminée, les artefacts (document, logiciel) continuent à être utilisés et nécessitent des adaptations. Cette sous-section débute en exposant deux faits qui mettent en valeur certains écueils sur la documentation. Ensuite, la sous-section présente ces écueils et en décrit les répercussions.

P16. Il y a une grande proportion de logiciels patrimoniaux, ce qui augmente par conséquent l'importance de la documentation. Des logiciels patrimoniaux sont issus «[...] d'une version dépassée et qui continuent d'être utilisés, après des ajustements, en même temps que la technologie contemporaine d'une entreprise. »<sup>13</sup>. Une des raisons de cette grande proportion est que la fin de vie d'une entité informatique est rarement planifiée. Ainsi, le référentiel des connaissances en gestion de projet (PMBOK, *Project Management Body of Knowledge* en anglais) à la version 6 [304] et à la version 7 [305] ainsi que dans l'*Information Technology Infrastructure Library* [19] survolent le sujet en quelques lignes, mais ne l'intègre pas aux procédés de gestion.

En plus, une estimation provenant d'une revue de littérature réalisée en 2024 ( $n = 60$ ) sur le sujet [345] indique qu'entre 39 % et 50 % des logiciels, dont une majorité des logiciels critiques, utilisés dans des pays industrialisés sont des logiciels patrimoniaux. Sans documentation appropriée, la maintenance ou l'adaptation de ces logiciels peut s'avérer plus difficile, voire impossible.

(voir énoncé[s] lié[s] : ED19, ED20, ED21 p.197)

P17. Durant une partie importante de la vie d'un logiciel, la fréquence des changements aux entités informatiques est rapide, il en va de même pour la documentation. Cette rapidité est un défi en soi. Ce phénomène est énoncé comme un constat par plusieurs experts en informatique. Barry W. Boehm indique que cela représente un défi [357]. Ian F. Sommerville en donne les raisons suivantes :

[Traduction libre]

- Nouvelles approches du développement de systèmes, en particulier la construction par configuration. [...]

---

13. Plusieurs autres définitions font intervenir par exemple le niveau de criticité ou de connaissances de l'entité informatique [345]. Néanmoins, ils peuvent appartenir à la définition proposée.

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- Le besoin d’une livraison rapide de logiciels. [...]
- Le rythme croissant de l’évolution des exigences. [...]
- Le besoin d’un meilleur retour sur investissement des actifs logiciels. [...] <sup>14</sup>  
[368]

Pareillement au code source, le professionnel informatique sera amené à faire plusieurs de modifications à la documentation. Ainsi, il doit continuer à comprendre les répercussions de ces changements et à maintenir le tout cohérent.

(voir énoncé[s] lié[s] : ED22 p.198)

**P18. Les changements aux contextes et aux entités informatiques peuvent finir par se répercuter négativement sur la documentation.** D’abord, les acquis du lectorat changent et cela devient plus probable avec les entités informatiques patrimoniales. Cette situation peut rendre l’interprétation d’un document plus incertaine.

- Entre les années 1980 et la fin des années 1990, les changements sur le lectorat des documents informatique étaient suffisamment importants pour devoir en tenir compte, par exemple en considérant le niveau d’expertise en plus du champ d’expertise [218].
- Entre les années 1950 et les années 1990, l’utilisation répandue du diagramme de flux est devenue celle du diagramme de flux de données, puis celle du diagramme d’activités [223].

Ensuite, les documents peuvent ne pas être mis à jour à la suite de changements. Plusieurs sondages menés auprès de professionnels informatiques indiquent qu’il est pratique courante de ne pas mettre à jour la documentation [88, 206, 347]. Si les professionnels informatiques ne disposent pas de temps suffisant pour correctement faire la documentation lors de la réalisation, il est vraisemblable que cela soit de même après la réalisation.

(voir énoncé[s] lié[s] : ED23, ED24 p.198)

---

14.

- *New approaches to systems development in particular, construction by configuration.* [...]
- *The need for rapid software delivery.* [...]
- *The increasing rate of requirements change.* [...]
- *The need for improved ROI on software assets.* [...]

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

P19. La documentation continue d’être négligée, même en sachant les répercussions. Cela fait intervenir le concept de dette technique. Une dette technique est le passif technique résultant de choix opportunistes dont le coût de mise à niveau augmente avec le temps. Déjà lors d’un sondage ( $n = 69$ ), les professionnels informatiques ont indiqué que la documentation peut contribuer à la dette technique [121]. Philippe Kruchten inclut la documentation comme une des multiples activités de développements contribuant à la dette technique [166]. Par ailleurs, une revue de littérature ( $n = 89$ ) [374] conclut même que cette dette conduit à l’échec de l’évolution et de la maintenance du logiciel. Pour plusieurs professionnels informatiques, la documentation est considérée comme une source non fiable d’information et ils préfèrent utiliser le code source comme source d’informations [167, 369].

Pourtant, l’une des particularités de plusieurs documents informatiques est que le professionnel informatique fait partie de l’atorat et du lectorat [316]. C’est qu’un auteur n’est pas sensibilisé aux lectorats, même si un professionnel informatique peut faire partie à la fois de l’atorat et du lectorat [149].

(voir énoncé[s] lié[s] : ED25, ED26, ED27, ED28 p.198)

Le contexte actuel comportant une grande proportion de logiciels patrimoniaux et une fréquence élevé de changements rend la documentation d’autant plus importante. Car, la documentation s’avère être une source d’information pour le professionnel informatique qui devra maintenir ou adapter les logiciels. Néanmoins, cette documentation doit demeurer de qualité.

### 1.1.3 Des approches

Parmi les approches, il y a des instruments, des méthodes et des techniques. Il existe quelques propositions d’instruments qui combinent méthodes et techniques, elles sont présentées au début. Cependant, la majorité des instruments utilisés présentent certaines difficultés sur l’accessibilité et les fonctions offertes. Ainsi, les méthodes et les techniques sont là pour expliquer le fonctionnement que pourraient avoir des instruments pour la documentation informatique. Les méthodes proviennent de l’informa-

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

tique. Les techniques proviennent de l'informatique, du traitement automatique des langues et de l'apprentissage profond.

P20. Voici les définitions d'instrument, de méthode et de technique :

- instrument : « Objet fabriqué servant à exécuter [quelque chose], à faire une opération. » [327] ;
- méthode : « Ensemble de démarches raisonnées, suivies pour parvenir à un but. » [331] ;
- technique : « Ensemble des applications de la science dans le domaine de la production. » [194].

(voir énoncé[s] lié[s] : AD01, AD02, AD03 p.198)

### Des instruments

En premier sont présentées des propositions de solutions pour maintenir cohérent la documentation. S'ensuit une présentation des principaux instruments utilisés en documentation qui pointent vers certaines difficultés d'accessibilité.

P21. Il existe des propositions de logiciels qui combinent des techniques informatiques visant à améliorer la qualité de la documentation. Un article, publié en 2002, intitulé *Lightweight validation of natural language requirements* propose un prototype qui regroupe langage formel et modélisation informatique [100]. D'une façon sommaire, voici les huit étapes comprenant la préparation et l'exécution de la solution :

1. Mise en place d'un ensemble de règles pour le langage appelé aussi grammaire.
2. Mise en place d'un ensemble de règles pour les propriétés du modèle.
3. Mise en place d'un ensemble de règles pour extraire le modèle.
4. Prétraitement sur le texte.
5. Analyse lexicale et syntaxique du texte grâce à la grammaire.
6. Transformation de l'arbre syntaxique en modèle à partir des règles d'extraction,
7. Application des règles pour les propriétés du modèle.

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

8. Validation et correction du document par l'utilisateur, puis retour à l'étape 4.

Dans ce procédé, c'est l'utilisateur qui détermine si le document est valide ou non et qui décide s'il a besoin d'itérer. Ainsi, il demeure en contrôle. À propos de leur approche, les auteurs affirment :

[Traduction libre] Le faible coût de notre approche, tant en termes de formation humaine que de ressources informatiques nécessaires, la rend particulièrement adaptée à l'introduction initiale de méthodes formelles dans une organisation.<sup>15</sup> [100]

(voir énoncé[s] lié[s] : AD04 p.199)

P22. Dans les fonctionnalités demandées, plusieurs professionnels informatiques veulent que les instruments gèrent les relations entre les documents et à l'intérieur de ceux-ci. L'objectif étant que, même lors des mises à jour, la cohérence soit maintenue. Il est même suggéré, dans une revue de littérature sur la documentation, que les documents deviennent exécutables [382]. De cette façon, il n'y aurait plus de désynchronisation et elle serait vérifiable avec des tests. Un autre article propose différentes solutions pour y arriver (tableau 1.3) et comprend méthodes et techniques [66].

(voir énoncé[s] lié[s] : AD05 p.199)

P23. Les détails sur le fonctionnement des instruments peuvent être inaccessibles. D'abord, un article de 2002 relève lors d'un sondage ( $n = 60$ ) les instruments de documentation utilisés par des professionnels informatiques [88]. Plus récemment, une revue de littérature ( $n = 63$ ) de 2022 relève aussi les instruments abordés dans les articles sur lesquels portent cette même revue de littérature. Le tableau 1.4 compare les deux relevés d'instruments. À l'intérieur figurent les instruments Word et Confluence qui sont très utilisés et demeurent des solutions propriétaires.

Ensuite, dans les éléments qui peuvent aider à comprendre le fonctionnement d'un instrument, il y a la documentation (à la condition qu'elle soit de qualité [à jour,

---

15.

*The low cost of our approach, both in terms of human training and of computational resources needed, makes it particularly well suited for the initial introduction of formal methods in an organization.*

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABLEAU 1.3** – Sommaire des problèmes et des solutions [Traduction libre] (inspiré de [66])

| Problème                                                                                                                                  | Solution                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| • Comment structurer l'information dans la documentation ?                                                                                | • Organiser les contenus en fonction de leur domaine, de sorte que la forme d'information soit directement liée aux concepts du domaine.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| • Comment garantir la cohérence entre des artefacts indépendants ayant un contenu similaire ?                                             | • Utiliser un environnement intégré, où plusieurs types d'artefacts et leurs relations peuvent être gérés de manière uniforme.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| • Comment préserver la cohérence de la documentation lorsque des fragments de contenu connexes sont dispersés dans différents documents ? | <ul style="list-style-type: none"><li>• Veiller à ce que les fragments de contenu connexes soient facilement accessibles les uns aux autres, par exemple via une source unique, des liens, des vues ou la transclusion, afin de faciliter l'évaluation de leur cohérence.</li><li>• Utiliser des relations explicites entre les différentes ressources pour séparer les contenus connexes et permettre aux lecteurs de naviguer aisément entre elles. (variante : liens)</li><li>• Intégrer les fragments d'information connexes dans un même document pour les maintenir facilement accessibles. (variante : source unique)</li><li>• Importer le contenu d'un fragment d'information dans un document en utilisant une référence à celui-ci. (variante : transclusion)</li><li>• Créer un document virtuel composé de différents fragments d'information. (variante : vues)</li></ul> |
| • Quand faut-il mettre à jour une information connexe dans la documentation ?                                                             | <ul style="list-style-type: none"><li>• Lorsqu'une modification est introduite, mettre à jour les informations associées.</li><li>• Lorsqu'une modification est introduite, mettre à jour toutes les informations associées. (variante : coévolution synchrone)</li><li>• Lorsqu'une modification est introduite, mettre en place des mécanismes de suivi des modifications associées en attente et mettre à jour les informations les plus pertinentes uniquement lorsque cela est nécessaire. (variante : coévolution décalée dans le temps)</li></ul>                                                                                                                                                                                                                                                                                                                                |

<sup>a</sup> Il peut exister plusieurs solutions pour un même problème, elles sont appelées variantes et peuvent s'exécuter sous des conditions similaires ou différentes.

cohérente, etc.]) et le code source. Ces deux artefacts peuvent venir avec des conditions d'accessibilité. Pour un utilisateur d'une solution propriétaire, il arrive que la documentation nécessite un coût supplémentaire, tandis que le code source lui sera rarement accessible.

(voir énoncé[s] lié[s] : AD06 p.199)

## Des méthodes informatiques

Les méthodes en informatique peuvent comprendre des méthodes informatiques de développement et des méthodes informatiques de documentation. Toutefois, certaines



## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABEAU 1.4** – Relevés des catégories d’instruments et leurs fréquences. (inspiré de [88, 382])

| Instruments utilisés par des professionnels informatiques provenant d'un sondage de 2002                                                                                                                                                                                                                             | Instruments avec descriptions abordés lors d'articles provenant d'une revue entre 2004 à 2018                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• traitement de texte (MS Word) &lt;22&gt;;</li><li>• générateur de documentation (JavaDoc, Doxygen, Doc++) &lt;21&gt;;</li><li>• éditeur de texte &lt;9&gt;;</li><li>• le système Rational Rose &lt;5&gt;;</li><li>• autre système (Control Centre, IDE) &lt;3&gt;.</li></ul> | <ul style="list-style-type: none"><li>• traitement de texte (Word, Excel, Powerpoint) &lt;40&gt;;</li><li>• moteur de wiki (Confluence) &lt;39&gt;;</li><li>• gestionnaire de code source (Git, Mercurial, SVN) &lt;5&gt;;</li><li>• instrument personnalisé (Script) &lt;9&gt;;</li><li>• éditeur de langage balisé (Markdown) &lt;2&gt;;</li><li>• communication verbale &lt;7&gt;.</li></ul> |

<sup>a</sup> Des exemples sont fournis entre parenthèses.

<sup>b</sup> La fréquence d'apparition de la catégorie d'instrument dans les articles est présentée entre chevrons.

méthodes peuvent emprunter des techniques à d'autres méthodes. C'est le cas des deux méthodes suivantes : **Software design documentation** et **Literate programming**.

P24. Ces méthodes nécessitent de définir les concepts suivants :

- organiser : « Doter d'une structure, d'une constitution déterminée, d'un mode de fonctionnement. » [332] ;
- module : est un composant comprenant une interface ;
- couplage : mesure l'interdépendance entre des composants logiciels ;
- adaptabilité : la capacité d'une entité à être modifiée afin de répondre à des changements d'environnements ou de besoins.

Il est utilisé le terme **cohérence** à **cohésion** dans le mémoire. Il s'est avéré que le concept de cohésion comme celui de couplage possède une grande variabilité en informatique appliquée [53]. Lorsque le terme cohésion est décrit, il fait souvent référence à l'utilisation d'une logique (séquentielle, temporelle, fonctionnelle, etc.). Ainsi, le terme cohérence est plus approprié que cohésion. D'ailleurs, Bertrand Meyer indique que la cohérence est la caractéristique désirée et résultant d'une encapsulation appropriée [221].

(voir énoncé[s] lié[s] : AD07, AD08, AD09, AD10, AD11 p.199)

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

P25. Une méthode est d'utiliser les techniques qui améliorent l'adaptabilité des documents comme il est possible de le faire pour le logiciel. D'abord, la programmation structurée et l'encapsulation avec la spécification des antécédents et des conséquents permettent de diminuer le couplage et de maintenir la cohérence et, ainsi d'obtenir une modularité qui améliore l'adaptabilité d'un logiciel [26].

Un article de S. D. Hester et David L. Parnas présente une méthode, la **Software design documentation**, qui applique ces techniques à la documentation [115]. Constatant la faible adhésion à cette méthode, David L. Parnas, Paul Clements et David Weiss proposent un exemple plus réaliste en s'appuyant sur le Operational Flight Program de l'avion A-7E [291].

Néanmoins, même lors de l'utilisation de langage formel, ces techniques sont peu efficacement utilisées. Une analyse réalisée sur cent logiciels en code source libre sur GitHub détermine qu'il faudrait un effort important pour diminuer le couplage et pour augmenter la cohérence [47].

(voir énoncé[s] lié[s] : AD12, AD13, AD14 p.200)

P26. La programmation littéraire<sup>16</sup> est une méthode qui veut aider à la compréhension du lectorat. Donald E. Knuth, l'auteur de la programmation littéraire, veut prioriser le lectorat à la machine [163]. Dans un même document, les explications (documentation) sont jumelées à des instructions (code source). Puis un instrument s'occupe de traiter ces informations. D'ailleurs, les instruments qui permettent d'exécuter de la programmation littéraire ont continué de s'adapter pour ajouter de nouvelles fonctionnalités comme [301] :

- utiliser d'autres langages dans les documents en entrées ;
- fournir d'autres formats de sorties ;
- utiliser des hyperliens pour faire des références croisées ;
- fournir des intégrations avec d'autres instruments.

Toutefois, maintenir la cohérence lors de modifications demeure problématique, conclut l'auteur d'Elucidator un instrument de programmation littéraire [232]. Le Theme-based

---

16. Cette traduction de *literate* fait référence à la langue utilisée dans les essais (scientifiques), par opposition à la langue littéraire (romans, poésie) et aux langages formels (utilisés en mathématiques et en programmation).

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

literate programming propose l'idée de faire correspondre les informations à un modèle et d'utiliser différentes techniques pour maintenir le modèle cohérent [154]. Cette approche propose l'utilisation de ces trois langages :

- langage de balisage extensible (XML, *Extensible Markup Language* en anglais) pour représenter le document ;
- langage de transformation XML (XSLT, *eXtensible Stylesheet Language Transformations* en anglais) pour spécifier les transformations ;
- langage pour la définition de type de document (DTD, *Document Type Definitions* en anglais) pour spécifier le modèle.

Depuis plusieurs années, un instrument du nom de Jupyter notebook gagne en popularité. Cet instrument permet de créer une documentation à partir de texte et de code source exécutable. Plusieurs l'associent à de la programmation littéraire [302]. Pourtant, l'objectif n'est pas de créer une entité informatique, mais de partager des idées [109]. Par conséquent, les Jupyter notebooks ont une faible reproductibilité, au plus 15 %, pour un échantillon ( $n = 1\,159\,568$ ) provenant de GitHub [302] et encourageraient de mauvaises pratiques de programmation [298].

(voir énoncé[s] lié[s] : AD15, AD16, AD17, AD18 p.200)

### Des techniques informatiques (donc, formelles)

Dans les techniques informatiques, il y a les langages formels. L'utilisation de langages formels permet de traiter formellement les documents avec des machines, comme ce fût le cas avec la programmation littéraire. À cela, s'ajoutent quelques exemples d'utilisation de modèles relationnels pour maintenir la cohérence entre les documents et à l'intérieur de ceux-ci.

P27. Avant d'aborder les techniques utilisées pour documenter en informatique appliquée, il est nécessaire de définir ces concepts :

- langage : système pour communiquer comprenant des symboles et des règles ;
- langage formel : est composé d'ensembles explicites de symboles et d'un ensemble explicite de règles appelé grammaire formelle ;

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- langage naturel : est un langage non formel ;
- architecture informatique : organisation d'entités informatiques résultant de la phase de conception ;
- langage de description d'architecture (ADL, *architecture description language* en anglais) : langage avec une syntaxe et une sémantique spécifiées permettant de décrire une architecture ;
- ontologie informatique : « corpus structuré de concepts, qui est modélisé dans un langage permettant l'exploitation par un ordinateur des relations sémantiques ou taxonomiques établies entre ces concepts. »<sup>17</sup> [267].

(voir énoncé[s] lié[s] : AD19, AD20, AD21, AD22, AD23, AD24 p.200)

P28. Les langages formels permettent à des instruments d'assurer entre autres l'adaptabilité des documents. D'abord, le XML est un exemple de langage formel qui permet d'exprimer des informations selon une structure. Il est une déclinaison du Standard Generalized Markup Language dont l'objectif est de permettre le partage de document [161].

Ensuite, il y a les langages de schémas qui servent à vérifier les XML. Une norme sur les langages de schémas en comprend six, dont la définition des types de document (DTD, *Document Type Definition* en anglais), la définition du schéma XML (XSD, *XML Schema Definition* en anglais) et le Schematron [134].

Néanmoins, le XML et les langages de schéma ne permettent pas d'exprimer pleinement la sémantique des données, et ce, même pour un langage de schéma aussi poussé que le Schematron [151]. Cela entraîne des difficultés pour réaliser des transformations adéquates vers le format Web Ontology Language (OWL) qui sert aux ontologies informatiques [17].

Pour finir, le XML, le DTD ou le XSD sont utilisés par deux instruments de documentation et une architecture de documentation. Ce sont DocBook, S1000D et l'architecture des types d'informations Darwin (DITA, *Darwin Information Type Architecture* en anglais) [86, 358]. Dans les fonctionnalités offertes par le DITA, il y a au moins la modularité et la réutilisation de contenu qui aident à l'adaptabilité [358].

(voir énoncé[s] lié[s] : AD25, AD26, AD27, AD28, AD29 p.201)

---

17. Une **ontologie informatisée** a pour source (origine) une **ontologie générale** à laquelle il est présumé qu'elle tente d'être le plus fidèle possible.

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

P29. L'utilisation de langages de descriptions d'architecture est une technique, mais plusieurs utilisateurs préfèrent les langages non formels. Un article répertorie 177 différents langages pour architecture [211]. Plusieurs utilisateurs des langages d'architecture voient un intérêt à bien définir la sémantique, il s'agit de la troisième fonctionnalité la plus utile, selon eux, après les multiples vues architecturales ou un langage exprimé graphiquement.

Néanmoins, plusieurs articles affirment le désintérêt des utilisateurs dans l'utilisation de langages formels pouvant aller jusqu'à exprimer la sémantique [211, 285, 347]. Selon plusieurs sondages, l'Unified Modeling Language (UML) est plus souvent utilisé que les langages de descriptions d'architecture (ADL, Architecture Description Language en anglais) [211, 347].

(voir énoncé[s] lié[s] : AD30, AD31, AD32 p.202)

P30. Utiliser une base de données relationnelle pour maintenir la cohérence de la documentation<sup>18</sup>. D'abord, Scott R. Tilley, dans un article publié en 1993, souligne que la documentation a peu évolué en trente ans et identifie ces trois problèmes [383] : [Traduction libre] « [...] est conçu pour de petites entités informatiques, est généralement obsolète et fournit une seule perspective [...] »<sup>19</sup>. Il modélise une solution qui consiste à maintenir cohérents le code source et la documentation d'architecture. Il n'explicite pas l'utilisation d'une base de données, mais y explique son modèle relationnel et la forme des tuples (*objects, relationships*).

Ensuite, un article portant sur l'utilisation du S1000D dans le secteur critique des trains à haute vitesse indique utiliser Common Resource Data Base, une base de données relationnelle pour maintenir la cohérence [401]. À l'intérieur de la spécification du S1000D, il y est décrit que le Common Resource Data Base contient que la structure de données (objets d'informations), c'est-à-dire le modèle sans fournir les fonctions nécessaires aux applications l'utilisant [5].

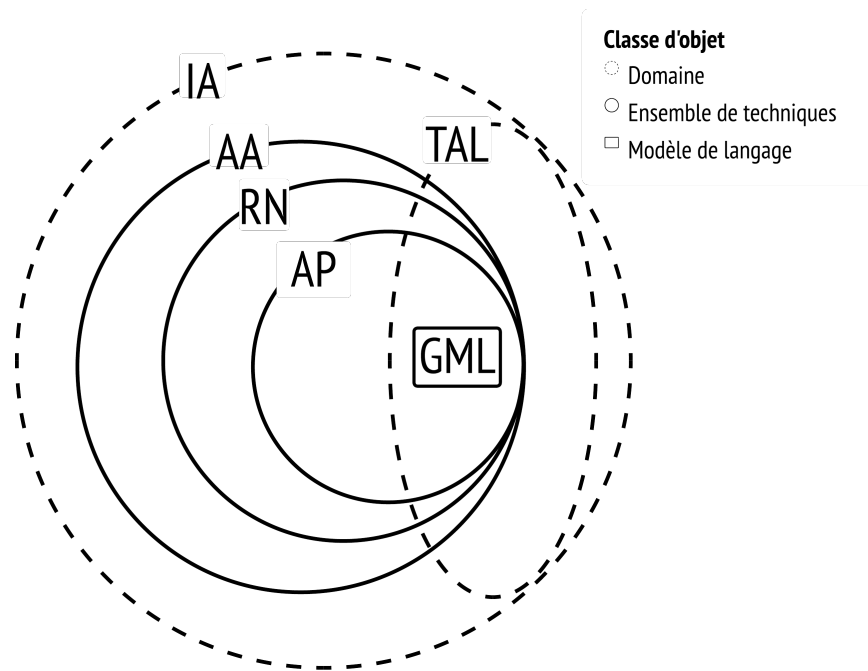
(voir énoncé[s] lié[s] : AD33 p.202)

---

18. Le modèle est le résultat de la modélisation qui est une méthode. La modélisation en informatique regroupe un ensemble de techniques (diagramme entité-association, diagramme de flux de données et etc.).

19. « *in-the-small, usually out-of-date, provides just one perspective* »

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE 1.1** – Diagramme de Venn sur les domaines, techniques et modèle de langage

### Des techniques de traitement automatique des langues

Cette sous-section aborde les machines en mesure de traiter les documents, peu importe les langages employés. Il y aura une présentation des domaines, des techniques et des modèles de langage ainsi que des limitations existantes.

P31. Pour éviter les ambiguïtés, ce paragraphe regroupe des définitions, puis un diagramme des relations logiques entre des ensembles (figure 1.1) :

- intelligence artificielle (IA ou AI, *Artificial Intelligence* en anglais : [Traduction libre] « [...] <discipline> recherche et développement des mécanismes et applications des systèmes d'IA [...]. »<sup>20</sup> [143] ;
- traitement automatique des langues (TAL ou NLP, *Natural Language Processing* en anglais) : « Technique d'apprentissage automatique qui permet à l'ordinateur de comprendre le langage humain. » [279] ;

20. « [...] <discipline> research and development of mechanisms and applications of AI systems [...]. »

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- apprentissage automatique (AA ou ML, *Machine Learning* en anglais) : [Traduction libre] « [...] processus d’optimisation des paramètres d’un modèle par des techniques de calcul [...]. »<sup>21</sup> [143];
- réseaux de neurones (RN ou NN, *Neural Networks* en anglais) : [Traduction libre] « [...] réseau d’une ou plusieurs couches de neurones [...] »<sup>22</sup> [143];
- apprentissage profond (AP ou DL, *Deep Learning* en anglais) : « Mode d’apprentissage automatique généralement effectué par un réseau de neurones artificiels composé de plusieurs couches de neurones hiérarchisées [...]. » [238];
- grand modèle de langage (GML ou LLM, *Large Language Model* en anglais) : « Modèle de langage constitué par apprentissage profond à partir de mégadonnées. » [254];
- boîte noire : « Système fermé dont on ignore le fonctionnement interne, qui n’est connu que par son comportement extérieur en fonction des sollicitations qui lui sont appliquées. » [242].

(voir énoncé[s] lié[s] : AD34, AD35, AD36, AD37, AD38, AD39 p.202)

P32. Toute la documentation écrite en langage non formel peut alimenter le développement de techniques de traitement automatique des langues puisque ces techniques traitent ce type d’information. D’abord, plusieurs professionnels informatiques affirment que l’architecture et les exigences s’expriment couramment en langage naturel. Selon deux sondages, cette proportion atteint 90 % ( $n = 109$ ) pour l’architecture [347] et 69 % ( $n = 117$ ) pour les exigences [159].

Ensuite, les documents de spécification des exigences sont ceux qui emploient le plus les techniques de TAL. C’est ce qui ressort d’une revue de littérature portant sur des articles publiés entre 1983 et 2019 [411].

Pour finir, les techniques de TAL sont de plus en plus exploitées dans la littérature scientifique. Une tentative de taxinomie est réalisée sur le TAL à partir de quatre-vingt mille articles provenant de l’Association for Computational Linguistics

---

21. « [...] *process of optimizing model parameters through computational techniques* [...]. »

22. « [...] *network of one or more layers of neurons* [...]. »

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

et le nombre d'articles suit une courbe exponentielle entre 1952 et 2023 [356].

(voir énoncé[s] lié[s] : AD40, AD41, AD42 p.203)

P33. Plusieurs techniques de TAL<sup>23</sup> ont des lacunes scientifiques. D'abord, il n'y a pas de règles exactes pour sélectionner adéquatement la chaîne ou sélectionner la classe de modèle à entraîner [98]. En fait, une chaîne (*pipeline*) prépare, développe et exécute un modèle de langage qui servira pour le TAL. La pratique actuelle se contente de fournir des exemples de chaînes et certains experts appellent à utiliser l'intuition [351]. À la fin, les résultats obtenus de plusieurs de ces techniques de TAL sont en fait des modèles de type « boîte noire » qui sont plus difficilement explicables. [124]. Néanmoins, il y a de plus en plus d'articles qui s'intéressent à comprendre les modèles [23] et certains peuvent fournir des explications demeurant incertaines, voire le plus souvent douteuses. Cynthia Rudin souligne des problèmes existants avec plusieurs de ces explications :

[Traduction libre]

- Les méthodes d'apprentissage automatique explicables fournissent des explications qui ne reflètent pas fidèlement les calculs du modèle original.
  - Ces explications sont souvent incohérentes ou insuffisamment détaillées pour comprendre le fonctionnement des modèles de type « boîte noire ».
- <sup>24</sup> [350]

Pour finir, il y a des manquements importants pour assurer la reproductibilité.

- Un article publié en 2025 énonce cinq items essentiels à la reproductibilité : versionner le code, avoir accès aux données, versionner les données, avoir les journaux de l'expérimentation ainsi que la chaîne qui a servi à créer le modèle [359].

---

23. Pour rappel, le traitement automatique des langues comprend les techniques d'apprentissage automatique, de réseaux de neurones et d'apprentissage profond.

24.

- *Explainable ML methods provide explanations that are not faithful to what the original model computes.*
- *Explanations often do not make sense or do not provide enough detail to understand what the black box is doing.*



## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- Une analyse sur 24 expérimentations importantes sur des modèles conclut que [317] :
  - 14 n'ont pas indiqué de signification statistique ;
  - 17 ne comportaient pas de scripts pour la réplication des résultats ;
  - la plupart souffraient d'une documentation insuffisante.
- Plusieurs experts vont jusqu'à affirmer qu'il n'y a pas d'accumulation fiable des connaissances [364, 399].

(voir énoncé[s] lié[s] : AD43, AD44, AD45, AD46, AD47 p.203)

### Des techniques d'apprentissage profond

Certaines techniques de TAL sont si répandues qu'elles nécessitent leurs propres sous-sections, il s'agit des techniques d'apprentissage profond. Il est expliqué que, même avec leurs limitations, les professionnels informatiques sont prêts à les utiliser et à les offrir. Il est présenté qu'il existe une absence d'expérimentations démontrant sans équivoque les avantages pour la documentation et une absence d'analyses approfondies des risques. La sous-section se termine en envisageant l'intégration ou la coexistence de ces techniques.

**P34. Malgré certaines limites en matière de reproductibilité, l'apprentissage profond demeure utile pour de nombreux individus.** Les techniques d'apprentissage profond comptent parmi les techniques les plus utilisées en TAL [351]. Elles fournissent des résultats jugés satisfaisants. Les experts du domaine vont jusqu'à changer les bancs d'essai lorsqu'ils leur semblent suffisamment réussis [213]. Un sondage mené auprès de quatre-vingt-dix mille professionnels informatiques indique qu'une majorité utilise les instruments basés sur les grands modèles de langage (GML), donc produits par l'apprentissage profond[214]. Les réponses indiquent notamment que :

- 82 % s'en servent pour écrire du code source ;
- 48 % s'en servent comme une aide envers du code source ;
- 55 % voudraient s'en servir pour tester du code source ;

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

— 50 % voudraient s’en servir pour documenter du code source.

(voir énoncé[s] lié[s] : AD48, AD49, AD50 p.204)

**P35. Plusieurs caractéristiques de qualité ne sont pas respectées ou démontrées comme telles par les grands modèles de langage.** D’abord, la cohérence ainsi que les ambiguïtés provenant de la langue et du contexte demeurent même pour les GML [160, 407]. Les ellipses<sup>25</sup> en sont un exemple [52]. Ces ambiguïtés peuvent conduire les GML à donner deux réponses contradictoires.

Ensuite l’efficacité n’est pas démontrée pour la rédaction des documents de spécifications des exigences. Pourtant, il s’agit d’un sujet plusieurs fois abordé dans l’utilisation des GML. Une revue de littérature trouve 22 articles qui abordent le sujet [212]. Parmi ces articles, 15 abordent l’efficacité, mais seulement quatre établissent un comparatif. L’accès à trois des quatre comparatifs confirme l’utilisation d’un échantillon de dix personnes ou moins [169, 342, 395].

Puis ce qui est de la réfutabilité, un article soulève les risques de contamination des jeux de données servant à l’entraînement. Cette situation aurait comme effet d’invalider des comparatifs entre les GML et de former des hypothèses trompeuses lors de l’étude des techniques de TAL [352].

Enfin, en ce qui concerne l’éthique, Laura Weidinger publie en 2022, avec 22 autres auteurs, l’article *Taxonomy of Risks posed by Language Models*. Plusieurs des risques soulevés sont des enjeux éthiques, et les techniques de mitigation réduisent le risque, mais ne l’élimine pas entièrement [400]. Par ailleurs, plusieurs de ces risques se concrétiseront inévitablement à une fréquence difficile, voire impossible à évaluer (attendu la faible documentation des méthodes et des instruments). Dès lors il faudra avoir recours à des techniques de palliation qui sont très rarement abordées suite à l’exposé de la mitigation. Une analyse judicieuse des coûts-bénéfices des instruments d’IA devrait contenir les coûts de mitigation et de palliation, comme cela le serait pour tout projet [310].

(voir énoncé[s] lié[s] : AD51, AD52, AD53, AD54 p.204)

---

25. Il s’agit de l’« omission d’un ou plusieurs mots dans une phrase qui reste cependant compréhensible. » [325]

## 1.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

P36. Les techniques d'IA ne remplacent pas les techniques informatiques. Un sondage mené en 2020 auprès de cent trente experts en méthodes formelles [378] révèle que 93 % d'entre eux estiment que l'IA ne remplacera probablement pas, ou ne remplacera définitivement pas, les méthodes formelles. Parmi les explications fournies, il y a :

[Traduction libre]

[—] [...] seules les méthodes formelles peuvent garantir l'exactitude [...]

[—] [...] le rôle crucial des méthodes formelles dans la spécification des exigences : « En fin de compte, tant le contexte ambigu que la traduction vers un contexte non ambigu sont caractéristiques des activités humaines. J'ai de la difficulté à imaginer que ces aspects créatifs puissent être entièrement automatisés. »

[—] [...] nous avons besoin d'approches combinant les techniques basées sur les modèles et les techniques basées sur les données [...] <sup>26</sup> [378]

De plus, les auteurs de l'ouvrage de référence *Artificial Intelligence A Modern Approach* expliquent pourquoi, avec un titre semblable, un grand nombre des techniques informatiques y sont précisément décrites et que les modèles transformateurs <sup>27</sup> utilisés pour les GML ne sont pas suffisant :

[Traduction libre] Il se peut que les modèles transformateurs et leurs dérivés apprennent des représentations latentes qui capturent les mêmes idées fondamentales que les grammaires et l'information sémantique, ou bien il se peut que quelque chose de totalement différent se produise au sein de ces modèles gigantesques ; nous l'ignorons tout simplement. <sup>28</sup> [351]

---

26.

[—] [...] *only formal methods can provide guarantees about correctness* [...]

[—] [...] *the crucial role of formal methods in requirements specification : " at the end of the day, both the ambiguous setting and the mapping to the unambiguous setting are characteristic of human activities. I have a hard time imagining that these creative aspects can be fully automated "*.

[—][...] *we need approaches that combine model-driven and data-driven techniques* [...].

27. Le terme **modèle transformateur** signifie « Architecture de réseau de neurones artificiels fondée sur le mécanisme d'attention [...] ». [281]

28.

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

Durant toute la revue de littérature, le terme **modèle** a été utilisé pour décrire les résultats obtenus des différentes techniques de TAL.

Néanmoins, l’usage de ce terme en IA désigne tout autre chose et cela est expliqué dans la prochaine section.

(voir énoncé[s] lié[s] : AD55 p.204)

## 1.2 Des modèles en informatique appliquée

Plusieurs documents en informatique appliquée véhiculent des modèles au point que, pour des professionnels informatiques, les documents et les modèles se confondent. Pourtant, ils ne partagent pas exactement les mêmes objectifs, écueils et approches. C’est ce qui est présenté dans cette section.

- P37. Avant de définir le type de modèle utilisé dans ce mémoire (modèle formel), il convient de définir ce qu’est un modèle en règle générale. Un modèle est défini comme étant la représentation d’un sujet pour en permettre le traitement. Il existe un lien entre le modèle et le langage. Guy Caplat l’illustre à l’aide d’un diagramme du langage de modélisation unifié (UML, *Unified Modeling Language* en anglais) présentant le modèle pouvant s’exprimer dans un langage artificiel ou naturel [49]. Van Gigch va plus loin en associant la création d’un modèle à l’utilisation de langages, de métalangages et de logiques [389].

(voir énoncé[s] lié[s] : M01 p.205)

---

*It may be that transformer models and their relatives are learning latent representations that capture the same basic ideas as grammars and semantic information, or it may be that something entirely different is happening within these enormous models ; we simply don’t know.*

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

### 1.2.1 Des objectifs

Les modèles sont abondamment utilisés en informatique, notamment pour traiter les informations. Les modèles en informatique répondent aussi à des objectifs plus globaux. L'informatique étant une science, les objectifs des modèles en science s'appliquent aussi à l'informatique.

P38. Pour en faire découvrir les différents objectifs, plusieurs concepts sont nécessaires. Les concepts suivants reposent sur l'utilisation de langages formels :

- système logique : est composé d'un langage formel et d'un ensemble de règles (axiome, proposition, logique) permettant d'inférer ;
- machine abstraite : système formel comprenant les instructions nécessaires au traitement automatique de données<sup>29</sup> ;
- modèle formel : est composé d'un langage formel et d'un ensemble de règles et vise la représentation d'un sujet pour en permettre le traitement.

(voir énoncé[s] lié[s] : OM01, OM02, OM03 p.205)

P39. Les systèmes logiques permettent de raisonner sur les systèmes formels, et plus particulièrement, sur les modèles formels à travers des machines abstraites. Cependant, pour qu'un logiciel forme un système formel ou un modèle formel [69] et qu'une machine abstraite soit en mesure de l'exécuter avec l'aide d'un système logique, il faut que le système formel et le système logique partagent le même langage formel. D'ailleurs, les machines abstraites sont nombreuses. L'ouvrage *Foundations of computer science* d'Alfred V. Aho et d'Ullman D. Jeffrey énumère sept niveaux d'abstractions faisant intervenir différents types de machines abstraites [8]. Ces niveaux varient de la logique numérique jusqu'au programme applicatif. C'est la capacité des machines à se modifier elles-mêmes qui rend possibles ces niveaux d'abstraction [113]. Cette capacité a été atteinte avec l'introduction de mémoires réinscriptibles sur les composantes matérielles. Aussi, il n'existe pas de système logique unique. Ils peuvent varier tout comme les logiques qui y sont employées [94].

(voir énoncé[s] lié[s] : OM04, OM05, OM06 p.205)

---

29. Les données sont la forme explicite de l'information. Elles constituent des artefacts.

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

P40. Des modèles sont utilisés et créés lors d'un procédé de développement. L'informatique appliquée crée des modèles de problèmes et des modèles de solutions [147]. Le modèle du problème sert à définir, à s'approprier, à circonscrire et à caractériser les solutions acceptables ou tout autre problème physique et tangible. Le modèle de la solution conduira à réaliser et établir une entité informatique. Un modèle de solution peut répondre à un problème réel (calcul du carburant requis par un avion pour un trajet donné) ou même à un problème technique de l'informatique (machine abstraite, compilateur). Comme les autres sciences, ces modèles peuvent servir à apprendre, à développer et à appliquer les connaissances [387].

Cependant, l'informatique va souvent plus loin dans l'instrumentalisation des modèles en créant des modèles traitants d'autres modèles.

(voir énoncé[s] lié[s] : OM07, OM08 p.205)

### 1.2.2 Des écueils

P41. Les écueils aux modèles sont ceux par rapport à des caractéristiques de qualité d'un modèle. Il existe plusieurs modèles de qualité. Certaines caractéristiques de qualité reviennent de manière récurrente : la cohérence, la couverture, la complétude et la compréhensibilité. On les retrouve dans le corpus de connaissances en génie logiciel (SWEBOK, *Software Engineering Body of Knowledge* en anglais) [396], dans un ouvrage consacré à la qualité des modèles conceptuels [233] ainsi que dans le manuel de John Krogstie *Quality in Business Process Modeling*, comprenant entre autres la qualité des documents de spécification des exigences [165]. Ainsi, les écueils sont énoncés selon ces caractéristiques de qualité.

(voir énoncé[s] lié[s] : EM01 p.206)

P42. La définition de ces caractéristiques de qualité peuvent varier d'une référence à une autre. Par conséquent, il est donc préférable de les définir précisément :

- cohérent : système logique où aucune contradiction n'a encore été découverte ;
- couverture : consiste à couvrir tous les éléments d'un ensemble avec une sous-famille d'un autre ensemble, et ce, la plus petite possible ;

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

- complétude : un système formel est complet lorsque toutes les vérités qu'il contient peuvent être déduites des axiomes et des règles d'inférence ;
- compréhensible : qui peut être intelligible par le lectorat.

(voir énoncé[s] lié[s] : EM02, EM03, EM04, EM05 p.206)

P43. Il y a une augmentation de l'utilisation de langages non formels, cela nuit à la cohérence. L'UML en est un des exemples les plus représentatifs [233, 283]. Ce langage non formel de modélisation vise à faciliter les actions suivantes :

[Traduction libre]

- Spécifier, visualiser, comprendre et documenter les problèmes.
- Capter, communiquer et exploiter les connaissances pour résoudre les problèmes.
- Spécifier, visualiser, construire et documenter les solutions. <sup>30</sup> [11]

L'UML aide à la compréhension, mais son insuffisance est telle qu'il nécessite la création d'extensions [373]. Le langage de modélisation des système (SysML, *SYStems Modeling Language* en anglais) en est un exemple [120]. Les auteurs de l'ouvrage *Model-Driven Software Development : Technology, Engineering, Management* cantonnent l'utilité de l'UML à la visualisation et à la documentation [391]. Ils soulignent que l'UML n'est pas un langage formel qui décrit la sémantique [391]. Cependant, l'ouvrage *SysML for systems engineering* classe le SysML comme pouvant contribuer à des *semi-formal scenarios* [120]. Ce qui les distingue des méthodes formelles.

Dans le contexte où les professionnels informatiques ont tendance à utiliser des langages non formels, la description des systèmes peut être réalisée de manière formelle, mais, en pratique, les approches plus intuitives utilisant des langages de modélisation graphique sont utilisées [75]. Les documents de spécification des exigences comprennent plus souvent des langages non formels que formels [43]. Avec l'augmentation de l'utilisation de langages non formels, l'utilisation de systèmes logiques pour déceler

---

30.

- *Specifying, visualizing, understanding, and documenting problems.*
- *Capturing, communicating, and leveraging knowledge in problem solving.*
- *Specifying, visualizing, constructing, and documenting solutions.*

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

les incohérences sans traitement préalable n'est plus envisageable.

(voir énoncé[s] lié[s] : EM06, EM07, EM08, EM09 p.206)

P44. Obtenir les différentes couvertures nécessite une synchronisation entre le sujet, le modèle et l'artefact. D'abord, le partage d'informations pour obtenir de meilleurs modèles devrait être amélioré. Victor R. Basili écrit :

[Traduction libre] Les chercheurs en génie logiciel ont besoin de laboratoires industriels leur permettant d'observer, de construire et d'analyser des modèles. [...] Les modèles de processus et de produits élaborés par les chercheurs doivent être adaptés aux données recueillies au sein de l'organisation et pouvoir évoluer en continu en fonction de l'expérience acquise par celle-ci.<sup>31</sup> [24]

Ensuite, les informations sur le sujet, le modèle et l'artefact peuvent être inaccessibles, éparpillées et changeantes. Alan W. Brown décrit ces problématiques :

[Traduction libre]

- On parvient rarement, voire jamais, à une compréhension complète de l'espace du problème.
- [...] de nombreuses contraintes pèsent sur les solutions à envisager [...].
- De nombreux types de modifications doivent être pris en compte à toutes les étapes du développement [...].
- Un projet de développement logiciel de grande envergure est un projet d'ingénierie impliquant des équipes de personnes aux compétences diverses, interagissant sur une période prolongée [...].<sup>32</sup> [41]

---

31.

*Software engineering researchers need industry-based laboratories that allow them to observe, build and analyze models. [...] The models of process and product generated by researchers should be tailored based upon the data collected within the organization and should be able to continually evolve based upon the organization's evolving experiences.*

32.

- *We rarely, if ever, get a full understanding of the problem space.*
- *[...] here are many constraints on the solutions to consider [...].*
- *Many kinds of changes must be addressed at all stages of development [...].*



## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

La synchronisation est nécessaire, mais pas suffisante. Il faut aussi que les trois procèdent de la même intention (intention qui se traduit par une vision, des attentes et un but commun).

(voir énoncé[s] lié[s] : EM10, EM11 p.207)

P45. Plusieurs logiciels concrétisant des systèmes formels n’arrivent pas à démontrer leur cohérence ou leur complétude et encore moins les deux à la fois. D’abord, les théorèmes d’incomplétude de Gödel démontrent [29, 286] :

- qu’un système formel cohérent pouvant interpréter une arithmétique de Peano est incomplet ;
- qu’un système formel cohérent pouvant interpréter une arithmétique de Peano ne peut pas vérifier sa cohérence.

Or, l’arithmétique de Peano est un système formel de l’arithmétique élémentaire qui permet des raisonnements par récurrence.

Ainsi, un logiciel concrétisant un tel système aura les mêmes limitations. Néanmoins, les nombreuses bornes existantes en informatique<sup>33</sup> permettent à certains logiciels de démontrer leur cohérence. Il est important de garder cela à l’esprit.

(voir énoncé[s] lié[s] : EM12, EM13, EM14, EM15 p.207)

P46. La base de connaissances sur les modèles n’est pas établie, ce qui peut nuire à la compréhension. Tout d’abord, il n’existe pas de consensus concernant les modèles. Il y a une absence de classification adéquate des modèles [78]. Plusieurs ouvrages sont consacrés entièrement à ce sujet. L’ouvrage *Body of Knowledge for Modeling and Simulation : A Handbook by the Society for Modeling and Simulation International* explique que les différents contextes et les différentes disciplines constituent probablement l’une des raisons pour lesquelles aucun consensus n’existe [283].

Ensuite, il n’y a pas de terminologie établie pour la modélisation en ingénierie. En 2018, pour y remédier, un article propose de développer un corpus de connaissances

---

— *A large software development task is an engineering project involving teams of people with different skills interacting over an extended period of time [...].*

33. Derrière les types ou le temps se trouvent des modèles finis. Les machines ont aussi des ressources finies.

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

en génie logiciel basé sur les modèle (MBEBOK, *Model-Based Software Engineering Body of Knowledge* en anglais) [57]. En 2026, le site officiel du MBEBOK présente encore une publication d'Alfonso Pierantonio de 2020 qui laisse croire que le glossaire n'a toujours pas été établi<sup>34</sup>.

Finalement, le vocabulaire utilisé peut confondre un modèle avec ses artefacts. C'est ce qui se produit souvent entre l'échéancier et le modèle d'échéancier. Le PMBOK fait la distinction, mais dans la langue courante ce n'est pas le cas [280, 310, 336]. David L. Parnas indique le problème dans les articles portant sur les méthodes formelles où modèle et spécification sont couramment interchangeables [293], puisque les deux peuvent s'avérer être des descriptions (artefacts) où la spécification contient ou réfère à des modèles.

(voir énoncé[s] lié[s] : EM16, EM17, EM18 p.207)

- P47. Les modèles peuvent être incompris et les répercussions peuvent devenir importantes. Le modèle de procédé de développement en cascade en offre un exemple. D'abord, il ne s'agit pas d'un modèle selon l'auteur Winston Royce, mais d'un exemple à éviter. Dans son article *Managing the development of large software systems*, l'auteur illustre ce qu'il ne faut pas faire [287]. Le modèle en cascade (*waterfall* en anglais) n'est pas un modèle de développement, mais un contre-exemple dont l'objectif est de motiver l'itération des tâches, des activités et des processus et d'introduire d'autres modèles l'utilisant. Frederick P. Brooks Jr. dans l'ouvrage *The design of design : essays from a computer scientist* revient sur son repentir d'avoir mal interprété l'exemple du cascade :

[Traduction libre] Royce a présenté sa théorie de la cascade comme un épouvantail qu'il a ensuite réfuté, mais beaucoup ont cité et suivi cet homme de paille plutôt que son modèle plus élaboré. J'ai moi-même fait cette erreur [...].<sup>35</sup> [40]

---

34. Alfonso Pierantonio, site web, « A Body of Knowledge for Model-Based Software Engineering », <https://modeling-languages.com/body-of-knowledge-model-based-software-engineering> (consulté le 2026-04-16).

35.

*Royce introduced his waterfall as a straw man that he then argued against, but many people have cited and followed the straw man rather than his more sophisticated model. I made this mistake myself [...].*

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

Aujourd’hui, le terme *cascade* s’est imposé dans l’usage. Le PMBOK indique qu’il est devenu synonyme des approches prédictives [305].

(voir énoncé[s] lié[s] : EM19 p.207)

### 1.2.3 Des approches

P48. Comme pour les documents, différents instruments, méthodes et techniques existent afin de faciliter la création et la manipulation des modèles. Harald Kühn fournit un modèle des instruments de modélisations où, en plus, il est précisé que les langages et les domaines de connaissances occupent une place nécessaire [168]. Ainsi les approches décrites comprennent ces éléments.

(voir énoncé[s] lié[s] : AM01 p.208)

P49. La description des approches nécessite de définir plusieurs termes :

- outil de génie logiciel assisté par ordinateur (CASE tool, *Computer-Aided Software Engineering tool* en anglais) : instrument servant à réaliser ou à établir des entités informatiques ;
- méta : « L’élément méta- vient du grec, qui peut signifier “au milieu”, “avec” ou “après”. [...] il exprime la succession, le changement, la participation, ou un concept qui englobe un autre concept. » [266] ;
- langage spécifique à un domaine (DSL, *Domain-Specific Language* en anglais) : langage destiné aux utilisateurs dans un domaine spécifique dont la syntaxe ou la sémantique peut être définie formellement ;
- automate : cas particulier de machine abstraite formellement définie et conçue pour en permettre l’étude.

(voir énoncé[s] lié[s] : AM02, AM03, AM04 p.208)

### Des instruments de modélisation

Une dénomination des instruments de modélisation en informatique est les *Meta-CASE tools*. Dans cette sous-section, il y est décrit que les *Meta-CASE tools* sont

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

rarement utilisés par les professions informatiques et que plusieurs n'ont pas les fonctionnalités jugées nécessaires en contexte professionnel.

**P50.** Pour de multiples raisons, la majorité des instruments de modélisation ne rejoint pas les professionnels informatiques. D'abord, la durée de vie des instruments de modélisation est souvent courte : la médiane serait de six ans [155]. Il s'agit de l'une des conclusions de la revue menée par Nafiseh Kahani *et al.*, qui analyse soixante instruments existants entre 1990 et 2017. Il existe tout de même des instruments qui résistent au passage du temps, comme MetaEdit, dont une description à l'intérieur d'un article remonte à 1992 [346].

Ensuite, lorsqu'un sondage interroge les utilisateurs de ces instruments, une majorité provient de secteurs critiques [284]. Par ailleurs, la revue menée par Nafiseh Kahani *et al.* montre que les instruments de modélisation sont plus utilisés dans le milieu académique<sup>36</sup> ( $n = 60, 92 \%$ ) que professionnel<sup>37</sup> ( $n = 60, 37 \%$ ) [155].

(voir énoncé[s] lié[s] : AM05, AM06 p.208)

**P51.** La couverture de certaines fonctionnalités par les instruments de modélisation n'est pas adéquate pour assurer certaines caractéristiques de qualité. La revue de Nafiseh Kahani sur les instruments de modélisation ( $n = 60$ ) indique la couverture des instruments sur des fonctionnalités en pourcentage. Voici certaines de ces fonctionnalités présentées sous la forme facette - attributs (pourcentage) :

[Traduction libre]

- Sortie - Artefacts de base de données de conversion modèle-texte (23 %) ;
- Comparaison de modèles - L'outil compare des modèles homogènes (12 %) ;
- Comparaison de modèles - L'outil compare des modèles hétérogènes (3 %) ;
- Vérification - Correction syntaxique (35 %) ;
- Vérification - Correction sémantique (33 %) ;
- Validation - L'outil fournit un environnement de test (33 %) ;

---

36. Instrument généralement conçu pour illustrer des concepts théoriques ou répondre à de nouveaux enjeux de recherche susceptibles d'intéresser d'autres chercheurs.

37. Instrument conçu pour répondre aux besoins industriels existants.

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

- Validation - L'outil fournit un environnement de simulation (25 %). <sup>38</sup>  
[155]

Plusieurs des instruments ne permettent pas de comparer, de vérifier et de valider. Cela a comme conséquence de rendre l'adaptabilité des modèles difficile à réaliser dans le temps.

(voir énoncé[s] lié[s] : AM07 p.208)

### Des méthodes de modélisation

Dans les méthodes de modélisation présentées, il y a : la création de nouveaux langages, le traitement des modèles comme des artefacts et l'utilisation de pratiques réflexives.

- P52. Créer de nouveaux langages pour parvenir à mieux communiquer des modèles aux individus, pour définir des machines ou pour les utiliser comme intrants aux machines. <sup>39</sup> Dès 1972, créer des langages pour rendre les utilisateurs des machines autonomes était un objectif qui suscitait de l'intérêt dans un article intitulé *Programming Languages : History and Future*[353]. Martin Fowler affirme que les langages spécifiques à un domaine (DSL, *Domain-Specific Language* en anglais) peuvent faciliter la communication [89].

---

38.

- *Output - model-to-text (M2T) Database artifacts (23 %)* ;
- *Model Comparison - The tool compares homogeneous models (12 %)* ;
- *Model Comparison - The tool compares heterogeneous models (3 %)* ;
- *Verification - Syntactic correctness (35 %)* ;
- *Verification - Semantic correctness (33 %)* ;
- *Validation - The tool provides a testing environment (33 %)* ;
- *Validation - The tool provides a simulation environment (25 %)*.

39. Pour rappel, il y a :

- les langages non formels qui ne permettent pas de décrire un exécutable ;
- les langages syntaxiquement formels qui permettent de décrire un exécutable et dont le sens n'est pas partagé ;
- les langages sémantiquement formels qui permettent de décrire un exécutable et son sens.

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

Le *Handbook of Model-Based Systems Engineering* indique que les SysML n'étant pas très accessibles, il est préférable de passer à un DSL [210].

Par la suite, les langages de programmation deviennent nombreux. Dans le même article de 1972, l'autrice Jean E. Sammet observe déjà ce phénomène : elle indique que cent soixante-dix langages de programmation sont utilisés, et ce, uniquement aux États-Unis. Selon l'opinion de l'auteure, les explications à ce phénomène seraient rationnelles (dont l'utilité économique) ou irrationnelles (dont l'effet de mode) [353]. Pour les DSL, il est vraisemblable qu'ils aient considérablement augmenté. Une étude de cartographie systématique de 2012 inventorie 1 440 articles, dont 1 000 se situent entre 2005 et 2012 [227]. L'examen de ces articles portant sur les DSL révèle que 1 142 articles présentent des propositions de solutions qui conduisent souvent à la création de nouveaux DSL.

Toutefois, si le nombre de langages de programmation ou de conception (spécifique ou non, formelle ou non) n'a pas de limites, les classes de langages formels généralement utilisées en ont une. Noam Chomsky et plusieurs autres individus ont démontré que des classes d'automates sont nécessaires et suffisantes à la reconnaissance des classes de langages formels<sup>40</sup>[126]. Ainsi, il est possible de choisir l'automate nécessaire et d'analyser des langages de la même classe avant même de créer un nouveau langage. Ce genre d'approche plus méthodique permettrait aux créateurs de nouveaux langages d'avoir de meilleurs fondements pour mieux répondre aux objectifs.

(voir énoncé[s] lié[s] : AM08, AM09, AM10 p.209)

### P53. Considérer le modèle comme un artefact qu'il est possible de transformer selon ses besoins.

Pour le Model Driven Architecture (MDA), la représentation du modèle et le code source sont deux éléments distincts. D'ailleurs, l'existence de chacun de ces éléments est nécessaire pour permettre une vérification mutuelle. Ainsi, les modèles ne sont plus considérés comme des artefacts intermédiaires générés et utilisés lors d'un processus

---

40. La hiérarchie de Chomsky n'aborde pas la sémantique des langages, mais la syntaxe. Les machines d'un type pouvant reconnaître un langage formel peuvent reconnaître les langages formels de type égal ou inférieur.

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

[45]. Il existe des transformations entre ces deux éléments. Un ouvrage sur le MDA donne trois transformations <sup>41</sup> :

[Traduction libre]

- (1) Les transformations de refactorisation réorganisent un modèle selon des critères bien définis. Le résultat est alors une version révisée du modèle original, appelée modèle refactorisé. [...]
- (2) Les transformations de modèle à modèle convertissent les informations d'un ou plusieurs modèles vers un ou plusieurs autres modèles, généralement lorsque le flux d'informations s'effectue au-delà des frontières d'abstraction. [...]
- (3) Les transformations de modèle en code sont familières à quiconque a déjà utilisé la fonction de génération de code d'un outil de modélisation UML. Ces transformations convertissent un élément de modèle en un fragment de code. [...] <sup>42</sup> [28]

Cette manière de considérer le modèle aurait conduit à de nombreuses méthodologies de modélisation, notamment : le (MDA) et le Business Process Management (BPM). Le MDA se distingue du UML par le fait que ses modèles ont une sémantique définie formellement [391]. Cependant, même si certains artefacts peuvent concrétiser le modèle, ils n'en sont pas pour autant le modèle. Kristen Nygaard, créateur du langage Simula a cru bon de le préciser :

[Traduction libre] Il est important de noter que la modélisation signifie que les actions et interactions des objets créés par le programme reproduisent

---

41. D'autres personnes classent les instruments de modélisations selon ces transformations : *model-to-model* (M2M), *model-to-text* (M2T) et *text-to-model* (T2M) [27, 155].

42.

- (1) *Refactoring transformations reorganize a model based on some well-defined criteria. In this case the output is a revision of the original model, called the refactored model. [...]*
- (2) *Model-to-model transformations convert information from one model or models to another model or set of models, typically where the flow of information is across abstraction boundaries. [...]*
- (3) *Model-to-code transformations are familiar to anyone who has used the code generation capability of a UML modeling tool. These transformations convert a model element into a code fragment. [...]*

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

les actions et interactions des objets du monde réel qu'ils sont conçus pour simuler. Ce n'est pas le code Simula qui modélise le système réel, mais les objets créés par ce code.<sup>43</sup> [30]

Par conséquent, l'interprétation d'un artefact par une machine abstraite est importante. Il faut maintenir la sémantique entre l'artefact et son interprétation. Les transformations doivent aussi maintenir cette sémantique. Ainsi, le passage d'une spécification des exigences informelles à une décrite formellement ne peut pas se faire par une transformation totalement automatisée [170].

(voir énoncé[s] lié[s] : AM11, AM12, AM13 p.209)

- P54. Utiliser une pratique réflexive pour assurer la cohérence et la couverture lors du développement d'un modèle. D'abord, il est possible d'interpréter des artefacts pour en obtenir un modèle. Un article décrit une pratique réflexive qui utilise des artefacts de code source [224]. À partir de ces artefacts, le modèle est construit et validé de manière successive comprenant différentes interventions. Une de ces interventions est l'utilisation d'un instrument pour vérifier la cohérence du modèle. Dans l'article, l'exécution pouvait prendre plus de trente minutes<sup>44</sup>. Parmi les avantages à cette méthode, il y a celui de permettre l'obtention de la couverture souhaitée par la sélection des artefacts (et plus particulièrement des modèles), voire de parties de ceux-ci. Aussi, ces bornes facilitent les vérifications et validations pour les intervenants et permettent de diminuer le délai d'exécution de l'instrument.

Ensuite, un ouvrage abordant les méthodes formelles décrit le procédé Balzer [21]. Ce procédé, proposé au début des années 1980, a comme objectif de faciliter l'utilisation des méthodes formelles, peu importe le procédé de développement utilisé, puisqu'il se situe en amont. Dans le procédé Balzer, le document de spécification des exigences formelles (la description du modèle) est construit suivant un cycle dont fait partie le

---

43.

*Notice that modelling means that the actions and interactions of the objects created by the program model the actions and interactions of the real-world objects that they are designed to simulate. It is not the Simula code that models the real-world system, but the objects created by that code.*

44. L'exécution était de 40 minutes en 2001 sur un Pentium II et le modèle était celui du produit Excel.



## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

simulacre et le document d'analyse informelle des exigences. Ce cycle, en comprenant les différentes vérifications et validations, améliore la cohérence et la couverture au modèle développé.

(voir énoncé[s] lié[s] : AM14 p.[209](#))

### Des techniques de modélisation

Dans cette sous-section, il y est présenté d'abord la technique offrant un langage formel avec une représentation textuelle et graphique, puis, l'ensemble des techniques appelées méthodes formelles.

- P55. L'une des techniques servant à la modélisation consiste à développer un langage formel offrant à la fois une représentation textuelle et une représentation graphique. Le modèle et notation des procédés métier (BPMN, *Business Process Model and Notation* en anglais), particulièrement sa version 2, en est un exemple. Selon les auteurs, le BPMN est décrit formellement en XSD et dans la mise en place complète de méta-objets (CMOF, *Complete Meta-Object Facility* en anglais) [[235](#)]. Le XSD est un langage pour décrire une structure et des contraintes sur les documents XML, tandis que le CMOF comprend le Meta-Object Facility (MOF), un langage pour décrire des métamodèles [[46](#)].

De prime abord, l'objectif du BPMN est de rendre un langage de modélisation accessible aux utilisateurs métiers, analystes et développeurs [[55](#)]. Le BPMN est inspiré du diagramme d'activité qui provient de l'UML.

Néanmoins, si formalisme il y a, les instruments ne permettent pas de profiter de ce formalisme. Un article publié en 2018 fait l'analyse de 47 instruments se déclarant conformes au standard BPMN [[99](#)]. Parmi eux, seuls 3 instruments parviennent à exécuter le format spécifié par le standard. Un autre article, publié en 2020, présente les résultats d'une analyse qui consistait à exécuter un analyseur syntaxique sur 8 904 artefacts BPMN disponibles sur GitHub. Cette expérience a parmi notamment

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

de trouver des erreurs dans 82 %<sup>45</sup> d'entre eux [114].

(voir énoncé[s] lié[s] : AM15, AM16 p.209)

P56. Les méthodes formelles comprennent des techniques éprouvées utilisant logique et formalisme, mais ne s'imposent pas largement. D'abord, les méthodes formelles comprennent des langages formels et des systèmes logiques. Jean-Raymond Abrial décrit ses préférences en langage et en logique :

[Traduction libre] [...] le langage de la logique classique et de la théorie des ensembles. Ces conventions nous permettront de communiquer facilement nos modèles à autrui [...] nous permettront de raisonner sous forme de démonstrations mathématiques.<sup>46</sup> [2]

Dines Bjørner soulève toute l'importance des systèmes logiques pour les méthodes formelles :

[Traduction libre] Un langage, même doté d'une sémantique, est appauvri s'il est dépourvu de logique : il ne fournirait aucun moyen d'obtenir ces conséquences de manière méthodique, reproductible et consensuelle.<sup>47</sup> [29]

De plus, Markus Roggenbach dans son ouvrage, explique en détail six projets à succès où les méthodes formelles ont joué un rôle vital [340]. Un sondage réalisé par Maurice H. Ter Beek *et al.*, auquel ont participé cent trente experts en méthodes formelles, aborde différentes questions liées à ce sujet. Lorsqu'il est demandé en quoi les méthodes formelles peuvent contribuer, il en ressort que :

---

45. Plusieurs des erreurs sont en fait des définitions non conformes des flux de séquence dans le format de sérialisation.

46.

[...] *the language of classical logic and set theory. These conventions will allow us to easily communicate our models to others [...] will allow us to do some reasoning in the form of mathematical proofs.*

47.

*A language, even one with a semantics, is impoverished if there is no logic : it would provide no means for obtaining those consequences in a methodical, reproducible and agreed fashion.*

## 1.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

[Traduction libre] Les experts ont confirmé, de manière quasi unanime, que les méthodes formelles garantissent la qualité, la sûreté, la sécurité, une certification plus facile et une maintenance simplifiée.<sup>48</sup> [378]

Cependant, les méthodes formelles ne s'imposent pas largement. Le même sondage de Maurice H. Ter Beek *et al.* demande les causes de cette non-utilisation, voici les plus couramment citées :

[Traduction libre]

[—] Les ingénieurs manquent de formation adéquate aux méthodes formelles ;

[—] Les outils académiques présentent des limitations et ne sont pas maintenus de manière professionnelle ;

[—] Les méthodes formelles ne sont pas correctement intégrées au cycle de vie de la conception industrielle ;

[—] L'apprentissage des méthodes formelles est complexe ;

[—] Les développeurs sont réticents à modifier leurs méthodes de travail.

<sup>49</sup> [378]

D'ailleurs, plusieurs articles sont écrits dans l'objectif d'informer et, ultimement, d'encourager leur utilisation [340]. Un autre sondage mené en 2020 révèle que 40 des 216 participants n'ont pas utilisé de méthodes formelles ni en milieu académique ni industriel, et ce, même si tous avaient été formés ou avaient travaillé dans des secteurs critiques [103].

(voir énoncé[s] lié[s] : AM17, AM18, AM19, AM20 p.209)

---

48.

*Quasi unanimously, the experts confirmed that formal methods deliver quality, safety, security, easier certification, and easier maintenance.*

49.

[—] *Engineers lack proper training in formal methods ;*  
[—] *Academic tools have limitations and are not professionally maintained ;*  
[—] *Formal methods are not properly integrated in the industrial design life cycle ;*  
[—] *Formal methods have a steep learning curve ;*  
[—] *Developers are reluctant to change their way of working.*

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

## 1.3 Des connaissances en informatique appliquée

À l’instar des documents et des modèles, les connaissances en informatique appliquée ont des objectifs, des écueils et des approches. L’un des objectifs aux connaissances en informatique appliquée est de contribuer à la réalisation et à l’établissement des entités informatiques. Ces connaissances demeurent donc attachées au procédé de développement. Ainsi, les écueils relatifs aux connaissances<sup>50</sup> se répercutent sur le procédé de développement et, par conséquent, sur les documents et les modèles. Pour cette raison, il ne sera pas possible de les présenter tous. Aussi, leurs présentations se limitent aux éléments du procédé de développement : les participants informatiques<sup>51</sup>, les processus, les artefacts et les instruments. Plusieurs approches existent pour surmonter ces écueils, mais elles ne les résolvent pas forcément tous ni entièrement. À cela s’ajoutent d’autres approches qu’il est possible de qualifier de sociétales, comme la réglementation et la formation.

P57. Il est important de bien définir certains concepts, notamment la connaissance, le savoir en informatique appliquée et la gestion des connaissances :

- connaissance : une croyance justifiée qui à force de légitimités devient un fait ;
- savoir en informatique appliquée : ensemble des connaissances s’appliquant à l’information, aux traitements de l’information et aux entités informatiques utilisés en pratique ;
- gestion de la connaissance : organiser, contrôler et rendre accessibles les représentations des connaissances au bénéfice d’une organisation.

(voir énoncé[s] lié[s] : S01, S02, S03 p.210)

---

50. Rencontrés lors de l’acquisition ou de l’application. Découlant de leurs incohérences, leurs incomplétudes et, etc.

51. Un participant informatique est une personne participant à des processus utilisant l’informatique.

## 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

### 1.3.1 Des objectifs

L'objectif fondamental de l'informatique est de solutionner des problèmes. Cela s'effectue lors d'un procédé de développement où différentes activités et autres procédés vont utiliser et produire des artefacts. La gestion de la connaissance indique que les artefacts comprenant les documents et les modèles sont d'importante représentation des connaissances. Toutefois, l'informatique appliquée nécessite des connaissances provenant de l'informatique, provenant d'autres domaines et qu'elles aient une forme explicite.

- P58. Le procédé de développement veut créer une solution en mesure de traiter un problème. À l'intérieur d'un article intitulé *The world and the Machine*, Michael Jackson met en relation les quatre facettes que sont la modélisation, l'interfaçage, l'ingénierie et le problème. Il ajoute :

[Traduction libre] Le développement logiciel relève de l'ingénierie, car il vise à créer des dispositifs physiques utiles pour répondre à des besoins pratiques. Un logiciel est la description d'une machine. Nous construisons cette machine en la décrivant et en présentant cette description à un ordinateur généraliste [...].<sup>52</sup> [148]

Pour Daniel Jackson, le développement logiciel est la conception d'abstraction [145]. Il nomme cette abstraction **modèle**, et cela avec regret, puisqu'elle ne représente pas forcément un sujet. Le point commun à ces visions est que le développement logiciel conduit à des machines abstraites, souvent à des modèles, et toujours dans l'objectif de traiter un problème.

(voir énoncé[s] lié[s] : OS01 p.210)

- P59. La gestion de la connaissance spécifie que les participants informatiques, les procédés et les artefacts, dont certains sont des instruments, comprennent d'importantes connaissances

---

52.

*Software development is engineering because it is concerned to make useful physical devices to serve practical purposes in the world. Software is a description of a machine. We build the machine by describing it and presenting our description to a general-purpose computer [...].*

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

pour le procédé de développement. D'abord, un ouvrage sur la gestion de la connaissance en logiciel donne l'exemple d'une entreprise où la conservation des connaissances passe par les procédés, certains artefacts, les configurations des instruments et les capacités des équipes [394].

Ensuite, un ouvrage intitulé *Managing Software Engineering Knowledge* explique que la connaissance, tant en général qu'en développement logiciel, est reliée à de nombreux concepts [16] : la gestion documentaire, les réseaux d'experts, la gestion de la configuration, les instruments (*CASE Tools*), etc.

Enfin, un article soulève quelques faits relatifs à la transmission des connaissances [72] : de nombreux artefacts en constituent des représentations. Toutefois, une partie importante des connaissances demeure dans la tête des participants informatiques, et seuls certains d'entre eux valident des artefacts comme étant des connaissances.

(voir énoncé[s] lié[s] : OS02 p.210)

P60. Le procédé de développement nécessite des connaissances explicites et provenant d'autres domaines que l'informatique. Pour les connaissances explicites, H. Dieter Rombach explique que toutes les connaissances explicites en informatique appliquée sont contenues dans ses propres artefacts<sup>53</sup> (la documentation, le code source, les modèles, etc.) [16]. Les problèmes de documentation occupent une place importante lorsque vient le temps de partager les connaissances. Une revue de littérature ( $n = 61$ ) sur le partage de connaissances lors de développement décentralisé en témoigne [410].

Pour les connaissances provenant d'autres domaines, le terme **appliquée** en informatique appliquée signifie que le domaine de l'informatique est appliqué à un autre domaine. Ian K. Bray décrit le métier d'informaticien et sa relation avec la connaissance dans l'ouvrage *An introduction to requirements engineering* [37]. L'auteur affirme qu'il faut inciter les individus à expliciter leurs connaissances et trouver la connaissance contenue à l'intérieur des artefacts.

(voir énoncé[s] lié[s] : OS03, OS04 p.210)

---

53. Toutes connaissances importantes devraient être explicitées, ce qui n'est pas toujours le cas.

## 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

### 1.3.2 Des écueils

Les écueils liés aux connaissances en informatique appliquée concernent les participants informatiques, les processus, les artefacts et les instruments. Du côté des participants informatiques et, plus précisément des professionnels informatiques, plusieurs ne possèdent souvent pas les qualifications nécessaires. De plus, plusieurs quittent la profession, ce qui peut nuire au transfert des connaissances. Quant aux processus, ils sont rarement étudiés pour construire ou vérifier des connaissances. Les artefacts, pour leur part, sont fréquemment délaissés, ce qui nuit à la conservation des connaissances. Enfin, plusieurs instruments échappent au contrôle des participants informatiques, alors qu'ils sont importants, puisque les instruments contrôlent les processus et les artefacts.

#### Les participants informatiques

Cette sous-section débute par une catégorisation des participants informatiques, puis utilise la catégorie la plus représentée, celle des professionnels informatiques. Dans les écueils, il apparaît que le transfert de connaissances peut s'avérer problématique et qu'une majorité des professionnels informatiques n'ont pas d'assise solide permettant entre autres de faciliter ce transfert.

P61. Il est important de distinguer l'informaticien, l'informaticien appliqué, le praticien informatique, le professionnel informatique et le participant informatique. Les définitions sont les suivantes :

- informaticien : spécialiste du traitement automatique et rationnel de l'information ;
- informaticien appliqué : informaticien spécialiste dans l'information provenant de domaines autres que l'informatique ;
- praticien informatique : personne qui réalise ou établit des entités informatiques ;
- professionnel informatique : personne exerçant une activité rémunérée liée à l'informatique ;
- participant informatique : personne participant à des processus utilisant l'informatique.

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

(voir énoncé[s] lié[s] : ES01, ES02, ES03, ES04, ES05, ES06 p.211)

- P62. Le groupe des professionnels informatiques est suffisant pour identifier et décrire plusieurs écueils. Suivant la définition précédente, il est possible d’y regrouper 15 métiers provenant de la classification nationale des professions (CNP). Il y a notamment : le spécialiste informatique, le technicien de réseau informatique et Web ainsi que le développeur et programmeur Web. Ce sont tous des métiers rémunérés pour réaliser ou établir des entités informatiques. Dans un procédé de développement, tous ces métiers n’ont pas la même importance, cependant, ils ont tous une influence. Selon les données de Statistique Canada sur les professions de 2022 [372], il y a environ 180 000 professionnels informatiques au Québec. Pour donner une comparaison, il y a 10 fois moins de professeurs et chargés de cours universitaires à temps plein pour tout domaine confondu au Québec.

(voir énoncé[s] lié[s] : ES07, ES08 p.211)

- P63. Le transfert de connaissances entre les professionnels informatiques présente un risque. Pour les professionnels informatiques, les tranches d’âges plus élevés sont moins représentées que dans d’autres métiers (tableau 1.5). Sans connaître les causes, cela représente un risque au niveau du passage des connaissances. D’ailleurs, la rétention des professionnels informatiques est plus faible que dans d’autres professions. À titre d’illustration, un sondage<sup>54</sup> réalisé en 2001 aux États-Unis indique que seulement 19 % des personnes qui graduent en informatique demeurent dans le secteur après vingt ans, comparativement à 52 % en génie civil [13]. Dans les raisons qui incitent à quitter le métier, plusieurs professionnels informatiques pointent la lourde charge de travail et la menace d’obsolescence. Ces causes reviennent dans différents sondages [63, 215, 234].

(voir énoncé[s] lié[s] : ES09, ES10, ES11 p.212)

---

54. Les données proviennent du The National Survey of College Graduates. Sur le site officiel, il est indiqué qu’il y a actuellement 71,7 millions d’individus ce n’est probablement pas l’état du sondage de 2001 [229].



### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**TABLEAU 1.5** – Pourcentage des tranches d'âge parmi les postes à temps plein occupés en 2021 au Québec

| Métier                                                  | 15 à 24 ans/Total | 25 à 64 ans/Total | 65 ans et plus/Total | 65 ans et plus/ 15 à 24 ans |
|---------------------------------------------------------|-------------------|-------------------|----------------------|-----------------------------|
| Professionnel informatique                              | 0,06              | 0,91              | 0,01                 | 0,26                        |
| Ingénieur sauf informaticien et électricien             | 0,02              | 0,93              | 0,03                 | 1,48                        |
| Assistant d'enseignement et de recherche au niveau      | 0,23              | 0,74              | 0,01                 | 0,08                        |
| Enseignant niveau collégial et instructeur en formation | 0,02              | 0,89              | 0,07                 | 3,50                        |
| Professeur et chargé de cours universitaire             | 0,00              | 0,85              | 0,14                 | 21,50                       |
| Avocat                                                  | 0,03              | 0,87              | 0,09                 | 2,74                        |
| Médecin et vétérinaire                                  | 0,01              | 0,86              | 0,12                 | 10,77                       |
| Physicien et astronome                                  | 0,01              | 0,91              | 0,07                 | 5,50                        |

<sup>a</sup> Données sur les professions provenant de Statistique Canada [372].

<sup>b</sup> Pourcentage des tranches d'âge (tronqué à l'inférieur) parmi les postes à temps plein occupés en 2021 au Québec, par métier.

P64. Les connaissances informatiques d'une majorité de professionnels informatiques n'ont pas d'assise solide. En premier lieu, le nombre de professionnels informatiques ayant un diplôme de premier cycle est faible comparativement à celui observé dans plusieurs autres métiers (tableau 1.6) [372]. De plus, la discipline de ces diplômes n'est pas précisée. Une étude menée aux États-Unis en 2015 indique qu'il s'agissait d'un peu moins de la moitié [64]. Plus précisément, les analystes de systèmes informatiques, les informaticiens et les développeurs de logiciels n'étaient majoritairement pas titulaires d'un diplôme en informatique. La combinaison de ces deux résultats statistiques suggère qu'environ le quart des professionnels informatiques pourraient être qualifiés. Ensuite, un professionnel informatique n'a pas besoin d'être qualifié pour exercer au Québec. Il n'y a pas de référence à des qualifications dans la définition d'informaticien [255], contrairement à celles d'ingénieur [257], de physicien [269] ou d'avocat [241]. Il est important de souligner que, parmi les métiers qui ont le plus de diplômés, figurent ceux attribués à un ordre professionnel. Cependant, cela peut aussi arriver lorsque

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**TABEAU 1.6** – Nombre de postes occupés en 2021 au Québec selon le métier et le diplôme universitaire de premier cycle

| Métier                                        | Possède un diplôme universitaire de 1er | Total  | Proportion de titulaires d'un diplôme universitaire de 1er |
|-----------------------------------------------|-----------------------------------------|--------|------------------------------------------------------------|
| Professionnel informatique                    | 86025                                   | 179675 | 0,48                                                       |
| Ingénieur sauf informaticien et électricien   | 36870                                   | 40860  | 0,90                                                       |
| Assistant d'enseignement et de recherche au   | 21955                                   | 27420  | 0,80                                                       |
| Enseignant niveau collégial et instructeur en | 26180                                   | 38805  | 0,67                                                       |
| Professeur et chargé de cours universitaire   | 18305                                   | 18345  | 1,00                                                       |
| Avocat                                        | 24470                                   | 24525  | 1,00                                                       |
| Médecin et vétérinaire                        | 28685                                   | 28685  | 1,00                                                       |
| Physicien et astronome                        | 635                                     | 695    | 0,88                                                       |

<sup>a</sup> Données sur les professions provenant de Statistique Canada [372].

le diplôme est une exigence des employeurs, comme c'est le cas pour les physiciens (tableau 1.6).

Enfin, plusieurs professionnels informatiques estiment, dans un sondage ( $n = 33$ ), que leur métier est trop exigeant pour leur permettre de maintenir ou développer de nouvelles compétences [376].

(voir énoncé[s] lié[s] : ES13, ES14, ES15, ES16 p.212)

### Les processus

Pour comprendre un processus, il faut en faire un procédé, mais d'abord, plusieurs écueils doivent être surmontés. Cette sous-section en présente deux : il faut une base de connaissances fiable et il faut obtenir les informations nécessaires à la construction des

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

modèles. Éventuellement, un premier procédé développé (modèle) pourra en devenir un second de type descriptif, prescriptif, vérificatif, etc.

#### P65. Pour les processus en informatique appliquée, il manque une base de connaissances fiable.

Cela devrait débiter avec une taxinomie. Une taxinomie est l'activité et le résultat de l'étude d'un sujet pour en obtenir des règles de classification.

D'abord, les taxinomies sont importantes pour comprendre rigoureusement les théories et les modèles. Paul Ralph écrit dans un article publié 2019 que :

[Traduction libre] [...] les théories et taxinomies des processus sont nécessaires en génie logiciel, car elles fournissent les concepts et le langage technique nécessaires à une communication et à une formation précises.<sup>55</sup>  
[315]

Un ouvrage distingue le modèle formel du modèle informel, entre autres, avec le formalisme utilisé sur la taxinomie [394].

Ensuite, dans le même article, Paul Ralph met en garde contre le manque de rigueur des taxinomies développées. Par ailleurs, une revue de littérature de 2017 portant sur 270 articles comportant 271 taxinomies ( $n = 271$ ) en informatique appliquée confirme plusieurs manquements à l'intérieur de ces articles [388] :

- pour 22 (8 %) taxinomies le sujet est soit les procédés, soit les modèles, soit les méthodes (une taxinomie a été évaluée et deux taxinomies ont été validées) ;
- pour 262 (96 %) taxinomies la procédure de classification est qualitative ;
- pour 227 (83 %) taxinomies la procédure de classification n'est pas décrite explicitement.

Néanmoins, depuis la fin des années 1990, le CMMI et le SWEBOK contribuent tout de même à améliorer la situation en offrant respectivement une procédé d'amélioration des procédés et une base de connaissances en informatique appliquée.

(voir énoncé[s] lié[s] : ES17, ES18, ES19 p.213)

---

55.

[...] *process theories and taxonomies are needed in software engineering because they provide concepts and technical language needed for precise communication and education.*

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

P66. Les informations nécessaires à la construction de modèles basés sur les procédés de développement sont manquantes, ce qui empêche de développer des connaissances. D’abord, le contexte n’est pas propice à la cueillette d’informations pour les raisons suivantes :

- le courant dominant, c’est-à-dire les méthodes agiles, discrédite les procédés de développement <sup>56</sup>[220] ;
- il n’existe pas de définitions explicites des procédés de développement et des critères de succès d’un projet, même dans les rapports qui sont populaires [371].

Ensuite, les études rigoureuses sur les procédés en informatique sont peu nombreuses :

- une revue de littérature répertorie 43 études ayant eu lieu entre 1987 et 2008 portant sur les modèles de développement et incluant des données : parmi elles, seulement 9 études comportent des données quantitatives [22] ;
- un article publié en 2022 décrit les précédentes études sur les méthodes agiles, donc les procédés courants, d’anecdotiques [59] ;
- l’ouvrage *Experimentation in Software Engineering* cite plusieurs études soulignant des problèmes avec les expérimentations en général :

[Traduction libre] Une revue systématique de la littérature sur les expériences en génie logiciel (1993-2002) [100] a permis d’identifier 40 théories dans 23 articles, sur un total de 113 articles. Seules deux de ces théories étaient présentes dans plus d’un article. <sup>57</sup> [404]

(voir énoncé[s] lié[s] : ES20, ES21, ES22 p.213)

#### Les artefacts

La disparition de certains artefacts a mené à la création d’une nouvelle discipline : la rétro-ingénierie. Cependant, elle ne résout pas tout et plusieurs écueils sont en fait des limitations à cette discipline.

---

56. Kent Beck *et al.*, site web, « Manifesto for Agile Software Development », <https://agilemanifesto.org/iso/fr/manifesto.html> (consulté le 2025-05-12).

57.

*systematic literature review of software engineering experiments, 1993–2002 [100]. They found 40 theories in 23 articles, out of the 113 articles in the review. Only two of the theories were used in more than one article !*

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**P67. La perte de connaissances en informatique appliquée a conduit à la naissance d’une nouvelle discipline : la rétro-ingénierie.** La rétro-ingénierie consiste à étudier un groupe d’artefacts pour en déterminer le fonctionnement et la méthode de fabrication. En effet, les artefacts se dégradent et finissent par disparaître. Ehsan Zabardast identifie 3 causes à la dégradation [409] : délibérée (dette technique), involontaire (pertes ou destructions d’artefacts) ou par entropie (adaptation successive). Lors d’un sondage mené auprès de 219 professionnels informatiques, 26 personnes ont indiqué qu’elles supprimeraient les exigences lorsqu’elles devenaient obsolètes [403]. Certains choisissent de ne conserver qu’un seul type d’artefact ; c’est le cas du projet Software Heritage, qui ne conserve que le code source, car il s’agit d’artefacts plus petits et plus faciles à traiter [74].

(voir énoncé[s] lié[s] : ES23, ES24, ES25 p.213)

**P68. La rétro-ingénierie présente de nombreuses limitations.** D’abord, un article de 1991 rappelle que certains artefacts ne contiennent pas toutes les informations [44]. Le code source explique précisément à la machine la procédure à suivre, mais n’offre ni une vision globale ni des explications sur les raisons. À cela s’ajoutent les biais lors de l’implémentation, les informations contradictoires pouvant provenir de la documentation ou le contexte changeant.

Ensuite, un article publié en 2024 résume les difficultés et défis à construire des modèles en rétro-ingénierie [362]. Parmi celles-ci, il y a : l’exactitude, l’hétérogénéité, l’adaptabilité, la gestion des comportements dynamiques, la compréhension de la sémantique du code source, l’absence de techniques d’apprentissage automatique et la traduction de programmes.

Enfin, les instruments de rétro-ingénierie nécessitent les connaissances des experts pour obtenir des résultats satisfaisants.

En 2011, un article qualifie les résultats produits automatiquement comme étant incomplets ou imprécis [48].

(voir énoncé[s] lié[s] : ES26 p.213)

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

#### Les instruments

De tous les instruments, le groupe des *CASE tools* est suffisamment représentatif pour démontrer les écueils associés aux instruments. C'est qu'ils sont moins bien définis et subissent les influences de l'économie et des organisations qui les contrôlent.

**P69. Le flou autour des définitions des *CASE tools* et de leurs catégorisations va en s'accroissant.**

Tout d'abord, les définitions des *CASE tools* sont précaires, et le terme lui-même est de moins en moins utilisé. Même le terme n'est plus autant utilisé. Roger S. Pressman, dans la septième édition de son ouvrage *Software engineering : a practitioner's approach* publiée en 2010, n'y fait d'ailleurs plus mention [308]. Pourtant, dans la cinquième édition publiée en 2000, un chapitre entier (chapitre 31) y est consacré et l'auteur y présente 24 définitions de différentes catégories de *CASE tools* [307].

Ensuite, le vocabulaire employé pour définir des *CASE tools* peut prêter à confusion. Ils peuvent être désignés par les termes suivants : [Traduction libre] « Environnements axés sur les processus, environnement CASE intégré (I-CASE), environnements d'ingénierie logicielle axés sur les processus, gestion du cycle de vie des applications (ALM) »<sup>58</sup>. Tous ces instruments prennent en charge des procédés de développement. Toutefois, dans le cas du ALM, Alfonso Fuggetta indique que cet aspect de ces instruments n'est pas explicite, cela peut ajouter à la confusion [92].

(voir énoncé[s] lié[s] : ES27, ES28 p.214)

**P70. Les instruments subissent les aléas de l'économie.** Tout d'abord, les *CASE tools* sont influencés par les tendances des méthodes de développement. Dans un ouvrage consacré aux *CASE tools*, l'auteur affirme que ce sont les exigences du marché qui motivent la création d'environnement CASE intégré (I-CASE tools, *Integrated CASE Environment* en anglais) [365]. Ce serait aux professionnels informatiques et aux spécialistes du marketing d'identifier les fonctionnalités les plus utiles et les plus demandées [306]. La popularité des méthodologies agiles a un impact incontestable sur ces instruments [50, 106].

---

<sup>58</sup>. « *Process-centered environments, integrated CASE environment (I-CASE), Process-centered software engineering environments, Application lifecycle management (ALM)* »

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

Ensuite, les participants informatiques se font leurrer quant aux fonctionnalités offertes par certains *CASE tools*. Depuis au moins le début d'années 1980, les fournisseurs d'instruments gonflent les fonctionnalités offertes [365]. À la fin des années 2000, les ALM promettent de gérer l'ensemble du processus d'une entité informatique, mais ce ne fut pas le cas [54, 92]. À la fin des années 2010, cette prétention demeure [384]. Pourtant, à la même période, un cas d'étude démontre qu'une entreprise a dû se reprendre à trois reprises pour réussir à intégrer leurs procédés à l'intérieur d'ALM [205]. Jeffrey D. Ullman décrit cette volonté mercantile derrière l'utilisation de certains termes :

[Traduction libre] [...] le terme « connaissance » se vend bien. Il semble qu'attribuer la notion de « connaissance » à votre logiciel, ou affirmer qu'il utilise « l'intelligence artificielle », le rend plus attractif, même si ses performances et ses fonctionnalités ne sont pas nécessairement meilleures que celles d'un produit similaire [...].<sup>59</sup> [386]

(voir énoncé[s] lié[s] : ES29, ES30 p.214)

P71. Ce sont les organisations qui contrôlent une grande partie des *CASE tools*. Dans l'ouvrage *Introduction to Digital Economics : Foundations, Business Models and Case Studies*, dix stratégies permettant aux organisations de garder captifs les utilisateurs sont énumérées [414]. Parmi celles-ci figurent les contrats difficiles à résilier, la perte d'information potentielle ainsi que la maintenance et les pièces de rechange. Une analyse propose même d'instaurer une réglementation obligeant les organisations à assurer la portabilité et l'interopérabilité des données emprisonnées dans les logiciels [349]. L'instrument Rational Synergy d'IBM est un exemple où une entreprise décide du sort d'un *CASE tool* :

---

59.

[...] "knowledge" sells well. It appears that attributing "knowledge" to your software product, or saying that it uses "artificial intelligence" make the product more attractive, even though the performance and functionality may be no better than that of a similar product [...].

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

- sur la page web officielle de l'entreprise, le retrait des intégrations avec sept autres produits est indiqué<sup>60</sup> ;
- sur une autre page, il est indiqué que les clients doivent impérativement télécharger la documentation, puisqu'elle sera retirée du site web<sup>61</sup>.

Ainsi, le fournisseur pousse le client à choisir entre migrer vers un nouvel instrument ou continuer avec l'ancien. Selon le type d'instrument, cela peut conduire le client à changer son procédé de développement<sup>62</sup>.

(voir énoncé[s] lié[s] : ES31 p.214)

#### 1.3.3 Des approches

Dans les approches, certaines relèvent du cadre sociétal. Cela comprend les normes, les standards, les conventions et les actes réservés. Toutefois, la formation a une place importante pour l'informatique appliquée. Elle offre les connaissances pour maîtriser les méthodes et les techniques, et, ainsi, mieux comprendre les documents et les modèles. Cependant, toutes ces approches ne sont pas infaillibles.

#### Améliorer le cadre sociétal

Le contexte dans lequel l'informatique appliquée se pratique est influencé ou même imposé par la société. Cela comprend la création et l'application de conventions, de standards, de normes et de réglementations. Cette sous-section en présente les écueils.

P72. Des termes comme **norme** et **standard** sont définis pour éviter des ambiguïtés :

---

60. IBM, site web, « Deprecated integrations », <https://www.ibm.com/docs/en/engineering-lifecycle-management-suite/workflow-management/7.0.3?topic=integrating-deprecated-integrations> (consulté le 2025-04-12).

61. IBM, site web, « How can I access the Rational Synergy and Rational Change online documentation after the products' end of life? », <https://www.ibm.com/support/pages/how-can-i-access-rational-synergy-and-rational-change-online-documentation-after-products-end-life> (consulté le 2025-04-12).

62. Rational Synergy est un logiciel de gestion de configuration (SCM : software configuration management) qui est une catégorie d'instruments importants et très intégré au procédé de développement.



### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

- norme : « Document, établi par consensus et approuvé par un organisme reconnu, qui fournit, pour des usages communs et répétés, des règles [...]. » [128] ;
- organisme de normalisation : « [...] l'une des principales fonctions, en vertu de ses statuts, est la préparation, l'approbation ou l'adoption de normes [...]. » [128] ;
- règlement : « Document qui contient des règles à caractère obligatoire et qui a été adopté par une autorité. » [128] ;
- standard : document qui contient des règles de pratiques communes ne provenant pas d'un organisme de normalisation ;
- convention : « Accord de volonté entre deux ou plusieurs personnes physiques ou morales, par lequel elles s'engagent à faire ou à ne pas faire quelque chose. » [246].

(voir énoncé[s] lié[s] : AS01, AS02, AS03, AS04, AS05 p.214)

#### P73. La création et l'application de standards ou de normes ne sont pas des méthodes infaillibles.

D'abord, il arrive que la réglementation ne force pas l'application d'un standard ou d'une règle. C'est ce qu'affirme Ralf Kneuper dans son ouvrage *Software Processes and Life Cycle Models : An Introduction to Modelling, Using and Managing Agile, Plan-Driven and Hybrid Processes* [162]. Il revient donc à la gouvernance et à la direction de les appliquer, puisque la réglementation ne le fait pas toujours. Deux exemples l'illustrent. Une analyse réalisée en 2021 pour vérifier la réglementation sur l'application de la norme ISO 27001<sup>63</sup> [311] recense 47 circonstances où son application est volontaire et 19 où elle est obligatoire. Par ailleurs, la réglementation peut évoluer dans le temps. En 2016, le 21st Century Cures Act retire à la United States Food and Drug Administration (US FDA) le contrôle de l'évaluation de certains logiciels, notamment les dossiers médicaux électroniques et les logiciels d'aide à la décision [343].

Par ailleurs, les informations ou les contrôles découlant des standards ou des normes peuvent s'atténuer avec le temps. En voici deux exemples. Lorsque le Department Of

---

63. La norme ISO 27001 concerne la sécurité de l'information et les systèmes de gestion de la sécurité de l'information [142]

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

Defense (DOD) a transféré la responsabilité de standards à l’Institute of Electrical and Electronics Engineers (IEEE) au milieu des années 1990, les gabarits de documents ont été abandonnés [61]. De plus, depuis 2017, il est possible pour une organisation d’être certifiée conforme à une culture de la qualité, et suffisante pour éviter les inspections ultérieures de l’US FDA [67]. En pratique, cela pourrait conduire à une forme de déréglementation.

Pour finir, même dans les secteurs critiques où les standards et normes existent, plusieurs problèmes surviennent. Selon l’US FDA, entre 2005 et 2011, 5 792 rappels sur des appareils médicaux ont été recensés, et dans 19,4 % des cas, la cause était le logiciel [105].

(voir énoncé[s] lié[s] : AS06, AS07, AS08 p.215)

P74. L’utilisation de standards sur des langages de modélisation a déjà été plus élevée. La première version du langage UML paraît en 1997 [341]. Après 2004, son utilisation commence à décliner, menant à son retrait de Microsoft Visual Studio en 2016. À partir de 2017, un regain d’intérêt se manifeste avec l’apparition des fichiers UML textuels, qui permettent de la lisibilité et du versionnage, les rendant ainsi plus adaptables<sup>64</sup>. Cette tendance se confirme par l’analyse de dépôts GitHub. Il est importe également de rappeler que le langage UML présente des limitations ayant conduit au développement des langages SysML et MOF.

Suit le MOF, publié<sup>65</sup> en 2014 [162]. Le MOF intègre l’UML et est utilisé par le BPMN ainsi que par le cadre de modélisation Eclipse (EMF, *Eclipse Modeling Framework* en anglais). Bien que l’EMF soit qualifié, en 2023, de populaire [210], une autre analyse de dépôts GitHub ne permet pas d’observer un accroissement de l’utilisation de l’EMF, du moins dans l’univers du code ouvert [20]. Des explications possibles sont avancées par ces articles :

- un sondage de 2012 mettant en évidence les difficultés des utilisateurs avec le MOF et le langage basé sur la logique Object Constraint Language (OCL) utilisé [46] ;

---

64. PlantUML et Mermaid en sont des exemples.

65. L’organisation de standardisation derrière l’UML, le MOF et le BPMN est le Object Management Group (OMG).

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

- une analyse de 2016 démontrant que la majorité des problèmes dans l'EMF concerne l'instrumentation et sa documentation [156].

(voir énoncé[s] lié[s] : AS09, AS10, AS11 p.215)

**P75. Les conventions de code sont un exemple où une convention peut aider à la compréhension de documents, mais l'utilisation des langages formels les rendent en partie inutiles.** Dans son ouvrage *Open Verification Methodology Cookbook* [102], Mark Glasser consacre un chapitre à une convention de code source. Il le conclut en tentant de persuader le lecteur de l'importance de suivre une convention.

Néanmoins, les langages de programmation possèdent déjà les caractéristiques (le formalisme) pour ne pas avoir besoin d'une convention. C. A. R. Hoare, dans un article de 1983, affirme que les bons langages de programmation devraient imposer des conventions et en faciliter sa documentation [119]. Ces langages sont développés par et pour l'informatique appliquée, et, s'ils ne conviennent pas, il suffit d'en développer d'autres.

(voir énoncé[s] lié[s] : AS12, AS13 p.215)

**P76. L'appropriation d'actes réservés en informatique est davantage une lutte d'économique.**

L'article de Tracey L. Adams, publié en 2007, résume bien la situation entourant le titre d'ingénieur logiciel aux États-Unis, en Grande-Bretagne et au Canada [4]. Le modèle de professionnalisation anglo-américain se distingue par le fait que les dirigeants professionnels font appel à l'État pour obtenir des privilèges professionnels, ce qui mène à des luttes interprofessionnelles. Dans cette perspective, l'auteur affirme que les informaticiens au Canada ont tardé à s'approprier leur profession. Le terme **génie logiciel** est devenu plus conflictuel au Canada qu'aux États-Unis ou en Grande-Bretagne, et l'auteur l'explique ainsi :

- les organisations de professionnels informatiques coopèrent moins avec les organisations universitaires et d'ingénieries ;
- les professionnels informatiques sont moins organisés que les ingénieurs donc, ils n'ont pas autant d'influence ;
- le cadre réglementaire entre les professionnels informatiques et les ingénieurs est vraiment contrasté, celui des professionnels informatiques étant très léger.

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

(voir énoncé[s] lié[s] : AS14 p.216)

#### **Améliorer la formation**

Cette sous-section précise que la formation consacrée aux modèles et à la modélisation devrait occuper une place croissante en science, mais qu'elle périclité en informatique. À cet écueil s'ajoute l'absence de consensus sur la formation à offrir en informatique. Cependant, plusieurs s'entendent pour affirmer que la modélisation et la documentation qui s'y rattache demeurent importantes, voire, selon certains, nécessaires.

- P77. La formation sur la modélisation est indispensable pour répondre aux changements rapides. Cela fait plusieurs années que le développement des compétences en modélisation est considéré comme essentiel en éducation, et ce, par plusieurs experts. Pour Mei-Hung Chiu et Jing-Wen Lin, cela serait essentiel à la culture scientifique de tous les citoyens [56]. Un article dans *Towards a Competence-Based View on Models and Modeling in Science Education* indique qu'un consensus semble se développer autour de l'importance de l'apprentissage de la modélisation [387]. En réponse aux changements rapides dans les sciences, les technologies, l'ingénierie et les mathématiques, de nouvelles politiques apparaissent. Elles misent sur le développement de la pensée computationnelle ainsi que des compétences en modélisation interdisciplinaire [164]. Néanmoins, il reste du travail à accomplir concernant la formation en modélisation. Les études empiriques sur le sujet sont peu nombreuses et, celles portant sur la qualité des modèles le sont encore moins [56]. Une revue de littérature portant sur l'utilisation des ontologies en éducation [370] recense 95 articles publiés entre 2015 et 2019, ce qui témoigne d'un intérêt croissant pour la question.

(voir énoncé[s] lié[s] : AS15, AS16 p.216)

- P78. Plusieurs experts s'entendent pour dire que la formation en informatique est insuffisante, mais il n'y a pas de consensus sur la formation à donner. Depuis de nombreuses années, des experts jugent la formation en informatique appliquée insuffisante. David L. Par-

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

nas a écrit au moins cinq articles sur le sujet [288, 289, 290, 292, 295]. Mary Shaw formule également quelques recommandations à ce propos :

[Traduction libre]

[—] Identifier les rôles distincts dans le développement logiciel et proposer une formation adaptée à chacun ;

[—] Insuffler une approche d'ingénierie dans les programmes de formation ;

[—] Assurer l'actualisation des formations face à l'évolution rapide du secteur ;

[—] Créer des certifications reflétant fidèlement les compétences. <sup>66</sup> [361]

En 2004, apparaît la version finale de la base de connaissances SWEBOK. Elle servira à l'Association for Computing Machinery (ACM) pour la création des différentes recommandations en matière de programmes d'études.

Cependant, les changements trop nombreux en informatique appliquée mettent à mal l'accumulation de connaissances. David L. Parnas, dans un article publié en 2021, cite trois raisons données par Theodore von Kármán :

[Traduction libre]

[—] L'ensemble des concepts et des faits connus des développeurs de logiciels est énorme ; [...] Ce qui semble important pour certains semble inutile pour d'autres.

[—] La base de connaissances logiciel connaît une croissance toujours rapide ; de plus, une grande partie devient inutile quelques années seulement après son ajout.

[—] Une grande partie de ces connaissances est liée à des outils spécifiques.

[...] les outils peuvent évoluer ou devenir obsolètes ; [...]. <sup>67</sup> [295]

---

66.

[—] *Identifying distinct roles in software development and providing appropriate education for each ;*

[—] *Instilling an engineering attitude in educational programs ;*

[—] *Keeping education current in the face of rapid change ;*

[—] *Establishing credentials that accurately represent ability.*

67.

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

(voir énoncé[s] lié[s] : AS17, AS18, AS19 p.216)

P79. La formation sur les documents et les modèles en informatique demeurent tout aussi importante, et plusieurs experts reconnaissent qu'elle présente encore des lacunes. Tout d'abord, en 1976, Barry W. Boehm décrit les compétences les plus importantes à développer [397]. Selon lui, les champs dans lesquels les nouveaux diplômés doivent être mieux formés sont : la documentation technique, l'analyse des exigences logicielles et de conception, le développement de logiciels applicatifs, l'aspect économique du traitement des données, les techniques de gestion, la performance des équipes, les considérations et méthodes de maintenance logicielle.

Ensuite, une revue de littérature ( $n = 30$ ) couvrant la période de 1995 à 2012 détermine que les lacunes qui reviennent le plus souvent sont, entre autres, la communication écrite, la communication orale, la gestion de projet et les instruments logiciels [313]. Une méta-analyse souligne également de grands écarts pour des domaines de connaissances importants. Des domaines de connaissances où la documentation et la modélisation jouent un rôle fondamental, tels les exigences, les conceptions, les évaluations puis la qualité. Il importe de souligner que tant la revue de littérature que la méta-analyse comprenaient des informations issues de différentes sources, notamment d'anciens étudiants et des employeurs.

(voir énoncé[s] lié[s] : AS20 p.216)

P80. Il est généralement admis que l'expertise est primordiale pour utiliser les méthodes formelles. Dans l'article *Ten Commandments Of Formal Methods*, les auteurs affirment que l'accompagnement d'un expert en méthode formelle est un gage de succès pour un projet utilisant des méthodes formelles [35]. Selon un sondage mené auprès de 216 participants reliés aux secteurs critiques, le manque de compétence et de forma-

---

[—] *The set of concepts and facts known to software developers is huge; [...] What seems important to some seems useless to others.*

[—] *The software BoK is always rapidly growing; further, much of it becomes irrelevant just a few years after it has been added.*

[—] *Much of that knowledge is tied to specific tools. [...] the tools may evolve or become out of date; [...].*

### 1.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

tion est jugé comme le deuxième obstacle à l'application de méthodes formelles [103]. L'étude de quatre expérimentations menées à la NASA a permis de conclure qu'il est plus facile d'intégrer un expert en méthodes formelles que de former les individus [80].

Cependant, les méthodes formelles sont de moins en moins enseignées. Dans un article, Maurice H. Ter Beek *et al.* soulèvent que les méthodes formelles ne figurent pas au programme du CS2023 qui est la recommandation de l'ACM pour le programme d'informatique [377]. Maurice H. Ter Beek *et al.* décrivent dans un précédent article la situation :

[Traduction libre] Après des décennies de gloire, les méthodes formelles semblent à un tournant. Dans l'industrie, de nombreux ingénieurs experts en méthodes formelles se voient confier de nouvelles priorités, notamment en intelligence artificielle. Parallèlement, l'enseignement de la vérification formelle dans l'enseignement supérieur est fragmenté. Dans de nombreuses universités, les cours de méthodes formelles sont en baisse, probablement parce qu'ils sont jugés trop difficiles. La transmission de nos connaissances à la prochaine génération n'est pas assurée. Nous ne pouvons pas nous relâcher.<sup>68</sup> [378]

(voir énoncé[s] lié[s] : AS21, AS22 p.216)

Il est retenu de cette revue de littérature que l'atteinte d'un niveau de qualité pour des logiciels est indissociable de la qualité des modèles utilisés. Car, les modèles sont nécessaires aux spécifications, mais aussi à la réalisation et l'établissement des logiciels. Plusieurs documents sont dans les faits un véhicule pour ces modèles. Par conséquent, les documents, les modèles et les connaissances sont intimement liés.

---

68.

*After decades of glory, formal methods seem at a turning point. In industry, many engineers with expertise in formal methods are assigned new priorities, especially in artificial intelligence. At the same time, the formal verification landscape in higher education is scattered. At many universities, formal methods courses are shrinking, likely because they are deemed too difficult. The transmission of our knowledge to the next generation is not guaranteed. So we cannot lean back.*

# Chapitre 2

## Une définition du problème

Derrière les différents problèmes en lien avec les documents et les modèles se cache un problème important : celui du transfert des connaissances. Ce problème étant trop grand, il est circonscrit, dans ce mémoire, à celui de la formalisation et de la compréhension des documents servant aux modèles. Dans ce chapitre, nous présentons d’abord ce problème pour ensuite le modéliser. Par la suite, nous établissons une liste des attentes et une liste des besoins à considérer pour la solution. Ces listes font partie de la définition du problème et elles sont nécessaires au développement et à la validation d’une solution. Ils s’avèrent être les objectifs de la recherche.

### 2.1 Une présentation du problème

Sur la base de la revue de littérature, il est possible d’affirmer qu’il existe des documents nécessaires à la concrétisation de modèles et de logiciels. Ces modèles et ces logiciels expriment des connaissances, dont celles nécessaires à la réalisation et à l’établissement d’un logiciel. Ainsi, en informatique appliquée, les trois sujets que sont les documents, les modèles ou les connaissances sont interreliés. Il en va de même pour plusieurs de leurs écueils, puisque le manque de formalisation et de compréhension des documents se répercute négativement sur les modèles, puis sur les connaissances. Or, puisque la formalisation passe par la modélisation, ce problème touche fondamentalement l’informatique appliquée, ce qui en accentue l’importance.



## 2.1. UNE PRÉSENTATION DU PROBLÈME

Sommairement, les objectifs et les écueils montrent un manque de formalisation et de compréhension qu'il est possible d'énumérer comme ceci :

- une mauvaise compréhension dans les objectifs visés par les langages formels ;
- une mauvaise compréhension dans les objectifs visés par les artefacts ;
- une mauvaise compréhension dans l'usage de langages formels ;
- une mauvaise compréhension dans l'usage des artefacts.

Cela se reflète dans l'utilisation de langages non formels ainsi que dans l'utilisation de mesures et de critères subjectifs. Pour ce qui est des conséquences aux écueils, il est perçu un manque de cohérence, de couverture et d'efficacité des artefacts produits. D'ailleurs, ce qui est contenu dans les documents et les modèles a généralement un impact visible ou non immédiatement visible dans les logiciels produits. Il faut également prendre en considération d'autres conséquences qui méritent une attention particulière.

Ces conséquences s'avèrent être des caractéristiques de qualité à respecter pour les documents, les modèles et les logiciels qui peuvent être définies comme étant absolues, relatives et universelles. Il y a :

- les caractéristiques de qualité absolues : cohérent, couverture et efficace ;
- les caractéristiques de qualité relatives : complétude, efficience et adaptabilité ;
- les caractéristiques de qualité universelles : réfutabilité et éthique.

Ces mêmes caractéristiques sont utilisées au chapitre consacré à l'évaluation de la solution.

Ces caractéristiques sont également prises en considération, à des niveaux variables, par les diverses approches présentées lors de la revue de littérature. Ces approches, prenant souvent la forme d'instruments, utilisant différents langages et différentes règles. Ainsi, il y a :

- des langages :
  - avec une sémantique ou une pragmatique formellement définie (langages formels sémantiquement) ;

## 2.2. UNE MODÉLISATION DU PROBLÈME

- avec une syntaxe formellement <sup>69</sup> définie (langages formels syntaxiquement) ;
- non formellement définis (langages non formels) ;
- des règles :
  - avec inférence logique ;
  - avec inférence statistique ;
  - sans inférence.

Il est important de souligner que l'utilisation de règles avec inférence logique nécessite l'emploi d'un langage formel. Se restreindre à cette approche mène à l'obtention d'un système logique ou d'un modèle logique et susceptible d'être vérifié, en grande partie, par une machine abstraite. Cependant, l'utilisation d'autres combinaisons de langages et de règles ne l'assure pas.

En plus des caractéristiques de qualité, le couplage doit être pris en compte par les instruments servant à la documentation. Parce que les coûts d'évolution et de maintenance sont directement proportionnels au nombre de couplages, donc influence l'adaptabilité. Néanmoins, cela est rarement appliqué objectivement à la documentation. Ainsi, même s'il faut assurer la cohérence tout en minimisant le couplage, l'organisation des artefacts dépendra souvent du choix des artefacts retenus et de leurs contenus. Ces choix répondent à des critères objectifs et subjectifs. Certains peuvent opter pour regrouper toutes les règles servant au développement d'un modèle dans un document de spécification des exigences. D'autres peuvent distribuer les règles à travers différents documents et les mettre sous la forme de diagrammes UML ou de *user stories*. Ces choix ont un effet sur le couplage et la cohérence.

## 2.2 Une modélisation du problème

Le problème est celui de la formalisation et de la compréhension des documents servant aux modèles. Du point de vue d'un praticien informatique, ce problème apparaît

---

69. Un langage avec une sémantique formelle comprend une syntaxe formelle.

## 2.2. UNE MODÉLISATION DU PROBLÈME

lors de l'utilisation d'instruments servant à faire des logiciels. Une mauvaise compréhension des documents a des répercussions négatives sur la qualité des logiciels. En plus, ces instruments peuvent être eux-mêmes des logiciels développés à l'aide de documents pour lesquels une mauvaise compréhension existait. Tous ces logiciels font partie de processus et en exécutent d'autres. Des processus qui ont été précédemment modélisés de manière formelle ou non par des praticiens informatiques. Ainsi, la modélisation du problème est en fait la rétro-ingénierie de la revue de littérature. Elle s'articule autour de trois modèles représentant des processus. Ces modèles contraignent ces procédés sans pour autant les définir, soit :

- un procédé pour l'obtention d'un instrument ;
- un procédé pour l'obtention d'un logiciel ;
- un procédé pour l'obtention d'un modèle.

Ces modèles sont décrits, dans les paragraphes qui suivent, en utilisant des prédicats et le langage de diagramme EA. Un prédicat est une

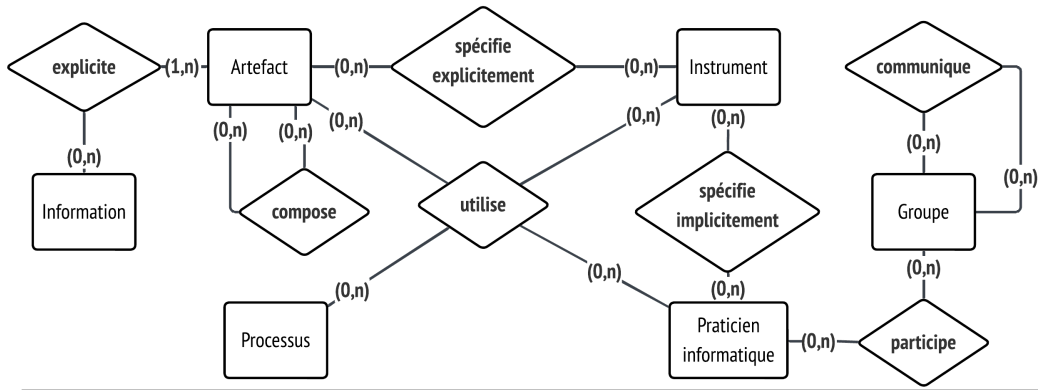
Fonction mettant en œuvre un ou plusieurs arguments, utilisée pour déclarer quelque chose concernant des objets, et dont le résultat est vrai ou faux. [273]

Ces prédicats pourront être précisés en fonction des procédés décrits ou désirés. Ils peuvent être davantage restreints. Le prochain paragraphe comprend la description du langage de diagramme entité-association (EA). En complément de la modélisation de ces processus, plusieurs observations fournissent des indices quant aux besoins que la solution devra satisfaire.

Le modèle d'un procédé d'obtention d'un instrument est composé de l'ensemble de ces prédicats :

$explicite(i, a) :=$  un artefact  $a$  rend manipulable une ou  
plusieurs informations  $i$   
 $compose(a1, a2) :=$  un artefact  $a1$  compose zéro, un ou plusieurs artefacts  $a2$   
 $specifie\_exp(a, i) :=$  un artefact  $a$  participe à la  
spécification explicite de zéro, un ou plusieurs instruments  $i$

## 2.2. UNE MODÉLISATION DU PROBLÈME



**FIGURE 2.1** – Diagramme EA du procédé d'obtention d'un instrument

$utilise(f, p, i, a) :=$  dans le cadre d'un processus  $f$ , zéro, un ou plusieurs praticien informatique  $p$  utilise zéro, un ou plusieurs instruments  $i$  pour créer zéro, un ou plusieurs artefacts  $a$

$communique(g1, g2) :=$  un groupe  $g1$  transmet des informations à zéro, un ou plusieurs groupes  $g2$

$participe(p, g) :=$  un praticien informatique  $p$  appartient à un zéro, un ou plusieurs groupes  $g$

$specifie\_imp(p, i) :=$  un praticien informatique  $p$  partage l'usage qu'il fait de zéro, un ou plusieurs instruments  $i$  <sup>70</sup>.

Le langage de diagramme EA fournit un diagramme représentant le modèle du procédé d'obtention d'un instrument (figure 2.1). Plusieurs observations sont faites sur le modèle de ce procédé d'obtention d'un instrument à partir des prédicats ou de la figure 2.1 :

- La présence même de l'artefact explicite de l'information et son absence le peut également.
- Un processus n'existe pas s'il n'a pas d'intrant et d'extrant, c'est-à-dire qu'il nécessite un participant informatique, un instrument ou un artefact. Il n'y a

<sup>70</sup>. Il existe de nombreux instruments qui sont détournés de leurs usages principaux sans que personne ne le consigne dans un artefact. Ils sont spécifiés implicitement.

## 2.2. UNE MODÉLISATION DU PROBLÈME

pas de distinction entre l'intrant et l'extrant si ce n'est son usage lors d'un processus.

- c) Un chemin emprunté par les connaissances en informatique appliquée est celui du traitement de l'information. Les connaissances sont des informations et leurs communications nécessitent une forme explicite, ainsi ils deviennent les artefacts du procédé d'obtention d'un instrument.
- d) Un autre chemin emprunté par les connaissances en informatique appliquée est le transfert des connaissances par l'intermédiaire de groupes. Cela est modélisé par les entités et les associations **participe, groupe, communique**.
- e) L'instrument accumule des représentations des connaissances, puis les met à la disposition.
- f) Certains documents et certains logiciels sont des instruments pour les participants informatiques.
- g) Des praticiens informatiques peuvent utiliser des instruments selon un usage différent de celui prévu et sans en spécifier explicitement l'usage.
- h) Le niveau d'utilisation d'un instrument peut varier selon sa maîtrise. Cependant, la revue de littérature n'a pas exposé assez de connaissances à ce sujet pour en permettre la modélisation.
- i) La spécification explicite d'un instrument passe par le processus.

Le modèle d'un procédé d'obtention d'un logiciel est composé de l'ensemble des prédicats suivants :

$rédige(p, d) :=$  un praticien informatique  $p$  apporte zéro, un ou plusieurs modifications de rédaction à un document  $d$

$consulte(p, d) :=$  un praticien informatique  $p$  consulte zéro, un ou plusieurs documents  $d$

$concrétise\_m(d, m) :=$  un document  $d$  rend manipulable un ou plusieurs modèles  $m$

$est(l, i) :=$  un logiciel  $l$  est zéro, un ou plusieurs instruments  $i$

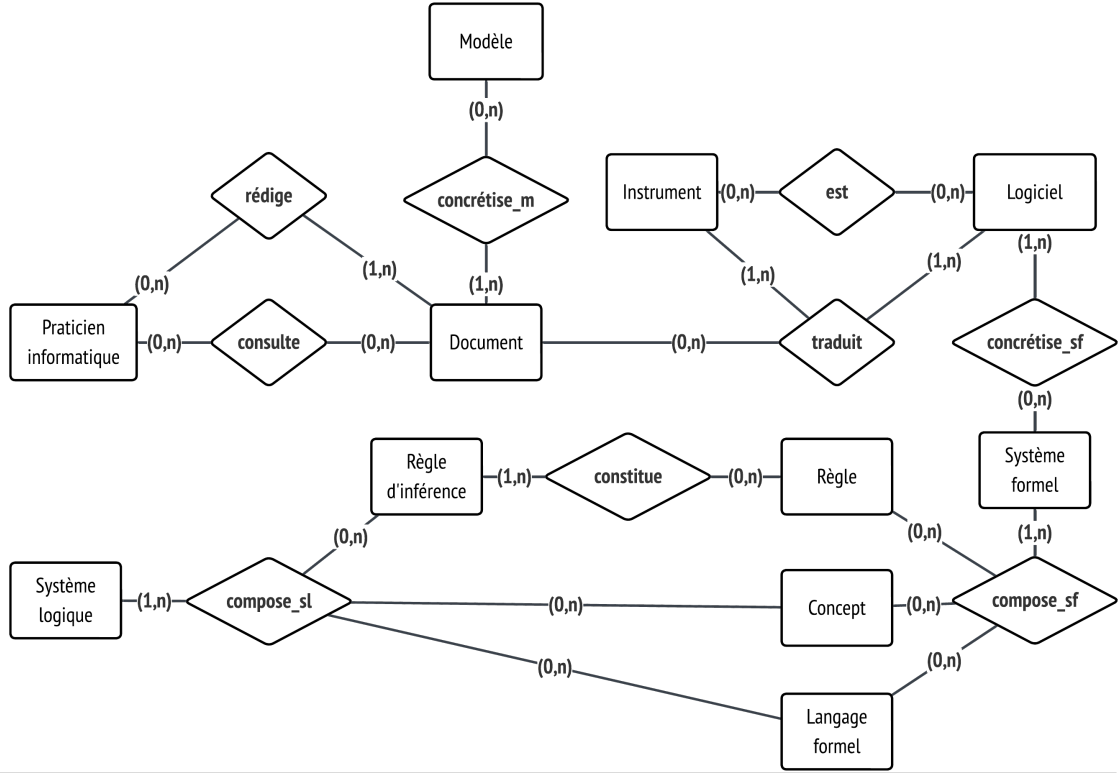
## 2.2. UNE MODÉLISATION DU PROBLÈME

- $traduit(i, d, l) :=$  un instrument  $i$  traduit un ou plusieurs documents  $d$   
vers un ou plusieurs logiciels  $l$
- $concrétise\_sf(l, s) :=$  un logiciel  $l$  matérialise un ou plusieurs systèmes formels  $s$
- $compose\_sf(s, r, c, l) :=$  un système formel  $s$  est composé  
de zéro, une ou plusieurs règles  $r$ , concepts  $c$   
et langages formels  $l$
- $constitue(i, r) :=$  une règle d'inférence  $i$  est une ou plusieurs règles  $r$
- $compose\_sl(s, i, c, l) :=$  un système logique  $s$  est composé  
de zéro, une ou plusieurs règles d'inférences  $i$ , concepts  $c$   
et langage formel  $l$
- $partage(sl, sf, r, c, l) :=$  un système formel  $sf$  et un système logique  $sl$   
partagent une ou plusieurs règles  $r$ ,  
concepts  $c$  et langages formels  $l$

Le langage de diagramme EA fournit un diagramme représentant le modèle d'un procédé d'obtention d'un logiciel (figure 2.2). Plusieurs observations sont faites sur le modèle du procédé d'obtention d'un logiciel à partir des prédicats ou de la figure 2.2 :

- a) En informatique appliquée, le modèle à concrétiser peut provenir du domaine du problème ou du domaine de la solution.
- b) Les documents, les instruments, les logiciels sont tous des artefacts.
- c) Les logiciels concrétisent des systèmes et certains sont des modèles.
- d) Les instruments de traduction et d'exécution reposent sur un système formel et celui-ci repose sur un système logique.
- e) Les méthodes formelles visent à spécifier et à vérifier des logiciels à l'aide des systèmes logiques.
- f) Les logiciels ne se limitent pas aux systèmes formels, puisqu'ils intègrent des interfaces utilisateurs. Ces logiciels ne font pas partie de la modélisation.
- g) Les systèmes logiques et formels étant des entités évolutives, elles peuvent à un moment être composées d'aucune règle, concept ou langage formel.

## 2.2. UNE MODÉLISATION DU PROBLÈME



**FIGURE 2.2** – Diagramme EA du procédé d'obtention d'un logiciel

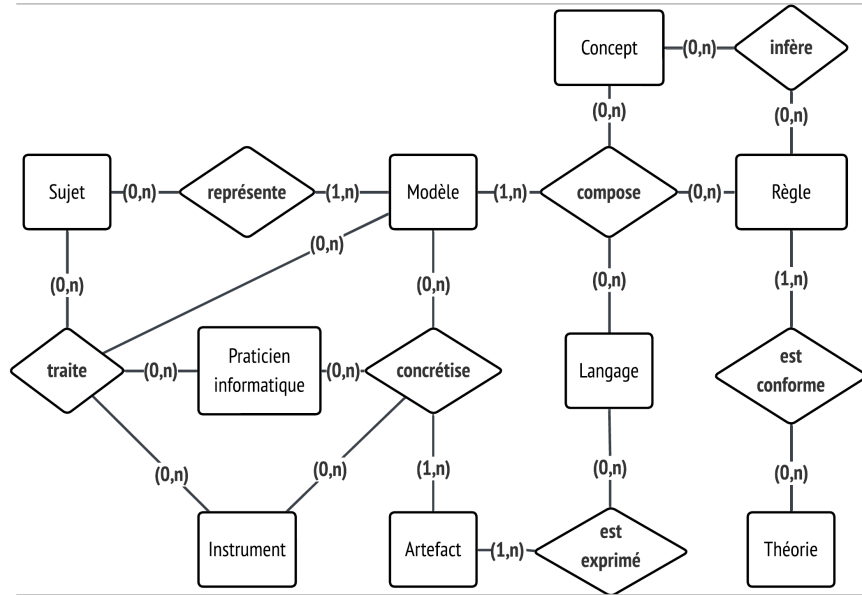
Le modèle d'un procédé d'obtention d'un modèle est composé de l'ensemble des prédicats suivants :

$infère(r, c) :=$  zéro, une ou plusieurs règles  $r$  établit des relations  
entre zéro, un ou plusieurs concepts  $c$

$représente(m, s) :=$  un modèle  $m$  est une représentation qui décrit un  
ou plusieurs sujets  $s$

$compose(c, l, r, m) :=$  zéro, un ou plusieurs concepts  $c$ , langages  $l$   
et réglementations  $r$  composent un ou plusieurs modèles  $m$

## 2.2. UNE MODÉLISATION DU PROBLÈME



**FIGURE 2.3** – Diagramme EA du procédé d'obtention d'un modèle

$traite(a, s, m, i) :=$  un praticien informatique  $a$  traite zéro, un ou plusieurs  
sujets  $s$  avec zéro, un ou plusieurs modèles  $m$   
ou instruments  $i$

$concrétise(m, a, p, i) :=$  un modèle  $m$  est concrétisé avec un artefact  $a$   
par zéro, un ou plusieurs praticiens informatiques  $p$   
ou instruments  $i$

$est\_conforme(r, t) :=$  zéro, une ou plusieurs règles  $r$  sont conformes à une  
ou plusieurs théories  $t$

$est\_exprimé(a, s) :=$  un artefact  $a$  est exprimé avec un ou plusieurs langages  $l$

Le langage de diagramme EA fournit un diagramme représentant le modèle du procédé d'obtention d'un modèle ( figure 2.3). Plusieurs observations sont faites sur le modèle du procédé d'obtention d'un modèle à partir des prédicats ou de la figure 2.3 :

- a) En informatique appliquée, le sujet peut être une machine abstraite, un modèle ou un artefact.



## 2.3. DES ATTENTES À CONSIDÉRER POUR LA SOLUTION

- b) Lorsque le sujet est un modèle, il s'agit du procédé d'obtention d'un métamodèle.
- c) Le traitement <sup>71</sup> d'un modèle peut comprendre la vérification ou la validation : cela dépend du sujet et de l'instrument.
- d) L'instrument peut provenir de la concrétisation d'un modèle.
- e) L'adéquation entre un modèle et sa concrétisation dépend des participants informatiques, des langages, des concepts et des règles.
- f) Un artefact peut réaliser un modèle, mais ce n'est pas forcément le cas. Par exemple, le code source (artefact) doit s'exécuter sur une machine abstraite pour réaliser un modèle. Néanmoins, le code source concrétise le modèle.

## 2.3 Des attentes à considérer pour la solution

Les attentes sont celles de l'ensemble des participants informatiques gravitant autour du problème et de sa solution, et s'inspirent de la revue de littérature. Ces désirs, une fois comblés, pourraient assurer l'adoption de la solution. Cependant, leurs formes subjectives peuvent conduire à des ambiguïtés ne permettant pas de confirmer leur respect avec la logique.

Cinq attentes ont été choisies et il peut en exister d'autres :

AT01 : La solution devrait améliorer la compréhension des utilisateurs quant aux documents servant aux modèles [P46, P47, P77, P78].

AT02 : La solution devrait fournir, dans le cadre d'un environnement de développement, les fonctionnalités nécessaires à la production et à la maintenance des documents servant aux modèles [P50, P51].

AT03 : La solution devrait rendre plus efficiente la manipulation des documents servant aux modèles, plus particulièrement en regard du temps requis et de la charge

---

71. En informatique, il peut arriver que le traitement ne comprenne pas la vérification ou la validation, ce n'est pas le cas ici.

## 2.4. DES BESOINS À CONSIDÉRER POUR LA SOLUTION

cognitive induite par le processus, et ce, pour l'ensemble des professionnels informatiques et des participants informatiques [P08, P09].

AT04 : La solution devrait être utilisable par des utilisateurs ayant des acquis distincts, et elle devrait finir par être maîtrisée [P56, P62, P64, P79, P80].

AT05 : La solution devrait s'utiliser dans différents environnements de développement faisant appel à différents procédés, gabarits de documents, langages, modèles, métamodèles et configurations [P03, P58].

## 2.4 Des besoins à considérer pour la solution

Les besoins sont justifiés et plus précis que les attentes. Ils regroupent ce qui est fondamentalement nécessaire pour satisfaire les parties prenantes. Dans le contexte de l'informatique appliquée, il y a deux caractéristiques majeures qui subsistent. La première caractéristique développe et utilise des modèles pour réaliser et établir des logiciels. La seconde applique à l'informatique appliquée des instruments provenant de logiciels. Ces deux caractéristiques font que le terme **modèle** est privilégié à celui de **document** pour exprimer les besoins.

Parmi les modèles, certains sont disponibles à l'utilisateur, tandis que d'autres sont réservés au système. Ceux qui sont disponibles à l'utilisateur sont les modèles développés qui correspondent aux logiciels à réaliser et à établir. Les modèles qui sont disponibles au système sont en fait des instruments se retrouvant dans la solution, ils sont nommés modèles systèmes. Les deux catégories de modèles peuvent contenir des métamodèles.

L'analyse de la revue de littérature a permis d'identifier au moins cinq besoins. Voici la liste avec leurs justifications :

BE01 : Permettre l'extraction des attributs des modèles développés depuis les documents qui les décrivent.

## 2.4. DES BESOINS À CONSIDÉRER POUR LA SOLUTION

- Séparer les éléments des documents pour être en mesure de les traiter syntaxiquement ou sémantiquement de façon à obtenir de nouveaux artefacts variant tant par la forme que le contenu [P26, P53].
- Rechercher, voire régénérer, les modèles à partir des artefacts peut constituer le seul moyen de les retrouver (rétro-ingénierie) [P67, P68].

BE02 : Permettre la création de nouveaux modèles développés depuis l’usage des modèles systèmes et des modèles développés.

- Au sein d’une collectivité, l’usage détermine souvent l’existence et la nécessité des différentes catégories de documents qui peuvent être des modèles ou des gabarits [P69, P75].
- Il existe déjà des modèles concrétisés par des logiciels dont plusieurs sont patrimoniaux [P16].
- Les documents utilisent différents types de langages formels ou non formels [P29, P32, P43].
- Les modèles touchent différents domaines de connaissances, donc les concepts sont différents [P40].
- L’informatique développe des modèles depuis différents domaines de connaissances, cela a une répercussion sur les concepts [P60].

BE03 : Permettre d’identifier les changements apportés aux documents pour déterminer au mieux les répercussions potentielles sur les modèles développés.

- Les documents sont utilisés à travers tout le processus de développement, la traçabilité ajoute la dimension temporelle et permet différents suivis, comme celui des coûts [P06].
- Maintenir à jour les documents pour éviter que cela ne devienne de la dette technique tant dans son contenu que dans sa forme, ainsi que dans le respect des modèles (gabarit, procédé) en place [P07, P18].
- La fréquence des changements apportés aux logiciels est de plus en plus élevée et certains touchent aux exigences, cela peut mener à des répercussions importantes [P17]<sup>72</sup>.

---

72. Ce justificatif peut aussi se retrouver dans le besoin BE04, car il laisse entendre qu’il existe de nombreux environnements distincts.

## 2.4. DES BESOINS À CONSIDÉRER POUR LA SOLUTION

BE04 : Permettre l'adaptation des modèles développés pour différents contextes ou systèmes.

- Différents systèmes logiques sont utilisés, certains sont seulement plus utilisés que d'autres, de sorte qu'il ne semble pas réaliste d'en imposer qu'un [P39].
- Les modèles peuvent varier selon le contexte, ce qui inclut le procédé de développement, le cadre réglementaire et les expertises employées [P04, P05].

BE05 : Permettre l'évaluation de la qualité d'un modèle développé, les caractéristiques de qualité étant définies par un modèle système.

- Le contrôle de la documentation est minime, car il y a souvent un manque d'encadrement, de supervision et fondamentalement d'intérêt envers ces artefacts [P11, P12, P13, P14].
- La qualité d'un document et d'un logiciel est reliée et exprimée par un modèle [P22, P28, P54, P55].
- Les caractéristiques de qualité des documents et des logiciels sont aussi celles qu'un modèle doit respecter<sup>73</sup> [P15, P21, P25, P30, P44, P45, P52].

---

73. Conséquemment, les mêmes méthodes ou techniques pourraient être utilisées.

# Chapitre 3

## Le développement d'une solution

Un logiciel est la concrétisation d'une solution informatique qui résulte d'un procédé de développement. Ce chapitre présente un modèle de solution en exposant des exigences, des éléments de conception et d'implémentation. Les exigences peuvent émaner du problème à résoudre ou de la solution proposée. Certaines de ces exigences exposent un prolongement à la solution. Les exigences sont réparties dans trois sections. En ce qui a trait au modèle de la solution, il prend la forme d'un instrument d'aide à la rédaction (IAR) pour les documents servant aux modèles en informatique appliquée.

Cet instrument, qui est en fait une preuve de concept, s'attarde sur la création d'un compilateur qui répondra aux besoins. Toutefois, des travaux ont été entamés pour étendre les fonctionnalités de l'instrument. L'IAR prenant la forme d'un atelier, il devient plus aisé de manipuler les configurations et les modèles.

### 3.1 Des exigences de la modélisation du problème

Les besoins, les diagrammes et les prédicats explicités dans le chapitre 2 constituent des règles pour un modèle du problème. Les exigences ont pour but de caractériser les solutions acceptables (voire adéquates) en regard du problème. Il est donc nécessaire qu'elles soient comprises (donc compréhensibles) puis évaluées (donc évaluables) par

### 3.1. DES EXIGENCES DE LA MODÉLISATION DU PROBLÈME

les parties prenantes. Ces règles peuvent s’inspirer du modèle du problème ou provenir de la conception et font toutes partie d’un modèle de solution.

Les exigences en lien avec le modèle du problème sont :

EX01 : Le système doit permettre la création et la modification des modèles <sup>74</sup>.

EX02 : Le système doit permettre l’importation et l’exportation des modèles <sup>75</sup>.

EX03 : Le système doit permettre la réutilisation des modèles développés.

EX04 : Le système doit permettre la comparaison des modèles développés.

EX05 : La structure d’un modèle au sein du système doit comprendre des concepts, des règles et des langages.

EX06 : Le système doit permettre de former un modèle en fonction des métamodèles utilisés par le système.

EX07 : Le système doit opérer toutes les transformations permises par les modèles sur la base du formalisme qui les décrit.

EX08 : Le système doit extraire les éléments requis à la synthèse d’un modèle développé à partir des documents et de la configuration soumise.

EX09 : Le système doit vérifier la description proposée d’un modèle en fonction des intrants (des modèles, des configurations, des documents), et ce, à différents niveaux (lexical, syntaxique et sémantique).

EX10 : Le système doit permettre de présenter les modèles développés.

EX11 : Le système doit donner le moyen de contextualiser les documents (y compris les modèles et métamodèles).

EX12 : Le système doit fournir des informations visant à permettre à l’utilisateur d’évaluer la couverture, l’efficience et l’adaptabilité du modèle développé.

Il convient de souligner que ces exigences n’explicitent pas le respect de certaines caractéristiques de qualités. En voici les raisons :

---

74. Ce sont des modèles développés qui répondent aux objectifs d’un procédé de développement et qui sont formulés en termes de métamodèles.

75. Ce sont des modèles développés qui peuvent eux-mêmes devenir des instruments pour d’autres modèles.

### 3.2. UNE CONCEPTION D'UNE SOLUTION

- cohérent : un modèle formel a de l'intérêt que s'il s'appuie sur un système logique qui permettra de trouver des incohérences ;
- efficace : se vérifie avec l'aide d'un deuxième modèle, les relations entre modèles sont comprises à l'intérieur des exigences supplémentaires (futur développement) ;
- complétude : la complétude n'étant pas atteignable, c'est à l'utilisateur de déterminer les critères satisfaisants ;
- réfutabilité : nécessite l'intégration de procédés scientifiques à même l'instrument qui permettra entre autres la définition de conséquences éventuellement vérifiable en pratique ;
- éthique : nécessite la formalisation des règles éthiques provenant de connaissances dépendantes à la fois du contexte et de l'usage.

## 3.2 Une conception d'une solution

Cette conception se déroule à partir du niveau le plus abstrait. Cela permet de donner une vue d'ensemble et d'ajouter les détails par étapes. Cette conception comprend un modèle logique de traitement et un modèle logique de données. Le modèle conceptuel de traitement<sup>76</sup> est concrétisé par un diagramme de contexte et des diagrammes de flux de données, tandis que le modèle conceptuel de données<sup>77</sup> est concrétisé par un diagramme entité association. Les descriptions formelles de ces diagrammes se retrouvent en annexe C.

Les éléments du diagramme de contexte (figure 3.1) :

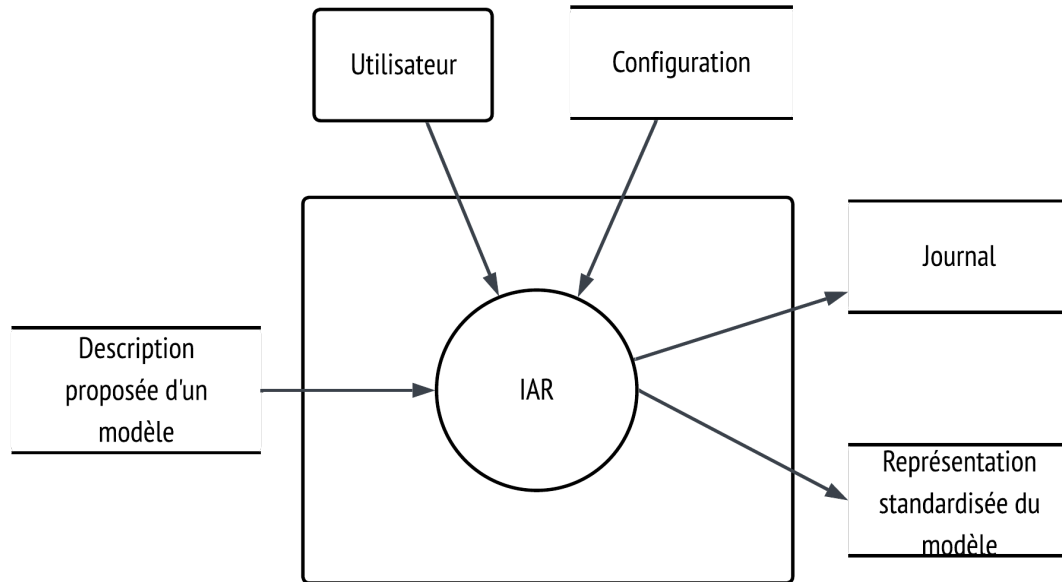
- **Utilisateur** : toute personne modifiant le modèle ;

---

76. Le modèle conceptuel de traitement appartient au domaine de la solution et le modèle conceptuel de flux appartient au domaine du problème ; ces modèles peuvent s'exprimer de manière similaire.

77. Le modèle conceptuel de données peut devenir calculable et le modèle logique de données est calculable ; ces modèles peuvent s'exprimer de manière similaire.

### 3.2. UNE CONCEPTION D'UNE SOLUTION

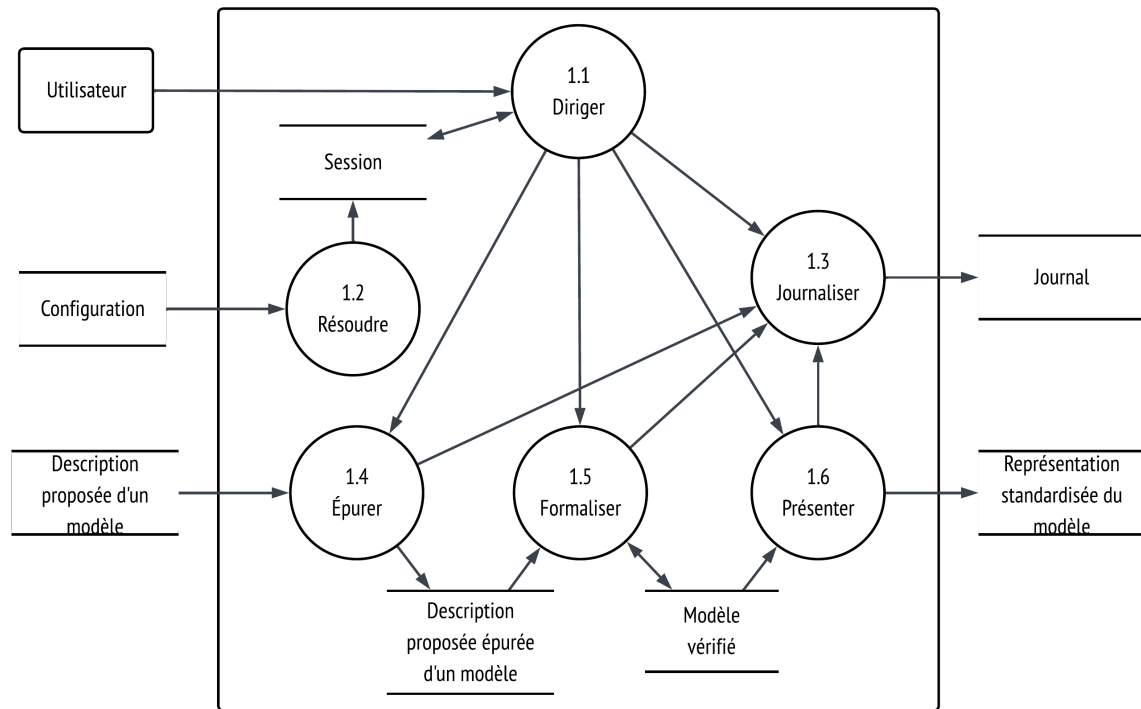


**FIGURE 3.1** – Diagramme de contexte de l'IAR

- **Configuration** : ensemble de descriptions formelles exprimant des métamodèles ainsi que des règles ;
- **Journal** : ensemble de documents informant l'utilisateur du déroulement du processus ;
- **Description proposée d'un modèle** : ensemble de documents définissant un modèle ou des changements à un modèle ;
- **IAR** : applique des transformations pour obtenir un modèle formel respectant un système logique ;
- **Représentation standardisée du modèle** : ensemble de documents produits à partir de la description proposée d'un modèle et respectant les métrarègles du système et celles de la configuration ;
- **règle** : décrit formellement une extraction, une transformation ou une contrainte à appliquer au modèle formel dans un objectif donné, elle est elle-même décrite en utilisant une description formelle.



### 3.2. UNE CONCEPTION D'UNE SOLUTION



**FIGURE 3.2** – DFD du système IAR

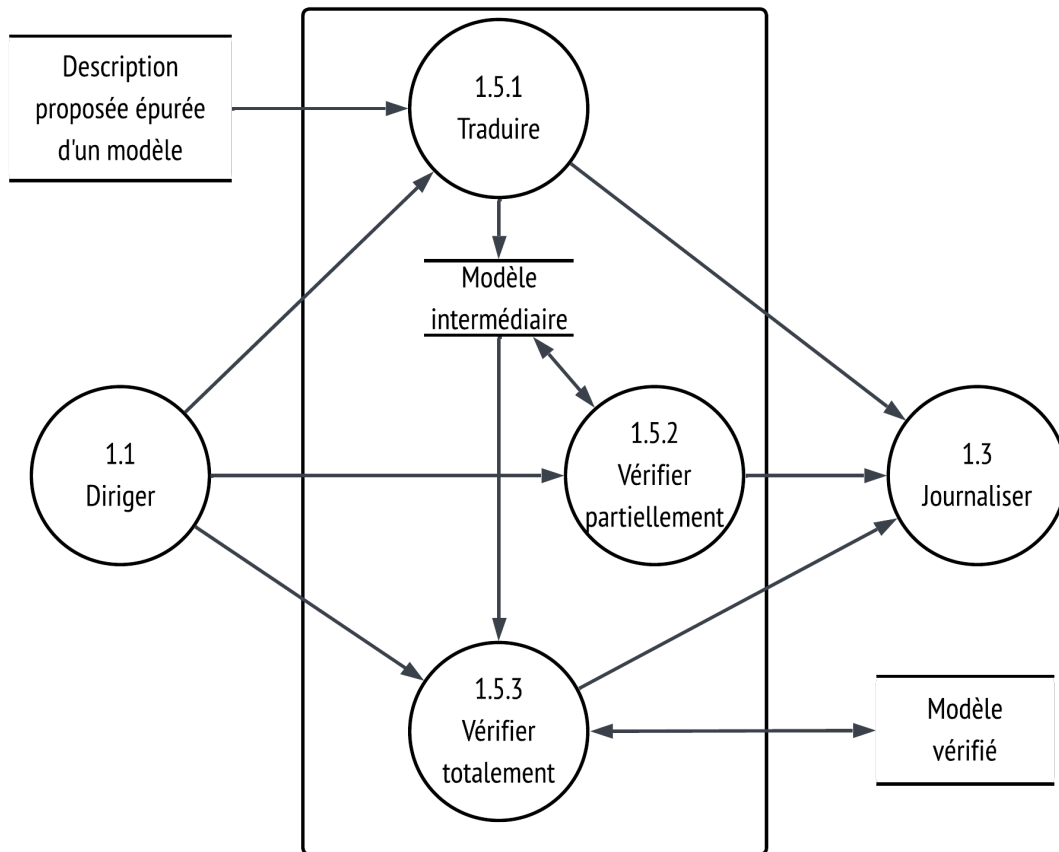
Les éléments du DFD du système IAR (figure 3.2) :

- **Diriger** : coordonne les différents processus et sélectionne les transformations pour les processus **Épurer**, **Formaliser** et **Présenter** ;
- **Session** : contient les références aux descriptions formelles utilisées pour les transformations lors du déroulement ;
- **Résoudre** : rend accessibles les descriptions formelles aux transformations<sup>78</sup> ;
- **Journaliser** : regroupe et transmet le déroulement et les résultats des différentes transformations ;
- **Épurer** : retire des informations non nécessaires à un modèle selon des règles d'épurations<sup>79</sup> ;

78. Une future amélioration serait un flux entre 1.1 et 1.2 afin de manipuler plus facilement la configuration à utiliser.

79. Des exemples de règles sont donnés en annexe D.

### 3.2. UNE CONCEPTION D'UNE SOLUTION



**FIGURE 3.3** – DFD du processus 1.5 Formalise

- **Formaliser** : construit un modèle ou apporte des changements à un modèle selon des règles servant à traduire, à certifier et à assurer ;
- **Présenter** : présente des informations provenant du modèle selon des règles de présentation ;
- **Description proposée épurée d'un modèle** : contient l'ensemble des données nécessaires pour un modèle ;
- **Modèle vérifié** : contient le modèle et le persiste dans la mémoire.

Les éléments du DFD du processus 1.5 Formalise (figure 3.3) :

### 3.2. UNE CONCEPTION D'UNE SOLUTION

- **Traduire** : prends les informations pour en faire un modèle fragmentaire qui suit le métamodèle selon des règles de traduction ;
- **Modèle intermédiaire** : contient des changements à un modèle, une partie du modèle ou un modèle ;
- **Vérifier partiellement** : vérifie le comportement interne du modèle intermédiaire et applique certaines adaptations, et ce, toujours selon des règles<sup>80</sup> ;
- **Vérifier totalement** : vérifie le comportement interne et externe du modèle selon des règles et un système logique.

Le modèle conceptuel de données de la preuve de concept est composé de l'ensemble des prédicats suivants :

$possède(j, e) :=$  un journal  $j$  contient un ou plusieurs événements  $e$

$renvoie(s, m) :=$  un système de référence  $s$  concerne exactement deux modèles  $m$

$comprend(s, r) :=$  un système de référence  $s$  regroupe aucune, une ou plusieurs références  $r$

$concrétise(d, m) :=$  le document  $d$  concrétise aucun, un ou plusieurs modèles

$instancie(m1, m2) :=$  un modèle  $m1$  est un sujet représenté par aucun, un ou plusieurs métamodèles  $m2$

$estconforme\_1(d, m) :=$  une description proposée d'un modèle  $d$  est conforme à un ou plusieurs métamodèles de modèle  $m$

$estconforme\_2(r, m) :=$  une représentation standardisée d'un modèle  $r$  est conforme à un ou plusieurs métamodèles de modèle  $m$

$estconforme\_m(d, m) :=$  un document  $d$  est conforme à aucun, un ou plusieurs métamodèles de document  $m$

$remplit(s, j) :=$  une session  $s$  fait un journal  $j$

$crée(u, s) :=$  un utilisateur  $u$  fait une ou plusieurs sessions  $s$

---

80. Il agit comme un analyseur de code statique.

### 3.2. UNE CONCEPTION D'UNE SOLUTION

$emploie(u, s) :=$  un utilisateur  $u$  utilise une ou plusieurs sessions  $s$

$relève(d, s) :=$  un document  $d$  dépend d'aucune, une ou plusieurs sessions  $s$

$contient\_i(d, i) :=$  un document  $d$  comprend aucune, une  
ou plusieurs informations  $i$

$renferme(i1, i2, i3) :=$  une information  $i1$  a peut-être un ancêtre  $i2$   
ou peut-être plusieurs descendants  $i3$

$provient(i, s) :=$  une information  $i$  vient d'aucune,  
une ou plusieurs sources  $s$

$réfère\_1(i, c) :=$  une information  $i$  concerne un ou plusieurs concepts  $c$

$réfère\_2(f, c) :=$  un formalisme  $f$  concerne un ou plusieurs concepts  $c$

$compose\_e(s, e) :=$  une session  $s$  a une ou des étapes  $e$

$compose\_t(e, t) :=$  une étape  $e$  a aucune, une ou plusieurs  
transformations  $t$

$exploite(t, f) :=$  une transformation  $t$  utilise aucun,  
un ou plusieurs formalismes  $f$

$utilise(f, r) :=$  un formalisme  $f$  utilise aucune, une ou plusieurs règles  $r$

$participe(p, s) :=$  un énoncé logique  $p$  appartient à aucun, un ou  
plusieurs systèmes  $s$

$appartient(r, s) :=$  une règle  $r$  participe à aucun, un ou plusieurs systèmes  $s$

$respecte(s1, s2) :=$  un système  $s1$  respecte  
aucun, un ou plusieurs autres systèmes  $s2$

$unit(r, s1, s2) :=$  une référence  $r$  unit deux sources distinctes  $s1$  et  $s2$

Le langage de diagramme EA fournit un diagramme représentant le modèle conceptuel de données de la preuve de concept (figure 3.4). Les observations suivantes sont faites sur le modèle conceptuel de données à partir des prédicats ou de la figure 3.4 :

- a) L'entité information est peu décrite ; son modèle logique est néanmoins fourni plus loin.

### 3.2. UNE CONCEPTION D'UNE SOLUTION

- b) Le système de références se construit automatiquement avec les entités sources.

### 3.2. UNE CONCEPTION D'UNE SOLUTION

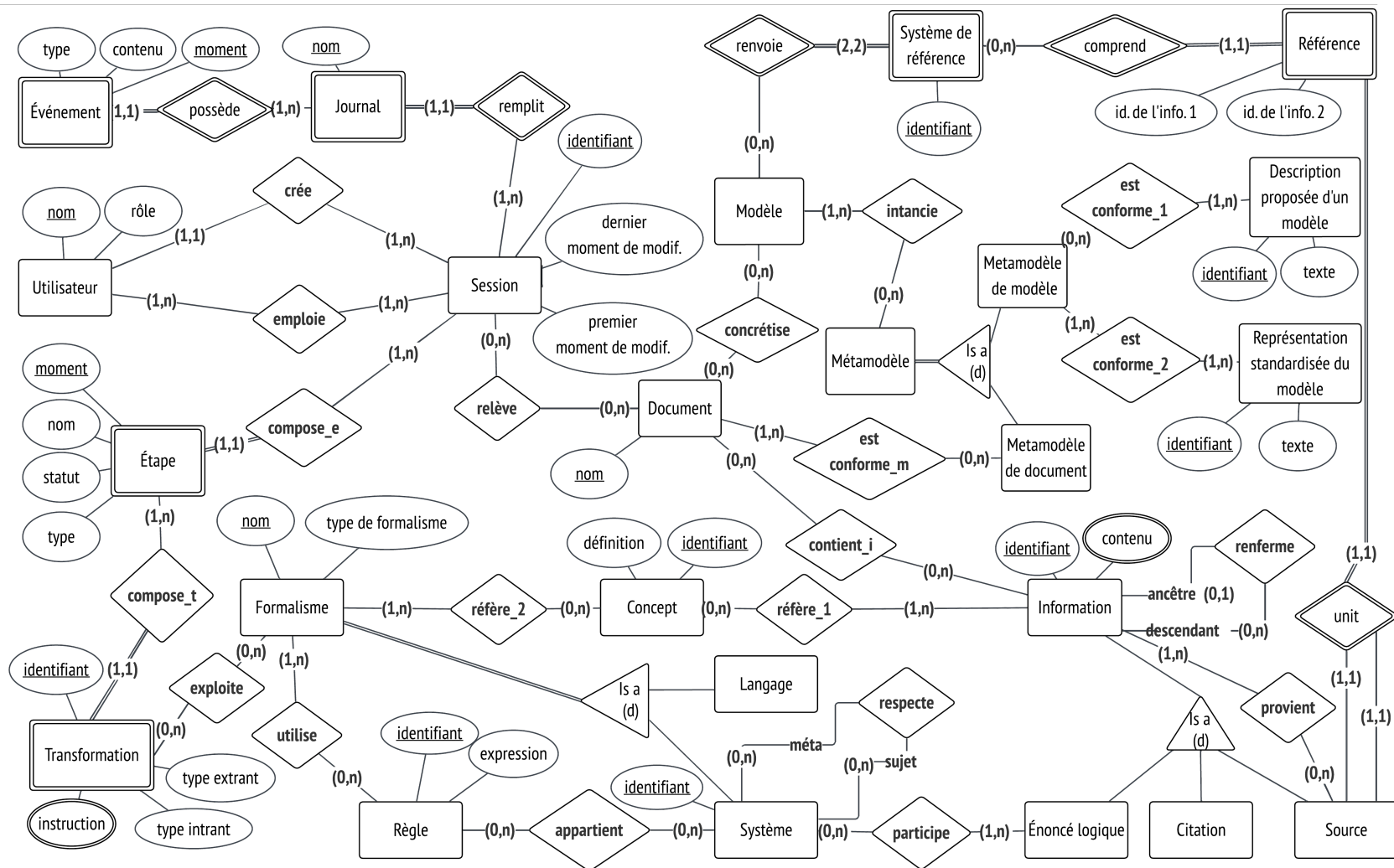


FIGURE 3.4 – Diagramme EA du modèle conceptuel de données de la preuve de concept

### 3.3. DES EXIGENCES DE LA MODÉLISATION D'UNE SOLUTION

## 3.3 Des exigences de la modélisation d'une solution

En développant une solution, de nouvelles exigences apparaissent et s'ajoutent à celles provenant de la modélisation du problème. Elles servent à mettre en place une solution et peuvent comprendre des aspects techniques :

- EX13 : Le système doit épurer la description proposée d'un modèle.
  - Les documents peuvent contenir des informations non nécessaires au modèle ou contenir des fonctions (macro, micro) qui encapsulent des informations importantes.
- EX14 : Le système doit informer les utilisateurs sur le processus lui-même et son déroulement.
- EX15 : Le système doit permettre la contextualisation d'une session de modélisation<sup>81</sup>.

## 3.4 Des explications sur l'implémentation d'une solution

Les explications sur l'implémentation portent sur les instruments employés pour permettre l'utilisation du système, le passage du texte en modèle et la calculabilité du modèle. L'utilisation du système se résume à l'interface utilisateur. L'extraction du texte en modèle se fait grâce à des descriptions formelles. La calculabilité du modèle découle de sa définition en termes de langages formels par l'emploi de systèmes logique, et ce, pour le calcul des transformations autres que linguistiques.

Les instruments servant à l'utilisation du système (interface utilisateur) doivent permettre le développement rapide d'une preuve de concept rapidement et doivent être relativement facile à maîtriser. Dans le cadre de ce mémoire, les recherches ont conduit

---

81. Dans le preuve de concept, le métamodèle de la configuration permet un paramétrage de la part de l'utilisateur.

### 3.4. DES EXPLICATIONS SUR L'IMPLEMENTATION D'UNE SOLUTION

à opter pour l'utilisation de la composante web Ace Editor. De plus, la réutilisation de l'exemple Ace Playground a diminué considérablement le temps de développement. Cette composante s'utilise à travers Qute qui génère des documents tels que des pages web.

- Ace Editor : éditeur de code pour le web écrit en JavaScript<sup>82</sup> ;
- Ace Playground : un exemple sophistiqué de l'utilisation d'Ace Editor<sup>83</sup> ;
- Qute : un moteur de *templates* pour faire des rendus web<sup>84</sup>.

Puisque ce n'est pas le sujet principal de ce mémoire, cette partie ne sera pas davantage détaillée.

Pour extraire et interpréter des informations, il y a l'utilisation de grammaires décrivant des langages formels et des descriptions formelles. Il s'agit de la fonction des compilateurs. L'IAR possède des procédés qui, une fois décomposés, correspondent aux procédés qui peuvent exister à l'intérieur d'un compilateur. La figure 3.5. présente un DFD qui expose un enchaînement possible pour un compilateur. Ce procédé nécessite quelques définitions avec lesquelles il est possible de déduire la grammaire rationnelle et la grammaire algébrique, les voici :

- grammaire : [Traduction libre] « Une grammaire formelle (ou une grammaire, en abrégé) est un quadruplet  $\langle V_T, V_N, P, S \rangle$ , où :
  1.  $V_T$  est un ensemble fini de symboles, appelés symboles terminaux,
  2.  $V_N$  est un ensemble fini de symboles, appelés symboles non terminaux ou variables, tel que  $V_T \cap V_N = \emptyset$ ,
  3.  $P$  est un ensemble fini de paires de chaînes de caractères, appelées productions, chaque paire  $\langle \alpha, \beta \rangle$  étant notée  $\alpha \rightarrow \beta$ , où  $\alpha \in V^+$  and  $\beta \in V^*$  , with  $V = V_T \cup V_N$ , et
  4.  $S$  est un élément de  $V_N$  , axiome ou symbole de départ.»<sup>85</sup> [299] ;

---

82. <https://ace.c9.io> sous licence BSD.

83. <https://github.com/mkslanc/ace-playground> sous licence MIT.

84. <https://quarkus.io/guides/qute> sous license Apache 2.0

85. « A formal grammar (or a grammar, for short) is a 4-tuple  $\langle V_T, V_N, P, S \rangle$ , where :



### 3.4. DES EXPLICATIONS SUR L'IMPLÉMENTATION D'UNE SOLUTION

- règles de production pour une grammaire rationnelle : « *a production in  $P$  is of type 3 (or regular) iff it is of the form  $\alpha \rightarrow \beta$ , where  $\alpha \in V_N$  and  $\beta \in \{\varepsilon\} \cup V_T \cup V_T V_N$*  »<sup>86</sup> [299] ;
- règles de production pour une grammaire algébrique : « *a production in  $P$  is of type 2 (or context-free) iff it is of the form  $\alpha \rightarrow \beta$ , where  $\alpha \in V_N$  and  $\beta \in V^*$*  »<sup>87</sup> [299] ;
- lexème : [Traduction libre] « l'analyseur lexical lit les caractères du programme source, les regroupe en unités lexicalement significatives appelées lexèmes et produit en sortie des jetons représentant ces lexèmes. »<sup>88</sup> [9] ;
- arbre syntaxique abstrait (AST, *abstract syntax tree* en anglais) : [Traduction libre] « Un arbre syntaxique abstrait est une version de l'arbre syntaxique dans laquelle seuls les nœuds sémantiquement importants sont conservés. La notion de “sémantiquement important” est laissée à l'appréciation du concepteur du compilateur. »<sup>89</sup> [111] ;

Des éléments du procédé représenté par la figure 3.5 sont :

- texte : ensemble de caractères que le compilateur prend en entrée ;
- analyse lexicale : [Traduction libre] « Ce composant analyse le programme source et reconnaît tous les tokens, c'est-à-dire les sous-chaînes de caractères consécutifs qui forment un tout logique. Les mots-clés et les identificateurs sont des exemples courants de tokens, mais il en existe beaucoup d'autres. »<sup>90</sup> [122] ;

- 
1.  $V_T$  is a finite set of symbols, called terminal symbols,
  2.  $V_N$  is a finite set of symbols, called nonterminal symbols or variables, such that  $V_T \cap V_N = \emptyset$ ,
  3.  $P$  is a finite set of pairs of strings, called productions, each pair  $\langle \alpha, \beta \rangle$  being denoted by  $\alpha \rightarrow \beta$ , where  $\alpha \in V^+$  and  $\beta \in V^*$ , with  $V = V_T \cup V_N$ , and
  4.  $S$  is an element of  $V_N$ , called axiom or start symbol.

»

86.  $V^+ = V^* \setminus \{\varepsilon\}$

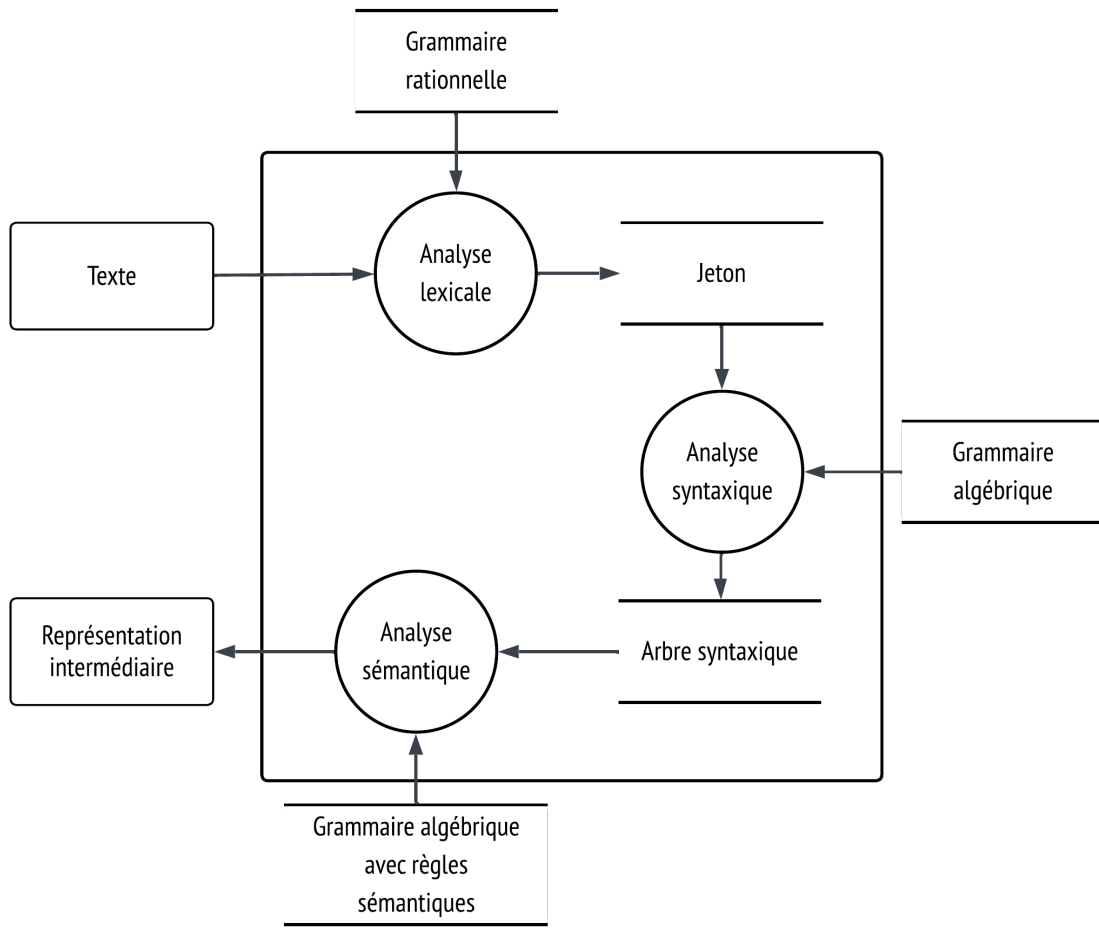
87.  $\varepsilon$  est la chaîne vide.

88. « *lexical analyzer reads the characters of the source program, groups them into lexically meaningful units called lexemes, and produces as output tokens representing these lexemes.* »

89. « *An abstract syntax tree is a version of the syntax tree in which only the semantically important nodes are retained. What is “semantically important” is up to the compiler writer.* »

90. « *This component scans the source program and recognizes all tokens those substrings of consecutive characters that belong together logically. Keywords and identifiers are common examples of tokens but there are many others.* »

### 3.4. DES EXPLICATIONS SUR L'IMPLÉMENTATION D'UNE SOLUTION



**FIGURE 3.5** – DFD pour certains types de compilateurs

- jeton : [Traduction libre] « Un jeton est composé de deux éléments : un nom et une valeur d'attribut. Les noms de jetons sont des symboles abstraits utilisés par l'analyseur syntaxique [...]. La valeur d'attribut, si elle est présente, est un pointeur vers la table des symboles qui contient des informations supplémentaires sur le jeton. »<sup>91</sup> [9] ;
- analyse syntaxique : [Traduction libre] « L'analyseur syntaxique utilise les premières composantes des jetons produits par l'analyseur lexical pour créer une

91. « A token consists of two components, a token name and an attribute value. The token names are abstract symbols that are used by the parser [...] The attribute value, if present, is a pointer to the symbol table that contains additional information about the token. »

### 3.4. DES EXPLICATIONS SUR L'IMPLÉMENTATION D'UNE SOLUTION

représentation intermédiaire arborescente qui décrit la structure grammaticale du flux de jetons. » <sup>92</sup> [9] ;

- arbre syntaxique : [Traduction libre] « Les arbres syntaxiques sont une forme de représentation intermédiaire ; ils sont couramment utilisés lors de l'analyse syntaxique et sémantique. » <sup>93</sup> [9] ;
- grammaire algébrique avec règles sémantiques (*syntax-directed definition*) : [Traduction libre] « une grammaire hors contexte est une grammaire dotée d'attributs et de règles. Les attributs sont associés aux symboles grammaticaux et les règles aux productions. » <sup>94</sup> [9] ;
- analyse sémantique : [Traduction libre] « L'analyseur sémantique utilise l'arbre syntaxique et les informations de la table des symboles pour vérifier la cohérence sémantique du programme source avec la définition du langage. Il collecte également les informations sur les types et les enregistre dans l'arbre syntaxique ou la table des symboles, pour une utilisation ultérieure lors de la génération du code intermédiaire. » <sup>95</sup> [9] ;
- représentation intermédiaire : représentation compréhensible par une machine, c'est-à-dire interprétable ou exécutable.

Les principaux processus de l'IAR forment aussi un enchaînement qui peut ressembler à celui d'un compilateur. Dick Grune dans *Modern compiler design* donne un aperçu de la manière qu'un compilateur construit successivement les relations :

[Traduction libre] L'analyse lexicale établit des relations locales entre les caractères, l'analyse syntaxique établit des relations d'imbrication entre

---

92. « *The parser uses the first components of the tokens produced by the lexical analyzer to create a tree-like intermediate representation that depicts the grammatical structure of the token stream.* »

93. « *Syntax trees are a form of intermediate representation ; they are commonly used during syntax and semantic analysis.* »

94. « *is a context-free grammar together with attributes and rules. Attributes are associated with grammar symbols and rules are associated with productions.* »

95. « *The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition. It also gathers type information and saves it in either the syntax tree or the symbol table, for subsequent use during intermediate-code generation.* »

### 3.4. DES EXPLICATIONS SUR L'IMPLÉMENTATION D'UNE SOLUTION

les jetons et la gestion du contexte établit des relations à longue portée entre les nœuds de l'AST.<sup>96</sup> [111]

Pour l'IAR, les ressemblances sont les suivantes :

- le processus **Épurer** conserve les éléments importants, comme l'analyse lexicale ;
- le processus **Formaliser** établit les liens entre les éléments et vérifie ces liens comme le ferait une analyse syntaxique et sémantique ;
- le processus **Présenter** crée une représentation intermédiaire dans ce cas pour un individu.

Pour faciliter la compréhension des techniques employées, voici quelques descriptions :

- EBNF : langage couramment utilisée pour spécifier une grammaire algébrique et pouvant servir à spécifier une grammaire rationnelle ;
- ANTLR : [Traduction libre] « À partir d'une description formelle du langage appelée grammaire, ANTLR génère un analyseur syntaxique pour ce langage qui peut construire automatiquement des arbres d'analyse syntaxique [...] génère également automatiquement des parcours d'arbres que vous pouvez utiliser pour visiter les nœuds de ces arbres afin d'exécuter du code spécifique à l'application. »<sup>97</sup> [297] ;
- la fonctionnalité **action** : [Traduction libre] « [...] sont des blocs de texte écrits dans la langue cible et encadrés par des accolades. Le système de reconnaissance les déclenche en fonction de leur position dans la grammaire. »<sup>98</sup> [297] ;
- table de symboles : [Traduction libre] « [...] sont des structures de données qui contiennent des informations sur les identificateurs. Ces informations sont

---

96.

*Lexical analysis establishes local relationships between characters, syntax analysis establishes nesting relationships between tokens, and context handling establishes long-range relationships between AST nodes.*

97. « From a formal language description called a grammar, ANTLR generates a parser for that language that can automatically build parse trees [...] also automatically generates tree walkers that you can use to visit the nodes of those trees to execute application-specific code. »

98. « [...] are blocks of text written in the target language and enclosed in curly braces. The recognizer triggers them according to their locations within the grammar. »

### 3.4. DES EXPLICATIONS SUR L'IMPLÉMENTATION D'UNE SOLUTION

insérées dans la table des symboles lors de l'analyse de la déclaration d'un identificateur. »<sup>99</sup> [9].

L'utilisation de plusieurs instruments permet de faciliter la mise en place d'un compilateur. Le premier instrument est le langage dénommé la forme étendue de Backus-Naur (EBNF : *Extended Backus-Naur form* en anglais). Ce langage permet de décrire un langage formel. Cet instrument se combine avec un autre instrument dont le nom est **un autre outil pour la reconnaissance du langage** (ANTLR : *ANother Tool for Language Recognition* en anglais). Les avantages de l'ANTLR sont :

- création automatique d'un analyseur lexical et d'un analyseur syntaxique ;
- mise à disposition d'un API en Java (il s'agit d'ailleurs du langage dans lequel la preuve de concept est écrite ;
- génération d'analyseurs pour d'autres langages (comme C#, C++ et JavaScript) qui procure une plus grande interopérabilité.

Un problème subsiste avec l'ANTLR : il ne permet pas de générer un analyseur sémantique. Cependant, il donne des moyens pour parcourir l'arbre syntaxique concret. Pour arriver à créer un analyseur sémantique, il faudrait être en mesure d'exprimer les règles sémantiques (règles de vérification et règles de traduction), et ce, en considérant le contexte (lexical et syntaxique). L'ANTLR propose comme moyens l'implémentation d'une classe *Visitor* ou d'un *Listener* en Java qui requiert des connaissances en Java et des modifications lorsqu'un changement est apporté à la grammaire. Pour surmonter ce problème, la fonctionnalité d'action d'ANTLR a été utilisée de manière à faire la traduction et à conserver le contexte à l'intérieur d'une table de symboles. Tandis que les règles de vérifications sont contenues dans le modèle intermédiaire et exécuté lors du processus **1.5.1 Traduire**. Ainsi, le résultat obtenu est que les aspects sémantiques sont directement intégrés dans l'analyseur syntaxique et cela demeure transparent pour l'utilisateur, car l'instrument s'occupe de cette gestion.

Cette stratégie de réutiliser le même enchaînement de processus pour arriver à extraire les informations d'un texte s'applique à l'IAR à deux moments :

---

99. « [...] are data structures that hold information about identifiers. Information is put into the symbol table when the declaration of an identifier is analyzed. »

### 3.4. DES EXPLICATIONS SUR L'IMPLÉMENTATION D'UNE SOLUTION

- lors de l'épuration, il est possible de décrire un langage qui va servir à éliminer les éléments inutiles ou à étendre des éléments essentiels cachés par l'usage de fonctions macro ou micro ;
- lors de la formalisation et, plus précisément, de la traduction où il est nécessaire de décrire un langage qui va servir à extraire l'information pour en faire un modèle.

Cette stratégie peut également s'appliquer à deux processus : le processus **1.6 Présenter** (figure 3.2) et le processus **1.5.2 Vérifier partiellement** (figure 3.3). Toutefois, pour la preuve de concept, il a été décidé d'utiliser des langages existants venant avec leurs instruments. Cela a permis d'épargner du temps<sup>100</sup> :

- lors de la vérification partielle, il est possible de décrire un langage qui va servir à décrire des règles à appliquer au modèle. L'instrument Easy-Rules a été préféré, car il permet de décrire les règles en YAML. De plus, il vient avec un modèle logique minimal intégré avec Java permettant de vérifier le modèle intermédiaire (voir la figure 3.3) ;
- lors de la présentation, il est possible de décrire un langage qui va servir à décrire des règles de présentation à appliquer au modèle. Pour la présentation, l'instrument *StringTemplate* proposé par l'auteur d'ANTLR a été préféré, car il s'agit d'un langage avec une syntaxe répandue. Même s'il est similaire dans ses fonctions à Qute, la décision a été de conserver les deux. Qute provient de l'écosystème Quarkus de RedHat, qui a été utilisé dans cette preuve de concept. L'effort nécessaire pour ne conserver qu'un seul des deux instruments tout en conservant les intégrations existantes n'était pas efficient à ce moment de la preuve de concept.

---

100. Dans la preuve de concept, le code source est encore là.

### 3.4. DES EXPLICATIONS SUR L'IMPLÉMENTATION D'UNE SOLUTION

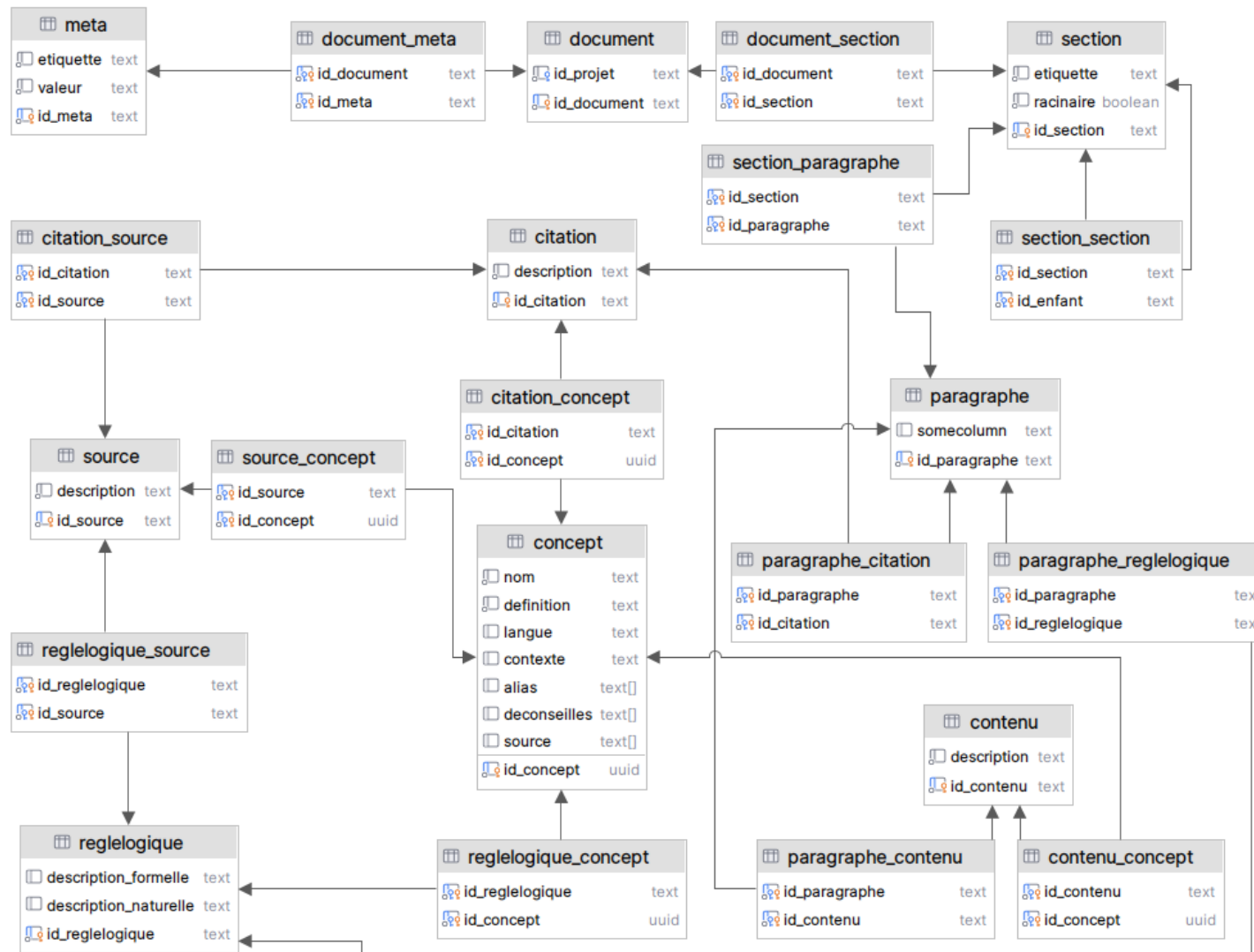


FIGURE 3.6 – Diagramme relationnel du métamodèle de document

### 3.5. DES EXIGENCES À SATISFAIRE POUR LA SOLUTION

Pour rendre ce modèle de document calculable, l'utilisation d'un langage et d'un système logique est nécessaire. Le système logique utilisé dans la preuve de concept est celui qui provient du langage de spécification Alloy. Il est écrit en Java alors, il s'intègre facilement dans la preuve de concept. Néanmoins, cela impose pour le moment l'utilisation de ce langage pour écrire les énoncés logiques contenus dans le modèle. Aussi, le métamodèle définit formellement le modèle de document et le rend en partie calculable. La figure 3.6 présente la concrétisation du métamodèle dans la preuve de concept. Elle utilise le langage d'un diagramme relationnel. Plusieurs observations sont faites sur ce diagramme :

- a) Les éléments sont des informations, car ils contiennent que des identifiants et des descriptions en texte.
- b) Les identifiants sont des références absolues, puisqu'une même information peut se retrouver dans plusieurs modèles apportés par un même document ou non.
- c) Les métainformations permettent de conserver des informations sur le contexte des documents et d'un modèle.
- d) L'ensemble des règles d'un même modèle se retrouve dans les énoncés logiques.
- e) Grâce à tous les énoncés logiques décrits formellement, il est possible d'utiliser un système logique pour découvrir des incohérences.
- f) Dans le cas d'une combinaison d'énoncés logiques décrits formellement et d'autres non, il faut réviser l'ensemble pour déterminer les incohérences.
- g) Les énoncés logiques qui ne sont pas formellement décrits peuvent l'être plus tard.

## 3.5 Des exigences à satisfaire pour la solution

Par usage et facilité, les règles du modèle du problème sont restreintes aux besoins, et les règles du modèle d'une solution sont restreintes aux exigences. La couverture des besoins par les exigences est l'une des caractéristiques de qualité que la solution doit respecter. Le tableau de correspondance (tableau 3.1) à la fin de cette sous-section sert à vérifier cette caractéristique. Ce tableau permet aussi d'illustrer que ce qui



### 3.5. DES EXIGENCES À SATISFAIRE POUR LA SOLUTION

n'est pas formalisé n'est pas pris en charge par le système, mais par les utilisateurs du système. Mais avant, voici une liste qui présente des exigences futures donnant un aperçu des développements possibles à l'IAR.

L'effort, c'est-à-dire le temps, consacré à la preuve de concept est limité. Dans l'état actuel, il ne serait pas surprenant que certaines caractéristiques de qualité relatives ou absolues ne soient pas respectées. Pour améliorer la solution, la liste d'exigences supplémentaires suivante pourrait être envisagée :

- EX16 : Le système doit permettre à l'utilisateur d'ajouter, de modifier ou de retirer des descriptions formelles.
  - Dans la preuve de concept, la majorité des descriptions formelles est disposée à l'intérieur d'un dossier accessible uniquement par le configurateur.
- EX17 : Le système doit permettre à l'utilisateur de récupérer directement les descriptions proposées d'un modèle.
  - Dans la preuve de concept, tout se joue au niveau de la session pour l'utilisateur. Il récupère une session et, du même coup, la description proposée d'un modèle et le modèle développé de cette session.
- EX18 : Le système doit permettre à l'utilisateur de récupérer directement un modèle développé.
  - Par exemple, la preuve de concept ne permet pas, depuis une interface utilisateur de recherche, d'ajouter à une session une description proposée ou un modèle développé.
- EX19 : Le système doit permettre de relier un modèle développé à un autre modèle développé.
  - L'accès aux informations d'un précédent modèle pour en faire des sources <sup>101</sup> dans un nouveau modèle pendant une session nécessite des changements à l'interface utilisateur. Ensuite, il sera possible de construire un système de références et, par la suite, de connaître toutes les répercussions après avoir effectué un changement à un modèle.

---

101. Chaque information contenue dans un document peut provenir d'une source. Cette source servira à créer un système de références entre des modèles.

### 3.5. DES EXIGENCES À SATISFAIRE POUR LA SOLUTION

- Chaque information n’ayant pas de provenance (source) devrait faire l’objet d’une attention particulière.
- EX20 : Le système doit permettre la réutilisation d’une même information pour plusieurs modèles développés.
  - Il faut s’assurer que l’information est la même et qu’il n’y ait pas de duplicata selon la sémantique. Car, si la détection reposant sur la syntaxe est simple, celle des équivalences, et même, des simples variations lexicales ou de présentation, n’est pas simple. Le changement à l’intérieur d’une information doit aussi relancer les vérifications et les validations pour l’ensemble des modèles l’utilisant. Il suffit de penser à un prédicat non formel dont le texte a changé.
- EX21 : Le système doit permettre à l’utilisateur de définir les métamodèles incarnés par le système.
  - Dans la preuve de concept, le métamodèle est décrit une fois avec du code java et une fois avec du code SQL. Un métamétamodèle est nécessaire pour permettre à l’utilisateur de faire des modifications. L’instanciation en métamodèle du métamétamodèle conduirait à générer du code dynamique avec les désavantages que cela peut occasionner. Parmi les désavantages, il y a la sécurité et la fiabilité plus faible, puisqu’il est permis de réécrire certaines parties du logiciel. Aussi, la plupart des instruments ne sont pas conçus pour être employés de cette façon, comme les éditeurs de code source ou les gestionnaires de versions.
- EX22 : Le système doit permettre des changements aux métamodèles incarnés par le système.
  - Pour tout changement, le système doit s’assurer que chaque partie utilisant précédemment ces métamodèles continue de fonctionner.
- EX23 : Le système doit fournir des vues pour réviser certaines informations provenant de différents modèles.
  - Advenant que le dépôt interne **Modèle vérifié** devienne un agent, ces vues pourraient se réaliser à l’aider d’un autre instrument.

### 3.5. DES EXIGENCES À SATISFAIRE POUR LA SOLUTION

- EX24 : Le système doit permettre à l'utilisateur d'appliquer plusieurs systèmes logiques à un même modèle.
  - Il existe des systèmes logiques différents qui utilisent un même langage formel et qui peuvent arriver à des résultats différents. Pour le moment, cela empêche d'intégrer des modèles en monde ouvert<sup>102</sup> pour avoir des logiques multivaluées, des logiques floues et des logiques probabilistes.
  - Il serait préférable de restreindre les relations entre des modèles reposant sur un même système logique.

---

102. « En logique formelle, l'hypothèse du monde ouvert est la supposition selon laquelle la véracité d'une affirmation ne dépend pas de la connaissance d'un agent ou d'un observateur. En d'autres termes, "ce n'est pas parce qu'on ne connaît pas une information que cette information est fausse." » [Wikipedia, « Hypothèse du monde ouvert », [https://fr.wikipedia.org/wiki/Hypoth%C3%A8se\\_du\\_monde\\_ouvert](https://fr.wikipedia.org/wiki/Hypoth%C3%A8se_du_monde_ouvert) (consulté le 2026-05-10)]

### 3.5. DES EXIGENCES À SATISFAIRE POUR LA SOLUTION

**TABLEAU 3.1** – Correspondances des besoins et des exigences

|      |                                                                                                        |                                                                                                         |                                                                                                                   |                                                                      |                                                                                                                                         |
|------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| EX09 | Oui, l'EAMDD nécessite de comprendre les intrants avec différents formalismes et à différents niveaux. | Oui, la CNMD peut utiliser un élément de configuration portant sur un modèle système.                   | Oui, l'ICAD porte sur des documents.                                                                              | Non, l'ADM pourrait se faire avec des transformations non formelles. | Oui, l'EQMD peut comprendre le résultat des différentes vérifications utilisées dans le procédé pour obtenir un modèle.                 |
| EX10 | Oui, l'EAMDD permet de former le modèle développé à présenter.                                         | Oui, la CNMD utilisant des précédents modèles développés, la présentation de ceux-ci peut-être utile.   | Non, l'ICAD ne nécessite pas de présenter le nouveau modèle développé.                                            | Non, l'ADM n'a pas de lien avec la présentation du modèle.           | Non, l'EQMD est objective tandis que la présentation d'un modèle permet à l'utilisateur de l'apprécier que sur des critères subjectifs. |
| EX11 | Oui, l'EAMDD doit considérer le contexte des intrants pour tenter de conserver la sémantique.          | Oui, la CNMD crée un nouveau modèle cela comprend un contexte.                                          | Non, l'ICAD devrait conserver le même contexte pour les documents.                                                | Oui, l'ADM nécessite de connaître le contexte.                       | Non, l'EQMD prend pour acquis que le contexte est déjà considéré.                                                                       |
| EX12 | Oui, l'EAMDD permet le traitement d'informations.                                                      | Non, la CNMD arrive avant d'en permettre l'évaluation.                                                  | Oui, l'ICAD montre les impacts d'un changement dont décrit indirectement la couverture et l'adaptativité.         | Non, l'ADM pourrait ne pas exister que l'évaluation serait possible. | Oui, l'EQMD comprend les caractéristiques de qualités portant sur la couverture, l'efficacité et l'adaptativité.                        |
| EX13 | Oui, l'EAMDD peut devenir plus difficile lorsqu'il y a la présence d'informations superflue.           | Non, la CNMD peut ne pas faire intervenir des documents.                                                | Non, l'ICAD n'est pas impossible avec des informations superflues.                                                | Non, l'ADM du modèle n'est pas celle des intrants.                   | Non, l'EQMD peut être réalisable, peu importe les transformations précédentes.                                                          |
| EX14 | Oui, l'EAMDD étant une étape du procédé, il apparaît au processus.                                     | Oui, la CNMD fait partie du procédé, il va s'y retrouver journalisé.                                    | Oui, l'ICAD fait partie du procédé, il va s'y retrouver journalisé.                                               | Oui, les l'ADM peuvent se retrouver à plusieurs endroits du procédé. | Non, l'EQMD repose d'abord sur l'utilisateur du système.                                                                                |
| EX15 | Oui, l'EAMDD doit considérer le contexte de l'activité pour tenter de conserver la sémantique.         | Oui, la CNMD récupère le contexte provenant des précédents modèles développés pour en faire un nouveau. | Oui, l'ICAD arrive lors de changements dans une nouvelle session, il est probablement que le contexte ait changé. | Oui, l'ADM doit considérer le contexte dans lequel, il est réalisé.  | Non, l'EQMD prend pour acquis que le contexte est déjà considéré.                                                                       |

<sup>a</sup> Les cases grises indiquent qu'une exigence ne couvre pas un besoin.

<sup>b</sup> Raccourci vers les [besoins 1 à 5](#), les [exigences 1 à 8](#)

### 3.5. DES EXIGENCES À SATISFAIRE POUR LA SOLUTION

**TABEAU 3.1 – Correspondances des besoins et des exigences (suite)**

| Exigence<br>Besoin | BE01 : Permettre l'extraction des attributs des modèles développés depuis les documents (EAMDD) qui les décrivent. | BE02 : Permettre la création de nouveaux modèles développés (CNMD) depuis l'usage des modèles systèmes et des modèles développés. | BE03: Permettre d'identifier les changements apportés aux documents (ICAD) pour déterminer au mieux les répercussions potentielles aux modèles développés. | BE04 : Permettre l'adaptation des modèles développés (AMD) pour différents contextes ou systèmes.                        | BE05 : Permettre l'évaluation de la qualité d'un modèle développé (EQMD), les caractéristiques de qualité étant définies par un modèle système. |
|--------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| EX01               | Oui, l'EAMDD fait partie du procédé de création et de modification des modèles développés.                         | Oui, la CNMD est incluse dans la création des modèles.                                                                            | Oui, l'ICAD est possible si est seulement s'il est possible de modifier les modèles développés.                                                            | Oui, l'ADM est comprise comme une modification.                                                                          | Oui, l'EQMD est possible que si un modèle développé est ajouté au système.                                                                      |
| EX02               | Oui, l'EAMDD est nécessaire pour importer le modèle dans le système.                                               | Oui, la CNMD comprend l'importation et l'exportation.                                                                             | Oui, l'ICAD peuvent provenir d'une importation.                                                                                                            | Oui, l'ADM en facilite l'importation ou l'exportation.                                                                   | Non, l'EQMD est possible même sans importation ou exportation de modèles.                                                                       |
| EX03               | Oui, l'EAMDD permet la réutilisation d'un modèle.                                                                  | Oui, la CNMD peut consister à utiliser des modèles développés.                                                                    | Oui, la CNMD peut consister à utiliser des modèles développés.                                                                                             | Oui, l'ADM facilite la réutilisation.                                                                                    | Oui, l'EQMD d'un modèle permet de choisir celui à réutiliser.                                                                                   |
| EX04               | Oui, l'EAMDD est préalable à toute comparaison des modèles.                                                        | Non, la CNMD n'a pas à comparer les modèles.                                                                                      | Oui, l'ICAD ont un impact sur les comparaisons.                                                                                                            | Oui, l'ADM peut aider dans sa comparaison                                                                                | Oui, l'EQMD peut se faire entre différents modèles.                                                                                             |
| EX05               | Oui, l'EAMDD des attributs d'un modèle nécessite la compréhension de la structure d'un modèle.                     | Oui, la CNMD repose la structure d'un modèle.                                                                                     | Oui, l'ICAD porte sur un modèle dont sa structure.                                                                                                         | Oui, l'ADM nécessite une structure                                                                                       | Oui, l'EQMD porte sur un modèle dont sa structure.                                                                                              |
| EX06               | Oui, l'EAMDD doit remplir un métamodèle pour les documents.                                                        | Oui, la CNMD nécessite un métamodèle pour les modèles.                                                                            | Oui, l'ICAD nécessite un métamodèle pour les documents.                                                                                                    | Oui, l'ADM nécessite l'utilisation de métamodèles.                                                                       | Oui, l'EQMD nécessite un métamodèle pour les modèles et un modèle pour les caractéristiques de qualité.                                         |
| EX07               | Oui, l'EAMDD utilise le formalisme comme moyen pour comprendre les intrants.                                       | Oui, la CNMD ne doit pas dénaturer les autres modèles dans son usage.                                                             | Oui, l'ICAD est réalisable lorsque des déductions sont possibles depuis les opérations.                                                                    | Non, l'ADM pourrait se faire avec des transformations non formelles.                                                     | Non, l'EQMD peut être réalisable, peu importe les transformations précédentes.                                                                  |
| EX08               | Oui, l'EAMDD nécessite en intrants documents et configuration.                                                     | Oui, la CNMD nécessite l'emploi de modèles systèmes qui peuvent se retrouver dans la configuration.                               | Oui, l'ICAD porte sur des documents.                                                                                                                       | Oui, l'ADM n'utilise pas les documents servant au modèle, mais peut utiliser des intrants provenant de la configuration. | Oui, l'EQMD porte aussi sur les intrants.                                                                                                       |

<sup>a</sup> Les cases grises indiquent qu'une exigence ne couvre pas un besoin.

<sup>b</sup> Raccourci vers les [besoins 1 à 5](#), les [exigences 9 à 12](#) et les [exigence 13 à 15](#).

# Chapitre 4

## L'utilisation de la solution

L'informatique appliquée développe des solutions pour différents domaines et différents utilisateurs. Cette solution s'adresse au domaine de l'informatique appliquée et est destinée aux personnes qui développent des modèles. Ce chapitre expose des situations dans lesquelles il existe un intérêt à utiliser cette solution et poursuit en expliquant en détail la façon d'utiliser cette solution.

### 4.1 Applications selon différents contextes

Parmi toutes les situations où l'utilisation de la solution pourrait améliorer les artefacts, il y en a deux qui sont présentées :

- lors du développement d'un modèle faisant intervenir plusieurs participants avec des acquis distincts ;
- lors du maintien d'un modèle sur une longue période où le sujet, le traitement, le modélisateur ou le contexte peuvent varier.

Le développement d'un modèle faisant intervenir plusieurs participants nécessite l'utilisation d'un langage commun pour être en mesure de correctement communiquer. Cependant, l'utilisation de langages non formels peut conduire à davantage d'ambiguïtés. Par conséquent, les participants vont chercher à mieux définir et restreindre

#### 4.1. APPLICATIONS SELON DIFFÉRENTS CONTEXTES

le langage. Cela peut débuter par la création d'un glossaire pour ensuite se terminer avec une grammaire formelle. D'ailleurs, cela peut se faire de manière incrémentale et en parallèle avec le développement d'un modèle. L'instrument d'aide à la rédaction (IAR) propose d'en faire un procédé et de l'appliquer à la documentation décrivant un modèle. Pour ce faire, elle offre d'utiliser des langages formels et des systèmes logiques grâce aux instruments informatiques suivants : la base de données relationnelle (PostgreSQL), le langage de grammaires (EBNF), le générateur d'analyseurs (ANTLR), le système à règles (Easy-Rules), le système logique (Alloy) et le langage de présentation (StringTemplate). Pendant le développement d'un modèle, l'utilisateur a le choix d'utiliser ces instruments informatiques et ceux-ci peuvent être adaptés au procédé. Un exemple serait l'ajout d'un glossaire à la base de données, puis la restriction des documents à l'utilisation de ce glossaire.

Les logiciels et, en particulier, leurs modèles exprimés par des documents et du code source sont voués à être adaptés ou abandonnés. Les connaissances relatives aux documents ou au code source qui concrétisent des modèles peuvent disparaître, comme c'est souvent le cas avec les logiciels patrimoniaux. Pour le code source, il est possible de faire des vérifications avec la compilation et les tests (dans la mesure où les tests ont été développés et maintenus à un niveau de couverture suffisant). Cela aidera au maintien de la cohérence et à la compréhension d'un modèle par un nouvel individu. Cependant, pour les documents représentant des modèles, il existe rarement de telles vérifications. L'IAR cherche à fournir aux modélisateurs ces vérifications, en offrant dans un premier temps un procédé semblable à la compilation du code source, puis dans un deuxième temps, un environnement permettant de créer des modèles servant à la vérification d'autres modèles<sup>103</sup>. Ce deuxième temps nécessite le respect des exigences EX16 à EX24 qui devront éventuellement être remplies dans le prolongement de la preuve de concept.

---

103. Il s'agit du concept de vérification de modèles (*Model checking, en anglais*).

## 4.2. EXPLICATIONS SUR LE FONCTIONNEMENT

### 4.2 Explications sur le fonctionnement

Les explications consistent à décrire un exemple hypothétique de l'utilisation de l'IAR inspiré de la première situation énumérée au début du chapitre. Plus précisément, il s'agit de donner des exemples d'intrants et d'extrants de la figure 3.1 Diagramme de contexte de l'IAR. Aussi, des exemples des intrants et des extrants sont donnés en annexe D.

Dans ce scénario, une équipe de développement décentralisé s'emploie à réaliser un logiciel. Avant cela, plusieurs membres de l'équipe ont décrit à l'intérieur d'un document un modèle en utilisant l'IAR. Ce modèle est décrit à l'aide d'un langage non formel et d'un langage formel. Un nouveau membre de l'équipe est chargé d'inclure de nouvelles contraintes à ce modèle sous la forme de règles logiques.

Bien avant l'arrivée de ce nouveau membre, l'équipe s'est dotée de standards sur la documentation représentant des modèles. Le respect de ces standards passe par des règles qui permettent d'extraire, de transformer ou de contraindre. Les membres de l'équipe ont dû décrire ces règles en utilisant des descriptions formelles qu'ils ont ajoutées à l'intérieur de la **Configuration** de l'IAR. Sommairement, les standards consistent à :

ST01 : le langage utilisé pour les documents est un sous-ensemble du langage balisé **Asciidoc**<sup>104 105</sup> ;

ST02 : le langage formel utilisé pour décrire les règles logiques du modèle est **Alloy**<sup>106</sup> ;

ST03 : les documents contiennent notamment un identifiant, un titre, une section et des règles logiques ;

ST04 : il est possible d'utiliser certaines macros comme des conditions à l'intérieur des documents ;

ST05 : la présentation du modèle exprimée en **Asciidoc** comprend avant tout les méta-informations, puis les sections.

---

104. <https://asciidoc.org>

105. À ce jour, les développeurs du langage n'offrent pas une description formelle en EBNF <https://gitlab.eclipse.org/eclipse/asciidoc-lang/asciidoc-lang>.

106. <https://alloytools.org>



## 4.2. EXPLICATIONS SUR LE FONCTIONNEMENT

Ainsi, la configuration des fichiers contient deux grammaires sous la forme EBNF, des règles Easy-Rules, des règles de présentations StringTemplate <sup>107</sup>.

Le nouveau membre récupérera le modèle et cela conduira à l'interface utilisateur (figure 4.1) où cinq boîtes (zones textuelles dynamiques) sont identifiées (A, B, C, D, E) pour faciliter la description du fonctionnement. Dans cette interface, il trouvera la description précédente du modèle ainsi que l'ensemble des règles sélectionnées pour exécuter l'IAR. Dans la preuve de concept, l'ensemble des règles sélectionnées pour exécuter l'IAR prend la forme d'un JSON (boîte A). Ce JSON est fourni par l'Utilisateur. Il pourrait le modifier pour sélectionner de nouvelles règles à appliquer, mais ce n'est pas vraiment souhaité dans ce cas. Néanmoins, en lisant le fichier, il en apprend davantage sur l'ordre et sur le fonctionnement de l'exécution de l'IAR. Puisqu'il n'est pas familier avec les règles sélectionnées, il décide d'aller lire leurs descriptions dans la configuration.

Ensuite, le nouveau membre exécute l'IAR pour obtenir le **Journal** (boîte E) et une **Représentation standardisée du modèle** (boîte C). Avec ces informations, il peut lire, tenter de comprendre le modèle, puis le modifier. Toutefois, il hésite à modifier la précédente **Description proposée du modèle** (boîte B) <sup>108</sup>. Il décide plutôt d'écrire les nouvelles règles logiques dans un autre document (boîte D). Ensemble, ces deux éléments deviendront la nouvelle **Description proposée du modèle**. Il peut maintenant exécuter l'IAR et observer dans le journal que la nouvelle description proposée du modèle respecte les différentes règles sélectionnées et par conséquent, le standard de l'équipe.

Malgré tout, il observe dans le journal que l'exécution a trouvé des incohérences. C'est que l'IAR a extrait les règles logiques qu'il avait écrites en langage formel pour ensuite les vérifier à l'aide d'un système logique. Le nouveau membre, en prenant conscience de son erreur, décide d'en discuter avec un autre membre de l'équipe. Ce

---

107. Pour le moment, seuls le langage et le système logique Alloy peuvent être utilisés pour les règles formelles.

108. Les boîtes B et D sont des entrées à l'exécution, la précédente **Description proposée du modèle** aurait pu se retrouver en D ou séparer en deux documents en B et D.

## 4.2. EXPLICATIONS SUR LE FONCTIONNEMENT

faisant, il prend conscience de sa compréhension erronée et corrige la règle logique problématique.

## 4.2. EXPLICATIONS SUR LE FONCTIONNEMENT

Session ID :  Début : 2026-08-19T12:49:59

Session titre :  Fin : 2026-08-19T12:49:59

Transformation :

```
[
  {
    "etape": "epure",
    "transformation": "AsciidocMacro",
    "intran": "description1",
    "extran": "epure1"
  },
  {
    "etape": "epure",
    "transformation": "AsciidocMacro",
```

**exécute** **conserve**

| description 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | présente                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>:projetid: projetdescription1 [b12c7bf9-3232-46cd-8a4c-de838b23e673] :documentid: DocumentModelev1 :var1: vrai  = DocumentModele_v1  == SectionA [4edd432a-05fa-4fb3-95d6-c58acec01f1b] Voici un modèle. ifdef::var1[] une utilisation de macro var1 := {var1} endif::var1[]  === SectionAa [9849d016-d120-4f54-b593-bb110cbc7ac0] [quote] Une citation. ----- Règle(s) informelle(s) .... RegleTest=error .... ~~~~~  === SectionAb [Secalla7-6a46-4cb3-832e-dal316a02270] Règle(s) formelle(s) .... b=10</pre> | <pre>:documentid: DocumentModelev1 [6458005a-ce4f-40d1-bba6-c07e09ca499b] :var1: vrai [d15e1881-ba35-4e90-8ad8-32935clefc4] :projetid: projetdescription1 [b12c7bf9-3232-46cd-8a4c-de838b23e673]  = DocumentModelev1  == SectionA[4edd432a-05fa-4fb3-95d6-c58acec01f1b]Voici un modèle. une utilisation de macro var1 := vrai  === SectionAb[Secalla7-6a46-4cb3-832e-dal316a02270]Règle(s) formelle(s) .... b=10 .... a=10 ....  === SectionAa[9849d016-d120-4f54-b593-bb110cbc7ac0]Règle(s) informelle(s) .... RegleTest=error [quote] Une citation. -----</pre>                                                                                                                                                                                       |
| <p>description 2</p> <pre>:projetid: projetdescription1 [b12c7bf9-3232-46cd-8a4c-de838b23e673] :documentid: DocumentModelev1  = DocumentModele_v1  == SectionA [4edd432a-05fa-4fb3-95d6-c58acec01f1b] Voici un modèle.  === SectionAa [9849d016-d120-4f54-b593-bb110cbc7ac0] [quote] Une citation. ----- Règle(s) informelle(s) .... RegleTest=bien ....</pre>                                                                                                                                                        | <p>journal</p> <pre>....  === SectionB [9ecalla7-6a46-4cb3-832e-dal316a02270] Nouvelle section. ....  2026-08-19T12:49:59.47 - INFO - Fin du traitement de l'épuration. 2026-08-19T12:49:59.47 - INFO - Début du traitement de formalisation. 2026-08-19T12:49:59.47 - INFO - Début de la traduction. 2026-08-19T12:49:59.47 - TRACE - Transformation (identifiant):Asciidoc 2026-08-19T12:49:59.47 - TRACE - Intran 1 (nbr de caractère):497 2026-08-19T12:49:59.47 - TRACE - Extran 1 (identifiant):= DocumentModele_v1 2026-08-19T12:49:59.47 - INFO - Fin de la traduction. 2026-08-19T12:49:59.47 - INFO - Debut de la vérification partielle et ajustement. 2026-08-19T12:49:59.47 - INFO - Fin de la vérification partielle et ajustement.</pre> |

<sup>1</sup> La signification des boîtes est : l'ensemble des règles sélectionnées pour exécuter l'IAR (A), la précédente description proposée (peut être B et D), la représentation standardisée du modèle (C), la nouvelle description proposée du modèle (peut être B et D) et le journal (E).

**FIGURE 4.1** – Une interface utilisateur de la preuve de concept

## 4.2. EXPLICATIONS SUR LE FONCTIONNEMENT

Cependant, certaines des nouvelles règles logiques sont exprimées en langage non formel. Le nouveau membre doit suivre ensuite le procédé de développement qui consiste à faire valider l'ensemble des règles du modèle par un autre membre de l'équipe ayant un rôle d'approbateur. L'IAR aide en ce sens, puisqu'il est en mesure de fournir une présentation du modèle approprié pour ce rôle avec les règles de présentation à sa disposition. L'approbateur, quant à lui, ne soulève aucune autre problématique et inscrit les nouvelles règles dans une méta-information du modèle.

Pour finir, il est obtenu un modèle qui n'est pas totalement vérifié logiquement, mais qui pourrait l'être éventuellement. Il suffit d'ajouter un effort supplémentaire dans les itérations suivantes pour réécrire formellement les règles logiques. D'ailleurs, ce type de mandat pourrait être confié à un expert. Néanmoins, l'IAR a permis à ce nouveau membre d'intégrer des langages formels et de comprendre les standards employés par l'équipe.

# Chapitre 5

## L'évaluation de la solution

En informatique appliquée, l'implémentation, qui est une concrétisation d'un modèle de solution, peut se vérifier avec des tests dérivés du modèle développé. Une bonne conception des tests permet de vérifier la cohérence, la couverture et l'efficacité de la solution. Cependant, cela ne permet pas d'affirmer qu'une solution informatique est une bonne solution en situation réelle. Il faut valider la solution dans la réalité de façon à ce que l'évaluation devienne plus complète. Dans le contexte d'une preuve de concept, la portée de l'évaluation a été volontairement réduite à s'assurer de la validité du modèle, de déterminer la faisabilité d'une solution qui en découlerait, de confirmer la pertinence de cette solution et d'envisager les ressources requises pour la réalisation d'un produit correspondant. Cette évaluation de l'instrument d'aide à la rédaction (IAR) comprend deux sections : la première section porte sur la vérification du respect des exigences avec des cas de tests, tandis que la deuxième section prend la forme d'un guide qui explique la manière de valider rigoureusement certaines attentes.

### 5.1 La vérification du respect des exigences

La description des tests et la manière de conduire les essais sur un logiciel se retrouvent couramment dans un document nommé spécification des essais. C'est pour cette raison que cette section comprend des descriptions portant sur les mêmes sujets : l'environnement, les cas de tests et la couverture des exigences. Puisqu'il existe

## 5.1. LA VÉRIFICATION DU RESPECT DES EXIGENCES

une variabilité dans la définition des cas de tests, voici celle provenant d'un ouvrage portant sur l'International Software Testing Qualifications Board (ISTQB) :

[Traduction libre] Un ensemble de préconditions, d'entrées, d'actions (le cas échéant), de résultats attendus et de postconditions, élaboré en fonction des conditions de test.<sup>109</sup> [108]

Parmi les conditions importantes pour avoir des cas de tests efficaces, il y a un procédé clair et réalisable ainsi qu'une couverture complète des exigences. Relativement à l'exécution, il y a trois composantes importantes :

- le protocole d'exécution (commun à toutes les séances) ;
- le journal d'exécution (propre à chacune des séances) ;
- le résultat de l'exécution (propre à chacune des séances).

Ce mémoire présente uniquement les résultats afin d'en limiter le volume.

L'environnement d'essai doit être en mesure de faire fonctionner la solution. L'environnement utilisé dans le cadre du présent mémoire comprend :

- des composantes matérielles :
  - une unité centrale de traitement AMD Ryzen 7 7840HS à 5,1 GHz avec 8 cœurs ;
  - de la mémoire primaire à hauteur de 16 Go ;
  - de la mémoire secondaire à hauteur de 500 Go ;
- des composantes logicielles :
  - un système d'exploitation Ubuntu (version 24.04) ;
  - une machine virtuelle Java Temurin (version 21) ;
  - une base de données PostgreSQL (version 14.2) ;

---

109.

*A set of preconditions, inputs, actions (where applicable), expected results and postconditions, developed based on test conditions.*

## 5.1. LA VÉRIFICATION DU RESPECT DES EXIGENCES

- les composantes de la solution (les informations suivantes sont fournies pour chaque composante : nom <sup>110</sup>, rôle, position dans le diagramme de flux de données (DFD) du système IAR [figure 3.2] et les instruments inclus) <sup>111</sup> :
  - **editor-web-library** est la composante web principale et exprime une partie du processus 1.1 Diriger. Cette composante inclut les instruments Ace Editor et Ace Playground ;
  - **editor-ldm** est la composante persistante et exprime le modèle vérifié. Cette composante inclut les instruments SQL, PostgreSQL, qui sont respectivement un langage et une machine abstraite ;
  - **editor-application** est la composante exécutable principale et exprime les parties Session, Description proposée épurée d'un modèle, les processus 1.2 Résoudre, 1.3 Journaliser, 1.4 Épurer, 1.5 Formaliser, 1.6 Présenter ainsi qu'une partie du processus 1.1 Diriger. Cette composante inclut les instruments Java, Quarkus, Qute, ANTLR, StringTemplate, Easy-rule et Alloy.

Ces composantes peuvent être différentes, mais resteront constantes durant le déroulement des essais. Les explications pour l'installation et la préparation se trouvent dans les documents accompagnant la preuve de concept.

La stratégie employée est de décrire des cas de tests. Plus spécifiquement, la description d'un cas de tests débute avec une instance de problème en lien avec une ou plusieurs exigences. Par la suite, une procédure est décrite. Cette procédure répond de manière formelle au problème en utilisant le système ; elle a donc des antécédents et des conséquents. Ainsi, il y a un ou plusieurs prérequis, une procédure et une ou plusieurs conditions de réussite. Par la suite, ces cas de tests sont exécutés avec un ou plusieurs intrants lors d'une session de tests et les résultats (couverture) sont rapportés dans un tableau nommé Résultats des tests. En annexe D se trouvent des exemples d'intrants et d'extrants.

---

110. Les composantes ont comme qualificatif **editor** et ils forment ensemble l'IAR.

111. L'ensemble du code source de ces composantes se retrouve à cette adresse : <https://github.com/mbjolin/IAR>

## 5.1. LA VÉRIFICATION DU RESPECT DES EXIGENCES

Les intrants pour ces tests doivent être conçus de manière à atteindre une certaine couverture de la description formelle (code source). Plus précisément, il existe des couvertures sur différents éléments, comme les déclarations, les branches et les chemins [300]. Une couverture de 100 % sur les déclarations ou les branches permettrait de confirmer que tout le code source est utilisé, mais ne permet pas d'affirmer l'absence d'erreur. Pour une preuve de concept, une couverture sur les déclarations du moteur d'exécution <sup>112</sup> d'au moins 80 % <sup>113</sup> est raisonnable. Dans le cas d'un prototype ou d'un produit, il peut être souhaitable d'atteindre le plus haut niveau possible de couverture, et ce, autant sur les déclarations, les décisions et les conditions. Cependant, même avec une couverture de 100 %, il peut exister des erreurs, il faut s'en remettre aux différentes techniques de tests. Dans ce mémoire, la technique des cas de tests sert à détecter les inéquations entre les exigences et leurs implémentations [108], c'est-à-dire entre les deux modèles développés.

Plusieurs cas de tests emploient les mêmes rôles. Dans le cas de documents, les rôles existants sont l'autorat et le lectorat. Puisque l'IAR vise les modèles en logiciel, les rôles de modélisateur et d'approbateur sont plus appropriés. Le modélisateur crée ou modifie un modèle, tandis que l'approbateur valide des éléments du modèles que l'IAR n'assure pas.

CA01 : Vérifier que l'IAR permet la création ou la modification d'un modèle à partir d'une description proposée, ainsi que la journalisation du système, l'épuration de la description proposée et la présentation du modèle.

— Prérequis :

— une description proposée d'un modèle *description\_1* ;

---

112. La preuve de concept comprend l'interface utilisateur, l'utilisation du système de gestion de base de données et d'autres fonctionnalités, comme le début d'un atelier qui comporte, entre autres, l'acquisition, la vérification et l'application des éléments de configuration, dont la couverture importe moins pour le moment.

113. 75 % est qualifié de louable chez Google [Carlos Arguelles *et al.*, « Code Coverage Best Practices », <https://testing.googleblog.com/2020/08/code-coverage-best-practices.html> (consulté le 2026-03-31)] et l'ouvrage *Effective software testing : a developer's guide* cite plusieurs études où 90 % et plus a une utilité débattable pour trouver des erreurs [14].



## 5.1. LA VÉRIFICATION DU RESPECT DES EXIGENCES

- une description proposée d'un modèle *description\_2* modifiant certains éléments de *description\_1* ;
- une description formelle *formel\_1* fonctionnelle dans le processus 1.5.1 Traduire ;
- une description formelle *epure\_1* fonctionnelle épurant certains éléments dans le processus 1.4 Épurer ;
- une description formelle *presente\_1* fonctionnelle pour le processus 1.6 Présenter ;
- Procédure :
  1. Le modélisateur crée une session avec : *description\_1*, *formel\_1*, *epure\_1* et *presente\_modele*.
  2. Le modélisateur compile<sup>114</sup> avec l'aide du système.
  3. Le modélisateur sauvegarde le modèle développé.
  4. Le modélisateur réutilise la session, mais avec *description\_2*.
  5. Le modélisateur compile avec l'aide du système.
  6. Le modélisateur sauvegarde le modèle développé.
- Conditions de réussite :
  - la base de données (**Modèle vérifié**) contient les informations provenant de *description\_1* dont la *description\_2* ne les modifie pas [EX01] ;
  - la base de données (**Modèle vérifié**) contient les informations provenant de *description\_2* changeant des informations de la *description\_1* [EX01] ;
  - la base de données (**Modèle vérifié**) contient les informations provenant de *description\_2* ne changeant pas des informations de la *description\_1* [EX01] ;
  - le système est en mesure de présenter le modèle développé depuis *description\_1* et *description\_2* [EX10] ;
  - les informations sélectionnées comme superflues ont été épurées et ne se retrouvent pas dans la base de données [EX13] ;

---

114. Compile comprend les processus 1.4 Épurer, 1.5 Formaliser et 1.6 Présenter.

## 5.1. LA VÉRIFICATION DU RESPECT DES EXIGENCES

- le système est en mesure de fournir un journal sur le processus et son déroulement [EX14].

CA02 : Vérifier que l'IAR ne repose que sur du formalisme.

- Prérequis :
  - une description proposée d'un modèle *description\_1* ;
  - une description formelle *formel\_1* fonctionnelle dans le processus 1.5.1 Traduire ;
  - une description formelle *epure\_1* fonctionnelle dans le processus 1.4 Épurier ;
  - une description formelle *presente\_1* fonctionnelle dans le processus 1.6 Présenter ;
- Procédure :
  1. Le modélisateur crée une session avec : *description\_1*, *formel\_1*, *epure\_1* et *presente\_1*.
  2. Le modélisateur compile avec l'aide du système.
  3. Le modélisateur sauvegarde le modèle développé.
- Conditions de réussite :
  - un modèle vérifié contenu dans la base de données est associé à des concepts, des règles et des langages [EX05] ;
  - la journalisation affiche la conformité des informations provenant de documents, de descriptions proposées d'un modèle ou d'une représentation standardisée du modèle par rapport à leurs métamodèles [EX06] ;
  - la journalisation affiche les transformations ainsi que les formalismes qu'elles ont utilisés [EX07] ;
  - la journalisation affiche les intrants aux transformations [EX08] ;
  - La journalisation affiche les différentes vérifications avec leurs intrants (des modèles, des configurations, des documents), puis leurs niveaux (lexical, syntaxique et sémantique) [EX09].

## 5.1. LA VÉRIFICATION DU RESPECT DES EXIGENCES

CA03 : Vérifier que l'utilisateur peut contextualiser l'IAR, incluant la session, les documents, les modèles et les métamodèles. La contextualisation ne se limite pas à annoter les artefacts, mais aussi à adapter l'IAR à un contexte particulier.

- Prérequis :
  - une description proposée d'un modèle *description\_1* ;
  - une description formelle *formel\_1* fonctionnelle dans le processus 1.5.1 Traduire ;
  - une description formelle *epure\_1* fonctionnelle dans le processus 1.4 Épurier ;
  - une description formelle *presente\_approbateur* fonctionnelle et contextualisée pour le lectorat approbateur dans le processus 1.6 Présenter ;
- Procédure :
  1. Le modélisateur crée une session avec : *description\_1*, *formel\_1*, *epure\_1* et *presente\_1*.
  2. Le modélisateur contextualise la session et utilise un métamodèle déjà contextualisé en changeant la transformation présente pour utiliser la description formelle *presente\_approbateur*.
  3. Le modélisateur contextualise le document en l'annotant cela prend la forme d'une méta-information ajoutée à la description proposée.
  4. Le modélisateur contextualise le modèle en ajoutant une méta-information dans la description proposée.
  5. Le modélisateur compile avec l'aide du système.
  6. Le modélisateur sauvegarde le modèle développé.
- Conditions de réussite :
  - le modèle vérifié comprend la contextualisation du document et du modèle [EX11] ;
  - la journalisation mentionne les contextualisations utilisées pour la session [EX15] ;

## 5.1. LA VÉRIFICATION DU RESPECT DES EXIGENCES

- le modèle développé est présenté selon la contextualisation de la session [EX15].

CA04 : Vérifier que l'IAR permet de former un seul modèle à partir de plusieurs documents provenant de différents contextes.

- Prérequis :
  - un modèle développé *modele\_1* contenu dans la base de données ;
  - une session *session\_1* qui a créé le modèle développé *modele\_1* <sup>115</sup> ;
  - une description formelle *formel\_1* fonctionnelle dans le processus de traduction ;
  - une description formelle *epure\_1* fonctionnelle dans le processus *épurer* ;
  - une description formelle *presente\_1* fonctionnelle qui permet de présenter le modèle comme la description proposée dans le processus 1.6 Présenter ;
- Procédure :
  1. Le modélisateur récupère la session *session\_1*.
  2. Le modélisateur conserve la représentation standardisée <sup>116</sup> du modèle *represente\_1*.
  3. Le modélisateur crée une session avec *formel\_1*, *epure\_1*, *presente\_1* et avec *represente\_1* (document supplémentaire) à mettre à la description proposée.
  4. Le modélisateur ajoute un document avec une information (prédicat) à la description proposée.
  5. Le modélisateur ajuste le contexte dans la description proposée.
  6. Le modélisateur compile avec l'aide du système.
  7. Le modélisateur sauvegarde le modèle développé *modele\_2*.
- Condition de réussite :
  - Le modèle développé *modele\_2* comprend une copie des informations provenant du modèle développé *modele\_1* [EX02, EX03].

---

115. Actuellement, passer par la session est actuellement le seul moyen de récupérer la description proposée.

116. La représentation standardisée est celle issue du processus *présente*.

## 5.1. LA VÉRIFICATION DU RESPECT DES EXIGENCES

CA05 : Vérifier que l'IAR permet de comparer des modèles.

- Prérequis :
  - un modèle développé *modele\_1* contenu dans la base de données ;
  - une session *session\_1* qui a créé le modèle développé *modele\_1* ;
  - un modèle développé *modele\_2* contenu dans la base de données ;
  - une session *session\_2* qui a créé le modèle développé *modele\_1* ;
  - une description formelle *formel\_1* fonctionnelle dans le processus 1.5.1 Traduire ;
  - une description formelle *epure\_1* fonctionnelle dans le processus 1.4 Épurier ;
  - une description formelle *presente\_1* fonctionnelle qui permet de présenter le modèle comme la description proposée dans le processus 1.6 Présenter ;
- Procédure :
  1. Le modélisateur récupère la session *session\_1*.
  2. Le modélisateur compile avec l'aide du système.
  3. Le modélisateur conserve la journalisation et la présentation à l'extérieur de l'IAR.
  4. Le modélisateur récupère la session *session\_2*.
  5. Le modélisateur compile avec l'aide du système.
- Conditions de réussite :
  - les deux modèles sont présentés sous une représentation standardisée [EX04] ;
  - la journalisation affiche des métriques<sup>117</sup> sur le texte (nombre de caractères ou de lignes), sur la structure (nombre d'informations ou de groupes d'énoncés logiques) ou sur la logique (nombre de contradictions trouvées) pour les modèles développés. Cela permet à l'utilisateur de les comparer [EX04, EX12].

---

117. Ces métriques devraient découler d'un modèle de qualité, ce n'est pas un objectif entièrement rempli par cette preuve de concept.

## 5.1. LA VÉRIFICATION DU RESPECT DES EXIGENCES

Les résultats des cas de tests sont résumés dans le tableau 4.1 (les descriptions formelles des cas de tests se retrouvent dans la preuve de concept<sup>118</sup>) tandis que la couverture des exigences par les cas de tests est complète et s'exprime dans le tableau 4.2. Ces cas de tests possèdent quelques caractéristiques :

- La plupart des prérequis sont des intrants aux procédures et ils ne sont pas spécifiés. Ainsi, il peut exister un grand nombre d'implémentations de ces tests. Néanmoins, ils devraient tous être en mesure de vérifier les exigences couvertes par le cas.
- Faire varier les intrants revient à tester les descriptions formelles (grammaire). Ce n'est pas ce qui est recherché.
- Plusieurs cas pourraient vérifier les mêmes exigences, c'est-à-dire des exigences communes à l'usage de l'engin d'exécution. Actuellement, les cas regroupent les vérifications d'exigences similaires dans un objectif de simplification.

**TABLEAU 5.1** – Résultats des cas de tests

| Cas de test   | Couverture de l'engin d'exécution |
|---------------|-----------------------------------|
| CA01          | 78%                               |
| CA02          | 77%                               |
| CA03          | 75%                               |
| CA04          | 63%                               |
| CA05          | 62%                               |
| Union des cas | 80%                               |

---

118. À cette adresse : <https://github.com/mbjolin/iar>

## 5.1. LA VÉRIFICATION DU RESPECT DES EXIGENCES

**TABEAU 5.2** – Correspondances des cas de tests et des exigences

| Cas<br>Exigence | CA01 : Vérifier que l'IAR permet la création ou la modification d'un modèle à partir d'une description proposée. Ainsi que la journalisation du système, l'épuration de la description proposée et la présentation du modèle. | CA02 : Vérifier que l'IAR repose que sur du formalisme. | CA03 : Vérifier que l'utilisateur peut contextualiser l'IAR cela comprend la session, les documents, les modèles et metamodels. | CA04 : Vérifier que l'IAR permet de former un seul modèle à partir de plusieurs documents provenant de différents contextes. | CA05 : Vérifier que l'IAR permet de comparer des modèles. |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| EX01            | O                                                                                                                                                                                                                             | P                                                       | P                                                                                                                               | P                                                                                                                            | P                                                         |
| EX02            | N                                                                                                                                                                                                                             | N                                                       | N                                                                                                                               | O                                                                                                                            | N                                                         |
| EX03            | N                                                                                                                                                                                                                             | N                                                       | N                                                                                                                               | O                                                                                                                            | N                                                         |
| EX04            | N                                                                                                                                                                                                                             | N                                                       | N                                                                                                                               | N                                                                                                                            | O                                                         |
| EX05            | P                                                                                                                                                                                                                             | O                                                       | P                                                                                                                               | P                                                                                                                            | P                                                         |
| EX06            | P                                                                                                                                                                                                                             | O                                                       | P                                                                                                                               | P                                                                                                                            | P                                                         |
| EX07            | P                                                                                                                                                                                                                             | O                                                       | P                                                                                                                               | P                                                                                                                            | P                                                         |
| EX08            | P                                                                                                                                                                                                                             | O                                                       | P                                                                                                                               | P                                                                                                                            | P                                                         |
| EX09            | P                                                                                                                                                                                                                             | O                                                       | P                                                                                                                               | P                                                                                                                            | P                                                         |
| EX10            | O                                                                                                                                                                                                                             | P                                                       | P                                                                                                                               | P                                                                                                                            | P                                                         |
| EX11            | N                                                                                                                                                                                                                             | N                                                       | O                                                                                                                               | N                                                                                                                            | N                                                         |
| EX12            | N                                                                                                                                                                                                                             | N                                                       | N                                                                                                                               | N                                                                                                                            | O                                                         |
| EX13            | O                                                                                                                                                                                                                             | P                                                       | P                                                                                                                               | P                                                                                                                            | P                                                         |
| EX14            | O                                                                                                                                                                                                                             | P                                                       | P                                                                                                                               | P                                                                                                                            | P                                                         |
| EX15            | P                                                                                                                                                                                                                             | P                                                       | O                                                                                                                               | P                                                                                                                            | P                                                         |

<sup>a</sup> La signification des valeurs est : O (oui), N (non) et P (partiel).

<sup>b</sup> Les cases grises indiquent qu'un cas ne couvre pas une exigence.

<sup>c</sup> Raccourcis vers les [exigences 1 à 12](#) et vers les [exigence 13 à 15](#).

### 5.2 Une guide pour la validation des attentes

Une expérimentation se déroule habituellement en suivant un protocole expérimental. Cette section donne des descriptions comprenant les sujets couverts par un protocole expérimental typique. Il y a la définition de la recherche, la collecte des données et l'analyse des données. La mise en place de cette expérimentation n'a pas encore eu lieu, car elle nécessite un prototype découlant d'une preuve de concept, qui permettra alors de définir les critères de validation et un protocole expérimental. Il s'agit d'une prochaine étape découlant de ce travail. Ainsi, cette section est en fait un guide qui donne plusieurs explications sur la manière de procéder pour arriver à valider les attentes d'une façon rigoureuse. De plus, il faut rappeler que, même avec plusieurs expérimentations, la démonstration du respect des attentes n'est pas assurée pour autant.

La définition de la recherche reprend les attentes vis-à-vis de l'IAR. Ces attentes s'adressent aux caractéristiques de qualité relatives suivantes : la complétude, l'efficacité et l'adaptation. Pour chacune de ces attentes, des questions sont formulées et l'expérimentation vise à répondre à ces questions.

- AT01 : La solution devrait améliorer la compréhension des utilisateurs pour les documents servant aux modèles.
  - Est-ce que le nombre d'itérations se trouve diminué s'il y a un cycle d'approbation des modèles ?
  - Est-ce que l'utilisation de l'IAR dans une expérimentation rétrospective permet de réduire les erreurs trouvées plus loin dans le procédé de développement ?
- AT02 : La solution devrait fournir, dans le cadre d'un environnement de développement, les fonctionnalités nécessaires à la production et à la maintenance des documents servant aux modèles.
  - Est-ce que les fonctionnalités de l'IAR utilisées retournent le résultat souhaité par les utilisateurs ?
  - Est-ce que des fonctionnalités manquantes sur l'IAR ont été soulevées pendant une période donnée de la part des utilisateurs ?



## 5.2. UNE GUIDE POUR LA VALIDATION DES ATTENTES

- AT03 : La solution devrait rendre plus efficace la manipulation des documents servant aux modèles, plus particulièrement en regard du temps requis et de la charge cognitive induite par le processus, et ce, pour l'ensemble des agents.
  - Quel est le temps consacré à la documentation ?
  - Subjectivement, selon les utilisateurs, quels sont les facteurs qui peuvent faire varier le temps consacré à la documentation ?
  - Est-ce que l'utilisation de l'IAR change le temps consacré à la documentation ?
- AT04 : La solution devrait être utilisable par des utilisateurs ayant des acquis distincts et finirait par être maîtrisée.
  - Combien de temps doit-on consacrer à la maîtrise de l'IAR ?
  - Est-ce que les acquis influencent le temps devant être consacré à la maîtrise de l'IAR ?
- AT05 : La solution devrait s'utiliser dans différents environnements de développement faisant appel à différents procédés, gabarits de documents, langages, modèles, métamodèles et configurations.
  - Est-ce que l'IAR peut s'adapter lorsque le procédé de développement change ?
  - Est-ce que l'IAR peut se déployer dans deux environnements distincts ?
  - Quelles sont les formations à offrir en fonction des acquis qui permettent de faciliter et d'accélérer la prise en main de l'IAR et du processus ?

La stratégie de recherche employée est celle d'une recherche de développement. Cela comprend l'élaboration d'un instrument, tel que décrit dans les précédents chapitres, et sa mise à l'épreuve [65]. Dans une telle stratégie, il faut expliciter la mise en place de l'instrument, le contexte et les interventions réalisées. Ainsi, des résultats sont obtenus sur l'instrument et sur sa mise en place.

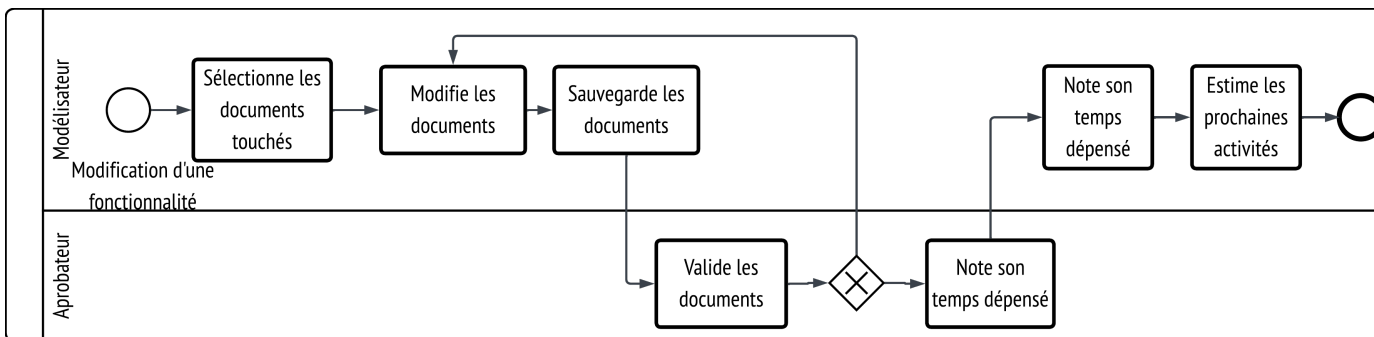
La mise en place de l'IAR peut se faire dans tous les environnements où des documents décrivent en fait des modèles. Cependant, c'est seulement là où il y a une surveillance et un contrôle du travail qu'il sera possible de mettre en place une expérimentation en

## 5.2. UNE GUIDE POUR LA VALIDATION DES ATTENTES

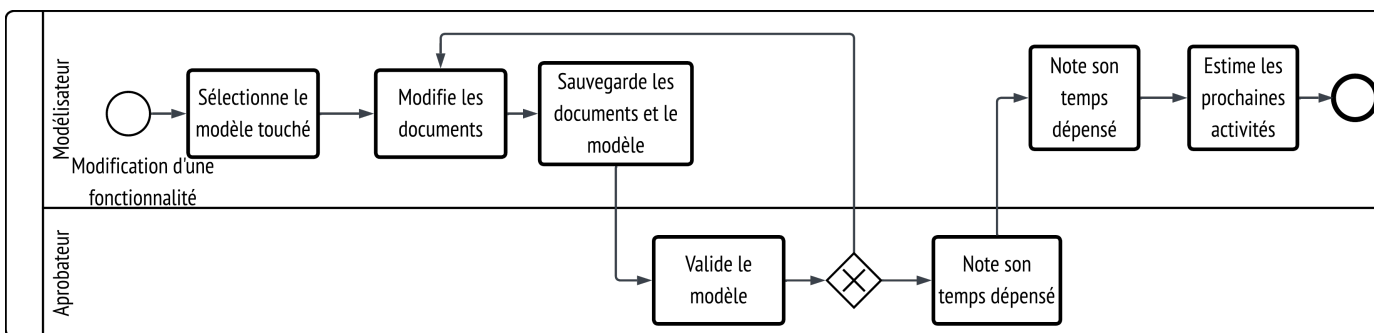
vue de vérifier les bénéfices de l'IAR. Cela implique que certaines hypothèses doivent être respectées, notamment les suivantes :

- les membres de l'organisation suivent au moins un procédé, même s'il n'est pas forcément clairement défini ;
- les activités et les tâches sont organisées avec l'aide d'un instrument servant à la gestion de projet ;
- les dépenses en ressources sont consignées et accessibles avec l'aide d'un instrument servant au suivi de temps.

Avec ces hypothèses respectées, il est possible de modéliser le processus avant (figure 5.1) et après la mise en place de l'IAR (figure 5.2). Le cas modélisé est celui d'une modification d'une fonctionnalité induisant des changements à la documentation. Il est important de minimiser les différences entre les deux processus pour assurer un contrôle plus grand au moment de l'expérimentation.



**FIGURE 5.1** – Diagramme BPMN sur des changements aux documents sans l'IAR



**FIGURE 5.2** – Diagramme BPMN sur des changements aux documents avec l'IAR

## 5.2. UNE GUIDE POUR LA VALIDATION DES ATTENTES

Idéalement, il n’y aurait que l’ajout de l’IAR lors de l’expérimentation sans aucune autre modification. Cela n’est pas possible, puisque l’expérimentation elle-même demande des ajustements :

- une formation sur l’IAR dont le premier objectif est d’éveiller l’intérêt et le second objectif est d’en permettre la maîtrise ;
- des échanges avec les utilisateurs pour confirmer la signification de concepts, le respect de procédés ou la présence de certains biais ;
- l’aide des utilisateurs pour obtenir des données supplémentaires.

La collecte des données s’opère principalement autour des utilisateurs de l’IAR. Dans un contexte professionnel, ces utilisateurs occupent le plus souvent des postes de professionnels informatiques. Ils font déjà l’usage d’autres instruments, l’IAR se retrouve donc à compétitionner avec ces autres instruments. S’il n’y a pas d’intérêt de la part de ces utilisateurs, et ce, même avec la démonstration de l’utilité, il n’y aura pas d’avenir pour l’IAR. Aussi, les usagers<sup>119</sup> ne sont pas considérés pour le moment, mais ils devraient l’être. Parmi les usagers, il a tous ceux qui interagissent avec les artefacts de documentation. Puisque l’IAR est utilisé dans le procédé de développement d’un logiciel, il se trouve à influencer de nombreux artefacts de documentation, comme la spécification d’interface utilisateur, le plan d’installation ou la description d’une version d’un logiciel. Ces artefacts peuvent s’en trouver amélioré ou détérioré. Ainsi, les usagers peuvent encourager ou décourager l’utilisation de l’IAR.

Définir le protocole d’expérimentation devrait comprendre une classification des différentes variables [65]. Ces variables peuvent être qualifiées d’indépendantes, de dépendantes et de contrôlées, le tableau 5.3 énonce une classification de plusieurs variables. Les variables indépendantes et dépendantes fournissent les données à analyser. Plusieurs facteurs agissant sur les variables dépendantes rendent l’analyse des données plus difficile. Les variables contrôlées, quant à elles, sont celles qui peuvent demeurer constantes durant l’expérimentation et, par conséquent, ne devraient pas influencer les autres variables.

---

119. Les usagers n’utilisent pas directement l’IAR comme les utilisateurs.

## 5.2. UNE GUIDE POUR LA VALIDATION DES ATTENTES

**TABLEAU 5.3** – Différentes variables pour l'expérimentation

| Variable indépendantes                                                                                                                                                                                                                                                                                                                                                                                                                                    | Variable dépendantes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Variable contrôlées                                                                                                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• niveau de scolarité de l'auteur;</li> <li>• niveau de scolarité de l'approbateur;</li> <li>• nombre d'années d'expérience de l'auteur;</li> <li>• nombre d'années d'expérience de l'approbateur;</li> <li>• nombre de documents;</li> <li>• nombre de mots;</li> <li>• nombre de prédicats formels ou non formels (pour évaluer l'instrument par rapport à lui-même);</li> <li>• nombre de citations.</li> </ul> | <ul style="list-style-type: none"> <li>• nombre d'heures à modifier les documents;</li> <li>• nombre d'heures à modifier un modèle;</li> <li>• nombre de cycles de validation pendant un développement;</li> <li>• nombre de problèmes soulevés sur les documents lors du développement;</li> <li>• nombre de problèmes soulevés sur les modèles lors du développement;</li> <li>• justesse des estimations pour les prochaines activités;</li> <li>• nombre d'incohérences soulevées par l'IAR;</li> <li>• nombre de cycles de développement pour le produit;</li> <li>• nombre de problèmes signalés sur l'IAR;</li> <li>• nombre d'améliorations signalées sur l'IAR;</li> <li>• nombre d'améliorations apportées sur l'IAR.</li> </ul> | <ul style="list-style-type: none"> <li>• système pour la prise de temps;</li> <li>• système pour la prise des estimations;</li> <li>• modifications de fonctionnalités aux produits;</li> <li>• système pour signaler les problèmes et les améliorations sur les produits et sur l'IAR.</li> </ul> |

La collecte des données, qui comprend la prise de mesure sur le travail des utilisateurs ou des artefacts, peut se faire par le biais : d'extractions, d'opérations provenant des utilisateurs ou de questionnaires adressés aux utilisateurs.

L'analyse des données doit prendre en compte les biais des données et des instruments. Les biais peuvent toucher trois types de variables :

- le niveau de scolarité et le nombre d'années d'expérience sont possiblement insuffisants pour décrire exactement les acquis des utilisateurs;
- le nombre de documents, de mots, de prédicats et de citations demeurent des indicateurs partiels de la complexité;
- la prise des mesures peut varier selon les utilisateurs (certains peuvent entrer une heure après avoir consacré réellement cinquante minutes à la tâche);
- la maîtrise des instruments peut provoquer des variations (les utilisateurs maîtrisant davantage l'IAR vont signaler plus de problèmes ou proposer plus d'améliorations);
- certains instruments sont utilisés à distance, ce qui empêche leurs contrôles complets; ils pourraient être modifiés au cours d'une expérimentation.

## 5.2. UNE GUIDE POUR LA VALIDATION DES ATTENTES

Quatre conditions doivent être respectées pour poursuivre la mise en place de l'expérimentation. Ces conditions sont :

1. Passer de la preuve de concept à un prototype. L'implication de personnes sur une période de temps significative pour une expérimentation a effectivement un coût significatif. Il est nécessaire que la solution proposée remplisse les caractéristiques de qualité absolues et qu'elle essaie le plus possible de remplir les critères de qualités relatives, puisqu'un dysfonctionnement entraînerait l'invalidation de l'expérimentation. Pour transformer la preuve de concept en prototype qui s'approche davantage d'un produit logiciel, il faut :
  - (a) Construire un document de spécification des exigences logiciel ;
    - (i) Valider les besoins et exigences auprès de plusieurs parties prenantes attachées au problème ou à la solution.
    - (ii) Vérifier que les exigences forment un système formel.
  - (b) Construire un document du plan de test et l'exécuter ;
    - (i) Utiliser d'autres techniques de tests : cela peut être des revues techniques, des tests exploratoires, de l'analyse des valeurs limites, etc.
    - (ii) Sélectionner des types et des pourcentages de couverture plus exigeants.
  - (c) Construire des documents pour l'établissement du logiciel.
    - (i) S'assurer de transmettre le fonctionnement du logiciel.
    - (ii) S'assurer de transmettre l'usage du logiciel.
    - (iii) S'assurer de transmettre la mise en place du logiciel.
2. Avoir à la disposition un environnement pour réaliser un banc d'essai. L'environnement prévu à l'origine possédait les caractéristiques nécessaires :
  - les membres de l'organisation suivent un procédé défini dans un diagramme d'activité utilisant le *Unified Modeling Language* (UML) ;
  - les activités et les tâches sont organisées à l'aide de JIRA, un instrument de gestion de projet ;

## 5.2. UNE GUIDE POUR LA VALIDATION DES ATTENTES

- les dépenses en ressources sont consignées à l'aide de JIRA et du module d'extension Timesheet Tracking.

Cependant, des changements ont conduit à ce que la gestion de projet ou le suivi du temps ne soient plus effectués par des instruments informatiques. Cela a entraîné l'inaccessibilité des variables dépendantes et la disparition des variables contrôlées. Ainsi, il faut trouver un nouvel environnement.

3. Construire le document du protocole expérimental complet, ce mémoire en décrit que certaines parties. Le protocole expérimental aide à assurer plusieurs caractéristiques de qualité, comme son efficacité, sa couverture par rapport aux interrogations et sa réfutabilité en décrivant la façon de reproduire l'expérimentation. D'ailleurs, cette condition est nécessaire à la suivante, qui est d'obtenir les autorisations éthiques.
4. Obtenir les autorisations nécessaires en suivant le procédé d'évaluation éthique des projets de recherche de l'Université de Sherbrooke. Cette démarche est obligatoire, puisqu'il y a des données liées à des personnes. D'ailleurs, l'établissement d'un protocole adéquat, conforme aux bonnes pratiques d'expérimentation impliquant des humains, est en fait un problème en soi, qu'il convient de traiter séparément.

Même avec l'expérimentation terminée, cela ne permet pas d'affirmer que des exigences aussi subjectives soient respectées, mais elle permet d'obtenir des réponses. En effet, il s'agit d'une recherche appliquée où le contexte varie irrémédiablement selon les acteurs, les procédés et les instruments. Il devient donc difficile d'établir l'apport quantitatif ou qualitatif d'un seul instrument. Cependant, il est probable qu'une expérimentation comme celle-là puisse appuyer et corriger un protocole expérimental, ce qui pourrait faire l'objet d'une publication. Éventuellement, d'autres expérimentations suivant le même protocole seront peut-être menées. Après un nombre suffisant d'expérimentations, une méta-analyse pourrait enfin être entreprise, ce qui pourrait fournir des réponses.

# Conclusion

La revue de littérature portant sur les documents en informatique appliquée a montré que les documents, les modèles et les connaissances sont intimement liés. Elle conduit à considérer les documents qui contribuent aux modèles, puisqu'ils sont abondamment utilisés et nécessaires en informatique appliquée. Avec ces nouvelles connaissances, le problème de la formalisation et de la compréhension des documents apparaît comme important et résoluble. S'ensuit, l'élaboration d'une solution qui prend la forme d'une preuve de concept appelée Instrument d'aide à la rédaction (IAR). Cette solution utilise plusieurs autres instruments, dont les principaux sont les modèles, les langages formels et la logique. Après la description des instruments, une évaluation de l'IAR est menée. Il est vérifié que la preuve de concept respecte les exigences, tandis que la validation des attentes fera partie d'une prochaine étape. De tout ce travail, plusieurs contributions peuvent être déduites. Pour finir, trois prolongements sont présentés, comprenant une contextualisation avec les GML.

## Contributions

La revue de littérature a permis de documenter et d'illustrer les aboutissements suivants :

- La documentation informatique demeure le vecteur privilégié de transmission des connaissances entre parties prenantes et praticiens.
- La modélisation est au cœur de l'informatique ; certains modèles sont destinés aux parties prenantes du projet et d'autres, aux praticiens responsables de la réalisation et de l'établissement.



## CONCLUSION

- La vérification et la validation des modèles ne peuvent être menées de façon satisfaisante sans la formalisation de ceux-ci.
- La documentation doit permettre d'associer intimement la description informelle des connaissances et la description formelle des modèles.
- Un instrument permettant de réaliser, de vérifier et de valider cette association s'avère nécessaire.

Il est raisonnable de penser qu'à défaut de méthodes bien définies, d'instruments appropriés et de formations adéquates, beaucoup d'informaticiens aient négligé ces faits et repoussé les bonnes pratiques qui en découlent. Il faut ajouter que les pressions commerciales induites ont eu elles aussi un rôle négatif.

Une définition du problème, la réalisation d'une solution et l'évaluation de la solution contribuent à :

- partager des modèles touchant au problème sur les processus d'obtention d'un instrument, d'un logiciel et d'un modèle ;
- fournir une solution décrite formellement et dont les exigences sont vérifiées ;
- fournir une manière de faire pour valider la solution par rapport aux attentes ;
- donner des exemples de modèles en utilisant des descriptions formelles ;
- constituer, avec ce mémoire, un exemple de procédé qui réalise et établit un logiciel (la preuve de concept) en informatique appliquée.

Ensemble, ces propositions forment un exemple de procédé de développement qui permet de réaliser un logiciel (la preuve de concept) qui sera à même de respecter plusieurs des caractéristiques de qualité souhaitées pour tout logiciel, modèle ou document.

La preuve de concept, l'IAR, contribue à :

- donner un instrument pour contrôler les documents ;
- se familiariser avec le concept de modèle, incluant les langages formels et les règles ;

## CONCLUSION

- se réappropriier la création de certains modèles, comme des grammaires et des métamodèles ;
- évaluer la façon et l’impact de mettre en place de bonnes pratiques.

En somme, un instrument développé dans l’objectif de réaliser de la bonne documentation informatique sera, du moins dans ce cas-ci, aussi en mesure de promouvoir de bonnes pratiques informatiques.

## Prolongements

Le prolongement le plus souhaitable serait de réaliser l’évaluation des attentes. D’ailleurs, la deuxième partie du chapitre 5 donne plusieurs éléments d’informations à ce sujet. Cette évaluation des attentes consiste à atteindre les quatre objectifs suivants :

- passer de la preuve de concept à un prototype ;
- avoir à la disposition un environnement pour réaliser un banc d’essai ;
- construire le document du protocole expérimental complet ;
- obtenir les autorisations nécessaires en suivant le procédé d’évaluation éthique.

À l’heure actuelle, la popularité des GML est telle que l’inclusion de ce sujet dans les prolongements peut susciter un intérêt marqué. Un deuxième prolongement envisageable serait d’utiliser un GML pour trouver des incohérences. Cependant, rien ne peut garantir la véracité des réponses. D’ailleurs, pour fonder et faciliter l’évaluation des réponses données, il est préférable de faire intervenir un système logique dans le procédé [152]. Le GML pourrait suggérer des reformulations, voire des modifications à la description proposée d’un modèle, que l’utilisateur pourra appliquer. Les modifications finiraient toutes vérifiées par l’IAR.

Un troisième prolongement serait de mener des expérimentations impliquant l’entraînement des GML. L’IAR permet d’obtenir des documents plus structurés et cohérents et, lorsqu’il s’exécute, permet d’obtenir trois groupes de documents :

- les descriptions proposées d’un modèle avant la compilation ;

## CONCLUSION

- les descriptions proposées d'un modèle après la compilation ;
- les représentations standardisées du modèle après la compilation.

Advenant des groupes suffisamment grands pour entraîner un GML, il serait intéressant de réaliser des entraînements différents avec différents groupes et d'observer les résultats. D'ailleurs, une expérimentation consistant à utiliser des analyseurs syntaxiques et statiques pour améliorer des données d'entraînement a été menée [93]. Le GML, une fois entraîné, était plus efficient. Par conséquent, il n'est pas impossible que, comme les humains, des GML puissent tirer des avantages de données cohérentes respectant une structure.

Un dernier prolongement serait la constitution d'une bibliothèque de modèles documentés prêts à l'usage. Cela nécessite tout d'abord une modification de la preuve de concept pour respecter les exigences de gestion des métamodèles. S'ensuit l'intégration d'instruments permettant, entre autres, les échanges et les modifications des métamodèles, comme le Meta-Object Facility (MOF) et l'Eclipse Modeling Framework (EMF). Ainsi, les utilisateurs seraient à même de pouvoir former une base entre les modèles pour mieux les comparer et les mesurer sans être contraints par les métamodèles se retrouvant à l'intérieur de l'implémentation. Aussi, l'utilisation du MOF assure le respect des standards et permet de plus facilement importer ou exporter les modèles.

Pour finir, ces prolongements exigent des efforts supplémentaires, mais nécessitent aussi le passage de la preuve de concept à un prototype solide et davantage adaptable. Étant donné l'importance que peut avoir la documentation informatique pour l'informatique appliquée, les efforts demandés pour créer ces instruments en valent largement la peine.

# Bibliographie

- [1] A. ABRAN, *Software Metrics and Software Metrology*, 1<sup>re</sup> éd., Wiley, mai 2010, ISBN 978-0-470-60683-4, doi:[10.1002/9780470606834](https://doi.org/10.1002/9780470606834).
- [2] J.-R. ABRIAL, *Modeling in Event-B : System and Software Engineering*, Cambridge University Press, Cambridge, 2010, ISBN 978-0-511-72976-8.
- [3] J.-R. ABRIAL et S. HALLERSTEDE, « Refinement, Decomposition, and Instantiation of Discrete Models : Application to Event-B », *Fundamenta Informaticae*, vol. 77(1-2), 2007, p. 1–28.
- [4] T. L. ADAMS, « Interprofessional Relations and The Emergence of A New Profession : Software Engineering in the United States, United Kingdom, and Canada », *The Sociological Quarterly*, vol. 48(3), août 2007, p. 507–532, ISSN 0038-0253, 1533-8525, doi:[10.1111/j.1533-8525.2007.00087.x](https://doi.org/10.1111/j.1533-8525.2007.00087.x).
- [5] AEROSPACE AND DEFENCE INDUSTRIES ASSOCIATION OF EUROPE, « S1000D International Specification for Technical Publications Using a Common Source Database », 2024.  
Récupéré de <https://s1000d.org> (consulté le 2026-04-01).
- [6] E. AGHAJANI, C. NAGY, M. LINARES-VÁSQUEZ, L. MORENO, G. BAVOTA, M. LANZA et D. C. SHEPHERD, « Software documentation : the practitioners’ perspective », dans « Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering », ACM, Seoul South Korea, juin 2020, ISBN 978-1-4503-7121-6, p. 590–601, doi:[10.1145/3377811.3380405](https://doi.org/10.1145/3377811.3380405).

## BIBLIOGRAPHIE

- [7] E. AGHAJANI, C. NAGY, O. L. VEGA-MARQUEZ, M. LINARES-VASQUEZ, L. MORENO, G. BAVOTA et M. LANZA, « Software Documentation Issues Unveiled », dans « 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE) », IEEE, Montreal, QC, Canada, mai 2019, ISBN 978-1-7281-0869-8, p. 1199–1210, doi:[10.1109/ICSE.2019.00122](https://doi.org/10.1109/ICSE.2019.00122).
- [8] A. AHO et J. D. ULLMAN, *Foundations of computer science*, Principles of computer science series, Computer science press, New York, 1992, ISBN 978-0-7167-8233-9.
- [9] A. V. AHO, M. S. LAM, R. SETHI et J. D. ULLMAN, éd., *Compilers : principes, techniques, & tools*, 2<sup>e</sup> éd., Pearson Addison-Wesley, Boston Munich, 2007, ISBN 978-0-321-48681-3 978-0-321-49169-5.
- [10] A. ALDERSON, « Meta-case technology », dans « Software Development Environments and CASE Technology », édité par G. GOOS, J. HARTMANIS, A. ENDRES et H. WEBER, vol. 509, Springer Berlin Heidelberg, Berlin, Heidelberg, 1991, ISBN 978-3-540-54194-3 978-3-540-47485-2, p. 81–91, doi:[10.1007/3-540-54194-2\\_27](https://doi.org/10.1007/3-540-54194-2_27). Series Title : Lecture Notes in Computer Science.
- [11] S. S. ALHIR, *UML in a nutshell : a desktop quick reference*, 1<sup>re</sup> éd., O'Reilly, Sebastopol, Calif, 1998, ISBN 978-1-56592-448-2.
- [12] K. ALPERNAS, Y. M. Y. FELDMAN et H. PELEG, « The wonderful wizard of LoC : paying attention to the man behind the curtain of lines-of-code metrics », dans « Proceedings of the 2020 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software », ACM, Virtual USA, novembre 2020, ISBN 978-1-4503-8178-9, p. 146–156, doi:[10.1145/3426428.3426921](https://doi.org/10.1145/3426428.3426921).
- [13] L. ANDERSON SNYDER, D. E. RUPP et G. C. THORNTON, « Personnel selection of information technology workers : The people, the jobs, and issues for human resource management », dans « Research in personnel and human resources management », Emerald Group Publishing Limited, 2006, p. 305–376, doi:[10.1016/S0742-7301\(06\)25008-4](https://doi.org/10.1016/S0742-7301(06)25008-4).

## BIBLIOGRAPHIE

- [14] M. ANICHE, *Effective Software Testing : A developer's guide*, Manning Publications, 2022, ISBN 9781638350583.
- [15] ANSI et ANS, « American National Standard for Documentation of Computer Software », 1995.  
Récupéré de <https://webstore.ansi.org/standards/ansi/ansians101995> Aperçu de l'ANSI/ANS-10.3-1995.
- [16] A. AURUM, R. JEFFERY, C. WOHLIN et M. HANDZIC, éd., *Managing Software Engineering Knowledge*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2003, ISBN 978-3-642-05573-7 978-3-662-05129-0, doi:[10.1007/978-3-662-05129-0](https://doi.org/10.1007/978-3-662-05129-0).
- [17] N. AUSSENAC-GILLES, C. COMPAROT, A. DUPUY, N. EL MALIKI, R. TOURNIER, B.-H. TRAN et C. T. DOS SANTOS, « eCISE-OWL : Représentation OWL du Schéma de Données de l'Environnement Commun d'Échange d'Information pour le Domaine Maritime », dans « 33èmes Journées Francophones d'Ingénierie des Connaissances (IC 2022) », AFIA, 2022,  
Récupéré de <https://hal.science/hal-03760512v1> p. 82–91.
- [18] P. AVGERIOU, P. KRUCHTEN, R. L. NORD, I. OZKAYA et C. SEAMAN, « Reducing Friction in Software Development », *IEEE Software*, vol. 33(1), janvier 2016, p. 66–73, ISSN 0740-7459, doi:[10.1109/MS.2016.13](https://doi.org/10.1109/MS.2016.13).
- [19] AXELOS, *ITIL Foundation : ITIL 4 Edition*, Stationery Office Books, Norwich, England, 2019, ISBN 9780113316076.
- [20] Ö. BABUR, E. CONSTANTINOU et A. SEREBRENIK, « Language usage analysis for EMF metamodels on GitHub », *Empirical Software Engineering*, vol. 29(1), 2024, p. 23, doi:[10.1007/s10664-023-10368-x](https://doi.org/10.1007/s10664-023-10368-x).
- [21] J. BACELAR ALMEIDA, M. J. FRADE, J. SOUSA PINTO et S. MELO DE SOUSA, *Rigorous software development : An introduction to program verification*, 2011<sup>e</sup> éd., Springer, London, England, 2011, ISBN 978-0-85729-018-2, doi:[10.1007/978-0-85729-018-2](https://doi.org/10.1007/978-0-85729-018-2).

## BIBLIOGRAPHIE

- [22] X. BAI, H. ZHANG et L. HUANG, « Empirical Research in Software Process Modeling : A Systematic Literature Review », dans « 2011 International Symposium on Empirical Software Engineering and Measurement », IEEE, Banff, AB, Canada, septembre 2011, ISBN 978-1-4577-2203-5 978-0-7695-4604-9, p. 339–342, doi:[10.1109/ESEM.2011.43](https://doi.org/10.1109/ESEM.2011.43).
- [23] A. BARREDO ARRIETA, N. DÍAZ-RODRÍGUEZ, J. DEL SER, A. BENNETOT, S. TABIK, A. BARBADO *et al.*, « Explainable Artificial Intelligence (XAI) : Concepts, taxonomies, opportunities and challenges toward responsible AI », *Information Fusion*, vol. 58, juin 2020, p. 82–115, ISSN 15662535, doi:[10.1016/j.inffus.2019.12.012](https://doi.org/10.1016/j.inffus.2019.12.012).
- [24] V. R. BASILI, *The Role of Experimentation in Software Engineering : Past, Current, and Future*, chap. 0, Springer, Berlin Heidelberg, 2005, ISBN 978-3-540-24547-6 978-3-642-06389-3, p. 1–13.
- [25] R. BASQUE, *CMMI 1.2, le modèle : un itinéraire fléché vers le Capability maturity model integration*, 3<sup>e</sup> éd., InfoPro, Dunod, Paris, 2009, ISBN 978-2-10-052233-0.
- [26] L. BASS, P. CLEMENTS et R. KAZMAN, *Software Architecture in Practice*, 4<sup>th</sup> Edition, 4<sup>e</sup> éd., Addison-Wesley Professional, 2021, ISBN 978-0-13-688597-9.
- [27] M. BERNARDO, V. CORTELLESA et A. PIERANTONIO, éd., *Formal Methods for Model-Driven Engineering : 12th International School on Formal Methods for the Design of Computer, Communication, and Software Systems, SFM 2012, Bertinoro, Italy, June 18-23, 2012. Advanced Lectures, Lecture Notes in Computer Science*, vol. 7320, Springer Berlin Heidelberg, Berlin, Heidelberg, 2012, ISBN 978-3-642-30981-6 978-3-642-30982-3, doi:[10.1007/978-3-642-30982-3](https://doi.org/10.1007/978-3-642-30982-3).
- [28] S. BEYDEDA, M. BOOK et V. GRUHN, éd., *Model-driven software development*, Springer, Berlin ; New York, 2005, ISBN 978-3-540-25613-7.
- [29] D. BJØRNER et M. C. HENSON, éd., *Logics of Specification Languages*, Monographs in Theoretical Computer Science. An EATCS Series, Springer Berlin

## BIBLIOGRAPHIE

- Heidelberg, Berlin, Heidelberg, 2008, ISBN 978-3-540-74106-0 978-3-540-74107-7, doi:[10.1007/978-3-540-74107-7](https://doi.org/10.1007/978-3-540-74107-7).
- [30] A. P. BLACK, « Object-oriented programming : Some history, and challenges for the next fifty years », *Information and Computation*, vol. 231, 2013, p. 3–20, doi:[10.1016/j.ic.2013.08.002](https://doi.org/10.1016/j.ic.2013.08.002).
- [31] S. L. BLODGETT, S. BAROCAS, H. D. III et H. WALLACH, « Language (Technology) is Power : A Critical Survey of "Bias" in NLP », mai 2020. Non publié, doi:[10.48550/arXiv.2005.14050](https://doi.org/10.48550/arXiv.2005.14050).
- [32] B. BOEHM et P. PAPACCIO, « Understanding and controlling software costs », *IEEE Transactions on Software Engineering*, vol. 14(10), octobre 1988, p. 1462–1477, ISSN 00985589, doi:[10.1109/32.6191](https://doi.org/10.1109/32.6191).
- [33] B. W. BOEHM, éd., *Software cost estimation with Cocomo II*, Prentice Hall, Upper Saddle River, NJ, 2000, ISBN 978-0-13-026692-7.
- [34] B. W. BOEHM et R. TURNER, *Balancing Agility and Discipline : A Guide for the Perplexed*, 6<sup>e</sup> éd., Addison-Wesley, Boston, Mass., 2008, ISBN 978-0-321-18612-6.
- [35] J. P. BOWEN et M. G. HINCHEY, « Ten commandments of formal methods », *Computer*, vol. 28(4), 2002, p. 56–63, doi:[10.1109/2.375178](https://doi.org/10.1109/2.375178).
- [36] M. BRAMBILLA, J. CABOT et M. WIMMER, *Model-driven software engineering in practice*, Morgan & Claypool Publishers, 2017, ISBN 978-3-031-02549-5, doi:[10.1007/978-3-031-02549-5](https://doi.org/10.1007/978-3-031-02549-5).
- [37] I. K. BRAY, *An introduction to requirements engineering*, Pearson Education, 2002, ISBN 0201 767929.
- [38] R. J. BROCKMANN, « The why, where and how of minimalism », dans « Proceedings of the 8th annual international conference on Systems documentation », 1990, p. 111–119. 1990 ACM 0-89791-414-719011000-0111 1.5.



## BIBLIOGRAPHIE

- [39] F. P. BROOKS, *The mythical man-month : essays on software engineering*, anniversary ed éd., Addison-Wesley Pub. Co, Reading, Mass, 1995, ISBN 978-0-201-83595-3.
- [40] F. P. BROOKS, *The design of design : essays from a computer scientist*, Addison-Wesley, Upper Saddle River, N.J, 2010, ISBN 978-0-201-36298-5 978-0-321-70208-1.
- [41] A. W. BROWN, J. CONALLEN et D. TROPEANO, « Introduction : Models, modeling, and model-driven architecture (MDA) », dans « Model-Driven Software Development », Springer, 2005, ISBN 978-3-540-25613-7, p. 1–16.
- [42] M. BROY, A. D. BRUCKER, A. FANTECHI, M. GLEIRSCHER, K. HAVE-LUND, M. A. KUPPE *et al.*, « Does every computer scientist need to know formal methods? », *Formal Aspects of Computing*, vol. 37(1), 2024, p. 1–17, doi:[10.1145/3670795](https://doi.org/10.1145/3670795).
- [43] J.-M. BRUEL, S. EBERSOLD, F. GALINIER, M. MAZZARA, A. NAUMCHEV et B. MEYER, « The Role of Formalism in System Requirements », *ACM Computing Surveys*, vol. 54(5), juin 2022, p. 1–36, ISSN 0360-0300, 1557-7341, doi:[10.1145/3448975](https://doi.org/10.1145/3448975).
- [44] E. J. BYRNE, « Software reverse engineering : A case study », *Software : Practice and Experience*, vol. 21(12), décembre 1991, p. 1349–1364, ISSN 0038-0644, 1097-024X, doi:[10.1002/spe.4380211206](https://doi.org/10.1002/spe.4380211206).
- [45] J. CABOT, C. GÓMEZ, O. PASTOR, M. R. SANCHO et E. TENIENTE, éd., *Conceptual Modeling Perspectives*, Springer International Publishing, Cham, 2017, ISBN 978-3-319-67270-0 978-3-319-67271-7, doi:[10.1007/978-3-319-67271-7](https://doi.org/10.1007/978-3-319-67271-7).
- [46] J. CADAVID, B. COMBEMALE et B. BAUDRY, *Ten years of Meta-Object Facility : an analysis of metamodeling practices*, Thèse de doctorat, INRIA, 2012.
- [47] I. CANDELA, G. BAVOTA, B. RUSSO et R. OLIVETO, « Using Cohesion and Coupling for Software Remodularization : Is It Enough? », *ACM Transactions*

## BIBLIOGRAPHIE

- on Software Engineering and Methodology*, vol. 25(3), août 2016, p. 1–28, ISSN 1049-331X, 1557-7392, doi:[10.1145/2928268](https://doi.org/10.1145/2928268).
- [48] G. CANFORA, M. DI PENTA et L. CERULO, « Achievements and challenges in software reverse engineering », *Communications of the ACM*, vol. 54(4), avril 2011, p. 142–151, ISSN 0001-0782, 1557-7317, doi:[10.1145/1924421.1924451](https://doi.org/10.1145/1924421.1924451).
- [49] G. CAPLAT, *Modèles et métamodèles*, Collection des sciences appliquées de l'INSA de Lyon, Presses polytechniques et universitaires romandes, Lausanne (Suisse), 2008, ISBN 978-2-88074-749-7.
- [50] L. F. CAPRETZ, « A brief history of the object-oriented approach », *ACM SIGSOFT Software Engineering Notes*, vol. 28(2), mars 2003, p. 6, ISSN 0163-5948, doi:[10.1145/638750.638778](https://doi.org/10.1145/638750.638778).
- [51] A. CAUSEVIC, D. SUNDMARK et S. PUNNEKKAT, « An Industrial Survey on Contemporary Aspects of Software Testing », dans « 2010 Third International Conference on Software Testing, Verification and Validation », IEEE, Paris, France, 2010, ISBN 978-1-4244-6435-7, p. 393–401, doi:[10.1109/ICST.2010.52](https://doi.org/10.1109/ICST.2010.52).
- [52] D. CAVAR, Z. TIGANJ, L. V. MOMPELAT et B. DICKSON, « Computing Ellipsis Constructions : Comparing Classical NLP and LLM Approaches », *Society for Computation in Linguistics*, vol. 7(1), 2024.
- [53] W. CHAMI, *Analyse de l'évolution des concepts de cohésion et de couplage : 1979-2008*, Thèse de doctorat, Université du Québec à Montréal, 2008.
- [54] D. CHAPPELL *et al.*, « What is application lifecycle management », *Chappell & Associates*, 2008, p. 8.
- [55] M. CHINOSI et A. TROMBETTA, « BPMN : An introduction to the standard », *Computer Standards & Interfaces*, vol. 34(1), janvier 2012, p. 124–134, ISSN 09205489, doi:[10.1016/j.csi.2011.06.002](https://doi.org/10.1016/j.csi.2011.06.002).
- [56] M.-H. CHIU et J.-W. LIN, « Modeling competence in science education », *Disciplinary and Interdisciplinary Science Education Research*, vol. 1(1), décembre 2019, p. 12, ISSN 2662-2300, doi:[10.1186/s43031-019-0012-y](https://doi.org/10.1186/s43031-019-0012-y).

## BIBLIOGRAPHIE

- [57] F. CICCOCCHI, M. FAMELIS, G. KAPPEL, L. LAMBERS, S. MOSSER, R. F. PAIGE *et al.*, « Towards a body of knowledge for model-based software engineering », dans « Proceedings of the 21st ACM/IEEE International Conference on Model Driven Engineering Languages and Systems : Companion Proceedings », 2018, ISBN 978-1-4503-5965-8, p. 82–89, doi:[10.1145/3270112.3270121](https://doi.org/10.1145/3270112.3270121).
- [58] O. CICO, L. JACCHERI, A. NGUYEN-DUC et H. ZHANG, « Exploring the intersection between software industry and Software Engineering education - A systematic mapping of Software Engineering Trends », *Journal of Systems and Software*, vol. 172, février 2021, p. 110736, ISSN 0164-1212, doi:[10.1016/j.jss.2020.110736](https://doi.org/10.1016/j.jss.2020.110736). Publisher : Elsevier BV.
- [59] D. CIRIC LALIC, B. LALIC, M. DELIĆ, D. GRACANIN et D. STEFANOVIC, « How project management approach impact project success? From traditional to agile », *International Journal of Managing Projects in Business*, vol. 15(3), avril 2022, p. 494–521, ISSN 1753-8378, doi:[10.1108/IJMPB-04-2021-0108](https://doi.org/10.1108/IJMPB-04-2021-0108).
- [60] CLEMENTS, *Documenting software architectures : views and beyond*, 2<sup>e</sup> éd., SEI series in software engineering, Addison-Wesley, Upper Saddle River, NJ, 2014, ISBN 978-0-321-55268-6.
- [61] K. B. CODUR et A. H. DOGRU, « Regulations and software evolution : An example from the military domain », *Science of Computer Programming*, vol. 77(5), mai 2012, p. 636–643, ISSN 01676423, doi:[10.1016/j.scico.2011.12.001](https://doi.org/10.1016/j.scico.2011.12.001).
- [62] F. N. COLAKOGLU, A. YAZICI et A. MISHRA, « Software Product Quality Metrics : A Systematic Mapping Study », *IEEE Access*, vol. 9, 2021, p. 44647–44670, ISSN 2169-3536, doi:[10.1109/ACCESS.2021.3054730](https://doi.org/10.1109/ACCESS.2021.3054730).
- [63] R. COLOMO-PALACIOS, C. CASADO-LUMBRERAS, S. MISRA et P. SOTO-ACOSTA, « Career Abandonment Intentions among Software Workers », *Human Factors and Ergonomics in Manufacturing & Service Industries*, vol. 24(6), novembre 2014, p. 641–655, ISSN 1090-8471, 1520-6564, doi:[10.1002/hfm.20509](https://doi.org/10.1002/hfm.20509).

## BIBLIOGRAPHIE

- [64] COMMITTEE ON THE GROWTH OF COMPUTER SCIENCE UNDERGRADUATE ENROLLMENTS, BOARD ON HIGHER EDUCATION AND WORKFORCE, POLICY AND GLOBAL AFFAIRS, COMPUTER SCIENCE AND TELECOMMUNICATIONS BOARD, DIVISION ON ENGINEERING AND PHYSICAL SCIENCES et NATIONAL ACADEMIES OF SCIENCES, ENGINEERING, AND MEDICINE, « Assessing and Responding to the Growth of Computer Science Undergraduate Enrollments », mars 2018. ISBN 978-0-309-46702-5, pages : 24926, doi:[10.17226/24926](https://doi.org/10.17226/24926).
- [65] A.-P. CONTANDRIOPOULOS, *Savoir préparer une recherche : la définir, la structurer, la financer*, Presses de l'Université de Montréal, 1990, ISBN 978-2-7606-1535-9. OCLC : 977023789.
- [66] F. F. CORREIA, A. AGUIAR, H. S. FERREIRA et N. FLORES, « Patterns for consistent software documentation », dans « Proceedings of the 16th Conference on Pattern Languages of Programs », ACM, Chicago Illinois USA, août 2009, ISBN 978-1-60558-873-5, p. 1–7, doi:[10.1145/1943226.1943241](https://doi.org/10.1145/1943226.1943241).
- [67] J. J. DARROW, J. AVORN et A. S. KESSELHEIM, « FDA Regulation and Approval of Medical Devices : 1976-2020 », *JAMA*, vol. 326(5), août 2021, p. 420, ISSN 0098-7484, doi:[10.1001/jama.2021.11171](https://doi.org/10.1001/jama.2021.11171).
- [68] C. J. DATE, *The new relational database dictionary : a comprehensive glossary of the concepts arising in connection with the relational model of data, with definitions and illustrative examples*, O'Reilly Media, Inc, Sebastopol, CA, 2016, ISBN 978-1-4919-5173-6.
- [69] P. DAVIDSSON, F. KLÜGL et H. VERHAGEN, « Simulation of complex systems », dans « Springer Handbook of Model-Based Science », Springer, 2017, ISBN 978-3-319-30526-4, p. 783–797, doi:[10.1007/978-3-319-30526-4](https://doi.org/10.1007/978-3-319-30526-4).
- [70] A. DAVIS, S. OVERMYER, K. JORDAN, J. CARUSO, F. DANDASHI, A. DINH *et al.*, « Identifying and measuring quality in a software requirements specification », dans « [1993] Proceedings First International Software Metrics Symposium », IEEE, 1993, p. 141–152.

## BIBLIOGRAPHIE

- [71] D. DAY, M. PRIESTLEY et D. SCHELL, « Introduction to the darwin information typing architecture », *IBM corporation*, 2005.
- [72] J. B. DE VASCONCELOS, C. KIMBLE, P. CARRETEIRO et A. ROCHA, « The application of knowledge management to software evolution », *International Journal of Information Management*, vol. 37(1), février 2017, p. 1499–1506, ISSN 02684012, doi:[10.1016/j.ijinfomgt.2016.05.005](https://doi.org/10.1016/j.ijinfomgt.2016.05.005).
- [73] J.-P. DELAHAYE, *Aux frontières des mathématiques : Kurt Godel et l'incomplétude*, Dunod, 2025, ISBN 978-2100883912.
- [74] R. DI COSMO et S. ZACCHIROLI, « Software Heritage : Why and How to Preserve Software Source Code. In iPRES 2017-14th International Conference on Digital Preservation. Kyoto, Japan, 1–10 », 2017.
- [75] C. E. DICKERSON et D. MAVRIS, « A brief history of models and model based systems engineering and the case for relational orientation », *IEEE Systems Journal*, vol. 7(4), 2013, p. 581–592.
- [76] A. M. DIMA et M. A. MAASSEN, « From Waterfall to Agile software : Development models in the IT sector, 2006 to 2018. Impacts on company management. », *Journal of International Studies (2071-8330)*, vol. 11(2), 2018, doi:[10.14254/20718330.2018/11-2/21](https://doi.org/10.14254/20718330.2018/11-2/21).
- [77] DOD, *MIL-STD-498 Military Standard : Software Development and Documentation* :, DOD, Fort Belvoir, VA, décembre 1994, doi:[10.21236/ADA291266](https://doi.org/10.21236/ADA291266).
- [78] S. M. DOWNES, *Models and Modeling in the Sciences : A Philosophical Introduction*, 1<sup>re</sup> éd., Routledge, juillet 2020, ISBN 978-1-315-64745-6, doi:[10.4324/9781315647456](https://doi.org/10.4324/9781315647456).
- [79] DRUIDE INFORMATIQUE INC., *Antidote 10 (Version 6.3)*, Druide informatique Inc., Montréal, QC, Canada, 2021.
- [80] S. EASTERBROOK, R. LUTZ, R. COVINGTON, J. KELLY, Y. AMPO et D. HAMILTON, « Experiences using lightweight formal methods for requirements mo-

## BIBLIOGRAPHIE

- deling », *IEEE Transactions on Software Engineering*, vol. 24(1), 1998, p. 4–14, doi:[10.1109/32.663994](https://doi.org/10.1109/32.663994).
- [81] N. U. EISTY, G. K. THIRUVATHUKAL et J. C. CARVER, « A Survey of Software Metric Use in Research Software Development », dans « 2018 IEEE 14th International Conference on e-Science (e-Science) », IEEE, Amsterdam, octobre 2018, ISBN 978-1-5386-9156-4, p. 212–222, doi:[10.1109/eScience.2018.00036](https://doi.org/10.1109/eScience.2018.00036).
- [82] R. ELMASRI et S. B. NAVATHE, *Fundamentals of database systems seventh edition*, pearson, 2016, ISBN 978-0-13-397077-7.
- [83] ERIC WEISSTEIN AT WOLFRAM RESEARCH, « Abstract Machine », site web, 2025.  
Récupéré de <https://mathworld.wolfram.com/AbstractMachine.html> (consulté le 2025-10-16).
- [84] R. E. FAIRLEY, *Managing and Leading Software Projects*, IEEE Computer Society publications, Wiley [u.a.], Hoboken, NJ, 2009, ISBN 978-0-470-29455-0.
- [85] R. E. D. FAIRLEY, P. BOURQUE et J. KEPPLER, « The impact of SWEBOK Version 3 on software engineering education and training », dans « 2014 IEEE 27th Conference on Software Engineering Education and Training (CSEE&T) », IEEE, Klagenfurt, Austria, avril 2014, ISBN 978-1-4799-4970-0, p. 192–200, doi:[10.1109/CSEET.2014.6816804](https://doi.org/10.1109/CSEET.2014.6816804).
- [86] F. FARAZI, C. CHAPMAN, P. RAJU et W. BYRNE, « Model-based documentation », *Encyclopedia with Semantic Computing and Robotic Intelligence*, vol. 01(01), mars 2017, p. 1630 009, ISSN 2529-7376, 2529-7392, doi:[10.1142/S2425038416300093](https://doi.org/10.1142/S2425038416300093).
- [87] A. M. FERNANDEZ-SAEZ, D. CAIVANO, M. GENERO et M. R. CHAUDRON, « On the use of UML documentation in software maintenance : Results from a survey in industry », dans « 2015 ACM/IEEE 18th International Conference on Model Driven Engineering Languages and Systems (MODELS) », IEEE, Ottawa, ON, Canada, septembre 2015, ISBN 978-1-4673-6908-4, p. 292–301, doi:[10.1109/MODELS.2015.7338260](https://doi.org/10.1109/MODELS.2015.7338260).

## BIBLIOGRAPHIE

- [88] A. FORWARD et T. C. LETHBRIDGE, « The relevance of software documentation, tools and technologies : a survey », dans « Proceedings of the 2002 ACM symposium on Document engineering - DocEng '02 », ACM Press, McLean, Virginia, USA, 2002, ISBN 978-1-58113-594-7, p. 26, doi:[10.1145/585058.585065](https://doi.org/10.1145/585058.585065).
- [89] M. FOWLER, *Domain-specific languages*, The Addison-Wesley signature series, Addison-Wesley, 2011, ISBN 978-0-321-71294-3 978-0-13-210754-9.
- [90] R. FRIGG, *Models and Theories : A Philosophical Inquiry*, 1<sup>re</sup> éd., Routledge, London, juin 2022, ISBN 978-1-003-28510-6, doi:[10.4324/9781003285106](https://doi.org/10.4324/9781003285106).
- [91] A. FUGGETTA, « A classification of CASE technology », *Computer*, vol. 26(12), décembre 1993, p. 25–38, ISSN 0018-9162, doi:[10.1109/2.247645](https://doi.org/10.1109/2.247645).
- [92] A. FUGGETTA et E. DI NITTO, « Software process », dans « Future of Software Engineering Proceedings », ACM, Hyderabad India, mai 2014, ISBN 978-1-4503-2865-4, p. 1–12, doi:[10.1145/2593882.2593883](https://doi.org/10.1145/2593882.2593883).
- [93] K. FUJII, Y. TAJIMA, S. MIZUKI, M. KAWAMURA, H. SHIMADA, T. SHIOTANI *et al.*, « Rewriting pre-training data boosts llm performance in math and code », 2025. Non publié, doi:[10.48550/arXiv.2505.02881](https://doi.org/10.48550/arXiv.2505.02881).
- [94] U. FURBACH et N. SHANKAR, *Automated Reasoning : Third International Joint Conference, IJCAR 2006, Seattle, WA, USA, August 17-20, 2006, Proceedings*, vol. 4130, Springer, 2006, ISBN 978-3-540-37187-8 978-3-540-37188-5, doi:[10.1007/11814771](https://doi.org/10.1007/11814771).
- [95] D. M. GABBAY, « What is a logical system? », dans « What is a Logical System? », Oxford University Press Oxford, 1994, ISBN 0 19 853859 6, p. 179–216.
- [96] V. GAROUSI, G. GIRAY, E. TÜZÜN, C. CATAL et M. FELDERER, « Aligning software engineering education with industrial needs : A meta-analysis », *Journal of Systems and Software*, vol. 156, 2019, p. 65–83, doi:[10.1016/j.jss.2019.06.044](https://doi.org/10.1016/j.jss.2019.06.044).

## BIBLIOGRAPHIE

- [97] J. GAUGER, M. M. ART et E. SONDERGELD, « Legacy Systems and Modernization : Core Systems Strategy for Policy Administration Systems. », 2013.  
Récupéré de <https://www2.deloitte.com/content/dam/Deloitte/us/Documents/financialservices/us-fsi-legacy-systems-and-modernization.pdf> (consulté le 2026-06-01).
- [98] L. GAZIT, M. GHAFARI et A. SAXENA, *Mastering NLP from foundations to LLMs : apply advanced rule-based techniques to LLMs and solve real-world business problems using Python*, packt, Birmingham, 2024, ISBN 978-1-80461-918-6 978-1-80461-638-3.
- [99] M. GEIGER, S. HARRER, J. LENHARD et G. WIRTZ, « BPMN 2.0 : The state of support and implementation », *Future Generation Computer Systems*, vol. 80, mars 2018, p. 250–262, ISSN 0167739X, doi:[10.1016/j.future.2017.01.006](https://doi.org/10.1016/j.future.2017.01.006).
- [100] V. GERVASI et B. NUSEIBEH, « Lightweight validation of natural language requirements », *Software : Practice and Experience*, vol. 32(2), 2002, p. 113–133, doi:[10.1002/spe.430](https://doi.org/10.1002/spe.430).
- [101] Y. GINGRAS, « Qu’est-ce qu’une connaissance ? », *Pour la Science*, vol. 579(1), 2026, p. 62–63.
- [102] M. GLASSER, *Open Verification Methodology Cookbook*, Springer New York, New York, NY, 2009, ISBN 978-1-4419-0967-1 978-1-4419-0968-8, doi:[10.1007/978-1-4419-0968-8](https://doi.org/10.1007/978-1-4419-0968-8).
- [103] M. GLEIRSCHER et D. MARMSOLER, « Formal methods in dependable systems engineering : a survey of professionals from Europe and North America », *Empirical Software Engineering*, vol. 25(6), 2020, p. 4473–4546, doi:[10.1007/s10664-020-09836-5](https://doi.org/10.1007/s10664-020-09836-5).
- [104] I. GOODFELLOW, Y. BENGIO et A. COURVILLE, *Deep learning*, Adaptive computation and machine learning, The MIT press, Cambridge, Mass, 2016, ISBN 978-0-262-03561-3.



## BIBLIOGRAPHIE

- [105] T. GORSCHKE, *Requirements Engineering for Safety-Critical Systems*, 1<sup>re</sup> éd., River Publishers Series in Software Engineering Ser, River Publishers, Aalborg, 2022, ISBN 978-87-7022-427-7 978-1-000-79319-2.
- [106] G. GOTH, « Agile Tool Market Growing with the Philosophy », *IEEE Software*, vol. 26(2), mars 2009, p. 88–91, ISSN 0740-7459, doi:[10.1109/MS.2009.30](https://doi.org/10.1109/MS.2009.30).
- [107] M. GÖTHE, *Collaborative application lifecycle management with IBM rational products*, IBM Redbooks, 2008, ISBN 0738431974.
- [108] D. GRAHAM, R. BLACK et E. VAN VEENENDAAL, *Foundations of software testing : ISTQB certification*, 4<sup>e</sup> éd., Cengage Learning EMEA, Andover, England, 2019, ISBN 978-1473764798.
- [109] B. E. GRANGER et F. PÉREZ, « Jupyter : Thinking and storytelling with code and data », *Computing in Science & Engineering*, vol. 23(2), 2021, p. 7–14, doi:[10.1109/MCSE.2021.3059263](https://doi.org/10.1109/MCSE.2021.3059263).
- [110] V. GRUHN, « Process-centered software engineering environments, a brief history and future challenges », *Annals of Software Engineering*, vol. 14(1), 2002, p. 363–382.
- [111] D. GRUNE, K. VAN REEUWIJK, H. E. BAL, C. J. H. JACOBS et K. LANGENDOEN, *Modern compiler design*, 2<sup>e</sup> éd., Springer, New York, NY, 2012, ISBN 978-1-4614-4699-6, doi:[10.1007/978](https://doi.org/10.1007/978).
- [112] T. HAIGH, M. PRIESTLEY et C. ROPE, « Reconsidering the stored-program concept », *IEEE Annals of the History of Computing*, vol. 36(1), 2013, p. 4–17, doi:[10.1109/MAHC.2013.56](https://doi.org/10.1109/MAHC.2013.56).
- [113] T. HAIGH, M. PRIESTLEY et C. ROPE, « Engineering "The Miracle of the ENIAC" : implementing the modern code paradigm », *IEEE Annals of the History of Computing*, vol. 36(2), 2014, p. 41–59, doi:[10.1109/MAHC.2014.15](https://doi.org/10.1109/MAHC.2014.15).
- [114] T. S. HEINZE, V. STEFANKO et W. AMME, « Mining BPMN Processes on GitHub for Tool Validation and Development », dans « Enterprise,

## BIBLIOGRAPHIE

- Business-Process and Information Systems Modeling », édité par S. NURCAN, I. REINHARTZ-BERGER, P. SOFFER et J. ZDRAVKOVIC, vol. 387, Springer International Publishing, Cham, 2020, ISBN 978-3-030-49417-9 978-3-030-49418-6, p. 193–208, doi:[10.1007/978-3-030-49418-6\\_13](https://doi.org/10.1007/978-3-030-49418-6_13). Series Title : Lecture Notes in Business Information Processing.
- [115] S. D. HESTER, D. L. PARNAS et D. F. UTTER, « Using Documentation as a Software Design Medium », *Bell System Technical Journal*, vol. 60(8), octobre 1981, p. 1941–1977, ISSN 00058580, doi:[10.1002/j.1538-7305.1981.tb00304.x](https://doi.org/10.1002/j.1538-7305.1981.tb00304.x).
- [116] B. HETZEL, *Making software measurement work : Building an effective measurement program*, John Wiley & Sons, Inc., 1993, ISBN 978-0-89435-465-6.
- [117] V. HILDERMAN, *The Aviation Development Ecosystem : applying Do-178C, ARP 4754A, DO-254, & related guidelines*, Aviation Press International, Los Angeles, California, 2021, ISBN 978-1-950336-16-6 978-1-950336-17-3.
- [118] A. HIRA et B. BOEHM, « Function Point Analysis for Software Maintenance », dans « Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement », ACM, 2016, ISBN 978-1-4503-4427-2, p. 1–6, doi:[10.1145/2961111.2962613](https://doi.org/10.1145/2961111.2962613).
- [119] T. HOARE, « Hints on Programming Language Design », dans « Programming Languages », Springer Berlin Heidelberg, Berlin, Heidelberg, 1983, ISBN 978-3-662-09509-6 978-3-662-09507-2, p. 31–40, doi:[10.1007/978-3-662-09507-2\\_3](https://doi.org/10.1007/978-3-662-09507-2_3).
- [120] J. HOLT, *SysML for Systems Engineering : A Model-Based Approach*, 3<sup>e</sup> éd., Computing and Networks Ser, Institution of Engineering & Technology, 2019, ISBN 978-1-78561-554-2 978-1-78561-555-9.
- [121] J. HOLVITIE, S. A. LICORISH, R. O. SPÍNOLA, S. HYRYNSALMI, S. G. MACDONELL, T. S. MENDES *et al.*, « Technical debt and agile software development practices and processes : An industry practitioner survey », *Information and Software Technology*, vol. 96, avril 2018, p. 141–160, ISSN 09505849, doi:[10.1016/j.infsof.2017.11.015](https://doi.org/10.1016/j.infsof.2017.11.015).

## BIBLIOGRAPHIE

- [122] J. E. HOPCROFT, R. MOTWANI et J. D. ULLMAN, *Introduction to automata theory, languages, and computation*, 3<sup>e</sup> éd., Pearson, Boston San Francisco Munich, 2007, ISBN 978-0-321-45536-9.
- [123] E. HOROWITZ, éd., *Practical strategies for developing large software systems*, Addison-Wesley Pub. Co, Reading, Mass, 1975, ISBN 978-0-201-02977-2.
- [124] W. HSIEH, Z. BI, C. JIANG, J. LIU, B. PENG, S. ZHANG *et al.*, « A Comprehensive Guide to Explainable AI : From Classical Models to LLMs », décembre 2024. Non publié, doi:[10.48550/arXiv.2412.00800](https://doi.org/10.48550/arXiv.2412.00800).
- [125] G. HUNTER, *Metalogic : An introduction to the metatheory of standard first order logic*, Univ of California Press, 1973, ISBN 0-520-01822-2.
- [126] M. D. HYMAN, « Chomsky between revolutions », dans « Chomskyan (R) evolutions », John Benjamins Publishing Company, 2010, p. 265–298, doi:[10.1075/z.154.09hym](https://doi.org/10.1075/z.154.09hym).
- [127] ISO, « 26514-2021 - ISO/IEC/IEEE International Standard - Systems and software engineering – Design and development of information for users », 2022. ISBN 978-1-5044-8249-3.
- [128] ISO et IEC, « Standardization and related activities – General vocabulary », 2004.  
Récupéré de <https://www.iso.org/standard/39976.html> Guide providing general terms and definitions concerning standardization.
- [129] ISO et IEC, « Software engineering. COSMIC : A functional size measurement method », 2011. ISBN 978-0-580-68679-5, OCLC : 9521559510.
- [130] ISO, IEC et IEEE, « Software engineering — Product quality Part 1 : Quality model », 2001.
- [131] ISO, IEC et IEEE, « Software engineering — Product quality Part 2 : External metrics », 2003.

## BIBLIOGRAPHIE

- [132] ISO, IEC et IEEE, « Software engineering — Product quality Part 3 : Internal metrics », 2003.
- [133] ISO, IEC et IEEE, « Software engineering — Product quality Part 4 : Quality in use metrics », 2004.
- [134] ISO, IEC et IEEE, « Information technology – Document Schema Definition Languages (DSDL) – Part 11 : Schema association », 2011. Defines xml-model processing instruction for associating XML documents with external schema languages.
- [135] ISO, IEC et IEEE, « ISO/IEC/IEEE 15289 :2011 Systems and software engineering – Content of life-cycle information products (documentation) », 2011. ISBN 978-0-7381-6589-9.
- [136] ISO, IEC et IEEE, « ISO/IEC 19506 : 2012(E) : ISO/IEC Information technology – Object Management Group Architecture-Driven Modernization (ADM) – Knowledge Discovery Meta-Model (KDM). Standard. International Organization for Standardization, Geneva, CH », 2012.
- [137] ISO, IEC et IEEE, « IEC 61513 :2013 Nuclear power plants. Instrumentation and control important to safety. General requirements for systems », 2013. ISBN 978-0-580-76695-4, oCLC : 9521589556.
- [138] ISO, IEC et IEEE, « Systems and software engineering : system life cycle processes », 2015. ISBN 978-92-67-21161-9.
- [139] ISO, IEC et IEEE, « ISO/IEC/IEEE 12207 :2017, First Edition : Systems and software engineering - Software life cycle processes », 2017. ISBN 978-92-67-11029-5.
- [140] ISO, IEC et IEEE, « ISO/IEC/IEEE 24765 : 2017(E) : ISO/IEC/IEEE International Standard - Systems and software engineering–Vocabulary », 2017. ISBN 978-1-5044-4118-6.

## BIBLIOGRAPHIE

- [141] ISO, IEC et IEEE, « 29148-2018 - ISO/IEC/IEEE International Standard - Systems and software engineering – Life cycle processes – Requirements engineering », 2018. ISBN 978-1-5044-5302-8.
- [142] ISO, IEC et IEEE, « Information security, cybersecurity and privacy protection — Information security management systems Requirements », 2022.
- [143] ISO, IEC et IEEE, « Information technology - Artificial intelligence - Artificial intelligence concepts and terminology », 2022.
- [144] ISO, IEC et IEEE, « ISO/IEC/IEEE 42010 :2022 Systems and software engineering – Architecture description », <https://www.iso.org/standard/74393.html>, 2022. ISBN 978-1-5044-9155-6 STD25753, international standard.
- [145] D. JACKSON, *Software Abstractions : logic, language, and analysis*, MIT press, 2012, ISBN 978-0262528900.
- [146] M. JACKSON, *CASE Tools and Development Methods*, chap. 8, Wiley, Chichester, England ; New York, 1990, ISBN 978-0-471-92893-5, p. 1.
- [147] M. JACKSON, *Software Requirements And Specifications*, chap. Models, Addison Wesley, Boston, MA, 1995, ISBN 978-0201877120, p. 228.
- [148] M. JACKSON, *The World and the Machine*, chap. 2, Good Friends Publishing Company, 2010, ISBN 978-1257175222, p. 373–391.
- [149] A. JANSEN, P. AVGERIOU et J. S. VAN DER VEN, « Enriching software architecture documentation », *Journal of Systems and Software*, vol. 82(8), août 2009, p. 1232–1248, ISSN 01641212, doi:[10.1016/j.jss.2009.04.052](https://doi.org/10.1016/j.jss.2009.04.052).
- [150] JEAN-FRANCOIS MONIN, *Understanding Formal Methods*, Springer London, 2003, doi:[10.1007/978-1-4471-0043-0](https://doi.org/10.1007/978-1-4471-0043-0).
- [151] R. JELLIFFE, « Schemas do not imply any semantics of documents », site web, 2017.  
Récupéré de [https://schematron.com/opinion/\\_schemas\\_do\\_not](https://schematron.com/opinion/_schemas_do_not)

## BIBLIOGRAPHIE

- [\\_imply\\_any\\_semantics\\_of\\_documents\\_.html](#) (consulté le 2026-03-10).
- [152] S. JHA, S. K. JHA, P. LINCOLN, N. D. BASTIAN, A. VELASQUEZ et S. NEEMA, « Dehallucinating large language models using formal methods guided iterative prompting », dans « 2023 IEEE International Conference on Assured Autonomy (ICAA) », IEEE, 2023, p. 149–152, doi:[10.1109/ICAA58325.2023.00029](#).
- [153] P. JOHNSTON et R. HARRIS, « The Boeing 737 MAX saga : lessons for software organizations », *Software Quality Professional*, vol. 21(3), 2019, p. 4–12.
- [154] A. KACOFEGITIS et N. CHURCHER, « Theme-based literate programming », dans « Ninth Asia-Pacific Software Engineering Conference, 2002. », IEEE Comput. Soc, Gold Coast, Qld., Australia, 2002, ISBN 978-0-7695-1850-3, p. 549–557, doi:[10.1109/APSEC.2002.1183079](#).
- [155] N. KAHANI, M. BAGHERZADEH, J. R. CORDY, J. DINGEL et D. VARRÓ, « Survey and classification of model transformation tools », *Software & Systems Modeling*, vol. 18(4), août 2019, p. 2361–2397, ISSN 1619-1366, 1619-1374, doi:[10.1007/s10270-018-0665-6](#).
- [156] N. KAHANI, M. BAGHERZADEH, J. DINGEL et J. R. CORDY, « The problems with eclipse modeling tools : a topic analysis of eclipse forums », dans « Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems », 2016, ISBN 978-1-4503-4321-3, p. 227–237, doi:[10.1145/2976767.2976773](#).
- [157] S. KANDAR, *Introduction to automata theory, formal languages and computation*, Pearson Education India, 2013, ISBN 9789332516328.
- [158] J. I. KANTOLA, S. NAZIR et T. BARATH, éd., *Advances in Human Factors, Business Management and Society : Proceedings of the AHFE 2018 International Conference on Human Factors, Business Management and Society, July 21-25, 2018, Loews Sapphire Falls Resort at Universal Studios, Orlando, Florida*,

## BIBLIOGRAPHIE

- USA, Advances in Intelligent Systems and Computing*, vol. 783, Springer International Publishing, Cham, 2019, ISBN 978-3-319-94708-2 978-3-319-94709-9, doi:[10.1007/978-3-319-94709-9](https://doi.org/10.1007/978-3-319-94709-9).
- [159] M. KASSAB et P. LAPLANTE, « The Current and Evolving Landscape of Requirements Engineering in Practice », *IEEE Software*, vol. 39(5), septembre 2022, p. 76–83, ISSN 0740-7459, 1937-4194, doi:[10.1109/ms.2022.3147692](https://doi.org/10.1109/ms.2022.3147692). Publisher : Institute of Electrical and Electronics Engineers (IEEE).
- [160] D. KHURANA, A. KOLI, K. KHATTER et S. SINGH, « Natural language processing : state of the art, current trends and challenges », *Multimedia tools and applications*, vol. 82(3), 2023, p. 3713–3744, doi:[10.1007/s11042-022-13428-4](https://doi.org/10.1007/s11042-022-13428-4).
- [161] P. KILPELÄINEN et D. WOOD, « SGML and XML document grammars and exceptions », *Information and Computation*, vol. 169(2), 2001, p. 230–251, doi:[10.1006/inco.2000.2964](https://doi.org/10.1006/inco.2000.2964).
- [162] R. KNEUPER, *Software Processes and Life Cycle Models : An Introduction to Modelling, Using and Managing Agile, Plan-Driven and Hybrid Processes*, Springer International Publishing, Cham, 2018, ISBN 978-3-319-98844-3 978-3-319-98845-0, doi:[10.1007/978-3-319-98845-0](https://doi.org/10.1007/978-3-319-98845-0).
- [163] D. E. KNUTH, *Literate programming*, n° 27 dans CSLI lecture notes, Center for the Study of Language and Information, Stanford, Calif., 1992, ISBN 978-0-937073-80-3 978-0-937073-81-0.
- [164] T. KNUUTTILA, N. CARRILLO et R. KOSKINEN, *The Routledge Handbook of Philosophy of Scientific Modeling*, 1<sup>re</sup> éd., Routledge, London, août 2024, ISBN 978-1-003-20564-7, doi:[10.4324/9781003205647](https://doi.org/10.4324/9781003205647).
- [165] J. KROGSTIE, *Quality in Business Process Modeling*, Springer International Publishing, Cham, 2016, ISBN 978-3-319-42510-8 978-3-319-42512-2, doi:[10.1007/978-3-319-42512-2](https://doi.org/10.1007/978-3-319-42512-2).

## BIBLIOGRAPHIE

- [166] P. KRUCHTEN, R. L. NORD et I. OZKAYA, « Technical Debt : From Metaphor to Theory and Practice », *IEEE Software*, vol. 29(6), novembre 2012, p. 18–21, ISSN 0740-7459, doi:[10.1109/MS.2012.167](https://doi.org/10.1109/MS.2012.167).
- [167] J. KRÜGER, « To Memorize or to Document : A Survey of Developers' Views on Knowledge Availability », dans « Product-Focused Software Process Improvement : 24th International Conference, PROFES 2023, Dornbirn, Austria, December 10–13, 2023, Proceedings, Part I », édité par R. KADGIEN, A. JED-LITSCHKA, A. JANES, V. LENARDUZZI et X. LI, *Lecture Notes in Computer Science*, vol. 14483, Springer Nature Switzerland, Cham, 2024, ISBN 978-3-031-49265-5 978-3-031-49266-2, p. 39–56, doi:[10.1007/978-3-031-49266-2](https://doi.org/10.1007/978-3-031-49266-2).
- [168] H. KÜHN, « Metamodeling Platforms : Observations and Evolutions », dans « Metamodeling : Applications and Trajectories to the Future : Essays in Honor of Dimitris Karagiannis », Springer, 2024, p. 95–110, doi:[10.1007/978-3-031-56862-6\\_7](https://doi.org/10.1007/978-3-031-56862-6_7).
- [169] T. KUTZNER et J. GRÖPLER, « Supporting students in the creation of requirements and functional specifications in interdisciplinary software development projects with the help of AI-based text generation tools », 2023. ISBN 978-84-09-51520-2.
- [170] A. V. LAMSWEERDE, « Formal specification : a roadmap », dans « Proceedings of the Conference on the Future of Software Engineering », 2000, ISBN 978-1-58113-253-3, p. 147–159, doi:[10.1145/336512.336546](https://doi.org/10.1145/336512.336546).
- [171] K. LANO et S. Y. TEHRANI, *Introduction to Software Architecture : Innovative Design Using Clean Architecture and Model-driven Engineering*, Springer Nature, 2023, ISBN 978-3-031-44142-4 978-3-031-44143-1, doi:[10.1007/978-3-031-44143-1](https://doi.org/10.1007/978-3-031-44143-1).
- [172] LAROUSSE, « Adéquat », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/ad%c3%a9quat/1058> (consulté le 2025-10-16).



## BIBLIOGRAPHIE

- [173] LAROUSSE, « Adéquation », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/ad%C3%A9quation/1060> (consulté le 2025-10-16).
- [174] LAROUSSE, « Assertion », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/assertion/5806> (consulté le 2025-10-16).
- [175] LAROUSSE, « Axiome », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/axiome/7191> (consulté le 2025-10-16).
- [176] LAROUSSE, « Citation », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/citation/16228> (consulté le 2025-10-16).
- [177] LAROUSSE, « Cohésion », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/coh%C3%A9sion/17019> (consulté le 2025-10-16).
- [178] LAROUSSE, « Complet », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/complet/17678> (consulté le 2025-10-16).
- [179] LAROUSSE, « Complétude », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/compl%C3%A9tude/17687> (consulté le 2025-10-16).
- [180] LAROUSSE, « Composant », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/composant/17735#17605> (consulté le 2025-10-16).
- [181] LAROUSSE, « Compréhensible », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/compr%C3%A9hensible/17770> (consulté le 2025-10-16).

## BIBLIOGRAPHIE

- [182] LAROUSSE, « concret », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/concret/17968> (consulté le 2025-04-12).
- [183] LAROUSSE, « Conjecture », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/conjecture/18235> (consulté le 2025-10-16).
- [184] LAROUSSE, « Corollaire », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/corollaire/19369> (consulté le 2025-10-16).
- [185] LAROUSSE, « Coupler », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/coupler/19858> (consulté le 2025-10-16).
- [186] LAROUSSE, « Document », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/document/26265> (consulté le 2025-10-16).
- [187] LAROUSSE, « Développement », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/d%C3%A9veloppement/24951> (consulté le 2025-10-16).
- [188] LAROUSSE, « Informatique », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/informatique/42996> (consulté le 2025-10-16).
- [189] LAROUSSE, « Ordinateur », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/ordinateur/56358> (consulté le 2025-10-16).
- [190] LAROUSSE, « Phase », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/phase/60161> (consulté le 2025-10-16).

## BIBLIOGRAPHIE

- [191] LAROUSSE, « Professionnel », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/professionnel/64160> (consulté le 2025-10-16).
- [192] LAROUSSE, « Standard », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/standard/74452> (consulté le 2025-10-16).
- [193] LAROUSSE, « Tangible », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/tangible/76579> (consulté le 2025-10-16).
- [194] LAROUSSE, « Technique », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/technique/76949> (consulté le 2025-04-12).
- [195] LAROUSSE, « Théorème », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/th%c3%a9or%c3%a8me/77729> (consulté le 2025-10-16).
- [196] LAROUSSE, « Établir », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/%c3%a9tablir/31239> (consulté le 2025-10-16).
- [197] LAROUSSE, « Évolution », site web, 2025.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/%c3%a9volution/31900> (consulté le 2025-10-16).
- [198] LAROUSSE, « Appliqué », site web, 2026.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/appliqu%C3%A9/4709> (consulté le 2026-04-11).
- [199] LAROUSSE, « Donnée », site web, 2026.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/donn%c3%a9e/26436> (consulté le 2026-05-16).

## BIBLIOGRAPHIE

- [200] LAROUSSE, « Participant », site web, 2026.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/participant/58369> (consulté le 2026-04-11).
- [201] LAROUSSE, « Participer », site web, 2026.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/participer/58375> (consulté le 2026-04-11).
- [202] LAROUSSE, « Passif », site web, 2026.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/passif/58517> (consulté le 2026-04-11).
- [203] LAROUSSE, « Praticien », site web, 2026.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/praticien/63251> (consulté le 2026-04-11).
- [204] LAROUSSE, « Théorique », site web, 2026.  
Récupéré de <https://www.larousse.fr/dictionnaires/francais/th%C3%A9orique/77736> (consulté le 2026-04-11).
- [205] X. LARRUCEA, I. SANTAMARIA, R. V. O’CONNOR et R. MESSNARZ, éd., *Systems, Software and Services Process Improvement : 25th European Conference, EuroSPI 2018, Bilbao, Spain, September 5-7, 2018, Proceedings, Communications in Computer and Information Science*, vol. 896, chap. ALM Tool Infrastructure with a Focus on DevOps Culture, Springer International Publishing, Cham, 2018, ISBN 978-3-319-97924-3 978-3-319-97925-0, p. 291–303, doi:[10.1007/978-3-319-97925-0](https://doi.org/10.1007/978-3-319-97925-0).
- [206] T. LETHBRIDGE, J. SINGER et A. FORWARD, « How software engineers use documentation : the state of the practice », *IEEE Software*, vol. 20(6), novembre 2003, p. 35–39, ISSN 0740-7459, doi:[10.1109/MS.2003.1241364](https://doi.org/10.1109/MS.2003.1241364). Number : 6.
- [207] N. G. LEVESON, « The Therac-25 : 30 years later », *Computer*, vol. 50(11), 2017, p. 8–11, doi:[10.1109/MC.2017.4041349](https://doi.org/10.1109/MC.2017.4041349).

## BIBLIOGRAPHIE

- [208] N. G. LEVESON et C. S. TURNER, « An investigation of the Therac-25 accidents », *Computer*, vol. 26(7), 1993, p. 18–41, doi:[10.1109/MC.1993.274940](https://doi.org/10.1109/MC.1993.274940).
- [209] R. LI, P. LIANG, Y. WANG, Y. CAI, W. SUN et Z. LI, « Unveiling the Role of ChatGPT in Software Development : Insights from Developer-ChatGPT Interactions on GitHub », mai 2025. Non publié, doi:[10.48550/arXiv.2505.03901](https://doi.org/10.48550/arXiv.2505.03901).
- [210] A. M. MADNI, N. AUGUSTINE et M. SIEVERS, *Handbook of model-based systems engineering*, Springer Nature, 2023, ISBN 978-3-030-93582-5, doi:[10.1007/978-3-030-93582-5](https://doi.org/10.1007/978-3-030-93582-5).
- [211] I. MALAVOLTA, P. LAGO, H. MUCCINI, P. PELLICCIONE et A. TANG, « What Industry Needs from Architectural Languages : A Survey », *IEEE Transactions on Software Engineering*, vol. 39(6), juin 2013, p. 869–891, ISSN 0098-5589, 1939-3520, doi:[10.1109/TSE.2012.74](https://doi.org/10.1109/TSE.2012.74).
- [212] N. MARQUES, R. R. SILVA et J. BERNARDINO, « Using ChatGPT in Software Requirements Engineering : A Comprehensive Review », *Future Internet*, vol. 16(6), mai 2024, p. 180, ISSN 1999-5903, doi:[10.3390/fi16060180](https://doi.org/10.3390/fi16060180). Number : 6.
- [213] N. MASLEJ, L. FATTORINI, R. PERRAULT, Y. GIL, V. PARLI, N. KARIUKI *et al.*, « Artificial intelligence index report 2025 », 2025. Non publié.
- [214] N. MASLEJ, L. FATTORINI, R. PERRAULT, V. PARLI, A. REUEL, E. BRYNJOLFSSON *et al.*, « Artificial intelligence index report 2024 », 2024. Non publié.
- [215] T. MASSONI, R. DUARTE et R. OLIVEIRA, « Exit the Code : A Model for Understanding Career Abandonment Intention Among Software Developers », dans « 2025 IEEE/ACM 18th International Conference on Cooperative and Human Aspects of Software Engineering (CHASE) », IEEE, Ottawa, ON, Canada, avril 2025, ISBN 979-8-3315-3871-2, p. 1–11, doi:[10.1109/CHASE66643.2025.00009](https://doi.org/10.1109/CHASE66643.2025.00009).
- [216] P. W. MCBURNEY, « Automatic Documentation Generation via Source Code Summarization », dans « 2015 IEEE/ACM 37th IEEE International Conference

## BIBLIOGRAPHIE

- on Software Engineering », IEEE, Florence, Italy, mai 2015, ISBN 978-1-4799-1934-5, p. 903–906, doi:[10.1109/ICSE.2015.288](https://doi.org/10.1109/ICSE.2015.288).
- [217] C. McDONALD, « From Art Form to Engineering Discipline? A History of US Military Software Development Standards, 1974–1998 », *IEEE Annals of the History of Computing*, vol. 32(4), octobre 2010, p. 32–47, ISSN 1058-6180, doi:[10.1109/MAHC.2009.58](https://doi.org/10.1109/MAHC.2009.58).
- [218] B. MEHLENBACHER, « Documentation : not yet implemented, but coming soon », *The HCI handbook : Fundamentals, evolving technologies, and emerging applications*, 2003, p. 527–543.
- [219] M. MENG, S. STEINHARDT et A. SCHUBERT, « Application Programming Interface Documentation : What Do Software Developers Want ? », *Journal of Technical Writing and Communication*, vol. 48(3), 2018, p. 295–330, ISSN 0047-2816, 1541-3780, doi:[10.1177/0047281617721853](https://doi.org/10.1177/0047281617721853). Number : 3.
- [220] B. MEYER, *Agile! : The Good, the Hype and the Ugly*, Springer International Publishing, Cham, 2014, ISBN 978-3-319-05154-3 978-3-319-05155-0, doi:[10.1007/978-3-319-05155-0](https://doi.org/10.1007/978-3-319-05155-0).
- [221] O. MEYER, *Object-Oriented Software Construction*, 2<sup>e</sup> éd., Prentice-Hall, London, England, 1997.
- [222] MING LI et C. SMIDTS, « A ranking of software engineering measures based on expert opinion », *IEEE Transactions on Software Engineering*, vol. 29(9), septembre 2003, p. 811–824, ISSN 0098-5589, doi:[10.1109/TSE.2003.1232286](https://doi.org/10.1109/TSE.2003.1232286).
- [223] S. J. MORRIS et O. GOTEL, « Flow diagrams : rise and fall of the first software engineering notation », dans « International Conference on Theory and Application of Diagrams », Springer, 2006, p. 130–144.
- [224] G. MURPHY, D. NOTKIN et K. SULLIVAN, « Software reflexion models : bridging the gap between design and implementation », *IEEE Transactions on Software Engineering*, vol. 27(4), avril 2001, p. 364–380, ISSN 00985589, doi:[10.1109/32.917525](https://doi.org/10.1109/32.917525).

## BIBLIOGRAPHIE

- [225] P. A. MÉNARD et S. RATTE, « Concept extraction from business documents for software engineering projects », *Automated Software Engineering*, vol. 23(4), décembre 2016, p. 649–686, ISSN 0928-8910, 1573-7535, doi:[10.1007/s10515-015-0184-4](https://doi.org/10.1007/s10515-015-0184-4).
- [226] NASA, *Nasa - Software Measurement Guidebook*, NASA, 1995. SEL-94-102.
- [227] L. M. DO NASCIMENTO, D. L. VIANA, P. NETO, D. MARTINS, V. C. GARCIA et S. MEIRA, « A systematic mapping study on domain-specific languages », dans « The Seventh International Conference on Software Engineering Advances (ICSEA 2012) », Xpert Publishing Services, 2012, ISBN 978-1-61208-230-1, p. 179–187.
- [228] NATIONAL BUREAU STANDARD, « NBS 500 :87 Management Guide for Software Documentation », document pdf, 1987.  
Récupéré de <https://www.govinfo.gov/content/pkg/GOVPUB-C13-ddd47a2eca7877b72195f405e4a9fb8d/pdf/GOVPUB-C13-ddd47a2eca7877b72195f405e4a9fb8d.pdf> (consulté le 2025-04-12).
- [229] NATIONAL CENTER FOR SCIENCE AND ENGINEERING STATISTICS, « The National Survey of College Graduates », 2026.  
Récupéré de <https://ncesdata.nsf.gov> (consulté le 2026-05-15).
- [230] I. G. NDUKWE, S. A. LICORISH, A. TAHIR et S. G. MACDONELL, « How have views on Software Quality differed over time ? Research and practice viewpoints », *Journal of Systems and Software*, vol. 195, janvier 2023, p. 111524, ISSN 01641212, doi:[10.1016/j.jss.2022.111524](https://doi.org/10.1016/j.jss.2022.111524).
- [231] S. NIELEBOCK, D. KROLIKOWSKI, J. KRÜGER, T. LEICH et F. ORTMEIER, « Commenting source code : is it worth it for small programming tasks ? », *Empirical Software Engineering*, vol. 24(3), 2019, p. 1418–1457, ISSN 1382-3256, 1573-7616, doi:[10.1007/s10664-018-9664-z](https://doi.org/10.1007/s10664-018-9664-z). Number : 3.
- [232] K. NORMARK, M. ANDERSEN, C. CHRISTENSEN, V. KUMAR, S. STAUN-PEDERSEN et K. SORENSEN, « Elucidative Programming in Java », dans « 18th

## BIBLIOGRAPHIE

- Annual Conference on Computer Documentation. ipcc sigdoc 2000. Technology and Teamwork. Proceedings. IEEE Professional Communication Society International Professional Communication Conference and ACM Special Interest Group on Documentation Conferenc », IEEE, Cambridge, MA, USA, 2000, ISBN 978-0-7803-6431-8, p. 483–495, doi:[10.1109/IPCC.2000.887305](https://doi.org/10.1109/IPCC.2000.887305).
- [233] A. OLIVÉ, *Conceptual modeling of information systems*, Springer, 2007, ISBN 978-3-540-39389-4.
- [234] R. OLIVEIRA, T. MASSONI, N. ARAÚJO, C. SARMENTO et F. SANTOS, « Ants Doing Legwork : Investigating Motivators for Software Development Career Abandonment », dans « Brazilian Symposium on Software Engineering », ACM, Joinville Brazil, septembre 2021, ISBN 978-1-4503-9061-3, p. 353–362, doi:[10.1145/3474624.3474644](https://doi.org/10.1145/3474624.3474644).
- [235] OMG, « Business Process Model and Notation (BPMN) version 2.0.2 », document pdf, 2013.  
Récupéré de <https://www.omg.org/spec/BPMN> (consulté le 2025-04-12).
- [236] OQLF, « Abandon », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8874792/abandon> (consulté le 2025-04-12).
- [237] OQLF, « Apprentissage automatique », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8395061/apprentissage-automatique> (consulté le 2025-04-12).
- [238] OQLF, « Apprentissage profond », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/26532876/apprentissage-profond> (consulté le 2025-04-12).
- [239] OQLF, « Artéfact », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8367467/artefact> (consulté le 2025-04-12).



## BIBLIOGRAPHIE

- [240] OQLF, « Automate », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8435097/automate> (consulté le 2025-04-12).
- [241] OQLF, « Avocat, avocate », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8875479/avocat> (consulté le 2025-10-24).
- [242] OQLF, « Boîte noire », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8408432/boite-noire> (consulté le 2025-04-12).
- [243] OQLF, « Cohérence », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/26504552/coherence> (consulté le 2025-04-12).
- [244] OQLF, « Composant logiciel », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8385525/composant-logiciel> (consulté le 2025-04-12).
- [245] OQLF, « Consistance », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/26504550/consistance> (consulté le 2025-04-12).
- [246] OQLF, « convention », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8871842/convention> (consulté le 2025-04-12).
- [247] OQLF, « Cycle », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8370299/cycle> (consulté le 2025-04-12).
- [248] OQLF, « Cycle de vie du développement logiciel », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fi>

## BIBLIOGRAPHIE

- [che-gdt/fiche/3293325/cycle-de-vie-du-developpement-logiciel](#) (consulté le 2025-04-12).
- [249] OQLF, « Difference entre procédé, processus et procédure », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/22247/le-vocabulaire/nuances-semanticques/difference-entre-procede-processus-et-procedure> (consulté le 2025-04-12).
- [250] OQLF, « Documentation », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8369506/documentation> (consulté le 2025-04-12).
- [251] OQLF, « Définition », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/26500953/definition> (consulté le 2025-04-12).
- [252] OQLF, « Entité », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8354296/entite> (consulté le 2025-04-12).
- [253] OQLF, « Gestion des connaissances », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8392040/gestion-des-connaissances> (consulté le 2025-04-12).
- [254] OQLF, « Grand modèle de langage », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8390901/modele-de-langage> (consulté le 2025-04-12).
- [255] OQLF, « Informaticien », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8363786/informaticien> (consulté le 2025-04-12).

## BIBLIOGRAPHIE

- [256] OQLF, « Informatique », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/2071561/informatique> (consulté le 2025-04-12).
- [257] OQLF, « Ingénieur », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8349078/ingenieur> (consulté le 2025-04-12).
- [258] OQLF, « Ingénieur en logiciel », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/26500449/ingenieur-en-logiciels> (consulté le 2025-10-24).
- [259] OQLF, « Ingénieur informaticien », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8349068/ingenieur-informaticien> (consulté le 2025-10-24).
- [260] OQLF, « Langage », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8366883/langage> (consulté le 2025-04-12).
- [261] OQLF, « Langage formel », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8408168/langage-formel> (consulté le 2025-04-12).
- [262] OQLF, « Langage naturel », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8372907/langage-naturel> (consulté le 2025-04-12).
- [263] OQLF, « Langage spécifique à un domaine », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/26542376/langage-specifique-a-un-domaine> (consulté le 2025-04-12).

## BIBLIOGRAPHIE

- [264] OQLF, « Lectorat », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8418390/lectorat> (consulté le 2025-04-12).
- [265] OQLF, « Logiciel », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8355564/logiciel> (consulté le 2025-04-12).
- [266] OQLF, « Méta- », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/index.php?id=24340> (consulté le 2025-04-12).
- [267] OQLF, « Ontologie informatique », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/26561359/ontologie-informatique> (consulté le 2025-04-12).
- [268] OQLF, « Patrimonial », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8390423/patrimonial> (consulté le 2025-04-12).
- [269] OQLF, « Physicien », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/17023711/physicien> (consulté le 2025-04-12).
- [270] OQLF, « Processus », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/26503184/processus> (consulté le 2025-04-12).
- [271] OQLF, « Processus », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8377717/processus> (consulté le 2025-04-12).
- [272] OQLF, « Programme », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/2089188/programme> (consulté le 2025-04-12).

## BIBLIOGRAPHIE

- [273] OQLF, « Prédicat », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8393945/predicat> (consulté le 2025-04-12).
- [274] OQLF, « Rétro-ingénierie », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/26539952/retro-ingenierie> (consulté le 2025-04-1).
- [275] OQLF, « Savoir », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8366773/savoir> (consulté le 2025-04-12).
- [276] OQLF, « Savoir », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/26503727/savoir> (consulté le 2025-04-12).
- [277] OQLF, « Standard », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8383249/standard> (consulté le 2025-04-12).
- [278] OQLF, « Taxinomie », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8359505/taxinomie> (consulté le 2025-04-12).
- [279] OQLF, « Traitement automatique des langues », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8396637/traitement-automatique-des-langues> (consulté le 2025-04-1).
- [280] OQLF, « Échéancier », site web, 2025.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8871625/echeancier> (consulté le 2025-04-12).
- [281] OQLF, « Modèle transformateur », site web, 2026.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/>

## BIBLIOGRAPHIE

- [fiche-gdt/fiche/26579066/modele-transformeur](#) (consulté le 2026-04-12).
- [282] OQLF, « Praticien », site web, 2026.  
Récupéré de <https://vitrinelinguistique.oqlf.gouv.qc.ca/fiche-gdt/fiche/8470357/praticien> (consulté le 2026-04-12).
- [283] T. ÖREN, B. P. ZEIGLER et A. TOLK, *Body of knowledge for modeling and simulation : a handbook by the society for modeling and simulation international*, Springer, 2023, ISBN 978-3-031-11085-6, doi:[10.1007/978-3-031-11085-6](https://doi.org/10.1007/978-3-031-11085-6).
- [284] M. OZKAYA et D. AKDUR, « What do practitioners expect from the meta-modeling tools ? A survey », *Journal of Computer Languages*, vol. 63, 2021, p. 101 030, doi:[10.1016/j.cola.2021.101030](https://doi.org/10.1016/j.cola.2021.101030).
- [285] M. OZKAYA et C. KLOUKINAS, « Are We There Yet ? Analyzing Architecture Description Languages for Formal Analysis, Usability, and Realizability », dans « 2013 39th Euromicro Conference on Software Engineering and Advanced Applications », IEEE, Santander, Spain, septembre 2013, ISBN 978-0-7695-5091-6, p. 177–184, doi:[10.1109/SEAA.2013.34](https://doi.org/10.1109/SEAA.2013.34).
- [286] G. O'REGAN, *Mathematics in Computing : An Accessible Guide to Historical, Foundational and Application Contexts*, Undergraduate Topics in Computer Science, Springer International Publishing, Cham, 2020, ISBN 978-3-030-34208-1 978-3-030-34209-8, doi:[10.1007/978-3-030-34209-8](https://doi.org/10.1007/978-3-030-34209-8).
- [287] M. S. PALMQUIST, M. A. LAPHAM, S. MILLER, T. CHICK et I. OZKAYA, « Parallel Worlds : Agile and Waterfall Differences and Similarities », *Software Engineering Institute*, 2013, doi:[10.1184/R1/6576047.v1](https://doi.org/10.1184/R1/6576047.v1).
- [288] PARNAS, « Why software developers should be licensed », *Engineering Dimensions*, mai 2001.
- [289] D. PARNAS, « Education for computing professionals », *Computer*, vol. 23(1), janvier 1990, p. 17–22, ISSN 0018-9162, doi:[10.1109/2.48796](https://doi.org/10.1109/2.48796).

## BIBLIOGRAPHIE

- [290] D. PARNAS et L. CHIK-PARNAS, « Goals for software engineering student education », *ACM SIGSOFT Software Engineering Notes*, vol. 30(4), juillet 2005, p. 6–8, ISSN 0163-5948, doi:[10.1145/1082983.1082986](https://doi.org/10.1145/1082983.1082986).
- [291] D. PARNAS, P. CLEMENTS et D. WEISS, « The Modular Structure of Complex Systems », *IEEE Transactions on Software Engineering*, vol. SE-11(3), mars 1985, p. 259–266, ISSN 0098-5589, doi:[10.1109/TSE.1985.232209](https://doi.org/10.1109/TSE.1985.232209).
- [292] D. L. PARNAS, « The Professional Responsibilities of Software Engineers », *IFIP*, 1994.
- [293] D. L. PARNAS, « Really Rethinking 'Formal Methods' », *Computer*, vol. 43(1), janvier 2010, p. 28–34, ISSN 0018-9162, doi:[10.1109/MC.2010.22](https://doi.org/10.1109/MC.2010.22).
- [294] D. L. PARNAS, « Precise Documentation : The Key to Better Software », dans « The Future of Software Engineering », édité par S. NANZ, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011, ISBN 978-3-642-15186-6 978-3-642-15187-3, p. 125–148, doi:[10.1007/978-3-642-15187-3\\_8](https://doi.org/10.1007/978-3-642-15187-3_8).
- [295] D. L. PARNAS, « Software Engineering : A Profession in Waiting », *Computer*, vol. 54(5), mai 2021, p. 62–64, ISSN 0018-9162, 1558-0814, doi:[10.1109/MC.2021.3057685](https://doi.org/10.1109/MC.2021.3057685).
- [296] D. L. PARNAS, D. M. HOFFMAN et D. M. WEISS, *Software fundamentals : collected papers by David L. Parnas*, Addison-Wesley, Boston, 2001, ISBN 978-0-201-70369-6.
- [297] T. PARR, *The definitive ANTLR 4 reference*, The pragmatic programmers, The Pragmatic Bookshelf, 2012, ISBN 978-1-934356-99-9.
- [298] J. M. PERKEL, « By Jupyter, it all makes sense », *Nature*, vol. 563(7729), 2018, p. 145–146, doi:[10.1038/d41586-018-07196-1](https://doi.org/10.1038/d41586-018-07196-1).
- [299] A. PETTOROSSO, *Automata theory and formal languages : fundamental notions, theorems, and techniques*, Undergraduate topics in computer science, Springer, 2022, ISBN 978-3-031-11965-1 978-3-031-11964-4.

## BIBLIOGRAPHIE

- [300] M. PEZZÈ et M. YOUNG, *Software testing and analysis : process, principles, and techniques*, John Wiley & Sons, 2008, ISBN 13-978-0-471-45593-6.
- [301] V. PIETERSE, D. G. KOURIE et A. BOAKE, « A case for contemporary literate programming », dans « ACM International Conference Proceeding Series », vol. 75, 2004, p. 2–9.
- [302] J. F. PIMENTEL, L. MURTA, V. BRAGANHOLO et J. FREIRE, « Understanding and improving the quality and reproducibility of Jupyter notebooks », *Empirical Software Engineering*, vol. 26(4), 2021, p. 65, doi:[10.1007/s10664-021-09961-9](https://doi.org/10.1007/s10664-021-09961-9).
- [303] R. PLOSCHE, A. DAUTOVIC et M. SAFT, « The Value of Software Documentation Quality », dans « 2014 14th International Conference on Quality Software », IEEE, Allen, TX, USA, octobre 2014, ISBN 978-1-4799-7198-5 978-1-4799-7197-8, p. 333–342, doi:[10.1109/QSIC.2014.22](https://doi.org/10.1109/QSIC.2014.22).
- [304] PMI, éd., *A guide to the project management body of knowledge / Project Management Institute*, sixth edition éd., PMBOK guide, Project Management Institute, Newtown Square, PA, 2017, ISBN 978-1-62825-184-5.
- [305] PMI, éd., *The standard for project management and a guide to the project management body of knowledge (PMBOK guide)*, seventh edition éd., Project Management Institute, Inc, Newtown Square, Pennsylvania, 2021, ISBN 978-1-62825-667-3 978-1-62825-665-9 978-1-62825-666-6.
- [306] G. POST, A. KAGAN et R. T. KEIM, « A comparative evaluation of CASE tools », *Journal of Systems and Software*, vol. 44(2), 1998, p. 87–96.
- [307] R. S. PRESSMAN, *Software engineering : a practitioner's approach*, 5<sup>e</sup> éd., McGraw-Hill series in computer science, McGraw Hill, Boston, Mass, 2000, ISBN 978-0-07-365578-9.
- [308] R. S. PRESSMAN, *Software engineering : a practitioner's approach*, 7<sup>e</sup> éd., McGraw-Hill Higher Education, New York, 2010, ISBN 978-0-07-337597-7. OCLC : ocn271105592.



## BIBLIOGRAPHIE

- [309] J. PRINTZ, *Architecture logicielle : concevoir des applications simples, sûres et adaptables*, 3<sup>e</sup> éd., InfoPro, Dunod, Paris, 2012, ISBN 978-2-10-057865-8.
- [310] PROJECT MANAGEMENT INSTITUTE, éd., *Guide du Corpus des connaissances en management de projet : Guide PMBOK*, sixième édition éd., Project Management Institute, Newtown Square, PA, 2017, ISBN 978-1-62825-187-6 978-1-62825-430-3.
- [311] D. S. K. PUTRA, S. TISTIYANI et S. U. SUNARINGTYAS, « The Use of ISO/IEC 27001 Family of Standards in Regulatory Requirements in Some Countries », dans « 2021 2nd International Conference on ICT for Rural Development (IC-ICTRuDev) », IEEE, 2021, p. 1–6, doi:[10.1109/IC-ICTRUDEV50538.2021.9656529](https://doi.org/10.1109/IC-ICTRUDEV50538.2021.9656529).
- [312] QIAN HU, « Software documentation writing on the project of software upgrade and maintenance », dans « 2012 International Conference on Computer Science and Information Processing (CSIP) », IEEE, Xian, Shaanxi, China, août 2012, ISBN 978-1-4673-1411-4 978-1-4673-1410-7 978-1-4673-1409-1, p. 1400–1403, doi:[10.1109/CSIP.2012.6309125](https://doi.org/10.1109/CSIP.2012.6309125).
- [313] A. RADERMACHER et G. WALIA, « Gaps between industry expectations and the abilities of graduates », dans « Proceeding of the 44th ACM technical symposium on Computer science education », ACM, 2013, ISBN 978-1-4503-1868-6, p. 525–530, doi:[10.1145/2445196.2445351](https://doi.org/10.1145/2445196.2445351).
- [314] M. RAGLIANTI, C. NAGY, R. MINELLI, B. LIN et M. LANZA, « On the Rise of Modern Software Documentation (Pearl/Brave New Idea) », *LIPICs, Volume 263, ECOOP 2023*, vol. 263, 2023, p. 43 :1–43 :24, ISSN 1868-8969, doi:[10.4230/LIPICS.ECOOP.2023.43](https://doi.org/10.4230/LIPICS.ECOOP.2023.43).
- [315] P. RALPH, « Toward Methodological Guidelines for Process Theories and Taxonomies in Software Engineering », *IEEE Transactions on Software Engineering*, vol. 45(7), juillet 2019, p. 712–735, ISSN 0098-5589, 1939-3520, 2326-3881, doi:[10.1109/TSE.2018.2796554](https://doi.org/10.1109/TSE.2018.2796554).

## BIBLIOGRAPHIE

- [316] M. RETTIG, « Nobody reads documentation », *Communications of the ACM*, vol. 34(7), juillet 1991, p. 19–24, ISSN 0001-0782, 1557-7317, doi:[10.1145/105783.105788](https://doi.org/10.1145/105783.105788).
- [317] A. REUEL, A. HARDY, C. SMITH, M. LAMPARTH, M. HARDY et M. J. KOCHENDERFER, « BetterBench : Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices », novembre 2024. Non publié, doi:[10.48550/arXiv.2411.12990](https://doi.org/10.48550/arXiv.2411.12990).
- [318] L. RIERSON, *DO-178C Developing Safety-Critical Software : A Practical Guide for Aviation Software and DO-178C Compliance*, 1<sup>re</sup> éd., CRC Press, décembre 2017, ISBN 978-1-315-21816-8, doi:[10.1201/9781315218168](https://doi.org/10.1201/9781315218168).
- [319] ROBERT, « Analyse », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/analyse> (consulté le 2025-04-12).
- [320] ROBERT, « Conception », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/conception> (consulté le 2025-04-12).
- [321] ROBERT, « Concevoir », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/concevoir> (consulté le 2025-04-12).
- [322] ROBERT, « Concret », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/concret> (consulté le 2025-04-12).
- [323] ROBERT, « Concretisation », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/concretisation> (consulté le 2025-04-12).
- [324] ROBERT, « Cycle », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/cycle> (consulté le 2025-10-12).

## BIBLIOGRAPHIE

- [325] ROBERT, « Ellipse », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/ellipse> (consulté le 2025-10-12).
- [326] ROBERT, « Exhaustif », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/exhaustif> (consulté le 2025-10-16).
- [327] ROBERT, « Instrument », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/en/definition/instrument> (consulté le 2025-04-12).
- [328] ROBERT, « Langage », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/langage> (consulté le 2025-10-16).
- [329] ROBERT, « Langue », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/langue> (consulté le 2025-10-16).
- [330] ROBERT, « Modèle », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/modèle> (consulté le 2025-10-16).
- [331] ROBERT, « Méthode », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/méthode> (consulté le 2025-04-12).
- [332] ROBERT, « Organiser », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/organiser> (consulté le 2025-10-12).
- [333] ROBERT, « Professionnel », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/professionnel> (consulté le 2025-10-16).

## BIBLIOGRAPHIE

- [334] ROBERT, « Réaliser », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/realiser> (consulté le 2025-04-12).
- [335] ROBERT, « Transition », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/transition> (consulté le 2025-04-12).
- [336] ROBERT, « Échéancier », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/echeancier> (consulté le 2025-04-12).
- [337] ROBERT, « Établir », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/etablir> (consulté le 2025-04-12).
- [338] ROBERT, « Évaluation », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/evaluation> (consulté le 2025-04-12).
- [339] ROBERT, « Évaluer », site web, 2025.  
Récupéré de <https://dictionnaire.lerobert.com/definition/evaluer> (consulté le 2025-04-12).
- [340] M. ROGGENBACH, A. CERONE, B.-H. SCHLINGLOFF, G. SCHNEIDER et S. A. SHAIKH, « Formal Methods for Software Engineering », *Springer Nature Switzerland, Gewerbestrasse*, vol. 11, 2021, p. 6330, doi:[10.1007/978-3-030-38800-3](https://doi.org/10.1007/978-3-030-38800-3).
- [341] J. ROMEO, M. RAGLIANTI, C. NAGY et M. LANZA, « UML is Back. Or is it? Investigating the Past, Present, and Future of UML in Open Source Software », dans « 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE) », IEEE, Ottawa, ON, Canada, avril 2025, ISBN 979-8-3315-0569-1, p. 2342–2354, doi:[10.1109/ICSE55347.2025.00155](https://doi.org/10.1109/ICSE55347.2025.00155).
- [342] K. RONANKI, C. BERGER et J. HORKOFF, « Investigating ChatGPT’s Potential to Assist in Requirements Elicitation Processes », dans « 2023 49th Euromi-

## BIBLIOGRAPHIE

- cro Conference on Software Engineering and Advanced Applications (SEAA) », IEEE, Durres, Albania, septembre 2023, ISBN 979-8-3503-4235-2, p. 354–361, doi:[10.1109/SEAA60479.2023.00061](https://doi.org/10.1109/SEAA60479.2023.00061).
- [343] J. G. RONQUILLO et D. M. ZUCKERMAN, « Software-Related Recalls of Health Information Technology and Other Medical Devices : Implications for FDA Regulation of Digital Health », *The Milbank Quarterly*, vol. 95(3), septembre 2017, p. 535–553, ISSN 0887-378X, 1468-0009, doi:[10.1111/1468-0009.12278](https://doi.org/10.1111/1468-0009.12278).
- [344] W. ROSA, B. K. CLARK, R. MADACHY et B. W. BOEHM, « Empirical Effort and Schedule Estimation Models for Agile Processes in the US DoD », *IEEE Transactions on Software Engineering*, vol. 48(8), août 2022, p. 3117–3130, ISSN 0098-5589, 1939-3520, 2326-3881, doi:[10.1109/TSE.2021.3080666](https://doi.org/10.1109/TSE.2021.3080666).
- [345] S. ROSENKRANZ, D. STAEGEMANN, M. VOLK et K. TUROWSKI, « Explaining the Business-Technological Age of Legacy Information Systems », *IEEE Access*, vol. 12, 2024, p. 84 579–84 611, ISSN 2169-3536, doi:[10.1109/ACCESS.2024.3414377](https://doi.org/10.1109/ACCESS.2024.3414377).
- [346] M. ROSSI, M. GUSTAFSSON, K. SMOLANDER, L.-A. JOHANSSON et K. LYYTINEN, « Metamodeling editor as a front end tool for a CASE shell », dans « Active Flow and Combustion Control 2018 », édité par R. KING, vol. 141, Springer International Publishing, Cham, 1992, ISBN 978-3-319-98176-5 978-3-319-98177-2, p. 546–567, doi:[10.1007/BFb0035153](https://doi.org/10.1007/BFb0035153). Series Title : Notes on Numerical Fluid Mechanics and Multidisciplinary Design.
- [347] D. ROST, M. NAAB, C. LIMA et C. VON FLACH GARCIA CHAVEZ, « Software Architecture Documentation for Developers : A Survey », dans « Software Architecture », édité par D. HUTCHISON, T. KANADE, J. KITTLER, J. M. KLEINBERG, F. MATTERN, J. C. MITCHELL *et al.*, vol. 7957, Springer Berlin Heidelberg, Berlin, Heidelberg, 2013, ISBN 978-3-642-39030-2 978-3-642-39031-9, p. 72–88, doi:[10.1007/978-3-642-39031-9\\_7](https://doi.org/10.1007/978-3-642-39031-9_7). Series Title : Lecture Notes in Computer Science.

## BIBLIOGRAPHIE

- [348] W. ROYCE, *Software Project Management : A Unified Framework*, 13<sup>e</sup> éd., Addison-Wesley, Boston Munich, 2004, ISBN 978-0-321-73402-0.
- [349] D. RUBINFELD, « Data portability and interoperability : An E.U.-U.S. comparison », *European Journal of Law and Economics*, vol. 57(1-2), avril 2024, p. 163–179, ISSN 0929-1261, 1572-9990, doi:[10.1007/s10657-023-09781-w](https://doi.org/10.1007/s10657-023-09781-w).
- [350] C. RUDIN, « Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead », *Nature Machine Intelligence*, vol. 1(5), mai 2019, p. 206–215, ISSN 2522-5839, doi:[10.1038/s42256-019-0048-x](https://doi.org/10.1038/s42256-019-0048-x).
- [351] S. J. RUSSELL et P. NORVIG, *Artificial intelligence : a modern approach*, fourth edition éd., Pearson Series in Artificial Intelligence, Pearson, Hoboken, NJ, 2021, ISBN 978-0-13-461099-3.
- [352] O. SAINZ, J. A. CAMPOS, I. GARCÍA-FERRERO, J. ETXANIZ, O. L. D. LACALLE et E. AGIRRE, « NLP Evaluation in trouble : On the Need to Measure LLM Data Contamination for each Benchmark », octobre 2023. Non publié, doi:[10.48550/arXiv.2310.18018](https://doi.org/10.48550/arXiv.2310.18018).
- [353] J. E. SAMMET, « Programming languages : History and future », *Communications of the ACM*, vol. 15(7), 1972, p. 601–610, doi:[10.1145/361454.361485](https://doi.org/10.1145/361454.361485).
- [354] J. SANTOS et F. F. CORREIA, « A Review of Pattern Languages for Software Documentation », dans « Proceedings of the European Conference on Pattern Languages of Programs 2020 », ACM, Virtual Event Germany, juillet 2020, ISBN 978-1-4503-7769-0, p. 1–14, doi:[10.1145/3424771.3424786](https://doi.org/10.1145/3424771.3424786).
- [355] S. R. SCHACH, *Object-oriented and classical software engineering*, 8<sup>e</sup> éd., McGraw-Hill, New York, 2011, ISBN 978-0-07-337618-9.
- [356] T. SCHOPF, K. ARABI et F. MATTHES, « Exploring the landscape of natural language processing research », 2023. Non publié.
- [357] R. W. SELBY, *Being a Software Engineer in the Software Century*, chap. 10, Wiley-IEEE Press, 2007, p. 797–806, doi:[10.1109/9780470187562.ch10](https://doi.org/10.1109/9780470187562.ch10).

## BIBLIOGRAPHIE

- [358] T. SELF, *The DITA style guide : best practices for authors*, Scriptorium Publishing, Research Triangle Park, N.C., 2011, ISBN 978-0-9828118-1-8. OCLC : 775007515.
- [359] H. SEMMELROCK, T. ROSS-HELLAUER, S. KOPEINIK, D. THEILER, A. HABERL, S. THALMANN et D. KOWALD, « Reproducibility in machine-learning-based research : Overview, barriers, and drivers », *AI Magazine*, vol. 46(2), juin 2025, p. e70 002, ISSN 0738-4602, 2371-9621, doi:[10.1002/aaai.70002](https://doi.org/10.1002/aaai.70002).
- [360] P. SERRADOR et J. K. PINTO, « Does Agile work? — A quantitative analysis of agile project success », *International Journal of Project Management*, vol. 33(5), juillet 2015, p. 1040–1051, ISSN 02637863, doi:[10.1016/j.ijproman.2015.01.006](https://doi.org/10.1016/j.ijproman.2015.01.006).
- [361] M. SHAW, « Software engineering education : a roadmap », dans « Proceedings of the Conference on The Future of Software Engineering », ACM, 2000, ISBN 978-1-58113-253-3, p. 371–380, doi:[10.1145/336512.336592](https://doi.org/10.1145/336512.336592).
- [362] H. A. SIALA, K. LANO et H. ALFRAIHI, « Model-Driven Approaches for Reverse Engineering—A Systematic Literature Review », *IEEE Access*, vol. 12, 2024, p. 62 558–62 580, ISSN 2169-3536, doi:[10.1109/ACCESS.2024.3394732](https://doi.org/10.1109/ACCESS.2024.3394732).
- [363] E. SIEGEL, *Schematron : a language for validating XML*, first edition éd., XML Press, 2022, ISBN 978-1-937434-81-6 978-1-937434-80-9.
- [364] M. SILBERZTEIN, *Linguistic Resources for Natural Language Processing : On the Necessity of Using Linguistic Methods to Develop NLP Software*, Springer Nature, 2024, ISBN 978-3-031-43811-0, doi:[10.1007/978-3-031-43811-0](https://doi.org/10.1007/978-3-031-43811-0).
- [365] A. R. SIMON, *The Integrated CASE Tools Handbook*, John Wiley & Sons, Inc., 1993, ISBN 0-442-30890-6.
- [366] D. J. SMITH et K. G. L. SIMPSON, *The safety critical systems handbook : a straightforward guide to functional safety :IEC 61508 (2010 edition), IEC 61511 (2016 edition) & related guidance : including machinery and other industrial*

## BIBLIOGRAPHIE

- sectors*, fourth edition éd., BH/Elsevier, Amsterdam ; Boston, 2016, ISBN 978-0-12-805121-4.
- [367] I. SOMMERVILLE, « Software documentation Sommerville », document pdf, 2001.  
Récupéré de <http://literateprogramming.com/documentation.pdf> (consulté le 2025-02-12).
- [368] I. SOMMERVILLE, « Integrated requirements engineering : a tutorial », *IEEE Software*, vol. 22(1), janvier 2005, p. 16–23, ISSN 0740-7459, doi:[10.1109/MS.2005.13](https://doi.org/10.1109/MS.2005.13).
- [369] S. C. B. DE SOUZA, N. ANQUETIL et K. M. DE OLIVEIRA, « A study of the documentation essential to software maintenance », dans « Proceedings of the 23rd annual international conference on Design of communication documenting & designing for pervasive information - SIGDOC '05 », ACM Press, Coventry, United Kingdom, 2005, ISBN 978-1-59593-175-7, p. 68, doi:[10.1145/1085313.1085331](https://doi.org/10.1145/1085313.1085331).
- [370] K. STANCIN, P. POSCIC et D. JAKSIC, « Ontologies in education—state of the art », *Education and Information Technologies*, vol. 25(6), 2020, p. 5301–5320, doi:[10.1007/s10639-020-10226-z](https://doi.org/10.1007/s10639-020-10226-z).
- [371] STANDISH, « CHAOS Report »,  
Récupéré de [https://wiki.sj.ifsc.edu.br/images/3/3b/CHAOSReport2015\\_rev.pdf](https://wiki.sj.ifsc.edu.br/images/3/3b/CHAOSReport2015_rev.pdf) 2015.
- [372] STATISTIQUES CANADA, « Profession, selon le principal domaine d'études (STIM et SACHES, détaillé) : Canada, provinces et territoires »,  
Récupéré de <https://www150.statcan.gc.ca/t1/tbl1/fr/cv.action?pid=9810040501> 2022.
- [373] V. ŠTUIKYS et R. DAMAŠEVIČIUS, *Meta-programming and model-driven meta-program development : principles, processes and techniques*, vol. 5, Springer Science & Business Media, 2012, ISBN 978-1-4471-4126-6, doi:[10.1007/978-1-4471-4126-6](https://doi.org/10.1007/978-1-4471-4126-6).



## BIBLIOGRAPHIE

- [374] D. A. TAMBURRI, F. PALOMBA et R. KAZMAN, « Success and Failure in Software Engineering : A Followup Systematic Literature Review », *IEEE Transactions on Engineering Management*, vol. 68(2), avril 2021, p. 599–611, ISSN 0018-9391, 1558-0040, doi:[10.1109/TEM.2020.2976642](https://doi.org/10.1109/TEM.2020.2976642).
- [375] J. TEAM, « The Jupyter Notebook », 2026.  
Récupéré de <https://jupyter-notebook.readthedocs.io/en/stable/notebook.html> (consulté le 2026-06-01).
- [376] TECHNOCOMPÉTENCES, « DIAGNOSTIC SECTORIEL 2025-2028 », 2025.  
Récupéré de <https://www.technocompetences.qc.ca/publications/diagnostic-sectoriel-2025-2028-technocompetences/> ISBN 978-0-309-46702-5.
- [377] M. H. TER BEEK, R. CHAPMAN, R. CLEAVELAND, H. GARAVEL, R. GU, I. TER HORST *et al.*, « Formal Methods in Industry », *Formal Aspects of Computing*, août 2024, p. 3689374, ISSN 0934-5043, 1433-299X, doi:[10.1145/3689374](https://doi.org/10.1145/3689374).
- [378] M. H. TER BEEK et D. NIČKOVIĆ, éd., *Formal Methods for Industrial Critical Systems : 25th International Conference, FMICS 2020, Vienna, Austria, September 2–3, 2020, Proceedings, Lecture Notes in Computer Science*, vol. 12327, Springer International Publishing, Cham, 2020, ISBN 978-3-030-58297-5 978-3-030-58298-2, doi:[10.1007/978-3-030-58298-2](https://doi.org/10.1007/978-3-030-58298-2).
- [379] A. TERRASA, J. PRIVAT et G. TREMBLAY, « Using Natural Language Processing for Documentation Assist. », dans « AAAI Workshops », 2018, p. 787–790.
- [380] TEXAS DEPARTMENT OF INFORMATION RESOURCES, « Legacy Systems Study Public Report : Assessment and Recommendations. », 2014.  
Récupéré de <https://dir.texas.gov/resource-libraryitem/legacy-systems-study-public-report> (consulté le 2026-06-01).
- [381] THE JOINT ACM/AIS IS2020 TASK FORCE, *A Competency Model for Undergraduate Programs in Software Engineering*, ACM, New York, NY, USA, février 2014.

## BIBLIOGRAPHIE

- [382] T. THEUNISSEN, U. VAN HEESCH et P. AVGERIOU, « A mapping study on documentation in Continuous Software Development », *Information and Software Technology*, vol. 142, février 2022, p. 106 733, ISSN 09505849, doi:[10.1016/j.infsof.2021.106733](https://doi.org/10.1016/j.infsof.2021.106733).
- [383] S. R. TILLEY, « Documenting-in-the-large vs. documenting-in-the-small. », dans « CASCON », 1993, p. 1083–1090.
- [384] E. TÜZÜN, B. TEKINERDOGAN, Y. MACIT et K. İNCE, « Adopting integrated application lifecycle management within a large-scale software company : An action research approach », *Journal of Systems and Software*, vol. 149, mars 2019, p. 63–82, ISSN 01641212, doi:[10.1016/j.jss.2018.11.021](https://doi.org/10.1016/j.jss.2018.11.021).
- [385] G. UDDIN et M. P. ROBILLARD, « How API Documentation Fails », *IEEE Software*, vol. 32(4), juillet 2015, p. 68–75, ISSN 0740-7459, 1937-4194, doi:[10.1109/MS.2014.80](https://doi.org/10.1109/MS.2014.80).
- [386] J. D. ULLMAN, *Principles of Database and Knowledge*, vol. 1, W H Freeman and Co., 1988, ISBN 978-0-88175-188-8.
- [387] A. UPMEIER ZU BELZEN, D. KRÜGER et J. VAN DRIEL, éd., *Towards a Competence-Based View on Models and Modeling in Science Education, Models and Modeling in Science Education*, vol. 12, Springer International Publishing, Cham, 2019, ISBN 978-3-030-30254-2 978-3-030-30255-9, doi:[10.1007/978-3-030-30255-9](https://doi.org/10.1007/978-3-030-30255-9).
- [388] M. USMAN, R. BRITTO, J. BÖRSTLER et E. MENDES, « Taxonomies in software engineering : A systematic mapping study and a revised taxonomy development method », *Information and Software Technology*, vol. 85, 2017, p. 43–59, doi:[10.1016/j.infsof.2017.01.006](https://doi.org/10.1016/j.infsof.2017.01.006).
- [389] J. P. VAN GIGCH, *System Design Modeling and Metamodeling*, Springer US, Boston, MA, 1991, ISBN 978-1-4899-0678-6 978-1-4899-0676-2, doi:[10.1007/978-1-4899-0676-2](https://doi.org/10.1007/978-1-4899-0676-2).

## BIBLIOGRAPHIE

- [390] K. VELTEN, D. M. SCHMIDT et K. KAHLEN, *Mathematical modeling and simulation : introduction for scientists and engineers*, second edition éd., Wiley-VCH, Weinheim, Germany, 2024, ISBN 978-3-527-83939-1 978-3-527-84960-4 978-3-527-41414-7.
- [391] M. VÖLTER, T. STAHL, J. BETTIN, A. HAASE et S. HELSEN, *Model-Driven Software Development : Technology, Engineering, Management*, 1<sup>re</sup> éd., Wiley Software Patterns Series, Wiley, s.l., 2013, ISBN 978-1-118-72576-4 978-0-470-02570-3.
- [392] D. D. WALDEN et INTERNATIONAL COUNCIL ON SYSTEMS ENGINEERING, éd., *Systems engineering handbook : a guide for system life cycle processes and activities*, fifth edition éd., Wiley, Hoboken, NJ, 2023, ISBN 978-1-119-81429-0 978-1-119-81431-3 978-1-119-81430-6.
- [393] J. WANG et W. M. TEPFENHART, *Formal methods in computer science*, Textbooks in mathematics, CRC press, 2020, ISBN 978-1-4987-7532-8 978-0-367-22570-4.
- [394] P. WARREN, J. DAVIES et E. SIMPERL, éd., *Context and Semantics for Knowledge Management : Technologies for Personal Productivity*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011, ISBN 978-3-642-19509-9 978-3-642-19510-5, doi:[10.1007/978-3-642-19510-5](https://doi.org/10.1007/978-3-642-19510-5).
- [395] M. WASEEM, T. DAS, A. AHMAD, P. LIANG, M. FEHMIDEH et T. MIKKONEN, « ChatGPT as a Software Development Bot : A Project-based Study », février 2024. Non publié, doi:[10.48550/arXiv.2310.13648](https://doi.org/10.48550/arXiv.2310.13648).
- [396] H. WASHIZAKI, « Guide to the Software Engineering Body of Knowledge », *IEEE Computer Society*, 2024.
- [397] A. I. WASSERMAN et P. FREEMAN, éd., *Software Engineering Education*, Springer New York, New York, NY, 1976, ISBN 978-0-387-90216-6 978-1-4612-9898-4, doi:[10.1007/978-1-4612-9898-4](https://doi.org/10.1007/978-1-4612-9898-4).

## BIBLIOGRAPHIE

- [398] L. WEIDINGER, J. MELLOR, M. RAUH, C. GRIFFIN, J. UESATO, P.-S. HUANG *et al.*, « Ethical and social risks of harm from Language Models », décembre 2021. Non publié, doi:[10.48550/arXiv.2112.04359](https://doi.org/10.48550/arXiv.2112.04359).
- [399] L. WEIDINGER, I. D. RAJI, H. WALLACH, M. MITCHELL, A. WANG, O. SALAUDEEN *et al.*, « Toward an Evaluation Science for Generative AI Systems », mars 2025. Non publié, doi:[10.48550/arXiv.2503.05336](https://doi.org/10.48550/arXiv.2503.05336).
- [400] L. WEIDINGER, J. UESATO, M. RAUH, C. GRIFFIN, P.-S. HUANG, J. MELLOR *et al.*, « Taxonomy of Risks posed by Language Models », dans « 2022 ACM Conference on Fairness Accountability and Transparency », ACM, Seoul Republic of Korea, juin 2022, ISBN 978-1-4503-9352-2, p. 214–229, doi:[10.1145/3531146.3533088](https://doi.org/10.1145/3531146.3533088).
- [401] Z. WEIJIAO, S. PENG et C. YAN, « S1000D Based Standardized Technical Information Management for High-Speed Trains », dans « 2015 Fifth International Conference on Instrumentation and Measurement, Computer, Communication and Control (IMCCC) », IEEE, Qinhuangdao, China, septembre 2015, ISBN 978-1-4673-7723-2, p. 987–991, doi:[10.1109/IMCCC.2015.214](https://doi.org/10.1109/IMCCC.2015.214).
- [402] W. WINSTON, « Royce. Managing the development of large software systems », *Tech Pap West Electron Show Conv*, vol. 8, 1970, p. 1–9.
- [403] K. WNUK, T. GORSCHER et S. ZAHDA, « Obsolete software requirements », *Information and Software Technology*, vol. 55(6), juin 2013, p. 921–940, ISSN 09505849, doi:[10.1016/j.infsof.2012.12.001](https://doi.org/10.1016/j.infsof.2012.12.001).
- [404] C. WOHLIN, P. RUNESON, M. HÖST, M. C. OHLSSON, B. REGNELL et A. WESSLÉN, *Experimentation in Software Engineering*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2024, ISBN 978-3-662-69305-6 978-3-662-69306-3, doi:[10.1007/978-3-662-69306-3](https://doi.org/10.1007/978-3-662-69306-3).
- [405] R. WOLVERTON, « The Cost of Developing Large-Scale Software », *IEEE Transactions on Computers*, vol. C-23(6), juin 1974, p. 615–636, ISSN 0018-9340, doi:[10.1109/t-c.1974.224002](https://doi.org/10.1109/t-c.1974.224002). Publisher : Institute of Electrical and Electronics Engineers (IEEE).

## BIBLIOGRAPHIE

- [406] A. WĄSOWSKI et T. BERGER, *Domain-Specific Languages : Effective Modeling, Automation, and Reuse*, Springer International Publishing, 2023, ISBN 978-3-031-23668-6 978-3-031-23669-3, doi:[10.1007/978-3-031-23669-3](https://doi.org/10.1007/978-3-031-23669-3).
- [407] C. YANG et C. HUANG, « Natural Language Processing (NLP) in Aviation Safety : Systematic Review of Research and Outlook into the Future », *Aerospace*, vol. 10(7), juin 2023, p. 600, ISSN 2226-4310, doi:[10.3390/aerospace10070600](https://doi.org/10.3390/aerospace10070600). Publisher : MDPI AG.
- [408] E. YOURDON, *Modern structured analysis*, Yourdon press, 1989, ISBN 0-13-598632-X.
- [409] E. ZABARDAST, J. FRATTINI, J. GONZALEZ-HUERTA, D. MENDEZ, T. GORSCHKEK et K. WNUK, « Assets in Software Engineering : What are they after all? », *Journal of Systems and Software*, vol. 193, novembre 2022, p. 111 485, ISSN 01641212, doi:[10.1016/j.jss.2022.111485](https://doi.org/10.1016/j.jss.2022.111485).
- [410] M. ZAHEDI, M. SHAHIN et M. ALI BABAR, « A systematic review of knowledge sharing challenges and practices in global software development », *International Journal of Information Management*, vol. 36(6), décembre 2016, p. 995–1019, ISSN 02684012, doi:[10.1016/j.ijinfomgt.2016.06.007](https://doi.org/10.1016/j.ijinfomgt.2016.06.007).
- [411] L. ZHAO, W. ALHOSHAN, A. FERRARI, K. J. LETSHOLO, M. A. AJAGBE, E.-V. CHIOASCA et R. T. BATISTA-NAVARRO, « Natural Language Processing for Requirements Engineering : A Systematic Mapping Study », *ACM Computing Surveys*, vol. 54(3), avril 2022, p. 1–41, ISSN 0360-0300, 1557-7341, doi:[10.1145/3444689](https://doi.org/10.1145/3444689).
- [412] J. ZHI, V. GAROUSI-YUSIFOĞLU, B. SUN, G. GAROUSI, S. SHAHNEWAZ et G. RUHE, « Cost, benefits and quality of software development documentation : A systematic mapping », *Journal of Systems and Software*, vol. 99, janvier 2015, p. 175–198, ISSN 01641212, doi:[10.1016/j.jss.2014.09.042](https://doi.org/10.1016/j.jss.2014.09.042).
- [413] H. ZUSE, *A Framework of Software Measurement*, DE GRUYTER, décembre 1998, ISBN 978-3-11-015587-7, doi:[10.1515/9783110807301](https://doi.org/10.1515/9783110807301).

## BIBLIOGRAPHIE

- [414] H. ØVERBY et J. A. AUDESTAD, *Introduction to Digital Economics : Foundations, Business Models and Case Studies*, chap. 12, Classroom Companion : Business, Springer International Publishing, Cham, 2021, ISBN 978-3-030-78236-8 978-3-030-78237-5, p. 177–192, doi:[10.1007/978-3-030-78237-5](https://doi.org/10.1007/978-3-030-78237-5).

# Glossaire

**Adaptabilité** La capacité d'une entité à être modifiée afin de répondre à des changements d'environnements ou de besoins. (inspirée de : [26, 79, 130, 197])

**Apprentissage profond (AP)** « Mode d'apprentissage automatique généralement effectué par un réseau de neurones artificiels composé de plusieurs couches de neurones hiérarchisées. » (tirée de [238])

**Architecture informatique** Organisation d'entités informatiques résultant de la phase de conception. (inspirée de : [26, 144, 220, 309])

**Arithmétique de Peano** Système formel de l'arithmétique élémentaire qui permet des raisonnements par récurrence. (inspirée de : [73])

**Artefact** Intrant tangible ou extrant tangible à un processus. (inspirée de : [136, 239])

**Automate** Cas particulier de machine abstraite formellement défini et conçu pour en permettre l'étude. (inspirée de : [122, 157, 240])

**Boîte noire** « Système fermé dont on ignore le fonctionnement interne, qui n'est connu que par son comportement extérieur en fonction des sollicitations qui lui sont appliquées. » [242]

**Cohérent** Système logique où aucune contradiction n'a encore été découverte. (inspirée de : [243, 245, 286])

**Complétude** Un système formel est complet lorsque toutes les vérités qu'il contient peuvent être déduites des axiomes et des règles d'inférence. (inspirée de : [179, 286])

**Composant logiciel** Logiciel ou partie d'un logiciel. (inspirée de : [140, 180, 244])

**Compréhensible** Qui peut être intelligible par le lectorat. (inspirée de : [165, 181, 233])

**Connaissance** Une croyance justifiée qui à force de légitimités devient un fait. (inspirée de : [101])

**Convention** « Accord de volonté entre deux ou plusieurs personnes physiques ou morales, par lequel elles s’engagent à faire ou à ne pas faire quelque chose. » (tirée de [246])

**Couplage** Mesure l’interdépendance entre des composants logiciels. (inspirée de : [127, 171, 185])

**Couverture** Ratio de la représentation d’une chose par une autre. Cela peut être d’un modèle par un artefact, d’un sujet par un modèle ou d’une théorie par un modèle. (inspirée de : [79, 172, 173, 178, 326])

**Dette technique** Passif technique résultant de choix opportunistes dont le coût de mise à niveau augmente avec le temps. (inspirée de : [18, 121, 202])

**Document** Écrit servant d’information. (inspirée de : [140, 186])

**Documentation informatique** Ensemble de documents associé à un ensemble d’entités informatiques. (inspirée de : [60, 140, 250])

**Ellipse** « Omission d’un ou plusieurs mots dans une phrase qui reste cependant compréhensible. » [325]

**Entité informatique** Système, sous-système, composant logiciel ou composant matériel en lien avec l’informatique. (inspirée de : [140, 252, 256])

**Établir** Créer les conditions pour soutenir la pérennité. (inspirée de : [196, 337])

**Gestion de la connaissance** Organiser, contrôler et rendre accessibles les représentations des connaissances au bénéfice d’une organisation. (inspirée de : [253, 310])

**Grand modèle de langage (GML)** « Modèle de langage constitué par apprentissage profond à partir de mégadonnées. » (tirée de [254])

**Informaticien** Spécialiste du traitement automatique et rationnel de l’information. (inspirée de : [188, 256])



**Informaticien appliqué** Informaticien spécialiste dans l'information provenant de domaines autres que l'informatique. (inspiré de : [198, 258, 259])

**Instrument** « Objet fabriqué servant à exécuter [quelque chose], à faire une opération. » (tirée de [327])

**Langage** Système pour communiquer comprenant des symboles et des règles. (inspirée de : [260, 328, 329])

**Langage de description d'architecture** Langage avec une syntaxe et une sémantique spécifiées qui décrit l'architecture d'entité informatique. (inspirée de : [144])

**Langage formel** Est composé d'ensembles explicites de symboles et d'un ensemble explicite de règles appelé grammaire formelle. (inspirée de : [125, 140, 261, 286, 340])

**Langage naturel** Est un langage non formel. (inspirée de : [140, 262])

**Langage spécifique à un domaine (*Domain-specific language*, en anglais)**  
Langage destiné aux utilisateurs dans un domaine spécifique dont la syntaxe ou la sémantique peut être défini formellement. (inspirée de : [263, 406])

**Logiciel** Ensemble de suites d'instructions interprétables par un ordinateur. (inspirée de : [140, 265, 272])

**Machine abstraite** Système formel comprenant les instructions nécessaires au traitement automatique de données. (inspirée de : [83, 189])

**Module** Est un composant comprenant une interface. (inspirée de : [60, 140])

**Modèle** Représentation d'un sujet pour en permettre le traitement. (inspirée de : [49, 330, 389, 390])

**Modèle formel** Est composé d'un langage formel et d'un ensemble de règles et vise la représentation d'un sujet pour en permettre le traitement. (inspirée de : [2, 3, 69])

**Méta (*meta* en grec)** « L'élément méta- vient du grec meta, qui peut signifier “au milieu”, “avec” ou “après”. [...] il exprime la succession, le changement, la participation, ou un concept qui englobe un autre concept. » (tirée de [266])

**Méthode** « Ensemble de démarches raisonnées, suivies pour parvenir à un but. »  
(tirée de [331])

**Norme** « Document, établi par consensus et approuvé par un organisme reconnu, qui fournit, pour des usages communs et répétés, des règles [...] » (tirée de [128])

**Ontologie informatique** « Corpus structuré de concepts, qui est modélisé dans un langage permettant l'exploitation par un ordinateur des relations sémantiques ou taxonomiques établies entre ces concepts. » (tirée de [267])

**Organiser** « Doter d'une structure, d'une constitution déterminée, d'un mode de fonctionnement. » (tirée de [332])

**Organisme de normalisation** « [...] principales fonctions, en vertu de ses statuts, est la préparation, l'approbation ou l'adoption de normes [...]. » (tirée de [128])

**Outil de génie logiciel assisté par ordinateur (*CASE tool*, en anglais)** Instrument servant à réaliser ou à établir des entités. (inspirée de : [140, 146])

**Participant informatique** Personne participant à des processus utilisant l'informatique. (inspirée de : [200, 201, 256])

**Patrimonial** « Se dit de composants, logiciels ou matériels, issus d'une génération ou d'une version dépassée et qui continuent d'être utilisés, après des ajustements, en même temps que la technologie contemporaine d'une entreprise. » (tirée de [268])

**Phase d'un procédé de développement** Regroupement d'activités logiquement interreliées découlant d'un procédé de développement. (inspirée de : [140, 190])

**Phase d'un projet** Regroupement d'activités logiquement interreliées découlant de la gestion d'un projet. (inspirée de : [140, 190])

**Praticien informatique** Personne qui réalise ou établit des entités informatiques. (inspirée de : [203, 256, 282])

**Processus** Instance d'un procédé. (inspirée de : [247, 270, 271, 310])

**Procédé** Ensemble structuré d'activités visant un but logiquement déterminé par des antécédents et des conséquents où des intrants donnent lieu à des extrants. (inspirée de : [249, 270, 271, 331])

**Procédé de développement** Procédé dont le but est de créer des entités informatiques. (inspirée de : [140, 187, 248, 310])

**Professionnel informatique** Personne exerçant une activité rémunérée liée à l'informatique. (inspirée de : [191, 256])

**Règlement** « Document qui contient des règles à caractère obligatoire et qui a été adopté par une autorité. » (tirée de [128])

**Réaliser** Rendre tangible ce qui est abstrait. (inspirée de : [182, 193, 334])

**Rétro-ingénierie** Consiste à étudier un groupe d'artefacts pour en déterminer le fonctionnement et la méthode de fabrication. (inspirée de : [140, 274])

**Savoir en informatique appliquée** Ensemble des connaissances s'appliquant à l'information, aux traitements de l'information et aux entités informatiques utilisé en pratique. (inspirée de : [275, 276])

**Standard** Document qui contient des règles de pratiques communes ne provenant pas d'un organisme de normalisation. (inspirée de : [128, 192, 277])

**Système logique** Est composé d'un langage formel et d'un ensemble de règles (axiome, proposition, logique) permettant d'inférer. (inspirée de : [95, 125, 286])

**Taxinomie** Activité et résultat de l'étude d'un sujet pour en obtenir des règles de classification. (inspirée de : [278, 315])

**Technique** « Ensemble des applications de la science dans le domaine de la production. » (tirée de [194])

**Traitement automatique des langues (TAL)** « Technique d'apprentissage automatique qui permet à l'ordinateur de comprendre le langage humain. » (tirée de [279])

# Annexe A

L'annexe A est construite à la manière d'un système formel, il permet au lecteur de valider chacun des énoncés ainsi que l'absence de contradiction. Ce système est un intermédiaire entre le texte principal et des informations supplémentaires (Annexe B) en appoint.

**Conjecture R01 : Ces principes rendent accessibles ces sujets.**

- des énoncés clairs, pertinents et catégorisés ;
- des citations contextualisées de sources appropriées pour appuyer des énoncés ;
- une structure pour faciliter la compréhension et l'utilisation dans les illustrations, les raisonnements et les démonstrations subséquentes.

(voir paragraphe P01 p.5 et explication p.218)

## A.1 Des documents en informatique appliquée

**Définition D01 : Logiciel (entité)**

Ensemble de suites d'instructions interprétable par un ordinateur.

(voir paragraphe P02 p.5 et explication p.220)

**Définition D02 : Composant logiciel**

Logiciel ou partie d'un logiciel.

(voir paragraphe P02 p.5 et explication p.220)

**Définition D03 : Entité informatique**

Système, sous-système, composant logiciel ou composant matériel en lien avec l'infor-

## A.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

matique.

(voir paragraphe P02 p.5 et explication p.221)

### **Définition D04 : Document**

Écrit servant d'information.

(voir paragraphe P02 p.5 et explication p.222)

### **Définition D05 : Documentation informatique**

Ensemble de documents associé à un ensemble d'entités informatiques.

(voir paragraphe P02 p.5 et explication p.222)

### **Définition OD01 : Procédé**

Ensemble structuré d'activités visant un but logiquement déterminé par des antécédents et des conséquents où des intrants donnent lieu à des extrants.

(voir paragraphe P03 p.6 et explication p.223)

### **Définition OD02 : Procédé de développement**

Procédé dont le but est de créer des entités informatiques.

(voir paragraphe P03 p.6 et explication p.224)

### **Définition OD03 : Processus**

Instance de procédé.

(voir paragraphe P03 p.6 et explication p.225)

### **Définition OD04 : Artefact**

Intrant tangible ou extrant tangible à un processus.

(voir paragraphe P03 p.6 et explication p.225)

### **Définition OD05 : Phase d'un projet**

Regroupement d'activités avec un début et une fin logiquement interreliés et découlant de la gestion d'un projet.

(voir paragraphe P03 p.6 et explication p.226)

### **Définition OD06 : Phase d'un procédé de développement**

Regroupement d'activités avec un début et une fin logiquement interreliés et découlant d'un procédé de développement.

(voir paragraphe P03 p.6 et explication p.227)

**Assertion OD07 : Le nombre de catégories de documents à employer pour un projet de développement dépend minimalement de ces facteurs : la**

## A.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**taille du projet, les lectorats, les procédés utilisés et la réglementation applicable.**

(voir paragraphe P04 p.7 et explication p.227)

**Assertion ODO8 : L’attribution d’un document à une catégorie est souvent vague.**

(voir paragraphe P04 p.7 et explication p.228)

**Définition ODO9 : Autorat**

Ensemble des auteurs.

(voir paragraphe P04 p.7)

**Définition OD10 : Lectorat**

Ensemble des lecteurs (d’un quotidien, d’une revue, etc.). [264]

(voir paragraphe P04 p.7)

**Assertion OD11 : Plusieurs standards définissent des catégories de documents.**

(voir paragraphe P05 p.7 et explication p.230)

**Théorème OD12 : Certains artefacts sont nécessaires pour vérifier le respect d’un procédé de développement.**

(voir paragraphe P05 p.7 et explication p.231)

**Axiome OD13 : Les documents servent à faire du logiciel et à le compléter.**

(voir paragraphe P06 p.8 et explication p.232)

**Axiome OD14 : Il n’y a pas consensus sur les phases de développement.**

(voir paragraphe P06 p.8 et explication p.233)

**Assertion OD15 : Les phases peuvent être, entre autres, l’analyse, la conception, la concrétisation, l’évaluation et la transition.**

(voir paragraphe P06 p.8 et explication p.234)

**Définition OD16 : Réaliser**

Rendre tangible ce qui est abstrait.

(voir paragraphe P07 p.8 et explication p.235)

**Définition OD17 : Établir**

Créer les conditions pour soutenir la pérennité.

(voir paragraphe P07 p.8 et explication p.236)

## A.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### Pendant la réalisation

**Assertion ED01 : Le professionnel informatique (temps, compétences) est nécessaire pour la documentation.**

(voir paragraphe P08 p.[10](#) et explication p.[236](#))

**Assertion ED02 : Plusieurs professionnels informatiques considèrent le temps consacré à la documentation comme étant inutile.**

(voir paragraphe P09 p.[11](#) et explication p.[238](#))

**Assertion ED03 : L'aversion des professionnels informatiques pour la documentation a mené à la réalisation d'une documentation minimaliste.**

(voir paragraphe P09 p.[11](#) et explication p.[241](#))

**Assertion ED04 : Le courant dominant en développement de logiciel est les méthodologies agiles.**

(voir paragraphe P10 p.[12](#) et explication p.[243](#))

**Assertion ED05 : La communication est privilégiée à la documentation.**

(voir paragraphe P10 p.[12](#) et explication p.[247](#))

**Axiome ED06 : Il y a une dépendance entre la qualité du logiciel et la qualité de la documentation logicielle.**

(voir paragraphe P11 p.[12](#) et explication p.[248](#))

**Citation ED07 : Il y a un manque d'intérêt scientifique sur l'étude de la documentation.**

(voir paragraphe P11 p.[12](#) et explication p.[249](#))

**Assertion ED08 : Depuis 2010, l'attention se concentre désormais sur la qualité du code source plutôt que sur celle du produit.**

(voir paragraphe P11 p.[12](#) et explication p.[249](#))

**Assertion ED09 : Plusieurs affirment ne pas utiliser de modèles de qualité pour la documentation ou utiliser d'anciens modèles de qualité.**

(voir paragraphe P11 p.[12](#) et explication p.[250](#))

**Assertion ED10 : Les caractéristiques de qualité de documentation peuvent être déduites de la littérature ou des artefacts.**

(voir paragraphe P12 p.[14](#) et explication p.[251](#))

## A.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**Citation ED11 : Plusieurs professionnels informatiques affirment évaluer la documentation à travers des revues informelles.**

(voir paragraphe P12 p.14 et explication p.253)

**Assertion ED12 : Il y a une tendance à moins mesurer la documentation.**

(voir paragraphe P13 p.14 et explication p.254)

**Assertion ED13 : Les métriques courantes sur le code source et la documentation sont utiles une fois le projet terminé.**

(voir paragraphe P14 p.15 et explication p.256)

**Assertion ED14 : Il est difficile de connaître le coût de la documentation, même proportionnellement au code source.**

(voir paragraphe P14 p.15 et explication p.258)

**Assertion ED15 : Les documents contenant des modèles peuvent servir à effectuer un contrôle.**

(voir paragraphe P14 p.15 et explication p.259)

**Assertion ED16 : Plusieurs activités de différentes phases modifient des documents.**

(voir paragraphe P15 p.15 et explication p.261)

**Assertion ED17 : Un même document peut être utilisé lors de plusieurs activités.**

(voir paragraphe P15 p.15 et explication p.262)

**Assertion ED18 : Il y a un éparpillement des informations à travers les documents d'architecture.**

(voir paragraphe P15 p.15 et explication p.263)

### Après la réalisation

**Définition ED19 : Patrimonial**

« [...] issus d'une génération ou d'une version dépassée et qui continuent d'être utilisés [...] ». » [268]

(voir paragraphe P16 p.16)



## A.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**Assertion ED20 : La fin de vie des entités informatiques est faiblement expliquée à l'intérieur des ouvrages de référence.**

(voir paragraphe P16 p.15 et explication p.264)

**Assertion ED21 : Dans plusieurs pays industrialisés, il est estimé qu'entre 39 % et 50 % des systèmes d'information utilisés sont patrimoniaux.**

(voir paragraphe P16 p.16 et explication p.265)

**Axiome ED22 : Les logiciels doivent s'adapter plus fréquemment.**

(voir paragraphe P17 p.17 et explication p.267)

**Assertion ED23 : La documentation peut devenir obsolète.**

(voir paragraphe P18 p.17 et explication p.268)

**Assertion ED24 : Le temps peut changer le lectorat, les langages utilisés et les standards.**

(voir paragraphe P18 p.17 et explication p.269)

**Définition ED25 : Dette technique**

Passif technique résultant de choix opportunistes dont le coût de mise à niveau augmente avec le temps.

(voir paragraphe P19 p.18 et explication p.270)

**Assertion ED26 : La documentation peut contribuer à la dette technique menant à l'échec de l'évolution ou de la maintenance d'un logiciel.**

(voir paragraphe P19 p.18 et explication p.271)

**Assertion ED27 : Plusieurs professionnels informatiques considèrent la documentation comme une source non fiable d'informations.**

(voir paragraphe P19 p.18 et explication p.272)

**Assertion ED28 : Plusieurs professionnels informatiques ne se préoccupent pas du lectorat, bien qu'ils fassent partie eux-mêmes du lectorat.**

(voir paragraphe P19 p.18 et explication p.274)

**Définition AD01 : Instrument**

Objet fabriqué servant à exécuter [quelque chose], à faire une opération.[327]

(voir paragraphe P20 p.19)

## A.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### **Définition AD02 : Méthode**

Ensemble de démarches raisonnées, suivies pour parvenir à un but. [331]

(voir paragraphe P20 p.19)

### **Définition AD03 : Technique**

Ensemble des applications de la science dans le domaine de la production. [194]

(voir paragraphe P20 p.19)

## **Des instruments**

**Assertion AD04 : Il existe des propositions de logiciels qui combinent des techniques informatiques visant à améliorer la qualité de la documentation.**

(voir paragraphe P21 p.20 et explication p.274)

**Assertion AD05 : Un instrument de documentation devrait maintenir les relations dans les documents.**

(voir paragraphe P22 p.20 et explication p.278)

**Assertion AD06 : Entre les années 2002 et 2018, les instruments restent relativement similaires, à l'exception de l'utilisation des sites wiki.**

(voir paragraphe P23 p.21 et explication p.279)

## **Des méthodes informatiques**

### **Définition AD07 : Organiser**

Doter d'une structure, d'une constitution déterminée, d'un mode de fonctionnement. [332]

(voir paragraphe P24 p.22)

### **Définition AD08 : Module**

Est un composant comprenant une interface.

(voir paragraphe P24 p.22 et explication p.280)

**Assertion AD09 : Le terme cohérence est préférable à celui de cohésion pour discuter de modularité.**

(voir paragraphe P24 p.22 et explication p.281)

### **Définition AD10 : Couplage**

Mesure l'interdépendance entre des composants logiciels.

(voir paragraphe P24 p.22 et explication p.283)

## A.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### **Définition AD11 : Adaptabilité**

La capacité d'une entité à être modifiée afin de répondre à des changements d'environnements ou de besoins.

(voir paragraphe P24 p.22 et explication p.283)

**Assertion AD12 : La programmation structurée et l'encapsulation permettent d'améliorer l'adaptabilité d'un logiciel.**

(voir paragraphe P25 p.23 et explication p.284)

**Assertion AD13 : Il est possible d'organiser les documents comme les modules logiciels.**

(voir paragraphe P25 p.23 et explication p.286)

**Assertion AD14 : L'adaptabilité de plusieurs composants logiciels peut encore être amélioré.**

(voir paragraphe P25 p.23 et explication p.287)

**Citation AD15 : La programmation littéraire cherche à expliquer en premier aux humains et en second à la machine.**

(voir paragraphe P26 p.24 et explication p.288)

**Assertion AD16 : Les instruments qui exécutent la programmation littéraire ont continué de s'adapter.**

(voir paragraphe P26 p.24 et explication p.288)

**Assertion AD17 : Maintenir la cohérence lors de modifications avec la méthode de programmation littéraire est problématique.**

(voir paragraphe P26 p.24 et explication p.290)

**Assertion AD18 : Plusieurs individus croient à tort que l'instrument Jupiter est de la programmation littéraire.**

(voir paragraphe P26 p.24 et explication p.291)

### **Des techniques informatiques (donc, formelles)**

#### **Définition AD19 : Langage**

Système pour communiquer comprenant des symboles et des règles.

(voir paragraphe P27 p.25 et explication p.294)

## A.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### **Définition AD20 : Langage formel**

Est composé d'ensembles explicites de symboles et d'un ensemble explicite de règles appelé grammaire formelle.

(voir paragraphe P27 p.25 et explication p.294)

### **Définition AD21 : Langage naturel**

Est un langage non formel.

(voir paragraphe P27 p.25 et explication p.295)

### **Définition AD22 : Architecture informatique**

Organisation d'entités informatiques résultant de la phase de conception.

(voir paragraphe P27 p.25 et explication p.296)

### **Définition AD23 : Langage de description d'architecture (ADL, *Architecture Description Language* en anglais)**

Langage avec une syntaxe et une sémantique spécifiées qui décrit l'architecture d'entité informatique.

(voir paragraphe P27 p.25 et explication p.297)

### **Définition AD24 : Ontologie informatique**

« Corpus structuré de concepts, qui est modélisé dans un langage permettant l'exploitation par un ordinateur des relations sémantiques ou taxonomiques établies entre ces concepts. » [267]

(voir paragraphe P27 p.25)

### **Assertion AD25 : Le XML est un langage formel qui permet d'exprimer des informations selon une structure.**

(voir paragraphe P28 p.25 et explication p.299)

### **Assertion AD26 : Les langages de schéma XML (DTD, XSD, Schematron) permettent d'exprimer des vérifications sur le XML.**

(voir paragraphe P28 p.25 et explication p.300)

### **Assertion AD27 : Les langages de schéma XML ne permettent pas d'exprimer totalement la sémantique.**

(voir paragraphe P28 p.25 et explication p.301)

### **Assertion AD28 : Le XML, le DTD, le XSD sont utilisés par des instruments de documentation.**

(voir paragraphe P28 p.25 et explication p.302)

## A.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**Assertion AD29 : Le DITA permet une adaptabilité pour les documents.**

(voir paragraphe P28 p.[25](#) et explication p.[304](#))

**Assertion AD30 : Les langages d’architecture sont nombreux.**

(voir paragraphe P29 p.[26](#) et explication p.[304](#))

**Assertion AD31 : La possibilité d’exprimer la sémantique est une fonctionnalité appréciée et recherchée par plusieurs professionnels informatiques utilisant des langages d’architecture.**

(voir paragraphe P29 p.[26](#) et explication p.[305](#))

**Assertion AD32 : Plusieurs utilisateurs des langages d’architecture n’utilisent pas de langages formels, du moins pas ceux permettant d’exprimer la sémantique.**

(voir paragraphe P29 p.[26](#) et explication p.[307](#))

**Assertion AD33 : Il y a des exemples où un modèle relationnel est utilisé pour maintenir la documentation cohérente.**

(voir paragraphe P30 p.[26](#) et explication p.[309](#))

### Des techniques de traitement automatique des langues

**Définition AD34 : Traitement automatique des langues (TAL ou NLP, *Natural Language Processing* en anglais)**

« Technique d’apprentissage automatique qui permet à l’ordinateur de comprendre le langage humain. » [[279](#)]

(voir paragraphe P31 p.[28](#))

**Définition AD35 : Apprentissage profond (AP ou DL : *Deep Learning*)**

« Mode d’apprentissage automatique généralement effectué par un réseau de neurones artificiels composé de plusieurs couches de neurones hiérarchisées [...]. » [[238](#)]

(voir paragraphe P31 p.[28](#))

**Définition AD36 : Grand modèle de langage (GML ou LLM, *Large Language Model* en anglais)**

« Modèle de langage constitué par apprentissage profond à partir de mégadonnées. » [[254](#)]

(voir paragraphe P31 p.[28](#))

## A.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### **Définition AD37 : Boîte noire**

« Système fermé dont on ignore le fonctionnement interne, qui n'est connu que par son comportement extérieur en fonction des sollicitations qui lui sont appliquées. »  
[242]

(voir paragraphe P31 p.28)

### **Assertion AD38 : Plusieurs concepts de l'intelligence artificielle sont logiquement interreliés.**

(voir paragraphe P31 p.28 et explication p.312)

### **Assertion AD39 : Les domaines du traitement automatique des langues et de l'intelligence artificielle se croisent sur le plan historique.**

(voir paragraphe P31 p.28 et explication p.314)

### **Assertion AD40 : Plusieurs professionnels informatiques affirment que l'architecture et les exigences s'expriment couramment en langage naturel.**

(voir paragraphe P32 p.29 et explication p.315)

### **Assertion AD41 : Au moins jusqu'en 2020, les documents de spécification des exigences est la catégorie de documents utilisant le plus les techniques de TAL.**

(voir paragraphe P32 p.29 et explication p.316)

### **Énoncé AD42 : Les techniques de TAL sont de plus en plus utilisées dans la littérature scientifique.**

(voir paragraphe P32 p.29 et explication p.316)

### **Assertion AD43 : Il n'y a pas de règles exactes pour sélectionner une chaîne ou une classe de modèle à entraîner.**

(voir paragraphe P33 p.30 et explication p.317)

### **Assertion AD44 : Plusieurs techniques de TAL conduisent à la création de modèles de type boîte noire.**

(voir paragraphe P33 p.30 et explication p.318)

### **Assertion AD45 : Les tentatives pour expliquer ces modèles sont de plus en plus nombreuses, cela montre l'insuffisance des explications.**

(voir paragraphe P33 p.30 et explication p.318)

## A.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**Assertion AD46 : L'absence de culture sur la reproductibilité limite l'amélioration des connaissances.**

(voir paragraphe P33 p.[30](#) et explication p.[320](#))

**Assertion AD47 : Plusieurs experts affirment qu'il n'y a pas d'accumulation fiable des connaissances dans le domaine du TAL.**

(voir paragraphe P33 p.[30](#) et explication p.[321](#))

### Des techniques d'apprentissage profond

**Citation AD48 : L'apprentissage profond crée des grands modèles de langage qui améliorent les résultats en traitement automatique des langues.**

(voir paragraphe P34 p.[31](#) et explication p.[322](#))

**Assertion AD49 : Les bancs d'essai pour les GML sont davantage réussis, ce qui rend nécessaire le développement de nouveaux bancs d'essai.**

(voir paragraphe P34 p.[31](#) et explication p.[323](#))

**Assertion AD50 : Plusieurs professionnels informatiques utilisent les GML.**

(voir paragraphe P34 p.[31](#) et explication p.[324](#))

**Assertion AD51 : Les ambiguïtés provenant de la langue et du contexte demeurent même pour les GML.**

(voir paragraphe P35 p.[31](#) et explication p.[326](#))

**Assertion AD52 : Il n'a pas été encore démontré, dans un contexte réel, que les GML peuvent apporter une aide pour les documents de spécification des exigences.**

(voir paragraphe P35 p.[31](#) et explication p.[328](#))

**Assertion AD53 : Les contaminations des jeux de données servant à l'entraînement d'un modèle peuvent invalider les essais de celui-ci.**

(voir paragraphe P35 p.[31](#) et explication p.[329](#))

**Assertion AD54 : Les répercussions des techniques de TAL commencent à être formalisées et plusieurs sont néfastes.**

(voir paragraphe P35 p.[31](#) et explication p.[330](#))

## A.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

**Assertion AD55 : Les techniques d'IA ne remplacent pas les techniques informatiques.**

(voir paragraphe P36 p.33 et explication p.332)

## A.2 Des modèles en informatique appliquée

**Définition M01 : Modèle**

Représentation d'un sujet pour en permettre le traitement.

(voir paragraphe P37 p.33 et explication p.335)

**Définition OM01 : Système logique**

Est composé d'un langage formel et d'un ensemble de règles (axiome, proposition, logique) permettant d'inférer.

(voir paragraphe P38 p.34 et explication p.336)

**Définition OM02 : Modèle formel**

Est composé d'un langage formel et d'un ensemble de règles et vise la représentation d'un sujet pour en permettre le traitement.

(voir paragraphe P38 p.34 et explication p.337)

**Définition OM03 : Machine abstraite**

Système formel comprenant les instructions nécessaires au traitement automatique de données.

(voir paragraphe P38 p.34 et explication p.338)

**Assertion OM04 : L'utilisation des systèmes logiques peut varier tout comme les logiques employées.**

(voir paragraphe P39 p.34 et explication p.338)

**Assertion OM05 : L'informatique a développé plusieurs couches d'abstractions où sont utilisées différentes machines abstraites.**

(voir paragraphe P39 p.34 et explication p.340)

**Assertion OM06 : L'utilisation d'une mémoire réinscriptible permet d'obtenir des machines abstraites.**

(voir paragraphe P39 p.34 et explication p.341)



## A.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

**Assertion OM07 : L'informatique appliquée crée des modèles de problèmes et des modèles de solutions, puis cherche à les associer.**

(voir paragraphe P40 p.35 et explication p.343)

**Axiome OM08 : Les modèles en science ont, entre autres, comme objectifs d'apprendre, de développer et d'appliquer les connaissances.**

(voir paragraphe P40 p.35 et explication p.345)

**Assertion EM01 : Pour réussir une modélisation, il faut considérer, entre autres, la compréhensibilité, la couverture, la cohérence et la complétude.**

(voir paragraphe P41 p.35 et explication p.347)

**Définition EM02 : Cohérent**

Système logique où aucune contradiction n'a encore été découverte.

(voir paragraphe P42 p.36 et explication p.350)

**Définition EM03 : Couverture**

Consiste à couvrir tous les éléments d'un ensemble avec une sous-famille d'un autre ensemble, et ce, la plus petite possible.

(voir paragraphe P42 p.36 et explication p.350)

**Définition EM04 : Complétude**

Un système formel est complet lorsque toutes les vérités qu'il contient peuvent être déduites des axiomes et des règles d'inférence.

(voir paragraphe P42 p.36 et explication p.351)

**Définition EM05 : Compréhensible**

Qui peut être intelligible par le lectorat.

(voir paragraphe P42 p.36 et explication p.352)

**Définition EM06 : Modèle conceptuel**

Utilise des concepts pour représenter le sujet.

(voir paragraphe P43 p.37 et explication p.352)

**Assertion EM07 : L'utilisation de l'UML pour exprimer un modèle conceptuel est encouragée par plusieurs auteurs.**

(voir paragraphe P43 p.37 et explication p.353)

**Assertion EM08 : L'UML est un langage de modélisation non formel.**

(voir paragraphe P43 p.37 et explication p.354)

## A.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

**Assertion EM09 : Il y a une tendance à moins formaliser.**

(voir paragraphe P43 p.37 et explication p.357)

**Citation EM10 : L'accès aux sujets, aux modèles ou aux artefacts doit être amélioré.**

(voir paragraphe P44 p.38 et explication p.358)

**Assertion EM11 : Les informations sur le sujet, le modèle et l'artefact peuvent être inaccessibles, éparpillées et changeantes.**

(voir paragraphe P44 p.38 et explication p.359)

**Définition EM12 : Arithmétique de Peano**

Système formel de l'arithmétique élémentaire qui permet des raisonnements par récurrence.

(voir paragraphe P45 p.38 et explication p.361)

**Théorème EM13 : Un système formel cohérent pouvant interpréter une arithmétique de Peano est incomplet.**

(voir paragraphe P45 p.38 et explication p.362)

**Théorème EM14 : Un système formel cohérent pouvant interpréter une arithmétique de Peano ne peut pas vérifier sa cohérence.**

(voir paragraphe P45 p.38 et explication p.363)

**Corollaires EM15 : Une conséquence à l'incomplétude est qu'il n'existe pas d'algorithme permettant de tout vérifier dans un système formel en mesure d'interpréter l'arithmétique de Peano.**

(voir paragraphe P45 p.38 et explication p.364)

**Assertion EM16 : Il n'existe pas de consensus concernant les modèles.**

(voir paragraphe P46 p.39 et explication p.365)

**Assertion EM17 : Il n'existe pas de terminologie établie pour la modélisation en ingénierie.**

(voir paragraphe P46 p.39 et explication p.366)

**Assertion EM18 : Le vocabulaire peut confondre un modèle avec ses artefacts.**

(voir paragraphe P46 p.39 et explication p.367)

## A.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

**Assertion EM19 : Les modèles peuvent être mal interprétés.**

(voir paragraphe P47 p.40 et explication p.368)

**Assertion AM01 : Les instruments de modélisation comprennent notamment des langages, des techniques, des méthodes et les connaissances du domaine.**

(voir paragraphe P48 p.40 et explication p.370)

**Définition AM02 : Outil de génie logiciel assisté par ordinateur (*CASE tool : computer-aided software engineering tool*)**

Instrument servant à réaliser ou à établir des entités informatiques.

(voir paragraphe P49 p.40 et explication p.371)

**Définition AM03 : Langage spécifique à un domaine (DSL, *Domain-Specific Language*, en anglais)**

Langage destiné aux utilisateurs dans un domaine spécifique dont la syntaxe ou la sémantique peut être définie formellement.

(voir paragraphe P49 p.40 et explication p.372)

**Définition AM04 : Automate**

Cas particulier de machine abstraite formellement définie et conçue pour en permettre l'étude.

(voir paragraphe P49 p.40 et explication p.373)

### Des instruments de modélisation

**Assertion AM05 : Plusieurs instruments de modélisation ont une faible durée de vie (médiane de 6 ans)**

(voir paragraphe P50 p.41 et explication p.374)

**Assertion AM06 : Les instruments de modélisation ne rejoignent pas l'ensemble des professionnels informatiques.**

(voir paragraphe P50 p.41 et explication p.374)

**Assertion AM07 : Ce ne sont pas tous les instruments de modélisation qui couvrent l'ensemble des fonctionnalités possibles.**

(voir paragraphe P51 p.42 et explication p.376)

## A.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

### Des méthodes de modélisation

**Assertion AM08 : Plusieurs personnes visent à rendre des langages formels accessibles aux utilisateurs.**

(voir paragraphe P52 p.[43](#) et explication p.[378](#))

**Assertion AM09 : Le nombre de langages formels ne fait qu'augmenter.**

(voir paragraphe P52 p.[43](#) et explication p.[380](#))

**Théorème AM10 : Les classes de langages formels sont reliées aux classes d'automates.**

(voir paragraphe P52 p.[43](#) et explication p.[381](#))

**Assertion AM11 : Plusieurs méthodes considèrent le modèle comme un artefact, avec potentiellement, une syntaxe ou une sémantique formelle.**

(voir paragraphe P53 p.[45](#) et explication p.[383](#))

**Assertion AM12 : Certaines méthodes de modélisation consistent à ramener le modèle à un artefact qu'il est possible de transformer.**

(voir paragraphe P53 p.[45](#) et explication p.[385](#))

**Assertion AM13 : Les artefacts concrétisent le modèle, mais ce n'est pas le modèle.**

(voir paragraphe P53 p.[45](#) et explication p.[387](#))

**Assertion AM14 : Les pratiques réflexives permettent d'obtenir les modèles recherchés.**

(voir paragraphe P54 p.[46](#) et explication p.[388](#))

### Des techniques de modélisation

**Assertion AM15 : Le BPMN est un exemple d'un langage formel avec une représentation textuelle et graphique.**

(voir paragraphe P55 p.[47](#) et explication p.[391](#))

**Assertion AM16 : Les instruments ne permettent pas de profiter du formalisme du BPMN.**

(voir paragraphe P55 p.[47](#) et explication p.[393](#))

### A.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**Axiome AM17 :** Certaines méthodes formelles regroupent des techniques, tandis que d'autres constituent elles-mêmes des techniques.

(voir paragraphe P56 p.48 et explication p.394)

**Assertion AM18 :** Les méthodes formelles comprennent l'utilisation de langages formels et de systèmes logiques.

(voir paragraphe P56 p.48 et explication p.396)

**Assertion AM19 :** Les méthodes formelles sont définitivement utiles.

(voir paragraphe P56 p.48 et explication p.398)

**Assertion AM20 :** Les méthodes formelles ne s'imposent pas largement.

(voir paragraphe P56 p.48 et explication p.399)

## A.3 Des connaissances en informatique appliquée

**Définition S01 : Connaissance**

Une croyance justifiée qui à force de légitimités devient un fait.

(voir paragraphe P57 p.49 et explication p.401)

**Définition S02 : Savoir en informatique appliquée**

Ensemble des connaissances s'appliquant à l'information, aux traitements de l'information et aux entités informatiques utilisés en pratique.

(voir paragraphe P57 p.49 et explication p.402)

**Définition S03 : Gestion de la connaissance**

Organiser, contrôler et rendre accessibles les représentations des connaissances au bénéfice d'une organisation.

(voir paragraphe P57 p.49 et explication p.402)

**Axiome OS01 :** Une solution informatique traite un problème grâce à une machine abstraite définie par un modèle.

(voir paragraphe P58 p.50 et explication p.403)

**Assertion OS02 :** En informatique appliquée, la conservation des connaissances passe par les participants informatiques, les processus et les artefacts, notamment les instruments.

(voir paragraphe P59 p.51 et explication p.404)

### A.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**Assertion OS03 : La représentation explicite des connaissances est requise afin d'en permettre le traitement et le partage.**

(voir paragraphe P60 p.51 et explication p.407)

**Assertion OS04 : La connaissance provenant des autres domaines est une nécessité pour l'informatique appliquée, puisqu'elle a pour objectif de résoudre différents problèmes hors de son domaine.**

(voir paragraphe P60 p.51 et explication p.408)

#### Les participants informatiques

##### **Définition ES01 : Informaticien**

Spécialiste du traitement automatique et rationnel de l'information.

(voir paragraphe P61 p.53 et explication p.409)

##### **Définition ES02 : Informaticien appliqué**

Informaticien spécialiste dans l'information provenant de domaines autres que l'informatique.

(voir paragraphe P61 p.53 et explication p.410)

##### **Définition ES03 : Praticien informatique**

Personne qui réalise ou établit des entités informatiques.

(voir paragraphe P61 p.53 et explication p.410)

##### **Définition ES04 : Professionnel informatique**

Personne exerçant une activité rémunérée liée à l'informatique.

(voir paragraphe P61 p.53 et explication p.411)

##### **Définition ES05 : Participant informatique**

Personne participant à des processus utilisant l'informatique.

(voir paragraphe P61 p.53 et explication p.411)

**Assertion ES06 : Les personnes impliquées en informatique peuvent être catégorisées selon certains critères.**

(voir paragraphe P61 p.53 et explication p.412)

**Assertion ES07 : Le terme professionnel informatique regroupe plusieurs métiers plus spécialisés.**

(voir paragraphe P62 p.53 et explication p.413)

### A.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**Assertion ES08 : Les professionnels informatiques sont nombreux, près de cent quatre-vingt mille au Québec.**

(voir paragraphe P62 p.53 et explication p.414)

**Assertion ES09 : Les tranches d'âge plus élevées sont nettement plus représentées pour d'autres métiers.**

(voir paragraphe P63 p.53 et explication p.414)

**Assertion ES10 : La rétention des professionnels informatiques est plus faible que celle d'autres professions.**

(voir paragraphe P63 p.53 et explication p.415)

**Assertion ES11 : Les professionnels informatiques affirment quitter le métier, notamment en raison de la lourde charge de travail et de la menace d'obsolescence.**

(voir paragraphe P63 p.53 et explication p.416)

**Assertion ES13 : Un professionnel informatique n'a pas besoin d'être qualifié pour exercer au Québec.**

(voir paragraphe P64 p.55 et explication p.417)

**Assertion ES14 : La profession informatique est l'une des rares à ne pas avoir été catégorisée par la formation.**

(voir paragraphe P64 p.55 et explication p.418)

**Assertion ES15 : Aux États-Unis, parmi les analystes de systèmes informatiques, les informaticiens et les développeurs de logiciels titulaires d'un diplôme de premier cycle, moins de la moitié possèdent un diplôme en informatique.**

(voir paragraphe P64 p.55 et explication p.418)

**Citation ES16 : Plusieurs professionnels informatiques estiment que leur métier est trop exigeant pour maintenir ou développer de nouvelles compétences.**

(voir paragraphe P64 p.55 et explication p.419)

### A.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

#### Les processus

##### **Définition ES17 : Taxinomie**

Activité et résultat de l'étude d'un sujet pour en obtenir des règles de classification.  
(voir paragraphe P65 p.56 et explication p.420)

**Assertion ES18 : Les taxinomies sont importantes pour comprendre rigoureusement les théories et les modèles.**

(voir paragraphe P65 p.56 et explication p.421)

**Assertion ES19 : Les taxinomies portant sur les processus, les modèles et les méthodes sont peu nombreuses et subjectives.**

(voir paragraphe P65 p.56 et explication p.423)

**Citation ES20 : Le courant dominant discrédite les procédés de développement.**

(voir paragraphe P66 p.57 et explication p.425)

**Assertion ES21 : Il n'y a pas de définitions explicites des procédés de développement et des critères de succès d'un projet.**

(voir paragraphe P66 p.57 et explication p.426)

**Énoncé ES22 : Les études rigoureuses sur les procédés en informatique sont peu nombreuses.**

(voir paragraphe P66 p.57 et explication p.427)

#### Les artefacts

##### **Définition ES23 : Rétro-ingénierie**

Consiste à étudier un groupe d'artefacts pour en déterminer le fonctionnement et la méthode de fabrication.

(voir paragraphe P67 p.58 et explication p.429)

**Assertion ES24 : Les artefacts se dégradent volontairement, involontairement et par les changements dans l'environnement.**

(voir paragraphe P67 p.58 et explication p.429)

**Assertion ES25 : Plusieurs artefacts ne sont pas conservés.**

(voir paragraphe P67 p.58 et explication p.431)



### A.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**Assertion ES26 : Les artefacts ne contiennent pas toutes les connaissances.**

(voir paragraphe P68 p.[58](#) et explication p.[432](#))

#### Les instruments

**Assertion ES27 : Les définitions des *CASE tools* sont précaires.**

(voir paragraphe P69 p.[59](#) et explication p.[437](#))

**Assertion ES28 : Le vocabulaire utilisé pour définir des *CASE tools* crée de la confusion.**

(voir paragraphe P69 p.[59](#) et explication p.[438](#))

**Assertion ES29 : Les CASE tools suivent les tendances des méthodes de développement.**

(voir paragraphe P70 p.[60](#) et explication p.[440](#))

**Assertion ES30 : Les participants informatiques se font leurrer sur les fonctionnalités offertes par certains *CASE tools*.**

(voir paragraphe P70 p.[60](#) et explication p.[442](#))

**Assertion ES31 : Ce sont les organisations qui contrôlent les instruments.**

(voir paragraphe P71 p.[61](#) et explication p.[445](#))

#### Améliorer le cadre sociétal

##### Définition AS01 : Norme

Document, établi par consensus et approuvé par un organisme reconnu, qui fournit, pour des usages communs et répétés, des règles, des lignes directrices ou des caractéristiques, pour des activités ou leurs résultats, garantissant un niveau d'ordre optimal dans un contexte donné. [[128](#)]

(voir paragraphe P72 p.[62](#))

##### Définition AS02 : Organisme de normalisation

Organisme à activités normatives reconnu au niveau national, régional ou international, dont l'une des principales fonctions, en vertu de ses statuts, est la préparation, l'approbation ou l'adoption de normes qui sont mises à la disposition du public. [[128](#)]

(voir paragraphe P72 p.[62](#))

##### Définition AS03 : Règlement

Document qui contient des règles à caractère obligatoire et qui a été adopté par une

### A.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

autorité. [128].

(voir paragraphe P72 p.62)

#### **Définition AS04 : Standard**

Document qui contient des règles de pratiques communes ne provenant pas d'un organisme de normalisation.

(voir paragraphe P72 p.62 et explication p.447)

#### **Définition AS05 : Convention**

Accord de volonté entre deux ou plusieurs personnes physiques ou morales, par lequel elles s'engagent à faire ou à ne pas faire quelque chose. [246]

(voir paragraphe P72 p.62)

**Assertion AS06 : Le respect d'un standard ou d'une norme peut se faire ou non d'une manière coercitive.**

(voir paragraphe P73 p.63 et explication p.448)

**Assertion AS07 : Les standards ou les normes varient à travers le temps.**

(voir paragraphe P73 p.63 et explication p.449)

**Assertion AS08 : Même dans les secteurs critiques où les standards et normes existent, plusieurs problèmes surviennent.**

(voir paragraphe P73 p.63 et explication p.451)

**Assertion AS09 : Le langage UML était le langage le plus populaire entre 1997 et 2004**

(voir paragraphe P74 p.64 et explication p.452)

**Assertion AS10 : Le MOF et l'EMF sont respectivement le langage et le système de metamodélisation les plus utilisés.**

(voir paragraphe P74 p.64 et explication p.453)

**Assertion AS11 : Les instruments posent problème pour le MOF ou l'EMF.**

(voir paragraphe P74 p.64 et explication p.455)

**Citation AS12 : La convention de code est nécessaire pour faciliter la compréhension du lectorat.**

(voir paragraphe P75 p.64 et explication p.456)

### A.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**Citation AS13 :** Une convention de code source est superflue, puisqu'un bon langage de programmation facilite la compréhension du lectorat.

(voir paragraphe P75 p.64 et explication p.457)

**Assertion AS14 :** L'appropriation d'actes réservés en informatique est davantage une lutte d'économique.

(voir paragraphe P76 p.65 et explication p.457)

#### Améliorer la formation

**Axiome AS15 :** La formation sur la modélisation est nécessaire pour répondre aux changements rapides.

(voir paragraphe P77 p.65 et explication p.460)

**Assertion AS16 :** Il reste du travail à faire sur l'enseignement de la modélisation.

(voir paragraphe P77 p.65 et explication p.462)

**Assertion AS17 :** Cela fait de nombreuses années que des experts considèrent la formation en informatique appliquée insuffisante.

(voir paragraphe P78 p.67 et explication p.463)

**Assertion AS18 :** Il y a plus de vingt ans, des connaissances ont été regroupées et ont occupé un rôle important dans l'informatique appliquée.

(voir paragraphe P78 p.67 et explication p.464)

**Assertion AS19 :** Les changements trop nombreux en informatique appliquée mettent à mal l'accumulation de connaissances.

(voir paragraphe P78 p.67 et explication p.466)

**Assertion AS20 :** Les compétences importantes pour la formation touchent, entre autres, aux documents et aux modèles depuis au moins cinquante ans.

(voir paragraphe P79 p.67 et explication p.467)

**Assertion AS21 :** Un critère de réussite dans l'usage des méthodes formelles est l'expertise.

(voir paragraphe P80 p.68 et explication p.470)

### A.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**Assertion AS22 : L'enseignement des méthodes formelles est sur le déclin.**

(voir paragraphe P80 p.[68](#) et explication p.[471](#))

# Annexe B

L'annexe B contient toutes les informations nécessaires (citation, tableau, figure) pour appuyer la véracité des différents énoncés de l'annexe A. Cette grande quantité d'information ne permet pas d'être incluse entièrement dans le texte principale, mais demeurent accessible au lecteur.

**Explication de la conjecture R01 : Ces principes rendent accessibles ces sujets.**

Les énoncés sont caractérisés ainsi :

- un énoncé peut comprendre d'autres énoncés ;
- un énoncé peut être attribué à l'une des catégories décrites plus loin ;
- les explications aux énoncés sont ajoutées dans l'annexe ;

Les catégories d'énoncés avec leurs définitions sont :

- définition : « Énoncé linguistique qui décrit un concept et qui permet de le situer dans un système conceptuel. » [251] ;
- citation : « Action de citer, de rapporter les mots ou les phrases de quelqu'un ; paroles, passage empruntés à un auteur ou à quelqu'un qui fait autorité. » [176] ;
- assertion : « Proposition, de forme affirmative ou négative, qu'on avance et qu'on donne comme vraie. » [174] ;
- axiome : « Vérité admise sans démonstration et sur laquelle se fonde une science, un raisonnement ; principe posé hypothétiquement à la base d'une théorie déductive. » [175] ;

- théorème : « Expression d'un système formel démontrable à l'intérieur de ce système. » [195] ;
- conjecture : « Hypothèse formulée sur l'exactitude ou l'inexactitude d'un énoncé dont on ne connaît pas encore de démonstration. » [183] ;
- corollaire : « Fait résultant inévitablement d'un autre fait ; conséquence. » [184] ;

Les citations :

- sont accompagnées de qualificatifs, tels qu'opinion, étude, expérimentation, comparaison, sondage ou revue de littérature ;
- peuvent comporter des précisions quant à leurs auteurs et sur l'année de publication des sources ;
- lorsqu'elles proviennent de sondages ou d'études, indiquent souvent le nombre de participants ainsi que leurs occupations ;

Les sources sont, entre autres :

- des dictionnaires comme Le Robert, le Larousse et le Grand dictionnaire terminologique développé par l'Office québécois de la langue française (OQLF) ;
- des standards ou des normes provenant de différentes organisations comme : International Organization for Standardization (ISO), Institute of Electrical and Electronics Engineers (IEEE) et International Electrotechnical Commission (IEC) ;
- des ouvrages d'experts en informatique reconnus comme C. A. R. Hoare, Frederick P. Brooks Jr., David L. Parnas, Paul Clements et Ian F. Sommerville ;
- des articles scientifiques ;

La structure comprend :

- une division en sections et sous-sections ;
- chaque section porte soit sur le document, le modèle ou la connaissance ;
- chaque sous-section porte sur des objectifs, des écueils ou des approches.

(voir énoncé p.193)

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### B.1 Des documents en informatique appliquée

#### Explication de la définition D01 : Logiciel (entité)

La définition formulée est : ensemble de suites d'instructions interprétables par un ordinateur.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition de programme : « Suite d'instructions écrites sous une forme que l'ordinateur peut comprendre pour traiter un problème ou pour effectuer une tâche. » [272] ;
- définition 1 de logiciel : « Ensemble des programmes constituant une unité destinée à effectuer un traitement particulier sur un ordinateur. » [265] ;
- définition 2 de logiciel : [Traduction libre] « tout ou partie des programmes, procédures, règles et de la documentation associée d'un système de traitement de l'information » <sup>120</sup> [140].

Un logiciel est composé de programmes et compose une ou plusieurs entités informatiques. Lorsqu'il est employé seul, il désigne une entité logicielle.

(voir énoncé p.193)

#### Explication de la définition D02 : Composant logiciel

La définition formulée est : logiciel ou partie d'un logiciel.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition de composant : « Objet, substance, élément qui entre dans la composition de quelque chose [...]. » [180] ;
- définition de composant logiciel : « Logiciel, programme ou élément d'un logiciel ou d'un programme qui constitue un module indépendant utilisé comme élément d'un système plus complexe et qui est spécialement conçu pour fonctionner sans problèmes avec d'autres logiciels ou programmes. » [244] ;

---

120. « *all or part of the programs, procedures, rules, and associated documentation of an information processing system* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- description de composant : [Traduction libre] « Un composant peut être matériel ou logiciel et peut être subdivisé en d'autres composants. Un composant désigne une partie d'un tout, comme un composant d'un produit logiciel ou un composant d'une étiquette d'identification logicielle. » <sup>121</sup> [140].

(voir énoncé p.193)

### Explication de la définition D03 : Entité informatique

La définition formulée est : Système, sous-système, composant logiciel ou composant matériel en lien avec l'informatique.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition d'entité : « Chose considérée comme un être ayant son individualité. » [252] ;
- définition d'informatique : « Discipline qui s'intéresse à tous les aspects, tant théoriques que pratiques, reliés au traitement automatique de l'information, à la conception, à la programmation, au fonctionnement et à l'utilisation des ordinateurs. » [256] ;
- définition de système : [Traduction libre] « combinaison d'éléments interagissant organisés pour atteindre un ou plusieurs objectifs énoncés » <sup>122</sup> [140] ;
- définition 1 de produit : [Traduction libre] « un artefact produit, quantifiable et qui peut être soit un produit final en soi, soit un composant » <sup>123</sup> [140] ;
- définition 2 de produit : [Traduction libre] « ensemble complet de programmes informatiques, de procédures et de documentation associée conçu pour être livré à un utilisateur » <sup>124</sup> [140] ;

---

121. « *A component can be hardware or software and can be subdivided into other components. Component refers to a part of a whole, such as a component of a software product or a component of a software identification tag. [...]* »

122. « *combination of interacting elements organized to achieve one or more stated purposes* »

123. « *an artifact that is produced, is quantifiable, and can be either an end item in itself or a component item* »

124. « *complete set of computer programs, procedures, and associated documentation designed for delivery to a user* »



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

— définition 3 de produit : [Traduction libre] « résultat d'un processus »<sup>125</sup> [140].

Puisque les multiples définitions de **produit** peuvent porter à confusion, l'utilisation du terme **produit** est réservée comme étant le résultat d'un processus, et, dans ce mémoire, ce résultat est précisément désigné par les termes **entité informatique**, **composant logiciel** ou **logiciel**. Plus particulièrement, le produit qui importe le plus dans ce mémoire est le logiciel.

(voir énoncé p.193)

### Explication de la définition D04 : Document

La définition formulée est : écrit servant d'information.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 : « pièce écrite servant d'information, de preuve » [186] ;
- définition 2 : « objet quelconque servant de preuve, de témoignage » [186] ;
- définition 3 : [Traduction libre] « unité d'information identifiée de manière unique et destinée à l'usage humain, telle qu'un rapport, un cahier des charges, un manuel ou un livre, sous forme imprimée ou électronique »<sup>126</sup> [140] ;
- définition 4 : [Traduction libre] « support ainsi que l'information qui y est enregistrée, qui a généralement une permanence et peut être lue par une personne ou une machine. »<sup>127</sup> [140].

(voir énoncé p.194)

### Explication de la définition D05 : Documentation informatique

La définition formulée est : ensemble de documents associé à un ensemble d'entités informatiques.

La littérature fournit ces informations qui ont servi à construire cette définition :

---

125. « *result of a process* »

126. « *uniquely identified unit of information for human use, such as a report, specification, manual or book, in printed or electronic form* »

127. « *medium, and the information recorded on it, that generally has permanence and can be read by a person or a machine.* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- définition 1 de documentation informatique : « Ensemble de documents accompagnant et décrivant un produit informatique. » [250] ;
- note jointe à cette définition : « La documentation sert à faciliter l’installation, la compréhension et la maintenance d’un produit informatique. » [250] ;
- définition 2 de documentation informatique : [Traduction libre] « informations écrites ou illustrées décrivant, définissant, spécifiant, rapportant ou certifiant des activités, des exigences, des procédures ou des résultats »<sup>128</sup> [140] ;
- avis de Paul Clements : [Traduction libre] « La documentation peut être formelle ou informelle, selon le contexte, et peut contenir des modèles ou non, selon le contexte. Les documents peuvent décrire ou spécifier. Par conséquent, le terme est suffisamment général. »<sup>129</sup> [60].

La documentation logicielle est indissociable du produit informatique, c’est pour cette raison que le terme **documentation informatique** est aussi répandu.

(voir énoncé p.194)

### Explication de la définition OD01 : Procédé

La définition formulée est : ensemble structuré d’activités visant un but logiquement déterminé par des antécédents et des conséquents où des intrants donnent lieu à des extrants. La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de procédé : « une suite continue de faits, de phénomènes présentant une certaine unité ou une certaine régularité dans leur déroulement » [249] ;
- définition 2 de procédé : « une méthode utilisée en vue d’obtenir un résultat déterminé. » [249] ;
- définition 3 de procédé : « [...] un procédé est une activité humaine ayant des éléments d’entrées (matières premières ou personnes) et des éléments de sorties

---

128. « *written or pictorial information describing, defining, specifying, reporting, or certifying activities, requirements, procedures, or results* »

129. « *Documentation can be formal or not, as appropriate, and may contain models or not, as appropriate. Documents may describe, or they may specify. Hence, the term is appropriately general.* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- (produits finis ou personnes). Il y a donc bien transformation d'objets ayant certaines caractéristiques en objets en possédant d'autres. » <sup>130</sup> ;
- définition de méthode : « Ensemble de démarches que suit l'esprit pour découvrir et démontrer la vérité. » [331] ;
  - définition 1 de processus : « Suite continue de faits ou de phénomènes qui présentent une certaine unité ou une certaine régularité dans leur déroulement. » [270] ;
  - définition 2 de processus : « Ensemble d'activités logiquement interreliées qui produisent un résultat déterminé. » [271].

Le procédé et le processus peuvent être confondus. Il est retenu que le procédé a un objectif à remplir tandis que le processus se déroule.

(voir énoncé p.194)

### Explication de la définition OD02 : Procédé de développement

La définition formulée est : procédé dont le but est de créer des entités informatiques.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition de développement : « Mise au point d'un appareil, d'un produit en vue de sa vente ; période précédant la commercialisation. » [187] ;
- définition de procédé : suite d'activité visant un but ;
- définition de cycle de vie du développement logiciel : « Ensemble des étapes permettant le développement d'un logiciel de manière systématique. » [248] ;
- définition de cycle de développement logiciel : [Traduction libre] « [...] période de temps qui commence avec la décision de développer un produit logiciel et se termine avec la livraison du logiciel [...]. » <sup>131</sup> [140] ;
- définition de cycle de vie du produit : « Série de phases qui représentent l'évolution d'un produit, du concept à la livraison, la croissance, la maturité et jusqu'à son retrait du marché. » [310].

---

130. Wikipedia, « Procédé », <https://fr.wikipedia.org/wiki/Proc%C3%A9d%C3%A9> (consulté le 2026-05-15).

131. « [...] *period of time that begins with the decision to develop a software product and ends when the software is delivered* [...]. »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

Les procédés de développement logiciel et système peuvent se confondre, c'est pour cela que le terme **procédé de développement** est utilisé<sup>132</sup>. Aussi, le procédé de développement s'inscrit dans un procédé de gestion de projet.

(voir énoncé p.194)

### Explication de la définition OD03 : Processus

La définition formulée est : instance de procédé.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de processus : « Suite continue de faits ou de phénomènes qui présentent une certaine unité ou une certaine régularité dans leur déroulement. » [270] ;
- définition 2 de processus : « Ensemble d'activités logiquement interreliées qui produisent un résultat déterminé. » [271] ;
- définition 1 de cycle : « Suite de phénomènes se renouvelant sans arrêt dans un ordre immuable. » [324] ;
- définition 2 de cycle : « Ensemble d'opérations qui se répètent de façon constante à chaque intervalle de temps du déroulement d'un événement périodique. » [247] ;
- définition de cycle de vie du produit : « Série de phases qui représentent l'évolution d'un produit, du concept à la livraison, la croissance, la maturité et jusqu'à son retrait du marché. » [310] ;
- qualificatif de cycle de vie : incrémentiel, itératif, prédictif [310].

Les qualificatifs de cycle de vie en gestion de projet entremêlent les concepts de processus et de procédé. Néanmoins, il est possible de restreindre le cycle de vie au processus. Puisque le processus peut avoir une notion de régularité, le cycle de vie est un cas particulier de processus. Le processus se déroule et le but est facultatif.

(voir énoncé p.194)

---

132. Il existe une très grande variété de procédés de développement. Ils se construisent grâce des modèles et des méthodes. Il existe notamment le *Scrum*, l'*eXtreme Programming* (XP), le *Rational Unified Process* (RUP), le *Personal Software Process* (PSP), la *Cleanroom*, le modèle en V, le modèle en spirale ainsi que le *Capability Maturity Model Integration* (CMMI).

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### Explication de la définition OD04 : Artefact

La définition formulée est : intrant tangible ou extrant tangible à un processus.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition d’artefact : « Module d’information utilisé ou produit lors de la conception d’un logiciel. » [239] ;
- note jointe à cette définition : « L’artefact peut prendre la forme d’un modèle d’architecture, d’une description, d’une documentation, etc. » [239] ;
- définition d’artefact logiciel (*software artifact*, en anglais) : [Traduction libre] « Un artefact logiciel est un document tangible lisible par machine, créé lors du développement d’un logiciel. Les documents de spécification des exigences, les documents de conception, le code source et les exécutable en sont des exemples. »<sup>133</sup> [136].

Dans le cadre d’un processus informatique, les extrants sont des artefacts informatiques et certains sont des entités informatiques.

(voir énoncé p.194)

### Explication de la définition OD05 : Phase d’un projet

La définition formulée est : regroupement d’activités avec un début et une fin logiquement interreliés et découlant de la gestion d’un projet.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition de phase : « Chacun des états successifs d’un phénomène en évolution. » [190] ;
- définition de phase d’un projet : [Traduction libre] « [...] un ensemble d’activités de projet logiquement liées qui aboutissent à la réalisation d’un ou plusieurs livrables [...]. »<sup>134</sup> [140] ;

---

133. « *The software artifact is a tangible machine-readable document created during software development. Examples are requirement specification documents, design documents, source code and executables.* »

134. « *[...] a collection of logically related project activities that culminates in the completion of one or more deliverables [...].* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- Définition phase du projet : « Ensemble d'activités conjointes du projet qui aboutit à la finalisation d'un ou de plusieurs livrables. ». [310]

Les phases sont d'abord une relation entre deux points, puis vient des activités connexes ou logiquement reliées.

(voir énoncé p.194)

### Explication de la définition OD06 : Phases d'un procédé de développement

La définition formulée est : regroupement d'activités avec un début et une fin logiquement interreliés et découlant d'un procédé de développement.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition de phase : « Chacun des états successifs d'un phénomène en évolution. » [190] ;
- phase de développement : [Traduction libre] « Ensemble de tâches liées entre elles qui produisent un ou plusieurs livrables. Les phases de développement peuvent être imbriquées, se chevaucher et être itérées selon le processus de développement utilisé. » <sup>135</sup> [84] ;
- Note sur *software development cycle* : [Traduction libre] « Ce cycle comprend généralement une phase d'analyse des besoins, une phase de conception, une phase de mise en œuvre, une phase de tests et parfois une phase d'installation et de vérification. » <sup>136</sup> [140].

(voir énoncé p.194)

**Explication sur l'assertion OD07 : Le nombre de catégories de documents à employer pour un projet de développement dépend minimalement de ces facteurs : la taille du projet, les lectorats, les procédés utilisés et la réglementation applicable.**

---

135. « *Development phase A set of related tasks that produce one or more work products. Development phases can be interleaved, overlapped, and iterated as specified by the development process being used.* »

136. « *This cycle typically includes a requirements phase, design phase, implementation phase, test phase, and sometimes, installation and checkout phase.* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

La taille du projet, les lectorats et les procédés utilisés se retrouvent dans le *Management Guide for Software Documentation* de 1987 publiés par le National Bureau of Standards <sup>137</sup>. Voici ce qui y est inscrit :

[Traduction libre] Le nombre de types de documents produits pour un projet donné varie et dépend de la taille du projet, des publics cibles, du nombre de phases identifiées pour le contrôle de gestion et d'autres facteurs. <sup>138</sup> [228]

La réglementation applicable exige des catégories de documents, par exemple, dans l'avionique. Lenna Rierison l'explique pour le Software Configuration Index (SCI) et le Software Accomplishment Summary (SAS) :

[Traduction libre] Le SCI et le SAS sont les deux éléments de données du cycle de vie qui sont généralement soumis à l'autorité de certification pour démontrer la conformité à la norme DO-178C et aux réglementations. <sup>139</sup> [318]

(voir énoncé p.194)

**Explication sur l'assertion ODO8 : L'attribution d'un document à une catégorie est souvent vague.**

Les gabarits peuvent disparaître des standards. Un article résume l'évolution de standards dans le secteur militaire (tableau B.1) [61]. Il y a dans ce tableau les standards suivants : MIL-STD-1679 (1978), DOD-STD-2167 (1985), MIL-STD-498 (1994), IEEE/EIA 12207 (1996). Il n'y a aucun gabarit dans l'ISO/IEC/IEEE 12207:2017 [139].

---

137. En 1988, le nom a changé pour le National Institute of Standards and Technology (NIST).

138.

*The number of document types produced for any one project varies and depends on the size of the project, the audiences addressed, the number of phases identified for management control, and other factors.*

139.

*The SCI and SAS are the two life cycle data items that are typically submitted to the certification authority to demonstrate compliance with the DO-178C and the regulations.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABLEAU B.1** – Couverture des standards sur des concepts de génie logiciel (source : *Table 1* [61])

| <i>Coverage of some software engineering concepts by standards</i> |              |              |             |                |
|--------------------------------------------------------------------|--------------|--------------|-------------|----------------|
| Coverage of some software engineering concepts by standards.       |              |              |             |                |
| Concept                                                            | MIL-STD-1679 | DOD-STD-2167 | MIL-STD-498 | IEEE/EIA 12207 |
| Undelivered software                                               | X            | ✓            | ✓           | ✓              |
| Incremental and evolutionary software development                  | X            | ✓            | ✓           | ✓              |
| Tailoring                                                          | X            | ✓            | ✓           | ✓              |
| Reuse                                                              | X            | ✓            | ✓           | ✓              |
| Independence in tests                                              | X            | ✓            | ✓           | ✓              |
| Traceability                                                       | X            | ✓            | ✓           | ✓              |
| Versioning                                                         | X            | ✓            | ✓           | ✓              |
| Software transition                                                | ≈            | ≈            | ✓           | ✓              |
| Testability                                                        | X            | ≈            | ✓           | ✓              |
| Validation                                                         | X            | X            | X           | ✓              |
| Service                                                            | X            | X            | ✓           | ✓              |
| Stress tests                                                       | ✓            | X            | X           | ✓              |
| User training                                                      | X            | ≈            | ✓           | ✓              |
| Document templates                                                 | ✓            | ✓            | ✓           | X              |

(‘X’ indicates no coverage, ‘≈’ indicates partial coverage, “✓” indicates full coverage).  
Software Engineering concepts were extracted from the texts of the standards during content analysis.

Les catégories de documents peuvent être définies sommairement à l’intérieur de standards. Voici des exemples provenant de l’ISO/IEC/IEEE 24765:2017 ayant pour titre *Ingénierie des systèmes et du logiciel — Vocabulaire* [140] :

- définition de manuel d’installation : [Traduction libre] « [...] document fournissant les informations nécessaires à l’installation d’un système ou d’un composant, au paramétrage initial et à la préparation du système ou du composant pour son utilisation opérationnelle (cf. manuel de diagnostic, manuel d’installation, manuel de maintenance, manuel d’utilisation, manuel du programmeur, manuel utilisateur) [...]. » <sup>140</sup> [140] ;
- définition 1 des spécifications des exigences logicielles (SEL) : [Traduction libre] « [...] documentation des exigences essentielles (fonctions, performances,

140. « [...] document that provides the information necessary to install a system or component, set initial parameters, and prepare the system or component for operational use cf. diagnostic manual, installation manual, maintenance manual, operator manual, programmer manual, user manual [...]. »



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

contraintes de conception et attributs) du logiciel et de ses interfaces externes [...]. » <sup>141</sup> [140] ;

- définition 2 de SEL : [Traduction libre] « [...] recueil structuré des exigences (fonctions, performances, contraintes de conception et attributs) du logiciel et de ses interfaces externes [...]. » <sup>142</sup> [140].

Des catégories de documents peuvent être décrites avec des items les composant. Il y a au moins une norme qui le fait, l'ISO/IEC/IEEE 15289:2011 intitulé *Ingénierie des systèmes et du logiciel — Contenu des systèmes et produits d'information du processus de cycle de vie du logiciel (documentation)* [135]. Les éléments nécessaires pour la spécification des exigences logicielles sont énumérés dans la figure B.1.

(voir énoncé p.195)

### Explication de l'assertion OD11 : Plusieurs standards définissent des catégories de documents.

Voici des standards ou des normes qui définissent suffisamment les catégories de documents :

- la MIL-STD-498 *software development and documentation* est utilisée pour son aspect historique. Elle a influencé, entre autres, la DO-178 et l'ISO 12207 [77] ;
- la DO-178C, *Software Considerations in Airborne Systems and Equipment Certification* est la dernière version de la norme servant à certifier les logiciels critiques en avionique [105, 117, 318] ;
- l'IEC 61513 *Nuclear power plants Instrumentation and control important to safety — General requirements for systems* sert pour assurer la qualité du logiciel et du matériel utilisé dans les centrales nucléaires [137, 366] ;
- l'ISO 15289 *Systems and software engineering Content of life-cycle information items (documentation)* explicite les documents provenant des normes sur le cycle de vie système ou logiciel [135]. Quant au cycle de vie système, il est décrit dans l'ISO 15288, tandis que le cycle de vie logiciel est décrit dans l'ISO 12207.

---

141. « [...] *documentation of the essential requirements (functions, performance, design constraints, and attributes) of the software and its external interfaces* [...]. »

142. « [...] *structured collection of the requirements (functions, performance, design constraints, and attributes) of the software and its external interfaces* [...]. »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.1** – Liste des éléments d’une spécification des exigences logicielles (source : [135])

(voir énoncé p.195)

**Explication du théorème OD12 : Certains artefacts sont nécessaires pour vérifier le respect d’un procédé de développement.**

Ce n’est pas explicité, mais plusieurs l’insinuent. Richard Basque fournit une description d’artefact dans son ouvrage sur le CMMI :

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

[...] lorsqu'on réalise une activité de développement à l'aide d'un processus, on laisse des traces [...] À la façon d'un archéologue qui examine des artefacts comme preuve de l'activité d'une civilisation [...]. [25]

Leanna Rierison indique ces artefacts pour le DO-178C :

[Traduction libre] La section 11 de la norme DO-178C identifie les données du cycle de vie générées lors de la planification, du développement et de la vérification du logiciel. Dans de nombreux cas, ces données sont également appelées éléments de données ou artefacts.<sup>143</sup> [318]

(voir énoncé p.195)

### Explication de l'axiome OD13 : Les documents servent à faire du logiciel et à le compléter.

Plusieurs experts en informatique soulignent la place et l'importance de la documentation et expliquent où les documents interviennent dans le processus de développement :

- avis de Hoare : [Traduction libre] « L'idée que la documentation soit un élément ajouté à un programme après sa mise en service semble erronée par principe et contreproductive en pratique. Au contraire, la documentation doit être considérée comme une partie intégrante du processus de conception et de développement. »<sup>144</sup> [119]
- avis de Frederick P. Brooks Jr. : [Traduction libre] « promotion d'un programme en tant que produit informatique qui nécessite une documentation complète, afin que chacun puisse l'utiliser, le corriger et l'étendre. »<sup>145</sup> [39]

---

143.

*DO-178C section 11 identifies the life cycle data generated while planning, developing, and verifying the software. In many situations the term software life cycle data is also referred to as data items or artifacts.*

144. « *The view that documentation is something that is added to a program after it has been commissioned seems to be wrong in principle and counterproductive in practice. Instead, documentation must be regarded as an integral part of the process of design and coding.* »

145. « *promotion of a program to a programming product requires its thorough documentation, so that anyone may use it, fix it, and extend it.* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- avis de Parnas : [Traduction libre] « Si un produit n'est pas documenté dès sa conception, en utilisant la documentation comme outil de conception, nous économiserons un peu aujourd'hui, mais paierons beaucoup plus cher à l'avenir. »<sup>146</sup> [296] ;
- avis de Ian F. Sommerville : [Traduction libre] « 1. Ils doivent servir de support de communication entre les membres de l'équipe de développement. 2. Ils doivent constituer un référentiel d'informations système à l'usage des ingénieurs de maintenance. 3. Ils doivent fournir à la direction les informations nécessaires à la planification, au budget et à l'ordonnancement du processus de développement logiciel. 4. Certains documents doivent expliquer aux utilisateurs comment utiliser et administrer le système. »<sup>147</sup> [367].

(voir énoncé p.195)

### Explication sur l'axiome OD14 : Il n'y a pas consensus sur les phases de développement.

La littérature fournit ces informations qui ont servi à construire cet axiome :

- Phases provenant d'un ouvrage de Barry W. Boehm et Richard Turner : [Traduction libre] « développement de concepts, exigences, conception, développement, maintenance »<sup>148</sup> [34] ;
- Exemple de phases dans l'ISO : [Traduction libre] « conception, développement, production, utilisation, assistance et mise hors service »<sup>149</sup> [139] ;
- Note provenant de l'ISO qui suit l'exemple : [Traduction libre] « L'utilisation de ces termes pour définir les étapes n'est pas normative. »<sup>150</sup> [139].

(voir énoncé p.195)

---

146. « *If a product is not documented as it is designed, using documentation as a design medium, we will save a little today, but pay far more in the future.* »

147. « *1. They should act as a communication medium between members of the development team. 2. They should be a system information repository to be used by maintenance engineers. 3. They should provide information for management to help them plan, budget and schedule the software development process. 4. Some of the documents should tell users how to use and administer the system.* »

148. « *concept development, requirements, design, development, maintenance* »

149. « *concept, development, production, utilization, support, and retirement* »

150. « *Use of these terms to define stages is not normative.* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**Explication sur l’assertion OD15 : Des phases peuvent être, entre autres, l’analyse, la conception, la concrétisation, l’évaluation et la transition.**

Des noms de phases ont été sélectionnés par rapport à d’autres termes déjà employés par des procédés décrits à l’intérieur de normes ou d’ouvrages :

- les normes :
  - l’ISO 15288 [138, 392] ;
  - l’ISO 12207 [139] ;
  - le MIL-STD-498 [77] ;
- les ouvrages :
  - *Balancing Agility and Discipline : A Guide for the Perplexed* de Barry W. Boehm et et Richard Turner [34] ;
  - *Software Project Management : A Unified Framework* de Walker Royce [348] ;
  - *Managing and Leading Software Projects* de Richard E. Fairley [84] ;

La phase d’analyse :

- définition d’analyse : « Opération intellectuelle consistant à décomposer un tout en ses éléments constituants et d’en établir les relations. » [319] ;
- description : l’analyse de problèmes est en fait son internalisation et cela mène à des artefacts, tels que la spécification des exigences ;
- connexe aux phases : *Concept Development, Requirements, Requirements analysis, Business analysis, System analysis* ;

La phase de conception :

- définition de conception : « Action de concevoir. » [320] ;
- définition de concevoir : « Créer par l’imagination. » [321] ;
- description : la conception des solutions mène à des artefacts comme l’architecture ;
- connexe aux phases : *Design, Architecture* ;

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

La phase de concrétisation :

- définition de concrétisation : « Fait de se concrétiser, de devenir concret ou réel. » [323] ;
- description : la concrétisation des solutions mène à des artefacts comme le code source ;
- connexe aux phases : *Development, Implementation* ;

La phase d'évaluation :

- définition d'évaluation : « Action d'évaluer » [338] ;
- définition d'évaluer : « Porter un jugement sur la valeur, le prix de » [339] ;
- description : l'évaluation porte sur les artefacts ;
- connexe aux phases : *Assessment, Verification and Validation, Test* ;

La phase de transition :

- définition de transition : « Passage d'un état à un autre, en général lent et graduel ; état intermédiaire. » [335] ;
- description : la transition consiste à l'externalisation d'artefacts pour en nécessiter d'autres - il existe des transitions pour en faire l'exploitation ou la migration ;
- connexe aux phases : *Maintenance, Deployment, Utilization, Support, Retirement, Operation*.

(voir énoncé p.195)

### Explication de la définition OD16 : Réaliser

La définition formulée est : rendre tangible ce qui est abstrait.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition de réaliser : « Faire passer à l'état de réalité concrète (ce qui n'existait que dans l'esprit). » [334] ;
- définition 1 de concret : « Qui peut être perçu par les sens ou imaginé. » [322] ;
- définition 2 de concret : « Par opposition à abstrait, qui est directement perceptible par les sens ; palpable, tangible, matériel. » [182] ;

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- définition de tangible : « Qu'on connaît par le toucher ; matériel, sensible » [193].

(voir énoncé p.195)

### Explication de la définition OD17 : Établir

La définition formulée est : créer les conditions pour soutenir la pérennité.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 d'établir : « Mettre, faire tenir (une chose) dans un lieu et d'une manière stable » [337] ;
- définition 2 d'établir : Construire, fonder quelque chose sur quelque chose de solide » [196] ;
- définition d'être établi : S'instaurer, exister de façon durable » [196].

(voir énoncé p.195)

### Pendant la réalisation

#### Explication de l'assertion ED01 : Le professionnel informatique (temps, compétences) est nécessaire pour la documentation.

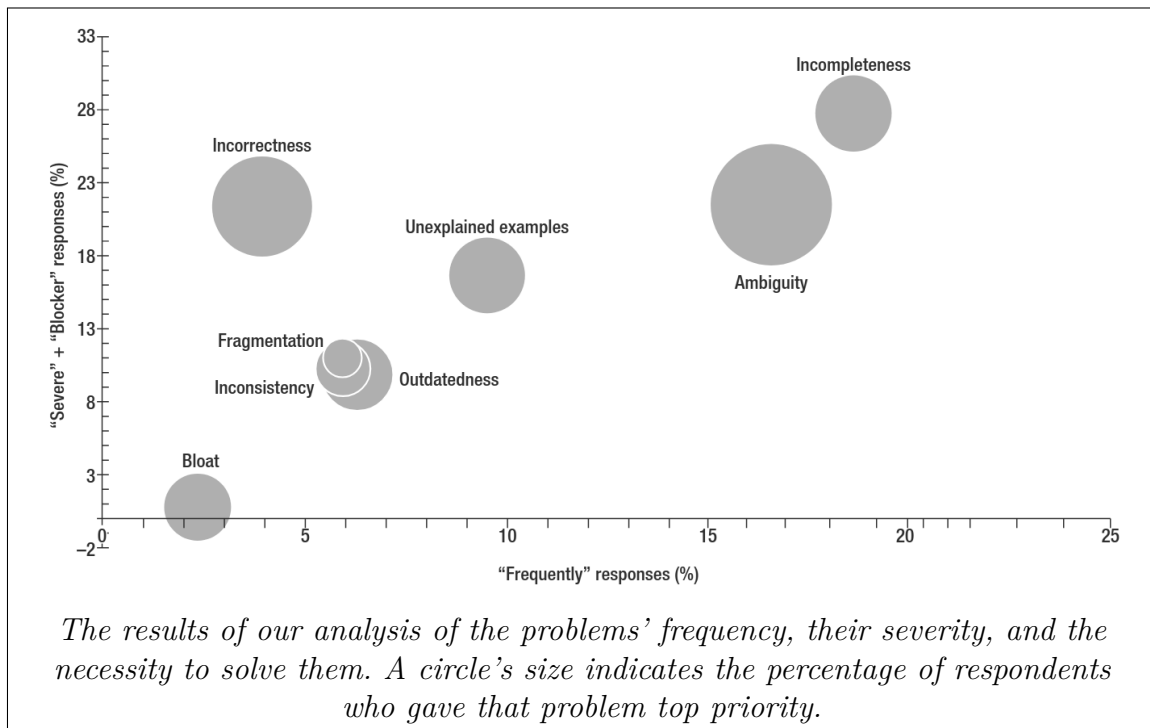
Un article de 2015 décrit deux sondages réalisés auprès de 323 professionnels informatiques d'IBM [385]. Le sujet porte sur les problèmes de documentation de l'interface du programme d'application (API, *Application Programme Interface* en anglais) et les résultats apparaissent à la figure B.2. L'article conclut que les compétences techniques sont importantes pour écrire la documentation :

[Traduction libre] Sans surprise, les problèmes les plus importants liés à la documentation des API étaient aussi ceux qui exigeaient le plus d'expertise technique. Compléter, clarifier et corriger la documentation requiert une connaissance approfondie et experte de l'implémentation de l'API. De ce fait, ces tâches s'avèrent difficiles à accomplir pour les non-développeurs ou les nouveaux contributeurs au projet.<sup>151</sup> [385]

---

151.

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.2** – Causes des problèmes de documentation selon des professionnels (source : *Figure 2* [385])

Un sondage effectué en 2020 interroge 146 professionnels en informatique, entre autres, sur les causes des problèmes de documentation [6] :

[Traduction libre] L'augmentation du budget alloué à la documentation a été une solution récurrente souvent mentionnée par les participants. Cela suggère que la documentation logicielle ne reçoit pas l'attention qu'elle mérite lors de la planification et de l'allocation des ressources logicielles. [...] Le manque de temps pour rédiger la documentation est le seul problème de cette catégorie qui a été jugé important par la majorité (65 %) des participants. De plus, il s'agit d'un problème fréquent. Les solutions

---

*Perhaps unsurprisingly, the biggest problems with API documentation were also the ones requiring the most technical expertise to solve. Completing, clarifying, and correcting documentation require deep, authoritative knowledge of the API's implementation. This makes accomplishing these tasks difficult for nondevelopers or recent contributors to a project.*



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

proposées se résument à : (i) allouer explicitement du temps, des efforts et des ressources à la documentation dans la planification du projet ; et (ii) démarrer les activités de documentation dès les premières étapes du cycle de vie du logiciel, [...].<sup>152</sup> [6]

(voir énoncé p.196)

### **Explication de l’assertion ED02 : Plusieurs professionnels informatiques considèrent le temps consacré à la documentation comme étant inutile.**

En 1974, une opinion devient répandue, elle provient d’un article écrit par Ray W. Wolverton :

[Traduction libre] La nature des programmeurs est telle que le travail intéressant est réalisé au détriment du travail ennuyeux, et la documentation est un travail ennuyeux.<sup>153</sup> [405]

En 2010, un sondage contenant 260 questions est réalisé auprès de 83 professionnels en informatique [51]. L’objectif du sondage est d’en apprendre davantage sur les tests lors d’un procédé de développement (tableau B.2). Plusieurs ont exprimé leur désaccord avec le fait qu’une documentation compréhensible devrait être essentielle.

En 2015, un sondage est réalisé sur le comportement de 178 professionnels en informatique pendant la maintenance logicielle :

[Traduction libre] Nous avons interrogé les répondants sur le type de documentation qu’ils utilisent lors de la maintenance logicielle. La majorité

---

152.

*Increasing the budget dedicated to documentation was a recurring solution often mentioned by participants. This suggests that software documentation does not receive the attention it deserves when planning and allocating software resources. [...] Lack of time to write documentation is the only issue in this category that was indicated as important by the majority (65%) of participants. Also, it is a frequently encountered issue. The proposed solutions boil down to : (i) explicitly allocating time/effort/resources to documentation in the project planning ; and (ii) starting documentation activities in the early stage of the software lifecycle, [...].*

153.

*The nature of programmers is such that interesting work gets done at the expense of dull work, and documentation is dull work.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABLEAU B.2** – Désaccord des professionnels envers des affirmations (source : [51])

| <i>Questions with the highest degree of dissatisfaction</i>                                       |                 |
|---------------------------------------------------------------------------------------------------|-----------------|
| Question                                                                                          | Dissatisfaction |
| Test cases should be written before writing code                                                  | 1.609           |
| Programming should start only after the design is completed                                       | 1.271           |
| Comprehensive documentation should be an essential part of software development                   | 1.250           |
| There should be general guidelines and principles for software development but not detailed rules | 1.204           |
| Management should encourage regular interaction between developers and customers/business people  | 1.200           |

<sup>a</sup> the dissatisfaction  $d_{q,R} = \frac{1}{|R|} \sum_{r \in R} |c_{q,r} - p_{q,r}|$  where  $q$  is the question on the practice of interest,  $R$  set of respondents in a particular category,  $c_{q,r}$  and  $p_{q,r}$  refer to the current and the preferred practice of question  $q$  as reported by the respondent  $r$

utilise des artefacts basés sur la documentation textuelle (66 %) et sur le code source lui-même (70 %), mais 40 % des personnes interrogées utilisent une notation graphique (26 % utilisent uniquement UML, 9 % une autre notation). [...] 14 % n'utilisent aucune documentation complémentaire (graphique ou textuelle) pour la maintenance du code source, c'est-à-dire qu'ils utilisent uniquement le code source comme documentation. <sup>154</sup> [87]

En 2018, un article présentait les résultats d'entretiens menés auprès de 17 professionnels en informatique et d'un questionnaire rempli par un second groupe composé de 114 répondants. Deux groupes se distinguent :

[Traduction libre] Certains développeurs semblent privilégier une approche descendante, axée sur les concepts. Ils consacrent du temps à acquérir une compréhension approfondie de l'API avant de tenter d'implémenter des

154.

We asked the respondents about what type of documentation they use during software maintenance. The majority use artifacts that are based on textual documentation (66%) and on the source code by itself (70%), but 40% of those surveyed use a graphical notation (26% use only UML, 9% another notation) [...] 14 % do not use any complementary documentation (graphical or textual) to maintain source code, i.e., they only employ the source code as documentation.

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

cas d'utilisation spécifiques. [...] Les développeurs de ce groupe ont souligné qu'ils recherchent systématiquement et consultent régulièrement la documentation fournie par le fournisseur de l'API. [...] D'autres développeurs, en revanche, adoptent une approche ascendante, axée sur le code. [...] Généralement, les développeurs de ce groupe souhaitent commencer à coder immédiatement. Les participants ont indiqué qu'ils apprennent en essayant des exemples qu'ils enrichissent progressivement. Ils ne consultent la documentation que s'ils ne trouvent pas de solution par eux-mêmes. <sup>155</sup> [219]

Une opinion revient fréquemment dans une étude menée en 2019 auprès de 227 professionnels informatiques sur l'efficacité des commentaires sur le code source :

[Traduction libre] Globalement, le message clé rapporté par certains professionnels est le suivant : « Seul le code explique le code. » <sup>156</sup> [231]

En 2022, une revue de littérature de Theo Theunissen *et al.* sur 63 articles publiés entre 2001 et 2018 dans le contexte des méthodes agiles. La conclusion met en évidence un constat important :

[Traduction libre] Les défis sont les suivants : la documentation informelle est difficile à comprendre, elle est perçue comme un gaspillage lorsqu'elle ne contribue pas au produit final, la productivité est limitée à la quantité de logiciels fonctionnels mesurée, la documentation se désynchronise

---

155.

*Some developers seem to follow a top-down approach that is concepts oriented. These developers spend some effort to build a deeper and more thorough understanding of the API before they attempt to implement particular use cases. [...] Developers in this group pointed out that they systematically search and regularly consult documentation provided by the API supplier.*

*[...] On the other hand, there are developers that take a bottom-up approach that is code oriented. [...] Typically, developers from this group want to start coding right away. Participants mentioned that they start learning by trying examples which they extend step-by-step. They check documentation only if they cannot find a solution on their own.*

156.

*Overall, the key message reported by some professionals is : « Only the code explains the code. »*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

facilement avec le code et la vision est souvent à court terme. Les bonnes pratiques incluent : une communication importante, verbale et informelle ; une utilisation pertinente des artefacts de développement à des fins de documentation, comme le TDD ; et l'utilisation de cadres d'architecture peut avoir un impact positif sur la qualité de la documentation.<sup>157</sup> [382]  
(voir énoncé p.196)

### **Explication de l'assertion ED03 : L'aversion des professionnels informatiques pour la documentation a mené à la réalisation d'une documentation minimaliste.**

En 1990, un article repasse en revue la documentation minimaliste [38]. L'article cite les différentes expérimentations menées par John Carroll chez IBM, HP, Xerox et Bell Northern Research. Plusieurs comportements sont énumérés, en voici quatre :

[Traduction libre]

- Ce sont des apprenants impatientes qui veulent se lancer rapidement dans un projet productif
- Ils parcourent les manuels et les documents en ligne sans les lire attentivement
- Ils font des erreurs, mais apprennent surtout en les corrigeant
- L'exploration autonome les motive au mieux
- Les manuels volumineux, où chaque tâche est décomposée en sous-tâches insignifiantes, les découragent plutôt que de les stimuler.<sup>158</sup> [38]

---

157.

*The challenges include : informal documentation is hard to understand, documentation is considered waste when it does not contribute to the end product, productivity is limited to the measured amount of working software, documentation is easily out-of-sync with the actual code, and there is short term focus. The practices include : a significant amount of communications happens verbally and informally ; there is a positive usage of development artifacts for documentation purposes, such as TDD ; and the use of architecture frameworks might positively influence documentation quality.*

158.

— *Are impatient learners and want to get started quickly on something productive*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

L'article soulève des problèmes avec la documentation minimaliste :

[Traduction libre]

- L'apprentissage par la découverte personnelle est moins prévisible et des lacunes ou un manque de profondeur dans l'apprentissage peuvent apparaître [...]
- Cette conception suppose un public motivé. Or, toutes les autres conceptions de documents, comme les scénarios et l'écriture structurée, supposent exactement le contraire. Laquelle est la plus juste ?
- En permettant le libre choix des objectifs, certains utilisateurs ont opté pour des objectifs inatteignables ou inefficaces.
- C'est très différent des méthodes traditionnelles d'écriture utilisées par les rédacteurs de documents [...]
- À quel moment la concision devient-elle obscure ? Les concepteurs minimalistes seront-ils plus ouverts aux questions de responsabilité que les rédacteurs de documents systématiques et exhaustifs ?
- Les concepteurs négligeront-ils l'élément essentiel des tests – le travail de fond – et se contenteront-ils de la solution de facilité, c'est-à-dire de supprimer des mots ? <sup>159</sup> [38]

- 
- *Skip around in manuals and on-line documents and rarely read them fully*
  - *Make mistakes but learn most often from correcting such mistakes*
  - *Are best motivated by self-initiated exploration*
  - *Are discouraged, not empowered, by large manuals with each task decomposed into its subtask minutiae.*

159.

- *Learning by self-discovery is less predictable and gaps or lack of depth in learning may appear [...]*
- *This design assumes a motivated audience, However, all the other document designs such as playscript and structured writing assume just the opposite. Which is more accurate ?*
- *In allowing the free choosing of goals, some users picked unattainable goals or ineffective goals.*
- *It's quite different from the ways documenters have traditionally been encouraged to write [...]*
- *When does one know when concision becomes cryptic ? Will minimalist designers be more open to liability challenges than systematic comprehensive documenters ?*
- *Will designers miss the all-important testing element-the hard work-and just employ the easiest step, i.e., cut out words.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

(voir énoncé p.196)

### **Explication de l’assertion ED04 : Le courant dominant en développement de logiciel est les méthodologies agiles.**

Des interviews menés en 2018 sont réalisés avec les experts de 19 entreprises internationales en technologie de l’information :

[Traduction libre] Le modèle le plus fréquemment mis en œuvre était le modèle Agile, sous ses trois formes : 68 % des experts ont mentionné Agile comme le modèle utilisé au sein de leur département, tandis que les autres ont mentionné le modèle en cascade. [...] La moitié d’entre eux ont indiqué que le modèle Agile avait été mis en œuvre après 2011 dans leur département.<sup>160</sup> [76]

Une exploration sur les articles portant sur l’enseignement du génie logiciel conduit à obtenir 147 articles. Les critères de sélection de ces articles portaient essentiellement sur les mots-clés (génie logiciel, apprentissage, étudiant, professeur, projet et, etc.) et s’étendaient de 2008 à 2018. Les tendances formées sont décrites par le tableau B.3. Le tableau B.4 met en évidence l’évolution du nombre d’articles par tendance, démontrant la popularité croissante des méthodes agiles dans le secteur de la recherche en enseignement du génie logiciel.

Un article indique la tendance de plus en plus grande à utiliser des méthodes agiles lors des exigences logicielles. Le titre de l’article est *The Current and Evolving Landscape of Requirements Engineering in Practice* :

[Traduction libre] Interrogés sur le cycle de vie du développement logiciel (SDLC) le plus approprié à leur projet, 51 % des projets ont déclaré utiliser plusieurs approches. Les méthodologies agiles (par exemple, Scrum,

---

160.

*most implemented model was the Agile model with its three forms, respectively : 68% of the experts mentioned Agile as the model used within their department, while the rest mentioned the Waterfall model, [...] while half of these mentioned the Agile model was implemented after 2011 in their department.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABLEAU B.3** – Catégorisation formée depuis les articles (modifié les *Tables 7* [58])

| Table 7: Keywords for common SE Trends       |  |                                                                                                                                       |
|----------------------------------------------|--|---------------------------------------------------------------------------------------------------------------------------------------|
| Category                                     |  | Properties                                                                                                                            |
| Agile Software Development                   |  | Agile practices, Lean and Agile, rapid prototyping, Enterprise Software Integration, DevOps, continuous software integration          |
| Software Implementation, Usability and Value |  | user-centered software, software usability, software non-functional requirements, software value, functionality development           |
| Global Software Engineering                  |  | global software engineering, multinational vs. local environment, cross-site, same-site software development, distributed development |
| Lean Software Startup                        |  | innovative practices in industry through Tech Startups, Lean Software Startups, BizOps, Tech Startups, Software Startups              |
| System of Systems                            |  | cloud systems, mobile systems, System of Systems, crowd-sourcing, open source software                                                |

<sup>a</sup> Retrait du filigrane *Journal Pre-proof*.

Extreme Programming ou le développement piloté par les fonctionnalités) dominant largement, puisqu'elles ont été choisies dans 70 % des projets analysés [Figure 1(a)]. Cela représente une augmentation de 24 % par rapport à l'enquête de 2013. Le modèle en cascade conserve sa part de marché habituelle (21 % des projets).<sup>161</sup> [159]

Un article décrit la récente utilisation des méthodes agiles dans le secteur critique :

161.

*When asked which software development life cycle (SDLC) best describes the ones used in the project, 51% of the projects reported using more than one approach and agile methodologies (for example, Scrum, Extreme Programming, or feature-driven development) were dominating the landscape as they were selected in 70% of the surveyed projects [Figure 1(a)]. This is an increase of 24% since the 2013 Survey. The Waterfall maintained its presence at the same level since 2013 (21% of the projects).*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABEAU B.4** – Tendances dans l’enseignement du génie logiciel (modifié la *Figure 8* [58])

| <i>The evolution of SE trends as topics in software engineering education</i> |      |      |      |      |      |      |      |      |      |      |      |
|-------------------------------------------------------------------------------|------|------|------|------|------|------|------|------|------|------|------|
|                                                                               | 2008 | 2009 | 2010 | 2011 | 2012 | 2013 | 2014 | 2015 | 2016 | 2017 | 2018 |
| Agile Software Development                                                    | 5    | 4    | 1    | 4    | 6    | 5    | 11   | 10   | 12   | 18   | 24   |
| Software Implementation.<br>Usability and Value                               | 2    | 1    | 0    | 2    | 3    | 3    | 5    | 1    | 1    | 1    | 2    |
| Global Software Engineering                                                   | 0    | 0    | 1    | 0    | 2    | 1    | 0    | 4    | 2    | 1    | 1    |
| Lean Software Startup                                                         | 1    | 0    | 0    | 0    | 0    | 0    | 0    | 0    | 3    | 4    | 4    |
| System of Systems                                                             | 0    | 0    | 0    | 0    | 0    | 1    | 0    | 0    | 0    | 0    | 1    |

<sup>a</sup> Découpage de la figure pour ne garder que le tableau et retrait du filigrane *Journal Pre-proof*.

[Traduction libre] Le développement d’un système critique pour la sécurité selon une méthodologie agile ne repose que sur peu de preuves empiriques de son efficacité. Néanmoins, quelques études de cas [24][25] sont disponibles dans la littérature et sont abordées dans les sections suivantes. L’étude de cas [24] porte sur une entreprise pharmaceutique où la méthodologie Scrum a été appliquée pour combiner le développement logiciel agile à un projet critique pour la sécurité traditionnelle. Le produit devait être conforme à plusieurs normes de sécurité, notamment celles de la FDA américaine, et plus de 100 responsables, ingénieurs et développeurs y ont participé. [...] Storer et Islam [25] présentent une étude de cas basée sur des entretiens semi-directifs menés auprès d’ingénieurs logiciels d’une grande entreprise d’avionique au Royaume-Uni. Leur objectif était d’étudier l’expérience de l’entreprise en matière d’adoption du développement logiciel agile pour les systèmes critiques pour la sécurité et les difficultés rencontrées.<sup>162</sup> [105]

162.

*The development of a safety-critical system following an agile process has little empirical evidence of its effectiveness. Nevertheless, some case studies [24][25] are available in the literature, and they are discussed in the following sections. A pharmaceutical company is the case study in which a Scrum process was applied to combine agile software development in a traditional safetycritical project in [24]. The product should be compliant with several safety standards, including those associated with the US FDA, and started with more than 100 managers, engineers, and developers involved. [...] Storer and Islam [25] present a case study where semi-structured interviews were performed with software*



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

Une analyse décrit l'adoption des méthodes agiles dans le secteur de la défense américaine :

[Traduction libre] En 2009, le Congrès américain a promulgué la loi d'autorisation de la défense nationale pour l'exercice 2010 (NDAA 2010), imposant au département de la Défense (DoD) la mise en œuvre d'un nouveau processus d'acquisition des systèmes informatiques [51]. Ce nouveau processus intégrait les principes du développement agile, tels que l'implication précoce et continue de l'utilisateur, le déploiement rapide de multiples incréments ou versions, le prototypage successif précoce pour soutenir une approche évolutive, et une approche modulaire de systèmes ouverts. Depuis la promulgation de la NDAA 2010, un nombre croissant de programmes d'acquisition du DoD, hors technologies de l'information, ont adopté le développement agile de logiciels comme méthode pour fournir aux combattants des capacités nouvelles et améliorées de manière rapide et reproductible, évitant ainsi les retards et les dépassements de coûts associés aux méthodes traditionnelles, telles que la méthode en cascade [51].<sup>163</sup> [344]

Cette même analyse conclut que :

[Traduction libre] Il convient toutefois de noter la disparité de l'échantillon : 36 projets agiles contre 176 pour les projets traditionnels. [...] Les

---

*engineers employed in a large avionics company in the United Kingdom. Their goal was to investigate the company's experiences in the adoption of agile software development to SCS and the difficulties experienced.*

163.

*In 2009, the U.S. Congress enacted the National Defense Authorization Act for Fiscal Year 2010 (2010 NDAA) requiring the DoD to implement a new acquisition process for IT systems [51]. This new process included principles of agile development such as early and continual involvement of the user, multiple rapidly executed increments or releases, early successive prototyping to support an evolutionary approach, and a modular open-systems approach. Since the enactment of the 2010 NDAA, an increasing number of nonIT DoD acquisition programs have turned to agile software development as a method for delivering new and enhanced capabilities to the warfighters on a rapid and repeatable basis, avoiding delays and cost overruns associated with traditional methods such as waterfall [51].*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

projets logiciels agiles semblent légèrement plus performants que les projets traditionnels en termes de gains de productivité. [...] La question de savoir si le développement agile est globalement plus efficace que le développement traditionnel reste ouverte, car les différences observées ne sont pas significatives.<sup>164</sup> [344]

(voir énoncé p.196)

### Explication de l’assertion ED05 : La communication est privilégiée à la documentation.

En 2002, un article publie les résultats d’un sondage mené auprès de 48 professionnels informatiques sur la documentation. Dans la conclusion, il est mentionné :

[Traduction libre]

- Le contenu d’un document peut être pertinent même s’il n’est pas à jour. (Toutefois, le maintenir à jour reste un objectif louable). De ce fait, les technologies devraient privilégier des solutions de documentation simples d’utilisation, légères et éphémères.
- La documentation est un outil de communication essentiel et doit toujours avoir une utilité. Les technologies devraient permettre de communiquer des idées rapidement et efficacement, plutôt que d’imposer des règles strictes de validation et de vérification pour établir des faits.<sup>165</sup> [88]

---

164.

*The caveat is the uneven sample of 36 for agile vs. 176 for traditional. [...] Agile software projects appear to perform slightly better than traditional in term of productivity gains. [...] Whether agile development is more effective than traditional in general remains unanswered as differences are not significant.*

165.

- *Document content can be relevant even if it is not up to date. (However, keeping it up to date is still a good objective). As such, technologies should strive for easy to use, lightweight and disposable solutions for documentation.*
- *Documentation is an important tool for communication and should always serve a purpose. Technologies should enable quick and efficient means to communicate ideas as opposed to providing strict validation and verification rules for building facts.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

Une analyse de 2023 porte sur les fichiers de documentation *README* (fichiers lisez-moi en français) à l'intérieur de projets ( $n > 10000$ ) en code source ouvert. L'analyse met en évidence que les instruments de communication prennent de plus en plus de place au détriment de la documentation :

[Traduction libre] La documentation logicielle classique cède la place à la communication. Du moins, dans le domaine des logiciels libres sur GitHub, cette dernière s'appuie sur une multitude de plateformes caractérisées par un débit élevé, une grande volatilité et une forte hétérogénéité. La vision initiale d'une documentation développeur à la demande [55] préconisait un changement de paradigme. Ce changement a bien eu lieu, mais pas dans le sens envisagé par Robillard *et al.* il y a cinq ans. Les nouvelles plateformes de communication soulèvent de nouveaux défis et offrent de nouvelles opportunités pour la documentation logicielle moderne. Il est temps d'explorer ces nouvelles formes de documentation.<sup>166</sup> [314]

Voici un résumé du tableau *Documentation tools in CSD retrieved from the studies for RQ2*. (Table 11) présenté dans la revue de littérature de 2022 de Theo Theunissen *et al.*, qui présente les outils et le nombre d'études en lien [382] : Word, Excel, Powerpoint, etc. (40) ; *Wiki* (39) ; *Source control* (5) ; *Scripts* (9) ; *Markdown editors* (2) ; *Verbal communication* (7).

(voir énoncé p.196)

**Explication de l'axiome ED06 : Il y a une dépendance entre la qualité du logiciel et la qualité de la documentation.**

Un article publié en 2003 analyse l'avis d'experts au sujet de différentes métriques [222]. Ces experts sont : Alain Abran, David Card, William Everett, Jon Hagar, Herbert Hecht, Watts Humphrey, Michael Lyu, Jean-Claude Laprie, William Petrick et Allen Nikora. Les trois mesures les plus importantes par phase de développement sont

---

166.

*Classical software documentation is being replaced by « communication ». At least in open source software on GitHub, it is supported by a plethora of platforms characterized by high throughput, volatility, and heterogeneity. The original vision of on-demand developer documentation [55] advocated for a paradigm shift. A shift did happen, but it was not in the direction foreseen by Robillard et al. five years ago. The new communication platforms bring new challenges and opportunities for modern software documentation. It is time to shed light on new forms of documentation.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABLEAU B.5** – Experts indiquant les trois mesures les plus importantes (source : *Table 9* [222])

| <i>Top Three Measures per Phase</i>        |                       |                       |                     |
|--------------------------------------------|-----------------------|-----------------------|---------------------|
| Requirements                               | Design                | Implementation        | Testing             |
| Fault density                              | Design defect density | Code defect density   | Failure rate        |
| Requirements specification change requests | Cyclomatic complexity | Design defect density | Code defect density |
| Error distribution                         | Fault density         | Cyclomatic complexity | Coverage factor     |

présentées au tableau B.5. Plusieurs d’entre elles concernent directement la documentation ou des artefacts contenus dans la documentation (*Requirements specification change request*, *Cyclomatic complexity*, etc.).

(voir énoncé p.196)

### Explication de la citation ED07 : Il y a un manque d’intérêt scientifique sur l’étude de la documentation.

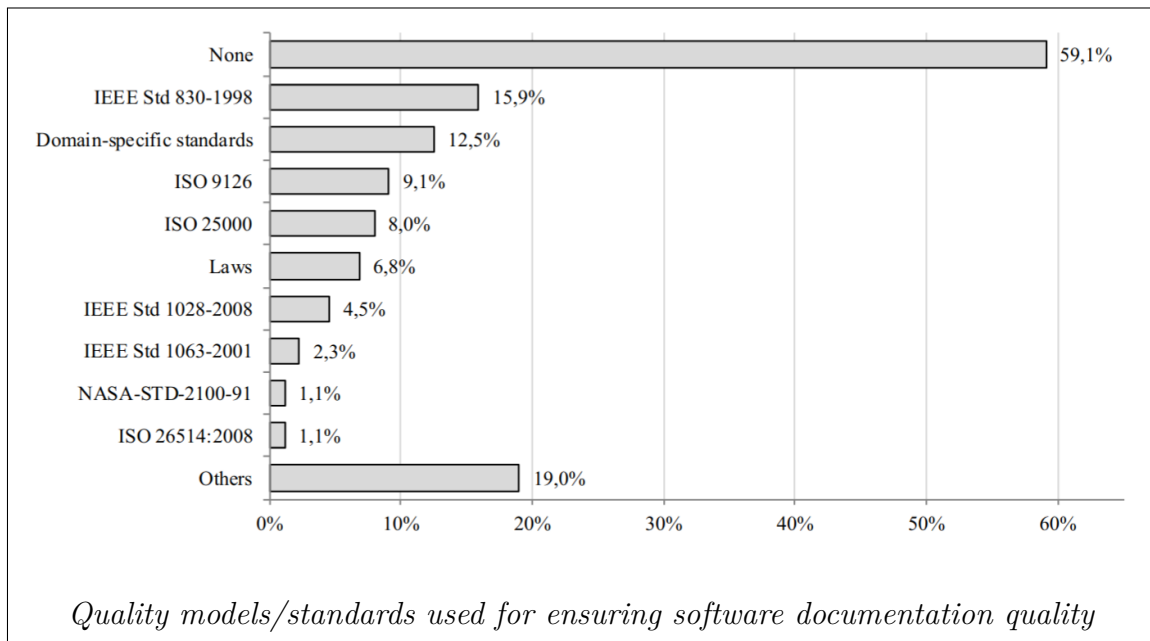
David L. Parnas explique le peu d’intérêt envers la documentation de la part de la recherche [294]. Selon lui, l’absence de modèle y est pour beaucoup. (voir P11 p.12 la citation complète)

(voir énoncé p.196)

### Explication de l’assertion ED08 : Depuis 2010, l’attention se concentre désormais sur la qualité du code source plutôt que sur celle du produit.

En 2023, un article a été publié qui regroupait deux revues de littérature et un sondage sur le thème de la qualité logiciel à travers le temps [230]. Les caractéristiques de qualité logiciel ont une plus grande fréquence en 2010. Les principales caractéristiques de qualité d’un produit en 2010 avec la fréquence d’apparition sont ; *functionality* (19), *maintainability* (17), *reliability* (14), *efficiency* (13), *usability* (12), *portability* (6). Cela coïncide avec la popularité des modèles de qualité provenant de l’ISO 9126 puis de l’ISO 25010. Après, un déclin s’amorce et arrive des caractéristiques de qualité sur le code source. En 2023, les caractéristiques de qualité observée pour le code

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.3** – Standards utilisés pour la documentation selon des professionnels (source : *Figure 8* [303])

source avec la fréquence d'apparition sont ; *reusability* (10), *security* (9), *readability* (6), *completeness* (3), *reliability* (3), *conciseness* (2), *correctness* (2).

Des métriques sur les documents s'appliqueraient sur toutes les phases. Une revue de littérature de 2021 porte sur les métriques se retrouvant dans des articles de 2009 à 2019 [62]. Ces articles peuvent s'adresser à toutes les phases ou à certaines, voici la distribution des articles par phase : *all* (14) ; *code* (43) ; *planification* (1) ; *verification and validation* (5) ; *requirement* (3) ; *design* (10).

(voir énoncé p.196)

**Explication de l'assertion ED09 : Plusieurs affirment ne pas utiliser de modèles de qualité pour la documentation ou utiliser d'anciens modèles de qualité.**

En 2014, un sondage sur la documentation interroge 88 professionnels informatiques [303]. Certains résultats se retrouvent dans la figure B.3 : près de 60 % des professionnels informatiques n'utilisent aucun standard ou modèle de qualité. Seuls 9 %

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

déclarent utiliser un standard ou un modèle de qualité actuel, soit l'ISO 25000 et l'ISO 26514. Voici une description de quelques standards :

- l'IEEE 830-1998 est remplacé en 2011 par l'ISO 29148, la dernière version date de 2018. Il s'agit d'un guide pour écrire les différents types d'exigences. Les types d'exigences sont : [Traduction libre] « [...] spécification des exigences métier, spécification des exigences des parties prenantes, spécification des exigences système, spécification des exigences logicielles [...]. »<sup>167</sup> [141] ;
- l'ISO 9126 est remplacée par l'ISO 25010 qui est comprise dans la série ISO 25000 qui date de 2005. À l'époque, elle fournit quelques métriques sur la documentation pour mesurer : l'efficacité de la documentation d'utilisateur, la complétude de la documentation d'utilisateur, la traçabilité entre les différents documents [130, 131, 132, 133] ;
- l'ISO 25000 décrit le *Systems and software Quality Requirements and Evaluation* (SQuaRE). L'ISO 25010 décrit quant à elle les qualités d'un produit.

(voir énoncé p.196)

### Explication de l'assertion ED10 : Les caractéristiques de qualité de documentation peuvent être déduites de la littérature ou des artefacts.

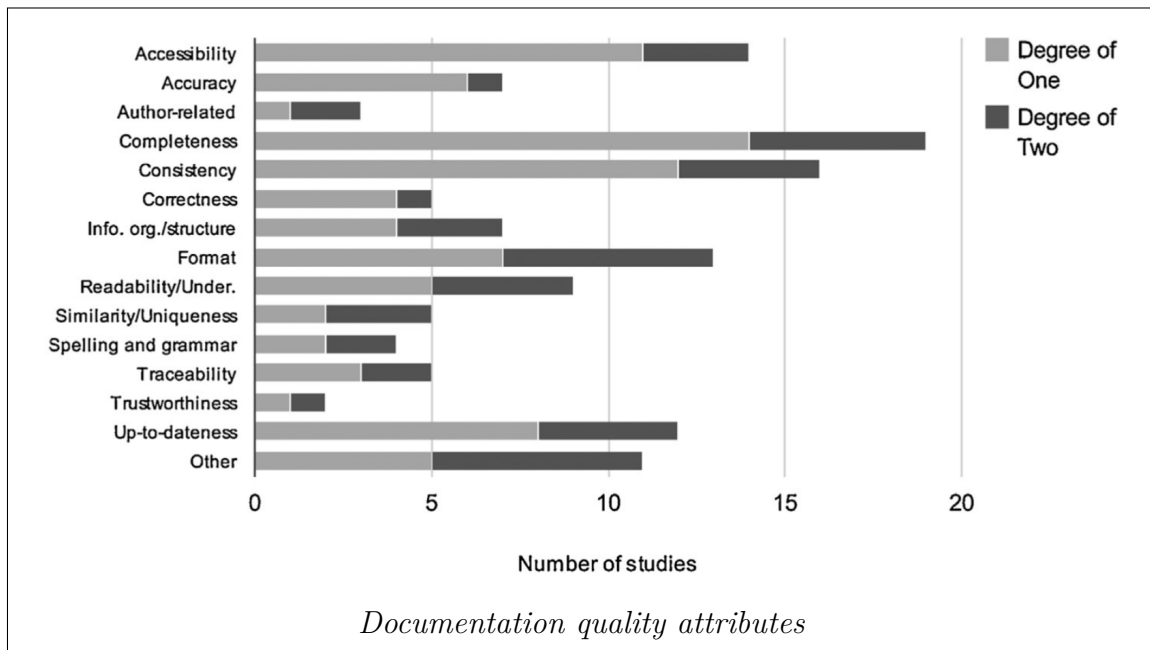
Une revue de littérature portant sur le coût, le bénéfice et la qualité de la documentation conduit à considérer 69 articles publiés sur une période s'étalant entre 1971 et 2011 [412]. Cette revue a permis de déduire de ces différents articles, des caractéristiques de qualité pour la documentation qui se trouvent illustrées dans la figure B.4.

En 2019, une étude a été menée sur les problèmes liés à la documentation [7]. Au total, 878 artefacts ont été examinés, incluant des courriels, des discussions sur des forums et diverses demandes, tous liés à la documentation. Voici le classement de ceux-ci sous différentes caractéristiques dans le tableau B.6. Il y a la complétude, l'usage et la lisibilité qui reviennent fréquemment.

---

167. « [...] *business requirements specification, stakeholder requirements specification, system requirements specification, software requirements specification* [...]. »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



<sup>a</sup> Degré un : articles traitant la caractéristique de manière qualitative.

<sup>b</sup> Degré deux : articles traitant la caractéristique de manière quantitative.

**FIGURE B.4** – Nombre d’articles abordant la qualité de la documentation (source : *Figure 18* [412])

**TABLEAU B.6** – Problèmes liés à la documentation (modifié de : [7])

|                                                           |                                          |
|-----------------------------------------------------------|------------------------------------------|
| <i>Information content - What (485) :</i>                 | <i>Information content - How (255) :</i> |
| — <i>correctness</i> (72) ;                               | — <i>usability</i> (138) ;               |
| — <i>completeness</i> (268) ;                             | — <i>maintainability</i> (21) ;          |
| — <i>up-to-dateness</i> (190) ;                           | — <i>readability</i> (107) ;             |
| <i>Process related - How (81) :</i>                       | — <i>usefulness</i> (20) ;               |
| — <i>internationalization</i> (20) ;                      | <i>Tool related - How (134) :</i>        |
| — <i>contributing to documentation</i> (50) ;             | — <i>bug/issue</i> (17) ;                |
| — <i>doc-generator configuration</i> (4) ;                | — <i>support/expectations</i> (34) ;     |
| — <i>development issues caused by documentation</i> (8) ; | — <i>help required</i> (90) ;            |
| — <i>traceability</i> (5) ;                               | — <i>tool migration</i> (4).             |

Plusieurs de ces caractéristiques de qualité reviennent encore lors d’une revue de littérature comportant 14 articles portant sur les problèmes de documentation (ta-

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABLEAU B.7** – Caractéristiques de qualité de la doc. abordées dans des articles  
(source : *Table 1* [354])

| <i>Quality attributes mentioned on each publication.</i> |            |            |            |             |            |             |             |            |             |            |             |            |             |             |
|----------------------------------------------------------|------------|------------|------------|-------------|------------|-------------|-------------|------------|-------------|------------|-------------|------------|-------------|-------------|
| <b>Attributes</b>                                        | <b>[6]</b> | <b>[2]</b> | <b>[7]</b> | <b>[18]</b> | <b>[9]</b> | <b>[12]</b> | <b>[13]</b> | <b>[8]</b> | <b>[20]</b> | <b>[4]</b> | <b>[10]</b> | <b>[1]</b> | <b>[15]</b> | <b>[19]</b> |
| Accessibility                                            |            |            | ✓          |             |            |             |             |            |             |            |             |            |             |             |
| Accuracy                                                 |            | ✓          |            |             |            |             |             |            |             |            |             | ✓          |             |             |
| Automatability                                           | ✓          |            |            |             |            |             |             |            |             |            |             |            |             |             |
| Completeness                                             |            | ✓          | ✓          | ✓           | ✓          |             |             |            | ✓           | ✓          | ✓           | ✓          |             |             |
| Consistency                                              |            | ✓          |            | ✓           |            |             |             |            |             |            |             |            |             | ✓           |
| Correctness                                              |            |            |            | ✓           |            |             | ✓           |            |             |            |             |            |             |             |
| Cost                                                     |            | ✓          | ✓          |             |            |             | ✓           |            |             |            |             |            |             | ✓           |
| Effort                                                   |            | ✓          |            |             |            |             |             |            |             |            |             |            |             |             |
| Readability                                              |            |            |            |             |            |             | ✓           |            |             |            |             | ✓          | ✓           | ✓           |
| Relevance                                                |            |            |            |             |            |             |             |            |             |            |             |            | ✓           | ✓           |
| Standardization                                          |            | ✓          | ✓          |             |            |             |             |            |             |            |             |            |             | ✓           |
| Sufficiency                                              |            |            | ✓          |             | ✓          |             | ✓           |            |             |            |             |            | ✓           | ✓           |
| Understandability                                        |            | ✓          |            |             |            | ✓           |             |            |             |            |             |            |             | ✓           |
| Up-to-dateness                                           | ✓          |            | ✓          | ✓           | ✓          |             | ✓           | ✓          | ✓           |            |             | ✓          | ✓           | ✓           |
| Usability                                                |            | ✓          |            |             |            |             |             |            |             |            |             | ✓          |             | ✓           |
| Usefulness                                               |            | ✓          |            |             |            |             | ✓           |            |             |            |             | ✓          |             | ✓           |

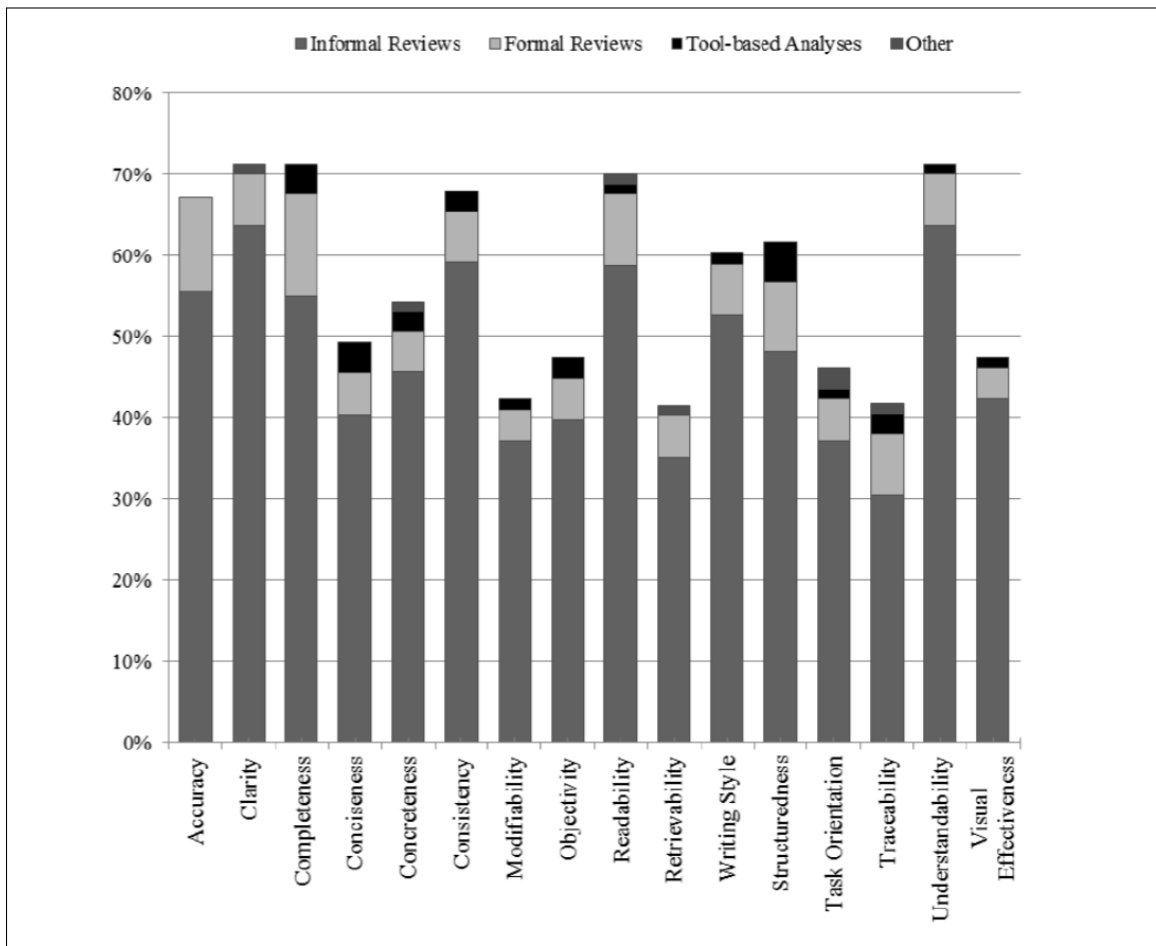
bleau B.7) [354].  
(voir énoncé p.196)

### Explication de la citation ED11 : Plusieurs professionnels informatiques affirment évaluer la documentation à travers des revues informelles.

En 2014, un sondage interroge 88 professionnels informatiques sur la documentation, sur les techniques de vérification et la validation employées (figure B.5) la revue informelle est la technique la plus utilisée [303]. D'ailleurs, dans le même article, plusieurs affirment ne pas utiliser de modèles de qualité (vers ED09).  
(voir énoncé p.196)



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.5** – Techniques d'évaluation de la qualité de la documentation utilisées par des professionnels (source : *Figure 13* [303])

**Explication de l'assertion ED12 : Il y a une tendance à moins mesurer la documentation.**

L'ouvrage *A framework of Software Measurement* de Horst Zuse cite de Bill Hetzel, les résultats sont dans la tableau B.8 et il en déduit ceci :

[Traduction libre] Il est intéressant de constater que la plupart des mesures énumérées ne caractérisent pas la qualité du produit. Nombre d'entre elles ne sont utiles qu'en phase de maintenance. Afin d'éviter les efforts et les problèmes de maintenance, il est essentiel d'intégrer les mesures logicielles

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABLEAU B.8** – Mesures les plus utilisées en industrie en 1990 (source : *Figure 9.3* [413])

|                                               |        |
|-----------------------------------------------|--------|
| 1. Number of defects found after release      | (61%), |
| 2. Number of changes or change requests       | (55%), |
| 3. User or customer satisfaction              | (52%), |
| 4. Number of defects found during development | (50%), |
| 5. Documentation completeness and accuracy    | (42%), |
| 6. Time to identify and correct defects       | (40%), |
| 7. Defect distributions by type or class      | (37%), |
| 8. Effort by mature function or feature       | (32%), |
| 9. Test coverage of specifications            | (31%), |
| 10. Test coverage of code                     | (31%), |
| 11. Defect density                            | (27%), |
| 12. Module or design complexity               | (24%), |
| 13. Number of source lines delivered          | (22%), |
| 14. Documentation size and complexity         | (20%), |
| 15. Number of reused source lines             | (15%), |
| 16. Number of function points                 | (10%). |

dès les premières phases du cycle de vie du logiciel. Nous sommes d'avis que la mesure logicielle est nécessaire tout au long de ce cycle de vie. <sup>168</sup> [413]

Dans l'ouvrage de Bill Hetzel, on trouve plus de détails sur le sondage ( $n = 800$ ) qui a permis de constituer le tableau B.8 :

[Traduction libre] En 1990, *Software Quality Engineering* a mené une vaste enquête visant à mesurer l'utilisation des mesures logicielles dans l'industrie et à identifier les meilleures pratiques des entreprises et des projets les plus performants. [...] L'enquête a été diffusée auprès de huit cents organisations logicielles à travers le monde. Les répondants ont été invités à indiquer le degré d'utilisation, au sein de leur organisation, d'une

---

168.

*It is interesting to observe that most of the listed measures do not characterize the product quality. Many of them can only be used in the maintenance phase. In order to avoid maintenance effort and problems, software measures also should be used in the very early phases of the software life-cycle. It is our view, that software measurement is necessary during the whole software life-cycle.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

liste représentative de mesures logicielles sélectionnées. Les pratiques des entreprises étaient très variables. Un très faible pourcentage d'entreprises ont déclaré utiliser régulièrement la plupart des mesures. Un tiers à la moitié n'en utilisaient aucune! <sup>169</sup> [116]

En 2018, un sondage interroge 129 professionnels informatiques dans le domaine de la recherche sur l'usage des mesures, il y aurait au plus deux personnes connaissant des mesures sur la documentation :

[Traduction libre] Autrement dit, (1,0) indique qu'une personne connaissait la métrique, mais que personne ne l'a utilisée. Chaque métrique a été mentionnée comme connue et comme utilisée. [...] Métriques de processus : quantité de documentation (1,0), nombre de lancements d'applications (1,1), auteurs/contributeurs (1,0), temps de cycle (3,3), heures de développement par *user story* (1,1), années-personnes de développement (1,0), documentation (1,0) [...]. <sup>170</sup> [81]

(voir énoncé p.197)

### Explication l'assertion ED13 : Les métriques courantes sur le code source et la documentation sont utiles une fois le projet terminé.

Un article de 1988 où Barry W. Boehm et P. N. Papaccio donne des résultats sur le coût de la documentation :

---

169.

*In 1990 Software Quality Engineering conducted a large-scale survey aimed at measuring how industry was using software measurements and to benchmark what best companies and projects were doing. [...] The survey was distributed to eight hundred software organizations around the world. Respondents were asked to report their organization's degree of usage of a representative list of selected software measures. Company practices were highly variable. A very small percentage of companies reported regular use of most of the measures. Another one-third to one-half used none of the measures!*

170.

*That is (1,0) indicates one person knew the metric, but no one actually used it. each metric was mentioned as known and as used, respectively. [...] Process Metrics : amount of documentation (1,0), app launch count (1,1), authors/committers (1,0), cycle time (3,3), development hours/story (1,1), development man [person] years (1,0), documentation (1,0) [...].*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

[Traduction libre] Le volume de documentation par rapport au nombre de lignes de code varie généralement selon l'application ; la référence [70] indique une moyenne de 28 pages de documentation pour mille instructions (pp/KDSI) pour les programmes commerciaux internes et de 66 pp/KDSI pour les logiciels commerciaux de taille équivalente (50 KDSI). Le rapport entre l'effort consacré à la documentation et celui consacré au code est en moyenne d'environ 60 :40 dans la base de données COCOMO [23] et d'environ 67 :33 pour les grands projets TRW [20].<sup>171</sup> [32]

En 2020, un article fait l'analyse de 81 articles provenant de différentes conférences en informatique. Parmi ces articles, 35 articles exposent la taille du code source, et 28 de ceux-là utilisent le nombre de lignes ou le nombre de lignes de code source pour le faire.

[Traduction libre] Nous avons analysé des articles récents issus de conférences de premier plan dans les domaines des langages de programmation, du génie logiciel et des systèmes : POPL, PLDI, OOPSLA, ICSE, ASE, ASPLOS, OSDI et SOSP. [...] La taille du code est très largement mesurée en lignes. Parmi les articles analysés, 80 % de ceux qui utilisent des métriques de code emploient le nombre de lignes (loc) ou le nombre de lignes de code source (sloc).<sup>172</sup> [12]

Alain Abran écrit dans son ouvrage *Software Metrics and Software Metrology* sur les limitations à compter les lignes de code source :

---

171.

*The volume of documentation with respect to lines of code tends to vary by application ; Reference [70] reports a typical 28 pages of documentation per thousand instructions (pp/KDSI) for internal commercial programs and a typical 66 pp/KDSI for commercial software products of the same size (50 KDSI). The proportion of documentation-related to code-related effort averaged about 60 :40 over the COCOMO data base of projects [23] and about 67 :33 for large TRW projects [20].*

172.

*we surveyed recent papers from top-tier conferences in Programming Languages, Software Engineering, and Systems : POPL, PLDI, OOPSLA, ICSE, ASE, ASPLOS, OSDI, and SOSP. [...] Overwhelmingly, code is measured in lines of code. Of the papers we surveyed, 80% of the papers who employ code metrics use loc or sloc.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

[Traduction libre] Les développeurs d'autres types de logiciels ont dû utiliser le nombre de lignes de code source (SLOC) comme mesure de taille. Cependant, comme cette taille n'est connue avec précision qu'une fois le logiciel terminé, il est difficile de l'utiliser pour des estimations. De plus, étant donné sa dépendance à la technologie, le nombre de lignes de code source est difficile à utiliser pour comparer les performances, notamment entre différentes technologies.<sup>173</sup> [1]

Aussi pour uniformiser l'usage de cette métrique Barry W. Boehm utilise une liste de contrôle créée par le Software Engineering Institute (SEI) [33].

(voir énoncé p.197)

**Explication sur l'assertion ED14 : Il est difficile de connaître le coût de la documentation, même proportionnellement au code source.**

En 1975, Barry W. Boehm produit un article ayant pour titre *The high cost of software* avec les données provenant du document (*October 1974*) *Air Force-Industry Workshop on Software Costs held at USAF Electronics Systems Division.*, il indique que le coût de la documentation peut représenter 10 % du coût total :

[Traduction libre] La plupart des données logicielles proviennent de l'Armée de l'air, principalement de l'étude CCIP-85 et du récent atelier conjoint Armée de l'air-industrie sur les coûts logiciels, mais elles sont probablement assez représentatives d'autres activités logicielles. [...] Quantité de documentation. L'expérience montre que les coûts de documentation représentent environ 10 % du coût total de développement logiciel. Pour la documentation non automatisée [...].<sup>174</sup> [123]

---

173.

*Developers of other types of software have had to use counts of Source Lines of Code (SLOC) as their size measure. But, as this size is only known accurately after the software is complete, it is difficult to use it for estimation. Moreover, being technology-dependent, SLOC counts are difficult to use for performance comparisons, especially across multiple technologies*

174.

*Most of the software data come from the Air Force, primarily from the CCIP-85 study<sup>4</sup> and the recent Air Force Industry Software Cost Workshop, but they are probably fairly*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

Un article de 1988 où Barry W. Boehm et P. N. Papaccio donnent des résultats sur le coût de la documentation qui avoisine les 60 % :

[Traduction libre] [...] l'effort consacré à la documentation et celui consacré au code est en moyenne d'environ 60 :40 dans la base de données COCOMO [23] et d'environ 67 :33 pour les grands projets TRW [20].<sup>175</sup> [32]

En 1995, la National Aeronautics and Space Administration (NASA) publie son *Software Measurement Guidebook* avec un résultat sur le coût de la documentation de 11 % (figure B.6) [226] :

[Traduction libre] Ces données sont faciles à collecter dans la plupart des entreprises de développement logiciel. La figure 6-7 présente les données recueillies auprès d'une grande organisation de la NASA.<sup>176</sup> [226]

Dans l'appendice du guide, il y a le *Personnel Resources Form* [226]. Ce formulaire confirme que certains artefacts ne sont pas considérés comme de la documentation, même s'ils se retrouvent inclus dans celle-ci.  
(voir énoncé p.197)

### Explication sur l'assertion ED15 : Les documents contenant des modèles peuvent servir à effectuer un contrôle.

La norme ISO 19761 intitulée *Software engineering COSMIC : A functional size measurement method* indique :

[Traduction libre] La norme internationale vise à répondre aux besoins a) des fournisseurs de logiciels qui doivent traduire les exigences des clients

---

*representative of other software activities elsewhere. [...] Amount of documentation. Experience indicates that documentation costs run about 10 percent of the total software development cost. For non automated documentation [...].*

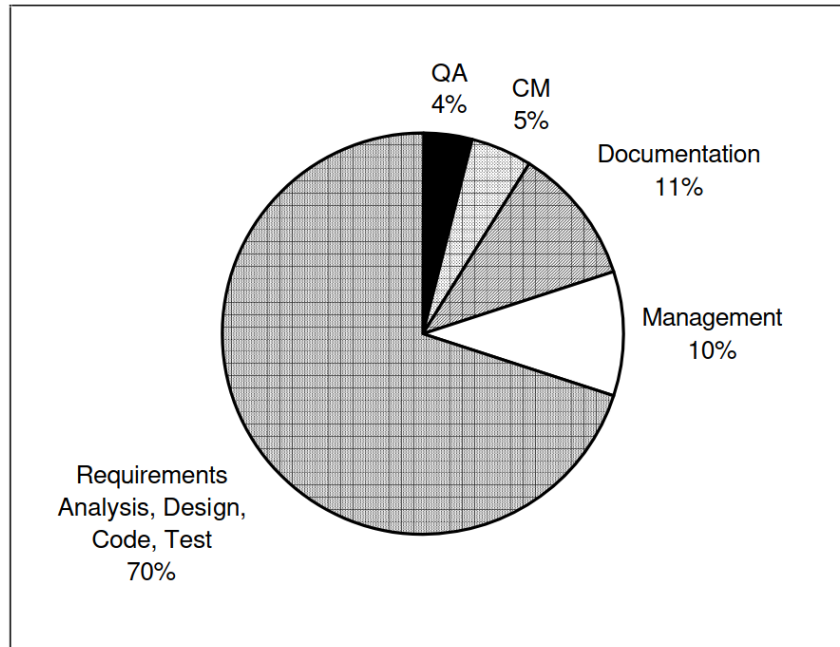
175.

*[...] effort averaged about 60 :40 over the COCOMO data base of projects [23] and about 67 :33 for large TRW projects [20].*

176.

*These data are easy to collect in most software organizations. figure 6-7 shows the data collected from one large NASA organization.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.6** – Répartition typique des ressources d'un projet logiciel affilié à la NASA  
(source : *Figure 6-7* [226])

en taille fonctionnelle du logiciel à produire, une activité clé dans l'estimation des coûts de leur projet, b) des clients qui souhaitent connaître la taille fonctionnelle du logiciel livré, un élément important pour mesurer la performance du fournisseur.<sup>177</sup> [129]

En 2016, Anandi Hira et Barry W. Boehm exposent l'efficacité des points de fonction dans les estimations de l'effort pour la maintenance logicielle et sa baisse d'efficacité lorsque cette métrique y est ajoutée :

---

177.

*The International Standard aims to meet the needs of*

- a) software suppliers facing the task of translating customer requirements into the functional size of software to be produced as a key activity in their project cost estimating,
- b) customers who want to know the functional size of delivered software as an important component of measuring supplier performance.

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

[Traduction libre] [...] Dans l’environnement de développement d’UCC, les points de fonction ont permis d’obtenir des estimations d’effort précises pour les projets intégrant de nouvelles fonctionnalités. L’utilisation d’un ratio points de fonction/lignes de code source (SLOC) accroît indéniablement l’incertitude ; par conséquent, les estimateurs de coûts ne devraient pas recourir à cette méthode s’ils attendent ou exigent des estimations d’effort précises. <sup>178</sup> [118]

(voir énoncé p.197)

### **Explication de l’assertion ED16 : Plusieurs activités de différentes phases modifient des documents.**

Dans son ouvrage *Object-Oriented and Classical Software Engineering*, Stephen R. Schach consacre une section à nommer les raisons pour lesquels la documentation n’est pas dans sa propre phase, il conclut :

[Traduction libre] Par conséquent, de même qu’il n’existe pas de phase de planification ou de phase de test distincte, il n’existe pas de phase de documentation distincte. Au contraire, la planification, les tests et la documentation doivent être des activités qui accompagnent toutes les autres activités lors du développement d’un produit logiciel. <sup>179</sup> [355]

Un sondage de 2014 chez 88 professionnels informatiques au sujet de la documentation (figure B.7) indique que les documents sont modifiés, peu importe la phase.

(voir énoncé p.197)

---

178.

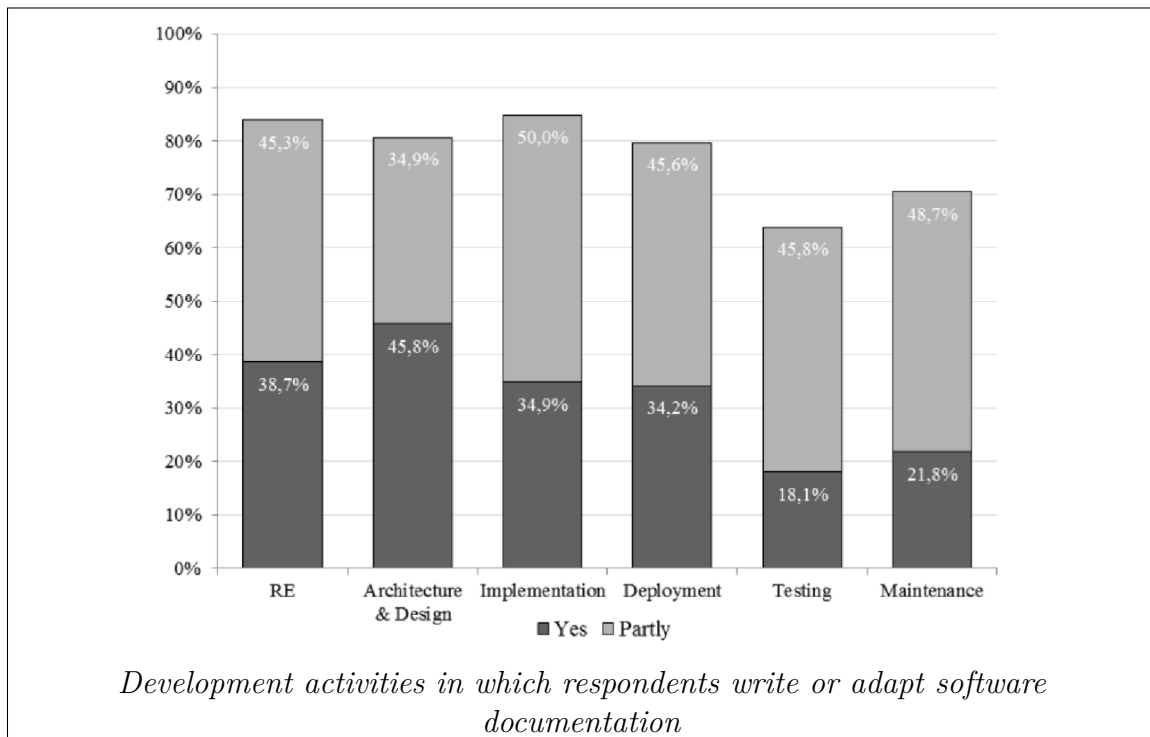
[...] *In UCC’s development environment, FPs led to accurate effort estimates for projects adding new functions. Using a FPs to SLOC ratio definitely adds additional uncertainty, and therefore, cost estimators should not use this method for effort estimation if they expect or require accurate effort estimates.*

179.

*Therefore, just as there is no separate planning phase or testing phase, there is no separate documentation phase. Instead, planning, testing, and documentation should be activities that accompany all other activities while a software product is being constructed.*



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



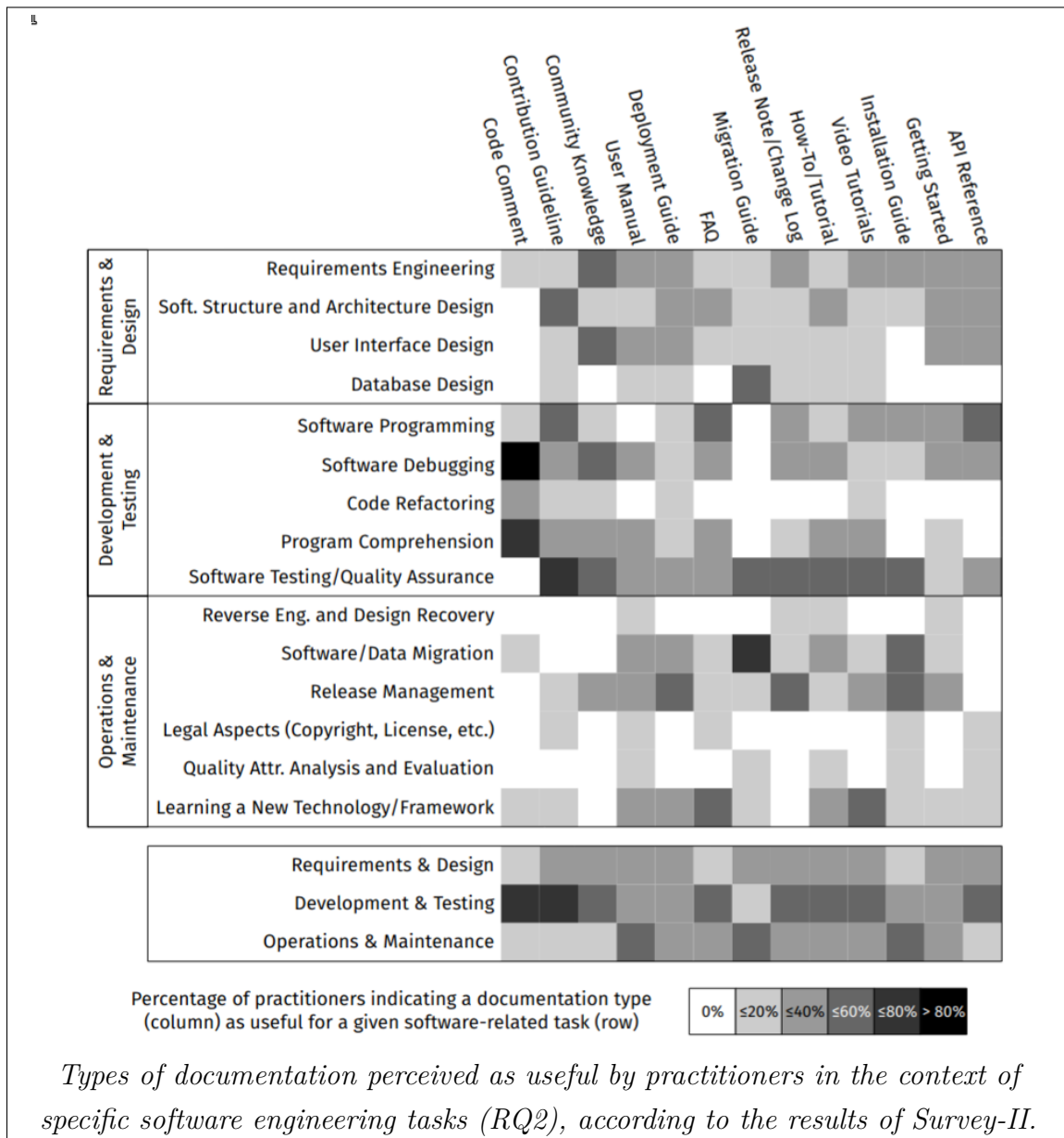
**FIGURE B.7** – Phases dans lesquelles des professionnels modifient la documentation (source : *Figure 8* [303])

**Explication l’assertion ED17 : Un même document peut être utilisé lors de plusieurs activités.**

Un sondage de 2020 chez 146 professionnels informatiques au sujet de la documentation (figure B.8) montre que certaines catégories de documents sont utilisées pour différentes activités de développement.

(voir énoncé p.197)

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

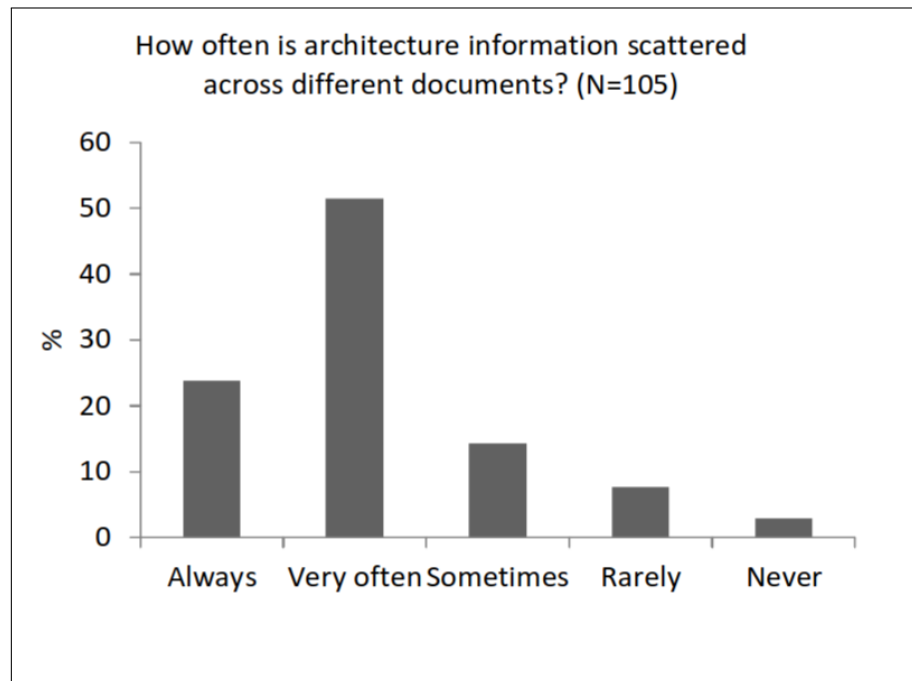


**FIGURE B.8** – Utilisation des documents par phase selon des professionnels (source : Figure 3 [6])

Explication l'assertion ED18 : Il y a un éparpillement des informations à travers les documents d'architecture.

Un sondage réalisé en 2013 auprès de 147 professionnels informatiques créant la docu-

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.9** – Fréquence de la répartition documentaire selon des professionnels (modifié la *Figure 7* [347])

mentation d’architecture confirme le phénomène de dispersion des informations, qui entraîne des duplicata ainsi que des problèmes de mise à jour [347] (figure B.09).  
(voir énoncé p.197)

### Après la réalisation

**Explication de l’assertion ED20 : La fin de vie des entités informatiques est faiblement expliquée à l’intérieur des ouvrages de référence.**

Plusieurs bases de connaissances importantes abordent très peu la fin de vie. Voici en exemple, deux versions du *Project Management Body of Knowledge*(PMBOK) et de l’*Information Technology Infrastructure Library* (ITIL) (le terme employé dans ces ouvrages est celui de *retirement*<sup>180</sup>) :

---

180. L’OQLF traduit le terme **abandon** par **retirement**. [236]

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- Apparaît deux fois dans la version 6 du PMBOK (2017) : [Traduction libre] « Le cycle de vie d'un projet est indépendant du cycle de vie d'un produit, même si celui-ci peut être induit par un projet. Le cycle de vie d'un produit correspond à la succession des phases qui représentent son évolution, de sa conception à sa mise hors service, en passant par sa commercialisation, sa croissance, sa maturité et son retrait du marché. » <sup>181</sup> [304] ;
- Apparaît six fois dans la version 7 du PMBOK (2021) : une figure explicite que le *retirement* est le dernier projet [305] ;
- Apparaît cinq fois dans l'*Information Technology Infrastructure Library* (ITIL) : [Traduction libre] « À moins que le retrait d'un produit ou d'un service ne soit soigneusement planifié, il pourrait entraîner des effets négatifs inattendus sur les clients ou l'organisation, effets qui auraient pu être évités autrement. » <sup>182</sup> [19].

(voir énoncé p.197)

**Explication de l'assertion ED21 : Dans plusieurs pays industrialisés, il est estimé qu'entre 39 % et 50 % des systèmes d'information utilisés sont patrimoniaux.**

En 2024, une revue de littérature s'intéresse aux entités informatiques dites patrimoniales. La revue de littérature porte sur 60 articles publiés sur une période s'étalant de 1973 à 2020. Il est décrit que :

[Traduction libre] De nombreuses études montrent que l'inventaire des systèmes d'information des organisations dans certains pays occidentaux se situe entre environ 39 % [2] et environ 50 % [3], [4]. Par conséquent, une part importante des systèmes d'information dans les pays industrialisés occidentaux doit être considérée comme obsolète. Cependant, ces

---

181. « *Project life cycles are independent of product life cycles, which may be produced by a project. A product life cycle is the series of phases that represent the evolution of a product, from concept through delivery, growth, maturity, and to retirement.* »

182. « *Unless the retirement of a product/service is carefully planned, it could cause unexpected negative effects on customers or the organization that might otherwise have been avoided.* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

systèmes constituent non seulement un problème économique généralisé, mais aussi un problème pour la société dans son ensemble, car les systèmes hérités représentent également un problème important pour les autorités publiques [5], [6], [7], [8].<sup>183</sup> [345]

Cette revue cite trois études ([2], [3], [4]), en voici des détails pour les deux premiers<sup>184</sup> :

- [2] Sondage de 2016 visant les systèmes d'administration des politiques utilisés par les assureurs ( $n = 58$ ) au Canada et aux États-Unis : [Traduction libre] « Système de gestion des polices d'assurance (SGA) [...] LIMRA et Deloitte ont mené un sondage auprès d'entreprises au cours de l'été 2016 afin d'en savoir plus sur leurs SGA actuels et sur leurs projets de modernisation. Cinquante-huit entreprises aux États-Unis et au Canada ont répondu ; une compagnie a répondu pour deux domaines. [...] Environ deux entreprises sur cinq (39 %) sondées possèdent encore un SGA ancien. »<sup>185</sup> [97]
- [3] Étude de 2014 portant sur les applications ( $n = 4130$ ) utilisées par l'État du Texas : [Traduction libre] « Le parc d'applications de l'État comprend plus de 4 130 applications d'affaires prises en charge par les systèmes de l'État. Parmi celles-ci, environ 58 % sont considérées comme obsolètes parce qu'elles contiennent des composants obsolètes. [...] Environ 36 % de l'ensemble du parc applicatif de l'État est jugé critique par les agences, alors que 64 % ne l'est pas. »<sup>186</sup> [380]

---

183.

*A vast number of studies show that the information system inventory of organizations in some Western countries ranges between around 39 % [2] to around 50 % [3], [4]. Therefore, a significant part of the IS in Western industrialized countries should be considered outdated. However, these systems are not only a widespread economic problem, but also a problem for society as a whole, as legacy systems are also a significant problem for public authorities [5], [6], [7], [8].*

184. Le troisième étant en allemand, il a été décidé de ne pas en faire de résumé.

185. « *policy administration system (PAS)* [...] LIMRA and Deloitte surveyed companies in the summer of 2016 to learn more about their current core PAS and plans for modernization. Fifty-eight companies in the United States and Canada responded ; one company responded for two areas of their company. [...] Roughly two in five companies (39 percent) surveyed have one legacy PAS. »

186. « *The state's application portfolio comprises more than 4,130 business applications supported by the state's systems. Of these, approximately 58% are listed as legacy because they contain legacy*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

(voir énoncé p.198)

### Explication de l'axiome ED22 : Les logiciels doivent s'adapter plus fréquemment.

En 2005, Ian F. Sommerville dans l'article *Integrated requirements engineering : a tutorial* explique quatre raisons pour lesquelles les changements aux entités informatiques vont devenir plus fréquent :

[Traduction libre]

[—] Nouvelles approches du développement de systèmes, en particulier la construction par configuration.

[—] Le besoin d'une livraison rapide de logiciels.

[—] Le rythme croissant de l'évolution des exigences.

[—] Le besoin d'un meilleur retour sur investissement des actifs logiciels.

<sup>187</sup> [368]

En 2007, Barry W. Boehm décrit dans un article les épreuves à surmonter :

[Traduction libre] Les défis que nous avons évoqués ici, à savoir la gestion simultanée des changements rapides, de l'incertitude et des situations d'urgence, de la fiabilité, de la diversité et de l'interdépendance, seront souvent redoutables, mais il y aura aussi des occasions d'apporter des contributions significatives qui feront une différence positive. <sup>188</sup> [357]

(voir énoncé p.198)

---

*components. [...] Approximately 36% of the entire state application portfolio is deemed mission critical by agencies, while 64% is not mission critical. »*

187.

[—] *New approaches to systems development in particular, construction by configuration.*

[—] *The need for rapid software delivery.*

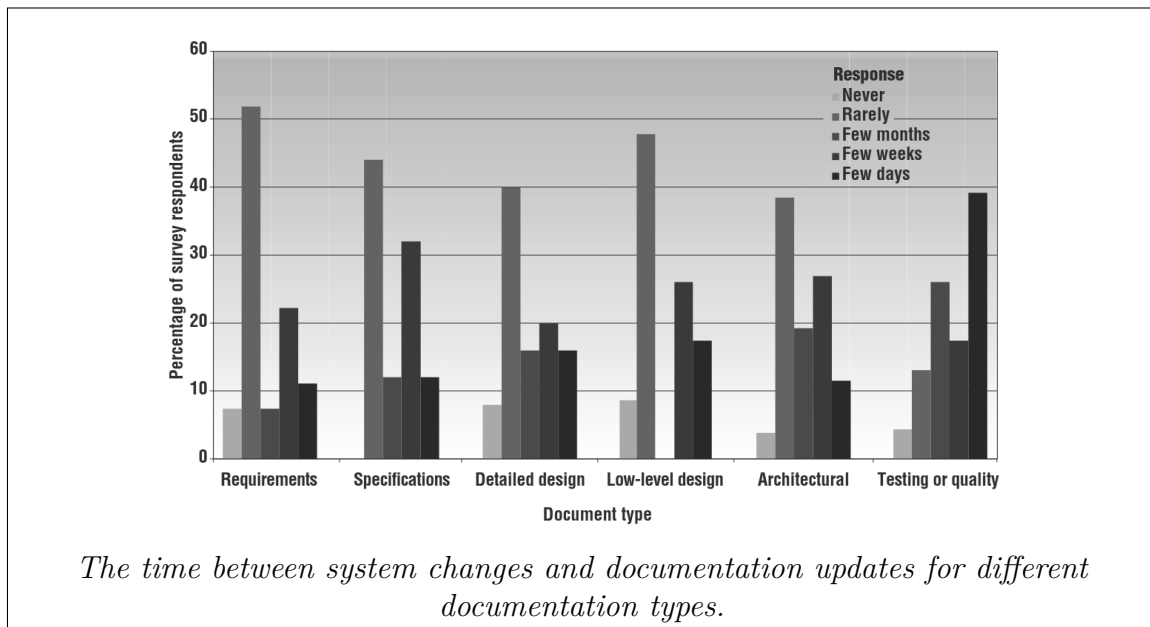
[—] *The increasing rate of requirements change.*

[—] *The need for improved ROI on software assets.*

188.

*The challenges we have discussed here of simultaneously dealing with rapid change, uncertainty and emergency, dependability, diversity, and interdependence will often be for-*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



<sup>a</sup> Dans l'article, le diagramme est en couleur, néanmoins les fréquences demeurent en ordre ce qui permet un affichage en gris.

**FIGURE B.10** – Délai de la mise à jour de la documentation selon des professionnels (source : *Figure 1* [206])

### Explication de l'assertion ED23 : La documentation peut devenir obsolète.

En 2002, un sondage interroge 48 professionnels informatiques sur la documentation [88]. Les professionnels informatiques affirment que la documentation est rarement mise à jour.

En 2003, un article analyse trois études au sujet de la documentation [206]. La troisième étude s'intéresse aux mises à jour de la documentation. La figure B.10 montre la fréquence subjective de mise à jour des documents selon les sondés ( $n = 48$ ). Une hypothèse est émise à la fin de l'article : les professionnels informatiques mettent l'effort sur les documents qu'ils estiment en valoir la peine.

En 2013, un sondage avec 147 experts en conception indique que l'obsolescence est le problème principal des documents d'architecture [347].

---

*midable, but there will also be opportunities to make significant contributions that will make a difference for the better.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

(voir énoncé p.198)

### Explication de l'assertion ED24 : Le temps peut changer le lectorat, les langages utilisés et les standards.

Le lectorat peut changer. Brad Mehlenbacher, dans un article, décrit différentes problématiques avec le lectorat de la documentation informatique [218]. L'expertise en informatique s'éparpille et se démocratise. Dans les années 1980, le lectorat était divisé ainsi : [Traduction libre] « expertise avec les ordinateurs, expertise dans les tâches du domaine, expertise avec les applications »<sup>189</sup>. En 1998, un autre axe est ajouté ; [Traduction libre] « novice, intermédiaire, expert »<sup>190</sup>.

Il arrive que les représentations graphiques changent. En 2006, un article décrit l'historique d'une représentation graphique, le diagramme de flux [223]. Le titre est *Flow diagrams : rise and fall of the first software engineering Notation*. Le diagramme d'activités est considéré comme le successeur au diagramme de flux et au diagramme de flux de données.

- Avant 1950, le diagramme de flux est déjà utilisé pour décrire le fonctionnement des machines ou des algorithmes. Le diagramme de flux est décrit comme cela ; [Traduction libre] « Le diagramme de flux en lui-même "exprime l'algèbre" du processus »<sup>191</sup>.
- Au milieu des années 1950, il y a eu un essor de ce type de diagramme, entre autres, à l'intérieur des guides d'utilisateur. Un déclin s'ensuivit.
- Dans les années 1970 et 1980, le développement structuré popularise le diagramme de flux de données.
- Dans les 1990, c'est au tour de l'*Unified Modeling Language* (UML) de populariser des types de diagrammes. Il y a, par exemple, le diagramme de séquence, le diagramme de classes et le diagramme d'activités.

Les standards peuvent changer. L'ANSI/ANS-10.3-1995 *Documentation of Computer Software* produit par l'American National Standards Institute (ANSI) et l'American

---

189. « *expertise with computer, expertise with task domain, expertise with application* »

190. « *novice, transfer, expert* »

191. « *The flow-diagram itself "expresses the 'algebra' of the process"* »



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

Nuclear Society (ANS) est un exemple de standard qui a été abandonné sans successeur [15].

(voir énoncé p.198)

### Explication de la définition ED25 : Dette technique

La définition formulée est : ensemble des artefacts résultants de choix opportunistes dont le coût de mise à niveau augmente avec le temps.

La définition de dette technique de Steve McConnell se retrouve dans plusieurs articles [18, 18, 166]. Elle précise la dette technique comme étant intentionnelle :

[Traduction libre] La définition de la dette technique a été réexaminée à plusieurs reprises par la suite, généralement pour généraliser la description antérieure de Cunningham à toutes les situations applicables, tout en catégorisant ses caractéristiques. La définition de Steve McConnell, qui distingue l'accumulation intentionnelle et non intentionnelle de dette technique [7], a été largement adoptée par le monde universitaire (par exemple, dans [8,9] et reconnue dans [5]).<sup>192</sup> [121]

La littérature fournit ces informations qui ont servi à construire cette définition :

— Définition de Steve McConnell :

[Traduction libre] une approche de conception ou de construction qui est opportune à court terme, mais qui crée un contexte technique dans lequel le même travail coûtera plus cher à réaliser plus tard qu'il ne coûterait à réaliser maintenant.<sup>193</sup> [18]

---

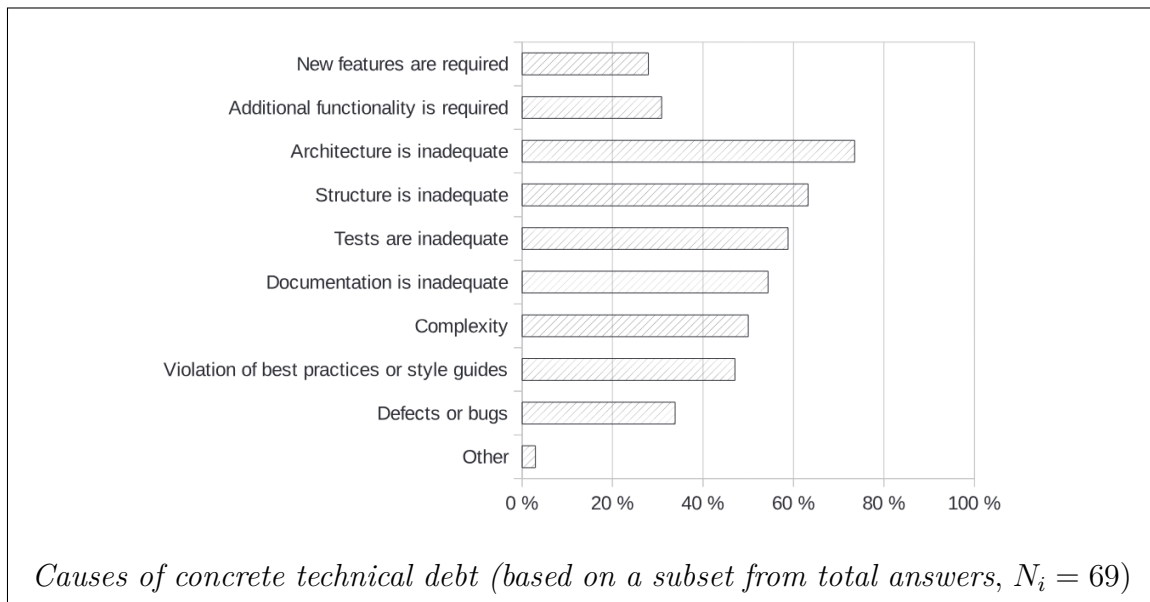
192.

*The definition of technical debt was later revisited on a number of occasions, usually to generalize what Cunningham had previously described for all applicable situations whilst categorizing its characteristics. Steve McConnell's definition, which separates out intentional and unintentional accumulation of technical debt [7], has been widely adopted by academia (e.g., in [8,9] and recognized in [5]).*

193.

*[...] a design or construction approach that is expedient in the short term but that creates a technical context in which the same work will cost more to do later than it would cost to do now.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.11** – Causes de la dette technique selon des professionnels (source : *Figure 4* [121])

— Définition de passif est : « Ensemble des dettes et des charges, évaluables en argent, qui grèvent un patrimoine. » [202].

Le terme **dette technique** s’est imposé dans l’usage, mais le terme **passif technique** est vraisemblablement plus approprié.

(voir énoncé p.198)

**Explication de l’assertion ED26 : La documentation peut contribuer à la dette technique menant à l’échec de l’évolution ou de la maintenance d’un logiciel.**

En 2018, dans un sondage mené auprès de 184 professionnels informatiques (figure B.11) révèle notamment que la documentation inadéquate est corrélée à l’accroissement de la dette technique [121].

Philippe Kruchten (ancien directeur du RUP à Rational Rose) *et al.* incluent la documentation comme l’une des multiples activités de développement contribuant à la dette technique :

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

[Traduction libre] L'architecture joue un rôle important dans le développement des grands systèmes, de même que d'autres activités de développement, telles que la documentation et les tests (souvent négligés). Ces activités peuvent considérablement accroître la dette technique et font donc partie intégrante de ce phénomène.<sup>194</sup> [166]

Publiée en 2021, une revue de littérature analyse les critères de succès ou d'échec en génie logiciel. La revue de littérature porte sur l'analyse de 89 articles publiés sur une période s'échelonnant de 1970 à 2019. Un des résultats observés est que :

[Traduction libre] Qualité de la documentation : Du point de vue de la maintenance et de l'évolution des logiciels, la documentation est un facteur discriminant dans les projets logiciels réussis ou en échec [32]–[34].<sup>195</sup> [374]

(voir énoncé p.198)

### **Explication l'assertion ED27 : Plusieurs professionnels informatiques considèrent la documentation comme une source non fiable d'informations.**

En 2005, 76 professionnels informatiques s'occupant de la maintenance sont sondés [369]. Selon eux, le code source a beaucoup plus d'importance que de nombreux autres artefacts qui sont généralement compris à l'intérieur de la documentation, les résultats sont dans la figure B.12. En 2024, un sondage est mené sur 37 professionnels informatiques à propos de la disponibilité de la connaissance [167]. Les sources de connaissances les plus précieuses, classées en ordre d'importance : connaissances propres, connaissance des collaborateurs, code source, documentation, système de contrôle de version, outils d'analyse.

(voir énoncé p.198)

---

194.

*Architecture plays a significant role in the development of large systems, together with other development activities, such as documentation and testing (which are often lacking). These activities can add significantly to the debt and thus are part of the technical debt landscape.*

195.

*Documentation quality : From the perspective of software maintenance and evolution, documentation is a discriminant in successful or failing software projects [32]–[34].*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

| rank | Structured analysis artifact        | Important |        |     |      | % very important | Does not known |
|------|-------------------------------------|-----------|--------|-----|------|------------------|----------------|
|      |                                     | no        | little | yes | very |                  |                |
| 1    | Source code                         | 0         | 0      | 5   | 63   | 92.6%            | 2              |
| 2    | Comments                            | 0         | 4      | 11  | 54   | 78.2%            | 1              |
| 3    | Logical data model (MER)            | 0         | 3      | 14  | 50   | 74.6%            | 3              |
| 4    | Physical data model                 | 0         | 1      | 24  | 42   | 62.6%            | 3              |
| 5    | Requirement description             | 3         | 7      | 28  | 41   | 59.4%            | 1              |
| 6    | Acceptance test plan                | 6         | 8      | 16  | 34   | 53.1%            | 6              |
| 7    | Requirement list                    | 6         | 9      | 17  | 36   | 52.9%            | 2              |
| 8    | Data dictionary                     | 1         | 10     | 24  | 31   | 46.9%            | 4              |
| 9    | Conceptual data model               | 4         | 7      | 25  | 29   | 44.6%            | 5              |
| 10   | System test plan                    | 4         | 10     | 23  | 29   | 43.9%            | 4              |
| 11   | User manual                         | 6         | 8      | 23  | 29   | 43.9%            | 4              |
| 12   | Implantation plan                   | 5         | 7      | 27  | 28   | 41.7%            | 3              |
| 13   | Unitary test plan                   | 5         | 12     | 22  | 25   | 39.0%            | 6              |
| 14   | Data migration plan                 | 6         | 10     | 25  | 23   | 35.9%            | 6              |
| 15   | Data flow diagram                   | 5         | 11     | 29  | 23   | 33.8%            | 2              |
| 16   | Functional prototype                | 7         | 12     | 24  | 21   | 32.8%            | 6              |
| 17   | Component specification             | 4         | 10     | 32  | 19   | 29.2%            | 5              |
| 18   | Architectural model                 | 5         | 15     | 26  | 18   | 28.1%            | 6              |
| 19   | Context diagram                     | 4         | 25     | 21  | 17   | 25.3%            | 3              |
| 20   | Hierarchical function diagram       | 5         | 15     | 30  | 15   | 23.0%            | 5              |
| 21   | Glossary                            | 4         | 21     | 26  | 15   | 22.7%            | 4              |
| 22   | Functions derived from requirements | 4         | 17     | 21  | 12   | 22.2%            | 16             |
| 23   | General transaction diagram         | 5         | 14     | 25  | 11   | 20.0%            | 15             |
| 24   | Non functional prototype            | 8         | 13     | 29  | 12   | 19.3%            | 8              |

| rank | object-orientation artifacts | Important |        |     |      | % very important | Does not known |
|------|------------------------------|-----------|--------|-----|------|------------------|----------------|
|      |                              | no        | little | yes | very |                  |                |
| 1    | Source code                  | 0         | 0      | 3   | 50   | 94.3%            | 1              |
| 2    | Comments                     | 0         | 4      | 9   | 41   | 75.9%            | 0              |
| 3    | Logical data model (MER)     | 0         | 3      | 12  | 38   | 71.6%            | 1              |
| 4    | Class diagram                | 0         | 2      | 18  | 33   | 62.6%            | 1              |
| 5    | Physical data model          | 0         | 2      | 19  | 32   | 60.3%            | 1              |
| 6    | Use case diagram             | 0         | 5      | 17  | 31   | 58.4%            | 1              |
| 7    | Use case specification       | 1         | 3      | 21  | 26   | 50.9%            | 3              |
| 8    | Acceptance test plan         | 2         | 8      | 15  | 25   | 50.0%            | 4              |
| 9    | Data dictionary              | 1         | 8      | 19  | 24   | 46.1%            | 2              |
| 10   | Implantation plan            | 2         | 9      | 18  | 23   | 44.2%            | 2              |
| 11   | User manual                  | 3         | 8      | 19  | 23   | 43.3%            | 1              |
| 12   | Conceptual data model        | 2         | 5      | 22  | 22   | 43.1%            | 3              |
| 13   | Unitary test plan            | 1         | 10     | 19  | 19   | 38.7%            | 5              |
| 14   | Data migration plan          | 3         | 9      | 18  | 19   | 38.7%            | 5              |
| 15   | System test plan             | 1         | 9      | 22  | 20   | 38.4%            | 2              |
| 16   | Sequence diagram             | 0         | 5      | 30  | 18   | 33.9%            | 1              |
| 17   | Activity diagram             | 3         | 7      | 27  | 15   | 28.8%            | 2              |
| 18   | Vision document              | 2         | 10     | 23  | 13   | 27.0%            | 6              |
| 19   | Functional prototype         | 6         | 13     | 19  | 14   | 26.9%            | 2              |
| 20   | Non functional prototype     | 6         | 10     | 22  | 11   | 22.4%            | 5              |
| 21   | Glossary                     | 3         | 13     | 25  | 11   | 21.1%            | 2              |
| 22   | Component diagram            | 5         | 9      | 28  | 8    | 16.0%            | 4              |
| 23   | State diagram                | 6         | 14     | 23  | 7    | 14.0%            | 4              |
| 24   | Distribution diagram         | 5         | 15     | 26  | 3    | 6.1%             | 5              |
| 25   | Collaboration diagram        | 6         | 14     | 28  | 1    | 2.0%             | 5              |

**FIGURE B.12** – Importance des artefacts selon des professionnels (source : *Table 1* [369])

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**Explication de l’assertion ED28 : Plusieurs professionnels informatiques ne se préoccupent pas du lectorat, bien qu’ils fassent partie eux-mêmes du lectorat.**

En 2009, une expérimentation est menée sur l’utilisation d’un instrument dont l’objectif est d’améliorer la documentation d’architecture [149]. Dans les limitations énoncées, il y a le découragement des auteurs. Le bénéfice à obtenir une bonne documentation profite surtout aux lectorats, et à peu près pas aux auteurs.

En 1991, un article d’opinion y va d’un titre provocateur *Nobody Reads Documentation* [316]. L’auteur Marc Rettig affirme que les professionnels informatiques se plaignent de la documentation et qu’ils ne peuvent en vouloir qu’à eux-mêmes. Il écrit [Traduction libre] « Médecin, guéris-toi toi-même »<sup>196</sup>.

En 2012, Qian Hu décrit les améliorations qu’il a apportées à son cours de rédaction [312]. Son objectif principal était de faire comprendre aux étudiants en informatique qu’ils étaient incapables de maintenir un logiciel sans la documentation. Ses étudiants ne disposaient que du code source et des tâches de maintenance et d’évolutivité que Qian Hu leur avait attribuées. Par la suite, les étudiants devaient s’entendre sur la documentation manquante et l’écrire. En expérimentant le rôle du lectorat, ils furent mieux à même de comprendre l’importance de la documentation.

(voir énoncé p.198)

### Des instruments

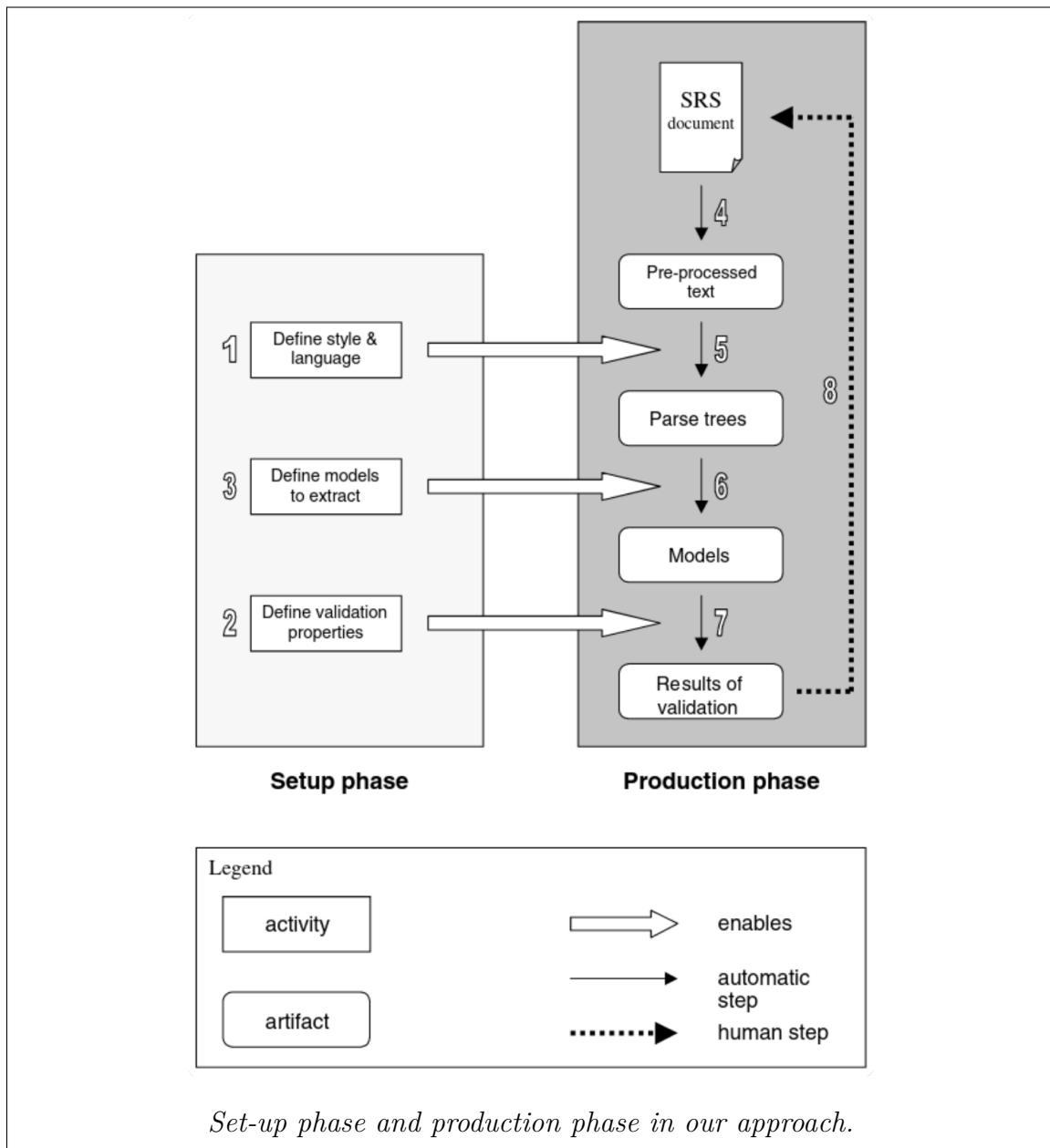
**Explication l’assertion AD04 : Il existe des propositions de logiciels qui combinent des techniques informatiques visant à améliorer la qualité de la documentation.**

Dans un article publié en 2002, Vincenzo Gervasi<sup>1</sup> et Bashar Nuseibeh proposent un prototype visant à valider les documents d’exigences. Le diagramme dans la figure B.13 décrit le procédé [100].

---

196. « *Physician, heal thyself* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.13** – Procédé de formalisation d'un document (source : *Figure 1* [100])

Ensuite, les auteurs décrivent chacune des huit étapes présentées dans la figure B.13 :

[Traduction libre]

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

[—] Définition d'un style, d'une structure et d'un langage pour le document d'exigences. Cette étape peut être menée de manière normative, c'est-à-dire par la production d'un guide de style prescriptif pour le document d'exigences (et dans ce cas, un éditeur syntaxique peut être utilisé pour faciliter la rédaction des exigences, comme dans [16]), ou de manière descriptive, c'est-à-dire par l'adaptation des capacités d'un outil d'analyse syntaxique à un document existant rédigé dans un style défini (comme dans le cas de l'expérience que nous rapportons dans cet article).

[—] Sélection des propriétés pertinentes à vérifier. Les propriétés « intéressantes » d'un document ou d'un système décrit dans un document dépendent du contexte spécifique de l'analyse. Comme c'est souvent le cas avec les méthodes formelles légères, une validation partielle est généralement acceptable à ce stade.

[—] Définition d'un ou plusieurs modèles permettant de vérifier les propriétés sélectionnées à l'étape précédente. Les propriétés sont toujours relatives à des modèles, c'est-à-dire des abstractions du document ou du système qu'il décrit, qui regroupent dans une structure analysable les informations nécessaires à la vérification de la propriété. Par exemple, une propriété de connexion entre les composants du système peut être vérifiée par rapport à un modèle décrivant toutes les communications entre ces composants.

[—] Prétraitement du document d'exigences : gestion du format, de la structure et des détails typographiques, et traduction du document d'exigences dans un format canonique adapté au traitement ultérieur.

[—] Analyse syntaxique du texte en langage naturel des exigences, aboutissant à une représentation analysable du contenu sémantique du texte. Là encore, l'analyse syntaxique peut être (et est généralement) partielle, afin de réduire le coût de la validation, à condition que cela n'entrave pas la collecte des informations nécessaires à cette validation.

[—] Construction des modèles définis à l'étape 3 ci-dessus, à partir des informations collectées lors de l'analyse syntaxique. Il est possible de

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

construire des modèles du document d'exigences (par exemple, la répartition des sujets entre les sections du document) et du système décrit par les exigences (par exemple, un modèle des chemins de communication dans un système distribué).

[—] Vérification que les modèles satisfont aux propriétés choisies. Comme à l'étape précédente, il est possible de vérifier les propriétés du document (dans notre exemple précédent, la cohérence des sujets au sein d'une même section) et du système (par exemple, l'existence de composants disjoints dans le modèle des chemins de communication).

[—] Évaluation des résultats et révision du cahier des charges en conséquence. Il est particulièrement important que les contrôles de validation fournissent autant de détails que possible sur le point et la raison d'une défaillance (c'est-à-dire sur les circonstances dans lesquelles une propriété de validation a été violée). À l'instar des contre-exemples fournis par d'autres méthodes formelles, ces informations aident l'ingénieur en exigences à identifier et à corriger les erreurs qui entraînent des violations.

<sup>197</sup> [100]

(voir énoncé p.199)

---

197.

[—] *Defining a style a structure and a language for the requirements document. This step can be undertaken either normatively, i.e. as the production of a prescriptive style manual for the requirements document (and in this case a syntax-guided editor can be used to support requirements writing, as in [16]), or descriptively, i.e. as an adaptation of the capabilities of a parsing tool to an already existing document written in a defined style (as in the case of the experience we report in this paper).*

[—] *Selecting desirable properties to check. Which properties of a certain document or system described in a document are 'interesting' depends on the particular context of the analysis. As is common with lightweight formal methods, partial validation is usually acceptable at this stage.*

[—] *Defining one or more models against which the properties selected in the previous step can be checked. Properties are always relative to models, i.e. abstractions of the document or of the system described in it, which collect in an analysable structure the information needed to check the property. For example, a connection property among system components can be checked against a model describing all the communications among system components.*

[—] *Pre-processing the requirements document, to handle format, structure and typographical details, and to translate the requirements document to a canonical form amenable to later processing.*

[—] *Parsing the NL text of the requirements, leading to an analysable representation of the semantic content of the text. Again, parsing can be (and usually is) partial, to help reduce the*



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABLEAU B.9** – Sommaire des problèmes et solutions en anglais (inspiré de [66])

| Problem                                                                                                       | Solution                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| • How to structure the information in documentation?                                                          | • Organize contents according to their domain, so that the information form directly relates to domain concepts.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| • How to support the maintenance of consistency between independent artifacts with related content?           | • Use an integrated environment, where several types of artifact and their relations may be maintained uniformly.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| • How to preserve documentation consistency when fragments of related content are scattered across documents? | <ul style="list-style-type: none"> <li>• Keep related information fragments easily accessible from each other, using a single source, links, views, or transclusion, for example, so that it may be easier to assess if they are in-sync.</li> <li>• Use explicit relations between different resources, so that related contents are kept separated and readers may easily travel between them. (Variant: Links)</li> <li>• Capture related information fragments in the same artifact, so that they may be easily maintained close to each other. (Variant: Single Source)</li> <li>• Import the contents of an information fragment into a document by using a reference to it. (Variant: Transclusion)</li> <li>• Create a virtual document, composed by different individual fragments of information. (Variant: Views)</li> </ul> |
| • When to update a related piece of information in documentation?                                             | <ul style="list-style-type: none"> <li>• When a change is introduced, update the related information parts.</li> <li>• Whenever a change is introduced, update every related piece of information. (Variant: Synchronous Co-Evolution)</li> <li>• Whenever a change is introduced, provide mechanisms to track the pending related changes, and update the most relevant pieces of information only when needed. (Variant: Time-Shifted Co-Evolution)</li> </ul>                                                                                                                                                                                                                                                                                                                                                                        |

*cost of validation, as long as this does not interfere with the collection of the information needed to perform the validation.*

*[—] Building the models defined in step 3 above, using the information collected during the parsing process. It is possible to build models of the requirements document (for example, distribution of topics among sections of the document) and of the system described by the requirements (for example, a model of the communication paths in a distributed system).*

*[—] Checking that the models satisfy chosen properties. As in the previous step, it is possible to check properties of the document (in our previous example, consistency of topics inside a single section) and of the system (for example, the existence of disjoint components in the communication paths model).*

*[—] Evaluating findings and revising the requirements specification accordingly. It is particularly important that the validation checks provide as much detail as possible about the point of and the reason for a failure (i.e. about the circumstances in which a validation property was violated). Like the counterexamples provided by other formal methods, this information helps the requirements engineer to identify and fix errors that cause violations.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**Explication l’assertion AD05 : Un instrument de documentation devrait maintenir les relations dans les documents.**

En 2009, l’article *Patterns for consistent software documentation*ait l’exercice de décrire ce qu’un instrument devrait utiliser pour éviter les contradictions dans les mises à jour de la documentation [66]. Le tableau B.9 regroupe les solutions proposées par problème.

En 2022, une revue de littérature suggère comme caractéristique que la documentation devrait être exécutable [382]. La revue porte sur 63 articles s’étalant de 2001 à 2018 dans le contexte des méthodes agiles. Trois avantages de cette caractéristique sont mis en évidence :

[Traduction libre]

[—] La documentation exécutable est toujours synchronisée ; son évolution est naturellement liée à celle des autres composants du logiciel. Elle ne se contente pas de décrire le logiciel, elle en fait partie intégrante.

[—] La documentation exécutable peut être testée. Si elle ne produit pas les résultats escomptés, c’est qu’il y a un problème. À cet égard, elle est comparable au code source.

[—] Le processus de création de la documentation exécutable n’est pas intrusif : les développeurs n’interrompent pas leur travail pour s’en occuper.

Coder et documenter font partie intégrante de la même tâche. <sup>198</sup> [382]  
(voir énoncé p.199)

**Explication l’assertion AD06 : Entre les années 2002 et 2018, les instruments restent relativement similaires, à l’exception de l’utilisation des sites**

---

198.

[—] *Executable documentation is never out-of-sync, it’s evolution is naturally connected to the evolution of the other parts of the software. Executable documentation does not just describe the software, but it is part of the software.*

[—] *Executable documentation can be tested. If it does not lead to the desired results, then something must be wrong. In that respect, executable documentation is just like source-code.*

[—] *The process of creating executable documentation is not intrusive, i.e. developers do not stop their work to take care of documentation ; coding and documenting are part of the same task.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### **wiki.**

En 2022, une revue de littérature porte sur 63 articles dans le contexte des méthodes agiles [382]. Les instruments utilisés sont relevés à l'intérieur d'articles publiés entre 2004 et 2018. Leurs descriptions et la fréquence d'articles correspondants entre parenthèses sont données ici :

- *Word, Excel, Powerpoint etc. : Used for reports and intended for long term usage* (40) ;
- *Wiki : Wiki-likes, Confluence* (39) ;
- *Source control : Tools for Git, Mercurial, SVN* (5) ;
- *Scripts : Executable lines of code to run tasks* (9) ;
- *Markdown editors : Light weight text editor, often used without editor but edited directly* (2) ;
- *Verbal communication : The proverbial water-cooler conversation* (7).

Un sondage de 2002 est réalisé auprès de 60 professionnels informatiques au sujet de la documentation [88]. Les instruments utilisés par les participants sont relevés, les voici avec la fréquence :

- *MS Word (and other word processors)* (22) ;
- *Javadoc and similar tools (Doxygen, Doc++)* (21) ;
- *Text Editors* (9) ;
- *Rational Rose* (5) ;
- *Together (Control Centre, IDE)* (3).

(voir énoncé p.199)

### **Des méthodes informatiques**

#### **Explication de la définition AD08 : Module**

La définition formulée est : est un composant comprenant une interface.

Voici des descriptions qui proviennent de la norme ISO 24765:2017 et de l'ouvrage *Documenting software architectures : views and beyond* de Paul Clements :

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- description 1 : [Traduction libre] « Note 1 : Les termes « module », « composant » et « unité » sont souvent employés indifféremment ou définis comme des sous-éléments les uns des autres de différentes manières selon le contexte. La relation entre ces termes n'est pas encore normalisée. » »<sup>199</sup> [140] ;
- description 2 : [Traduction libre] « Un module désigne avant tout une unité d'implémentation. Les travaux fondateurs de Parnas sur la conception modulaire (Parnas, 1972) ont utilisé le principe d'encapsulation comme critère d'attribution des responsabilités à un module. Les informations susceptibles d'évoluer au cours du cycle de vie d'un système, telles que le choix des structures de données ou des algorithmes, étaient attribuées à un module, lequel disposait d'une interface permettant d'accéder à ses fonctionnalités. »<sup>200</sup> [60] ;
- description 3 : [Traduction libre] « Un composant désigne une entité d'exécution. Szyperski précise qu'un composant « peut être déployé indépendamment et est susceptible d'être composé par des tiers » (Szyperski, 1998, p. 30). L'accent est clairement mis sur le produit fini et non sur les considérations d'implémentation qui ont contribué à sa réalisation. »<sup>201</sup> [60].

(voir énoncé p.199)

### Explication de l'assertion AD09 : Le terme cohérence est préférable à celui de cohésion pour discuter de modularité.

Avant tout, les définitions de cohésion et de couplage apportent de la confusion en informatique, une analyse de l'évolution de ces concepts datant de 2008 soulève l'ambiguïté existant :

---

199. « Note 1 to entry : The terms 'module', 'component,' and 'unit' are often used interchangeably or defined to be subelements of one another in different ways depending upon the context. The relationship of these terms is not yet standardized. »

200. « A module refers first and foremost to a unit of implementation. Parnas's foundational work in module design (Parnas 1972) used information hiding as the criterion for allocating responsibility to a module. Information that was likely to change over the lifetime of a system, such as the choice of data structures or algorithms, was assigned to a module, which had an interface through which its facilities were accessed. »

201. « A component refers to a runtime entity. Szyperski says that a component « can be deployed independently and is subject to composition by third parties » (Szyperski 1998, p. 30). The emphasis is clearly on the finished product and not on the implementation considerations that went into it. »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

Le grand nombre de recherches effectuées sur le couple C&C [(cohésion et couplage)], et ayant comme objectif général l'éclaircissement de ces deux concepts, est une preuve de l'utilité et du grand emploi du couple C&C en GL. Reste à savoir si cette utilisation est efficace ou non. Nous pensons qu'elle ne l'est pas vraiment en raison de l'ambiguïté qui existe encore autour de ces deux concepts. [53]

Ensuite, les différentes définitions de cohésion laissent transparaître qu'il s'agit d'une mesure appliquée selon une logique déterminée :

- définition 1 de cohésion : « Propriété d'un ensemble dont toutes les parties sont solidaires [...] » [177] ;
- définition 2 de cohésion : « Caractère d'une pensée, d'un exposé, etc., dont toutes les parties sont liées logiquement les unes aux autres [...] » [177] ;
- définition 3 de cohésion : [Traduction libre] « [...] en conception logicielle, une mesure de la force d'association des éléments au sein d'un module [...]. »<sup>202</sup> [127] ;
- définition 4 de cohésion : [Traduction libre] « La cohésion d'un composant signifie que ses fonctions et ses données sont liées sémantiquement (elles se rapportent à un objectif commun), au lieu d'être des collections arbitraires d'éléments. »<sup>203</sup> [171] ;
- description de cohésion : [Traduction libre] « La cohésion mesure le degré de cohésion entre les responsabilités d'un module. »<sup>204</sup> [26].

Pour finir, Bertrand Meyer, dans son livre *Object-Oriented Software Construction*, affirme qu'un des critères souhaités pour la modularité est la cohérence :

[Traduction libre] [...] nous savons, d'après une discussion précédente, à quel point l'encapsulation est importante pour la conception d'architectures cohérentes et flexibles.<sup>205</sup> [221]

---

202. « [...] in software design, a measure of the strength of association of the elements within a module [...]. »

203. « Cohesion of a component means that its functions and data are semantically related (they relate to a common purpose), instead of being arbitrary collections of elements. »

204. « Cohesion measures how strongly the responsibilities of a module are related. »

205.

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

(voir énoncé p.199)

### Explication de la définition AD10 : Couplage

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition de coupler : « Assembler des choses, les réunir, les relier afin de constituer un seul élément. » [185] ;
- description 1 de coupler : [Traduction libre] « [...] en conception logicielle, une mesure de l'interdépendance entre les modules d'un programme informatique [...]. »<sup>206</sup> [127] ;
- description 2 de coupler : [Traduction libre] « Nous pouvons quantifier ce chevauchement en mesurant la probabilité qu'une modification apportée à un module se propage à l'autre. Cette relation est appelée couplage [...]. »<sup>207</sup> [26].

(voir énoncé p.199)

### Explication de la définition AD11 : Adaptabilité

La définition formulée est : la capacité d'une entité à être modifiée afin de répondre à des changements d'environnements ou de besoins.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition d'évolution : « Ensemble de ces modifications, stade atteint dans ce processus, considéré comme un progrès [...]. » [197] ;

---

[...] *we know from earlier discussion how important information hiding is to the design of coherent and flexible architectures.*

206. « [...] *in software design, a measure of the interdependence among modules in a computer program [...].* »

207. « *We can quantify this overlap by measuring the probability that a modification to one module will propagate to the other. This relationship is called coupling [...].* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- la modifiabilité : [Traduction libre] « La modifiabilité est liée au changement, et notre intérêt pour elle est de réduire le coût et le risque des changements. » <sup>208</sup> [26] ;
- la maintenabilité : [Traduction libre] « La capacité du produit logiciel à être modifié. Les modifications peuvent inclure des corrections, des améliorations ou une adaptation du logiciel aux changements d’environnement, d’exigences et de spécifications fonctionnelles. » <sup>209</sup> [130] ;
- définition d’adaptatif : « Qui facilite ou constitue une adaptation. » [79] ;
- définition de s’adapter à : « s’habituer à (un environnement) en modifiant son comportement en fonction de (cet environnement). » [79] ;
- définition d’adaptabilité : « Caractère de ce qui est adaptable. » [79] ;
- définition d’adaptable : « Qui peut s’adapter, que l’on peut adapter. » [79]

L’évolution est souvent associée à l’idée de progrès. Cependant, un changement à un logiciel n’est pas forcément un progrès.

(voir énoncé p.200)

**Explication de l’assertion AD12 : La programmation structurée et l’encapsulation permettent d’améliorer l’adaptabilité d’un logiciel.**

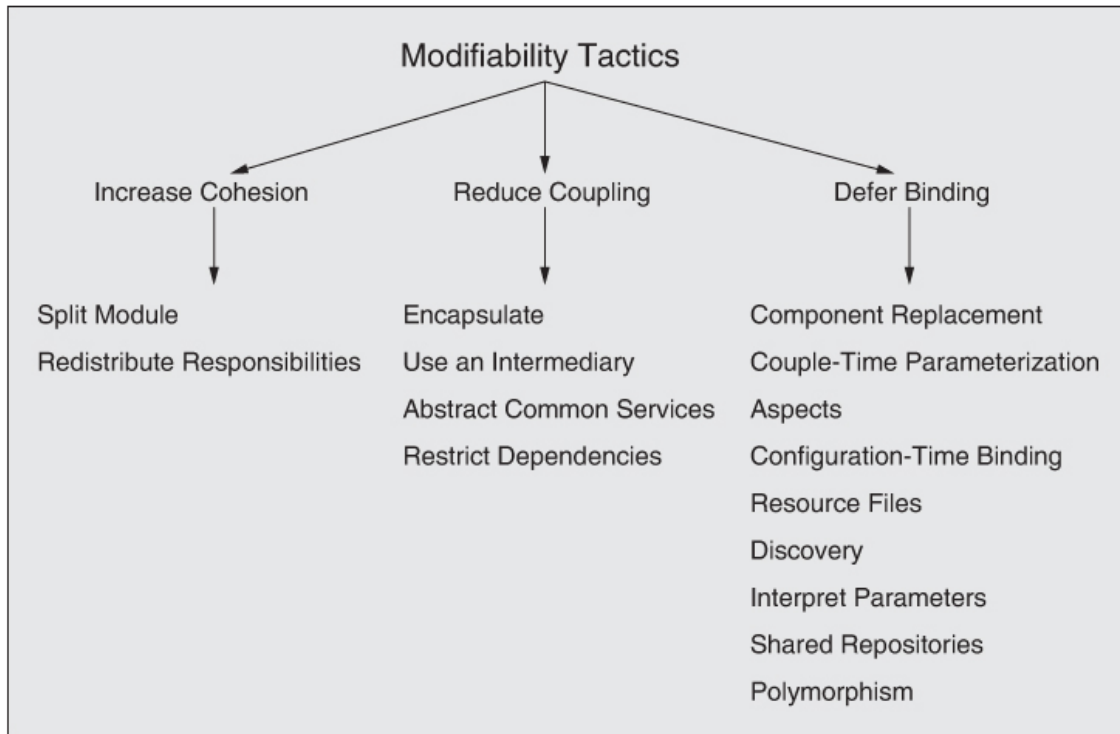
Dans leur ouvrage *Software Architecture in Practice*, Len Bass, Paul Clements et Rick Kazman explicitent, à l’aide d’un diagramme (figure B.14), comment accroître la cohérence, diminuer la couplage ou différer la liaison afin d’améliorer l’adaptabilité [26].

---

208. « *Modifiability is about change, and our interest in it is to lower the cost and risk of making changes.* »

209. « *The capability of the software product to be modified. Modifications may include corrections, improvements or adaptation of the software to changes in environment, and in requirements and functional specifications.* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.14** – Approches pour augmenter l’adaptabilité (source : *Figure 8.3* [26])

Ces techniques proviennent de la programmation structurée et de l’encapsulation :

[Traduction libre] L’article d’Edsger Dijkstra de 1968 sur le système d’exploitation T.H.E. a introduit le concept de couches [Dijkstra 68]. Les premiers travaux de David Parnas ont posé de nombreux fondements conceptuels, notamment l’encapsulation de l’information [Parnas 72], les familles de programmes [Parnas 76], les structures inhérentes aux systèmes logiciels [Parnas 74] et l’utilisation de ces structures pour construire des sous-ensembles et des sur-ensembles de systèmes [Parnas 79]. [...] En 1972, Dijkstra et Hoare, avec Ole-Johan Dahl, ont soutenu que les programmes devaient être décomposés en composants indépendants dotés d’interfaces simples et réduites. Ils ont nommé leur approche « programmation structurée » [...].<sup>210</sup> [26]

(voir énoncé p.200)

---

210.



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### Explication de l'assertion AD13 : Il est possible d'organiser la documentation comme les modules logiciels

En 1981, S. D. Hester et David L. Parnas ont publié un article intitulé *Using Documentation as a Software Design Medium* [115] qui résume cette méthode. Selon l'article, voici les avantages qu'on retire de son utilisation :

[Traduction libre]

- i) Facilité de modification. Les fonctions système susceptibles d'évoluer sont identifiées et l'encapsulation des informations est mise en œuvre afin de minimiser l'impact d'une modification de ces fonctions sur le logiciel.
- ii) Maîtrise des informations relatives aux fonctions du système. Un document de spécifications structuré avec soin est maintenu à jour tout au long du projet.
- iii) Planification des étapes de développement en fonction des objectifs du projet. La documentation des sous-ensembles utiles du système et des dépendances entre les modules logiciels permet d'orienter la planification de développement.
- iv) Clarification des accords entre développeurs. La documentation des interfaces entre les modules logiciels des différents développeurs permet d'éviter les malentendus et d'assurer une meilleure intégration du système. <sup>211</sup> [115]

---

*Edsger Dijkstra's 1968 paper on the T.H.E. operating system introduced the concept of layers [Dijkstra 68]. The early work of David Parnas laid many conceptual foundations, including information hiding [Parnas 72], program families [Parnas 76], the structures inherent in software systems [Parnas 74], and the uses structure to build subsets and supersets of systems [Parnas 79]. [...] In 1972, Dijkstra and Hoare, along with Ole-Johan Dahl, argued that programs should be decomposed into independent components with small and simple interfaces. They called their approach structured programming, [...].*

211.

- i) *Ease of change. System functions that are likely to change are identified and information hiding is applied to minimize the amount of software affected by a change in these functions.*
- ii) *Control of the information about the functions of the system. A carefully structured requirements document is to be maintained throughout the life of the project.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

En 1985, David L. Parnas, Paul Clements et David M. Weiss ont fourni des raisons expliquant l'inutilisation de la méthode et donnent finalement un exemple réaliste de son application [291] en s'appuyant sur l'Operational Flight Program de l'avion A-7E. (voir énoncé p.200)

**Explication l'assertion AD14 : L'adaptabilité de plusieurs composants logiciels peut encore être amélioré.**

Un article présente une étude sur le couplage et la cohérence du code source [47]. Les auteurs analysent cent projets code source libre et leur appliquent des mesures de couplage et de cohérence, à l'aide de neuf formules distinctes. Voici leurs conclusions qui confirment que la cohérence et le couplage pourraient être améliorés et donc l'adaptabilité :

[Traduction libre]

- Les résultats obtenus montrent que la modularisation des systèmes code source libre est généralement loin d'être optimale, tant du point de vue de la cohérence/du couplage structurel que conceptuel. À quelques rares exceptions près, l'effort de refactorisation nécessaire pour atteindre une modularisation optimale à partir de la conception originale, telle qu'évaluée par MoJoFM, est important.
- Globalement, les systèmes en code source libre tendent à privilégier la cohérence par rapport au couplage. Notre analyse montre que cela est principalement dû à la séparation nette entre les couches applicatives recherchée par les développeurs lors de la modularisation de leur système. De plus, les résultats indiquent que la cohérence et le couplage des packages ne sont peut-être pas les principaux facteurs pris en compte par les développeurs lors du partitionnement des classes d'un système. <sup>212</sup> [47]

- 
- iii) *Ordering of the development steps to meet the project objectives. Documentation of the useful subsets of the system and the dependencies between the software modules serve to guide the scheduling of the development effort.*
  - iv) *Making the agreements between developers explicit. Misunderstandings are avoided and a smoother system integration is achieved by documenting the interfaces between the software modules of individual developers.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

(voir énoncé p.200)

**Explication de la citation AD15 : La programmation littéraire cherche à expliquer en premier aux humains et en second à la machine.**

En 1984, Donald E. Knuth publie un article intitulé *Literate Programming*<sup>213</sup>, il y indique que :

[Traduction libre] Au lieu d’imaginer que notre tâche principale consiste à donner des instructions à un ordinateur sur ce qu’il doit faire, concentrons-nous plutôt sur le fait d’expliquer aux êtres humains ce que nous voulons qu’un ordinateur fasse.<sup>214</sup> [163]

(voir énoncé p.200)

**Explication l’assertion AD16 : Les instruments qui exécutent la programmation littéraire ont continué de s’adapter.**

En 2004, un article résume six différents environnements pour la programmation littéraire [301] :

- *the structure and dataflow of WEB (the first LPE)*;
- *the structure and dataflow of LIPED (a language independent LPE)*;

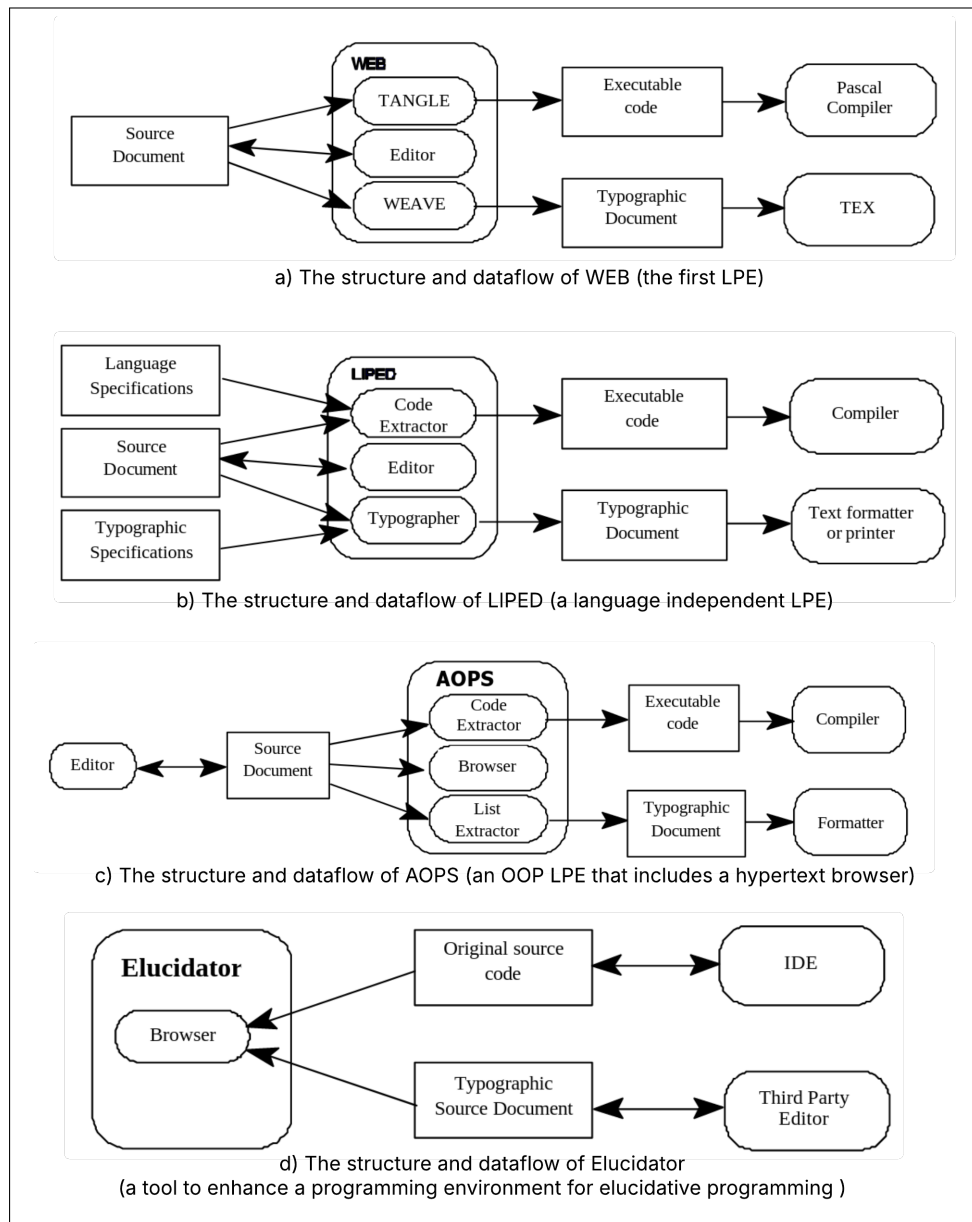
- 
- *The achieved results highlight that the modularization of open-source systems is generally far from an optimal modularization in terms of both structural and conceptual cohesion/coupling. With very few exceptions, the refactoring effort needed to reach an optimal modularization from the original design as assessed by the MoJoFM is high.*
  - *Overall, open-source systems tend to give higher priority to cohesion over coupling. Our analysis shows that this is mainly due to the clear separation between the application layers pursued by developers when modularizing their system. Also, the results show that package cohesion and coupling might not be the main factors considered by developers when partitioning the classes of a system.*

213. Il a développé TeX sur lequel repose LaTeX. LaTeX est le langage utilisé pour écrire ce mémoire.

214.

*Instead of imagining that our main task is to instruct a computer what to do, let us concentrate rather on explaining to human beings what we want a computer do.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.15** – Représentations d’environnements de programmation littéraire (modifiée de [301])

- the structure and dataflow of AOPS (an OOP LPE that includes a hypertext browser);
- the structure and dataflow of LPW (a LPE that integrates a third party editor);

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- *the structure and dataflow of warp (a LPE that integrates third party programming environment)*;
- *the structure and dataflow of Elucidator (a tool to enhance a programming environment for elucidative programming).*

La figure B.15 présente les diagrammes de quatre de ces environnements, dont voici la description textuelle :

- Le diagramme a) emploie deux processus distincts pour extraire le code exécutable ou le document typographique.
- Le diagramme b) montre qu'il est possible de spécifier le langage de programmation et le langage de formatage du document. Cela correspond aux *Language Specifications* et *Typographic Specifications*.
- Le diagramme c) représente un environnement où il est possible de naviguer entre les relations en utilisant un fureteur.
- Le diagramme d) montre que l'instrument Elucidator s'occupe de maintenir les relations entre les concepts provenant de deux endroits.

(voir énoncé p.200)

### **Explication de l'assertion AD17 : Maintenir la cohérence lors de modifications avec la méthode de programmation littéraire est problématique.**

L'auteur d'Elucidator indique à quel point cela vient difficile de maintenir la cohérence lors de modifications :

[Traduction libre] En revanche, il est plus complexe de garantir la mise à jour de la documentation suite à des modifications du programme, et inversement. Ce problème est accentué par le fait qu'un même fragment de programme peut être mentionné à plusieurs endroits dans la documentation.<sup>215</sup> [232]

Le *Theme-based literate programming* tente de maintenir la cohérence en utilisant modèles et techniques [154]. Ces techniques comprennent du XML, du XSLT et du DTD

---

215.

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

et, comme décrits, ils servent à représenter et traiter formellement les informations contenues dans les documents :

[Traduction libre]

[—] Nous utilisons XML [12] comme représentation fondamentale des documents LP [...].

[—] Notre outil de création utilise des outils d'analyse XML et des transformations XSLT [6] pour effectuer une grande partie du traitement [...].

[—] Les définitions de type de document (DTD) permettent de spécifier une grammaire pour chaque segment de données et leurs relations. <sup>216</sup>

[154]

(voir énoncé p.200)

**Explication de l'assertion AD18 : Plusieurs individus croient à tort que l'instrument Jupyter est de la programmation littéraire.**

Voici la description provenant de la documentation officielle de Jupyter qui combine deux composantes, elle provient de la documentation officielle :

[Traduction libre]

- Une application web : un programme d'édition basé sur un navigateur permettant la création interactive de carnets de calcul. Il offre un environnement interactif rapide pour le prototypage et l'explication de code, l'exploration et la visualisation de données, ainsi que le partage d'idées.
- Documents de carnets de notes pour le calcul : un document partageable qui combine du code informatique, des descriptions en langage clair, des

---

*On the negative side, it is less trivial to ensure a proper updating of the documentation upon program modifications, and vice versa. The problem is amplified by the fact that a single program fragment may be addressed at multiple places in the documentation.*

216.

[—] We use XML [12] as the fundamental representation of LP documents [...].

[—] XML parsing tools are used in our authoring tool and XSLT [6] transformations are used to perform much of the processing [...].

[—] Document Type Definitions (DTD) allow a grammar for individual chunks and their relationships to be specified.

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

données, des visualisations riches (modèles 3D, diagrammes, formules mathématiques, graphiques et figures) et des commandes interactives. <sup>217</sup> [375]

Un article décrit l'analyse de carnets Jupyter (1 159 568) provenant de répertoires GitHub (264 020). L'article utilise 24 fois le terme *literate programming* et décrit la popularité de Jupyter et les problèmes de reproductibilité :

[Traduction libre]

[—] Jupyter Notebook est le système le plus utilisé pour la programmation littéraire interactive (Shen 2014).

[—] [...] Le système a été lancé en 2013 et compte aujourd'hui plus de 9 millions de cahiers sur GitHub (Parente 2020).

[—] [...] La plupart des carnets de notes ne testent pas leur code et un grand nombre d'entre eux présentent des caractéristiques qui entravent le raisonnement et la reproductibilité, tels que des cellules mal ordonnées, des cellules de code non exécutées et la possibilité d'états cachés.

[—] [...] Nous avons obtenu un taux de reproductibilité compris entre 4,90 % et 15,04 %. <sup>218</sup> [302]

Un article cite des problèmes dans l'utilisation de code avec Jupyter, la modularité y semble particulièrement problématique :

---

217.

- *A web application : A browser-based editing program for interactive authoring of computational notebooks which provides a fast interactive environment for prototyping and explaining code, exploring and visualizing data, and sharing ideas with others*
- *Computational Notebook documents : A shareable document that combines computer code, plain language descriptions, data, rich visualizations like 3D models, charts, mathematics, graphs and figures, and interactive controls.*

218.

- [—] *Jupyter Notebook is the most widely-used system for interactive literate programming (Shen 2014).*
- [—] [...] *The system was released in 2013, and today there are over 9 million notebooks in GitHub (Parente 2020).*
- [—] [...] *most notebooks do not test their code and that a large number of notebooks has characteristics that hinder the reasoning and the reproducibility, such as out-of-order cells, non-executed code cells, and the possibility of hidden states.*
- [—] [...] *We achieved a reproducibility rate that ranged from 4.90% to 15.04%.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

[Traduction libre] Selon Joel Grus, les carnets Jupyter encouragent également les mauvaises pratiques de programmation, car ils rendent difficiles l’organisation logique du code, son découpage en modules réutilisables et le développement de tests pour garantir son bon fonctionnement. Grus concède que ces problèmes ne sont pas insurmontables, mais que l’utilisation des carnets exige de la rigueur lors de l’exécution du code.<sup>219</sup> [298]

Un autre article dont les auteurs sont Brian E. Grange et Fernando Pérez<sup>220</sup> explique qu’il s’agit de **computational narrative** et non de *literate programming*. Il s’en distingue par le fait que l’objectif est de faire comprendre au lectorat un modèle et non de permettre à une machine de l’exécuter :

[Traduction libre] Ces exemples d’utilisation humaine des récits informatiques illustrent en quoi les utilisations des carnets Jupyter diffèrent radicalement des outils de génie logiciel traditionnels, dont l’objectif est qu’un groupe de personnes développe un logiciel destiné ensuite à être déployé et utilisé par un tout autre groupe d’utilisateurs. Alors que le paradigme de programmation littéraire de Knuth intègre une documentation orientée utilisateur au processus de génie logiciel, le récit informatique se distingue par son approche centrée sur l’informatique interactive. Le résultat n’est pas ici un produit logiciel, mais des idées et une compréhension « déployées » auprès d’autres personnes.<sup>221</sup> [109]

(voir énoncé p.200)

---

219.

*Joel Grus [...] Jupyter notebooks also encourage poor coding practice, he says, by making it difficult to organize code logically, break it into reusable modules and develop tests to ensure the code is working properly. [...] Those aren’t insurmountable issues, Grus concedes, but notebooks do require discipline when it comes to executing code [...].*

220. Fernando Pérez est cofondateur du projet Jupyter.

221.

*These human uses of computational narratives illustrate how and why the ways that Jupyter notebooks are used are so dramatically different from traditional software engineering tools where the goal is for one group of people to write software that is subsequently deployed to and used by an entirely different group of users. While Knuth’s literate programming paradigm weaves human-oriented documentation into the software engineering process, a computational narrative is distinct in its incorporation of interactive computing as the central element. The outcome here is not a software product but ideas and understanding that are “deployed” to other humans.*



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### Des techniques informatiques (donc, formelles)

#### Explication de la définition AD19 : Langage

La définition formulée est : système pour communiquer comprenant des symboles et des règles.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de langage : « Faculté propre à l'être humain d'exprimer sa pensée ou de communiquer au moyen d'une langue et de comprendre les messages d'autrui. » [260] ;
- note sur la définition 1 de langage : « Le langage comporte différentes composantes (phonologique, lexicale, morphosyntaxique, pragmatique, etc.) et se développe dans les volets réceptif (compréhension) et expressif (production). » [260] ;
- définition 2 de langage : « Tout système de signes permettant la communication. » [328] ;
- définition de langue : « Système d'expression et de communication par des moyens phonétiques et éventuellement graphiques, commun à un groupe social. » [329].

(voir énoncé p.200)

#### Explication de la définition AD20 : Langage formel

La définition formulée est : est composé d'ensembles explicites de symboles et d'un ensemble explicite de règles appelé grammaire formelle.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de langage formel : « Langage qui utilise un ensemble de termes et de règles syntaxiques pour permettre de communiquer sans aucune ambiguïté. » [261] ;

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- définition 2 de langage formel : [Traduction libre] « Un langage formel est un ensemble bien défini de mots appartenant à un alphabet donné. Il est généralement défini par une grammaire, c'est-à-dire un ensemble de règles déterminant l'appartenance des mots au langage. »<sup>222</sup> [340] ;
- définition 3 de langage formel : [Traduction libre] « 1) un ensemble de symboles (l'alphabet de sa langue) et (2) un ensemble de règles de formation déterminant quelles séquences de symboles de son alphabet sont des formules bien formées de sa langue. »<sup>223</sup> [125] ;
- description de langage formel : [Traduction libre] « Une grammaire formelle génère un langage formel, qui est un ensemble de séquences de symboles de longueur finie créées en appliquant les règles de production de la grammaire. »<sup>224</sup> [286]
- définition 4 de langage formel : [Traduction libre] « [...] un langage dont les règles sont explicitement établies avant son utilisation [...]. »<sup>225</sup> [140].

(voir énoncé p.200)

### Explication de la définition AD21 : Langage naturel

La définition formulée est : est un langage non formel.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de langage naturel : [Traduction libre] « [...] langue dont les règles sont basées sur l'usage courant sans être spécifiquement prescrites [...]. »<sup>226</sup> [140] ;

---

222. « A formal language is a well-defined set of words from a given alphabet. Usually it is defined by a grammar, which is a set of rules determining the membership of the language. »

223. « 1) a set of symbols (the alphabet of his language) and (2) a set of formation rules determining which sequences of symbols from his alphabet are wffs of his language. »

224. « A formal grammar generates a formal language, which is set of finite length sequences of symbols created by applying the production rules of the grammar. »

225. « [...] language whose rules are explicitly established prior to its use [...]. »

226. « [...] language whose rules are based on current usage without being specifically prescribed [...]. »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- définition 2 de langage naturel : « Langage humain par opposition aux langages de programmation. » [262].

Dans le mémoire, il est souvent préféré le terme **langage non formel** à **langage naturel**, mais, puisque l'usage de langage naturel est répandu, il devait être défini.

(voir énoncé p.201)

### Explication de la définition AD22 : Architecture informatique

La définition formulée est : organisation d'entités informatiques résultant de la phase de conception.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition d'architecture : [Traduction libre] « [...] concepts ou propriétés fondamentaux d'une entité dans son environnement (3.13) et principes directeurs de la réalisation et de l'évolution de cette entité et de ses processus de cycle de vie associés [...]. » <sup>227</sup> [144] ;
- définition d'architecture d'un système : [Traduction libre] « L'architecture logicielle d'un système est l'ensemble des structures nécessaires pour raisonner sur ce système. Ces structures comprennent des éléments logiciels, les relations entre eux et leurs propriétés. » <sup>228</sup> [26] ;
- description d'*Architecture versus Design* : [Traduction libre] « L'architecture est une conception, mais toute conception n'est pas de l'architecture. Autrement dit, de nombreuses décisions de conception ne sont pas liées à l'architecture — qui est, après tout, une abstraction — et dépendent du discernement et du bon jugement des concepteurs et même des implémenteurs en aval. » <sup>229</sup> [26] ;
- description de *software system architecture* : [Traduction libre] « [Cite Barry W. Boehm] Un ensemble de composants logiciels et système, de connexions et

---

227. « [...] *fundamental concepts or properties of an entity in its environment (3.13) and governing principles for the realization and evolution of this entity and its related life cycle processes* [...]. »

228. « *The software architecture of a system is the set of structures needed to reason about the system. These structures comprise software elements, relations among them, and properties of both.* »

229. « *Architecture is design, but not all design is architecture. That is, many design decisions are left unbound by the architecture—it is, after all, an abstraction—and depend on the discretion and good judgment of downstream designers and even implementers.* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

de contraintes. Un ensemble d'énoncés de besoins des parties prenantes du système. Une justification démontrant que les composants, les connexions et les contraintes définissent un système qui, s'il était implémenté, satisferait l'ensemble des énoncés de besoins des parties prenantes du système. »<sup>230</sup> [309] ;

- description d'architecture : [Traduction libre] « Il y a peu de différence significative entre « conception » et « architecture ». Par souci de clarté, nous utiliserons ici « conception » pour désigner le processus et « architecture » pour son résultat [le verbe « architecturer » devient alors superflu]. La même convention s'applique à « implémentation » et « code ». »<sup>231</sup> [220] ;

(voir énoncé p.201)

### Explication de la définition AD23 : Langage de description d'architecture (ADL, *Architecture Description Language* en anglais))

La norme ISO 42010 intitulée *Logiciel, systèmes et entreprise — description de l'architecture* définit les langages de description d'architecture (ADL, *Architecture Description Languages* en anglais) :

[Traduction libre] Un langage de description architecturale (ADL) est une syntaxe et une sémantique spécifiées, destinées à décrire l'architecture d'une entité donnée. L'ADL est un langage utilisé par les parties prenantes, notamment celles impliquées dans la conception architecturale, pour exprimer les considérations architecturales au moyen d'éléments de description architecturale relatifs à l'entité et au contexte de conception. Une description architecturale peut utiliser plusieurs ADL, voire une ADL différente pour chaque point de vue, ou encore des ADL distinctes pour

---

230. « [Cite Barry W. Boehm] A collection of software and system components, connections, and constraints. A collection of system stakeholders' need statements. A rationale which demonstrates that the components, connections, and constraints define a system that, if implemented, would satisfy the collection of system stakeholders need statements. »

231. « There is little meaningful difference between « design » and « architecture ». For clarity, this discussion will use « design » for the process and « architecture » for its result (then we do not need « architect » as a verb). The same convention can be applied to « implementation » and « code ». »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

chaque type de modèle spécifié par un même point de vue architectural.<sup>232</sup>  
[144]

La norme comprend un modèle conceptuel qui peut tenir compte des différentes déclinaisons d'ADL, des exemples se retrouvent entre parenthèses :

[Traduction libre]

[—] un seul type de modèle (par exemple, Darwin[37]) ;

[—] plusieurs types de modèles (par exemple, UML, Rapide, AADL) ;

[—] un seul point de vue (par exemple, ACME[33]) ;

[—] plusieurs points de vue (par exemple, ArchiMate, SysML).<sup>233</sup> [144]

La norme ne prescrit pas l'usage de la modélisation formelle :

[Traduction libre] Ce document ne prescrit pas l'étendue ni les attentes concernant l'utilisation des méthodes de modélisation formelles dans une description d'architecture. Si les méthodes informelles peuvent être utilisées efficacement, les méthodes de modélisation formelles sont souvent moins ambiguës.<sup>234</sup> [144]

(voir énoncé p.201)

---

232.

*An ADL is a specified syntax and semantics intended for use in describing the architecture of an entity of interest. An ADL is a language for stakeholders, including those involved in the architecting effort that allows the expression of architecture considerations by means of AD elements pertaining to the entity of interest, and the architecting context. An AD can use more than one ADL, even a different ADL for each viewpoint, or even distinct ADLs for each model kind specified by a single architecture viewpoint.*

233.

*[—] a single model kind (e.g. Darwin[37]) ;  
[—] multiple model kinds (e.g. UML, Rapide, AADL) ;  
[—] single viewpoint (e.g. ACME[33]) ;  
[—] multiple viewpoints (e.g. ArchiMate, SysML).*

234.

*This document does not prescribe the extent or expectation regarding the use of formal modelling methods in an AD. While informal methods can be used effectively, formal modelling methods are often less ambiguous.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**Explication de l’assertion AD25 : Le XML est un langage formel qui permet d’exprimer des informations selon une structure.**

L’origine du XML vise à transmettre des informations comme cela est décrit :

[Traduction libre] Le langage SGML (Standard Generalized Markup Language) [12, 13] favorise l’interchangeabilité et la gestion indépendante des applications des documents électroniques en fournissant un métalangage syntaxique pour la définition des systèmes de balisage textuel. Le langage XML (Extensible Markup Language) [2] est, en substance, une version simplifiée et plus restrictive du SGML. L’objectif du XML est de permettre la diffusion, la réception et le traitement des documents SGML sur le Web. Il s’agit du métalangage syntaxique proposé pour la spécification des grammaires de documents W3C.<sup>235</sup> [161]

Le XML a permis de structurer les informations avec des métalangages, un article explique son utilisation dans le Darwin Information Type Architecture (DITA) :

[Traduction libre] SGML et XML sont reconnus comme des métalangages qui permettent aux communautés de propriétaires de données de décrire leurs actifs informationnels de manière à refléter la façon dont elles développent, stockent et traitent ces informations.<sup>236</sup> [71]

L’utilisation du XML s’est répandue, comme l’explique le guide du DITA :

---

235.

*The Standard Generalized Markup Language (SGML) [12, 13] promotes the interchangeability and application-independent management of electronic documents by providing a syntactic metalanguage for the definition of textual markup systems. The Extensible Markup Language (XML) [2] is, essentially, a simplified and more restrictive version of SGML. The goal of XML is to allow SGML documents to be served, received, and processed on the Web. It is the proposed syntactic metalanguage for the specification of document grammars for W3 documents.*

236.

*SGML and XML are recognized as meta languages that allow communities of data owners to describe their information assets in ways that reflect how they develop, store, and process that information.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

[Traduction libre] Le développement du XML à la fin des années 1990 a transformé la manière dont les connaissances étaient stockées. Le XML a permis la création de normes d'information structurées pour le stockage des connaissances et des données dans tous les secteurs d'activité.<sup>237</sup> [358]  
(voir énoncé p.201)

**Explication de l'assertion AD26 : Les langages de schéma XML (DTD, XSD, Schematron) permettent d'exprimer des vérifications sur le XML.**

La norme *Information technology — Document Schema Definition Languages (DSDL)* soutient six langages de schéma XML : DTD, W3C XML Schema, RELAX NG, RELAX NG - compact syntax, Schematron et NVDL.

Un article explique les grammaires et les exceptions qui se trouvent dans le SGML et le XML :

[Traduction libre] SGML et XML permettent tous deux de définir des définitions de type de document (DTD), qui sont essentiellement des grammaires hors contexte étendues, exprimées dans une notation similaire à la forme Backus-Naur étendue. La partie droite d'une production, appelée modèle de contenu, est à la fois une expression régulière étendue et une expression régulière restreinte.<sup>238</sup> [161]

Ces différents langages répondent à des besoins différents de vérifications. L'auteur de l'ouvrage sur le Schematron indique que ces vérifications peuvent arriver lors des différentes étapes de traitement :

[Traduction libre]

---

237.

*The development of XML in the late 1990s transformed the way in which knowledge was stored. XML permitted structured information standards to be created for the storage of knowledge and data for all types of industries.*

238.

*Both SGML and XML allow users to define document-type definitions (DTDs), which are essentially extended context-free grammars expressed in a notation that is similar to extended Backus-Naur form. The right-hand side of a production, called a content model, is both an extended and a restricted regular expression.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

[—] Pour XML, la vérification des règles fondamentales de la famille de langages est une condition préalable absolue à toute vérification ultérieure. Un document XML qui réussit cette étape est dit bien formé.

[—] Pour vérifier la grammaire, plusieurs langages de validation XML sont disponibles. Les plus courants sont DTD, W3C XML Schema et RELAX NG. Si cette étape est réussie, un document XML est dit valide.

[—] Schematron est un langage de validation qui vérifie les règles métier. Un ensemble de vérifications écrites en Schematron est appelé un schéma Schematron. Il n'existe pas de nom officiel pour un document qui réussit cette étape, mais nous l'appellerons « schéma valide ». <sup>239</sup> [363]

Ces étapes rappellent ceux d'un compilateur où différents analyseurs vont vérifier et traiter des informations spécifiques.

(voir énoncé p.201)

**Explication de l'assertion AD27 : Les langages de schéma XML ne permettent pas d'exprimer totalement la sémantique.**

Rick Jelliffe créateur du Schematron commente sur son blogue les propos de l'auteur d'un autre langage de schéma XML (RELAX NG) Makoto Murata sur le fait que ces langages ne concernent pas la sémantique. Selon lui, cela demeure plus nuancé :

[Traduction libre] Je pense donc que Murata-san a raison sur ce point : les schémas n'impliquent aucune sémantique. Certes, ils n'impliquent jamais toute la sémantique, et souvent, même pas toutes les règles structurales, de type de données ou de liaison. Mais cela ne signifie pas pour autant que les schémas ne reflètent pas pleinement les considérations sémantiques. Le

---

239.

[—] *For XML, checking the ground rules of the language family is an absolute precondition for any subsequent checks. An XML document that passes this stage is called well-formed.*

[—] *To check grammar you can choose from several XML validation languages. The most common ones are DTD, W3C XML Schema, and RELAX NG. If this stage is passed, an XML document is called valid.*

[—] *Schematron is a validation language that checks business rules. A set of checks written in the Schematron language is called a Schematron schema. There's no official name for a document that passes this stage, but let's call it Schematron valid.*



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

problème réside dans l'implication (finalement, par les humains) : l'idée qu'un schéma puisse exprimer complètement tout ce qui concerne un type de document.<sup>240 241</sup>

Ces vérifications ont des limites sur la sémantique, comme indiqué dans un article de 2022 qui propose une solution semi-automatique pour passer de XSD à OWL, puisque le XSD ne gère pas la sémantique :

Bien que le passage de données XML ou de schémas XSD à une représentation sémantique (RDF, RDFS ou OWL) soit une question étudiée de longue date dans le domaine du Web sémantique, une transformation automatique simple est rarement correcte. Ce processus se heurte à la difficulté de gérer des nœuds anonymes, de traiter la représentation de nœuds complexes, de capturer la sémantique des balises purement structurelles, ou encore de produire des constructeurs liés à la structuration [12, 10, 4, 22]. [17]

(voir énoncé p.201)

**Explication de l'assertion AD28 : Le XML, le DTD, le XSD sont utilisés par des instruments de documentation.**

En 2017, un article sur le *Model-based documentation* décrit des instruments servant à spécifier la documentation :

[Traduction libre]

---

240.

*So I think Murata-san has a good rule-of-thumb there, that schemas don't imply any semantics. Certainly schemas never imply all semantics, and often not even all the structural pr data type or linking rules. But that is not to say that schemas don't reflect semantic considerations through and through. The implying (ultimately, by humans) is the problem : the idea that any schema completely expresses everything about a document type.*

241. Rick Jelliffe, site web, « Schemas do not imply any semantics of documents », [https://scematron.com/opinion/\\_schemas\\_do\\_not\\_imply\\_any\\_semantics\\_of\\_documents\\_.html](https://scematron.com/opinion/_schemas_do_not_imply_any_semantics_of_documents_.html) (consulté le 2026-03-10).

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

[—] Darwin Information Type Architecture (DITA) est une architecture de documentation, basée sur XML, conçue pour fournir une prise en charge complète, de la création semi-automatique à la diffusion automatique de la documentation technique.

[—] DocBook est un système de documentation développé pour faciliter la production de publications relatives au matériel et aux logiciels, telles que des livres et des articles. [...] Les DTD DocBook et les langages de schémas, comme XML Schema, sont utilisés pour structurer la documentation résultante.

[—] S1000D est une spécification initialement conçue pour l'industrie aérospatiale et de défense, pour la préparation et la production de documentation technique. [...] Son principe fondamental repose sur la réutilisation de l'information, facilitée par les modules de données. Dans la version 4.0 de S1000D, ces modules sont représentés en XML. S1000D convient à la publication sous forme imprimée et électronique. L'architecture de génération de documents S1000D comprend un composant de stockage de données appelé base de données source commune (CSDB). <sup>242</sup> [86]

Le guide *The DITA style guide : best practices for authors* expose le contexte technologique du DITA dans la figure B.16 l'illustre. Le DITA repose sur le XML et fournit les moyens d'exprimer des informations sur le logiciel, le matériel, etc.

(voir énoncé p.201)

---

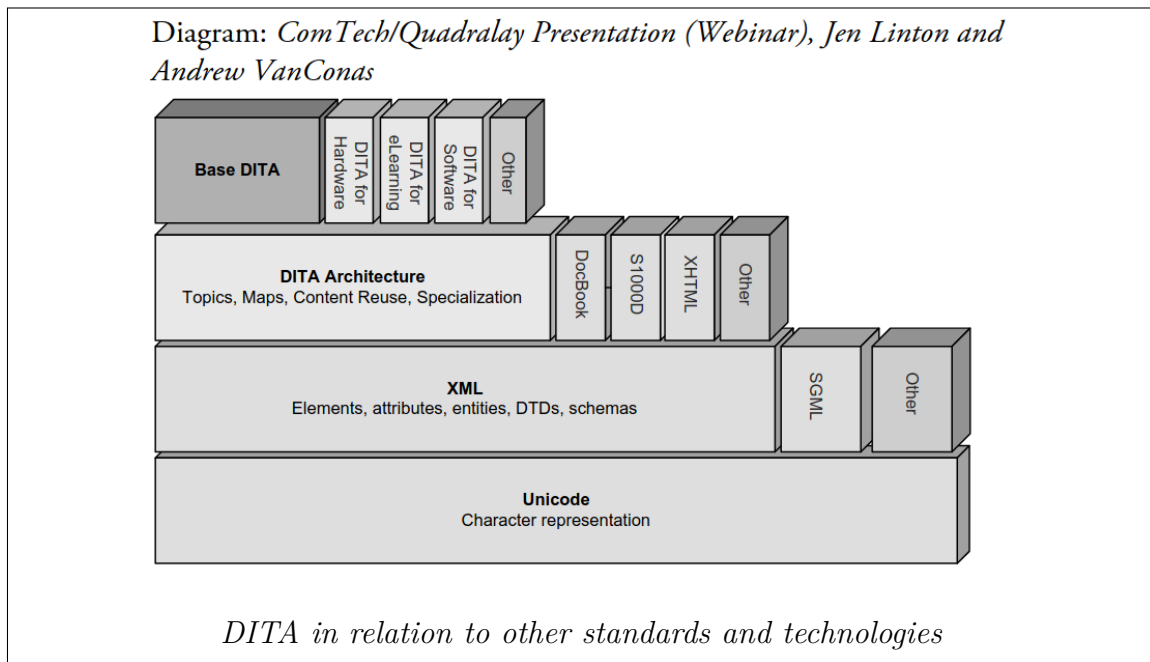
242.

[—] *Darwin Information Type Architecture (DITA) is a documentation architecture, based on XML, created to provide an end-to-end support, from semi-automatic authoring all the way through to automatic delivery of technical documentation.*

[—] *DocBook is a documentation system developed to facilitate the production of computer hardware and software related publications such as books and articles. [...] DocBook DTDs and schema languages such as XML Schema are employed to provide structure to the resulting documentation.*

[—] *S1000D is a specification originally designed for aerospace and defense industry for the preparation and production of technical documentation. [...] At its core, there is the principle of information reuse that is facilitated by Data Modules. The Data Modules in version 4.0 of S1000D are represented in XML. S1000D is suitable for publishing in both printed and electronic form. The S1000D document generation architecture includes a data storage component, which is called Common Source Database (CSDB).*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.16** – Technologies sur lesquelles le DITA repose (source : *figure 46* [358])

**Explication de l’assertion AD29 : Le DITA permet une adaptabilité pour les documents.**

Voici une liste des fonctions offertes par DITA [358] : [Traduction libre] « modularité, rédaction structurée (réduction du temps de rédaction, augmentation du temps d’analyse), typage de l’information, minimalisme, héritage, spécialisation, source unique, approche thématique, métadonnées, traitement conditionnel, publication par composants : concepts de rédaction DITA, orientation tâche, réutilisation du contenu, structure facilitant la traduction »<sup>243</sup>

(voir énoncé p.202)

**Explication de l’assertion AD30 : Les langages d’architecture sont nombreux.**

243. « *modularity, structured authoring (reduces authoring time, increases analysis time), information typing, minimalism, inheritance, specialization, single-source, topic-based, metadata, conditional processing, component publishing DITA authoring concepts, task-orientation, content re-use, translation-friendly structure* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

En 2013, un article inventorie avec des recherches et un sondage 177 langages d'architecture :

[Traduction libre] [...] nous avons découvert qu'une autre communauté utilise le terme ADL pour désigner les langages permettant de spécifier l'architecture pour la compilation adaptative [...]. La liste des 57 langages d'architecture (LA) produite à l'étape 1 est enrichie par les 120 LA trouvés à l'étape 2.7. Ces résultats sont utilisés dans la phase 2 pour constituer la communauté de praticiens ciblée par notre enquête.<sup>244</sup> [211]

(voir énoncé p.202)

**Explication l'assertion AD31 : La possibilité d'exprimer la sémantique est une fonctionnalité appréciée et recherchée par plusieurs professionnels informatiques utilisant des langages d'architecture.**

Un sondage comprenant 48 professionnels informatiques, utilisant des langages d'architecture, est réalisé [211]. Les participants ont été invités à donner leur avis sur certaines fonctionnalités, ainsi que sur leurs utilités passées et leurs utilités potentielles. La figure B.17 présente un résumé des résultats. Ces résultats mettent en évidence les observations suivantes :

- les trois fonctionnalités les plus utiles sont : support de plusieurs vues architecturales, syntaxe graphique, sémantique bien définie ;
- les trois fonctionnalités les plus utiles sont : support d'outils, support de plusieurs vues architecturales, sémantique bien définie.

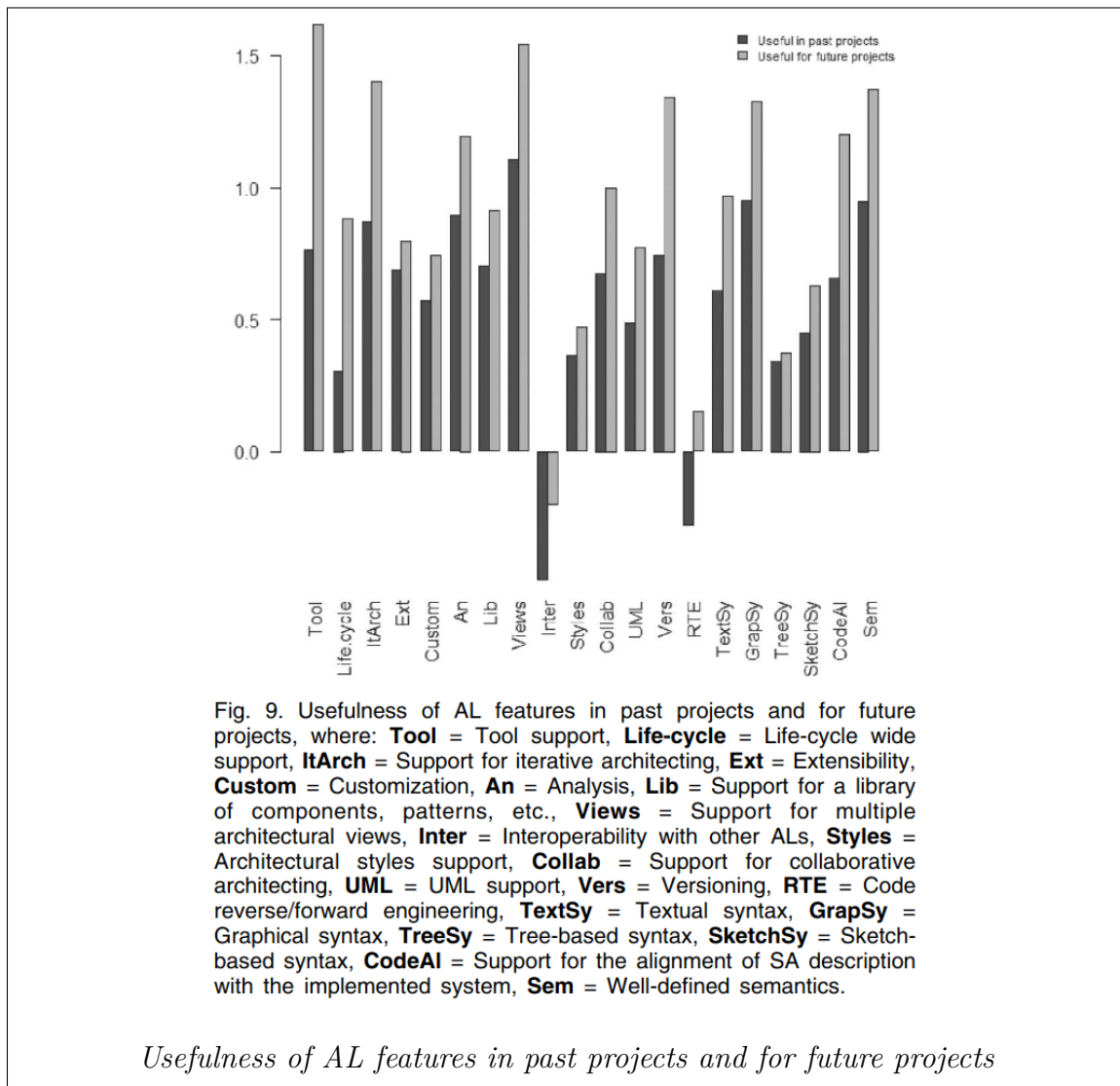
Après avoir exposé différents ADL, un autre article exprime trois problèmes qui restent non résolus [285] :

---

244.

[...] *we discovered that another community uses the term ADL to refer to languages for specifying the architecture for retargetable compilation [...]. The list of 57 ALs produced in step 1 is enriched by the 120 ALs found in step 2.7 This output is used in Phase 2 to build the community of practitioners target of our survey.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.17** – Fonctionnalités utiles pour les langages d’architecture selon des professionnels (source : *Figure 9* [211])

[Traduction libre]

[—] [...] le manque de support pour l’analyse formelle des architectures  
[...].

[—] [...] les notations qui ne sont pas adaptées aux systèmes complexes  
[...].

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

[—] [...] la non-faisabilité des systèmes [...]. <sup>245</sup> [86]

(voir énoncé p.202)

**Explication de l’assertion AD32 : Plusieurs utilisateurs des langages d’architecture n’utilisent pas de langages formels, du moins pas ceux permettant d’exprimer la sémantique.**

Dans l’article *What Industry Needs from Architectural Languages : A Survey*, Ivano Malavolta, Patricia Lago *et al.* affirment la sous-utilisation des langages formels :

[Traduction libre] Les outils de langages d’architecture issus de l’industrie peuvent répondre aux besoins des praticiens, mais ils manquent de la rigueur des langages d’architecture formels. Inversement, les langages d’architecture formels ne sont pas actuellement adoptés par l’industrie, principalement en raison de problèmes d’utilisabilité. Il est nécessaire de créer un pont pour combler cet écart. <sup>246</sup> [211]

Aussi, cet article présente les résultats d’un sondage ( $n = 48$ ) indiquant la préférence à l’égard de l’UML comparativement aux ADL.

[Traduction libre] Il ressort des données recueillies que 42 % des répondants utilisent exclusivement UML et les profils UML, 35 % utilisent conjointement les ADL et UML, tandis que seulement 17 % créent et utilisent exclusivement des ADL pour décrire les architectures logicielles. <sup>247</sup> [211]

---

245.

[—] [...] *lack of support for formal analysis of architectures* [...].

[—] [...] *notations that sometimes make specifying large and complex system architectures harder than it should be* [...].

[—] [...] *potential un-realizability of system architectures* [...].

246.

*AL tools from industry can cater to the needs of practitioners, but they lack the rigor of formal ALs. Conversely, formal ALs are not currently adopted by the industry, mainly for usability deficiencies. A bridge needs to be built to close this gap.*

247.

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

Le même article démontre le désintérêt envers la fonctionnalité « générer du code source ». Cette fonction découle directement de l'utilisation d'un langage formel :

[Traduction libre] Bien que la génération de code soit considérée comme une fonctionnalité intéressante, elle est rarement utilisée. Elle n'est pas non plus perçue comme très utile pour les projets futurs. Les raisons invoquées par les personnes interrogées sont très diverses [...].<sup>248</sup> [211]

Des raisons y sont invoquées :

[Traduction libre]

[—] [...] la description de l'architecture est trop complexe [...].

[—] [...] [la génération de code] exige trop de détails dans les descriptions [...].

[—] [...] [La génération de code] n'était pas l'objectif visé par la définition de l'architecture ! L'architecture avait d'autres objectifs [...].<sup>249</sup> [211]

Un autre article explique que les utilisateurs ne sont pas enclins à utiliser des langages formels qui permettent d'exprimer la sémantique :

[Traduction libre] La plupart des langages de description d'architectures (ADL) actuels (par exemple, Wright, LEDA, Darwin, PiLar, PRISMA, CONNECT et SOFA) adoptent une notation formelle pour spécifier le comportement des éléments architecturaux. Cette notation repose généralement sur une algèbre de processus (par exemple, FSP [28], CSP [18],

---

*What emerges from the collected data is that 42 percent of the respondents make exclusive use of UML and UML profiles, 35 percent use ADLs and UML together, while only 17 percent make and exclusively use ADLs when describing software architectures.*

248.

*While code generation is considered an interesting feature, it is rarely used. It is also not considered a very useful feature for future projects. The reasons provided by respondents are very different [...].*

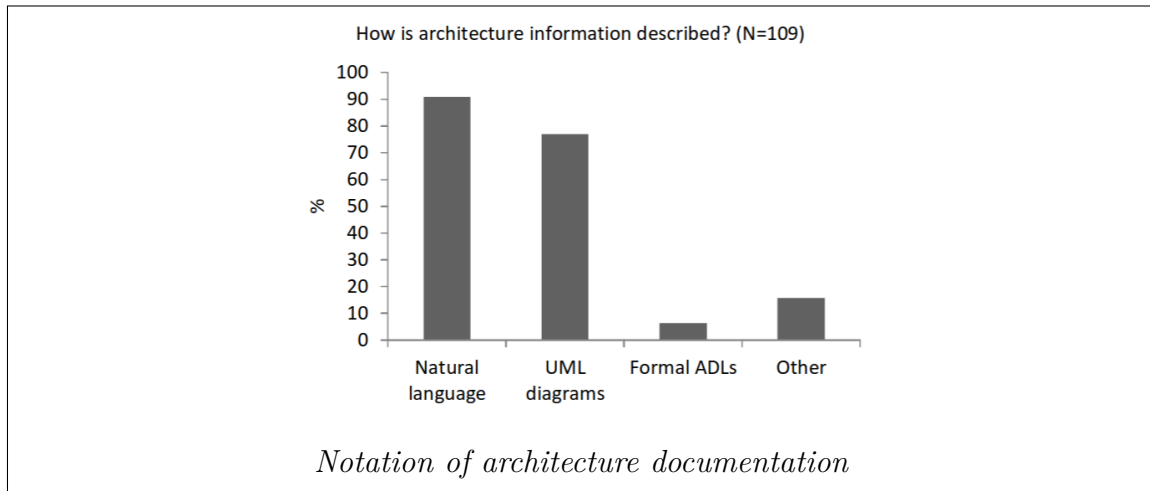
249.

*[—] [...] architecture description [is] on a too high level [...].*

*[—] [...] [code generation] Requires too much details in the descriptions [...].*

*[—] [...] [code generation] was not the intended goal of the architecture definition ! Architecture did have other focus [...].*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.18** – Langages d’architecture utilisés selon des professionnels (source : *Figure 7 part 1* [347])

CCS [35] ou le  $\pi$ -calcul [36]), même si d’autres formalismes sont également utilisés (par exemple, Z [47]). Bien qu’il soit important qu’ils fournissent un moyen formel de spécifier le comportement des architectures, les algèbres de processus, etc., sont malheureusement mal perçues par les praticiens [30].<sup>250</sup> [285]

Un sondage portant sur l’utilisation de la documentation d’architecture logicielle a été réalisé auprès de 147 professionnels informatiques provenant de divers pays et de multiples secteurs [347]. Comme l’illustre la figure B.18, les ADL formels ne sont utilisés qu’à environ 5 %.

(voir énoncé p.202)

**Explication de l’assertion AD33 : Il y a des exemples où un modèle relationnel est utilisé pour maintenir la documentation cohérente.**

---

250.

*The bulk of the current ADLs (e.g., Wright, LEDA, Darwin, PiLar, PRISMA, CONNECT, and SOFA) adopt a formal notation for specifying the behaviours of architectural elements. The notation is usually some process algebra (e.g., FSP [28], CSP [18], CCS [35], or  $\pi$ -calculus [36]), even though other formalisms are also used (e.g., Z [47]). Although it is important that they provide a formal means of specifying behaviours of architectures, process algebras, etc., are unfortunately not viewed favourably by practitioners [30].*



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

Scott R. Tilley dans un article de 1993 décrit la situation de la documentation :

[Traduction libre] Les techniques de documentation n'ont pas vraiment beaucoup changé en trente ans ; la plupart des documentations logicielles sont encore destinées à des activités de petite envergure.<sup>251</sup> [383]

Il y spécifie trois problèmes : [Traduction libre] « pour de petites entités informatiques »<sup>252</sup>, [Traduction libre] « généralement obsolète »<sup>253</sup> et [Traduction libre] « fournis une seule perspective »<sup>254</sup>. Voici la description qu'il fait du terme « *in-the-small* » :

[Traduction libre] La plupart des documentations logicielles sont de nature descriptive, car elles se limitent généralement à la description des algorithmes et des structures de données. Pour les grands systèmes existants, la compréhension des aspects structurels de l'architecture du système est plus importante que celle de tout composant algorithmique pris individuellement.<sup>255</sup> [383]

L'article propose également d'établir des relations entre les différents modèles :

[Traduction libre] Notre approche consiste à faire de la documentation une représentation « vivante » du code source, et non un texte séparé. Si la documentation reflète la structure, et, inversement, il n'y a pas de contradiction. La suite de cette section décrit les bases de la documentation de

---

251.

*Documentation techniques have not really changed very much in thirty years ; most software documentation is still targeted towards in-the-small activities.*

252. « *in-the-small* »

253. « *usually out-of-date* »

254. « *provides just one perspective* »

255.

*Most software documentation is documenting-in-the-small, since it typically describes the program at the algorithm and data structure level. For large legacy systems, an understanding of the structural aspects of the system's architecture is more important than any single algorithmic component.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

la structure logicielle : les stratifications de sous-systèmes virtuels et les modèles d'interconnexion logicielle. <sup>256</sup>

Un autre article décrit l'utilisation du S1000D pour le secteur des trains à haute vitesse :

[Traduction libre] Le module de données (MD) et la base de données de ressources communes (CSDB) sont deux concepts essentiels de la norme S1000D. Cette norme, qui définit un manuel d'utilisation de l'information (MUI), organise ses informations dans le MD et les gère dans la CSDB. [...] Après la compilation, les informations techniques (manuels) sont stockées dans la CSDB relationnelle de manière structurée et non redondante. <sup>257</sup> [401]

Voici une description de la base de données *Common Resource Data Base* (CSDB) utilisée dans le S1000D :

[Traduction libre] La norme S1000D ne précise pas les règles de conception et de mise en œuvre d'une CSDB. Elle spécifie uniquement la structure des données des objets d'information, indépendamment de toute implémentation logicielle. <sup>258</sup> [5]

(voir énoncé p.202)

---

256.

*Our approach is to make the documentation a « live » representation of the source code, not separate text. If the documentation is the structure, and vice versa, no discrepancy exists. The rest of this section describes the basis for documenting software structure : virtual subsystem stratifications (VSS's) and software interconnection models (IM's).*

257.

*Data module (DM) and common resource data base (CSDB) are two essential concepts of S1000D, a standardized IETM organizes its information with DM, while manages it in CSDB. [...] After compilation, the technical information (manuals) is stored in the relational CSDB with a structured and nonredundancy manner.*

258.

*S1000D does not specify the design and implementation rules for a CSDB. It only specifies the data structure of the information objects, which is independent from any software implementation.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### Des techniques de traitement automatique des langues

**Explication de l’assertion AD38 : Plusieurs concepts de l’intelligence artificielle sont logiquement interreliés.**

La littérature fournit ces informations qui ont servi à construire cette assertion :

- définition d’intelligence artificielle (IA) : [Traduction libre] « [...] <discipline> recherche et développement des mécanismes et applications des systèmes d’IA [...]. » <sup>259</sup> [143] ;
- définition de système d’intelligence artificielle : [Traduction libre] « [...] système conçu pour générer des résultats tels que du contenu, des prévisions, des recommandations ou des décisions en fonction d’un ensemble d’objectifs définis par l’humain [...] » <sup>260</sup> [143] ;
- définition d’apprentissage automatique : [Traduction libre] « [...] processus d’optimisation des paramètres d’un modèle par des techniques de calcul [...]. » <sup>261</sup> [143] ;
- définition de réseau de neurones (RN) : [Traduction libre] « [...] réseau d’une ou plusieurs couches de neurones [...] » <sup>262</sup> [143] ;
- synonymes de réseau de neurones : *neural net* et *artificial neural network* [143] ;
- définition de réseau de neurones : [Traduction libre] « [...] approche pour créer des représentations hiérarchiques riches grâce à l’entraînement de réseaux neuronaux avec de nombreuses couches cachées [...] » <sup>263</sup> [143] ;
- synonyme d’apprentissage profond : *deep neural network learning* [143] ;
- description d’apprentissage profond (AP) : [Traduction libre] « L’apprentissage profond est une famille élargie de techniques pour l’apprentissage machine où les

---

259. « [...] <discipline> research and development of mechanisms and applications of AI systems [...]. »

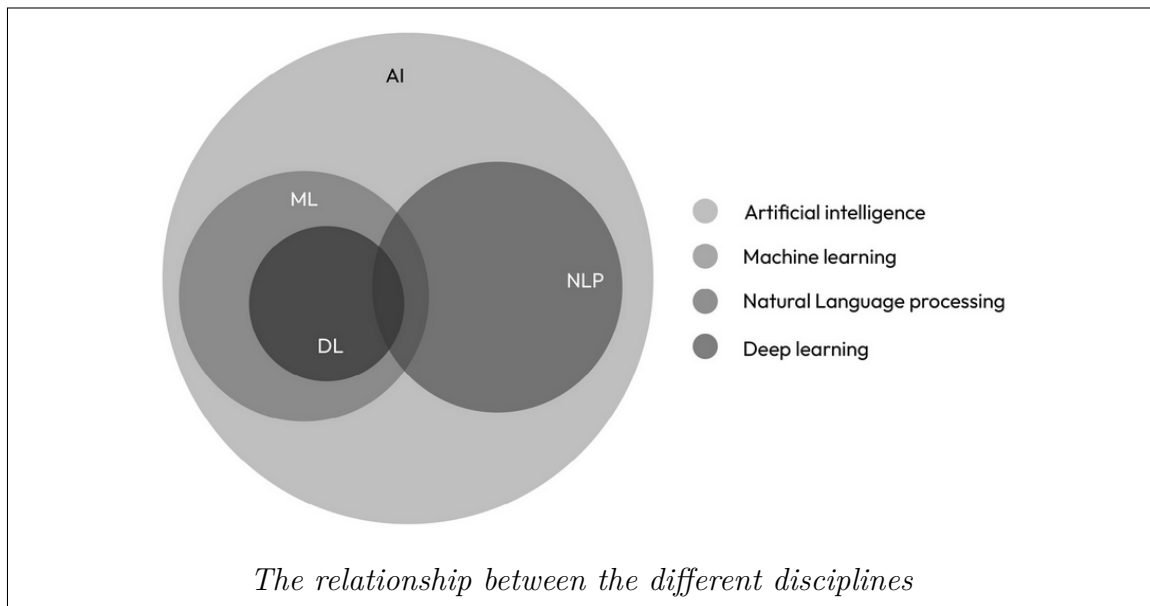
260. « [...] engineered system that generates outputs such as content, forecasts, recommendations or decisions for a given set of human-defined objectives [...] »

261. « [...] process of optimizing model parameters through computational techniques [...]. »

262. « [...] network of one or more layers of neurons [...]. »

263. « [...] approach to creating rich hierarchical representations through the training of neural networks with many hidden layers [...] »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



<sup>a</sup> Dans l'ouvrage, le diagramme est en couleur, puisqu'il comporte des noms, une version en gris a été retenu ici.

**FIGURE B.19** – Diagramme de Venn sur l'IA et le TAL (modifié la *Figure 1.2* [98])

hypothèses prennent la forme de circuits complexes d'algèbre avec des liaisons multiples. » <sup>264</sup> [351] ;

- définition 1 de grand modèle de langage (GML) : « Modèle de langage constitué par apprentissage profond à partir de mégadonnées. » [254] ;
- définition 2 de grand modèle de langage : [Traduction libre] « [...] Les GML comme GPT (Generative Pre-trained Transformer) utilisent des techniques d'apprentissage profond [...]. » <sup>265 266</sup>.

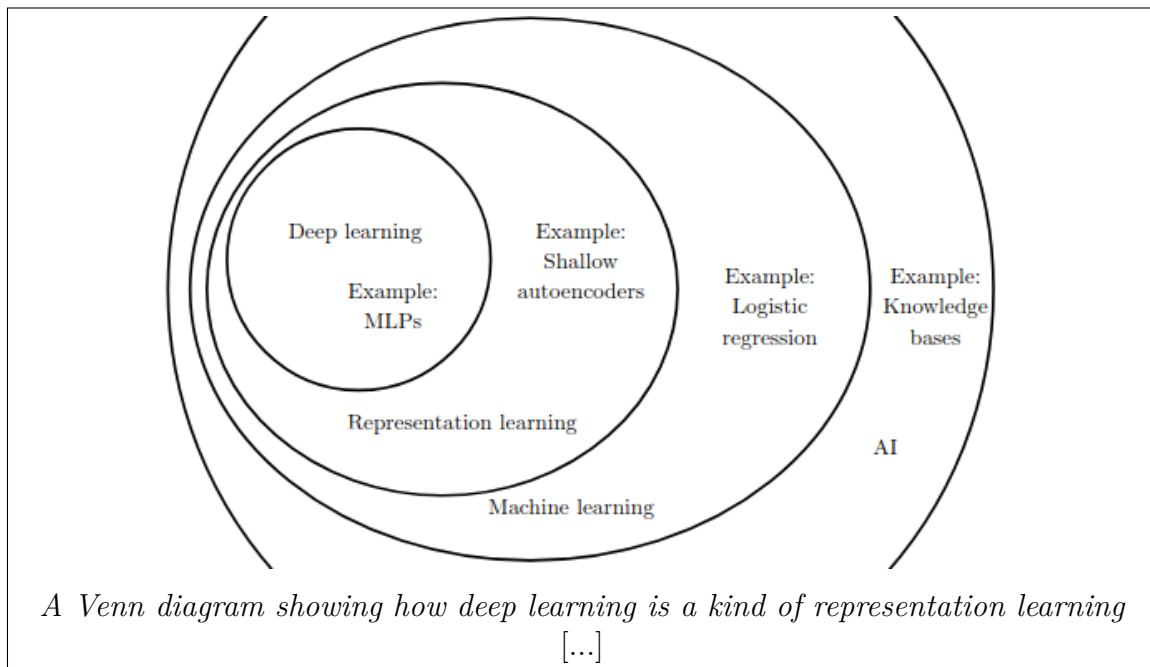
Dans l'ouvrage *Mastering NLP from foundations to LLMs* inclut un diagramme de Venn (figure B.19) représentant les relations entre les différentes disciplines d'IA [98]. Tandis que l'ouvrage *Deep learning* d'Ian Goodfellow, Yoshua Bengio et Aaron Courville, présentent notamment ce type de diagramme de Venn [104]. Il s'agit de la

264. « *Deep learning is a broad family of techniques for machine learning in which hypotheses take the form of complex algebraic circuits with tunable connection strengths.* »

265. « [...] *LLMs like GPT (Generative Pre-trained Transformer) use deep learning techniques [...].* »

266. University of British Columbia, site web, « Glossary of GenAI Terms », <https://ai.ctlt.ubc.ca/resources/glossary-of-genai-terms/> (consulté le 2025-10-16).

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



<sup>a</sup> Diagramme est en partie.

**FIGURE B.20** – Diagramme de Venn sur l’IA (modifié la *Figure 1.4* [104])

figure B.20.

(voir énoncé p.203)

**Explication de l’assertion AD39 : Les domaines du traitement automatique des langues et de l’intelligence artificielle se croisent sur le plan historique.**

Les ouvrages *Mastering NLP from foundations to LLMs* et *Artificial Intelligence A Modern Approach* proposent une synthèse de l’histoire du TAL [98, 351]. En voici un très bref résumé fondé sur les éléments présentés dans ces ouvrages :

- Dans les années 1950, les systèmes à base de règles de production étaient utilisés<sup>267</sup>. Ces systèmes sont devenus plus sophistiqués, grâce à des bases de connaissances plus grandes, et ce, jusqu’à la fin des années 1980.
- À la fin des années 1980, les méthodes statistiques se sont imposées sous l’appellation d’apprentissage automatique (ML)[237].

---

267. En anglais : *rule-based systems*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- Les années 2010 marquent le début de l'apprentissage profond (DL, *Deep Learning* en anglais), ce qui a relancé la popularité des réseaux de neurones (NN, *Neural Network* en anglais). Ces derniers existent depuis les années 1950 et ont déjà été populaires dans les années 1980.

Dans l'ouvrage *Deep learning*, l'auteur affirme que l'apprentissage profond (AP) est simplement le dernier nom donné, voici la citation complète ;

[Traduction libre] L'apprentissage profond n'est une nouveauté qu'en apparence, car il a été relativement impopulaire pendant plusieurs années avant sa popularité actuelle, et parce qu'il a connu de nombreux noms, jusqu'à être récemment appelé " apprentissage profond ". Ce domaine a connu de nombreuses refontes, reflétant l'influence de différents chercheurs et perspectives.<sup>268</sup> [104]

L'ensemble de ces techniques tente de produire des modèles. Dans le contexte du TAL, le résultat est appelé un modèle de langage. Parmi ces modèles de langage apparaissent les grands modèles de langage (GML) à la fin des années 2010.  
(voir énoncé p.203)

**Explication de l'assertion AD40 : Plusieurs professionnels informatiques affirment que l'architecture et les exigences s'expriment couramment en langage naturel.**

Un article de D. Rost [347] sonde l'utilisation de la documentation d'architecture logicielle. Selon son sondage, auquel 109 des 147 professionnels informatiques ont répondu, 90 % utilisent un langage non formel. (voir AD28 p.302 figure B.8 « Comment les informations architecturales sont-elles décrites ? »)

Un article publié en 2022 décrit la pratique des professionnels informatiques en ce qui concerne les exigences [159]. Il analyse quatre sondages qui ont été réalisés en 2003 ( $n = ?$ ), en 2008 ( $n = ?$ ), en 2013 ( $n = ?$ ) et en 2020 ( $n = 117$ ). Au cours de

---

268.

*Deep learning only appears to be new, because it was relatively unpopular for several years preceding its current popularity, and because it has gone through many different names, only recently being called "deep learning." The field has been rebranded many times, reflecting the influence of different researchers and different perspectives.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

ces sondages, le taux des professionnels informatiques jugeant que leurs exigences deviennent de plus en plus formelles est passé de 45 % à 69 %.

(voir énoncé p.203)

**Explication l’assertion AD41 : Au moins jusqu’en 2020, les documents de spécification des exigences est la catégorie de documents utilisant le plus les techniques de TAL.**

Une revue de littérature publiée en 2022 portant sur l’utilisation du TAL pour les exigences [411] analyse un corpus de 404 articles publiés entre 1983 et 2019 (dont plus de la moitié [214 articles] a été publiée après 2012. Parmi l’ensemble des articles recensés, 271 articles portent sur une proposition de solution, tandis que 239 articles traitent de spécification des exigences système ou logicielles comme type de documents, les autres types sont des *use cases*, des *user stories*, etc. D’ailleurs, les fonctions et les techniques les plus utilisées sont : étiquetage grammatical (*POS Tagging*) ( $\approx 185$ ), analyse lexicale (*Tokenization*) ( $\approx 80$ ), analyse syntaxique (*Parsing*) ( $\approx 75$ ), suppression des mots vides (*Stop-Words Removal*) ( $\approx 75$ ), extraction de termes (*Term Extraction*) ( $\approx 70$ ), racinisation (*Stemming*) ( $\approx 70$ ), lemmatisation (*Lemmatization*) ( $\approx 50$ ), mesures de similarité (*Similarity Measures*) ( $\approx 50$ ), TF-IDF ( $\approx 40$ ), etc.

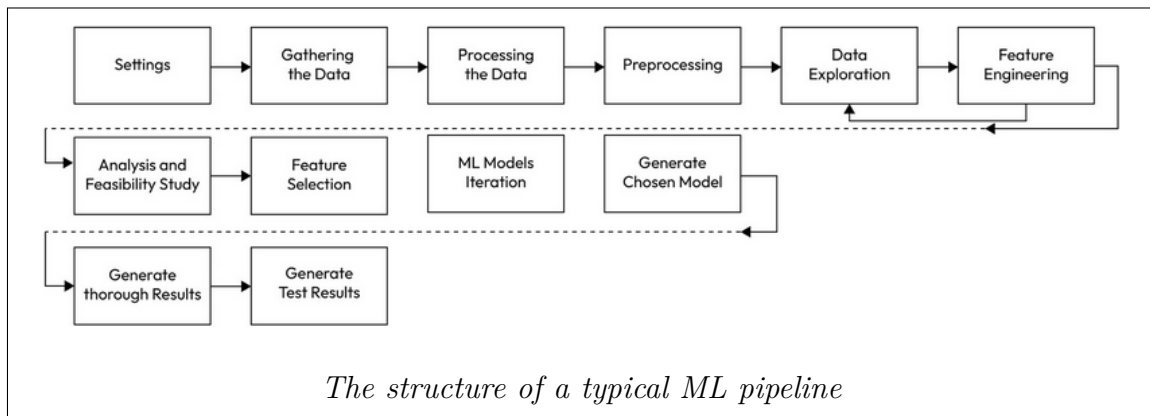
(voir énoncé p.203)

**Explication de l’énoncé AD42 : Les techniques de TAL sont de plus en plus utilisées dans la littérature scientifique.**

Les utilisations des techniques de TAL croissent, il y a de plus en plus de propositions et d’expérimentations. Des articles publiés entre 2014 et 2023 démontrent cette progression :

- un article publié en 2014 présente une solution pour générer un sommaire depuis le code source [216] ;
- un article publié en 2016 décrit qu’il est possible d’extraire les concepts des documents, tout en soulignant le caractère mitigé des résultats obtenus [225] ;
- un article publié en 2018 démontre qu’il est possible de se servir des techniques TAL comme instruments de suggestions ou de complétions pour la documentation [379] ;

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.21** – Procédé typique pour la création d’un modèle de langage (source : Figure 5.2 [98])

- un article publié en 2023 énumère tous les domaines d’application du TAL [356]. Le TAL couvre 12 domaines d’application, chacun divisé. Il s’agit d’une tentative de taxonomie réalisée à partir de 80 000 articles provenant de la bibliothèque *Anthology* (qui est consacrée au TAL) de l’Association for Computational Linguistics (ACL). Le nombre d’articles contenus dans cette bibliothèque suit une courbe exponentielle entre 1952 et 2023.

(voir énoncé p.203)

**Explication de l’assertion AD43 : Il n’y a pas de règles exactes pour sélectionner une chaîne ou une classe de modèle à entraîner.**

L’ouvrage *Mastering NLP from foundations to LLMs* montre une chaîne typique (figure B.21) pour l’entraînement d’un modèle avec de l’apprentissage automatique [98]. Dans l’ouvrage *Artificial Intelligence A Modern Approach*, il est écrit qu’il n’existe pas de procédé parfaitement établi :

[Traduction libre] Avec des données nettoyées et une compréhension intuitive de celles-ci, il est temps de construire un modèle. [...] Il n’existe aucune méthode infaillible pour choisir la meilleure classe de modèle, mais voici quelques lignes directrices générales.<sup>269</sup> [351]

(voir énoncé p.203)

---

269.



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### **Explication de l’assertion AD44 : Plusieurs techniques de TAL conduisent à la création de modèles de type boîte noire.**

En 2024, un guide consacré à l’explication des modèles d’IA est publié [124]. Rédigé par un collectif de 27 auteurs issus, majoritairement, du milieu universitaire, il énumère des modèles de types boîte blanche et boîte noire <sup>270</sup> :

- Les modèles « boîtes blanches » sont : [Traduction libre] « Régression linéaire, régression logistique, arbres de décision, systèmes à base de règles, K-plus proches voisins, classificateurs bayésiens naïfs, modèles additifs généralisés » <sup>271</sup>.
- Les modèles « boîtes noires » sont : [Traduction libre] « Réseaux de neurones (par exemple, modèles d’apprentissage profond), machines à vecteurs de support (SVM), méthodes d’ensemble (par exemple, forêts aléatoires, machines à gradient boosting), modèles de transformateurs (par exemple, BERT, GPT), réseaux de neurones graphiques » <sup>272</sup>.

(voir énoncé p.203)

### **Explication de l’assertion AD45 : Les tentatives pour expliquer ces modèles sont de plus en plus nombreuses, cela montre l’insuffisance des explications.**

Un article publié en 2020 décrit un domaine récent de l’AI, l’*Explainable Artificial Intelligence* (XAI) [23]. L’article réalise des recherches sur Scopus, lesquelles mettent en évidence une croissance du nombre de publications (figure B.22). Dans un article publié en 2019, Cynthia Rudin expose des problèmes d’interprétabilité, d’explicabilité, de compréhensibilité et de transparence avec l’apprentissage machine :

---

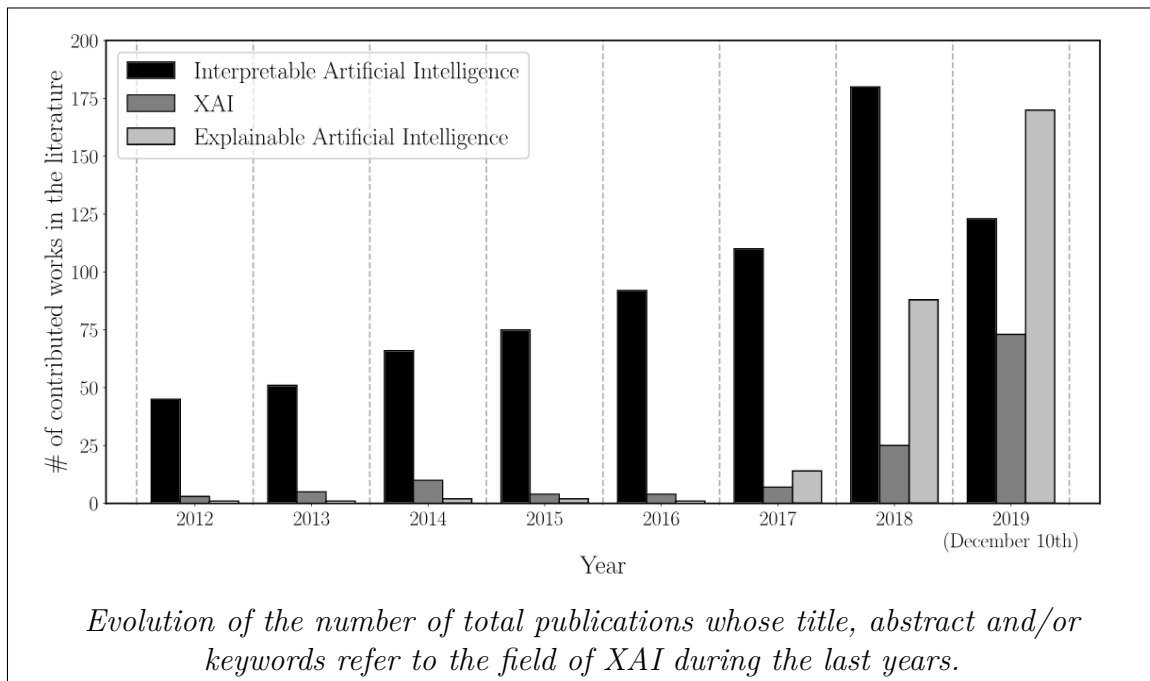
*With cleaned data in hand and an intuitive feel for it, it is time to build a model. [...] There is no guaranteed way to pick the best model class, but there are some rough guidelines.*

270. La boîte est blanche est le contraire de la boîte noire.

271. « *Linear Regression, Logistic Regression, Decision Trees, Rule-based Systems, K-Nearest Neighbors (KNN), Naive Bayes Classifiers, Generalized Additive Models (GAMs)* »

272. « *Neural Networks (e.g., Deep Learning Models), Support Vector Machines (SVMs), Ensemble Methods (e.g., Random Forests, Gradient Boosting Machines), Transformer Models (e.g., BERT, GPT), Graph Neural Networks (GNNs)* »

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.22** – Nombre d’articles voulant expliquer l’IA entre 2012 et 2019 (source : Figure 1 [23])

[Traduction libre]

[—] Il est faux de croire qu’il existe nécessairement un compromis entre précision et interprétabilité.

[—] Les méthodes d’apprentissage automatique explicables fournissent des explications qui ne reflètent pas fidèlement les calculs du modèle original.

[—] Ces explications sont souvent incohérentes ou insuffisamment détaillées pour comprendre le fonctionnement de la boîte noire.

[—] Les modèles de type « boîte noire » sont souvent incompatibles avec les situations où des informations externes à la base de données doivent être intégrées à une évaluation des risques.

[—] Les modèles de type « boîte noire », même avec explications, peuvent engendrer un processus de décision excessivement complexe, propice aux erreurs humaines. <sup>273</sup> [350]

273.

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

(voir énoncé p.203)

### **Explication l’assertion AD46 : L’absence de culture sur la reproductibilité limite l’amélioration des connaissances.**

En 2025, un article décrit les problèmes de reproductibilité des modèles en AA [359] et énonce cinq items essentiels à la reproductibilité ;

- versionner le code ;
- avoir accès aux données ;
- versionner les données ;
- avoir les journaux de l’expérimentation ;
- la chaîne qui a servi à créer le modèle.

En 2025, le populaire *Artificial Intelligence Index Report* de l’Université de Stanford cite un article décrivant les problèmes de reproductibilité et d’évaluation sur les expérimentations [213]. Le nom de l’article cité est *BetterBench* [317]. Au total, 24 expérimentations majeures ont été analysées et il en ressort que :

- 14 n’ont pas indiqué de signification statistique (*failed to report statistical significance*) ;
- 17 ne comportaient pas de scripts pour la réplication des résultats (*lacked scripts for result replication*) ;
- la plupart souffraient d’une documentation insuffisante (*most suffered from inadequate documentation*).

(voir énoncé p.203)

---

[—] *It is a myth that there is necessarily a trade-off between accuracy and interpretability.*

[—] *Explainable ML methods provide explanations that are not faithful to what the original model computes.*

[—] *Explanations often do not make sense or do not provide enough detail to understand what the black box is doing.*

[—] *Black box models are often not compatible with situations where information outside the database needs to be combined with a risk assessment.*

[—] *Black box models with explanations can lead to an overly complicated decision pathway that is ripe for human error.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**Explication de l’assertion AD47 : Plusieurs experts affirment qu’il n’y a pas d’accumulation fiable des connaissances dans le domaine du TAL.**

En 2024, Max Silberztei, auteur de l’ouvrage *Linguistic Resources for Natural Language Processing : On the Necessity of Using Linguistic Methods to Develop NLP Software*, met en garde contre les modèles de type boîte noire [364] :

[Traduction libre] L’utilisation de méthodes de type « boîte noire » basées sur des corpus d’entraînement, produisant des résultats satisfaisants, mais dépourvus de toute capacité d’explication ou de généralisation, constitue une approche viable pour résoudre les problèmes d’ingénierie logicielle non critiques en TAL. Cependant, il est temps que tous les scientifiques, linguistes et informaticiens, prennent conscience que cette approche est à l’opposé de ce que les approches scientifiques exigent : une accumulation fiable de connaissances.<sup>274</sup> [364]

En 2025, dans l’article *Toward an evaluation science for generative AI systems* [399], Laura Weidinger *et al.* affirment que la manière actuelle d’expérimenter n’est tout simplement pas suffisante pour améliorer nos connaissances :

[Traduction libre] Pour que l’évaluation de l’IA devienne une véritable « science », elle doit répondre à certains critères. Les sciences se caractérisent par des théories sur leurs objets d’étude, lesquelles peuvent être exprimées sous forme d’hypothèses testables. Les instruments de mesure utilisés pour tester ces hypothèses doivent garantir la cohérence expérimentale (fiabilité, validité interne) et la généralisabilité (validité externe). Enfin, les sciences se caractérisent par l’itération : au fil du temps, les approches et les instruments de mesure sont affinés et de nouvelles connaissances sont mises au jour. Collectivement, ces propriétés des sciences

---

274.

*Using training-corpus-based "black box" methods that produce satisfactory results, but without any power of explanation nor generalization, is a viable approach to non-critical NLP software engineering problems. However, it is time for all scientists—both linguists and computer scientists—to realize that this approach is the opposite of what scientific approaches require : reliable accumulation of knowledge.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

contrastent fortement avec la pratique consistant à développer rapidement des référentiels statiques pour évaluer les systèmes d'IA générative, tout en anticipant que, d'ici quelques mois, ces référentiels deviendront beaucoup moins utiles, voire obsolètes.<sup>275</sup> [399]

(voir énoncé p.204)

### Des techniques d'apprentissage profond

**Explication la citation AD48 : L'apprentissage profond crée des grands modèles de langage qui améliorent les résultats en traitement automatique des langues.**

L'AP est l'approche la plus utilisée en TAL et est celle qui a eu le plus d'impact, selon l'ouvrage d'*Artificial Intelligence A Modern Approach*, en voici deux extraits :

[Traduction libre] L'apprentissage profond est actuellement l'approche la plus utilisée pour des applications, telles que la reconnaissance visuelle d'objets, la traduction automatique, la reconnaissance vocale, la synthèse vocale et la synthèse d'images ; il joue également un rôle important dans les applications d'apprentissage par renforcement. [...] L'apprentissage profond a également eu un impact considérable sur les applications de traitement automatique du langage naturel (TAL), telles que la traduction automatique et la reconnaissance vocale. Parmi ses avantages pour ces applications, on peut citer la possibilité d'un apprentissage de bout en

---

275.

*For AI evaluation to mature into a proper "science," it must meet certain criteria. Sciences are marked by having theories about their targets of study, which can be expressed as testable hypotheses. Measurement instruments to test these hypotheses must provide experimental consistency (i.e., reliability, internal validity) and generalizability (i.e., external validity). Finally, sciences are marked by iteration : Over time, measurement approaches and instruments are refined and new insights are uncovered. Collectively, these properties of sciences contrast sharply with the practice of rapidly developing static benchmarks for evaluating generative AI systems, while anticipating that within a few months such benchmarks will become much less useful or obsolete.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

bout, la génération automatique de représentations internes du sens des mots et l’interchangeabilité des encodeurs et décodeurs appris.<sup>276</sup> [351]  
(voir énoncé p.204)

**Explication de l’assertion AD49 : Les bancs d’essai pour les GML sont davantage réussis, ce qui rend nécessaire le développement de nouveaux bancs d’essai.**

L’*Artificial Intelligence Index Report* de 2025 contient des informations sur la performance des techniques [213]. Il y est discuté majoritairement de GML, comme l’indique le tableau *Timeline : Significant Model and Dataset Release* [213]. Voici un sommaire du tableau (tableau B.10), GML y apparaît 14 fois sur un total de 38. Dans le même rapport se trouve un comparatif de différents bancs d’essai à la *Figure 2.1.33* [213]. Les résultats sont disposés de 0% à 120% où 100% équivaldraient aux compétences humaines. Il est observé qu’il y a un banc d’essai qui disparaît en 2021 et deux en 2022. Tandis que d’autres apparaissent, dont deux en 2019, un en 2021 puis deux en 2023. Les auteurs expliquent ces changements de la façon suivante :

[Traduction libre] Ces dernières années, les capacités de raisonnement des systèmes d’IA ont tellement progressé que les anciens *benchmarks*, comme SQuAD (pour le raisonnement textuel) et VQA (pour le raisonnement visuel), sont devenus saturés, ce qui indique un besoin de tests de raisonnement plus difficiles.<sup>277</sup> [213]

---

276.

*Deep learning is currently the most widely used approach for applications such as visual object recognition, machine translation, speech recognition, speech synthesis, and image synthesis ; it also plays a significant role in reinforcement learning applications. [...] Deep learning has also had a huge impact on natural language processing (NLP) applications such as machine translation and speech recognition. Some advantages of deep learning for these applications include the possibility of end-to-end learning, the automatic generation of internal representations for the meanings of words, and the interchangeability of learned encoders and decoders.*

277.

*In recent years, the reasoning abilities of AI systems have advanced so much that older benchmarks like SQuAD (for textual reasoning) and VQA (for visual reasoning) have become saturated, indicating a need for more challenging reasoning tests.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABEAU B.10** – Sommaire des étiquettes sur les techniques utilisées (modifié de : [213])

|                            |                                 |                              |
|----------------------------|---------------------------------|------------------------------|
| — <i>LLM</i> (14)          | — <i>text-to-text</i> (1)       | — <i>text-to-song</i> (1)    |
| — <i>dataset</i> (2)       | — <i>text-to-image</i> (3)      | — <i>song-to-song</i> (1)    |
| — <i>model/dataset</i> (1) | — <i>text-to-video</i> (2)      | — <i>text-to-podcast</i> (1) |
| — <i>multimodal</i> (3)    | — <i>image-to-video</i> (1)     | — <i>tool</i> (1)            |
| — <i>language</i> (2)      | — <i>agentic capability</i> (1) | — <i>iPhone feature</i> (1)  |
| — <i>vision</i> (1)        | — <i>math</i> (1)               | — <i>biology</i> (1)         |

<sup>a</sup> Il s’agit d’un sommaire de la colonne **Category**

<sup>b</sup> Le rapport parle de **catégories**, mais cela ressemble davantage à des étiquettes.

(voir énoncé p.204)

### Explication de l’assertion AD50 : Plusieurs professionnels informatiques utilisent les GML.

Un article publié en 2025 a analysé l’utilisation de ChatGPT par des professionnels informatiques [209]. Pour ce faire, les auteurs analysent les données provenant du jeu de données DevChat et conservent uniquement les messages qui possèdent des liens URL vers GitHub. On en dénombre 2 547, répartis comme suit : [209] ; 1 105 liens vers des fichiers de code source, 823 vers des *Commits*, 295 vers des *Issues*, 266 vers des *Pull Requests*, et 58 vers des fils de discussions. Les cinq tâches les plus demandées (présentées en ordre décroissant) étaient :

- la génération et complétion de code source (888) ;
- la modification et l’optimisation de code source (686) ;
- la revue de code source et le débogage (184) ;
- le déploiement et la configuration (123) ;
- l’explication de code source et la compréhension (111).

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

Dans le *Artificial Intelligence Index Reporte 2024*, il est présenté le sondage de Stack Overflow<sup>278</sup> portant sur la relation entre les professionnels informatiques et les instruments d'IA [214]. Réalisé en 2023, ce sondage a recueilli les réponses de plus de quatre-vingt-dix mille professionnels informatiques.

Selon ce sondage, les deux instruments les plus utilisés par catégories sont :

- outils de développement :
  - GitHub Copilot (56%) ;
  - Tabnine (12 %) ;
- outils de recherche :
  - ChatGPT (83 %) ;
  - Bing AI (19 %).

GitHub Copilot et ChatGPT utilisent des GML.

[Traduction libre] Copilot permet aux développeurs de choisir parmi plusieurs grands modèles de langage (GML), notamment les modèles GPT d'OpenAI, Claude d'Anthropic, Grok de xAI et Gemini de Google.<sup>279 280</sup>

Le figure B.23 présente les réponses des professionnels informatiques sur les usages actuels, les usages désirés et les usages non désirés quant aux instruments d'IA. Comme il est indiqué, les professionnels informatiques s'en servent pour écrire du code (82 %). Cependant, ils souhaiteraient s'en servir pour tester le code (55 %) et documenter le code (50 %).

---

278. Stack Overflow est un site Web proposant des questions et réponses dans le domaine de l'informatique.

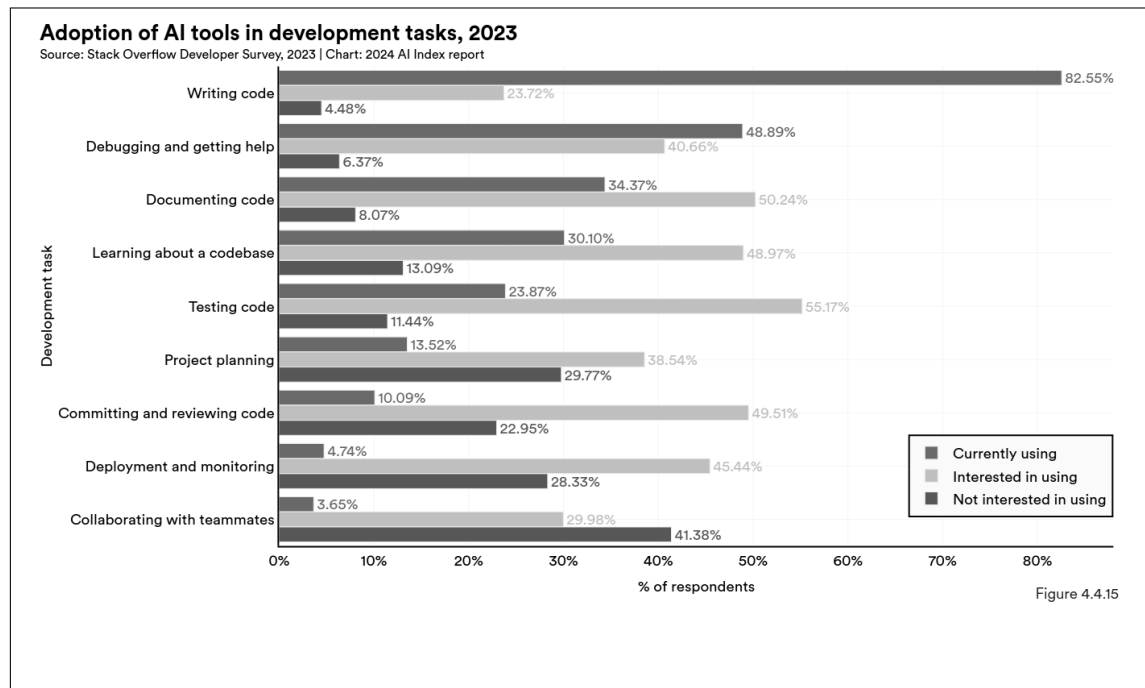
279.

*Copilot enables developers to choose between multiple large language models (LLMs), including OpenAI's GPT models, Anthropic's Claude, xAI's Grok, and Google's Gemini.*

280. Wikipedia, « Github Copilot », [https://en.wikipedia.org/wiki/GitHub\\_Copilot](https://en.wikipedia.org/wiki/GitHub_Copilot) (consulté le 2025-08-15).



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE



**FIGURE B.23** – Adoption d’outils d’IA pour des tâches de développement (source : *Figure 4.4.15* [214])

(voir énoncé p.204)

**Explication de l’assertion AD51 : Les ambiguïtés provenant de la langue et du contexte demeurent même pour les GML.**

Un article publié en 2022 fait un état de l’art sur le TAL [160]. Les auteurs dressent quelques constats :

[Traduction libre]

[—] Dans une langue, les mots et expressions contextuels peuvent avoir des significations différentes au sein d’une même phrase. Si ces mots et expressions sont faciles à comprendre pour les humains, leur utilisation représente néanmoins un défi.

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

[—] Les langues contenant des expressions familières, des tournures de phrase, des idiomes et un jargon culturel spécifique compliquent la conception de modèles destinés à un usage généralisé, [...].

[—] [...] les modèles de TAL peuvent être performants dans un domaine ou une zone géographique spécifique, mais pour une utilisation à grande échelle, ces difficultés doivent être surmontées <sup>281</sup> [160]

Un article publié en 2023 effectue une revue de littérature sur l'utilisation du TAL appliqué à la sécurité de l'aviation. [407].

- L'article résume vingt articles dont la majorité avait comme objectifs d'identifier les incidents.
- Les auteurs des différents articles utilisent des rapports ou des bases de données standardisés, treize utilisent la base de données *Aviation Safety Reporting System* (ASRS) qui provient de la NASA.
- Le langage utilisé est l'anglais dans dix-huit des articles.
- Les techniques de TAL utilisées peuvent être des combinaisons de ; *SVM*, *LSA*, *LDA*, *TF-IDF*, *Long short-term memory (LSTM)*, *Bag of Word (BoW)*, *Word2Vec*, *Diffusion Maps (DM)*, *Depth Neural Network (DNN)*, etc.
- Les techniques s'appliquent à la reconnaissance de texte et de la parole.

Le contexte revient comme un défi important :

[Traduction libre] Le traitement automatique du langage naturel peine à comprendre le contexte et le sens des mots et des expressions, notamment dans le jargon spécifique à l'aviation. Par exemple, le terme « piste » peut désigner à la fois la piste physique et les procédures de décollage et d'atterrissage qui y sont associées. <sup>282</sup>

---

281.

[—] *Contextual words and phrases in the language where same words and phrases can have different meanings in a sentence which are easy for the humans to understand but makes a challenging task.*

[—] *Language containing informal phrases, expressions, idioms, and culture-specific lingo make difficult to design models intended for the broad use, [...].*

[—] [...] *the models for NLP may be working good for an individual domain, geographic area but for a broad use such challenges need to be tackled.*

282.

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

Un article publié en 2024 démontre que les ellipses sont particulièrement difficiles pour le TAL et que les GML n’y échappent pas :

[Traduction libre]

[—] Les constructions d’ellipses restent évidemment un défi pour tous les pipelines TAL SOTA courants et les systèmes basés sur des règles.

[—] Le fait que des modèles spécifiques entraînés à prédire les ellipses dans les phrases surpassent les modèles de langage semble indiquer que l’absence de données explicites et l’apprentissage automatique autosupervisé pur ne suffisent pas non plus à traiter les éléments opaques du langage. L’entraînement des GML sur des données purement explicites ignore des propriétés importantes du langage. Les phénomènes d’ellipses sont grammaticaux et systématiques, et il semble problématique pour les GML actuels de deviner les suites implicites. <sup>283</sup> [52]

(voir énoncé p.204)

**Explication de l’assertion AD52 : Il n’a pas été encore démontré, dans un contexte réel, que les GML peuvent apporter une aide pour les documents de spécification des exigences.**

Il y a plusieurs articles qui s’intéressent aux performances des GML pour les exigences en logiciel. En 2024, une revue de littérature obtient à 22 articles à analyser sur l’utilisation de ChatGPT et les exigences logicielles [212]. Plusieurs articles s’intéressent aux performances des GML vis-à-vis de la spécification des exigences logicielles. En

---

*NLP struggles to understand the context and meaning of words and phrases, especially in aviation-specific jargon. For example, the term « runway » can refer to both the physical runway and the takeoff/landing procedures associated with it. [407]*

283.

*[—] Ellipsis constructions are obviously still challenging for all the common SOTA [State-of-the-art] NLP pipelines and rule-based systems.*

*[—] The fact that specific models trained on the prediction of ellipses in sentences outperform LLMs seems to indicate that the lack of explicit data and pure self-supervised machine learning is not sufficient to handle opaque elements in language, either. Training LLMs on purely overt data ignores significant properties of language. Ellipsis phenomena are grammatical and systematic, and it seems problematic for current LLMs to guess covert continuations.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

2024, une revue de littérature recense 22 articles portant sur l'utilisation de ChatGPT dans ce domaine. Les thèmes abordés par l'analyse de ces articles sont les suivants :

- efficacité dans la génération des exigences (15) ;
- rôle de l'intervention humaine (3) ;
- exploration de l'ingénierie rapide (6) ;
- défis et limites de l'IA (6) ;
- orientations futures de la recherche (15) ;
- expertise humaine comparée (4).

Parmi les articles faisant l'objet de la revue, seuls quatre présentaient un comparatif. L'accès à trois d'entre eux ne permet pas de conclure qu'il y a définitivement un bénéfice à utiliser ChatGPT, puisque les échantillons sont trop faibles [169, 342, 395] :

- Les trois articles arrivent à la conclusion qu'il y aurait un bénéfice à utiliser ChatGPT pour documenter les exigences.
- Les échantillons pour chaque article respectivement, sont : sept étudiants en informatique, dix étudiants en informatique, puis dix professionnels informatiques experts en exigence logiciel.

(voir énoncé p.204)

**Explication l'assertion AD53 : Les contaminations des jeux de données servant à l'entraînement d'un modèle peuvent invalider les essais de celui-ci.**

Les données d'entraînement peuvent contaminer. En 2023, un article met en garde sur la contamination [352] et indique les trois moments où la contamination peut avoir lieu :

- contamination lors du pré-entraînement (*Contamination during pre-training*) ;
- contamination lors du réglage fin supervisé (*Contamination on supervised fine-tuning*) ;
- contamination après déploiement (*Contamination after deployment*).

Les deux conséquences de la contamination sont également soulevées dans l'article :

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

[Traduction libre] Premièrement, les performances d'un GML évalué sur un ensemble de données de référence qu'il a déjà traité lors du préentraînement seront surestimées, ce qui le favorisera par rapport à d'autres GML. Cela affecte l'évaluation comparative de la qualité des GML. Deuxièmement, les articles proposant des hypothèses scientifiques sur certaines tâches de traitement automatique du langage naturel pourraient utiliser des GML contaminés, et ainsi formuler des affirmations erronées concernant leurs hypothèses et invalider des hypothèses alternatives qui pourraient être valides. Cette seconde conséquence a un impact négatif considérable sur notre domaine et constitue notre principal axe de recherche. <sup>284</sup>

[352]

(voir énoncé p.204)

**Explication de l'assertion AD54 : Les répercussions des techniques de TAL commencent à être formalisées et plusieurs sont néfastes.**

En 2020, une revue de littérature portant sur les biais dans le TAL est publiée [31]. Au total, 146 articles y sont analysés. L'auteur affirme que plusieurs articles manquent d'arguments et de connaissances pour étayer leurs propos. Il formule alors trois recommandations visant à améliorer la recherche sur ce sujet :

[Traduction libre]

(R1) Effectuer un travail de fond analysant les "biais" dans les systèmes de TAL à partir de la littérature pertinente, en dehors du domaine du TAL, qui explore les relations entre le langage et les hiérarchies sociales. Considérer les préjudices représentationnels comme nuisibles en soi.

---

284.

*The first one is that the performance of an LLM when evaluated on a benchmark it already processed during pre-training will be overestimated, causing it to be preferred with respect to other LLMs. This affects the comparative assessment of the quality of LLMs. The second is that papers proposing scientific hypotheses on certain NLP tasks could be using contaminated LLMs, and thus make wrong claims about their hypotheses, and invalidate alternative hypotheses that could be true. This second consequence has an enormous negative impact on our field and is our main focus.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

(R2) Fournir des énoncés explicites expliquant pourquoi les comportements du système décrits comme des "biais" sont nuisibles, de quelle manière et pour qui. Assumer clairement le raisonnement normatif (Green, 2019) sous-jacent à ces énoncés.

(R3) Examiner l'usage du langage en pratique en s'appuyant sur les expériences vécues des membres des communautés affectées par les systèmes de TAL. Interroger et repenser les rapports de pouvoir entre les technologues et ces communautés. <sup>285</sup> [31]

En 2021, Laura Weidinger et 22 coauteurs publient un article sur les risques associés aux modèles de langage [398]. Ils y présentent 21 risques préjudiciables, répartis selon des domaines de risque. Un article ultérieur approfondit encore cette analyse.

En 2022, Laura Weidinger et ses collègues de DeepMind proposent un regroupement plus structuré des risques dans un article intitulé *Taxonomy of Risks posed by Language Models* [400]. Leurs regroupements s'appuient sur 35 références, et les auteurs présentent également plusieurs techniques d'atténuation. Voici une partie du tableau sommaire qui se retrouve à l'intérieur de l'article (tableau B.11). Les zones de risques, les mécanismes et les techniques de mitigation demeurent parfois diffus, mais il s'agit néanmoins d'un pas en avant dans la compréhension des risques. Certaines zones de risques ne sont d'ailleurs pas propres aux modèles de langage et peuvent s'appliquer à bien d'autres disciplines.

- risques technologiques, en général : la récupération d'une technologie à des fins malveillantes (4) ou les conséquences de son utilisation pour l'environnement et la société (6) ;

---

285.

(R1) *Ground work analyzing "bias" in NLP systems in the relevant literature outside of NLP that explores the relationships between language and social hierarchies. Treat representational harms as harmful in their own right.*

(R2) *Provide explicit statements of why the system behaviors that are described as "bias" are harmful, in what ways, and to whom. Be forthright about the normative reasoning (Green, 2019) underlying these statements.*

(R3) *Examine language use in practice by engaging with the lived experiences of members of communities affected by NLP systems. Interrogate and reimagine the power relations between technologists and such communities.*

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

- risques en informatique : la fuite d'information (2) et la désinformation préjudiciable (3) ;
- risques à simuler l'humain : la discrimination, les discours haineux et l'exclusion (1) et les interactions personne-machine (5) ;

**TABEAU B.11 – Risques avec les modèles de langage.** [Traduction libre] (modifié de [400])

| Zone à risque                                  | Mécanisme                                                                                                                                                                | Approches d'atténuation techniques                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Discrimination, discours de haine et exclusion | Le modèle de langage reflète avec précision les discours injustes, toxiques et oppressifs présents dans les données de formation.                                        | <ul style="list-style-type: none"> <li>• Données de formation plus représentatives</li> <li>• Données de formation organisées et filtrées</li> <li>• Documentation des ensembles de données</li> <li>• Approches participatives pour détecter les cas de préjudice</li> <li>• Apprentissage en ligne pour la mise à jour du modèle</li> <li>• Modèle de récupération de formation avec un corpus de données distinct</li> <li>• Conception rapide</li> <li>• Recherche d'explicabilité et d'interprétabilité pour identifier les problèmes d'équité</li> </ul>                                                                                                                         |
| Sécurité de l'information                      | Le modèle de langage divulgue ou déduit correctement des informations sensibles.                                                                                         | <ul style="list-style-type: none"> <li>• Outils algorithmiques tels que la confidentialité différentielle;</li> <li>• Stratégies de diffusion responsables.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Désinformation préjudiciable                   | Le modèle de langage fournit des informations fausses, trompeuses, insensées ou de mauvaise qualité.                                                                     | <ul style="list-style-type: none"> <li>• Stratégie de diffusion</li> <li>• Innover sur les méthodes de filtrage des déclarations incorrectes</li> <li>• Entraîner le modèle de récupération avec un corpus de données distinct</li> <li>• Concevoir des modèles de langage pour ne pas fournir de sortie sur des domaines sensibles</li> <li>• Interventions sociotechniques telles que la formation à la véracité via des humains dans la boucle et l'élaboration de normes et d'institutions sur la vérité sur le terrain</li> <li>• Former des modèles de langages qui peuvent rechercher et référencer des sources sur Internet pour étayer des déclarations factuelles</li> </ul> |
| Utilisations malveillantes                     | Les humains utilisent intentionnellement le modèle de langage pour causer du tort.                                                                                       | <ul style="list-style-type: none"> <li>• Limiter l'accès aux modèles de langage et surveiller leur utilisation</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Interaction homme-machine préjudiciable        | Les humains sont trompés ou rendus vulnérables par une interaction directe avec un agent conversationnel puissant.                                                       | <ul style="list-style-type: none"> <li>• Conception de produits plus inclusive</li> <li>• Donner à un assistant une voix non genrée ou multiple</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Dommages environnementaux et socioéconomiques  | Les modèles de langage sont utilisés pour étayer des applications en aval largement utilisées qui profitent et nuisent de manière disproportionnée à différents groupes. | <ul style="list-style-type: none"> <li>• Innovations architecturales telles que le modèle de récupération de formation avec corpus de données séparé</li> <li>• Augmenter l'efficacité de la formation</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

<sup>a</sup> Il a été retiré les colonnes ; *Type of Harm* et *Evidence of this risk in large-scale LMs*

(voir énoncé p.204)

## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

### Explication de l’assertion AD55 : Les techniques d’IA ne remplacent pas les techniques informatiques.

Dans *Artificial Intelligence A Modern Approach* plusieurs chapitres sont consacrés aux différentes techniques permettant d’obtenir un modèle exprimé depuis un langage formel ou un langage non formel. Les auteurs prennent soin d’expliquer les raisons pour lesquelles toutes ces techniques sont réunies dans le même ouvrage :

[Traduction libre] Le lecteur curieux pourrait se demander pourquoi nous avons étudié les grammaires, l’analyse syntaxique et l’interprétation sémantique dans le chapitre précédent, pour ensuite les abandonner au profit de modèles purement axés sur les données dans ce chapitre. Pour l’instant, la réponse est simple : les modèles axés sur les données sont plus faciles à développer et à maintenir, et obtiennent de meilleurs résultats aux tests de performance standard, que les systèmes construits manuellement, avec un effort humain raisonnable, grâce aux approches décrites au chapitre 23. Il se peut que les modèles transformateurs et leurs équivalents apprennent des représentations latentes qui capturent les mêmes idées fondamentales que les grammaires et les informations sémantiques, ou qu’il se passe quelque chose de complètement différent au sein de ces modèles gigantesques ; nous l’ignorons. Nous savons qu’un système entraîné avec des données textuelles est plus facile à maintenir et à adapter à de nouveaux domaines et à de nouvelles langues naturelles qu’un système qui repose sur des caractéristiques conçues manuellement.<sup>286</sup> [351]

---

286.

*The curious reader may wonder why we learned about grammars, parsing, and semantic interpretation in the previous chapter, only to discard those notions in favor of purely datadriven models in this chapter ? At present, the answer is simply that the data-driven models are easier to develop and maintain, and score better on standard benchmarks, compared to the hand-built systems that can be constructed using a reasonable amount of human effort with the approaches described in Chapter 23. It may be that transformer models and their relatives are learning latent representations that capture the same basic ideas as grammars and semantic information, or it may be that something entirely different is happening within these enormous models ; we simply don’t know. We do know that a system that is trained with textual data is easier to maintain and to adapt to new domains and new natural languages than a system that relies on hand-crafted features.*



## B.1. DES DOCUMENTS EN INFORMATIQUE APPLIQUÉE

**TABLEAU B.12** – Avis d’experts en méthodes formelles sur un éventuel remplacement par des méthodes non formelles (source : [378])

| <i>Whether other rising approaches</i>                                                                                                        |                |                     |                       |           |
|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------|---------------------|-----------------------|-----------|
| <i>Do you believe that alternative approaches (e.g. artificial intelligence or quantum computing) will eventually replace formal methods?</i> |                |                     |                       |           |
| Definitely: 0.0%                                                                                                                              | Probably: 3.1% | Probably not: 41.5% | Definitely not: 51.5% | N/A: 3.8% |

En 2020, un article ayant pour titre *The 2020 Expert Survey on Formal Methods*, interroge 130 experts en méthodes formelles [378]. Le tableau B.12 présente les résultats obtenus à la question portant sur l’éventualité que les méthodes formelles soient remplacées :

Le sondage révèle également que la majorité des experts ne voit pas l’IA remplacer les méthodes formelles. Quelques-uns des commentaires de ces experts sont décrits ci-dessous :

[Traduction libre]

[—] Nombre d’entre eux soutiennent que seules les méthodes formelles peuvent garantir l’exactitude des résultats, par exemple : « L’intelligence artificielle se trompe dans 10 à 25 % des cas et doit être paramétrée manuellement. En quoi les méthodes formelles sont-elles pertinentes dans ce cas ? » et « L’intelligence artificielle devra être certifiée. Quelles méthodes seront utilisées pour cette certification ? ».

[—] [...] Deux commentaires, soulignant le rôle crucial des méthodes formelles dans la spécification des exigences, avancent une raison bien différente : « En fin de compte, l’ambiguïté du contexte et sa traduction en un contexte non ambigu sont caractéristiques de l’activité humaine. J’ai du mal à imaginer que ces aspects créatifs puissent être entièrement automatisés. »

[—] D’autres commentaires entrevoient des interactions fructueuses entre les deux domaines. [...] Un autre commentaire ajoute : "Nous avons besoin

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

d’approches combinant techniques basées sur les modèles et techniques  
basées sur les données.<sup>287</sup> [378]  
(voir énoncé p.204)

## B.2 Des modèles en informatique appliquée

### Explication la définition M01 : Modèle

La définition formulée est : représentation d’un sujet pour en permettre le traitement.  
La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de modèle : « Représentation simplifiée, souvent formalisée, d’un processus, d’un système. » [330] ;
- définition 2 de modèle : [Traduction libre] « Pour un observateur O, un modèle M et un objet A, O peut utiliser M pour répondre à des questions sur A. »<sup>288</sup> [390] ;
- description 1 de modèle : un modèle est exprimé dans un langage artificiel ou naturel. Il modélise un sujet d’étude et incarne un point de vue. Ce point de vue est porté sur le même sujet d’étude. (description d’une figure) [49] ;
- description 2 de modèle : [Traduction libre] « Le langage utilisé pour concevoir le modèle. Nous rappelons aux lecteurs la différence entre langage et métalangage. Langage et logique sont indissociables : nous utilisons le langage pour désigner

---

287.

[—] *Many of them argue that only formal methods can provide guarantees about correctness, e.g. « artificial intelligence is wrong in 10 to 25% of the cases and must be hand-tuned. What is formal there ? » and « artificial intelligence will need to be certified. What methods will be used to certify it ? ».*

[—] *[...] two comments that praise the crucial role of formal methods in requirements specification : « at the end of the day, both the ambiguous setting and the mapping to the unambiguous setting are characteristic of human activities. I have a hard time imagining that these creative aspects can be fully automated ».*

[—] *Other comments see fruitful interactions between both fields. Another comment adds : « we need approaches that combine model-driven and data-driven techniques ».*

288. « To an observer B, an object  $A^*$  is a model of an object A to the extent that B can use  $A^*$  to answer questions that interest him about A. »

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

des éléments, des événements et des choses, et le métalangage pour désigner leurs noms. »<sup>289</sup> [389].

La définition 1 du Robert fait référence au fait que le modèle peut être souvent formel, tandis que la définition 2 de Marvin Minsky provenant de l'ouvrage de Velten ne fait pas référence au terme **formel**. La description 1 de Guy Caplat ajoute à la description du terme **modèle** la relation entre le modèle et le langage.

(voir énoncé p.205)

### Explication de la définition OM01 : Système logique

La définition formulée est : est composé d'un langage formel et d'un ensemble de règles (axiome, proposition, logique) permettant d'inférer.

La littérature fournit ces informations qui ont servi à construire cette définition :

- description d'un système logique : [Traduction libre] « Les trois principaux composants de tout système logique sont : 1. Un langage formel 2. Une relation de conséquence 3. Applicabilité par le biais d'une notion de schéma. »<sup>290</sup> [95] ;
- définition d'un système logique : « Un système logique est un système formel dédié au raisonnement et aux déductions logiques. »<sup>291</sup> ;
- définition de système formel selon C. J. Date : « *A logical system, q.v* » [68] ;
- définition 1 de système formel : [Traduction libre] « Un système formel se compose de, un langage formel, un ensemble d'axiomes, des règles d'inférence. »<sup>292</sup> [286]
- définition 2 d'un système formel : [Traduction libre] « Un système formel S est un langage formel L muni d'un appareil déductif défini par : (1) l'établissement formel que certaines formules de L sont des axiomes de S et/ou (2) l'établissement formel d'un ensemble de règles de transformation (également appelées

---

289. « *The Language Used to Design the Model. We remind readers of the difference that exists between language and metalanguage. Language and logic are inextricably related : we use language to refer to items, events, and things and metalanguage to refer to their names.* »

290. « *The three main components of any logical system are : 1. A formal language 2. A consequence relation 3. Applicability through a notion of schematic substitution.* »

291. Wikipedia, « Système logique », [https://fr.wikipedia.org/wiki/Syst%C3%A8me\\_logique](https://fr.wikipedia.org/wiki/Syst%C3%A8me_logique) (consulté le 2025-11-15).

292. « *A formal system consists of, - A formal language, - A set of axioms, - Rules of inference.* »

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

règles d'inférence) qui déterminent quelles relations entre les formules de  $L$  sont des relations de conséquence immédiate dans  $S$ . (Intuitivement, les règles de transformation autorisent la dérivation de certaines formules à partir d'autres.) [...] L'appareil déductif doit être définissable sans référence à une quelconque interprétation du langage ; sinon, le système n'est pas un système formel. »<sup>293</sup> [125].

(voir énoncé p.205)

### Explication de la définition OM02 : Modèle formel

La définition formulée est : est composé d'un langage formel et d'un ensemble de règles et vise la représentation d'un sujet pour en permettre le traitement.

La littérature fournit ces informations qui ont servi à construire cette définition :

- description 1 de modèle formel : [Traduction libre] « Lors de la modélisation d'un système, nous structurons le modèle formel de telle sorte que les parties constantes et les parties variables soient conservées dans des entités, des contextes et des machines distincts, respectivement. »<sup>294</sup> [3] ;
- description 2 de modèle formel : [Traduction libre] « Un modèle peut contenir uniquement des contextes, uniquement des machines, ou les deux. Dans le premier cas, le modèle représente une structure mathématique pure avec des ensembles, des constantes, des axiomes et des théorèmes. »<sup>295</sup> [2] ;
- description 3 de modèle formel : [Traduction libre] « Formel signifie ici que le modèle est représenté à l'aide d'un langage formel dont la syntaxe et la

---

293. « A formal system  $S$  is a formal language  $L$  together with a deductive apparatus given by : (1) laying down by fiat that certain formulas of  $L$  are to be axioms of  $S$  and/or (2) laying down by fiat a set of transformation rules (also called rules of inference) that determines which relations between formulas of  $L$  are relations of immediate consequence in  $S$ . (Intuitively, the transformation rules license the derivation of some formulas from others.) [...] The deductive apparatus must be definable without reference to any intended interpretation of the language : otherwise the system is not a formal system. »

294. « When modeling a system we structure the formal model such that constant parts and variable parts are kept in distinct entities, contexts and machines respectively. »

295. « A model can contain contexts only, or machines only, or both. In the first case, the model represents a pure mathematical structure with sets, constants, axioms, and theorems. »

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

sémantique sont si clairement définies que le modèle peut être exécuté sur un ordinateur. » <sup>296</sup>[69].

(voir énoncé p.205)

### Explication de la définition OM03 : Machine abstraite

La définition formulée est : système formel comprenant les instructions nécessaires au traitement automatique d'information.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition de machine abstraite : [Traduction libre] « Une machine abstraite est un modèle de système informatique (considéré comme matériel ou logiciel) conçu pour permettre une analyse détaillée et précise de son fonctionnement. Ce modèle comprend généralement des entrées, des sorties et les opérations réalisables (l'ensemble des opérations), et peut donc être assimilé à un processeur. Les machines de Turing sont les machines abstraites les plus connues [...]. » <sup>297</sup>[83] ;
- définition d'ordinateur : « Machine automatique de traitement de l'information, obéissant à des programmes formés par des suites d'opérations arithmétiques et logiques. » [189] ;
- définition de donnée : « Représentation conventionnelle d'une information en vue de son traitement informatique. » [199].

(voir énoncé p.205)

### Explication de l'assertion OM04 : L'utilisation des systèmes logiques peut varier tout comme les logiques employées.

---

296. « *Formal means here that the model is represented using a formal language with so clearly defined syntax and semantics that the model can be executed using a computer.* »

297. « *An abstract machine is a model of a computer system (considered either as hardware or software) constructed to allow a detailed and precise analysis of how the computer system works. Such a model usually consists of input, output, and operations that can be preformed (the operation set), and so can be thought of as a processor. Turing machines are the best known abstract machines [...].* »

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

Dale Miller décrit l'utilisation de systèmes logiques pour le calcul à travers l'article *Representing and Reasoning with Operational Semantics* [94]. D'abord, il catégorise deux approches :

[Traduction libre]

- calcul en tant que modèle : [...] les calculs sont encodés sous forme de structures mathématiques, contenant des éléments, tels que des nœuds, des transitions et des états. La logique est utilisée de manière externe pour formuler des énoncés sur ces structures. Autrement dit, les calculs servent de modèles pour les expressions logiques. Les opérateurs intensionnels, tels que les modaux des logiques temporelles et dynamiques ou les triplets de la logique de Hoare, sont souvent utilisés pour exprimer des propositions sur le changement d'état. Cette utilisation de la logique pour représenter et raisonner sur le calcul est probablement l'utilisation la plus ancienne et la plus répandue de la logique pour spécifier le calcul.
- calcul-comme-déduction : [...] utilise directement des éléments de la syntaxe logique (formules, termes, types et preuves) comme éléments du calcul spécifié. <sup>298</sup> [94]

Par la suite, l'auteur décrit sommairement le fonctionnement pour *computation-as-model* :

[Traduction libre]

---

298.

- *computation-as-model* : [...] *computations are encoded as mathematical structures, containing such items as nodes, transitions, and state. Logic is used in an external sense to make statements about those structures. That is, computations are used as models for logical expressions. Intensional operators, such as the modals of temporal and dynamic logics or the triples of Hoare logic, are often employed to express propositions about the change in state. This use of logic to represent and reason about computation is probably the oldest and most broadly successful use of logic for specifying computation.*
- *computation-as-deduction* : [...] *uses pieces of logic's syntax (formulas, terms, types, and proofs) directly as elements of the specified computation.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

- 1- [...] les mathématiques sont implémentées par une théorie des ensembles ou une logique d'ordre supérieur (par exemple, HOL [14], Isabelle/ZF [46], PVS [44]).
- 2- [...] on ramène les problèmes de correction des programmes aux mathématiques. Ainsi, les structures de données, les états, les piles, les files, les invariants, etc., sont toutes représentés comme différents types d'objets mathématiques.
- 3- On raisonne ensuite directement sur ces objets en utilisant des techniques mathématiques standards (induction, récursion primitive, points fixes, ordres bien fondés, etc.). <sup>299</sup> [94]

(voir énoncé p.205)

**Explication de l'assertion OM05 : L'informatique a développé plusieurs couches d'abstractions où sont utilisé différentes machines abstraites.**

Dans l'ouvrage *Foundations of computer science*, les auteurs Alfred V. Aho et Ullman D. Jeffrey énumèrent sept niveaux d'abstractions :

[Traduction libre]

6. Programme d'application : [...] chacune avec son propre modèle de données, conçu pour résoudre des problèmes spécifiques dans un large éventail de disciplines.
5. Langage de programmation : des langages de haut niveau permettant aux programmeurs d'applications de résoudre les problèmes plus facilement qu'avec le langage assembleur.

---

299.

- 1- [...] one implements mathematics via some set-theory or higher-order logic (for example, HOL [14], Isabelle/ZF [46], PVS [44]).
- 2- [...] one reduces program correctness problems to mathematics. Thus, data structures, states, stacks, heaps, invariants, etc, all are represented as various kinds of mathematical objects.
- 3- One then reasons directly on these objects using standard mathematical techniques (induction, primitive recursion, fixed points, well-founded orders, etc).

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

4. Langage assembleur : [...] est une représentation symbolique des instructions que l'on trouve aux niveaux inférieurs.
3. Noyau du système d'exploitation : Le noyau du système d'exploitation lui-même peut être implémenté dans un langage de haut niveau qui a été traduit en langage machine.
2. Langage machine : [...] peuvent, par exemple, additionner deux nombres, déplacer des données d'un emplacement à un autre ou déterminer si un nombre est égal à zéro. Elles suffisent à exécuter n'importe quel programme ou application de haut niveau.
1. Microprogramme : Tous les ordinateurs ne possèdent pas ce niveau, mais pour ceux qui le possèdent, il s'agit d'un ensemble d'étapes permettant d'implémenter les instructions en langage machine.
0. Logique numérique : [...] est une abstraction des circuits électroniques d'un ordinateur. <sup>300</sup> [8]

(voir énoncé p.205)

**Explication de l'assertion OM06 : L'utilisation d'une mémoire réinscriptible permet d'obtenir des machines abstraites.**

---

300.

6. *Application program : [...] each with its own data model — designed to solve specific problems in a broad range of disciplines.*
5. *Programming language : [...] higher-level languages with which applications programmers can solve problems more readily than with assembly language*
4. *Assembly language : [...] is a symbolic representation for instructions found at the lower levels.*
3. *Operating system kernel : The operating system kernel itself may be implemented in a higher-level language that has been translated into machine language.*
2. *Machine language : [...] may add two numbers, move data from one location to another, or determine whether a number is equal to zero, for instance. Primitive instructions such as these are sufficient to execute any program or application at the higher levels.*
1. *Microprogram : Not all computers have this level, but for those that do it is a collection of steps used to implement the machine language instructions.*
0. *Digital logic : [...] is an abstraction of the electronic circuitry of a computer.*



## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

Ce qui permet autant de niveaux d'abstractions est la caractéristique d'une machine à se modifier. Thomas Haigh cite Swade à ce sujet :

[Traduction libre] Swade lui-même a renoncé à l'aveu de confusion, aussi touchant qu'audacieux, que nous avons cité précédemment, pour conclure que « le programme interne stocké [...] est la réalisation pratique de l'universalité de Turing » et conférait ainsi « la plasticité de la fonction, qui explique en grande partie la remarquable prolifération des ordinateurs et des artefacts de type informatique ». <sup>301</sup> [112]

Dans le même article, l'auteur soulève des raisons pour lesquelles cette caractéristique ne peut pas être attribuable à John von Neumann :

[Traduction libre]

[—] Dans le « Premier brouillon d'un rapport sur l'EDVAC » [...] le mot « programme » n'apparaît jamais. Von Neumann préférait systématiquement « code » à « programme » et parlait de « mémoire » plutôt que de « stockage ».

[—] Von Neumann a délibérément éliminé la possibilité qu'un programme s'écrase lui-même. <sup>302</sup> [112]

Dans un deuxième article, Thomas Haigh précise quel fut le premier ordinateur à présenter cette caractéristique, remontant la trace jusqu'à la fin des années 1950 :

[Traduction libre] [...] « on attribue à l'ENIAC le titre de premier ordinateur à fonctionner avec un programme stocké en lecture seule », mais

---

301.

*Swade himself retreated from the endearingly bold confession of confusion we quoted earlier to conclude that “the internal stored program [...] is the practical realization of Turing universality” and thus conferred “plasticity of function, which in large part accounts for the remarkable proliferation of computers and computer-like artifacts.”*

302.

*[—] The “First Draft of a Report on the EDVAC” [...] the word “program” never appears in the draft. Von Neumann consistently preferred “code” to “program” and wrote of “memory” rather than “storage.”*

*[—] Von Neumann deliberately eliminated the possibility of a program overwriting itself.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

au « BINAC et à l'EDSAC le titre de premiers ordinateurs à fonctionner avec un programme stocké modifiable dynamiquement. »<sup>303</sup> [113]

(voir énoncé p.205)

**Explication de l'assertion OM07 : L'informatique appliquée crée des modèles de problèmes et des modèles de solutions, puis cherche à les associer.**

Un problème à résoudre peut être modélisé. Dans l'ouvrage *Foundations of computer science*, les auteurs Alfred V. Aho et Jeffrey D. Ullman soulignent l'importance de cette modélisation en informatique :

[Traduction libre] Mais fondamentalement, l'informatique est une science de l'abstraction — créer le bon modèle pour un problème et concevoir les techniques mécanisables appropriées pour le résoudre.<sup>304</sup> [8]

Une solution qui permet de résoudre un problème peut se modéliser. Un article offre un survol historique des modèles et de l'ingénierie des systèmes basée sur des modèles (MBSE, *Model Based Systems Engineering* en anglais). Il présente successivement les concepts suivants :

[Traduction libre]

[—] Théorie des modèles de Tarski : La théorie des modèles du premier ordre s'intéresse aux relations entre les descriptions de systèmes formulées dans un langage formel du premier ordre (tel que le calcul des prédicats de la logique mathématique) et les structures qui satisfont ces descriptions.

[—] Analyse et conception structurées de Yourdon : Yourdon [11] a introduit une approche de développement logiciel basée sur les modèles, selon

---

303.

[...] "*credit the ENIAC as the first computer to be run with a read-only stored program*" but the "*BINAC and the EDSAC as being the first computers to be run with a dynamically modifiable stored program*".

304.

*But fundamentally, computer science is a science of abstraction — creating the right model for a problem and devising the appropriate mechanizable techniques to solve it.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

une perspective comportementale. Il a également introduit un langage de modélisation graphique appelé diagrammes de flux de données pour étayer son approche.

[—] Ingénierie des systèmes basée sur les modèles de Wymore : A. Wayne Wymore [12] fut l'un des pionniers de l'ingénierie des systèmes basée sur les modèles. Il adoptait une perspective comportementale du système, dans laquelle le modèle de comportement est composé du nom du système, de ses états, des entrées et sorti de ces états, et des transitions d'état suivantes.

[—] Méthodologies des systèmes généraux de Klir et Lin : Le concept d'homomorphisme de Wymore a été généralisé par Lin [7] dans son concept de système général, en utilisant une définition algébrique de l'homomorphisme.

[—] Conception axiomatique de N. P. Suh : Dans les années 1990, Suh a publié de nombreux travaux sur ce qu'il appelait la conception axiomatique. La référence [14] propose une introduction concise à son approche, illustrée par des exemples. Deux axiomes de conception sont à considérer : 1) maximiser l'indépendance fonctionnelle en découplant les éléments fonctionnels, et 2) minimiser la quantité d'information contenue dans la conception. <sup>305</sup> [75]

---

305.

[—] *Tarski Model Theory* : First-order model theory is concerned with the relationships between system descriptions made in a first-order formal language (such as the predicate calculus of mathematical logic) and the structures that satisfy these descriptions.

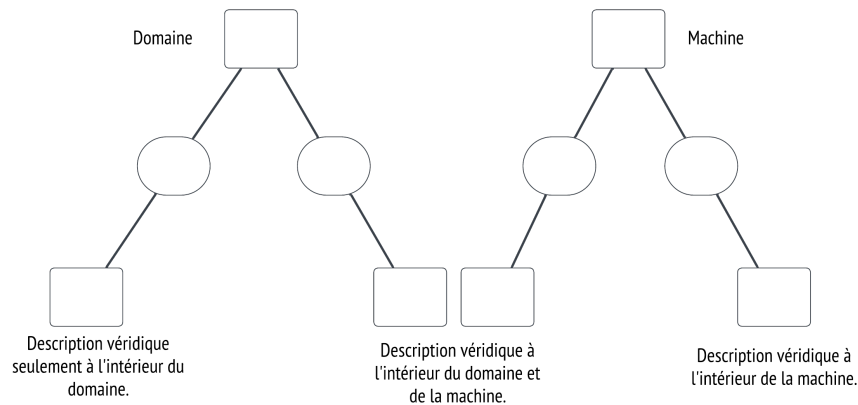
[—] *Yourdon Structured Analysis and Design* : Yourdon [11] introduced a model-based approach for software development from a behavioral viewpoint. He also introduced a graphical modeling language called data flow diagrams to support his approach.

[—] *Wymore MBSE* : A. Wayne Wymore [12] was one of the early engineering pioneers in the domain of model-based systems engineering. He had a behavioral viewpoint on system in which the model of behavior is comprised of the name of the system, its states, the inputs and outputs of those states, and next state transitions.

[—] *Klir and Lin General Systems Methodologies* : Wymore's concept of homomorphism was generalized by Lin [7] in his concept of a general system by using an algebraic definition of homomorphism.

[—] *N. P. Suh Axiomatic Design* : During the 1990s, Suh published an extensive body of literature on what he called axiomatic design. Reference [14] offers a concise introduction to

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.24** – Diagramme représentant l’association entre deux modèles [Traduction libre] (source : [147])

L’association entre un modèle de problème et un modèle de solution est décrite dans l’ouvrage *Software requirements & specifications* de Michael Jackson. Il décrit les termes **domaine** et **machine** ainsi :

[Traduction libre] Cette distinction entre la machine et le domaine d’application est essentielle à la compréhension de la différence, souvent citée, mais souvent mal comprise, entre le *quoi* et le *comment* : ce que fait le système se trouve dans le domaine d’application, tandis que le *comment* il le fait se trouve dans la machine elle-même. Le problème réside dans le domaine d’application. La machine est la solution.<sup>306</sup> [147]

Il illustre l’activité et la modélisation avec une figure (figure B.24) et ajoute « *The picture is easy to remember : M is for Modelling.* ».

(voir énoncé p.205)

---

*his approach to include examples. There are two design axioms : 1) maximize functional independence by decoupling functional elements, and 2) minimize the information content of the design.*

306.

*This distinction between the machine and the application domain is the key to the much-cited (but little understood) distinction between What and How : what the system does is to be sought in the application domain, while how it does it is to be sought in the machine itself. The problem is in the application domain. The machine is the solution.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

**Explication de l’axiome OM08 : Les modèles en science ont, entre autres, comme objectifs d’apprendre, de développer et d’appliquer les connaissances.**

Dans le recueil de Upmeier zu Belzen, les modèles sont décrits comme le produit principal de la science et se retrouvent dans toutes les disciplines [387]. Trois objectifs principaux sont décrits :

[Traduction libre]

- Apprendre les sciences — acquérir et développer des connaissances conceptuelles et théoriques.
- Apprendre les sciences — développer une compréhension des caractéristiques de la démarche scientifique, du rôle et du statut des connaissances qu’elle génère [...].
- Faire de la science — s’engager dans la recherche scientifique et la résolution de problèmes et développer une expertise en la matière. <sup>307</sup> [387]

À l’intérieur du *curriculum* de l’Association for Computing Machinery (ACM) pour l’ingénierie logiciel de 2014, l’ingénierie est placée comme l’utilisateur des modèles produits par la science, voici ce qui est décrit :

[Traduction libre] Alors que les scientifiques observent et étudient les comportements existants puis élaborent des modèles pour les décrire, les ingénieurs utilisent ces modèles comme point de départ pour concevoir et développer des technologies permettant de nouvelles formes de comportement. <sup>308</sup> [381]

---

307.

- *Learning science—acquiring and developing conceptual and theoretical knowledge.*
- *Learning about science—developing an understanding of the characteristics of scientific inquiry, the role and status of the knowledge it generates [...].*
- *Doing science—engaging in and developing expertise in scientific inquiry and problem-solving.*

308.

*Whereas scientists observe and study existing behaviors and then develop models to describe them, engineers use such models as a starting point for designing and developing technologies that enable new forms of behavior.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

(voir énoncé p.206)

**Explication de l’assertion EM01 : Pour réussir une modélisation, il faut considérer, entre autres, la compréhensibilité, la couverture, la cohérence et la complétude.**

Dans la version 4 du *Software Engineering Body of Knowledge* SWEBOK, l’analyse des modèles comprend :

[Traduction libre] [...] les techniques d’analyse courantes utilisées en modélisation pour vérifier la complétude, la cohérence, l’adéquation, la traçabilité et l’interaction.<sup>309</sup> [396]

Des caractéristiques de qualité d’un modèle sont recensées dans l’ouvrage *Conceptual Modeling of Information Systems* de Antoni Olivé. On y retrouve :

[Traduction libre]

- complet : [...] le schéma conceptuel doit inclure toutes les connaissances requises [...]. ;
- correct : [...] si les connaissances qu’il définit sont exactes pour le domaine et pertinentes pour les fonctions que le système doit exécuter. ;
- correcte syntaxiquement : [...] s’il respecte toutes les règles du langage dans lequel il est écrit. ;
- compréhensible : Tous les membres du public concerné devraient être en mesure de comprendre correctement la partie du schéma conceptuel qui les concerne. ;
- adaptable : [...] si des modifications mineures aux propriétés du domaine ou aux exigences des utilisateurs n’entraînent pas de modifications majeures du schéma.<sup>310</sup> [233]

---

309.

[...] *analysis techniques used in modeling to verify completeness, consistency, correctness, traceability and interaction.*

310.

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

John Krogstie aborde la qualité sur la modélisation dans *Quality in Business Process Modeling* [165]. Dans le chapitre 2, il présente les caractéristiques de qualité concernant la modélisation proposées par différents auteurs et appliquées à divers artefacts. Pour ce qui concerne un document de spécification des exigences logicielles, il se sert d'un article écrit par Alan M. Davis [70], les caractéristiques de qualité doivent être :

[Traduction libre]

[—] compréhensible : [...] si tous les types de lecteurs de spécifications fonctionnelles peuvent facilement comprendre la signification de toutes les exigences avec un minimum d'explications.

[—] sans ambiguïté : [...] si et seulement si chaque exigence énoncée n'a qu'une seule interprétation possible.

[—] organisé : [...] si et seulement si son contenu est agencé de manière à ce que les lecteurs puissent facilement localiser l'information et que les relations logiques entre les sections adjacentes soient apparentes.

[—] complet : [...] si : 1. Tout ce que le logiciel est censé faire est inclus dans les spécifications fonctionnelles. 2. Les réponses du logiciel à toutes les classes de données d'entrée réalisables sont décrites dans [...].

[—] concis : [...] s'il est aussi court que possible sans affecter aucune autre qualité des spécifications fonctionnelles.

[—] correct : [...] si et seulement si chaque exigence représente une exigence du système à construire.

[—] vérifiable : [...] s'il existe des techniques finies et rentables permettant de vérifier que chaque exigence énoncée est satisfaite par le système à construire. On utilise souvent le terme testable pour désigner ce type d'exigence.

- 
- *complete* : [...] *the conceptual schema must include all the required knowledge [...]. ;*
  - *correct* : [...] *if the knowledge that it defines is true for the domain and relevant to the functions that the system must perform. ;*
  - *syntactically correct* : [...] *if it respects all the rules of the language in which it is written. ;*
  - *understandable* : *All members of the relevant audience should be able to correctly understand the part of the conceptual schema that is relevant to them. ;*
  - *stability* : [...] *if minor changes in the properties of the domain or in the users' requirements do not entail major changes in the schema.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

[—] cohérent à l’interne : [...] si et seulement si aucune exigence individuelle énoncée dans le document n’entre en conflit avec une autre exigence. Davis suggère d’utiliser des langages dotés d’une syntaxe et d’une sémantique formelles afin de détecter et de supprimer les incohérences.

[—] cohérent à l’externet : [...] si et seulement si aucune exigence énoncée dans le document n’est en conflit avec la documentation de référence du projet ou de l’organisation.

[—] exécutable/interprétable/prototypable : [...] si et seulement s’il existe un outil logiciel capable de fournir un modèle comportemental dynamique basé sur (les parties pertinentes) du cahier des charges fonctionnel.

[—] réalisable : [...] si et seulement s’il existe au moins une conception et une implémentation système qui mettent correctement en œuvre toutes les exigences énoncées dans le cahier des charges fonctionnel. <sup>311</sup> [165]

Pour compléter cette liste, il y a également les caractéristiques de qualité suivantes : *design-independent, traceable, modifiable, electronically stored, annotated by relative*

---

311.

[—] *understandable* : [...] *if all types of SRS readers can easily comprehend the meaning of all of the requirements with a minimum of explanation.*

[—] *unambiguous* : [...] *if and only if every requirement stated therein has only one possible interpretation.*

[—] *organized* : [...] *if and only if its contents are arranged so that readers can easily locate information and logical relationships among adjacent sections are apparent.*

[—] *complete* : [...] *if : 1. Everything that the software is supposed to do is included in the SRS. 2. Responses of the software to all realizable classes of input data in [...].*

[—] *concise* : [...] *if it is as short as possible without affecting any other quality of the SRS.*

[—] *correct* : [...] *if and only if every requirement represents something required of the system to be built.*

[—] *verifiable* : [...] *if there are finite, cost-effective techniques that can be used to verify that every requirement stated therein is satisfied by the system to be built. Testable is another term often found for this.*

[—] *internally consistent* : [...] *if and only if no subset of individual requirements stated therein conflicts. Davis suggests using languages with formal syntax and semantics to be able to detect and remove inconsistency.*

[—] *externally consistent* : [...] *if and only if no requirement stated therein conflicts with any already baselined project or organizational documentation.*

[—] *executable/interpretable/prototypable* : [...] *if and only if there exists a software tool that is capable of providing a dynamic behavioral model based on (relevant parts of) the SRS.*

[—] *achievable* : [...] *if and only if there could exist at least one system design and implementation that correctly implements all of the requirements stated in the SRS.*



## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

*importance, annotated by relative stability, annotated by version, not redundant, at right level of detail, precise, reusable, traced, cross-referenced.*

(voir énoncé p.206)

### Explication la définition EM02 : Cohérent

La définition formulée est : système logique où aucune contradiction ne s'est encore manifestée.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de consistance : « Propriété d'un système logique caractérisé par l'impossibilité d'y trouver des contradictions. » [245] ;
- définition 2 de consistance : [Traduction libre] « Un système formel est cohérent s'il n'existe aucune formule  $A$  telle que  $A$  et  $\neg A$  soient démontrables dans le système (c'est-à-dire qu'il n'y a pas de contradictions dans le système). » <sup>312</sup> [286] ;
- définition de cohérence : « Propriété d'un système logique caractérisé par une absence, apparente ou réelle, de contradictions. » [243] ;
- note sur la définition de cohérence : « Ainsi, un système consistant est obligatoirement cohérent alors que l'inverse n'est pas nécessairement vrai. » [243].

Consistance et cohérence peuvent se confondre. Cependant, en informatique, il peut exister des contradictions qui vont se manifester lors de l'exécution du logiciel. Alors, il est plus juste d'utiliser la cohérence comme caractéristique de qualité.

(voir énoncé p.206)

### Explication la définition EM03 : Couverture

La définition formulée est : consiste à couvrir tous les éléments d'un ensemble avec une sous-famille d'un autre ensemble, et ce, la plus petite possible.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition du problème de couverture en mathématique : « Étant donné un ensemble  $A$ , on dit qu'un élément  $e$  est couvert par  $A$  si  $e$  appartient à  $A$ . Étant

---

312. « *A formal system is consistent if there is no formula  $A$  such that  $A$  and  $\neg A$  are provable in the system (i.e. there are no contradictions in the system).* »

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

donné un ensemble  $U$  et une famille  $S$  de sous-ensembles de  $U$ , le problème consiste à couvrir tous les éléments  $U$  avec une sous-famille de  $S$  la plus petite possible. » <sup>313</sup> ;

- définition de couverture : « Étendue de la zone où un service donné fonctionne correctement. » [79] ;
- définition d'exhaustif : « Qui traite complètement, épuise un sujet. » [326] ;
- définition de complet : « Qui comporte tous les éléments nécessaires, à quoi rien ne manque. » [178] ;
- définition d'adéquat : « Qui correspond parfaitement à son objet ; approprié, adapté. » [172] ;
- définition d'adéquation : « Conformité à l'objet, au but qu'on se propose. » [173].

La couverture peut être :

- celle d'un modèle par rapport au sujet ;
- celle d'un modèle par rapport à la théorie ;
- celle d'un artefact par rapport à un modèle ;

Les termes **adéquat** et **exhaustif** ont une signification plus catégorique qui aurait comme effet d'influencer toute mesure objective prise pour catégoriser des modèles. Tandis que le terme **couverture** est plus neutre.

(voir énoncé p.206)

### Explication la définition EM04 : Complétude

La définition formulée est : un système formel est complet lorsque toutes les vérités qu'il contient peuvent être déduites des axiomes et des règles d'inférence.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de complétude : « Propriété d'une théorie déductive consistante où toute formule est décidable. » ; [179]

---

313. Wikipedia, « Problème de couverture par ensembles », [https://fr.wikipedia.org/wiki/Probl%C3%A8me\\_de\\_couverture\\_par\\_ensembles](https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_couverture_par_ensembles) (consulté le 2026-08-15).

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

- définition 2 de complétude : [Traduction libre] « Un système formel est complet si toutes les vérités qu’il contient peuvent être déduites des axiomes et des règles d’inférence. »<sup>314</sup> [286].

(voir énoncé p.206)

### Explication la définition EM05 : Compréhensible

La définition formulée est : qui peut être intelligible par le lectorat.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de compréhensible : [Traduction libre] « Tous les membres du public concerné devraient être en mesure de comprendre correctement la partie du schéma conceptuel qui les concerne. »<sup>315</sup> [233]
- définition 2 de compréhensible : [Traduction libre] « Compréhensible : [...] si tous les types de lecteurs de spécifications fonctionnelles peuvent facilement comprendre la signification de toutes les exigences avec un minimum d’explications. »<sup>316</sup> [165] ;
- définition 3 de compréhensible : « Qui peut être compris, intelligible, clair. » [181].

(voir énoncé p.206)

### Explication la définition EM06 : Modèle conceptuel

La définition formulée est : utilise des concepts pour représenter le sujet.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de modèle conceptuel : [Traduction libre] « modèle des concepts pertinents pour un sujet intérêt donné »<sup>317</sup> [140] ;

---

314. « *A formal system is complete if all the truths in the system can be derived from the axioms and rules of inference.* »

315. « *All members of the relevant audience should be able to correctly understand the part of the conceptual schema that is relevant to them.* »

316. « *Understandable : [...] if all types of SRS readers can easily comprehend the meaning of all of the requirements with a minimum of explanation.* »

317. « *a model of the concepts relevant to some endeavor* »

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

- définition 2 de modèle conceptuel : [Traduction libre] « Les modèles conceptuels sont utilisés soit comme représentation intermédiaire, soit comme représentation directe dans le processus de développement et d'évolution des systèmes d'information. »<sup>318</sup> [45] ;
- description de la définition 2 de modèle conceptuel : [Traduction libre] « En principe, un même modèle conceptuel peut être appliqué à de nombreux domaines différents, et plusieurs modèles conceptuels peuvent être appliqués à un même domaine. »<sup>319</sup> [233].

(voir énoncé p.206)

**Explication de l'assertion EM07 : L'utilisation de l'UML pour exprimer un modèle conceptuel est encouragée par plusieurs auteurs.**

En 2007, dans l'ouvrage *Conceptual Modeling of Information Systems*, l'auteur Antoni Olivé indique que le schéma conceptuel et le modèle conceptuel peuvent se confondre. Il l'illustre ainsi :

- *Conceptual model = Conceptual schema* ;
- *Conceptual model = Conceptual schema + Information base.*

Il précise :

[Traduction libre] En modélisation conceptuelle, les connaissances générales sur un domaine sont représentées dans le schéma conceptuel, tandis que les faits simples sont représentés dans la base d'informations.<sup>320</sup> [233]

Plus loin, il affirme que la logique de premier ordre est suffisante dans les modèles conceptuels pour la plupart des cas et, pour les autres cas, il suggère l'UML :

---

318. « *Conceptual models are either used as an intermediate or directly used representation in the process of development and evolution of information systems.* »

319. « *In principle, the same conceptual model can be applied to many different domains, and several conceptual models can be applied to the same domain.* »

320.

*In conceptual modeling, the general knowledge about a domain is represented in the conceptual schema, while simple facts are represented in the information base.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

[Traduction libre] Le fondement formel des langages de modélisation conceptuelle repose sur la logique. Tout schéma conceptuel peut être spécifié à l'aide d'un langage logique. En particulier, la logique du premier ordre est suffisante pour la spécification de la plupart des schémas conceptuels. Dans les exemples présentés dans ce chapitre introductif, nous utilisons exclusivement la logique du premier ordre. Cependant, dans de nombreux projets, l'utilisation des langages logiques s'avère impraticable et des langages spécialisés sont plus appropriés. Le langage de modélisation unifié (UML) est l'un de ces langages. Dans cet ouvrage, nous expliquerons en détail l'utilisation d'UML en tant que langage de modélisation conceptuelle.<sup>321</sup> [233]

À l'intérieur du *Body of Knowledge for Modeling and Simulation : A Handbook by the Society for Modeling and Simulation International*, il est indiqué que l'UML a un formalisme adéquat :

[Traduction libre] Un modèle conceptuel décrit, au plus haut niveau d'abstraction, les connaissances générales relatives au système étudié. [...] UML et le langage naturel sont des exemples de formalisme adéquat pour ce niveau d'abstraction.<sup>322</sup> [283]

(voir énoncé p.206)

---

321.

*The formal basis of conceptual modeling languages is logic. Any conceptual schema can be specified in some kind of logical language. In particular, the first-order logic (FOL) language is sufficient for the specification of most conceptual schemas. In the examples given in this introductory chapter, we use the FOL language only. However, in many projects the use of logical languages is impractical, and specialized languages are more suitable. One such language is the Unified Modeling Language (UML). In this book, we shall explain in detail the use of UML as a conceptual modeling language.*

322.

*A conceptual data model describe, at the highest level of abstraction, the general knowledge about the system under study. [...] UML and natural language are examples of adequate formalism for this abstraction level.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

### Explication de l’assertion EM08 : L’UML est un langage de modélisation non formel.

L’ouvrage *UML in a nutshell* réaffirme que l’UML sert à la modélisation :

[Traduction libre] UML n’est pas un langage de programmation visuel, mais un langage de modélisation visuelle.<sup>323</sup> [11]

L’ouvrage *Meta-Programming and Model-Driven Meta-Program Development : Principles, Processes and Techniques* explique à quel point l’UML est insuffisant pour modéliser :

[Traduction libre] En revanche, les notations MDD, telles que UML, ne permettent pas de modéliser la variabilité et les familles de logiciels (gammes de produits). Les efforts déployés pour combler cette lacune comprennent l’extension du méta-modèle UML afin d’y inclure des fonctionnalités pour la modélisation de la variabilité [ZJ06] ou l’utilisation conjointe de modèles UML et de modèles de fonctionnalités pour la modélisation d’un domaine [...].<sup>324</sup> [373]

Le SysML est un exemple d’extension pour pallier certains problèmes d’UML :

[Traduction libre] SysML ajoute de nouveaux diagrammes et constructions qui ne se trouvent pas dans UML : le diagramme paramétrique, le diagramme d’exigences, les fonctionnalités requises et fournies et les propriétés de flux.<sup>325</sup> [120]

---

323.

*The UML is not : A visual programming language, but a visual modeling language.*

324.

*On the other hand, the MDD notations such as UML lack capabilities for modelling variability and software families (product lines). Efforts to overcome this gap include extension of the UML meta-model to include features for variability modelling [ZJ06] or using both UML and feature models for modelling a domain [...].*

325.

*SysML adds some new diagrams and constructs not found in UML : the parametric diagram, the requirement diagram, required and provided features and flow properties.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

Le SysML n'est pas pour autant formel comme indiqué dans l'usage de scénario :

[Traduction libre]

- Scénario formel : Un scénario mathématiquement prouvable, par exemple à l'aide de méthodes formelles.
- Scénario semi-formel : Un scénario démontrable, par exemple à l'aide de notations visuelles, telles que SysML, des tableaux, du texte, etc. <sup>326</sup> [120]

Dans *Model-Driven Software Development : Technology, Engineering, Management*, les auteurs indiquent que l'UML n'est pas défini formellement, contrairement à l'architecture pilotée par modèle (MDA, *Model Driven Architecture* en anglais), ils indiquent que l'UML sert davantage pour la visualisation :

[Traduction libre] De prime abord, les outils UML impressionnent par leur capacité à synchroniser les modèles et le code. Cependant, un examen plus approfondi révèle que ces outils n'augmentent pas immédiatement la productivité, mais constituent au mieux une méthode efficace pour générer une documentation ayant une belle apparence. Ils peuvent aussi faciliter la compréhension de grandes quantités de code existant. [...] Les modèles UML ne sont pas, à proprement parler, des modèles MDA. La principale différence entre les modèles UML courants (par exemple, les modèles d'analyse) et les modèles MDA réside dans le fait que la signification (sémantique) des modèles MDA est définie formellement. Ceci est garanti par l'utilisation d'un langage de modélisation correspondant, généralement mis en œuvre par un profil UML et ses règles de transformation associées. <sup>327</sup> [391]

---

326.

- *Formal Scenario : A Scenario that is mathematically provable using, for example, formal methods.*
- *Semi-formal Scenario : A Scenario that is demonstrable using, for example, visual notations such as SysML, tables, text, etc.*

327.

*Superficially, UML tools impress with their ability to keep models and code synchronized. However, on closer inspection one finds that such tools do not immediately increase productivity, but are at best an efficient method for generating good-looking documentation. They can also help in understanding large amounts of existing code. [...] UML models*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

(voir énoncé p.206)

### Explication de l’assertion EM09 : Il y a une tendance à moins formaliser.

Frigg, dans l’ouvrage *Models and Theories*, soutient que globalement les scientifiques préfèrent utiliser une logique informelle :

[Traduction libre] Par « formalisation standard », Suppes entend une formalisation en logique du premier ordre. L’inquiétude qu’il exprime est difficile à cerner, car la notion de « simple ou d’élégant » peut varier selon le contexte et, de fait, les goûts personnels. Malgré cela, il convient de prendre cette inquiétude au sérieux, car il est avéré, dans la pratique scientifique, que les formulations de théories au premier ordre sont rares. [...] Mais, globalement, les scientifiques semblent préférer travailler avec une version (informelle) de la logique d’ordre supérieur, ce qui laisse supposer qu’ils trouvent ce cadre plus pratique.<sup>328</sup> [90]

---

*are not per se MDA models. The most important difference between common UML models (for example analysis models) and MDA models is that the meaning (semantics) of MDA models is defined formally. This is guaranteed through the use of a corresponding modeling language which that is typically realized by a UML profile and its associated transformation rules.*

328.

*By “standard formalization” Suppes means a formalisation in first-order logic. The worry Suppes expresses is somewhat difficult to pin down because what counts as “simple or elegant” may depend on context and, indeed, taste. Despite this, the worry is one that ought to be taken seriously because it is a fact of scientific practice that first-order formulations of theories tend to be rare. [...] But, on the whole, scientists seem to prefer to work in some (informal) version of higher order logic, which would suggest that they find that framework more convenient.*



## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

Un article décrit la préférence historique pour les approches intuitives en informatique appliquée :

[Traduction libre] La formalisation mathématique de la modélisation des systèmes a toujours été recherchée et peut, en principe, être réalisée à l'aide de la théorie des modèles du premier ordre de Tarski et de l'homomorphisme des structures relationnelles décrit dans [7] et [13]. Cependant, en pratique, une approche moins formelle et plus intuitive a été privilégiée en génie logiciel et système, grâce à l'utilisation de langages de modélisation graphique.<sup>329</sup> [75]

En 2022, un article aborde le formalisme dans les artefacts des spécifications des exigences :

[Traduction libre] La précision est si importante qu'un observateur extérieur pourrait supposer que tout développement logiciel commence par une description formelle des exigences. Cette approche est loin d'être la norme aujourd'hui ; la plupart des projets, lorsqu'ils formalisent des exigences, les expriment de manière informelle, soit dans un « document d'exigences » en langage naturel, soit (dans les méthodes agiles) à travers des « récits utilisateurs » individuels, également rédigés en langage naturel.<sup>330</sup> [43]

(voir énoncé p.207)

---

329.

*The mathematical formalization of system modeling has consistently been sought and, in principle, can be accomplished using the first-order model theory of Tarski and the homomorphism of relational structures prescribed in [7] and [13]. However, in practice, a less formal and more intuitive approach has been followed in software and systems engineering using graphical modeling languages.*

330.

*Precision is so important that an outsider might assume that all software development starts with a formal description of the requirements. This approach is by far not the standard today ; most projects, if they do write requirements, express them informally, either in a natural-language “requirements document” or (in agile methods) through individual “user stories,” also expressed in natural language.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

**Explication la citation EM10 : L'accès aux sujets, aux modèles ou aux artefacts doit être amélioré.**

Victor R. Basili écrit un article en 1996 intitulé *The Role of Experimentation in Software Engineering : Past, Current, and Future* dans lequel il défend le partage des modèles et des données :

[Traduction libre] Les chercheurs en génie logiciel ont besoin de laboratoires industriels pour observer, construire et analyser des modèles. Parallèlement, les praticiens doivent mettre en place des systèmes de qualité performants et rentables, notamment pour estimer les coûts, suivre l'avancement des projets et évaluer la qualité. Les modèles de processus et de produits élaborés par les chercheurs doivent être adaptés aux données collectées au sein de l'organisation et évoluer en fonction de son expérience. Ainsi, la recherche et les applications commerciales du génie logiciel entretiennent une relation symbiotique.<sup>331</sup> [24]

(voir énoncé p.207)

**Explication de l'assertion EM11 : Les informations sur le sujet, le modèle et l'artefact peuvent être inaccessibles, éparpillées et changeantes.**

Cela fait partie des difficultés énumérées dans un article sur la modélisation :

[Traduction libre]

- Il est rare, voire exceptionnel, que nous parvenions à une compréhension exhaustive du problème. Les experts du domaine sont certes utiles, mais

---

331.

*Software engineering researchers need industry-based laboratories that allow them to observe, build and analyze models. On the other hand, practitioners need to build quality systems productively and profitably, e.g., estimate cost track progress, evaluate quality. The models of process and product generated by researchers should be tailored based upon the data collected within the organization and should be able to continually evolve based upon the organization's evolving experiences. Thus the research and business perspectives of software engineering have a symbiotic relationship.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

leur compréhension des domaines dans lesquels ils travaillent est également limitée, leurs perspectives peuvent différer et ils peuvent avoir des divergences sur les approches, les processus et les priorités. C'est pourquoi, en pratique, nous consacrons beaucoup de temps à analyser ces différentes contributions afin d'obtenir une vision commune et évolutive du domaine du client.

- Les solutions à envisager sont soumises à de nombreuses contraintes : par exemple, trouver un équilibre entre le temps et les efforts nécessaires à la mise en œuvre d'un système, l'intégrer aux applications et technologies existantes et coordonner les différentes équipes qui produisent les composants du système déployé.
- De nombreux types de changements doivent être pris en compte à toutes les étapes du développement : des erreurs sont découvertes, les conceptions sont mises à jour, les exigences sont affinées, les priorités évoluent, les implémentations sont remaniées, etc. La nature dynamique du processus de développement implique de consacrer beaucoup de temps à comprendre l'impact d'un changement, à définir un plan pour y remédier et à s'assurer que toutes les actions sont menées correctement.
- Un projet de développement logiciel de grande envergure est un projet d'ingénierie impliquant des équipes de personnes aux compétences diverses, interagissant sur une période prolongée pour mettre en œuvre une solution qui ne peut jamais être véritablement considérée comme « terminée ». Par conséquent, toutes les techniques de gestion de projets d'ingénierie doivent être prises en compte : planification, gestion des ressources, atténuation des risques, analyse du retour sur investissement, documentation, support, etc. <sup>332</sup> [41]

---

332.

— *We rarely, if ever, get a full understanding of the problem space. Domain experts help, but they, too, have a limited understanding of the areas they work in, view things from different perspectives, or disagree on approaches, processes, and priorities. So in practice, we spend a lot of time analyzing these different inputs and obtaining a common, evolving view of the customer's domain.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

(voir énoncé p.207)

### Explication de la définition EM12 : Arithmétique de Peano

La définition formulée est : système formel de l'arithmétique élémentaire qui permet des raisonnements par récurrence.

Voici des descriptions et une définition de Jean-Paul Delahaye provenant de l'ouvrage *Aux frontières des mathématiques : Kurt Gödel et l'incomplétude* [73] :

- « [...] système axiomatique de l'arithmétique usuelle - appelé arithmétique de Peano que nous noterons PEANO. »
- « Pour définir le système formel de l'arithmétique de PEANO [...] nous procédons en deux étapes en commençant par l'arithmétique de Robinson (notée ROB). Elle est définie par les sept axiomes suivants, qui s'ajoutent aux axiomes logiques du calcul des prédicats du premier ordre avec l'égalité pour le langage comportant les symboles,  $0, +, x, s$  (pour désigner la fonction successeur).  
[...]

L'arithmétique de Peano, PEANO, est alors ce que l'on obtient en ajoutant encore tous les axiomes de la forme :

$$(P(0) \text{ et } \forall x(P(x) \implies P(s(x)))) \implies \forall y P(y)$$

(Si  $P$  est vrai pour  $0$  et que de l'hypothèse que  $P$  est vraie pour  $x$  on peut déduire que  $P$  est vrai pour le successeur de  $x$ , alors  $P$  est vrai pour tout  $y$ ) Où  $P(x)$

- 
- *There are many constraints on the solutions to consider : for example, balancing the time and effort required to implement a system, integrating with existing applications and technologies already in place, and coordinating multiple teams producing different components of the deployed system. ;*
  - *Many kinds of changes must be addressed at all stages of development : errors will be discovered, designs will be updated, requirements will be refined, priorities will be changed, implementations will be refactored, and so on. The dynamic nature of the development process results in a lot of time spent on understanding the impact of a change, defining a plan to address the change, and ensuring that any actions are carried out appropriately.*
  - *A large software development task is an engineering project involving teams of people with different skills interacting over an extended period of time to implement a solution that can never truly be said to be "finished." As a result, all the techniques of managing engineering projects must be taken into account : scheduling, resource management, risk mitigation, ROI analysis, documentation, support, and so on.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

est une formule quelconque du langage de PEANO. Ces formules permettent de mener les raisonnements par récurrence, indispensables en arithmétique. »

— « PEANO : système formel de l'arithmétique élémentaire. »

(voir énoncé p.207)

**Explication du théorème EM13 : Un système formel pouvant interpréter une arithmétique de Peano est incomplet.**

À l'origine des théorèmes de complétude se trouvent les travaux de Tarski, qui a prouvé, sous certaines conditions, la complétude et la décidabilité. Ces travaux ont permis par la suite d'arriver aux théorèmes de complétude :

[Traduction libre] Dans [128], Tarski a démontré des résultats de complétude et de décidabilité pour une théorie des nombres réels, où les formules atomiques font intervenir l'égalité ( $=$ ) et les relations d'ordre ( $<$ ,  $\leq$ ,  $>$ ,  $\geq$ ) et où les termes sont construits à partir de constantes rationnelles et de variables (globales) à l'aide des opérations d'addition, de soustraction, de négation et de multiplication. Pour la théorie des nombres naturels, Presburger a donné, en 1930, un algorithme de décision pour l'arithmétique linéaire (à l'exclusion de la multiplication), tandis que Gödel, en 1931, a établi son célèbre théorème d'incomplétude pour une théorie ayant l'addition, la soustraction, la négation et la multiplication comme opérations. <sup>333</sup> [29]

Cette description du premier théorème sur l'incomplétude de Gödel sert à la construction de l'énoncé présent :

---

333.

*In [128] Tarski proved completeness and decidability results for a theory of reals, where atomic formulas involve equality ( $=$ ) and ordering relations ( $<$ ,  $\leq$ ,  $>$ ,  $\geq$ ) and terms are constructed from rational constants and (global) variables using operations for addition, subtraction, negation and multiplication. For natural-number theory, Presburger gave, in 1930, a decision algorithm for linear arithmetic (excluding multiplication), while Gödel, in 1931, established his famous incompleteness theorem for a theory having addition, subtraction, negation and multiplication as operations.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

[Traduction libre] Le premier théorème d'incomplétude de Gödel a démontré que l'arithmétique du premier ordre est incomplète, c'est-à-dire qu'il existe des vérités en arithmétique du premier ordre qui ne peuvent être démontrées dans le langage de son axiomatisation.<sup>334</sup> [286]

(voir énoncé p.207)

**Explication du théorème EM14 : Un système formel cohérent pouvant interpréter une arithmétique de Peano ne peut pas vérifier sa cohérence.**

La description du théorème d'indéfinissabilité de Tarski aide à la compréhension du deuxième théorème d'incomplétude de Gödel en plus de donner une base :

[Traduction libre] Le théorème de Tarski sur l'indéfinissabilité de la vérité nous apprend qu'aucun système logique (capable de formaliser une certaine quantité d'arithmétique) ne peut formaliser sa propre sémantique, et le second théorème d'incomplétude de Gödel nous indique qu'il ne peut en aucun cas prouver sa propre cohérence — sauf, bien sûr, s'il n'est pas cohérent, auquel cas il peut tout prouver [21]. Ainsi, indépendamment des détails d'implémentation, si nous voulons prouver la cohérence d'un vérificateur de preuves, nous devons utiliser une logique qui, à certains égards au moins, dépasse celle que le vérificateur prend en charge.<sup>335</sup> [94]

---

334.

*Gödel's first incompleteness theorem showed that first-order arithmetic is incomplete, i.e. there are truths in first-order arithmetic that cannot be proved in the language of the axiomatization of first-order arithmetic.*

335.

*Tarski's theorem on the undefinability of truth tells us that no logical system (capable of formalizing a certain amount of arithmetic) can formalize its own semantics, and Gödel's second incompleteness theorem tells us that it cannot prove its own consistency in any way at all — unless of course it isn't consistent, in which case it can prove anything [21]. So, regardless of implementation details, if we want to prove the consistency of a proof checker, we need to use a logic that in at least some respects goes beyond the logic the checker itself supports.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

Cette description du deuxième théorème sur l'incomplétude de Gödel sert à la construction de l'énoncé présent :

[Traduction libre] Le second théorème d'incomplétude de Gödel a démontré que tout système formel étendant l'arithmétique de base ne peut démontrer sa propre cohérence au sein de ce système formel.<sup>336</sup> [286]

(voir énoncé p.207)

**Explication des corollaires EM15 : Une conséquence à l'incomplétude est qu'il n'existe pas d'algorithme permettant de tout vérifier dans un système formel en mesure d'interpréter l'arithmétique de Peano.**

Des théorèmes d'incomplétudes, il en résulte ce corollaire qui est décrit ainsi :

[Traduction libre] Il existe des théories du premier ordre décidables. Cependant, la logique du premier ordre incluant les axiomes de Peano (ou tout système formel comprenant l'addition et la multiplication) ne peut être décidée par un algorithme. Autrement dit, il n'existe aucun algorithme permettant de déterminer si une proposition mathématique quelconque est vraie ou fausse. La logique propositionnelle, quant à elle, est décidable, car il existe une procédure (par exemple, l'utilisation d'une table de vérité) permettant de déterminer la validité d'une formule [...].<sup>337</sup> [286]

(voir énoncé p.207)

---

336.

*Gödel's second incompleteness theorem showed that any formal system extending basic arithmetic cannot prove its own consistency within the formal system.*

337.

*There are first-order theories that are decidable. However, first-order logic that includes Peano's axioms of arithmetic (or any formal system that includes addition and multiplication) cannot be decided by an algorithm. That is, there is no algorithm to determine whether an arbitrary mathematical proposition is true or false. Propositional logic is decidable as there is a procedure (e.g. using a truth table) to determine whether an arbitrary formula is valid [...].*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

### Explication de l’assertion EM16 : Il n’existe pas de consensus concernant les modèles.

En 2020, Stephen M. Downes écrit un ouvrage sur les modèles en sciences. Il fait les constatations suivantes :

[Traduction libre] Nous avons également constaté l’absence de système de classification adéquat pour les modèles. Classer les modèles en fonction de leur structure ou de leurs constituants aboutit à des cas relevant de deux des systèmes de classification que nous proposons, ou révèle des modèles très similaires qui ne semblent pourtant pas appartenir à la même classe. Enfin, nous avons constaté que la relation entre modèles et théories est complexe et difficile à expliquer.<sup>338</sup> [78]

En 2022, Roman Frigg consacre un ouvrage à décrire et expliquer les modèles. Dans le chapitre intitulé [Traduction libre] « Le modèle embrouillé »<sup>339</sup>, l’auteur soulève qu’il existe au moins 120 différents types de modèles, dont plusieurs partagent des relations de dérivations ou de similarités. Il ajoute que :

[Traduction libre] [...] nul ne pourra, sans un diplôme en « modélisation », naviguer dans la littérature sur les modèles sans risquer de se blesser.<sup>340</sup> [90]

En 2023, l’ouvrage *Body of Knowledge for Modeling and Simulation : A Handbook by the Society for Modeling and Simulation International* explique en partie pourquoi il n’y a pas de consensus sur la modélisation :

---

338.

*We also found that there is no adequate classification scheme for models. Classifying models by appealing to their structure or constituents leaves us with cases that fall into two of our proposed classification schemas or reveals very similar models that still do not appear to belong in the same class. Finally, we found that the relation between models and theories is not straightforward and cannot be easily accounted for.*

339. « *The model muddle* »

340.

*[...] no one without a degree in “model studies” will be able to navigate the literature on models without harm and injury.*



## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

[Traduction libre]

- [...] différents contextes (industriels, gouvernementaux ou militaires)[...].
- [...] différentes disciplines (que ce soit dans les sciences « dures » ou « molles » ou l'ingénierie) [...]. <sup>341</sup> [283]

(voir énoncé p.207)

### Explication de l'assertion EM17 : Il n'existe pas de terminologie établie pour la modélisation en ingénierie.

En 2017, l'ouvrage *Model-Driven software engineering in practice* nomme une section ainsi « LOST IN ACRONYMS : THE MD\* JUNGLE ». Il y est décrit : le *Model-Driven Development* (MDD), le *Model-Driven Architecture* (MDA), le *Model-Driven Engineering* (MDE) et le *Model-Based Engineering* (MBE). Il est ajouté que :

[Traduction libre] Il est à noter qu'on trouve aussi dans la littérature une multitude de variantes de ces acronymes. Par exemple, MDSE (Model-Driven Software Engineering), MDPE (Model-Driven Product Engineering), et bien d'autres. <sup>342</sup> [36]

En 2018, un article propose de développer un corpus de connaissances en génie logiciel basé sur les modèle (MBEBOK, *Model-Based Software Engineering Body of Knowledge* en anglais) [57]. L'article y aborde le corpus de connaissances en génie des langages logiciels (SLEBOK, *Software Language Engineering BoK* en anglais) et l'inspiration provenant du SWEBOK. Il y a soulevé l'intégration possible entre les trois corpus de connaissances.

---

341.

- [...] *different contexts (whether industrial, governmental, or military)*[...].
- [...] *disciplinary (whether in "hard" or "soft" science or engineering)* [...].

342.

*Notice that a huge set of variants of all these acronyms can be found in literature too. For instance, MDSE (Model-Driven Software Engineering), MDPE (Model-Driven Product Engineering), and many others exist.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

En 2023, l'ouvrage *Body of Knowledge for Modeling and Simulation : A Handbook by the Society for Modeling and Simulation International* décrit l'état problématique de la terminologie, mais invite quand même les gens à la considérer pour commencer un corpus de connaissances :

[Traduction libre] Certaines définitions peuvent fournir une bonne « première approximation » pour comprendre un terme et son utilisation, mais elles manquent de la précision et de la rigueur auxquelles un référentiel de connaissances devrait aspirer.<sup>343</sup> [283]

En 2026, le site web officiel du MBEBOK affiche toujours une publication de 2020 d'Alfonso Pierantonio qui peut laisser croire que le glossaire n'est pas encore établi :

[Traduction libre] De plus, nous travaillons à l'élaboration d'un glossaire de termes pour le MBEBOK, avec la terminologie qui définit l'ensemble des concepts centraux de la discipline et constitue l'ontologie acceptée pour le domaine MBE (par exemple, cela éviterait des discussions éternelles, comme la distinction entre MDE, MBE et MDA,...).<sup>344 345</sup>

(voir énoncé p.<sup>207</sup>)

### **Explication de l'assertion EM18 : Le vocabulaire peut confondre un modèle avec ses artefacts.**

Cette confusion se produit notamment avec l'échéancier et le modèle d'échéancier. Ces définitions illustrent la problématique :

---

343.

*Some definitions can provide a good “first approximation” to understanding a term and how it is used, and they lack the precision and rigor that a BoK should aspire to.*

344.

*Additionally, we are working on the development of a glossary of terms for the MBE-BOK, with the terminology that defines the set of central concepts of the discipline, and constitutes the accepted ontology for the MBE domain (e.g. this would avoid eternal discussions as the distinction between MDE vs MBE vs MDA,...).*

345. Alfonso Pierantonio, site web, « A Body of Knowledge for Model-Based Software Engineering », <https://modeling-languages.com/body-of-knowledge-model-based-software-engineering> (consulté le 2026-04-16).

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

- un modèle d'échéancier est : « Représentation du plan d'exécution des activités du projet, comprenant les durées, les dépendances et d'autres informations de planification, utilisées pour produire un échéancier du projet [...]. » [310] ;
- l'échéancier du projet est : « une donnée de sortie d'un modèle d'échéancier qui présente les activités liées avec des dates planifiées, des durées, des jalons et des ressources. » [310] ;
- définition 1 d'échéancier est : « Ensemble de délais à respecter. » [336] ;
- définition 2 d'échéancier est : « Document où figurent, par ordre chronologique, des échéances, les règlements à effectuer ou à recevoir. » [280].

L'échéancier est un document qui peut être à la fois une entrée ou une sortie du modèle d'échéancier.

David L. Parnas indique dans l'article *Really Rethinking 'Formal Methods'* qu'il ne faut plus interchanger modèle et spécification :

[Traduction libre] De nombreux articles sur les méthodes formelles utilisent les termes « modèle » et « spécification » de manière interchangeable, probablement en se basant sur une définition standard du dictionnaire qui désigne une information spécifique. Cette définition ne correspond pas à l'usage en ingénierie [...].<sup>346</sup> [293]

(voir énoncé p.207)

### Explication de l'assertion EM19 : Les modèles peuvent être mal interprétés.

En 2013, une note technique du Software Engineering Institute (SEI) décrit le modèle en cascade par rapport aux méthodes agiles. Les auteurs y indiquent que :

[Traduction libre] Le Dr Winston Royce, souvent, à tort, surnommé le « père du modèle en cascade » et auteur de l'article fondateur de 1970

---

346.

*Many papers on formal methods use the words "model" and "specification" interchangeably, perhaps based on a standard dictionary definition of specification as specific information. This definition does not correspond to engineering use [...].*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

intitulé *Managing the Development of Large Software Systems*, n'a apparemment jamais envisagé que la caricature en cascade de son modèle soit autre chose qu'un élément de la discussion académique de son article, menant à une version plus itérative [Royce 1970].<sup>347</sup> [287]

Les auteurs citent d'ailleurs le fils de Walker Royce qui rappelle que son père défendait l'itératif et avait émis son opposition quant au modèle en cascade :

[Traduction libre] Il a toujours été un fervent défenseur du développement itératif, incrémental et évolutif. Son article décrivait le modèle en cascade comme étant la description la plus simple, mais précisait qu'il ne convenait qu'aux projets les plus simples. Le reste de son article décrit [les pratiques itératives] dans le contexte des modèles de marchés publics des années 60/70 (un ensemble de contraintes importantes) [Larman 2003].<sup>348</sup> [287, 402]

Dans l'ouvrage *The design of design : essays from a computer scientist*, Frederick P. Brooks Jr. revient sur son repentir d'avoir confondu l'exemple du cascade :

[Traduction libre] Royce a présenté sa théorie de la cascade comme un épouvantail qu'il a ensuite réfuté, mais beaucoup ont cité et suivi cet homme de paille plutôt que son modèle plus élaboré. J'ai moi-même fait cette erreur dans ma jeunesse et je m'en suis repenti publiquement par la suite.<sup>349</sup> [40]

---

347.

*Dr. Winston Royce, the man who is often but mistakenly called the “father of waterfall” and the author of the seminal 1970 paper Managing the Development of Large Software Systems, apparently never intended for the waterfall caricature of his model to be anything but part of his paper’s academic discussion leading to another, more iterative version [Royce 1970].*

348.

*He was always a proponent of iterative, incremental, evolutionary development. His paper described the waterfall as the simplest description, but that it would not work for all but the most straightforward projects. The rest of his paper describes [iterative practices] within the context of the 60s/70s government contracting models (a serious set of constraints) [Larman 2003].*

349.

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

En 2021, le PMBOK indique que le cascade est devenu synonyme d'approche prédictive :

[Traduction libre] Une approche prédictive est utile lorsque les exigences du projet et du produit peuvent être définies, recueillies et analysées dès le début du projet. On parle alors d'approche en cascade.<sup>350</sup> [305]

(voir énoncé p.207)

**Explication de l'assertion AM01 : Les instruments de modélisation comprennent notamment des langages, des techniques, des méthodes et les connaissances du domaine.**

En 2024, dans l'article *Metamodeling Platforms : Observations and Evolutions* Harald Kühn étend ce qu'il a précédemment proposé pour mettre l'emphasis sur le domaine de connaissances (figure B.25). Les instruments de modélisation sont spécifiques aux domaines et une plateforme de création d'instruments est indépendante des domaines. L'avantage décrit par l'auteur d'avoir une approche personnalisable pour les modèles est que, dans la réalité, certains modèles ne sont pas adaptables à tous les contextes, voici ce qu'il écrit :

[Traduction libre] Cette approche est particulièrement pertinente dans les domaines où les méthodes standardisées, les langages de modélisation standardisés et/ou les mécanismes prédéfinis de modélisation ne permettent pas de saisir pleinement la spécificité et l'objectif d'une situation. Dans de tels cas, les solutions adaptées au domaine apportent une valeur ajoutée en termes d'efficacité, de qualité et/ou de réduction des coûts.<sup>351</sup>

(voir énoncé p.208)

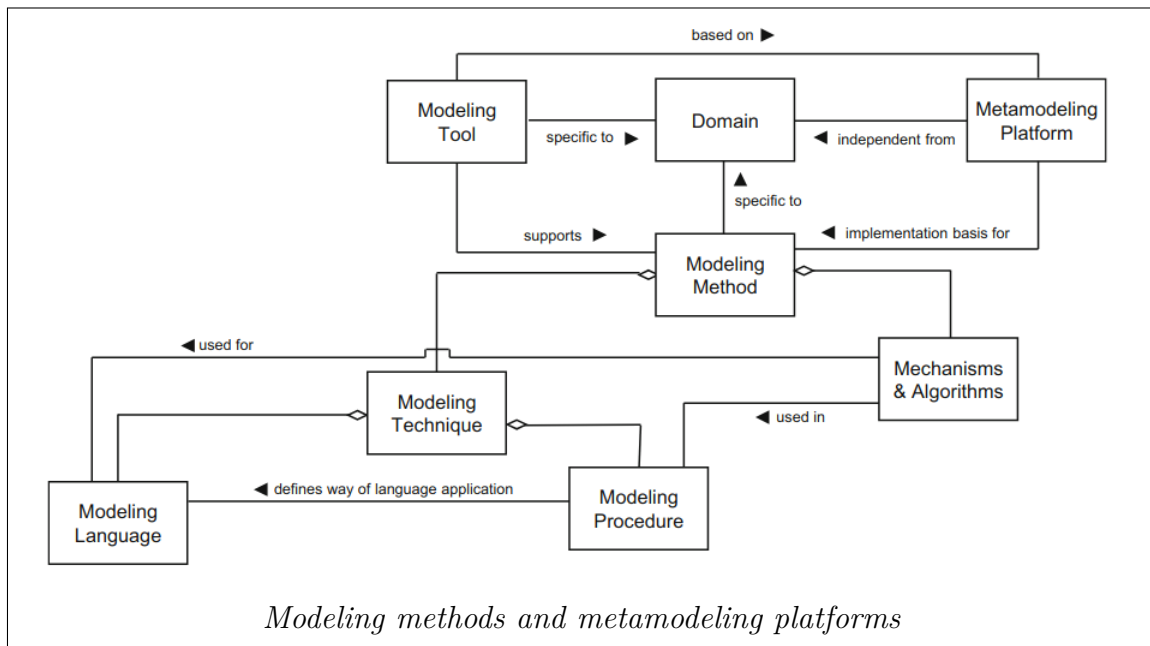
---

*Royce introduced his waterfall as a straw man that he then argued against, but many people have cited and followed the straw man rather than his more sophisticated model. I made this mistake myself in my younger days, and publicly repented of it later.*

350.

*A predictive approach is useful when the project and product requirements can be defined, collected, and analyzed at the start of the project. This may also be referred to as a waterfall approach.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.25** – Modèle au sujet des instruments de modélisation (source : *Figure 1* [168])

### Explication de la définition AM02 : Outil de génie logiciel assisté par ordinateur (*CASE tools : computer-aided software engineering tools*)

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de CASE tools : [Traduction libre] « [...] produit logiciel capable d’assister les ingénieurs logiciels en fournissant une assistance automatisée pour les activités du cycle de vie du logiciel [...]. »<sup>352</sup> [140];
- description de Michael Jackson : [Traduction libre] « Il existe une symbiose évidente entre les outils CASE et les méthodes : un outil mécanise les opérations

351. *This approach is highly relevant in fields where standardized methods, standardized modeling languages and/or pre-defined mechanisms working on models do not cover the full scope of a situation’s particularity and purpose. In such situations, domain-specific solutions create value in efficiency, quality and/or reduced costs.*

352. « [...] software product that can assist software engineers by providing automated support for software life-cycle activities [...]. »

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

prescrites par la méthode, et la méthode tire parti des outils pour prescrire des opérations, des notations et des types de description souhaitables qui ne seraient pas réalisables sans eux. »<sup>353</sup> [146].

(voir énoncé p.208)

### Explication de la définition AM03 : Langage spécifique à un domaine

La définition formulée est : langage destiné aux utilisateurs dans un domaine spécifique dont la syntaxe ou la sémantique peut être définie formellement.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 : « Langage de programmation conçu pour apporter des solutions aux problèmes d'un domaine spécifique. » [263] ;
- définition 2 : [Traduction libre] « [...] est un langage de programmation ou de modélisation informatique à expressivité limitée, axé sur un domaine particulier ou l'un de ses aspects. »<sup>354</sup> [406] ;
- définition de *general-purpose programming languages* (GPL) : [Traduction libre] « [...] vise à décrire les algorithmes et à structurer les systèmes logiciels. »<sup>355</sup> [406] ;
- description de sémantique statique (*static semantics*) : [Traduction libre] « La sémantique statique est généralement spécifiée par l'analyse des noms, la vérification des types et diverses contraintes de validité. »<sup>356</sup> [406] ;
- description de sémantique dynamique (*dynamic semantics*) : [Traduction libre] « La sémantique dynamique est généralement réalisée soit par l'interprétation (exécution, visualisation ou calcul) des modèles, soit par la traduction vers

---

353. « *There is an obvious symbiosis of CASE tools with methods : a tool mechanises the operations prescribed by the method, and the method takes advantage of the tools to prescribe desirable operations, notations, and description types that would not be practicable without them.* »

354. « *[...] is a computer programming or modeling language of limited expressiveness focused on a particular domain or its aspect.* »

355. « *[...] aim at describing algorithms and structuring software systems.* »

356. « *Typically, static semantics are specified by means of implementing a name analysis, type checking, and a number of validity constraints.* »

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

d'autres langages dont la signification est connue. Elle peut généralement être définie de plusieurs manières pour un même langage. » <sup>357</sup> [406] ;

- note sur la sémantique des DSL : [Traduction libre] « Il est vrai qu'en suivant la terminologie des compilateurs, nous utilisons à tort le terme de sémantique dynamique pour les DSL. Contrairement aux GPL, la sémantique de nombreux DSL n'est pas du tout dynamique ; ils ne sont pas exécutables. » <sup>358</sup> [406]

(voir énoncé p.208)

### Explication la définition AM04 : Automate

La définition formulée est : cas particulier de machine abstraite formellement définie et conçue pour en permettre l'étude.

La littérature fournit ces informations qui ont servi à construire cette définition :

- description 1 de la théorie des automates : [Traduction libre] « La théorie des automates est l'étude des dispositifs de calcul abstraits ou « machines ». » <sup>359</sup> [122] ;
- description 2 de la théorie des automates : [Traduction libre] « L'informatique théorique, ou théorie des automates, étudie les machines abstraites et les problèmes de calcul qui leur sont liés. Ces machines abstraites sont appelées automates. » <sup>360</sup> [157] ;
- définition d'automate : « Le terme automate est appliqué à des êtres mathématiques formalisant et décrivant le fonctionnement de "machines", c'est-à-dire de mécanismes ou de procédures logiques permettant d'aboutir en un nombre de

---

357. « *Dynamic semantics is typically realized either by interpreting (e.g., executing, visualizing, or calculating) the models, or by translating to other languages, for which the meaning is known. Typically, it can be defined in multiple ways for the same language.* »

358. « *Admittedly, by following the compiler terminology, we misuse the term dynamic semantics for DSLs. Unlike for GPLs, the semantics of many DSLs is not dynamic at all—they are not executable.* »

359. « *Automata theory is the study of abstract computing devices or "machines."* »

360. « *Theoretical computer science or automata theory is the study of abstract machines and the computational problems related to these abstract machines. These abstract machines are called automata.* »



## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

pas fini ou non fini à un état final, à partir d'un état initial donné à l'avance. » [240].

(voir énoncé p.208)

### Des instruments de modélisation

#### Explication de l'assertion AM05 : Plusieurs instruments de modélisation ont une faible durée de vie (médiane de 6 ans)

La durée de vie des instruments est aussi variable. En exemple, MetaEdit a une plus grande longévité.

En 1991, un article décrit le *Meta-CASE tools* [10]. Il est affirmé qu'il s'agit de la solution pour personnaliser le procédé de développement pour une organisation. Il permet de générer des *CASE tools*.

En 1992, un article décrit un *Meta-CASE tools* du nom de MetaEdit [346].

En 2019, un article fait un inventaire des instruments de modélisation de 1990 à 2017.

En tout, soixante instruments sont répertoriés. Voici la citation :

[Traduction libre] Les données montrent que 24 outils ont eu une durée de vie inférieure à 5 ans. 20 outils ont duré entre 5 et 10 ans, et seulement 16 ont survécu plus de 10 ans. La durée de vie moyenne et médiane des outils est respectivement de 7,5 et 6 ans. TXL et Metaedit+ affichent actuellement la plus grande longévité, tandis que l'outil actif ayant la plus courte durée de vie est Melange (car c'est aussi le plus récent).<sup>361</sup> [155]

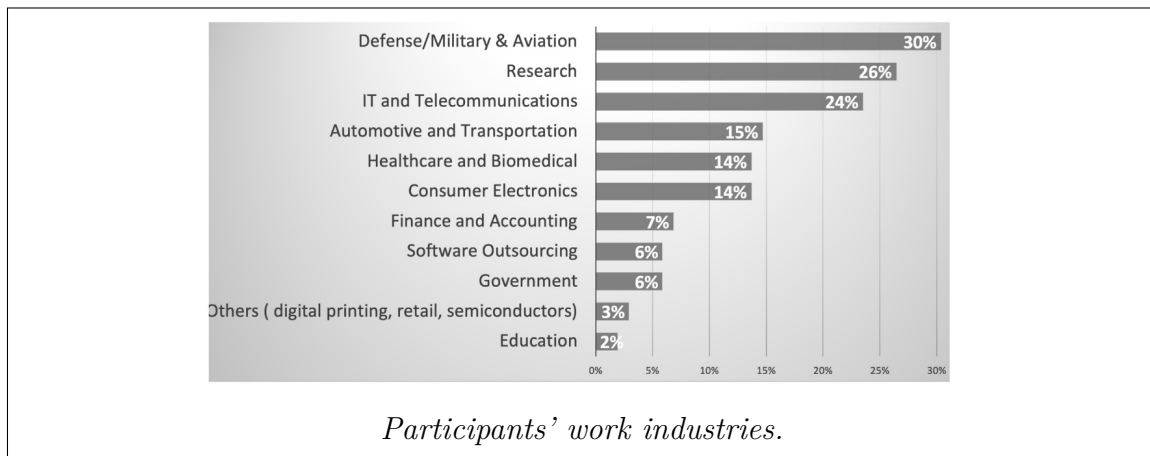
(voir énoncé p.208)

---

361.

*The data show that 24 tools had a lifetime of less than 5 years. 20 tools lasted between 5 and 10 years, and only 16 of tools have survived more than 10 years. The average and median longevity of tools is 7.5 and 6 years, respectively. TXL and Metaedit+ currently have the highest longevity, and the shortest-lived active tool is currently Melange (because it is also the newest).*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.26** – Secteurs où des instruments de modélisation sont utilisés par des professionnels (source : *Figure 3* [284])

**Explication de l’assertion AM06 : Les instruments de modélisation ne rejoignent pas l’ensemble des professionnels informatiques.**

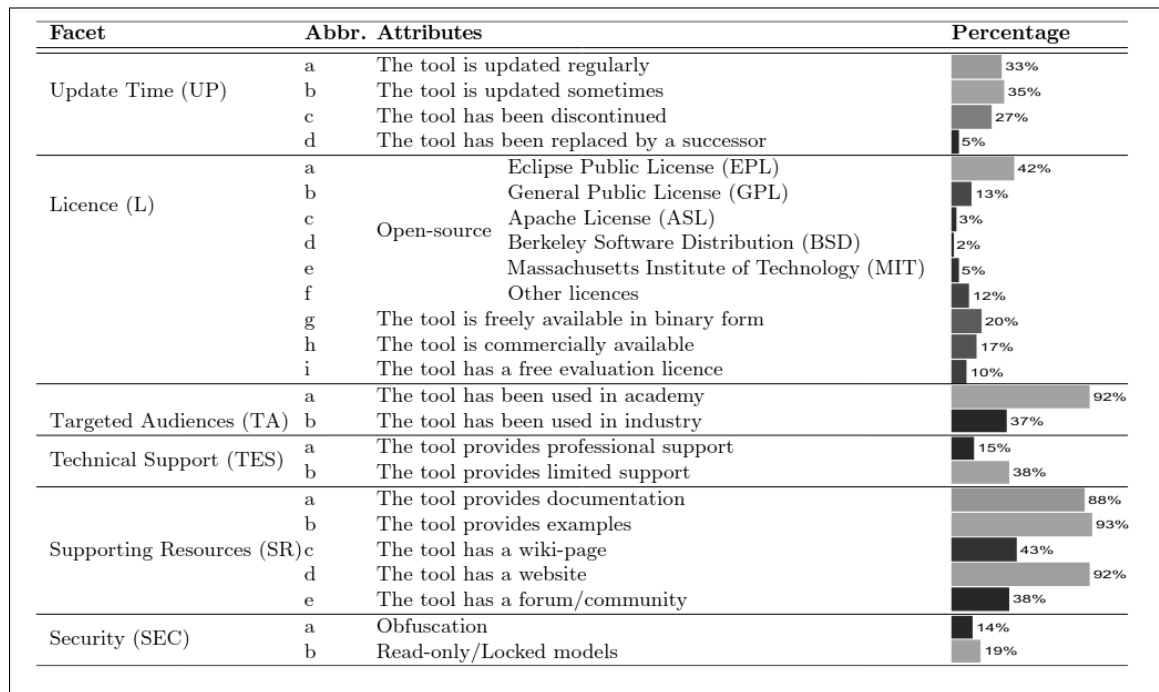
En 2021, un sondage mené auprès d’utilisateurs d’instruments de modélisation, a réuni 103 répondants dont 77 % Européens issus pour la plupart de secteurs critiques [284] (figure B.26). Le secteur critique couvre un sous-ensemble de tous les secteurs. En 2024, lors une revue des instruments de modélisation ( $n = 60$ ), il ressort que les instruments de modélisation sont plus utilisés dans un usage académique qu’industriel [155] (figure B.27). Plus précisément, le critère **public cible** est décrit ainsi :

[Traduction libre] Ce critère concerne le domaine d’utilisation actuel ou futur des outils : milieu universitaire, industrie, ou les deux. Les outils universitaires sont généralement conçus pour illustrer des concepts théoriques ou répondre à de nouveaux enjeux de recherche susceptibles d’intéresser d’autres chercheurs. En revanche, les outils industriels sont conçus pour répondre aux besoins industriels existants, ce qui les rend plus adaptés à une utilisation aussi bien dans l’industrie que dans le milieu universitaire.<sup>362</sup> [155]

---

362.

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.27** – Caractéristiques générales des instruments de modélisation (source : Table 3 [155])

(voir énoncé p.208)

**Explication de l’assertion AM07 : Ce ne sont pas tous les instruments de modélisation qui couvrent l’ensemble des fonctionnalités possibles.**

En 2019, un article dresse un inventaire des instruments de modélisation publiés entre 1990 et 2017 [155]. Au total, 60 instruments sont recensés. Plusieurs résultats sont révélés dans la figure B.28 et la figure B.29 portant sur les fonctionnalités offertes par ces instruments. Parmi les fonctionnalités, il y a : les transformations de modèles, la

*This facet refers to where the tools have been or are intended to be used : in academia, in industry, or both. Academic tools are typically developed to demonstrate theoretical concepts or to address new research challenges of interest to other researchers. On the other hand, industrial tools are designed to address existing industrial requirements, which can make them more practical for use in both industry and academia.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

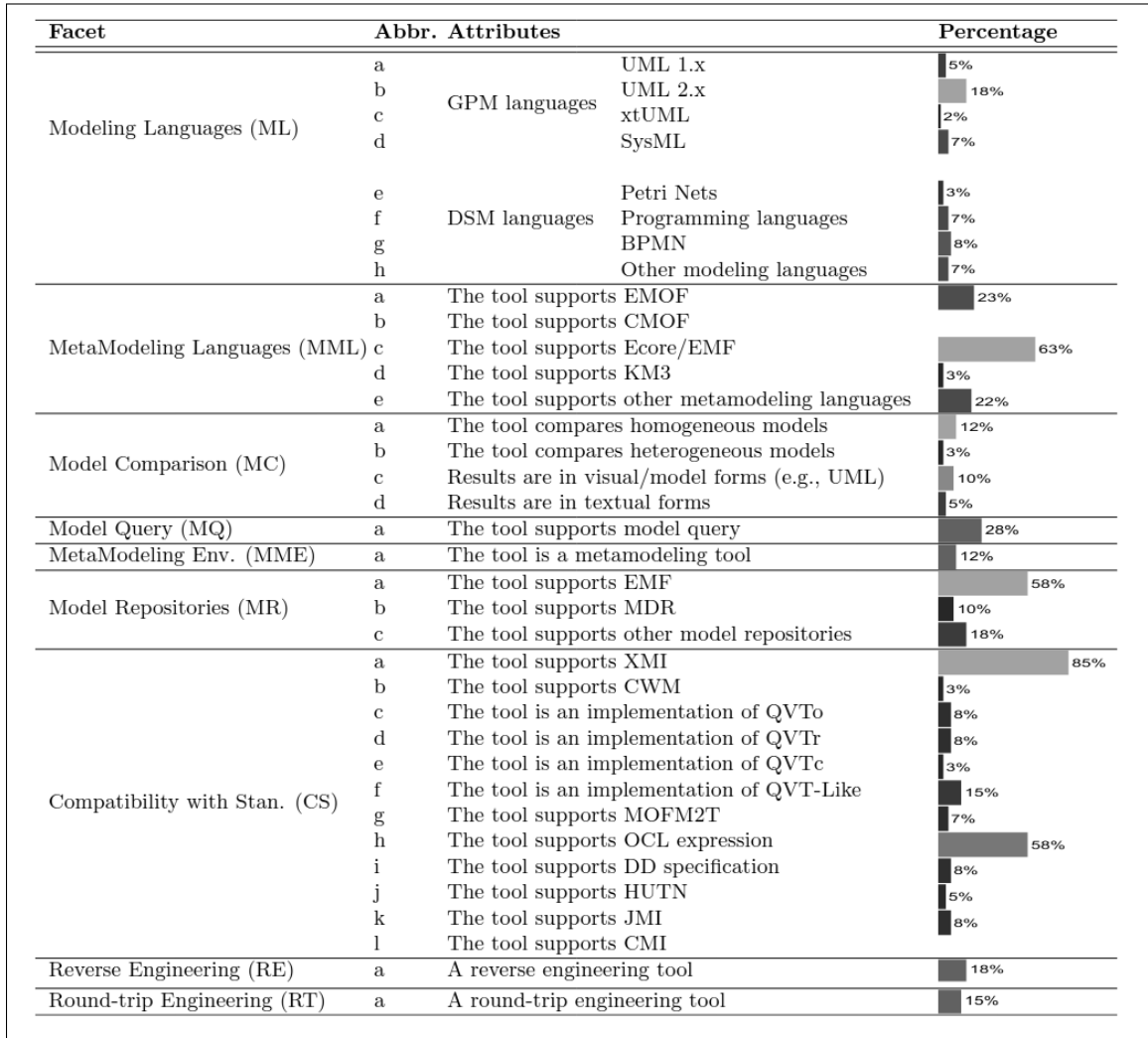
vérification et la validation de modèles, la comparaison de modèles et, etc.  
(voir énoncé p.208)

| Facet                     | Abbr. | Attributes                                 | Percentage |
|---------------------------|-------|--------------------------------------------|------------|
| MT Language Syntax (MTLX) | a     | The tool has graphical syntax              | 48%        |
|                           | b     | The tool has textual syntax                | 63%        |
| Output (O)                | a     | M2M<br>In-place/Destructive                | 58%        |
|                           | b     |                                            | 75%        |
|                           | c     | M2T<br>Textual artifacts                   | 40%        |
|                           | d     |                                            | 40%        |
|                           | e     |                                            | 23%        |
| Verification (V)          | a     | Syntactic correctness                      | 35%        |
|                           | b     | Termination                                | 18%        |
|                           | c     | Semantic correctness                       | 33%        |
|                           | d     | Completeness                               | 8%         |
|                           | e     | Determinism/Uniqueness/Confluence          | 12%        |
|                           | f     | Robustness                                 | 13%        |
|                           | g     | Definedness                                | 12%        |
| Validation (VA)           | a     | The tool provides a testing environment    | 33%        |
|                           | b     | The tool provides a simulation environment | 25%        |
| Traceability (TR)         | a     | Automatic                                  | 47%        |
|                           | b     | User-defined                               | 45%        |

<sup>a</sup> Plusieurs facettes ont été retirées parce que jugées moins informatives.

**FIGURE B.28** – Fonctionnalités offertes par les instruments concernant les transformations (modifiée de *Figure 5* [168])

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.29** – Fonctionnalités offertes par les instruments concernant les modèles (modifiée de *Figure 4* [168])

### Des méthodes de modélisation

**Explication de l’assertion AM08 : Plusieurs personnes visent à rendre des langages formels accessibles aux utilisateurs.**

Un article publié en 1972 souligne l’intérêt de confier aux utilisateurs la gestion de leurs langages :

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

[Traduction libre] Les langages définis par l'utilisateur suscitent un intérêt depuis de nombreuses années. Seules quelques tentatives primitives ont été réalisées ; elles sont brièvement décrites dans [18]. Permettre aux utilisateurs de définir leurs propres langages est un concept important, car il retire aux développeurs de langages le contrôle de ce qui est bon pour l'utilisateur et le place entre les mains des utilisateurs.<sup>363</sup> [353]

Dans l'ouvrage *Domain-Specific Language*, Martin Fowler expose les avantages des DSL, en soulignant particulièrement celui lié à l'accessibilité :

[Traduction libre] Je crois que la partie la plus difficile des projets logiciels, la source la plus fréquente d'échec, réside dans la communication avec les clients et les utilisateurs de ce logiciel. En fournissant un langage clair et précis pour gérer les domaines, un DSL peut contribuer à améliorer cette communication.<sup>364</sup> [89]

Dans le manuel *Handbook of Model-Based Systems Engineering*, il est préféré les langages spécifiques à un domaine (DSL) aux langages de modélisation des systèmes (SysML) :

[Traduction libre] SysML fournit un métamodèle commun permettant d'établir un cadre de référence partagé. Cependant, comme indiqué précédemment, la complexité de sa formulation « prête à l'emploi » est considérable. Par conséquent, il est conseillé aux projets et aux organisations de créer leur propre interprétation adaptée, basée sur un DSL.<sup>365</sup> [210]

---

363.

*User defined languages have been an area of interest for many years. Only a few primitive attempts have been made ; they are described briefly in [18]. Allowing people to define their own languages is a significant concept because it takes the control of "what is good for the user" out of the hands of language developers and puts it in the hands of the users.*

364.

*I believe that the hardest part of software projects, the most common source of project failure, is communication with the customers and users of that software. By providing a clear yet precise language to deal with domains, a DSL can help improve this communication.*

365.

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

(voir énoncé p.209)

### Explication de l’assertion AM09 : Le nombre de langages formels ne fait qu’augmenter.

En 1972, dans un article intitulé *Programming Languages : History and Future*, l’auteur Jean E. Sammet effectue une revue des langages de programmation. Elle souligne que, seulement, aux États-Unis, 170 langages sont utilisés. [353] Deux raisons sont données pour expliquer autant de langages :

[Traduction libre] Certains langages suscitent indéniablement un engouement qui les rend populaires – parfois jusqu’au fanatisme – malgré les difficultés. C’est l’équivalent du « charisme politique » qui influence souvent les résultats électoraux.[...] En résumé, il existe deux raisons principales pour lesquelles un langage est considéré comme important : d’une part, il est économiquement pratique et donc très utile, et d’autre part, il est techniquement nouveau.<sup>366</sup> [353]

En 2012, une revue analyse des articles abordant les DSL et les ADL. Comme l’illustre la figure B.30, le nombre d’articles portant sur ce sujet augmente considérablement au fil des années. En plus, la vaste majorité des articles proposent des solutions consistant à développer de nouveaux langages (figure B.31).

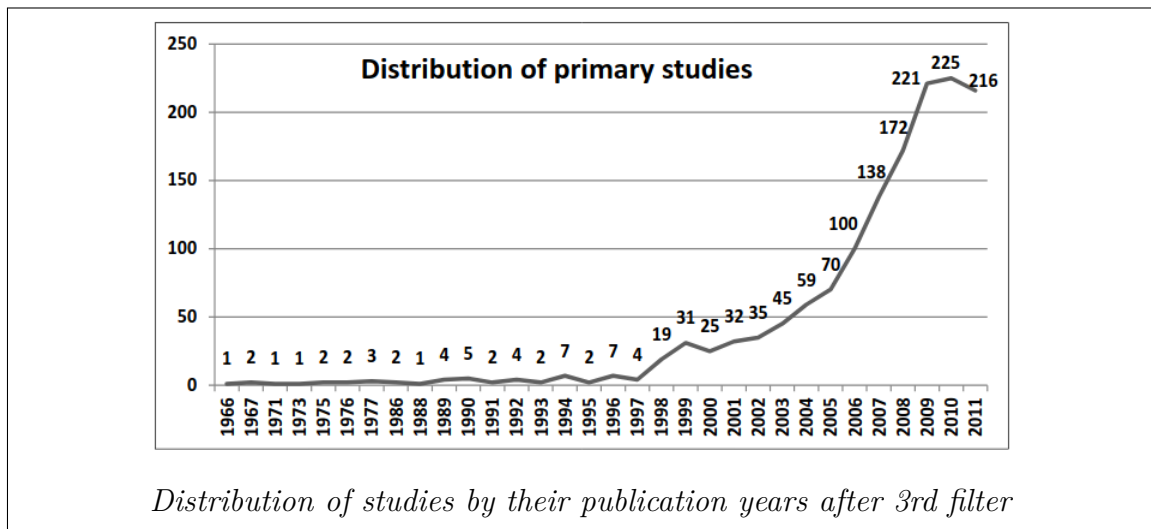
---

*The SysML provides a common metamodel that can be used to establish common grounds. However, as pointed out, its “out-of-the-box” form complexity is overwhelming. Therefore, projects and organizations are advised to create their own adapted DSL-based interpretation.*

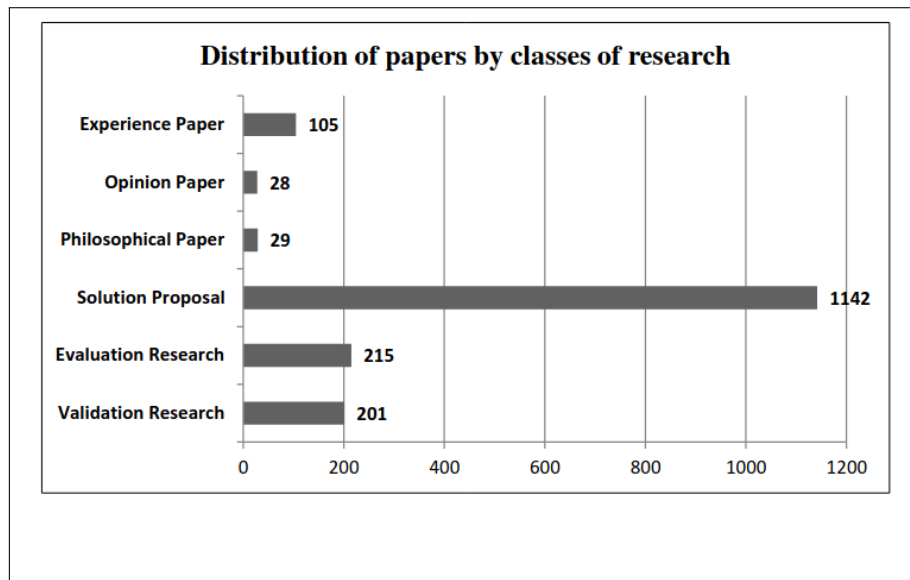
366.

*Some languages clearly create a spark, which causes the languages to become popular—sometimes to a level of fanaticism—regardless of the difficulties. This is equivalent to the “political charisma” which often affects election results.[...] In summary, there are really two major reasons for a language to be considered significant : one is that it is economically practical and hence very useful, and the other is that it is technically new.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.30** – Nombre d’articles sur les DSL et les ADL entre 1996 et 2011 (source : *Figure 2* [227])



**FIGURE B.31** – Distribution des articles sur les DSL et les ADL selon les catégories (source : *Figure 3* [227])  
(voir énoncé p.209)

**Explication du théorème AM10 :** Les classes de langages formels sont reliées aux classes d’automates.



## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

Un bref aperçu de l'histoire des automates, incluant l'étude des grammaires formelles, est présenté dans l'ouvrage de John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman :

[Traduction libre]

- Avant l'avènement des ordinateurs dans les années 1930, A. Turing a étudié une machine abstraite possédant toutes les capacités des ordinateurs actuels, du moins en ce qui a trait aux calculs qu'ils pouvaient effectuer.
- Dans les années 1940 et 1950, des machines plus simples, qu'on appelle aujourd'hui automates finis, ont été étudiées par plusieurs chercheurs.
- [...] à la fin des années 1950, le linguiste N. Chomsky entreprit l'étude des « grammaires » formelles. [...] ces grammaires sont étroitement liées aux automates abstraits et constituent aujourd'hui la base de certains composants logiciels importants, notamment des parties de compilateurs.
- En 1969, S. Cook a approfondi les travaux de Turing sur les limites du calcul. <sup>367</sup> [122]

Un historique de la relation entre les langages formels et les automates provenant de l'article *Chomsky between revolutions* permet d'observer ce rôle en informatique :

[Traduction libre]

[—] Backus (1960) et Naur et al. (1960) ont défini une métasyntaxe pour spécifier la syntaxe des langages de programmation (initialement ALGOL), connue sous le nom de forme Backus-Naur (BNF).

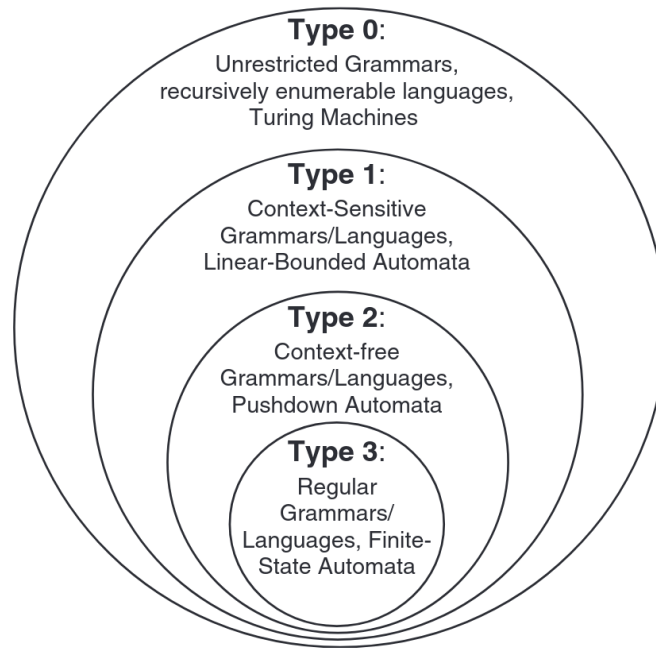
[—] Ginsburg et Rice (1962) ont démontré qu'ALGOL était équivalente à un langage hors contexte (ou de type 2, dans la typologie de Chomsky 1959b) ; [...]

---

367.

- *Before there were computers in the 1930's, A Turing studied an abstract machine that had all the capabilities of today's computers at least as far as in what they could compute.*
- *In the 1940's and 1950's simpler kinds of machines which we today call finite automata were studied by a number of researchers.*
- *[...] late 1950's, the linguist N Chomsky began the study of formal "grammars". While not strictly machines these grammars have close relationships to abstract automata and serve today as the basis of some important software components including parts of compilers.*
- *In 1969, S. Cook extended Turing's study of what could and what could not be computed.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.32** – Classes de langages formels (Hiérarchie de Chomsky) (source : [164])

[—] Kuroda (1964) a démontré que chaque langage de la typologie de Chomsky 1959b (désormais appelée « hiérarchie de Chomsky ») correspond à un type spécifique d'automate. <sup>368</sup> [126]

Les langages sont reliés aux machines abstraites, la hiérarchie de Chomsky en est l'illustration représentée par la figure B.32.

(voir énoncé p.209)

**Explication de l'assertion AM11 : Plusieurs méthodes considèrent le modèle comme un artefact avec, potentiellement, une syntaxe ou une sémantique formelle.**

---

368.

[—] Backus (1960) and Naur et al. (1960) defined a metasyntax for specifying the syntax of programming languages (originally ALGOL) that became known as Backus-Naur Form (BNF).

[—] Ginsburg and Rice (1962) demonstrated that ALGOL was equivalent to a context-free (or Type 2, in the typology of Chomsky 1959b) language; [...]

[—] Kuroda (1964) succeeded in showing that each of the languages in the typology of Chomsky 1959b (now known as the “Chomsky hierarchy”) corresponds to a specific kind of automaton.

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

En 2017, dans l'ouvrage *Conceptual Modeling Perspectives*, John Krogstie décrit le changement de tendance qui s'est opéré en modélisation :

[Traduction libre] Alors que les techniques de modélisation étaient traditionnellement utilisées pour créer des artefacts intermédiaires dans l'analyse et la conception de systèmes, les méthodologies de modélisation modernes confèrent aux modèles un rôle plus actif. Par exemple, dans la gestion des processus métier (BPM) [6], l'architecture pilotée par les modèles (MDA) et l'ingénierie logicielle pilotée par les modèles (MDSE) [3], la modélisation spécifique au domaine (DSM) [8], l'architecture d'entreprise (EA) [21], la modélisation d'entreprise (EM) [31], les modèles interactifs [19] et la modélisation active des connaissances (AKM) [22, 23], les modèles sont utilisés directement au sein du système d'information de l'organisation.<sup>369</sup> [45]

Dans *Model-Driven Software Development : Technology, Engineering, Management*, les auteurs décrivent l'UML comme n'étant pas défini formellement contrairement au MDA :

[Traduction libre] Les modèles UML ne sont pas, en soi, des modèles MDA. La principale différence entre les modèles UML courants (par exemple, les modèles d'analyse) et les modèles MDA réside dans le fait que la signification (sémantique) des modèles MDA est définie formellement. Ceci est garanti par l'utilisation d'un langage de modélisation correspondant, généralement réalisé par un profil UML et ses règles de transformation associées.<sup>370</sup> [391]

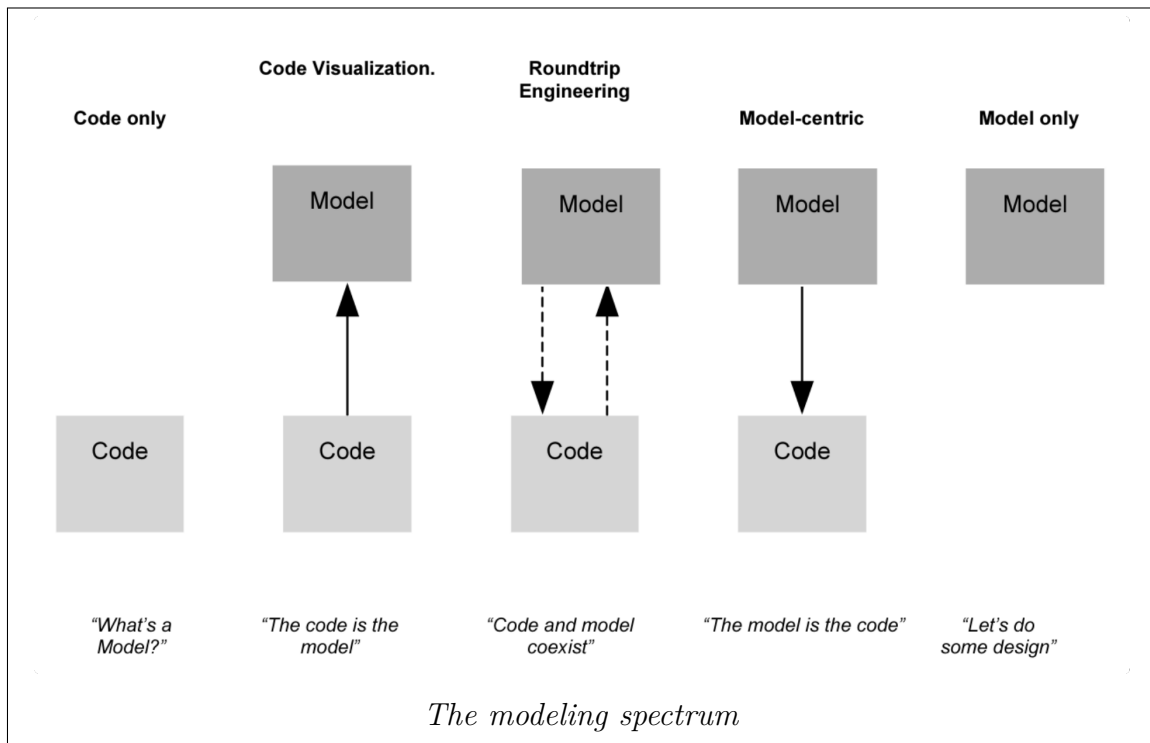
---

369.

*Whereas modeling techniques traditionally have been used to create intermediate artifacts in systems analysis and design, modern modeling methodologies support a more active role for the models. For instance in Business Process Management (BPM) [6], Model Driven Architecture (MDA) and Model-driven Software Engineering (MDSE) [3], Domain specific modeling (DSM) [8], Enterprise Architecture (EA) [21], Enterprise modeling (EM) [31], Interactive Models [19] and Active Knowledge Modelling (AKM) [22, 23], the models are used directly as part of the information system of the organization.*

370.

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.33** – Représentation de la dualité entre le modèle et le code source (source : Figure 1 [28])

(voir énoncé p.209)

**Explication de l’assertion AM12 :** Certaines méthodes de modélisation consistent à ramener le modèle à un artefact qu’il est possible de transformer.

En 2005, un article décrit la modélisation en informatique appliquée grâce à une fi-

---

*UML models are not per se MDA models. The most important difference between common UML models (for example analysis models) and MDA models is that the meaning (semantics) of MDA models is defined formally. This is guaranteed through the use of a corresponding modeling language which that is typically realized by a UML profile and its associated transformation rules.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

gure (figure B.33) illustrant cinq cas où différentes transformations possibles. Voici la description de ces cas :

[Traduction libre]

[—] *code only* : [...] ils expriment leur modèle du système qu'ils construisent directement dans une troisième génération de langage de programmation [...].

[—] *code visualization* : [...] Alors que les développeurs créent ou analysent une application, ils souhaitent souvent visualiser le code à l'aide d'une notation graphique qui facilite leur compréhension de la structure ou du comportement du code.

[—] *roundtrip engineering* : [...] Lorsque des modifications sont apportées au code, elles doivent à un moment donné être réconciliées avec le modèle original [...]

[—] *model-centric* : [...] les modèles du système sont établis avec suffisamment de détails pour que l'implémentation complète du système puisse être générée à partir des modèles eux-mêmes.

[—] *model only* : [...] les développeurs utilisent les modèles uniquement comme outils de réflexion pour comprendre le domaine d'affaires ou la solution, ou pour analyser l'architecture d'une solution proposée. <sup>371</sup> [28]

L'auteur ajoute qu'aujourd'hui, [Traduction libre] « la majorité des développeurs de logiciels adoptent encore une approche exclusivement axée sur le code [...] » <sup>372</sup> [28]

---

371.

[—] *code only* : [...] *they express their model of the system they are building directly in a third-generation programming language [...].*

[—] *code visualization* : [...] *As developers create or analyze an application they often want to visualize the code through some graphical notation that aids their understanding of the code's structure or behavior.*

[—] *roundtrip engineering* : [...] *As changes are made to the code they must at some point be reconciled with the original model [...]*

[—] *model-centric* : [...] *models of the system are established in sufficient detail that the full implementation of the system can be generated from the models themselves.*

[—] *model only* : [...] *developers use models purely as thought aids in understanding the business or solution domain, or for analyzing the architecture of a proposed solution.*

372. « [...] *a majority of software developers still take a code-only approach [...].* »

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

Une autre classification des instruments transformations est : *model-to-model* (M2M), *model-to-text* (M2T) et *text-to-model* (T2M) [27, 155]. Voici ce qu'un article décrit sur ces transformations :

[Traduction libre] Le résultat d'une transformation M2M est une instance d'un métamodèle cible, tandis que les approches M2T utilisent généralement des grammaires cibles pour décrire la structure de leur sortie textuelle. Les outils de transformation T2M, tels que MoDisco [40], acceptent des représentations textuelles en entrée et produisent des modèles de sortie décrits par des métamodèles. Les outils T2M sont le plus souvent utilisés pour la rétro-ingénierie et reposent généralement sur des technologies de compilation (comme la génération d'analyseurs syntaxiques) plutôt que sur des technologies de modélisation (comme la métamodélisation).<sup>373</sup> [155]

(voir énoncé p.209)

**Explication de l'assertion AM13 : Les artefacts concrétisent le modèle, mais ce n'est pas le modèle.**

Axel van Lamsweerde soulève des problèmes sur les spécifications d'exigences formelles, dont l'un concerne le passage du non formel au formel, il indique cela ne se résume pas à une simple traduction :

[Traduction libre] La spécification formelle ne se limite pas à une simple traduction de l'informel vers le formel. La spécification d'un système vaste et complexe exige l'identification, la mise en relation et la caractérisation des objets et phénomènes pertinents par leurs propriétés d'intérêt. La

---

373.

*The output of a M2M transformation is an instance of a target metamodel, whereas M2T approaches typically use target grammars to describe the structure of their textual output. T2M transformation tools, such as MoDisco [40], accept text representations as input and produce output models described by metamodels. T2M tools are most often used for reverse engineering, and are usually based on compiler technology (such as parser generation), rather than modeling technology (such as metamodeling).*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

construction du modèle et la description des propriétés sont donc des composantes étroitement liées de tout processus de spécification à grande échelle.<sup>374</sup> [170]

Un article décrit l'histoire derrière le paradigme orienté-objet. Cela remonte jusqu'à la création de Simula présenté en 1963. Ole-Johan Dahl et Kristen Nygaard utilisent les concepts d'objets, de classes et d'héritage pour créer un modèle pouvant représenter la fission nucléaire :

[Traduction libre] L'intérêt de Nygaard pour la modélisation des phénomènes liés à la fission nucléaire a conduit à concevoir Simula comme un langage de description de processus autant que comme un langage de programmation. [...] Il est important de noter que la modélisation signifie que les actions et interactions des objets créés par le programme reproduisent les actions et interactions des objets du monde réel qu'ils sont conçus pour simuler. Ce n'est pas le code Simula qui modélise le système réel, mais les objets créés par ce code.<sup>375</sup> [30]

(voir énoncé p.209)

**Explication de l'assertion AM14 : Les pratiques réflexives permettent d'obtenir les modèles recherchés.**

---

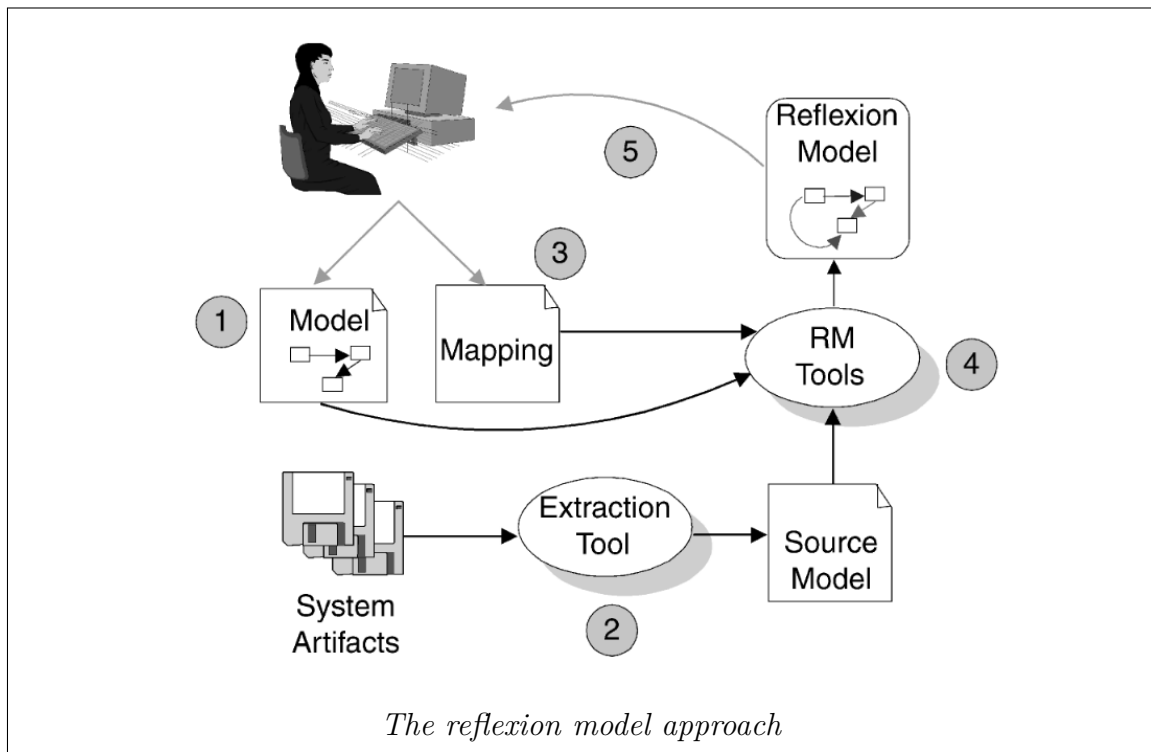
374.

*Formal specification is not a mere translation process from informal to formal. The specification of a large, complex system requires relevant objects and phenomena to be identified, interrelated, and characterized through properties of interest. Model construction and property description are thus tightly coupled components of any specification-in-the-large process.*

375.

*Nygaard's concern with modelling the phenomena associated with nuclear fission meant that Simula was designed as a process description language as well as a programming language. [...] Notice that modelling means that the actions and interactions of the objects created by the program model the actions and interactions of the real-world objects that they are designed to simulate. It is not the Simula code that models the real-world system, but the objects created by that code.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.34** – Procédé réflexif pour faire la rétro-ingénierie d'un modèle formel (source : [224])

Un article publié en 2001 décrit l'utilisation d'un modèle réflexif dans le cadre de la ré-ingénierie du produit Microsoft Excel [224]. La figure B.34 illustre le cycle pour la production du modèle ainsi que l'intervention des individus. Les avantages d'une pratique réflexive sont :

[Traduction libre]

- [...] léger et itératif, car un ingénieur peut calculer rapidement et facilement un modèle de réflexion, puis affiner sélectivement les entrées et recalculer les modèles jusqu'à obtenir les informations souhaitées, [...].
- [...] approximatif et partiel, car la correspondance utilisée dans le calcul d'un modèle de réflexion peut associer grossièrement les entités du modèle source aux entités du modèle de haut niveau et peut ne pas inclure toutes



## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

les entités d'un modèle source ou de haut niveau donné dans un calcul particulier. <sup>376</sup> [224]

L'auteur conclut que :

[Traduction libre] La technique de modélisation par réflexion logicielle offre un moyen de combler le fossé entre les modèles de haut niveau couramment utilisés par les ingénieurs pour raisonner sur un système logiciel et les artefacts du système qui constituent ce système logiciel. <sup>377</sup> [224]

Le procédé Balzer provenant de l'ouvrage *Rigorous software development : An introduction to program verification* met de l'avant qu'il est possible d'utiliser les méthodes formelles en amont d'un procédé de développement. D'ailleurs, plusieurs cycles s'y trouvent (figure B.35), illustrant une pratique réflexive lors de la construction. Il s'insère au début du procédé de développement :

[Traduction libre] Au début des années 80, Balzer et ses collègues ont proposé une notion de cycle de vie du logiciel [...] par rapport aux autres cycles de développement logiciel, car elle préconise explicitement l'utilisation de méthodes formelles à différentes étapes, et en particulier en ce qui concerne la relation entre les exigences, le modèle et l'implémentation. <sup>378</sup> [21]

---

376.

- [...] *lightweight and iterative, in that an engineer can quickly and easily compute a reflexion model and then may selectively refine the inputs and recompute models until the desired information is obtained, [...].*
- [...] *approximate and partial, in that the map used in the computation of a reflexion model may coarsely associate source model entities with high-level model entities and may not include all entities in a given source or high-level model in any particular computation.*

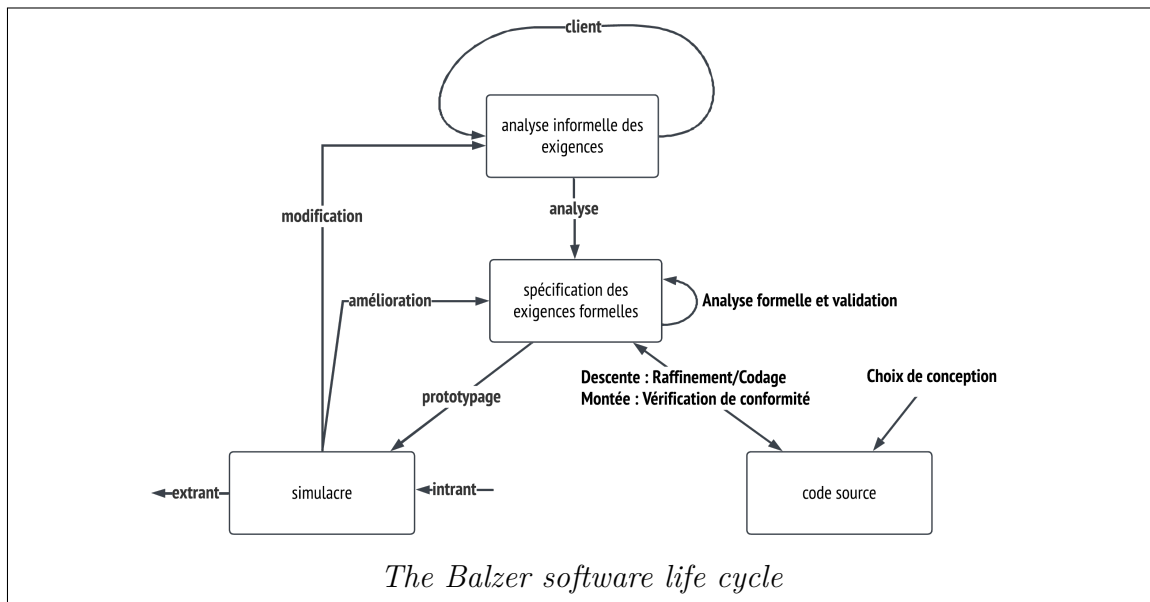
377.

*The software reflexion model technique provides a means of bridging the gap between the high-level models commonly used by engineers to reason about a software system and the system artifacts that are the software system.*

378.

*In the early 80's, Balzer and colleagues proposed a notion of software life cycle [...] with respect to other software development cycles is that it explicitly advocates the use of formal methods at different stages, and in particular where the relationship between the requirements, the model, and the implementation is concerned.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.35** – Procédé réflexif pour créer un modèle formel [Traduction libre] (modifiée de [21])

(voir énoncé p.209)

### Des techniques de modélisation

**Explication de l’assertion AM15 : Le BPMN est un exemple d’un langage formel avec une représentation textuelle et graphique.**

En 2011, un article décrit le *Business Process Model and Notation* (BPMN) :

[Traduction libre] L’objectif principal de BPMN est de fournir une notation facilement compréhensible par les utilisateurs métiers, allant des analystes métiers qui esquissent les premières versions des processus aux développeurs techniques chargés de les mettre en œuvre, et enfin au personnel métier qui déploie et surveille ces processus [37].<sup>379</sup> [55]

379.

*The primary goal of BPMN is to provide a notation that is readily understandable by business users, ranging from the business analysts who sketch the initial drafts of the*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

En 2018, un article examine le support de l'exécution du BPMN 2.0 avec différents instruments et décrit les avantages du BPMN 2.0 :

[Traduction libre] Avec la publication de BPMN 2.0 en janvier 2011, la prise en charge de l'exécution native des modèles de processus BPMN et un format de sérialisation standardisé a été ajoutée à la norme. L'objectif de cet ajout est de réduire l'écart entre la modélisation des processus d'une part et leur exécution par logiciel d'autre part [4].<sup>380</sup> [99]

Le standard du BPMN comprend plusieurs descriptions formelles en format XSD et CMOF (*Complete Meta-Object Facility*) [235]. En voici quelques-unes :

- <http://www.omg.org/spec/BPMN/20100501/BPMN20.cmf>;
- .../BPMN20.xsd ;
- .../Semantic.xsd ;
- .../Semantic.cmf.

Cet article décrit la différence entre MOF et CMOF, une différence importante est que CMOF intègre l'UML, ce qui peut mettre en doute le formalisme employé dans le CMOF :

[Traduction libre] Il est important de noter que MOF est présenté dans sa spécification en deux versions : Essential MOF (EMOF), qui constitue le noyau de concepts nécessaires à la construction de métamodèles, et Complete MOF (CMOF), qui n'ajoute pas de nouveaux concepts, mais fusionne EMOF avec les définitions fondamentales d'UML. [...] MOF (Meta-Object Facility) [6]. La spécification qui a créé la norme pour l'échange de métadonnées, et, par conséquent, le langage des métamodèles eux-mêmes.

---

*processes to the technical developers responsible for actually implementing them, and finally to the business staff deploying and monitoring such processes [37].*

380.

*With the publication of BPMN 2.0 in January 2011, support for the native execution of BPMN process models and a standardized serialization format has been added to the standard. The intention underlying this addition is to mitigate the gap between the modeling of processes on the one hand and their execution through software on the other hand [4].*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

Elle a été créée à partir des fondements de modélisation d'UML et comprend deux métamodèles : Essential MOF (EMOF) et Complete MOF (CMOF).<sup>381</sup> [46]

(voir énoncé p.209)

### Explication de l'assertion AM16 : Les instruments ne permettent pas de profiter du formalisme du BPMN.

En 2018, un article examine le support de l'exécution du BPMN 2.0 avec différents instruments :

[Traduction libre] Nous étudions comment les implémenteurs de BPMN 2.0 gèrent la norme, et montrons que l'exécution native de BPMN 2.0 est l'exception. Seuls trois des 47 systèmes de gestion de processus métier (BPMS) considérés prennent en charge le format d'exécution défini dans la norme, bien qu'ils prétendent tous s'y conformer.<sup>382</sup> [99]

En 2020, un article publie les résultats d'une analyse syntaxique sur des milliers d'artefacts BPMN disponibles sur GitHub. Trois résultats sont rapportés :

[Traduction libre]

---

381.

*It is important to notice that MOF is presented in its specification in two versions, namely Essential MOF (EMOF), which is the core of concepts necessary for the construction of metamodels, and Complete MOF (CMOF), which doesn't add new concepts but merges EMOF with the core definitions of UML. [...] MOF (Meta-Object Facility) [6]. The specification that created the standard for the exchange of metadata, therefore creating the language for metamodels themselves. It was created from the modeling foundations of UML and comprises two metamodels, Essential MOF (EMOF) and Complete MOF (CMOF).*

382.

*We investigate how BPMN 2.0 implementers deal with the standard, showing that native BPMN 2.0 execution is an exception. Only three out of 47 BPMS considered support the execution format defined in the standard, although all of them claim to comply to the BPMN 2.0 standard.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

[—] 1 251 dépôts contiennent au moins un artefact potentiel de modèle de processus BPMN, soit 21 306 artefacts potentiels et 16 907 modèles BPMN 2.0 sérialisés.

[—] Bien que la majorité des artefacts identifiés soient des contenus statiques et récents d'un dépôt de logiciels, nous avons également trouvé des artefacts datant de plus de 8 ans, mis à jour à plusieurs reprises et provenant de différentes régions géographiques. Au moins la moitié des artefacts potentiels de modèles de processus BPMN et la moitié des modèles de processus sérialisés sont distincts, ce qui constitue un corpus de 8 904 modèles de processus BPMN 2.0 uniques. Ce corpus contient des modèles de tailles variées.

[—] Les violations des règles de syntaxe et de sémantique de BPMN, telles que signalées par BPMNspector, sont fréquentes dans le corpus de modèles de processus BPMN, affectant 82,8 % des modèles.

[—] Nous avons également examiné de plus près les problèmes signalés et constaté qu'une grande partie d'entre eux constituaient des violations des règles Ext.023, Ext.101 et Ext.107. Les règles Ext.023, Ext.101 et Ext.107 font référence à la définition non conforme des flux de séquence dans le format de sérialisation BPMN 2.0 [...] <sup>383</sup> [114]

(voir énoncé p.209)

---

383.

[—] *There have been 1,251 repositories with at least one potential BPMN process model artifact, containing 21,306 potential artifacts and 16,907 serialized BPMN 2.0 models.*

[—] *While the majority of identified artifacts are static and recent contents of a software repository, we also found artifacts which are older than 8 years, updated more than once and come from different geographical regions. At least half of the potential BPMN process model artifacts and half of the serialized process models are distinct, yielding a corpus of 8,904 unique BPMN 2.0 process models. The corpus contains models of various sizes.*

[—] *Violations of BPMN's syntax and semantic rules as reported by BPMNspector are frequent in the corpus of BPMN process models, affecting 82,8% of the models.*

[—] *We also took a closer look into the reported issues and found a large fraction to be violations of rules Ext.023, Ext.101 and Ext.107 [...] Rules Ext.023, Ext.101, Ext.107 refer to the non-compliant definition of sequence flows in the BPMN 2.0 serialization format [...]*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

**Explication de l'axiome AM17 : Certaines méthodes formelles regroupent des techniques, tandis que d'autres constituent elles-mêmes des techniques.**

Dans l'ouvrage *Logics of Specification Languages* [29], Dines Bjørner décrit le *The Vienna Development Method* (VDM) comme une collection de techniques :

- [Traduction libre] « On peut probablement attribuer à VDM le mérite d'avoir été le premier langage de spécification formel (1974). »<sup>384</sup> [29] ;
- [Traduction libre] « La méthode de développement viennoise (VDM) est un ensemble de techniques pour la modélisation, la spécification et la conception de systèmes informatiques. »<sup>385</sup> [29]

Jean-François Monin dans l'ouvrage *Understanding formal methods* affirme qu'il s'agit de techniques, mais que dans l'usage, le terme **méthode** s'est répandu :

[Traduction libre] Ces méthodes fournissent, en substance, un cadre rationnel composé d'outils facilitant la modélisation et le raisonnement, mais elles n'apportent que peu d'éléments sur le plan méthodologique. Nous utiliserons le terme méthode formelle, car il est bien établi, mais « technique formelle » serait certainement plus approprié. Le domaine des techniques de compilation constitue peut-être une exception. Pour construire un compilateur, on définit d'abord la grammaire du langage source à l'aide de règles formelles appropriées. Après une éventuelle transformation de cette dernière, un analyseur syntaxique efficace est automatiquement dérivé grâce à des mécanismes généraux établis dans les années 1960. Nous avons ici tous les ingrédients d'une méthode formelle. Premièrement, nous disposons évidemment d'un langage formel permettant de décrire précisément les règles grammaticales : la BNF, forme normale de Backus-Naur.<sup>386</sup> [150]

---

384. « *VDM can probably be credited as being the first formal specification language (1974).* »

385. « *The Vienna Development Method (VDM), is a collection of techniques for the modelling, specification and design of computer-based systems.* »

386.

*Such methods provide, essentially, a rational framework composed of tools to aid in modeling and reasoning, but they don't bring much from a methodological perspective. We will use the term formal method, because it is well established, but formal technique would certainly be more appropriate. The domain of compilation techniques may be an*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

Une description de méthode formelle provenant du manuel *Formal Methods in Computer Science* indiquant que ce sont le regroupement de techniques :

[Traduction libre] Les méthodes formelles sont des techniques de spécification, de développement et de vérification des systèmes logiciels et matériels. Une méthode formelle est une méthode mathématique. En élaborant un modèle mathématique rigoureux d'un système, il est possible de vérifier ses propriétés de manière plus approfondie que par des tests empiriques. On considère que l'application de méthodes formelles à la conception et au développement de systèmes accroît leur fiabilité et leur robustesse.<sup>387</sup>  
[393]

(voir énoncé p.209)

**Explication de l'assertion AM18 : Les méthodes formelles comprennent l'utilisation de langages formels et de systèmes logiques.**

Dines Bjørner, dans l'ouvrage *Logics of specification languages*, souligne toute l'importance d'utiliser un système logique, car, sans lui, un langage formel perd de son utilité :

[Traduction libre] Nous considérons comme allant de soi que toute spécification formelle doit permettre d'obtenir des conséquences précises :

---

*exception. In order to construct a compiler, first the grammar of the source language is defined using suitable formal rules. After a possible transformation of the latter, an efficient parser is automatically derived thanks to general mechanisms determined in the 1960s. We have here all of the ingredients of a formal method. First, we obviously have a formal language for describing the grammar rules in a precise manner - a BNF, normal form of Backus-Naur.*

387.

*Formal methods are techniques for the specification, development, and verification of software and hardware systems. A formal method is a mathematical method. By building a mathematically rigorous model of a system, it is possible to verify the system's properties in a more thorough fashion than empirical testing. It is believed that applying formal methods in system design and development will increase the reliability and robustness of the system.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

l'accent, dans l'expression « méthode formelle », doit porter sur le second mot, et non sur le premier. Un langage, même doté d'une sémantique, est appauvri s'il est dépourvu de logique : il ne fournirait aucun moyen d'obtenir ces conséquences de manière méthodique, reproductible et consensuelle.<sup>388</sup> [29]

Jean-Raymond Abrial explique dans son ouvrage *Modeling in Even-B* ses préférences en langages formels et en systèmes logiques :

[Traduction libre] Les méthodes formelles sont des techniques utilisées pour élaborer des schémas adaptés à notre discipline. Ces schémas sont appelés modèles formels. Comme pour les schémas réels, nous utiliserons des conventions prédéfinies pour écrire nos modèles. Il est inutile d'inventer un nouveau langage ; nous utiliserons celui de la logique classique et de la théorie des ensembles. Ces conventions nous permettront de communiquer facilement nos modèles, car ces langages sont connus de toute personne ayant des connaissances en mathématiques. L'utilisation d'un tel langage mathématique nous permettra de raisonner sous forme de démonstrations mathématiques, que nous effectuerons comme d'habitude. Notons encore une fois que, comme pour les schémas réels, la base fait défaut ; notre modèle ne sera donc généralement pas exécutable. [...] <sup>389</sup> [2]

---

388.

*We take it as self-evident that any formal specification should permit precise consequences to be obtained : the emphasis in the term formal method should fall on the second word and not the first. A language, even one with a semantics, is impoverished if there is no logic : it would provide no means for obtaining those consequences in a methodical, reproducible and agreed fashion.*

389.

*Formal methods are techniques used to build blueprints adapted to our discipline. Such blueprints are called formal models. As for real blueprints, we shall use some pre-defined conventions to write our models. There is no point in inventing a new language ; we are going to use the language of classical logic and set theory. These conventions will allow us to easily communicate our models to others, as these languages are known by everyone having some mathematical backgrounds. The use of such a mathematical language will allow us to do some reasoning in the form of mathematical proofs, which we shall conduct as usual. Note again that, as with blueprints, the basis is lacking ; our model will thus not in general be executable. [...]*



## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

(voir énoncé p.210)

### Explication de l’assertion AM19 : Les méthodes formelles sont définitivement utiles.

Dans l’ouvrage de Markus Roggenbach, l’auteur présente des exemples de réussites [340] :

- *Model Checking at Intel* ;
- *Microsoft’s Protocol Documentation Program* ;
- *Electronic Voting* ;
- *The Operating Systems seL4 and PikeOS* ;
- *Model-Based Design with Certified Code Generation* ;
- *Transportation Systems in France*.

Un article, intitulé *The 2020 Expert Survey on Formal Methods*, présente les résultats d’un sondage auquel ont participé cent trente experts en méthodes formelles. Ces derniers ont répondu à des questions portant sur l’utilisation combinée de méthodes formelles et d’outils d’analyse formelle, les résultats sont présentés dans le tableau B.13. Un article qui affirme la nécessité des méthodes formelles qui serait indissociable de la profession d’informaticien ou d’ingénieur logiciel :

[Traduction libre] Tout informaticien a-t-il besoin de connaître les méthodes formelles ? Notre position, étayée par les arguments développés dans cet article, est affirmative. De plus, un développeur logiciel ignorant les nombreux avantages des méthodes formelles ne peut être considéré comme un informaticien ou un ingénieur logiciel.<sup>390</sup> [42]

(voir énoncé p.210)

---

390.

*Does every computer scientist need to know formal methods? Our stance, supported by the arguments made throughout this article, is “yes, they do”. Even more, software developers not being aware of the various benefits of formal methods cannot be called computer scientists or software engineers.*

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE

**TABLEAU B.13** – Bénéfices attendus de l'utilisation des méthodes formelles (source : [378])

| <i>Expected Benefits</i>                                                                                                        |            |          |              |                |       |
|---------------------------------------------------------------------------------------------------------------------------------|------------|----------|--------------|----------------|-------|
| <i>Do you believe that formal methods, together with the rigorous use of formal analysis tools, can deliver the promise of:</i> |            |          |              |                |       |
|                                                                                                                                 | Definitely | Probably | Probably not | Definitely not | N / A |
| Better software quality                                                                                                         | 81.5%      | 16.9%    | 0.8%         | 0.0%           | 0.8%  |
| Improved system safety                                                                                                          | 92.3%      | 7.7%     | 0.0%         | 0.0%           | 0.0%  |
| Enhanced cybersecurity                                                                                                          | 65.4%      | 31.5%    | 0.8%         | 0.0%           | 2.3%  |
| Higher performance systems                                                                                                      | 27.7%      | 46.2%    | 19.2%        | 0.0%           | 6.9%  |
| Cheaper software development                                                                                                    | 19.2%      | 40.8%    | 30.0%        | 5.4%           | 4.6%  |
| Reduced time to market                                                                                                          | 19.2%      | 37.7%    | 31.5%        | 4.6%           | 6.9%  |
| Easier certification                                                                                                            | 61.5%      | 35.4%    | 2.3%         | 0.0%           | 0.8%  |
| Easier long-term maintenance                                                                                                    | 60.0%      | 36.9%    | 2.3%         | 0.0%           | 0.8%  |

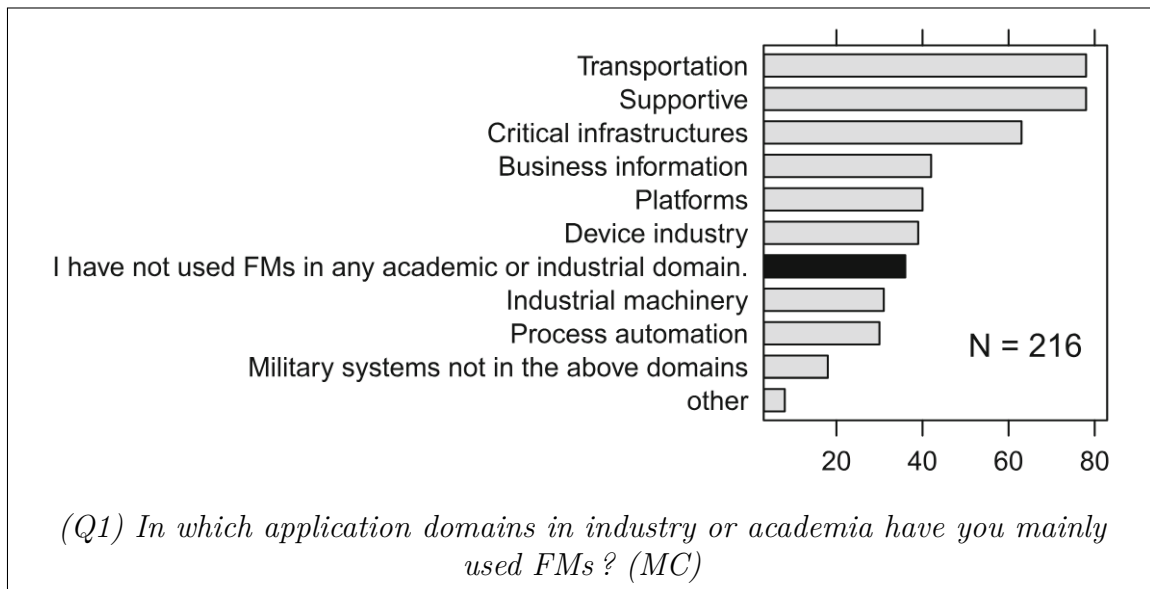
**Explication de l'assertion AM20 : Les méthodes formelles ne s'imposent pas largement.**

En 2022, Markus Roggenbach, dans l'ouvrage *Formal Methods for Software Engineering : Languages, Methods, Application Domains*, expose des articles qui déboulonnent les idées préconçues sur les méthodes formelles dans l'objectif de convaincre de potentiels utilisateurs et, du même coup, de les aiguiller dans l'usage des méthodes formelles [340] :

- *Seven Myths of Formal Methods* (1990) ;
- *Seven more myths of Formal Methods, Ten Commandments of Formal Methods* (1995) ;
- *Ten Commandments of Formal Methods ... Ten Years Later* (2006).

Un sondage de 2020 mène sur l'utilisation de méthode formelle. Plusieurs résultats sur l'utilisation de méthodes formelles par secteur se retrouvent dans la figure B.36. L'un des résultats les plus notables est le nombre de personnes ( $n = 216$ , 40 personnes) n'utilisant pas de méthodes formelles. Ce sondage vise des individus en lien avec

## B.2. DES MODÈLES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.36** – Secteurs où des méthodes formelles sont utilisés par des professionnels (source : *Figure 2* [103])

le secteur critique, un secteur qui est réputé comme étant plus enclin à utiliser des méthodes :

[Traduction libre] Notre groupe cible pour cette enquête comprend des personnes ayant (1) une formation en ingénierie et en sciences liées aux systèmes informatiques critiques ou aux systèmes à forte composante logicielle, de préférence ayant obtenu leur premier diplôme, ou (2) une expérience pratique en ingénierie dans un domaine de système ou de produit raisonnablement critique impliquant la pratique du logiciel.<sup>391</sup> [103]

L'article *The 2020 Expert Survey on Formal Methods*, qui rapporte les résultats d'un sondage mené auprès de cent trente experts en méthodes formelles, propose un tableau (tableau B.14) recensant les raisons évoquées par les répondants pour expliquer la

391.

*Our target group for this survey includes persons with (1) an educational background in engineering and the sciences related to critical computer-based or software-intensive systems, preferably having gained their first degree, or (2) a practical engineering background in a reasonably critical system or product domain involving software practice.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

faible utilisation des méthodes formelles. La formation et l'expertise en méthodes formelles trônent au sommet.

**TABLEAU B.14** – Raisons à la faible utilisation des méthodes formelles. (source : [378])

| <i>What are the limiting factors for a wider adoption of formal methods by industry?</i> |       |
|------------------------------------------------------------------------------------------|-------|
| Engineers lack proper training in formal methods                                         | 71.5% |
| Academic tools have limitations and are not professionally maintained                    | 66.9% |
| Formal methods are not properly integrated in the industrial design life cycle           | 66.9% |
| Formal methods have a steep learning curve                                               | 63.8% |
| Developers are reluctant to change their way of working                                  | 62.3% |
| Managers are not aware of formal methods                                                 | 57.7% |
| Many companies do not pay enough attention to software quality and security              | 56.2% |
| Formal methods are not properly integrated in standard certification processes           | 46.9% |
| Formal methods focus on relevant problems, but only on a small part of all problems      | 36.9% |
| Benefits of formal methods are not marketed enough                                       | 36.9% |
| There are too many formal methods, with a lack of independent comparison                 | 28.5% |
| Formal methods are too costly, with no perceived immediate added value                   | 26.9% |
| Formal methods are too slow to meet current time-to-market constraints                   | 17.7% |
| Professional tools are too expensive because of the small market for them                | 14.6% |
| Other approaches to software quality outperform formal methods                           | 13.1% |
| Industrial software development practices change too often and too quickly               | 8.5%  |
| Formal methods focus on the wrong problems                                               | 7.7%  |
| Other                                                                                    | 13.8% |

*(from 1 to 12 answers allowed; answers sorted by frequency)*

(voir énoncé p.210)

## B.3 Des connaissances en informatique appliquée

### Explication de la définition S01 : Connaissance

La définition formulée est : une croyance justifiée qui à force de légitimités devient un fait.

Dans un article, Yves Gingras décrit la connaissance ainsi :

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

Platon s'était déjà posé la question et proposait dans son dialogue *Théétète* une définition relativement simple de la notion de connaissance. Elle consiste selon lui en une « croyance vraie et justifiée ». Depuis, les philosophes ont débattu sans fin pour savoir comment déterminer ce qui est « vrai » de manière absolue, et nombreux sont ceux qui, à l'instar de Karl Popper, considèrent qu'il faut plutôt se satisfaire d'une définition qui ne conserve que le caractère *justifié* de l'énoncé. On doit alors préciser que les justifications légitimes sont celles qui *valident l'énoncé par des méthodes généralement acceptées et potentiellement accessibles à toute personne raisonnable* [...] Ainsi, c'est la justification qui fait passer de la croyance à la connaissance. Ce qui les distingue est le fait que la connaissance a été validée par des méthodes généralement acceptées, lesquelles évoluent au fil du temps en raison même de l'accroissement des connaissances et des instruments de mesure et d'observation disponibles - une caractéristique qui l'élève au rang de « fait » bien établi. [101]

(voir énoncé p.210)

#### **Explication de la définition S02 : Savoir en l'informatique appliquée**

La définition formulée est : ensemble des connaissances s'appliquant à l'information, aux traitements de l'information et aux entités informatiques utilisé en pratique.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de savoir : « Ensemble des connaissances acquises qui ont été approfondies par une activité mentale suivie. » [275] ;
- définition 2 de savoir : « Être capable d'effectuer des tâches grâce à des connaissances théoriques ou pratiques ainsi qu'à l'expérience. » [276].

(voir énoncé p.210)

#### **Explication de la définition S03 : Gestion de la connaissance**

La définition formulée est : organiser, contrôler et rendre accessibles les représentations des connaissances au bénéfice d'une organisation.

La littérature fournit ces informations qui ont servi à construire cette définition :

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

- définition 1 de gestion de la connaissance : « Gestion de la création, du stockage et du partage collaboratif des connaissances des salariés d’une organisation pour améliorer l’efficacité, la productivité et la rentabilité. » [253] ;
- définition 2 de gestion de la connaissance : « [...] consiste à s’assurer que les compétences, l’expérience et l’expertise de l’équipe projet et des autres parties prenantes sont utilisées avant, pendant et après le projet. » [310].

(voir énoncé p.210)

**Explication de l’axiome OS01 : Une solution informatique traite un problème grâce à une machine abstraite définie par un modèle.**

Dans un article, Michael Jackson décrit l’objectif du développement logiciel qui cherche à créer une machine abstraite pour résoudre des problèmes, il écrit :

[Traduction libre] Le développement logiciel relève de l’ingénierie, car il vise à créer des dispositifs physiques utiles répondant à des besoins pratiques. Un logiciel est la description d’une machine. Nous construisons cette machine en la décrivant et en présentant cette description à un ordinateur qui adopte alors les attributs et le comportement de la machine décrite.<sup>392</sup> [148]

Daniel Jackson, dans l’ouvrage *Software Abstractions : logic, language, and analysis* utilise le terme **modèle** pour décrire ces abstractions qui sont produites lors du développement logiciel :

[Traduction libre] Le cœur du développement logiciel réside donc dans la conception d’abstractions. Une abstraction n’est ni un module, ni une interface, ni une classe, ni une méthode : c’est une structure, pure et simple – une idée réduite à son essence. [...] J’utilise le terme « modèle » pour décrire une abstraction logicielle. Ce n’est pas idéal, car une abstraction

---

392.

*Software development is engineering because it is concerned to make useful physical devices to serve practical purposes in the world. Software is a description of a machine. We build the machine by describing it and presenting our description to a general-purpose computer that then takes on the attributes and behavior of the machine we have described.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

logicielle n'est pas nécessairement un "modèle" de quoi que ce soit. Mais c'est plus court que "description", et son usage s'est largement répandu (et reste vague!).<sup>393</sup> [145]  
(voir énoncé p.210)

**Explication de l'assertion OS02 : En informatique appliquée, la conservation des connaissances passe par les participants informatiques, les processus et les artefacts, notamment les instruments.**

Pour une entreprise de conception de puces électroniques, ce sont les éléments jugés importants pour la reproductibilité :

[Traduction libre] Il est important de recueillir des données sur l'exécution et le déroulement des activités de conception dans plusieurs projets menés simultanément. Idéalement, cette collecte devrait s'appuyer sur une application de gestion des connaissances permettant de recueillir des informations sur la manière dont une séquence d'activités de conception et de vérification est liée à un type particulier d'artefact conçu, aux configurations des outils de conception utilisés et aux compétences des équipes de conception.<sup>394</sup> [394]

L'ouvrage *Managing Software Engineering Knowledge* décrit précisément les éléments qui contribuent au savoir, en les répartissant dans deux catégories : les éléments

---

393.

*The core of software development, therefore, is the design of abstractions. An abstraction is not a module, or an interface, class or method : it is a structure, pure and simple - an idea reduced to its essential form. [...] I Use the term "model" for a description of a software abstractions. It's not ideal, because a software abstraction need not be a "model" of anything. But it's shorter than "description", and has come to have a well established (and vague !) usage.*

394.

*It is important to collect data about the actual execution and sequences of design activities in several concurrent design project flows. Ideally, this should be supported by a knowledge management application that assists in eliciting the knowledge about how a sequence of design and verification activities is related to a particular type of a designed artifact, the configurations of used design tools, and the capabilities of design teams.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

communs à toutes les organisations et ceux propres aux organisations du domaine du logiciel :

- [Traduction libre]
- [pour tout type d'organisation]
- Réutilisation des ressources : [...] La réutilisation logicielle vise à réduire les reprises en établissant un référentiel de réutilisation où les programmeurs soumettent les ressources logicielles qu'ils estiment utiles à d'autres. [...] peut s'appliquer à tous les artefacts du génie logiciel, tels que les documents d'exigences, les spécifications de conception et de test. [...]
- Gestion documentaire : [...] Les documents deviennent les ressources de l'organisation en capturant les connaissances explicites. Les systèmes de gestion documentaire aident les organisations à gérer ces ressources inestimables, à faciliter le transfert de connaissances des experts aux novices et à soutenir la localisation, l'organisation et la réutilisation des connaissances documentées. [...]
- Collaboration : La technologie et les processus peuvent être utilisés pour rassembler les personnes et générer de nouvelles connaissances.
- Gestion des compétences : [...] Alors que la gestion documentaire traite des connaissances explicites, la gestion des compétences assure le suivi des connaissances tacites. [...]
- Réseaux d'experts : [...] reposent généralement sur l'entraide entre pairs et peuvent réduire le temps consacré par les ingénieurs logiciels à la recherche de connaissances spécifiques au domaine. [...]
- [pour les organisations en développement logiciel]
- Gestion de la configuration et contrôle de version : [...] assure le suivi des documents d'un projet et les relie entre eux. [...]
- Justification de la conception : [...] La justification de la conception enregistre ces informations ainsi que celles relatives aux solutions envisagées et testées, mais non mises en œuvre. [...]
- Traçabilité : [...] La traçabilité est une approche qui rend explicite le lien entre les exigences et le système logiciel final. [...]



### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

- Rapports d'incidents et suivi des anomalies : [...] Ils contiennent des connaissances sur les caractéristiques et les propriétés des produits qui posent problème aux utilisateurs, ainsi que des informations sur la gestion des réclamations antérieures par l'organisation. [...]
- Outils CASE et environnements de développement logiciel : » [...] contribuent à la création de connaissances sur les produits et les processus à travers les artefacts créés [...]. <sup>395</sup> [16]

Un article publié en 2017 décrit une proposition de solution pour la gestion de connaissances et l'évolution d'une entité informatique [72]. Quelques relations entre les participants informatiques, les artefacts et les processus sont décrites :

[Traduction libre]

---

395.

*[for any type of organization]*

- *Asset Reuse* : [...] *Software reuse aims to reduce this rework by establishing a reuse repository to which programmers submit software assets they believe would be useful to others. [...] can be applied to all software engineering artifacts, such as requirements documents, design, and test specifications. [...]*
- *Document Management* : [...] *documents become the assets of the organization in capturing explicit knowledge. Document management systems help organizations manage these invaluable assets, enable knowledge transferal from experts to novices, and support the location, organization, and reuse of documented knowledge.[...]*
- *Collaboration* : *technology and process can be used to bring people together and generate new knowledge.*
- *Competence Management* : [...] *While document management deals with explicit knowledge assets, competence management keeps track of tacit knowledge. [...]*
- *Expert Networks* : [...] *are typically based on peer-to-peer support and can reduce the time spent by software engineers in looking for specific domain knowledge. [...]*
- *[for software development organizations]*
- *Configuration Management and Version Control* : [...] *keeps track of a project documents and relates them to each other. [...]*
- *Design Rational* : [...] *Design rationale captures this information as well as information about solutions that were considered and tested but not implemented. [...]*
- *Traceability* : [...] *Traceability is an approach that makes the connection between requirements and the final software system explicit [...]*
- *Trouble Reports and Defect Tracking* : [...] *They contain knowledge about product features and properties with which users have difficulties as we has knowledge regarding the organization's management of past complaints. [...]*
- *CASE Tools and Software Development Environments* : [...] *support the creation of product and process knowledge in terms of the artifacts that are created. [...]*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

[—] Beaucoup de connaissances sont documentées, mais beaucoup restent dans l'esprit des individus, créant ainsi un potentiel écart de connaissances (organisationnel) [;]

[—] Il existe une quantité importante de documents et d'artefacts produits au cours des phases du processus de développement, enregistrant les connaissances acquises à chaque sprint ou projet.

[—] Les responsables de la maintenance logicielle valident, corrigent et mettent à jour les connaissances issues des phases précédentes du processus de développement logiciel [...]. <sup>396</sup> [72]

(voir énoncé p.210)

**Explication de l'assertion OS03 : La représentation explicite des connaissances est requise afin d'en permettre le traitement et le partage.**

Dans l'ouvrage *Managing Software Engineering Knowledge*, H. Dieter Rombach explique :

[Traduction libre] La gestion des connaissances comprend l'extraction, la mise en forme, la gestion et la réutilisation des connaissances sous toutes leurs formes. Les connaissances explicites en génie logiciel incluent tous les types d'artefacts de génie logiciel, allant des artefacts logiciels traditionnels, tels que le code, la conception et les exigences aux connaissances de processus sous forme de modèles, de données et de normes, ainsi qu'aux leçons apprises. <sup>397</sup> [16]

---

396.

[—] *A lot of knowledge is documented but a lot remains in the individuals' mind, therefore creating a potential (organizational) knowledge gap [;]*

[—] *There is a significant amount of documents and artifacts produced during the phases of the SDLC, recording knowledge gathered along each sprint or project.*

[—] *Software maintainers validate, correct, and update knowledge from previous phases of software development life cycle [...].*

397.

*Knowledge management is comprised of the elicitation, packaging and management, and reuse of knowledge in all its different forms. Explicit software engineering knowledge includes all types of software engineering artifacts, ranging from traditional software*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

Une revue de littérature portant sur le partage de connaissances dans un contexte de développement décentralisé [410] s'appuie sur l'analyse de 61 articles pour identifier les défis que voici :

- coût du partage des connaissances ;
- différences contextuelles ;
- manque d'ouverture ;
- rotation du personnel ;
- problèmes de documentation.

(voir énoncé p.210)

**Explication de l'assertion OS04 : La connaissance provenant des autres domaines est une nécessité pour l'informatique appliquée, puisqu'elle a pour objectif de résoudre différents problèmes hors de son domaine.**

Ian K. Bray, dans l'ouvrage *An introduction to requirements engineering*, décrit le métier d'informaticien et de la nécessité de l'exploration des connaissances provenant d'autres domaines, le terme *Elicitation* est employé et y est décrit :

[Traduction libre] Le terme « collecte » a largement remplacé celui de « récolte », qui sous-entendait à tort que les exigences « mûres » (etc.) attendaient d'être « cueillies ». Or, c'est rarement le cas ; les exigences « brutes » n'existent généralement que sous une forme incomplète. Dans leur analyse très accessible de ce sujet, Gause et Weinberg (1989) proposent la métaphore pertinente de l'exploration : il s'agit de s'aventurer en territoire, au moins partiellement, inexploré, à la recherche d'informations qui peuvent être cachées ou encore partiellement ébauchées. Il ne s'agit donc pas d'un simple transfert de connaissances d'un endroit à un autre. Concernant les caractéristiques du domaine problématique, l'information existe sans doute quelque part, mais elle peut se trouver uniquement dans

---

*artifacts such as code, design and requirements to process knowledge in the form of models, data and standards, and lessons learned.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

l'esprit d'un utilisateur d'un système préexistant, et il se peut même que cet utilisateur n'en ait pas conscience.<sup>398</sup> [37]

(voir énoncé p.211)

#### Les participants informatiques

##### Explication de la définition ES01 : Informaticien

La définition formulée est : spécialiste du traitement automatique et rationnel de l'information.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 d'informatique : « Science du traitement automatique et rationnel de l'information considérée comme le support des connaissances et des communications. » [188] ;
- définition 2 d'informatique : « Discipline qui s'intéresse à tous les aspects, tant théoriques que pratiques, reliés au traitement automatique de l'information, à la conception, à la programmation, au fonctionnement et à l'utilisation des ordinateurs. » [256] ;
- définition d'informaticien : « Spécialiste du traitement automatique de l'information. » [255] ;
- notes sur la définition d'informaticien : « Le terme informaticien est un terme générique qui sert à désigner les programmeurs, les analystes de l'informatique, les concepteurs de logiciels, etc. » [255].

---

398.

*Elicitation has now largely replaced the term “gathering” which had the inappropriate implication that “ripe” requirements (etc.) are hanging around simply waiting to be “plucked”. This is seldom the case ; “raw” requirements usually exist only in an incomplete form. In their highly readable examination of this topic, Gause and Weinberg (1989) present the apt metaphor of exploration ; one is setting out into, at least partly, uncharted territory in search of information which may well be hidden or only part formed. So it is not a simple matter of transferring knowledge from one place to another. In the case of problem domain characteristics, the information doubtless exists somewhere but it may be only in the head of a user of some pre-existing system and it may be that said user does not even realise what they do know.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

L'informaticien comprend les parties théorique et appliquée de l'informatique.  
(voir énoncé p.211)

#### **Explication de la définition ES02 : Informaticien appliqué**

La définition formulée est : informaticien spécialiste dans l'information provenant de domaines autres que l'informatique.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition d'ingénieur informaticien : « Spécialiste de l'informatique qui travaille à la conception, au développement, à la mise en œuvre et à l'exploitation de systèmes de traitement ou de communication de données. » [259] ;
- définition d'ingénieur logiciel : « Ingénieur spécialisé qui travaille à la conception, au développement, à la mise en œuvre et à l'exploitation des systèmes logiciels d'une organisation. » [258].
- définition d'appliqué : « Se dit de tout domaine d'activité scientifique où les recherches théoriques sont mises en œuvre pour résoudre des problèmes pratiques. » [198]

L'ingénierie comprend la partie appliquée de l'informatique.

En opposition, il y a l'informaticien théorique ou fondamental et les définitions suivantes le corroborent :

- définition théorique : « Relatif à la théorie, à l'élaboration des théories, à leur contenu, par opposition à pratique ou à appliqué : Connaissances théoriques. » [204] ;
- antonyme de fondamental : « En parlant de la recherche — appliquée (recherche) » [79].

(voir énoncé p.211)

#### **Explication de la définition ES03 : Praticien informatique**

La définition formulée est : personne qui réalise ou établit des entités informatiques.

La littérature fournit ces informations qui ont servi à construire cette définition :

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

- définition 1 de praticien : « Personne qui exerce son art et qui a la connaissance et l’usage des moyens pratiques, par opposition à théoricien. » [203] ;
- définition 2 de praticien : « Personne engagée dans l’exercice d’un art ou d’une profession. » [282] ;
- définition d’informatique : « Discipline qui s’intéresse à tous les aspects, tant théoriques que pratiques, reliés au traitement automatique de l’information, à la conception, à la programmation, au fonctionnement et à l’utilisation des ordinateurs. » [256].

(voir énoncé p.211)

#### **Explication de la définition ES04 : Professionnel informatique**

La définition formulée est : personne exerçant une activité rémunérée liée à l’informatique.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de professionnel : « Qui exerce régulièrement une profession, un métier, par opposition à amateur [...]. » [191] ;
- définition 2 de professionnel : « Personne de métier (opposé à amateur) [...]. » [333] ;
- définition d’informatique : « Discipline qui s’intéresse à tous les aspects, tant théoriques que pratiques, reliés au traitement automatique de l’information, à la conception, à la programmation, au fonctionnement et à l’utilisation des ordinateurs. » [256].

(voir énoncé p.211)

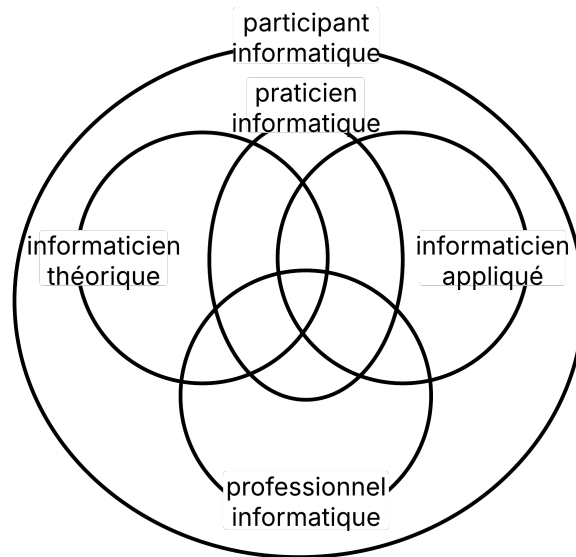
#### **Explication de la définition ES05 : Participant informatique**

La définition formulée est : personne participant à des processus utilisant l’informatique.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition de participant : « Qui participe à quelque chose. » [200] ;
- définition de participer : « Assumer une partie d’une action, d’une tâche : Participer aux travaux domestiques. » [201] ;

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.37** – Diagramme de Venn sur des titres en informatique

- définition d'informatique : « Discipline qui s'intéresse à tous les aspects, tant théoriques que pratiques, reliés au traitement automatique de l'information, à la conception, à la programmation, au fonctionnement et à l'utilisation des ordinateurs. » [256].

(voir énoncé p.211)

**Explication de l'assertion ES06 : Les personnes impliquées en informatique peuvent être catégorisées selon certains critères.**

Les personnes impliquées en informatique peuvent être catégorisées selon les critères de formation, d'application à un domaine, de production ou de rémunération.

- une personne est qualifiée (informaticien) ou pas ;
- une personne applique l'informatique à celle-ci (fondamentale) ou à un autre domaine (appliquée) ;
- une personne développe des entités informatiques (praticien) ou pas ;
- une personne est rémunérée (professionnel) ou pas.

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

La figure B.37 représente les différents titres en informatique. L’informaticien représente l’union entre l’informaticien théorique et l’informaticien appliqué.  
(voir énoncé p.211)

**TABEAU B.15** – Nombre de postes occupés par des professionnels informatiques au Québec en 2021

| Métier (CNP)                                                                  | Le nombre en poste |
|-------------------------------------------------------------------------------|--------------------|
| Gestionnaires des systèmes informatiques (20012)                              | 18035              |
| Scientifiques des données (21211)                                             | 2580               |
| Spécialistes de la cybersécurité (21220)                                      | 3220               |
| Spécialistes des systèmes commerciaux (21221)                                 | 4445               |
| Spécialiste informatique (21222)                                              | 31890              |
| Analystes de bases de données et administrateurs de données (21223)           | 7145               |
| Développeurs et programmeurs de systèmes informatiques (21230)                | 18225              |
| Ingénieurs et concepteurs en logiciel (21231)                                 | 12230              |
| Développeurs et programmeurs de logiciels (21232)                             | 18260              |
| Concepteurs Web (21233)                                                       | 2510               |
| Développeurs et programmeurs Web (21234)                                      | 15265              |
| Ingénieurs informatiques - sauf ingénieurs et concepteurs en logiciel (21311) | 6230               |
| Techniciens de réseau informatique et Web (22220)                             | 20965              |
| Agents de soutien aux utilisateurs (22221)                                    | 14795              |
| Évaluateurs de systèmes informatiques (22222)                                 | 3860               |
| Le total                                                                      | 179655             |

<sup>a</sup> Données sur les professions provenant de Statistique Canada [372].

**Explication de l’assertion ES07 : Le terme professionnel informatique regroupe plusieurs métiers plus spécialisés.**

Ces métiers sont présentés dans le tableau B.15. Les personnes qui les exercent sont



### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

rémunérées pour réaliser et établir des entités informatiques. Ces métiers varient selon les classifications, les régions et les époques pour servir de base. La classification nationale des professions (CNP) est utilisée.

(voir énoncé p.[211](#))

**Explication de l’assertion ES08 : Les professionnels informatiques sont nombreux, près de cent quatre-vingt mille au Québec.**

En 2021, il y a environ dix fois plus de professionnels informatiques ( $\approx 180\,000$ ) au Québec que de professeurs et chargés de cours universitaires à temps plein, tous domaines confondus ( $\approx 18\,000$ ), selon des données de Statistique Canada (présentées aux tableaux B.15 et B.16) sur les professions [[372](#)].

(voir énoncé p.[211](#))

**Explication de l’assertion ES09 : Les tranches d’âge plus élevées sont nettement plus représentées pour d’autres métiers.**

À partir des données de Statistique Canada, il est possible de comparer le métier de professionnel informatique à d’autres métiers, ce qui donne lieu au tableau B.16. En faisant le ratio entre la tranche d’âge le plus élevé (64 ans et plus) et la tranche d’âge le plus bas (15 à 24 ans), il est obtenu : professionnel informatique (0,26),

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**TABEAU B.16** – Nombre de postes occupés à temps plein en 2021 au Québec, par métier et tranche d'âge

| Métier                                                  | 15 à 24 ans | 25 à 64 ans | 65 ans et plus | Total  |
|---------------------------------------------------------|-------------|-------------|----------------|--------|
| Professionnel informatique                              | 12060       | 164505      | 3110           | 179675 |
| Ingénieur sauf informaticien et électricien             | 1055        | 38260       | 1560           | 40860  |
| Assistant d'enseignement et de recherche au niveau      | 6575        | 20330       | 510            | 27420  |
| Enseignant niveau collégial et instructeur en formation | 885         | 34820       | 3100           | 38805  |
| Professeur et chargé de cours universitaire             | 120         | 15645       | 2580           | 18345  |
| Avocat                                                  | 810         | 21500       | 2220           | 24525  |
| Médecin et vétérinaire                                  | 320         | 24920       | 3445           | 28685  |
| Physicien et astronome                                  | 10          | 635         | 55             | 695    |

<sup>a</sup> Données provenant de Statistique Canada sur les professions [372].

ingénieur (1,48), assistant d'enseignement (0,08), etc. (Pour plus de détails voir [P.84 tableau 1.5](#)). Dans ce comparatif, il n'y a que les assistants d'enseignements et de recherche qui présentent un ratio aussi faible que celui des professionnels informatiques. Toutefois, ils bénéficient de l'encadrement des professeurs et chargés de cours universitaires. Les professionnels informatiques, pour leur part, ne disposent pas nécessairement du soutien d'un autre corps de métier.

(voir énoncé [p.212](#))

#### **Explication de l'assertion ES10 : La rétention des professionnels informatiques est plus faible que celle d'autres professions.**

En 2006, un article de Deborah E. Rupp *et al.* décrit l'industrie des technologies de l'information aux États-Unis du point de vue des ressources humaines [13]. Plusieurs études et sondages sont cités et corroborent que la rétention dans le secteur est basse :

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

- [Traduction libre] « Une étude de l’Université George Mason a également constaté que le taux de changement de carrière chez les informaticiens était deux fois plus élevé que chez les travailleurs d’autres secteurs (Mandell, 1998). » <sup>399</sup> ;
- [Traduction libre] « Capelli (2001) présente une statistique éloquent : l’Enquête nationale auprès des diplômés universitaires a révélé que seulement 19 % des diplômés en informatique travaillaient encore dans ce domaine 20 ans plus tard, contre 52 % des diplômés en génie civil. » <sup>400</sup> ;
- [Traduction libre] « Gallivan et al. (2002) ont examiné les compétences requises dans les offres d’emploi pour les professionnels de l’informatique en 1988, 1995 et 2001, et ont conclu que le nombre de compétences mentionnées par annonce avait augmenté de 40 % entre 1988 et 2001. » <sup>401</sup>

Il est possible que les professionnels informatiques occupent des métiers plus propices au changement de rôle, comme la transition d’un poste de développeur à celui de gestionnaire de projet.

(voir énoncé p.212)

**Explication de l’assertion ES11 : Les professionnels informatiques affirment quitter le métier, notamment en raison de la lourde charge de travail et de la menace d’obsolescence.**

En 2014, un article présente les résultats d’une étude qualitative et d’une étude quantitative [63]. L’étude qualitative repose sur des entrevues menées auprès de 20 personnes, tandis que l’étude quantitative s’appuie sur un sondage mené auprès de 167 individus. Tous sont d’anciens professionnels informatiques.

- Dans l’étude qualitative, les raisons suivantes sont évoquées pour expliquer l’abandon du métier de professionnel informatique :

---

399. « *A study by George Mason University similarly found that career change among IT workers was double that of workers in other fields (Mandell, 1998).* »

400. « *Capelli (2001) presents a compelling statistic : The National Survey of College Graduates reported that only 19% of computer science graduates remained in the field 20 years later, while 52% of civil engineering graduates did so.* »

401. « *Gallivan et al. (2002) examined the skills specified in job ads for IT professionals in 1988, 1995, and 2001, and concluded that the number of skills mentioned per ad increased by 40 % from 1988 to 2001.* »

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

- le déséquilibre effort-récompense ;
  - l'épuisement émotionnel ;
  - le conflit travail-famille ;
  - la charge de travail perçue ;
  - la fatigue) ;
  - la menace d'obsolescence professionnelle.
- Dans l'étude quantitative, les raisons suivantes sont mentionnées :
- le déséquilibre effort-récompense ;
  - la charge de travail perçue ;
  - l'épuisement émotionnel ;
  - la menace d'obsolescence professionnelle ;
  - le faible contrôle sur le travail ;
  - le conflit travail-famille.

Plusieurs de ces raisons réapparaissent :

- en 2021, lors d'entrevues menées auprès de 15 anciens professionnels informatiques [234] ;
- en 2025, dans un sondage portant sur les intentions de quitter la profession auprès de 221 professionnels informatiques [215].

(voir énoncé p.212)

**Explication de l'assertion ES13 : Un professionnel informatique n'a pas besoin d'être qualifié pour exercer au Québec.**

Des connaissances minimales s'obtiennent avec des formations formelles et des qualifications. Il y a plusieurs métiers où la formation qualifiante se retrouve à même la définition du métier. Voici deux exemples :

- définition d'informaticien : « Spécialiste du traitement automatique de l'information » [255] ;

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

- note sur la définition d’informaticien : « Le terme informaticien est un terme générique qui sert à désigner les programmeurs, les analystes de l’informatique, les concepteurs de logiciels, etc. » [255] ;
- définition d’ingénieur logiciel : « Ingénieur spécialisé qui travaille à la conception, au développement, à la mise en œuvre et à l’exploitation des systèmes logiciels d’une organisation. » [258] ;
- définition de physicien : « Personne qualifiée pour exercer la profession de physicien et qui détient un diplôme en physique d’une université reconnue. » [269] ;
- définition d’avocat : « Personne qui, régulièrement inscrite à un barreau, [...] » [241] ;
- définition d’ingénieur : « Personne ayant reçu une formation reconnue qui la rend apte à concevoir, à réaliser et à mettre en œuvre dans l’industrie des systèmes, des procédés, des produits ou des services. » [257]

Il n’y a aucune référence à une formation qualifiante dans la définition d’informaticien, tandis qu’il en a une dans celles de physicien, d’avocat et d’ingénieur.

(voir énoncé p.212)

**Explication de l’assertion ES14 : La profession informatique est l’une des rares à ne pas avoir été catégorisée par la formation.**

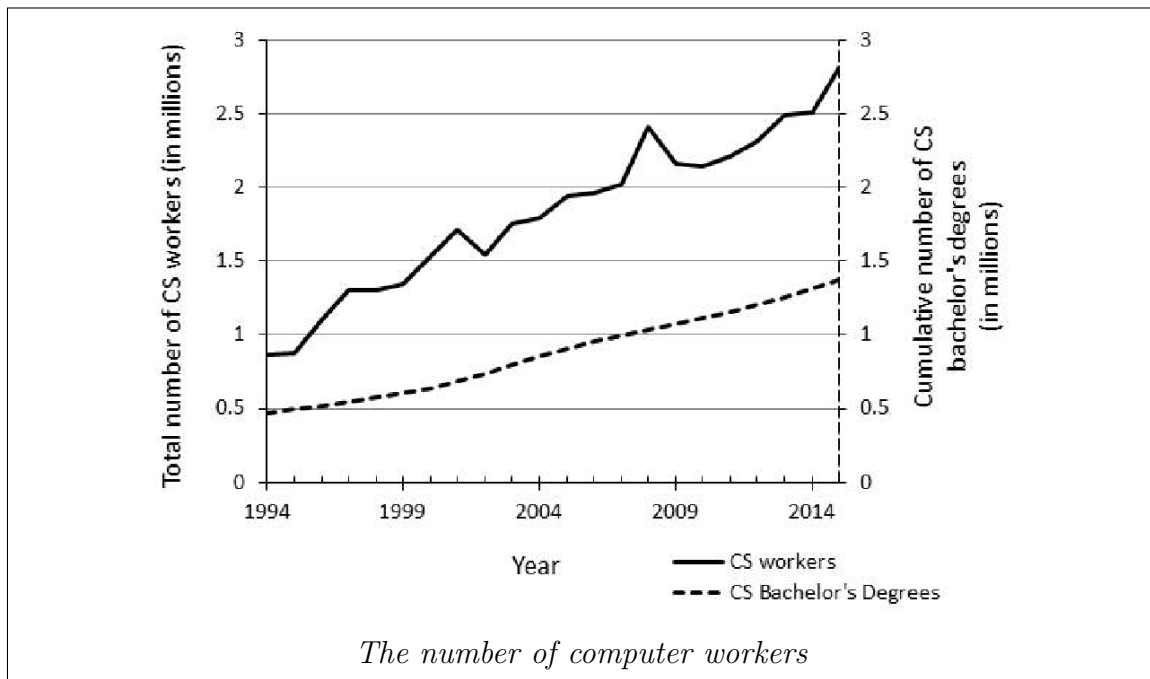
À l’aide de données provenant de Statistique Canada, il est possible de comparer le niveau de scolarité des professionnels informatiques avec d’autres métiers.

Les professionnels informatiques sont le groupe avec le plus petit taux de diplomation au premier cycle avec 0,48. Les métiers comme médecin, avocat, ingénieur ont un haut taux de diplomation au premier cycle, puisque c’est exigé par leurs ordres professionnels. Pour les métiers comme professeur, physicien, assistant d’enseignement ou de recherche, cela est exigé de la part de leurs employeurs. Le point commun à tous ces métiers est qu’ils ont été catégorisés par différent niveau de formation (professionnel, collégial, universitaire), ce qui n’est pas le cas des professionnels informatiques.

(voir énoncé p.212)

**Explication de l’assertion ES15 : Aux États-Unis, parmi les analystes de systèmes informatiques, les informaticiens et les développeurs de logiciels titulaires d’un diplôme de premier cycle, moins de la moitié possèdent un diplôme en informatique.**

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE



<sup>a</sup> Le rapport cite une étude aux États-Unis de 2015 dont les auteurs sont Bound and Morales, elle se retrouve en annexe C du rapport.

<sup>b</sup> « *workers (defined as “Computer Systems Analysts,” “Computer Scientists,” and “Computer Software Developers”) holding bachelor’s degrees (in any field)* »

<sup>c</sup> « *Cumulative CIS bachelor’s degrees calculated from Integrated Postsecondary Education Data System (IPEDS) completions survey results.* »

**FIGURE B.38** – Professionnels en informatique par rapport aux diplômés en informatique aux États-Unis (source : *Figure 4.1* [64])

En 2018, le National Academies Press publie un rapport sur la croissance des travailleurs informatiques. Il y est observé (figure B.38) le nombre de millions de personnes travaillant dans ce domaine, en distinguant les travailleurs qualifiés des non qualifiés [64].

(voir énoncé p.212)

**Explication de la citation ES16 : Plusieurs professionnels informatiques estiment que leur métier est trop exigeant pour maintenir ou développer de nouvelles compétences.**

L’organisme TechnoCompétence publie un rapport pour les technologies de l’infor-

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

mation et des communications (TIC) intitulé *Diagnostic sectoriel 2025-2028*<sup>402</sup> [376]. Ce rapport présente les résultats d'un sondage en ligne portant sur les freins à la formation, auquel 33 personnes ont répondu. Il s'agit d'un faible échantillon étant donné les cent quatre-vingt mille professionnels informatiques. Les trois principales raisons évoquées sont :

- le surplus de travail empêchant de libérer le personnel pour suivre de la formation ;
- les horaires des formations incompatibles avec ceux du travail ;
- les coûts élevés des activités de formation continue.

(voir énoncé p.212)

#### Les processus

##### Explication de la définition ES17 : Taxinomie

La définition formulée est : activité et résultat de l'étude d'un sujet pour en obtenir des règles de classification.

La littérature fournit ces informations qui ont servi à construire cette définition :

- une définition : « Étude théorique de la classification, de ses bases, de ses lois, de ses principes et de ses règles. » [278] ;
- une note sur la définition précédente : « La taxinomie ne s'intéressait à l'origine qu'à la classification des organismes vivants en bactériologie, en botanique et en zoologie mais son utilisation s'étend aujourd'hui à d'autres sciences, telles les sciences humaines et les sciences de l'information. » [278] ;
- une note supplémentaire : [Traduction libre] « La taxonomie est l'étude de la classification, et une taxonomie est une classification. »<sup>403</sup> [315].

Selon l'OQLF, l'utilisation répandue du terme **taxonomie** est un anglicisme qu'il est préférable d'éviter en faveur de **taximomie**.

(voir énoncé p.213)

---

402. TechnoCompétence est un organisme à but non lucratif dirigé par des entreprises en TIC qui fait la promotion des TIC.

403. « *Taxonomy is the study of classification, and a taxonomy is a classification.* »

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**Explication de l’assertion ES18 : Les taxinomies sont importantes pour comprendre rigoureusement les théories et les modèles.**

En 2019, Paul Ralph décrit l’importance de la taxinomie pour l’informatique dans l’article *Toward Methodological Guidelines for Process Theories and Taxonomies in Software Engineering* :

[Traduction libre]

[—]Une théorie des processus [...] est un système d’idées destiné à expliquer, comprendre (et parfois prédire) comment une entité change et évolue.

[—]En résumé, les théories des processus et les taxinomies sont nécessaires en génie logiciel, car elles fournissent les concepts et le langage technique indispensables à une communication et à un enseignement précis. Les taxinomies améliorent également l’efficacité cognitive en facilitant le raisonnement sur les classes plutôt que sur les instances, tandis que les théories des processus révèlent souvent des vérités contre-intuitives et combattent la pseudoscience. <sup>404</sup> [315]

Dans le recueil d’articles *Context and Semantics for Knowledge Management : Technologies for Personal Productivity*, la formalité de la taxinomie va déterminer si le modèle qui l’emploie est formel ou pas :

[Traduction libre]

- Les modèles informels sont classés par ordre croissant de formalité : vocabulaires contrôlés, glossaires, thésaurus et taxinomies informelles. Cette catégorie inclut également les folksonomies, ensembles de termes issus de processus d’étiquetage collaboratifs.

---

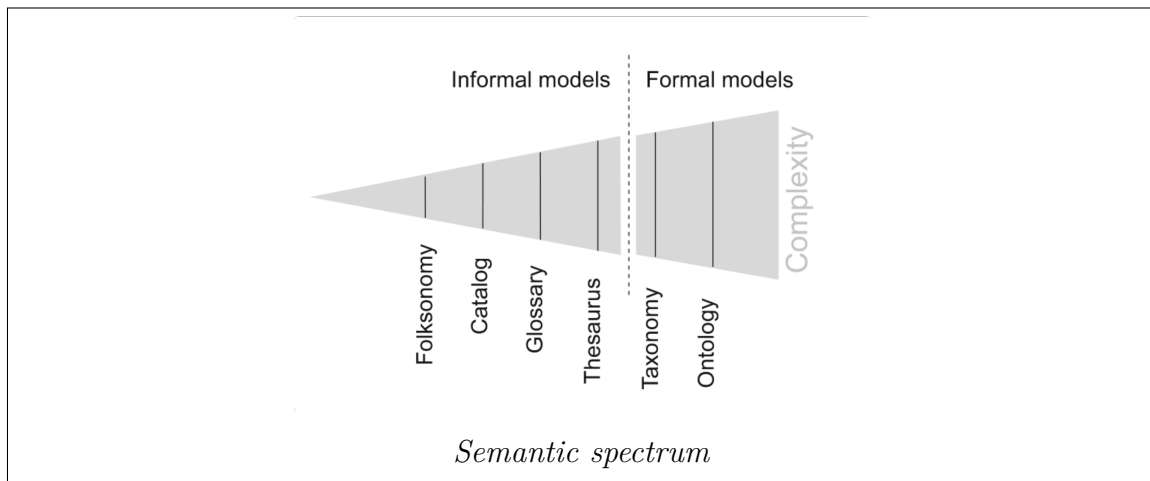
404.

[—]A process theory, contrastingly, is a system of ideas intended to explain, understand (and sometimes predict) how an entity changes and develops.

[—]In summary, process theories and taxonomies are needed in software engineering because they provide concepts and technical language needed for precise communication and education. Taxonomies also provide cognitive efficiency by facilitating reasoning about classes rather than instances while process theories often expose counterintuitive truths and combat pseudoscience.



### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE



<sup>a</sup> Définition de folksonomies : *sets of terms which are the result of collaborative tagging processes.*

**FIGURE B.39** – Niveaux de formalisme d’artefacts de représentation des connaissances (source : *Figure 3.1* [394])

- Les modèles formels sont classés de la même manière : partant des taxonomies formelles, qui définissent précisément la relation spécialisation/généralisation, des modèles plus formels sont dérivés par l’ajout progressif d’instances formelles, de propriétés/cadres, de restrictions de valeurs, de disjonction, de méronymie formelle, de contraintes logiques générales, etc.

<sup>405</sup> [394]

Il y a aussi cette figure B.39 provenant d’un autre auteur qui trace la taxinomie comme la frontière pour établir un modèle formel :

[Traduction libre] McGuiness (2003) définit un « spectre sémantique »” spécifiant un ordre global entre les types de modèles courants. Cela di-

405.

- *Informal models are ordered in ascending order of their formality degree as controlled vocabularies, glossaries, thesauri and informal taxonomies. In this category we can also include folksonomies, sets of terms which are the result of collaborative tagging processes.*
- *Formal models are ordered in the same manner : starting with formal taxonomies, which precisely define the meaning of the specialization/generalization relationship, more formal models are derived by incrementally adding formal instances, properties/frames, value restrictions, disjointness, formal meronymy, general logical constraints etc.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

visent essentiellement les ontologies ou les structures de type ontologie en informelles et formelles, comme suit (Fig. 3.1) :<sup>406</sup> [394]  
(voir énoncé p.213)

**Explication de l’assertion ES19 : Les taxinomies portant sur les processus, les modèles et les méthodes sont peu nombreuses et subjectives.**

En 2019, Paul Ralph met en garde contre cette situation :

[Traduction libre] L’ingénierie des systèmes bénéficie déjà de taxinomies des dimensions de la qualité logicielle, des facteurs clés de succès, des décisions de codage, des rôles des gestionnaires et des développeurs, et de nombreux autres phénomènes importants. Cependant, nombre de taxinomies existantes ne reposent pas sur des recherches rigoureuses. L’ingénierie des systèmes manque également d’une taxinomie des activités ou pratiques de développement (analogue à la taxinomie de Sim et Duffy pour la conception technique [140]).<sup>407</sup> [315]

En 2017, un article fait une revue de littérature sur les taxinomies en informatique appliquée [388]. Au total, 270 articles allant de 1975 à 2015 sont analysés pour un total de 271 taxinomies. Il y a une correspondance de faite entre les taxinomies et les domaines de connaissances provenant du SWEBOK. Les résultats sont affichés dans la figure B.40. À cette figure s’ajoutent des faits supplémentaires qui informent le lecteur sur la rigueur employé pour créer ces taxinomies :

[Traduction libre]

---

406.

*(McGuinness 2003) defines a “semantic spectrum” specifying a total order between common types of models. This basically divides ontologies or ontologylike structures in informal and formal as follows (Fig. 3.1) :*

407.

*SE already benefits from taxonomies of software quality dimensions, critical success factors, coding decisions, manager and developer roles, and many other important phenomena. However, many existing taxonomies are not based on rigorous research. SE also lacks a taxonomy of development activities or practices (analogous to Sim and Duffy’s taxonomy for engineering design [140]).*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

[—] Relativement peu de taxinomies sont rapportées dans les « articles d'évaluation » (34 – 12,54 %) et les « articles de validation » (11 – 4,06 %).

[—] Sans surprise, les 23 études qui ne fournissent pas de bases descriptives sont peu citées, ce qui pourrait indiquer leur faible utilité [...].

[—] La majorité des taxinomies utilisent une procédure de classification qualitative (262 – 96,68 %), suivie des procédures quantitatives (7 – 2,58 %) et des deux (2 – 0,74 %).

[—] La majorité des taxinomies ne comportent pas de description explicite de la procédure de classification (227 – 83,76 %), et seulement 44 taxinomies (16,24 %) en possèdent une. <sup>408</sup> [388]

(voir énoncé p.213)

---

408.

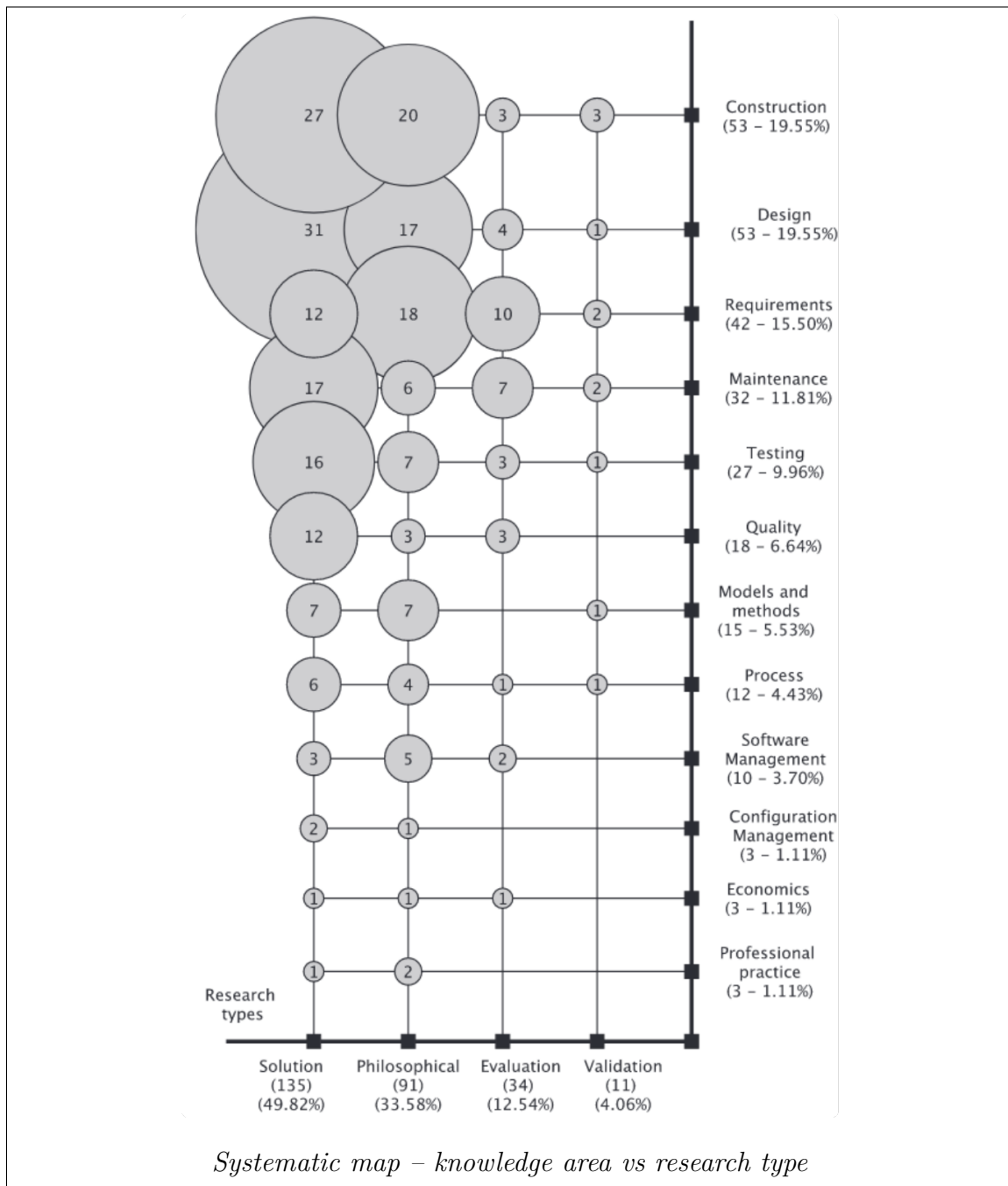
[—] *Relatively fewer taxonomies are reported in “evaluation papers” (34 – 12.54%) and “validation papers” (11 – 4.06%).*

[—] *Not surprisingly, the 23 studies that did not provide descriptive bases are lowly cited, it might indicate the low usefulness of them [...].*

[—] *The majority of the taxonomies employed a qualitative classification procedure (262 – 96.68%), followed by quantitative (7 – 2.58%) and both (2 – 0.74%).*

[—] *The majority of the taxonomies do not have an explicit description for the classification procedure (227– 83.76%) and only 44 taxonomies (16.24%) have an explicit description.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.40** – Développement de taxinomies par type de recherche et domaine de connaissances (source : Figure 8 [388])

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

#### **Explication de la citation ES20 : Le courant dominant discrédite les procédés de développement.**

En 2001 apparaît le Manifeste Agile. Le premier principe mentionné est celui-ci :

Les individus et leurs interactions plus que les processus et les outils. <sup>409</sup>

Bertrand Meyer décrit dans l'ouvrage *Agile ! : The Good, the Hype and the Ugly* la nécessité d'avoir des procédés :

[Traduction libre] Les discussions sur les modèles de cycle de vie oscillent souvent entre les deux mots-clés d'un ouvrage de Sigmund Freud : Totem et Tabou. Aucun des deux n'est approprié. Tout projet a besoin d'un cadre temporel pour prévoir et évaluer son évolution. <sup>410</sup> [220]

(voir énoncé p.213)

#### **Explication de l'assertion ES21 : Il n'y a pas de définitions explicites des procédés de développement et des critères de succès d'un projet.**

Le Standish Group publie le *CHAOS Report* et y oppose les approches *Agile* et *Waterfall*. Or, dans l'édition 2015, ces termes ne sont pas définis [371].

En 2019, une revue de littérature porte sur les articles abordant le succès des projets en méthodologie agile et souligne le fait qu'il n'y a pas de consensus sur les critères de succès d'un projet :

[Traduction libre] Du point de vue de la gestion de projet, les facteurs clés de succès (FCS) sont les caractéristiques, les conditions ou les variables qui ont un impact significatif sur la réussite du projet, à condition qu'elles soient correctement gérées [27]. L'approche par les FCS a fait l'objet de recherches au cours des trente dernières années. Cependant, il n'existe

---

409. Kent Beck *et al.*, site web, « Manifesto for Agile Software Development », <https://agilemanifesto.org/iso/fr/manifesto.html> (consulté le 2025-05-12).

410.

*Discussions of lifecycle models tend to oscillate between the two title words of a book by Sigmund Freud : Totem and Taboo. Neither is appropriate. Every project needs a temporal framework to predict and assess its progress.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

toujours pas de consensus sur les critères qui déterminent la réussite d'un projet [28].<sup>411</sup> [158]  
(voir énoncé p.213)

#### Explication de l'énoncé ES22 : Les études rigoureuses sur les procédés en informatique sont peu nombreuses.

En 2011, une revue de littérature portant sur les recherches consacrées à la modélisation des processus de développement logiciel décortique les sujets abordés [22]. Au total, 43 études réalisées entre 1987 et 2008 et fournissant des données sont sélectionnées (figure B.41). Il y a très peu d'expérimentations ou de méta-analyses reposant sur des données quantitatives. Or, ce type de données faciliteraient l'établissement de comparaisons et permettraient d'augmenter la réfutabilité de ces études. Avec des données qualitatives, cet exercice devient plus difficile.

| Empirical Research Method | Data Analysis |              | Total | Percentage |
|---------------------------|---------------|--------------|-------|------------|
|                           | Qualitative   | Quantitative |       |            |
| Action Research           | 12            | -            | 12    | 27.9%      |
| Case Study                | 13            | 4            | 13*   | 30.2%      |
| Experiment                | 4             | 1            | 4*    | 9.3%       |
| Meta-analysis             | 3             | 1            | 3     | 7.0%       |
| Phenomenology             | 9             | 1            | 9     | 20.9%      |
| Survey                    | 3             | 2            | 3     | 7.0%       |
| Sub-Total                 | 43            | 9            | 43*   | -          |

\* One study claimed using both case study and controlled experiment [6].

*The primary research objectives and mostly used research method for them*

**FIGURE B.41** – Types de recherches menées sur les procédés de développement entre 1987 et 2008 (source : Table 2 [22])

411.

*From a project management perspective, critical success factors (CSF) are the characteristics, conditions, or variables with significant impact on the success of the project provided they are properly managed [27]. The CSF approach has been researched over the last thirty years. However, there is still no consensus on the criteria that determine project success [28].*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

En 2015, l'analyse *Does Agile work? — A quantitative analysis of agile project success* portant sur 1 386 projets qualifie la majorité des recherches d'anecdotique où les échantillons sont trop petits pour être utiles :

[Traduction libre] À ce jour, la majorité des recherches examinant l'utilité de cette méthodologie sont anecdotiques, basées sur des études de cas portant sur de petits échantillons, ou des recherches limitées par la taille de l'échantillon, le secteur d'activité ou la zone géographique.<sup>412</sup> [360]

En 2024, l'ouvrage *Experimentation in Software Engineering* analyse 113 articles. Parmi ces articles, seuls deux concernaient une même théorie. Ceci signifie que les articles qui étaient ou réfutent un autre article sont rares :

[Traduction libre] Comme mentionné précédemment, des expériences peuvent être menées pour générer, confirmer et étendre des théories. Cependant, le recours à la théorie est rare en génie logiciel, comme l'ont conclu Hannay *et al.* dans leur revue systématique de la littérature sur les expériences en génie logiciel (1993-2002) [100]. Sur les 113 articles analysés, ils ont recensé 40 théories dans 23 articles. Seules deux de ces théories étaient utilisées dans plus d'un article ! De même, Hall *et al.* ont constaté un manque de références à la théorie générale de la motivation dans 92 études sur la motivation en génie logiciel, menées entre 1980 et 2006 [99].<sup>413</sup> [404]

(voir énoncé p.213)

---

412.

*To date, the majority of research examining the methodology's usefulness has been anecdotal, based on small-sample case studies, or research limited by sample size, industry or geography.*

413.

*Experiments may be conducted to generate, confirm, and extend theories, as mentioned above. However, the use of theory is scarce in software engineering, as concluded by Hannay et al. in their systematic literature review of software engineering experiments, 1993–2002 [100]. They found 40 theories in 23 articles, out of the 113 articles in the review. Only two of the theories were used in more than one article ! Similarly, Hall et al. identified a lack of references to general theory on motivation in 92 studies of motivation in software engineering, in the 1980–2006 period [99].*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

#### Les artefacts

#### Explication de la définition ES23 : Rétro-ingénierie

La définition formulée est : consiste à étudier un groupe d'artefacts pour en déterminer le fonctionnement et la méthode de fabrication.

La littérature fournit ces informations qui ont servi à construire cette définition :

- définition 1 de rétro-ingénierie : « Ensemble des opérations d'analyse d'un logiciel ou d'un matériel destinées à retrouver le processus de sa conception et de sa fabrication, ainsi que les modalités de son fonctionnement. » [274] ;
- définition 2 de rétro-ingénierie : [Traduction libre] « [...] déterminer ce que fera le logiciel existant et comment il est construit (pour apporter des modifications intelligentes) [...]. » <sup>414</sup> [140] ;
- définition 3 de rétro-ingénierie : [Traduction libre] « [...] une approche d'ingénierie logicielle qui déduit la conception ou les exigences d'un système à partir de son code [...]. » <sup>415</sup> [140] ;
- définition d'ingénierie inverse : [Traduction libre] « [...] processus d'obtention d'une représentation de haut niveau du logiciel à partir du code source voir rétro-ingénierie [...]. » <sup>416</sup> [140] ;
- note sur d'ingénierie inverse : [Traduction libre] « L'ingénierie inverse fournit une vue plus abstraite du système dans le but de récupérer les informations de conception et d'exigences. » <sup>417</sup> [140].

(voir énoncé p.213)

#### Explication de l'assertion ES24 : Les artefacts se dégradent volontairement, involontairement et par les changements dans l'environnement.

---

414. « [...] *determining what existing software will do and how it is constructed (to make intelligent changes [...].)* »

415. « [...] *a software engineering approach that derives a system's design or requirements from its code [...].* »

416. « [...] *process of obtaining a high-level representation of the software from the source code cf. reverse engineering [...].* »

417. « *Inverse engineering provides a more abstract view of the system with the intent of recapturing design and requirements information.* »



### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

Dans un article de 2022, Ehsan Zabardast décrit les dégradations possibles des artefacts :

[Traduction libre]

- délibérée : Prendre une décision consciente, mais sous-optimale, pour atteindre des objectifs à court terme au détriment de la valeur à long terme. L'actif se dégrade suite à une modification résultant d'une décision consciente et non optimale. On prend un raccourci, en connaissant ses conséquences, et on en subit finalement les conséquences.
- involontaire : Les décisions sous-optimales dues à un manque de diligence constituent des formes non intentionnelles de dégradation. L'actif se dégrade suite à une modification résultant d'une décision inconsciente et non optimale. On ignore l'existence d'autres alternatives, meilleures, à notre décision ; par conséquent, on ne prévoit pas les conséquences du choix d'une alternative. On n'a pas conscience du fait que l'utilisabilité de l'actif peut s'en trouver compromise.
- par entropie : Selon Lehman, « à mesure qu'un logiciel évolue, sa complexité augmente à moins qu'un travail ne soit entrepris pour la maintenir ou la réduire » et « [il] sera perçu comme étant de qualité déclinante à moins d'être rigoureusement maintenu et adapté à un environnement opérationnel changeant » (Lehman, 1979, 1996). Cette complexité croissante et la baisse de qualité perçue peuvent englober la dégradation des ressources impliquées dans le développement du système logiciel. La dégradation due à l'évolution continue du logiciel, et non à la manipulation directe de ces ressources par les développeurs, est appelée entropie. L'entropie peut induire une dégradation sous forme de dégradation naturelle due aux évolutions technologiques et du marché, même en présence de mécanismes de gestion de la qualité censés l'éviter. <sup>418</sup> [409]

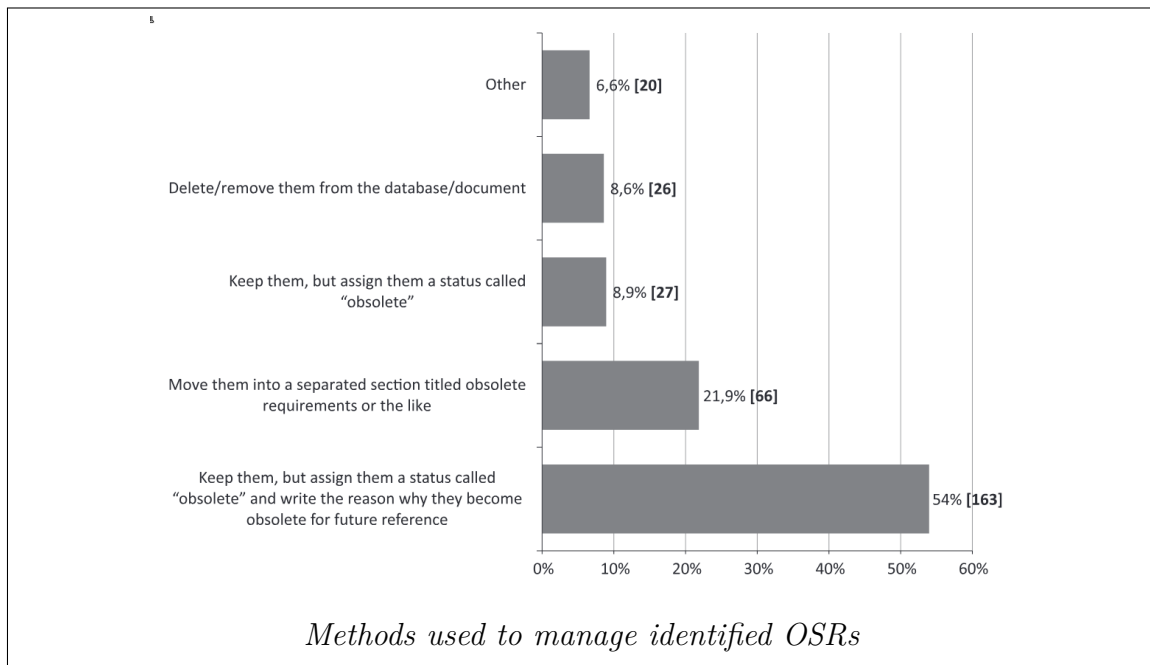
(voir énoncé p.213)

---

418.

— *deliberate* : Taking a conscious, sub-optimal decision to accommodate short-term goals at the expense of long-term value. The asset is degraded as a result of a modification comprising a

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE



<sup>a</sup> *Obsolete Software Requirements (OSR)*

**FIGURE B.42** – Manières utilisées par des professionnels pour gérer les exigences obsoètes (source : *Figure 12* [403])

**Explication de l’assertion ES25 : Plusieurs artefacts ne sont pas conservés.**

*conscious non-optimal decision. We take a shortcut, knowing its consequences, and eventually pay its price.*

- *unintentional* : Sub-optimal decisions based on lack of diligence are unintentional forms of degradation. The asset is degraded as a result of a modification comprising an unconscious non-optimal decision. We are not aware of the other, better alternatives to our decision ; therefore, we do not foresee the consequences of selecting an alternative. There is no awareness of the fact that the usability of the asset can be hindered.
- *entropy* : According to Lehman, “as a software program is evolved its complexity increases unless work is done to maintain or reduce it” and “[it] will be perceived as of declining quality unless rigorously maintained and adapted to a changing operational environment” (Lehman, 1979, 1996). This ever increasing complexity and perceived quality decline might encompass the degradation of the assets involved in the development of the software system. The degradation that is due to the continuous evolution of the software, and which is not coming directly from the manipulation of the asset by the developers is entropy. Entropy can introduce degradation in the form of natural decay due to technology and market changes on assets even when we have quality management mechanisms to avoid such degradation.

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

Un article publié en 2013 présente les résultats d’un sondage consacré aux exigences logicielles devenues obsolètes [403]. À la question portant sur les méthodes utilisées pour identifier les exigences logicielles obsolètes, 26 des 219 professionnels informatiques ont indiqué qu’ils les supprimaient. Les différents comportements des professionnels informatiques sont indiqués dans la figure B.42. En 2020, l’article *Software Heritage : Why and How to Preserve Software Source Code* décrit l’initiative de conserver le logiciel qu’avec le code source et explique les limitations avec les autres artefacts :

[Traduction libre] La première raison de ce choix est pragmatique : [...] Le code source est généralement petit comparé à d’autres objets numériques, dense en informations et coûteux à produire, contrairement aux millions de photos et vidéos (de chats) échangées sur les réseaux sociaux. De plus, le code source est fortement redondant/dupliqué, ce qui permet des méthodes de stockage efficaces (voir section 7). Deuxièmement, les logiciels sont aujourd’hui massivement développés en code source libre, ce qui nous donne accès à l’historique des projets logiciels depuis leurs toutes premières phases. Il s’agit d’une information précieuse pour comprendre comment un logiciel naît et évolue, et nous souhaitons la préserver pour tout projet « important ». Malheureusement, lorsqu’un projet est à ses débuts, il est extrêmement difficile de savoir s’il deviendra un succès retentissant ou un échec. [...].<sup>419</sup> [74]

(voir énoncé p.213)

**Explication de l’assertion ES26 : Les artefacts ne contiennent pas toutes les connaissances.**

---

419.

*The first reason behind this choice is pragmatic : [...] Source code is usually small in comparison to other digital objects, information dense and expensive to produce, unlike the millions of (cat) pictures and videos exchanged on social media. Additionally, source code is heavily redundant/duplicated, allowing for efficient storage approaches (see Section 7). Second, software is nowadays massively developed in the open, so we get access to the history of software projects since their very early phase. This is a precious information for understanding how software is born and evolves and we want to preserve it for any “important” project. Unfortunately, when a project is in its infancy it is extremely hard to know whether it will grow into a king or a peasant. [...].*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

Un article de 1991, décrivant le cas d'étude d'une rétro-ingénierie, présente les raisons pour lesquelles il est difficile de reconstituer des documents de conception originaux à partir des documents issus du code source :

[Traduction libre]

- 1- biais d'implémentation : L'un des enjeux majeurs de la récupération d'une conception est la distinction entre les informations de conception et celles d'implémentation. Au niveau de l'implémentation, il peut être difficile d'identifier les informations à ne pas consigner dans la conception récupérée. Il peut être complexe d'« éliminer » des informations lors du processus d'abstraction, notamment lorsque l'objectif est de réimplémenter le système. Il est essentiel de comprendre que la conception ne constitue pas une documentation de programme. La documentation de programme reflète les détails d'implémentation du code source. La documentation de conception, quant à elle, offre une vision globale du fonctionnement du programme. Parmi les détails d'implémentation à surveiller figurent les noms des éléments de données, les types de données de base, la structuration des données, les interfaces système, les sections de code optimisées, le code influencé par l'architecture informatique sous-jacente et le code dont l'expression était biaisée par le langage d'implémentation d'origine.
- 2- traçabilité : Une conception récupérée doit consigner les liens entre les informations récupérées et les sources originales. Ceci permet d'assurer la traçabilité entre la conception récupérée et l'implémentation originale. Cette expérience a démontré qu'un environnement de développement logiciel existant peut être utilisé pour consigner une conception récupérée. Toutefois, un tel outil doit offrir la flexibilité nécessaire pour enregistrer des informations non prévues lors de sa création.
- 3- information du domaine : Les informations enregistrées dans un programme suffisent à expliquer « ce qui » est fait, mais pas « pourquoi ». Pour comprendre l'importance d'une étape de traitement, des informations plus détaillées sont souvent nécessaires. Ces informations ne sont

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

généralement pas consignées dans le code source ni dans la documentation existante. La compréhension du domaine d'application peut faciliter la récupération d'informations sur la finalité d'une fonction et son importance.

- 4- re-engineering : La rétro-ingénierie est le plus souvent effectuée sur des systèmes logiciels anciens dont la documentation est obsolète ou inexistante. [...].
- 5- documentation existante : Un dernier point concerne l'utilisation des commentaires et de la documentation existante. Ces sources d'information peuvent être obsolètes ou erronées. Elles doivent être utilisées avec précaution. Elles peuvent fournir des informations utiles sur un programme, mais aussi être trompeuses ou incorrectes. Le code source constitue la déclaration finale du fonctionnement réel du programme. <sup>420</sup> [44]

---

420.

- 1- *implementation bias* : Perhaps the most important issue in recovering a design is to separate design information from implementation information. At the implementation level it can be difficult to recognize what information should not be recorded in the recovered design. It can be difficult to 'throw away' information during the abstraction process, particularly, when the goal is to reimplement the system. It is necessary to realize that the design does not serve as program documentation. Program documentation reflects the implementation details of the source code. Design documentation reflects a higher-level view of 'how' a program functions. Implementation details that need to be watched for are data item names, basic data types, data structuring, system interfaces, code sections written to be efficient, code that is biased by the underlying computer architecture and code whose expression was biased by the original implementation language.
- 2- *traceability* : A recovered design should record links between recovered information and the original sources. This permits traceability between the recovered design and the original implementation. This experiment has shown that an existing software development environment can be used to record a recovered design. However, such a tool must provide the flexibility to record information not envisioned when the tool was created.
- 3- *domain information* : The information recorded in a program is sufficient to explain 'what' is being done, but not 'why'. To understand the significance of a processing step, deeper information is often required. This information is often not recorded in the source code or existing documentation. An understanding of the application domain can aid the recovery of information about the purpose of a function and its significance.
- 4- *re-engineering* : Reverse engineering will most often be done on older software systems whose documentation is out-of-date or non-existent . [...].
- 5- *existing documentation* : One last issue is the use of comments and existing documentation. These sources of information may be out-dated or incorrect. They must be used carefully. They

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

Une revue de littérature publiée en 2024 porte sur l’analyse des articles de rétro-ingénierie utilisant une approche par modèle [362]. Il y est abordé le langage de contrainte d’objet (OCL, *Object Constraint Language* en anglais). Les principaux défis recensés ainsi que leurs descriptions sont :

[Traduction libre]

- a : EXACTITUDE [...] Fournir des modèles complets et précis de systèmes logiciels peut s’avérer difficile, notamment pour les systèmes vastes et complexes présentant des erreurs et/ou des incohérences.
- b : HÉTÉROGÉNÉITÉ [...] Il est nécessaire de prendre en compte les variations de langages et de plateformes pour analyser correctement le système logiciel [...].
- c : ADAPTABILITÉ [...] traiter des systèmes logiciels réels et d’extraire les informations nécessaires du code source. En effet, ces systèmes possèdent des architectures complexes ou sont développés dans plusieurs langages de programmation. De plus, la taille et la complexité d’un système logiciel exigent davantage de temps et de ressources.
- d : GESTION DES COMPORTEMENTS DYNAMIQUES [...] capturer les comportements d’exécution et les dépendances entre les composants logiciels et de les intégrer dans différents modèles. Les approches traditionnelles de rétro-ingénierie privilégient souvent les artefacts statiques, tels que le code source, ce qui rend l’extraction de diagrammes comportementaux, comme les diagrammes de séquence et de communication UML, extrêmement difficile.
- e : COMPRÉHENSION DE LA SÉMANTIQUE DU CODE SOURCE [...] Extraire automatiquement ces informations du code source peut s’avérer très difficile, notamment pour les systèmes logiciels vastes et complexes. Des techniques interactives (semi-automatisées) nécessitant l’intervention d’experts du système sont généralement indispensables.

---

*can provide useful information about a program, but they can also be misleading or wrong. The source code is the final statement about what the program actually does.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

#### f : MANQUE DE TECHNIQUES D'APPRENTISSAGE AUTOMATIQUE POUR LA RÉTROINGÉNIERIE DE SYSTÈMES TECHNIQUES (MDRE)

On constate un manque d'approches MDRE utilisant l'apprentissage automatique pour extraire les spécifications du code source. Cette lacune dans la recherche actuelle souligne la nécessité d'étudier plusieurs techniques d'apprentissage automatique. Ces techniques facilitent les approches MDRE en identifiant les informations pertinentes dans le code source, en comprenant sa sémantique et en prenant en charge plusieurs langages et cadres d'application. L'apprentissage automatique, en particulier les grands modèles de langage (GML), peut être utilisé pour réduire l'effort humain et les ressources nécessaires à la rétro-ingénierie de systèmes logiciels vastes et complexes.

g : TRADUCTION DE PROGRAMMES Traduire les modèles de rétro-ingénieries en code cible avec une sémantique équivalente à celle du code source, notamment dans le cas de plusieurs langages de programmation cibles, représente un défi complexe. Certaines approches abordent ce problème en extrayant d'abord des représentations UML et OCL précises du code source, puis en utilisant un outil MDE, tel qu'AgileUML pour traduire le programme dans les langages cibles choisis. <sup>421</sup> [362]

---

421.

- a : *ACCURACY* [...] *Providing complete and accurate models of software systems can be difficult, especially with large and complex software systems that have errors and/or inconsistencies.*
- b : *HETEROGENEITY* [...] *It is necessary to consider such language and platform variations to properly analyze the software system [...].*
- c : *SCALABILITY* [...] *dealing with real-world software systems and extracting the required information from the source code. This is because these systems have complex architectures or are developed in multiple programming languages. Additionally, more time and resources are required as the size and complexity of a software system grows.*
- d : *HANDLING DYNAMIC BEHAVIOUR* [...] *capture runtime behaviours and dependencies between software components and include them in different models. Traditional reverse engineering approaches often prioritize static artifacts such as source code, making the extraction of behavioural diagrams such as UML sequence and communication diagrams extremely challenging.*
- e : *UNDERSTANDING THE SEMANTICS OF THE SOURCE CODE* [...] *need to be able to understand the semantics of the source code. [...] Extracting this information automatically from source code can be very difficult, especially with large and complex software systems. Interactive (semi-automated) techniques involving input from system experts are typically necessary.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

En 2011, un article présente les raisons expliquant la nécessité des connaissances des experts lors de l'utilisation d'instruments de rétro-ingénierie :

[Traduction libre]

- Les résultats sont soit incomplets, soit imprécis, surtout lorsqu'ils sont entièrement automatisés ; et surtout, ils ne tiennent pas compte des connaissances des experts [...].
- Les résultats sont semi-automatiques ; par exemple, l'intervention d'experts humains est nécessaire pour compléter ou corriger les informations extraites automatiquement et pour produire des vues utiles au développeur [...]. <sup>422</sup> [48]

(voir énoncé p.213)

#### Les instruments

**Explication de l'assertion ES27 : Les définitions des *CASE tools* sont précaires.**

La précarité des définitions s'observe entre les différentes éditions de l'ouvrage de Roger S. Pressman, *Software engineering : a practitioner's approach* :

---

*f : LACK OF MACHINE LEARNING TECHNIQUES FOR MDRE There is a lack of MDRE approaches that use machine learning to extract specifications from source code. This appears to be a gap in current research and indicates the need to study several machine learning techniques. These techniques assist MDRE approaches by recognizing relevant information within source code, understanding its semantics, and dealing with multiple programming languages and frameworks. Machine learning, especially LLMs, can be used to reduce the human effort and various resources required to reverse engineer large and complex software systems.*

*g : PROGRAM TRANSLATION Translating reverse-engineered models to target code with equivalent semantics to the source, particularly in the case of multiple target programming languages, presents a complex challenge. Some approaches address this problem by first abstracting precise UML and OCL representations from the source code and then using an MDE tool such as AgileUML to translate the program into the chosen target languages.*

422.

- *They are either incomplete or imprecise, especially when they are completely automatic ; above all, they do not account for experts' knowledge [...].*
- *They are semi-automatic, for example, inputs from human experts are required to complement or correct the information automatically extracted and to produce useful views for the developer [...].*



### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

- Dans la cinquième édition, le chapitre 31 (*Computer-Aided Software Engineering*) fournit 24 définitions de types de *CASE tools* [307] : *Business process engineering tools*, *Process modeling and management tools*, *Project planning tools*, *Risk analysis tools*, *Project management tools*, *Requirements tracing tools*, *Metrics and management tools*, *Documentation tools*, *System software tools*, *Quality assurance tools*, *Database management tools*, *Software configuration management tools*, *Analysis and design tools*, *PRO/SIM tools*, *Interface design and development tools*, *Prototyping tools*, *Programming tools*, *Web development tools*, *Integration and testing tools*, *Static analysis tools*, *Dynamic analysis tools*, *Test management tools*, *Client/server testing tools*, *Reengineering tools*.
- Dans la 7e édition, le terme *CASE tool* est complètement absent. L’auteur y aborde toutefois certains instruments, tels que les *Process Modeling Tools*, les *use-case-tool*, etc. [308].

(voir énoncé p.214)

#### Explication de l’assertion ES28 : Le vocabulaire utilisé pour définir des *CASE tools* crée de la confusion.

Les instruments qui intègrent les processus ont des rôles similaires, mais des noms différents :

- environnements axés sur les processus : [Traduction libre] « [...] reposent sur une définition formelle du processus logiciel. Une application informatique, appelée pilote ou moteur de processus, utilise cette définition pour guider les activités de développement en automatisant des fragments de processus [...]. » <sup>423</sup> [91] ;
- environnement CASE intégré (I-CASE) : [Traduction libre] « [...] combine divers outils et un large éventail d’informations de manière à assurer la communication entre les outils, entre les personnes et tout au long du processus logiciel. » <sup>424</sup> [307] ;

---

423. « [...] is based on a formal definition of the software process. A computerized application called a process driver or process engine uses this definition to guide development activities by automating process fragments [...]. »

424. « [...] combines a variety of different tools and a spectrum of information in a way that enables closure of communication among tools, between people, and across the software process. »

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

- environnements d'ingénierie logicielle axés sur les processus : [Traduction libre] « [...] abandonne la notion de modèle de processus prédéfini. Ils prennent en charge une plus grande variété de processus [...] Les développeurs sont informés des activités à réaliser, les tâches automatiques sont exécutées sans intervention humaine et la cohérence entre les documents est assurée jusqu'à un certain niveau. » <sup>425</sup> [110] ;
- gestion du cycle de vie des applications (ALM) : [Traduction libre] « [...] comprend les disciplines fondamentales que sont la définition et la gestion des exigences, la gestion des actifs, le développement, la compilation, les tests et la mise en production. Toutes ces activités sont planifiées par la gestion de projet et orchestrées à l'aide d'un processus. [...] L'ALM implique la coordination des activités de développement logiciel et des actifs afin de produire et de gérer les applications logicielles tout au long de leur cycle de vie. » <sup>426</sup> [107].

Un article sur la recherche en processus logiciel d'Alfonso Fuggetta et Elisabetta Di Nitto explique que les ALM font partie des logiciels de gestion des processus, même si ce n'est pas explicitement décrit dans ces instruments :

[Traduction libre] Les suites de gestion du cycle de vie des applications (ALM) constituent une catégorie de produits issue à la fois de l'industrie du logiciel traditionnelle et de la communauté open source. Bien qu'elles ne soient pas explicitement liées à la recherche sur les processus logiciels, elles offrent des mécanismes d'automatisation de certaines tâches (généralement la compilation et les tests) et permettent de relier les tâches de gestion aux activités de développement logiciel. De plus, elles proposent des connecteurs vers des outils externes pour des fonctionnalités spéci-

---

425. « [...] *give up the notion of a predefined process model. They support a wider variety of processes [ . . . ] software developers are reminded of activities which have to be carried out, automatic activities are executed without human interaction and consistency between documents is enforced up to a certain level.* »

426. « [...] *consists of the core disciplines of requirements definition and management, asset management, development, build, test, and release that are all planned by project management and orchestrated by using some form of process. [ . . . ] ALM involves the coordination of software development activities and assets to produce and manage software applications throughout their life cycle.* »

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

fiques, telles que la gestion de la configuration, le suivi des bogues, la gestion des exigences, etc. <sup>427</sup> [92]

(voir énoncé p.214)

#### Explication le l'assertion ES29 : Les CASE tools suivent les tendances des méthodes de développement.

Dans un ouvrage de 1993 ayant pour titre *The Integrated CASE Tools Handbook* :

[Traduction libre] Je suis fermement convaincu que ce triumvirat — une compréhension plus claire du problème d'intégration qu'auparavant, l'engagement des fournisseurs à réaliser l'intégration (motivé par les exigences du marché) et les technologies sous-jacentes (matériel, logiciel et communications) — permettra d'atteindre le niveau d'intégration CASE souhaitée par les utilisateurs depuis les débuts de l'expérimentation des outils. <sup>428</sup> [365]

Un article de 1998 ayant pour titre *A comparative evaluation of CASE tools* :

[Traduction libre] Il peut être difficile d'évaluer des produits complexes dotés de multiples attributs. Chaque produit présente un ensemble différent de points forts et de points faibles. Les consommateurs (utilisateurs) doivent évaluer les compromis entre chaque attribut pour chaque produit.

---

427.

*Application Lifecycle Management (ALM) suites are a class of products originated partly by the conventional software industry and partly by the open source community. Even though they do not appear as explicitly connected to Software Process research, they offer mechanisms for automating some tasks (typically, build and test), and for connecting managerial tasks with software development activities. Moreover, they offer connectors to external tools for specific functionality such as configuration management, bug tracking, requirement management and the like.*

428.

*It is my firm belief that this triumvirate—a clearer understanding of the integration problem than in the past, vendor commitment to achieving integration (driven by market demands), and the underlying technologies (hardware, software, and communications)—will bring about the level of CASE integration desired by users since the early days of tool experimentation.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

Les concepteurs de logiciels et les spécialistes du marketing doivent identifier les attributs les plus utiles et les plus demandés. [...] <sup>429</sup> [306]

Un article de 2003 résume le remplacement des méthodes structurées par les méthodes agiles. Deux courants se sont produits ;

[Traduction libre]

- 1) Adaptation : elle a consisté à combiner une approche orientée objet avec des méthodologies de développement structurées reconnues ;
- 2) Assimilation : elle a privilégié l'utilisation d'une méthodologie orientée objet pour le développement de systèmes logiciels, tout en suivant le modèle traditionnel du cycle de vie logiciel en cascade. <sup>430</sup> [50]

Un article de 2009 ayant pour titre *Agile Tool Market Growing with the Philosophy* décrit la croissance de certains types d'instruments [106]. Il y a :

[Traduction libre]

[—] Selon Matt Light, analyste chez Gartner Group, les dernières tendances en matière de développement agile s'inscrivent dans la continuité des méthodes de développement rapide d'applications (RAD) apparues au début des années 90.

[—] Borland a dévoilé Borland Management Solutions (BMS), une suite logicielle de gestion composée de trois produits, conçue pour fonctionner avec les méthodes agiles, ainsi qu'avec d'autres modèles de développement itératifs, voire en cascade.

---

429.

*It can be difficult to evaluate complex products with multiple attributes. Each product provides a different set of strengths and weaknesses. Consumers (users) have to evaluate the trades between each attribute for each product. Software designers and marketers have to identify which attributes are the most useful and in demand. [...]*

430.

- 1) Adaptation : it has been concerned with mixing an objectoriented approach with well-known structured development methodologies ;
- 2) Assimilation : it has emphasized the use of an object-oriented methodology for developing software systems, but has followed the traditional waterfall software life cycle model.

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

[—] Holler, de VersionOne, compare la transformation du secteur vers l'agilité à l'essor des applications de gestion de la relation client (CRM) dans les années 90. « La différence, dans notre contexte, réside dans le fait qu'il ne s'agit pas seulement d'un défi d'automatisation, mais aussi d'un défi de processus, d'une façon fondamentalement différente de faire des affaires », explique-t-il. « J'estime que cette évolution s'étalera sur 10 à 15 ans, et nous n'en sommes qu'à la cinquième ou sixième année. Il nous reste donc encore cinq à dix ans devant nous. ». <sup>431</sup> [106]

(voir énoncé p.214)

#### **Explication de l'assertion ES30 : Les participants informatiques se font leurrer sur les fonctionnalités offertes par certains *CASE tools*.**

Dans un ouvrage de 1993 ayant pour titre *The Integrated CASE Tools Handbook* :

[Traduction libre] les limites des logiciels CASE des années 1980, notamment : [...] la surestimation des capacités des outils par les fournisseurs.  
[...] <sup>432</sup> [365]

En 1988, Jeffrey D. Ullman décrit l'utilisation de certains termes pour des objectifs mercantiles :

[Traduction libre] La notion de « connaissance » est complexe à définir formellement, et le fait qu'elle se vende bien aujourd'hui la rend encore

---

431.

[—] Gartner Group analyst Matt Light says the latest agile-development trends are a continuation of rapid-application-development (RAD) methods first employed in the early '90s.

[—] Borland unveiled its Borland Management Solutions (BMS) package, a three-product management suite intended to work with agile as well as other iterative and even waterfall development models.

[—] VersionOne's Holler compares the industry transformation to agile to the boom of customer-relationship-management (CRM) applications in the '90s. Holler says. "The slight difference with our environment is that this is not only an automation challenge but a process challenge, a fundamentally different way of doing business, and I see it as a 10- to 15-year evolution, and we're only in year five or six. So we have another five to 10 years ahead of us."

432.

the limitations of 1980s-era CASE, including : [...] vendors overselling tool capabilities.  
[...]

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

plus difficile à appréhender. Attribuer la « connaissance » à un logiciel ou affirmer qu'il utilise l'« intelligence artificielle » semble le rendre plus attractif, même si ses performances et ses fonctionnalités ne sont pas nécessairement supérieures à celles d'un produit similaire qui revendique ces mêmes qualités.<sup>433</sup> [386]

En 2014, Alfonso Fuggetta à l'intérieur d'un article résume la recherche effectuée sur les logiciels qui intègrent les processus, il cite D. Chappel<sup>434</sup> [54] :

[Traduction libre] Dans [9], la gestion du cycle de vie des applications est présentée comme une discipline visant à prendre en charge également les procédures d'exploitation des applications et leur gouvernance. Cependant, comme le souligne l'auteur, les outils ALM actuels ne permettent pas d'intégrer ces aspects à ceux liés au développement logiciel.<sup>435</sup> [92]

En 2018, un article décrit un cas où l'intégration comprend un ALM [205]. Le cas se produit dans une entreprise de 1300 professionnels informatique. Dans l'objectif d'améliorer les procédés trois tentatives d'intégrer différents instruments sont réalisées :

- la première tentative a été abandonnée à cause des difficultés d'intégrations et de la courbe d'apprentissage élevé à l'utilisation des instruments ;
- la deuxième tentative a été abandonnée à cause du coût élevé à la maintenance et d'une couverture inadéquate des besoins ;

---

433.

*"Knowledge" is a tricky notion to define formally, and it has been made trickier by the fact that today, "knowledge" sells well. It appears that attributing "knowledge" to your software product, or saying that it uses "artificial intelligence" make the product more attractive, even though the performance and functionality may be no better than that of a similar product and claimed to possess these qualities.*

434. L'article de Chappel est en allemand, alors il est utilisé cette de Fuggetta.

435.

*In [9] Application Lifecycle Management is presented as a discipline aiming at taking care also of application operation procedures and managerial governance. However, as the author claims, current ALM tools do not support the integration of these aspects with those related to software development.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**TABLEAU B.17** – Tentatives d'intégrations (inspiré de [205])

| tentative | fonction<br>utilisé<br>instrument | gestion<br>de<br>projet | gestion<br>d'exi-<br>gence | gestion<br>essai | concep-<br>tion | justifi-<br>catif<br>concep-<br>tion | gestion du<br>dévelop-<br>pement | gestion<br>de code<br>source |
|-----------|-----------------------------------|-------------------------|----------------------------|------------------|-----------------|--------------------------------------|----------------------------------|------------------------------|
| première  | Microsoft TFS                     | oui                     | oui                        | oui              |                 |                                      |                                  |                              |
|           | Enterprise Architect              |                         |                            |                  | oui             |                                      |                                  |                              |
|           | Atlassian Jira                    |                         |                            |                  |                 |                                      | oui                              |                              |
|           | Apache SVN                        |                         |                            |                  |                 |                                      |                                  | oui                          |
| deuxième  | HP ALM                            |                         | oui                        | oui              |                 |                                      |                                  |                              |
|           | Atlassian Jira                    | oui                     |                            |                  |                 | oui                                  | oui                              |                              |
|           | Visual SVN                        |                         |                            |                  |                 |                                      |                                  | oui                          |
| troisième | Atlassian Jira                    | oui                     | oui                        | oui              |                 |                                      | oui                              |                              |
|           | Atlassian Confluence              |                         |                            |                  | oui             |                                      |                                  |                              |
|           | Atlassian Bitbucket               |                         |                            |                  |                 |                                      |                                  | oui                          |
|           | Jenkins                           |                         |                            |                  |                 |                                      |                                  |                              |
|           | SonarSource<br>SonarQube          |                         |                            |                  |                 |                                      |                                  |                              |
|           | JBake<br>Artifactory              |                         |                            |                  |                 |                                      |                                  |                              |

<sup>a</sup> Les instruments Jenkins, SonarSource SonarQube et JBake Artifactory ont été introduit après la découverte de nouveaux besoins.

- la troisième tentative comprend plusieurs instruments d'une même entreprise, cela facilite l'usage, l'intégration et diminue le coût des licences.

Les instruments utilisés dans ces tentatives et besoins qu'ils remplissent sont énumérés dans le tableau B.17. Plusieurs ont le terme **ALM** dans leurs noms ou affirment être des ALM. Toujours en 2018, un autre article décrit l'adaptation des ALM dans les entreprises [384]. Il y est concédé les problèmes d'intégrations avec les ALM, mais l'article annonce qu'il n'y aura plus de problème avec les ALM 2.0. L'article cite

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

plusieurs ALM 2.0, il y a entre autres ; Atlassian Suite, HP ALM, IBM Jazz, Polarion, TFS <sup>436</sup>

(voir énoncé p.214)

**Explication de l’assertion ES31 : Ce sont les organisations qui contrôlent les instruments.**

L’ouvrage *Introduction to Digital Economics : Foundations, Business Models and Case Studies* de Harald Øverby décrit différentes stratégies qui permettent aux entreprises de garder captifs leurs clients [414]. Ces stratégies sont ;

- la maintenance et les pièces de rechange ;
- les formations ;
- les incompatibilités et compatibilités ;
- la perte d’information potentielle ;
- la non-viabilité économique de faire autrement ;
- les contrats difficiles à résilier ;
- les programmes de loyauté ;
- les produits disponibles en lots ;
- les algorithmes de recherche ;
- les produits liés.

En 2024, une analyse, réalisée sur la portabilité et l’interopérabilité des données, conduit à suggérer l’intervention publique afin d’imposer un cadre réglementaire pour améliorer la portabilité et l’interopérabilité des entités informatiques :

[Traduction libre] Le secteur privé s’est activement mobilisé, individuellement ou collectivement, pour améliorer la portabilité des données. Toutefois, les avantages privés et les avantages sociaux ne convergent pas pleinement, et l’intervention publique s’avère indispensable. Il est pertinent d’intégrer à notre cadre réglementaire actuel un cadre qui prenne en

---

436. TFS pour Team Foundation Server est un produit Microsoft qui a été renommé en 2018 Azure DevOps Server [Wikipedia, « Azure DevOps Server », [https://en.wikipedia.org/wiki/Azure\\_DevOps\\_Server](https://en.wikipedia.org/wiki/Azure_DevOps_Server) (consulté le 2025-11-03).]



### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

compte les effets potentiels de la portabilité et de l'interopérabilité des données.<sup>437</sup> [349]

En dernier lieu, voici un exemple où la société IBM décide du sort du *CASE tools* Rational Synergy qui est un logiciel de gestion de configuration (SCM, *Software Configuration Management* en anglais). Sur le site officiel, il est indiqué que les intégrations<sup>438</sup> seront retirées :

[Traduction libre] Les intégrations suivantes sont obsolètes. IBM continue de les prendre en charge, mais ne prévoit plus de les améliorer et pourrait les supprimer dans une version ultérieure du produit. Pour plus d'informations, consultez la section « Nouveautés de la gestion des flux de travail d'ingénierie dans la version 7.0.3 ». <sup>439</sup> <sup>440</sup>

La documentation aussi sera retirée :

[Traduction libre] Étapes : Les liens vers la documentation en ligne actuelle sont les suivants : [...]. Cette documentation en ligne sera retirée après le 30 septembre 2024. Pour accéder à la documentation de Rational Synergy et Rational Change, vous devez installer l'application IBM Documentation

---

437.

*The private sector has been active in making efforts, individually or jointly, to improve data portability. Nevertheless, private benefits and social benefits are not fully aligned and there is a clear role for public intervention. Adding a regulatory overlay to our current regulatory and enforcement environment that recognizes the potential effects of data portability and interoperability is appealing.*

438. Voici les intégrations : *Integrating Engineering Workflow Management and Rational Synergy, Integrating Engineering Workflow Management and Rational Change, Integrating Engineering Workflow Management and Subversion, Integrating Engineering Workflow Management and ClearCase, Integrating Engineering Workflow Management and Rational ClearQuest, Integrating with Bitbucket, Integrating with Sametime*

439.

*The following integrations are deprecated. IBM continues to support the deprecated integrations, but no longer plans to enhance it and might remove the capability in a subsequent release of the product. For the announcement, see What's new in Engineering Workflow Management in 7.0.3.*

440. IBM, site web, « Deprecated integrations », <https://www.ibm.com/docs/en/engineering-lifecycle-management-suite/workflow-management/7.0.3?topic=integrating-deprecated-integrations> (consulté le 2025-04-12).

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

Offline et télécharger la documentation avant son retrait. [...] Informations complémentaires : Les fichiers de documentation hors ligne pourraient ne plus être disponibles après la fin de vie de Rational Synergy et Rational Change.<sup>441 442</sup>  
(voir énoncé p.214)

#### Améliorer le cadre sociétal

#### Explication de la définition AS04 : Standard

La définition formulée est : document qui contient des règles de pratiques communes ne provenant pas d'un organisme de normalisation.

La littérature fournit ces informations qui ont servi à construire cette définition :

- observation : le guide « Normalisation et activités connexes — Vocabulaire général » exclut le terme standard du vocabulaire français [128] ;
- définition 1 : « Règle fixe à l'intérieur d'une entreprise pour caractériser un produit, une méthode de travail, une quantité à produire, le montant d'un budget. » [192] ;
- définition 2 : « Ensemble de règles ou de spécifications relatives à des activités ou à leurs produits, émanant du secteur privé, mais adopté comme cadre de référence par une large part des acteurs d'un domaine d'application donné. » [277] ;
- note : « Les standards peuvent émaner d'entreprises qui se sont imposées sur le marché, ou encore de consortiums formés expressément dans le but de promou-

---

441.

*Steps The current online documentation links are : [...]. This online documentation is expected to be withdrawn after 30 September 2024. To access the Rational Synergy and Rational Change documentation one needs to install the IBM Documentation Offline application and download the documentation before it is withdrawn online. [...] Additional Information The offline documentation files may not be available after the end-of-life of Rational Synergy and Rational Change.*

442. IBM, site web, « How can I access the Rational Synergy and Rational Change online documentation after the products' end of life? », <https://www.ibm.com/support/pages/how-can-i-access-rational-synergy-and-rational-change-online-documentation-after-products-end-life> (consulté le 2025-04-12).

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

voir l'adoption de pratiques communes, mais qui ne sont pas des organismes de normalisation. » [277].

(voir énoncé p.215)

**Explication de l'assertion AS06 : Le respect d'un standard ou d'une norme peut se faire ou non d'une manière coercitive.**

Dans son ouvrage *Software Processes and Life Cycle Models : An Introduction to Modelling, Using and Managing Agile, Plan-Driven and Hybrid Processes*, Ralf Kneuper partage que, dans la pratique, les règlements touchant aux standards ou aux normes ne sont pas toujours contraignants :

[Traduction libre] Il incombe à la gouvernance et à la direction (exécutive) de définir les règles, etc., dont le respect est attendu au sein d'une organisation. En théorie, les réglementations légales devraient bien sûr toujours être contraignantes, mais la pratique montre que ce n'est pas toujours le cas.<sup>443</sup> [162]

En 2021, une analyse se penche sur les règlements et la famille de normes ISO 27000 [311]. La norme 27001 concerne la sécurité de l'information et les systèmes de gestion de la sécurité de l'information. Elle définit notamment les exigences nécessaires pour établir un système d'information sécuritaire [142]. L'analyse vérifie l'application de cette norme selon différents États (l'Australie, l'Union européenne, l'Allemagne, l'Inde, le Royaume-Uni, etc), secteurs (le public, le privé, l'énergie, le financier, la santé, les communications et les médias) ainsi que certains domaines (la protection des données personnelles, la sécurité de l'information ainsi que la numérisation et l'archivage électronique). Au total, parmi les 66 combinaisons ainsi obtenues, 47 reposent sur une conformité acquise de manière volontaire et 19, sur une conformité obligatoire. Il est toutefois à noter que ni la Chine ni les États-Unis font partie des États analysés, même s'ils occupent une place importante.

---

443.

*It is the task of governance and (executive) management to define the rules etc. for which conformance is expected within an organisation. In theory, legal regulations should of course always be binding but practice shows that this is not always the case.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

Autre exemple : un article publié en 2017 décrit l'importance du logiciel dans le domaine de la santé et rappelle les dangers à diminuer la réglementation [343]. Cet article retrace notamment la chronologie des pouvoirs accordés à l'US FDA) :

- Depuis 1976, l'US FDA possède les pouvoirs lui permettant de faire respecter les standards sur les appareils médicaux.
- En 2016, le *21st Century Cures Act* retire à l'US FDA le pouvoir d'évaluer certains logiciels, comme les dossiers médicaux électroniques et les logiciels d'aide à la décision.

(voir énoncé p.215)

#### **Explication de l'assertion AS07 : Les standards ou les normes varient à travers le temps.**

Un article de 2010 raconte l'historique des standards militaires aux États-Unis, et en particulier ceux des logiciels :

[Traduction libre]

[—] Le ministère de la Défense a été créé à la demande du Congrès dans les années 1950. Bien qu'une fiabilité accrue ait été brièvement mentionnée comme objectif, les économies de coûts et les « cas où le manque de pièces de rechange standardisées handicapait les opérations militaires » étaient au cœur du Programme de normalisation de la défense, qui a finalement généré des milliers de normes militaires (MIL-STD).

[—] Elle a finalement obtenu l'approbation de la Marine sous la référence MIL-STD-1679 à la fin de 1978. [...] Les exigences de la norme MIL-STD-1679 incluaient une technique de génie logiciel courante appelée conception descendante. [...] On retrouve, sous une forme ou une autre, bon nombre des exigences de la norme MIL-STD-1679 dans l'article de synthèse de Boehm de 1976.

[—] En 1976, l'armée représentait encore un cinquième des achats de logiciels aux États-Unis, mais, entre la fin des années 1970 et les années 1990, le marché commercial a connu une croissance explosive.

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

[—] La norme MIL-STD-498 a donc été conçue avec une date d'expiration : une dérogation de deux ans. Cette dérogation a d'ailleurs été prolongée, mais elle a finalement expiré en 1998, lorsqu'une norme commerciale développée par l'IEEE a remplacé la MIL-STD-498. De plus, la nouvelle stratégie du Département de la Défense a totalement abandonné la notion de normes logicielles imposées contractuellement ; l'adoption de la norme de développement logiciel IEEE 12207 par un contractant pour un projet donné était entièrement volontaire. L'armée avait ainsi cédé l'intégralité du contrôle des processus de développement logiciel à ses contractants commerciaux. <sup>444</sup> [217]

En 2012, un article propose une analyse comparative sur l'évolution de ces normes [61]. Selon le comparatif, le seul concept abandonné lors du passage du DOD à l'IEEE est les gabarits de documents.

Un autre article répertorie les approbations données par l'US FDA entre 1976 et 2020, en voici quelques-unes :

[Traduction libre]

[—] Les amendements de 1976 permettaient aux fabricants de demander à la FDA une classification initiale en classe I ou II, mais le processus né-

---

444.

[—] *DoD at the behest of Congress in the 1950s. Although improved reliability received a brief mention as a goal, cost savings and “instances when the lack of standardized replacement parts handicapped military operations” were the main focus of the Defense Standardization Program, which eventually generated thousands of military standards (MIL-STDs).*

[—] *finally achieving Navy approval as MIL-STD-1679 at the end of 1978. [...] MIL-STD-1679's requirements included a popular software engineering technique called top-down design [...] many of the MIL-STD-1679 requirements can be found in Boehm's 1976 review article in some form.*

[—] *In 1976, the military still accounted for one-fifth of software purchases in the US, but between the late 1970s and the 1990s, the commercial market grew explosively.*

[—] *MIL-STD-498 therefore came into the world with an expiration date—a two-year waiver. In fact, the waiver was extended, but it finally expired in 1998, when a commercial standard developed by the IEEE replaced MIL-STD-498. What's more, the DoD's new strategy entirely abandoned the notion of contractually imposed software standards ; adoption of the IEEE 12207 software development standard by a contractor for a particular project would be wholly voluntary. The military had ceded control over software development processes entirely to their commercial contractors.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

cessitait un examen par un comité consultatif. [...] En 1997, ce processus a été simplifié en une voie de classification « de novo », mais ne pouvait intervenir qu’après le dépôt d’une demande 510(k) et la conclusion d’une absence d’équivalence substantielle.<sup>38</sup> En 2012, le Congrès a encore simplifié la procédure en supprimant l’obligation de soumettre au préalable une demande 510(k).

[—] Reconnaissant la nature itérative des logiciels, la FDA a annoncé en 2017 le programme pilote de précertification des logiciels (PreCert), destiné à évaluer si les sociétés de logiciels ayant démontré une « culture de la qualité » pouvaient commercialiser certains types de produits de santé numériques sans examen de la FDA ou après un examen abrégé. <sup>445</sup> [67]

(voir énoncé p.215)

**Explication de l’assertion AS08 : Même dans les secteurs critiques où les standards et normes existent, plusieurs problèmes surviennent.**

L’ouvrage *Requirements Engineering for Safety-Critical Systems* cite une étude pré-occupante de l’US FDA :

[Traduction libre] Une étude menée par la FDA entre 2005 et 2011 visait à identifier les rappels de dispositifs médicaux liés à des problèmes logiciels. Cette étude, portant sur 5 792 rappels, a révélé que 1 112 d’entre eux (19,4 %) étaient liés à des problèmes logiciels. <sup>446</sup> [105]

(voir énoncé p.215)

---

445.

[—] *The 1976 Amendments allowed manufacturers to petition the FDA for initial classification as class I or II, but the process required advisory panel review. [...] In 1997, this process was streamlined into the “de novo” classification pathway but could occur only after 510(k) submission and a finding of not substantially equivalent.<sup>38</sup> In 2012, Congress further simplified the procedure by removing the requirement to first submit a 510(k).*

[—] *Recognizing the iterative nature of software, <sup>48</sup> the FDA in 2017 announced the Software Precertification (PreCert) Pilot Program, intended to evaluate whether software companies with a demonstrated “culture of quality” could market certain types of digital health products without FDA review or after a shortened review.*

446.

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

#### Explication de l’assertion AS09 : Le langage UML était le langage le plus populaire entre 1997 et 2004

En 2025, un article présente un aperçu de l’utilisation du langage UML en analysant plus de treize mille projets GitHub, couvrant la période de 1999 à 2022, afin d’examiner les extensions utilisées [341]. Les extensions recensées sont : .zargo .argo .pgml .zuml .prj .diagram .dia .uxf .uml .xmi .ecore .ucls .umlclass .mmd .plantuml et .puml. Le jeu de données est formé comme ceci :

[Traduction libre] [...] nous avons sélectionné les projets comportant au moins 2 000 commits afin d’éliminer les dépôts de test. Nous avons également sélectionné ceux comptant au moins 10 contributeurs afin de garantir la nécessité de la collaboration, ce qui accroît l’utilité des diagrammes. Enfin, nous avons retenu les projets ayant reçu au moins 100 étoiles afin de nous assurer qu’ils sont considérés comme utiles par la communauté des logiciels libres. Notre ensemble de données final comprend 13 152 dépôts, que nous avons clonés localement le 1er avril 2023.<sup>447</sup> [341]

Il y est également décrit la popularité de ce langage à travers le temps :

[Traduction libre]

[—] L’adoption d’UML comme norme par l’Object Management Group en 1997 a marqué la fin des guerres de méthodes et le début de l’ère « post-UML » [...]

---

*A study to identify software-related recalls from 2005 to 2011 was conducted by FDA. This study analyzed 5,792 medical device recalls and found that 1,112 (19.4%) of them were related to software.*

447.

*[...] we selected projects with at least 2,000 commits to eliminate toy repositories. We selected projects with at least 10 contributors to ensure the need for collaboration, which increases the utility of diagrams. We selected projects with at least 100 stars to ensure projects are considered useful by the OSS community. Our final dataset consists of 13,152 repositories, which we cloned locally on April 1, 2023*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

[—] Des révisions mineures de la spécification UML ont été publiées à un rythme annuel moyen entre 1997 et 2003 [16]. La publication d’UML 2.0 a pris plus de deux ans, ce qui a freiné son développement.

[—] Le déclin de la popularité d’UML depuis 2004 a exacerbé le problème.

[—] Microsoft Visual Studio a supprimé la prise en charge d’UML en 2016 en raison de sa sous-utilisation par les clients.

[—] après 2017, avec l’arrivée de fichiers UML textuels lisibles, versionnables et faciles à maintenir (par exemple, PlantUML, Mermaid).

[—] À son apogée, environ 3 % des dépôts actifs contenaient un diagramme UML. <sup>448</sup> [341]

(voir énoncé p.215)

**Explication de l’assertion AS10 : Le MOF et l’EMF sont respectivement le langage et le système de metamodélisation les plus utilisés.**

En 2018, Ralf Kneuper l’indique dans son ouvrage :

[Traduction libre] Aujourd’hui, la notation principale utilisée pour cette tâche est le Meta Object Facility (MOF) [...] Le Meta Object Facility (MOF) est une norme de l’Object Management Group (OMG) qui décrit les différents niveaux nécessaires à la métamodélisation. Cette norme est également publiée sous la référence ISO/IEC 19508 :2014. Le MOF prend en charge la description de divers langages largement utilisés, principalement UML et BPMN, tous deux également définis par des normes

---

448.

[—] *The adoption of UML as a standard by the Object Management Group in 1997 marked the end of the method wars and the beginning of the “After UML” era [...]*

[—] *Minor revisions of the UML specification have been issued with an average yearly pace between 1997 and 2003 [16]. UML 2.0 took over two years to be released, losing the momentum.*

[—] *UML’s decline in popularity since 2004 exacerbated the problem.*

[—] *Microsoft Visual Studio removed UML support in 2016 due to underutilization by clients*

[—] *[...] after 2017, coinciding with the advent of human-readable, versionable, and easily maintainable textbased UML files (e.g., PlantUML, Mermaid).*

[—] *At its peak, about 3.0 % of active repositories contained a UML diagram.*



### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

OMG. La prise en charge du métamodèle d'ingénierie des processus logiciels (SPEM), qui sera décrite dans la section 2.4.2, est particulièrement importante dans ce contexte. À l'instar du CDIF, l'objectif principal du MOF est de fournir une base pour la prise en charge de ces langages de modélisation par les outils, notamment l'échange de modèles entre différents outils.<sup>449</sup> [162]

De plus, un ouvrage de 2023 qualifie l'Eclipse Modeling Frameworks (EMF) de populaire :

[Traduction libre] La pile EMF est un choix populaire, car elle offre un riche écosystème d'extensions permettant d'aborder des aspects importants, tels que la spécification des contraintes d'un langage de modélisation (par exemple, OCL, ECL), la transformation entre plusieurs langages (par exemple, QVT, Viatra) et la spécification de la sémantique d'un langage de modélisation (par exemple, Xsemantics).<sup>450</sup> [210]

Un article fait l'inventaire des dépôts GitHub avec des modèles sous la forme EMF, un type de format pour métamodèles très populaire. Cet examen démontre cependant que la popularité de ce format ne va pas en grandissant (figure B.43) du moins en code source ouvert. D'ailleurs, un seul dépôt fait augmenter considérablement le nombre de modèles sous la forme EMF qui est la forme la plus populaire :

---

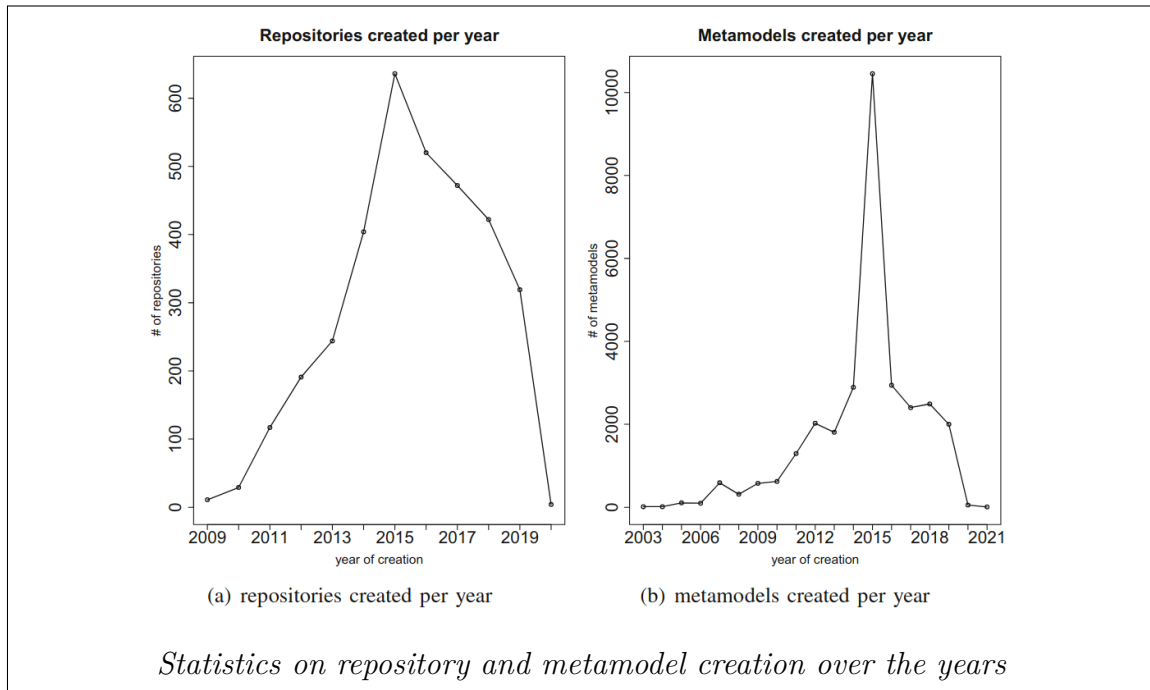
449.

*Today, the main notation used for this task is the Meta Object Facility (MOF) [...] The Meta Object Facility (MOF). MOF is an Object Management Group (OMG) standard that describes the multiple levels needed in meta-modelling, also published as ISO/IEC 19508 :2014. MOF supports the description of various widely-used languages, mainly UML and BPMN, both also defined by OMG standards. Particularly important in this context is the support for the Software Process Engineering Metamodel (SPEM) which will be described in Sect. 2.4.2. Similar to CDIF, the main purpose of MOF is to provide a basis for tool support for such modelling languages, including the exchange of models between different tools.*

450.

*The EMF stack is a popular choice because it provides a rich ecosystem of extensions for tackling important aspects such as specifying constraints for a modeling language (e.g., OCL, ECL), transformation across multiple languages (e.g., QVT, Viatra), and specifying the semantics of a modeling language (e.g., Xsemantics).*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.43** – Nombre de dépôts et de métamodèles créés sur GitHub entre 2003 et 2020 (source : *Figure 8* [20])

[Traduction libre] On observe une augmentation exceptionnelle du nombre de métamodèles créés en 2015, qui peut être principalement attribuée au dépôt *damenac/puzzle*, qui compte 7 483 métamodèles [...].<sup>451</sup> [20]  
(voir énoncé p.215)

**Explication de l’assertion AS11 : Les instruments posent problème pour le MOF ou l’EMF.**

Un sondage sur le MOF indique les difficultés présentes :

[Traduction libre] Ce sondage indique que l’utilisation conjointe d’OCL et de MOF est une tâche difficile et que les experts sont plus ou moins

451.

*There is an exceptional spike in the number of created metamodels in 2015, which can mostly be attributed to the repository *damenac/puzzle*, which has 7,483 metamodels [...].*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

susceptibles de maîtriser la logique d'OCL pour une métamodélisation précise.<sup>452</sup> [46]

Publié en 2016, l'article *The Problems with Eclipse Modeling Tools : A Topic Analysis of Eclipse Forums* soulève les principaux problèmes suivants :

[Traduction libre]

- Problèmes de plugiciels : les problèmes liés à l'architecture des plugiciels de l'outil, tels que les dépendances, la configuration, l'installation et le (re)déploiement des plugiciels, constituent de loin la source de difficultés la plus fréquente pour les utilisateurs.
- Problèmes de documentation : l'absence ou la mauvaise qualité de la documentation, des tutoriels et des manuels est le deuxième problème le plus fréquent.<sup>453</sup> [156]

(voir énoncé p.215)

**Explication de l'assertion AS12 : La convention de code est nécessaire pour faciliter la compréhension du lectorat.**

Dans l'ouvrage *Open Verification Methodology Cookbook*, Mark Glasser consacre un chapitre à une convention. Il conclut son chapitre de cette façon :

[Traduction libre] Comme pour tout texte, appliquer un style cohérent à votre code permet à celui qui le lit de le comprendre rapidement et précisément. Cette personne pourrait bien être vous!<sup>454</sup> [102]

(voir énoncé p.215)

---

452.

*This survey indicates that the conjunct use of OCL and MOF is a difficult task and that experts are more or less likely to master OCL's logic for precise metamodeling.*

453.

- *Plugin issues : problems related to the tool's plugin architecture, such as plugin dependencies, plugin configuration, plugin installation, and plugin (re-)deployment, are, by far, the most common source of headache for users.*
- *Documentation issues : missing or low quality documentation, tutorials, and manuals are the second most commonly occurring problem.*

454.

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**Explication de la citation AS13 : Une convention de code est superflue, puisqu'un bon langage de programmation facilite la compréhension du lectorat.**

Un langage de programmation devrait suffire en lui-même. C. A. R. Hoare, dans son article *Hints on programming language design*, affirme qu'un bon langage de programmation devrait déjà agir comme une convention de code source :

[Traduction libre] Il devrait contribuer à établir et à faire respecter les conventions et les disciplines de programmation qui garantiront une coopération harmonieuse des différentes parties d'un vaste programme, qu'elles soient développées séparément ou finalement assemblées.<sup>455</sup> [119]

Puis, il ajoute que le langage de programmation doit d'abord servir pour la lecture, et non à l'écriture de code source :

[Traduction libre] Un bon langage de programmation encourage et aide le programmeur à écrire un code clair et autodocumenté, et peut-être même à développer et à afficher un style d'écriture agréable. La lisibilité des programmes est infiniment plus importante que leur facilité d'écriture.<sup>456</sup> [119]

(voir énoncé p.215)

---

*Just like any writing, when you apply a consistent style to your code, you improve the ability for someone reading it to understand it quickly and accurately. That person might be you !*

455.

*It should assist in establishing and enforcing the programming conventions and disciplines which will ensure harmonious cooperation of the parts of a large program when they are developed separately and finally assembled together.*

456.

*A good programming language will encourage and assist the programmer to write clear self-documenting code, and even perhaps to develop and display a pleasant style of writing. The readability of e programs is immeasurably more important than their writeability.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

#### **Explication l’assertion AS14 : L’appropriation d’actes réservés en informatique est davantage une lutte économique.**

Dans un article publié en 2007, Tracey L. Adams fait un sommaire de la situation entourant le titre d’ingénieur logiciel aux États-Unis, en Grande-Bretagne et au Canada [4]. Dès le départ, l’article affirme que les actes réservés sont une lutte entre professions<sup>457</sup> :

[Traduction libre] [...] de nombreux leaders professionnels continuent de rechercher un statut professionnel plus élevé dans l’espoir d’accroître leur sécurité d’emploi, leur autonomie, leur estime sociale et leurs revenus. Leurs efforts en ce sens sont cependant entravés, comme ceux de leurs prédécesseurs, par les actions des professions plus établies qui cherchent à maintenir leur statut actuel et par les projets d’autres professions émergentes visant à garantir leur place sur le marché du travail et dans la société en général.<sup>458</sup> [4]

Par la suite, il répond à la question de savoir pourquoi ces luttes sont plus fréquentes en informatique au Canada, en invoquant le manque de coopération, le manque d’organisation et les différences importantes dans la réglementation :

[Traduction libre]

[—] Premièrement, bien qu’il existe une longue tradition de coopération entre les organisations informatiques et d’ingénierie aux États-Unis et au Royaume-Uni, ce n’est pas le cas au Canada. Le CIPS n’entretient aucun lien, ni formel ni informel, avec les organisations d’ingénierie canadiennes. Si l’ACM et l’IEEE-CS aux États-Unis semblent partager certains de leurs

---

457. Lors de la rédaction de l’article, l’organisation d’informaticiens la plus importante au Canada est la Canadian Information Processing Society (CIPS).

458.

[...] *many occupational leaders continue to seek greater professional status in the hopes of increasing their job security, autonomy, social esteem, and income. Their efforts to do so, however, are hampered, like those of their predecessors, by the actions of more established professions seeking to maintain an existing status and other aspiring professions’ projects to secure their own Interprofessional Relations and the Emergence of a New Profession places in the labor market and society more broadly.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

membres, c'est moins vrai au Canada, où le CIPS est principalement une organisation de professionnels de l'informatique de gestion : par exemple, au début des années 1990, 49 % des membres de le CIPS étaient des gestionnaires et 25 % des programmeurs et des analystes. La présence d'universitaires et d'ingénieurs en informatique est assez faible au sein de le CIPS, ne représentant que 3 % des membres (CIPS 1990).

[—] Deuxièmement, les professionnels de l'informatique au Canada sont beaucoup moins organisés et moins unifiés que les ingénieurs. Le CIPS est une petite organisation représentant environ 6 000 travailleurs du secteur des TI, soit une infime fraction de l'ensemble des professionnels du domaine, tandis que les organisations d'ingénierie prétendent représenter tous les ingénieurs agréés (160 000). Aux États-Unis et au Royaume-Uni, la taille et l'influence des organisations d'informatique et de génie électrique sont plus comparables.

[—] Troisièmement, et c'est peut-être le plus important, il y a la différence de réglementation professionnelle entre l'informatique et l'ingénierie. Comme nous l'avons vu, les informaticiens et les ingénieurs au Royaume-Uni sont soumis à une réglementation et à un statut professionnel similaires. La situation est assez proche aux États-Unis, où la plupart des ingénieurs et tous les informaticiens ne sont ni réglementés ni agréés par l'État. Il est à noter que le principal point de friction interprofessionnel aux États-Unis concerne précisément l'agrément. Au Canada, l'écart entre les informaticiens et les ingénieurs est le plus marqué. Les ingénieurs canadiens sont des professionnels agréés, hautement qualifiés et autoréglementés. L'accès à la profession est encadré par la législation, la formation et l'agrément ; les ingénieurs bénéficient d'un fort sentiment d'isolement social. En revanche, les informaticiens au Canada sont peu réglementés et seulement la moitié d'entre eux environ sont titulaires d'un diplôme universitaire (Wolfson 2004). Le CIPS a créé une certification pour les travailleurs de l'informatique, mais cette certification n'est détenue que

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

par certains de ses membres et n'est reconnue par la législation que dans cinq provinces canadiennes. <sup>459</sup> [4]  
(voir énoncé p.216)

#### Améliorer la formation

**Explication de l'axiome AS15 : La formation sur la modélisation est nécessaire pour répondre aux changements rapides.**

Dans un article publié en 2019 sur les modèles en éducation, Mei-Hung Chiu et Jing-Wen Lin soulignent l'importance de la formation sur la modélisation à travers la première des cinq affirmations contenues dans leur article :

---

459.

[—] First, while there is a long history of cooperation between computing and engineering organizations in the United States and the United Kingdom, there is none in Canada. CIPS has neither formal nor informal ties with Canadian engineering organizations. While the ACM and IEEE-CS in the United States appear to have overlapping memberships, this is less true in Canada, where CIPS is primarily an organization of business-computing workers : for instance, in the early 1990s, 49 percent of CIPS members were managers and a further 25 percent were programmers and analysts. The presence of computing academics and engineers is quite small within CIPS, accounting for only 3 percent of members (CIPS 1990).

[—] Second, computing workers in Canada are much less organized and less unified than are engineers. CIPS is a small organization representing about 6,000 IT workers, only a small fraction of those in the field, while engineering organizations claim to speak for all (160,000) licensed engineers. In the United States and the United Kingdom, the size and strength of computing and electrical engineering organizations are more comparable.

[—] Third, and perhaps most important, is the difference in professional regulation between computing and engineering. As we have seen, computer professionals and engineers in the United Kingdom have a similar type of regulation and degree of professional status. The situation is not too different in the United States where most engineers and all computer professionals are not government-regulated or licensed. It is notable that the key area of interprofessional conflict to emerge in the United States is precisely in the area of licensing. In Canada, the gap between computer workers and engineers is the largest. Canadian engineers are licensed, highly educated, self-regulating professionals. Entry into the profession is limited through legislation, education, and licensing ; engineers have a great deal of social closure. In contrast, computing workers in Canada are largely unregulated and only about half of them hold a university degree (Wolfson 2004). CIPS has established a credential for computing workers, but this credential is possessed by only some of its members and is only recognized by legislation in five of Canada's provinces.

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

[Traduction libre] Affirmation 1 : Le développement des compétences en modélisation est essentiel à la culture scientifique des citoyens du XXI<sup>e</sup> siècle<sup>460</sup> [56]

Le recueil d'articles *Towards a Competence-Based View on Models and Modeling in Science Education* en vient à la même conclusion :

[Traduction libre] Malgré cette diversité, il semble y avoir un consensus sur l'idée selon laquelle, depuis les premières années des élèves jusqu'au niveau universitaire, si l'enseignement des sciences au XXI<sup>e</sup> siècle vise à fournir aux élèves des expériences scientifiques à jour, un rôle central devrait être joué par l'apprentissage des modèles scientifiques, l'apprentissage de la modélisation et l'apprentissage des modèles et de la modélisation.<sup>461</sup> [387]

Le recueil d'articles *The Routledge Handbook of Philosophy of Scientific Modeling* indique les difficultés des systèmes d'éducation à soutenir les compétences en modélisation qui sont pourtant cruciales :

[Traduction libre] Face à l'évolution rapide des pratiques dans les domaines des sciences, des technologies, de l'ingénierie et des mathématiques (STEM) contemporains, les récentes initiatives de politique éducative ont mis l'accent sur le développement de la pensée computationnelle et des compétences en modélisation interdisciplinaire au lycée et dans l'enseignement supérieur. Cependant, les systèmes éducatifs du monde entier peinent à mettre en œuvre ce changement systémique pourtant indispensable, faute de modèles opérationnels clairs illustrant des méthodes

---

460.

*Claim 1 : The development of modeling competence is essential to scientific literacy for the twenty-first century citizens*

461.

*Despite this variety, there seemed to be a consensus about the idea that, ranging from students' early years to the university level, if science education in the twenty-first century aims to provide students with up-to-date scientific experiences, a central role should be played by learning scientific models, learning to model, and learning about models and modeling.*



### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

efficaces pour passer des pratiques actuelles aux pratiques interdisciplinaires.<sup>462</sup> [164]

(voir énoncé p.216)

#### **Explication de l’assertion AS16 : Il reste du travail à faire sur l’enseignement de la modélisation.**

L’article de Mei-Hung Chiu et Jing-Wen Lin cite la revue de littérature de Namdar et Shen portant sur l’enseignement de la modélisation aux niveaux primaire et secondaire, incluant des données empiriques [56] :

[Traduction libre] Cependant, les discussions approfondies sur la modélisation des produits sont rares. Dans sa définition de la compétence en modélisation, seules quelques études ont inclus la modélisation de « produits » (et lorsqu’elles existent, elles sont implicites) ou même évalué la qualité de ces produits dans les pratiques de modélisation. Par exemple, dans une revue systématique de 104 études empiriques sur l’enseignement de la modélisation dans l’enseignement primaire et secondaire entre 1980 et 2013, Namdar et Shen (2015) n’ont pu identifier que 15 articles relatifs à l’évaluation des produits de modélisation [...] <sup>463</sup> [56]

---

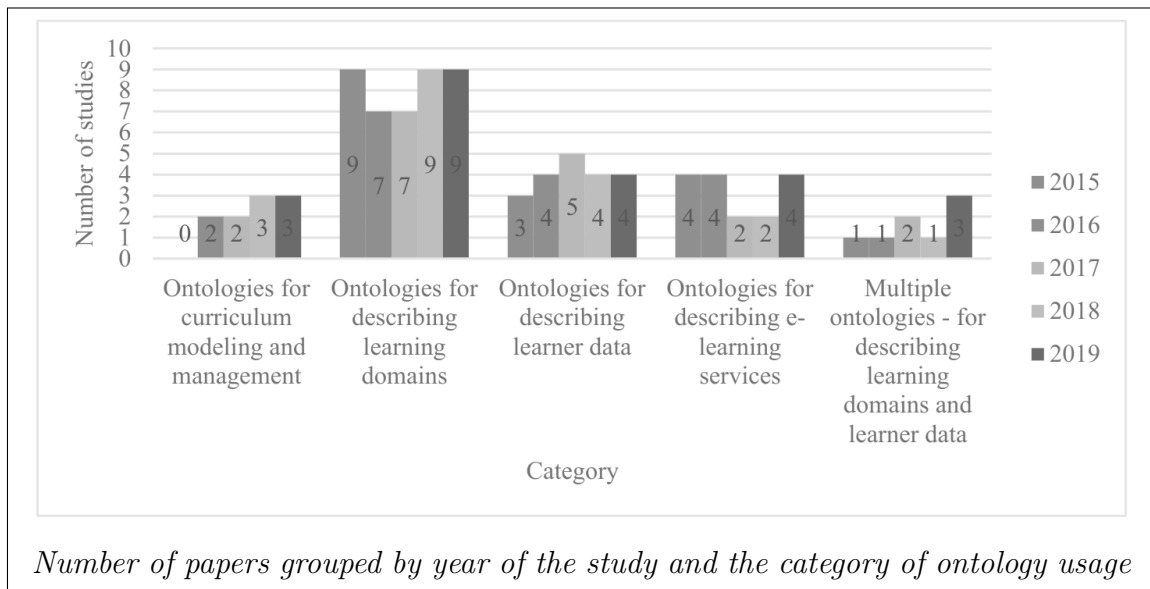
462.

*In response to the fast-changing practices in contemporary science, technology, engineering, and mathematics (STEM), recent educational policy initiatives have focused on developing computational thinking and interdisciplinary model-building skills at the high school and undergraduate levels. However, education systems around the world are struggling to implement this much-needed systemic change, as there are no clear operational models that illustrate effective ways to transition from existing practices to interdisciplinary ones.*

463.

*However, there is a lack of deep discussions on the topics of modeling products. As part of its definition of modeling competence, only a limited number of studies included modeling “products” (if any, there were implicit) or even assessment of quality of the products in the modeling practices. For instance, in a systemic review of 104 empirical studies on modeling instruction in K-12 between 1980 and 2013, Namdar and Shen (2015) were only able to identified 15 articles that are related to the assessment of modeling products [...]*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.44** – Types d’usage des ontologies entre 2015 et 2019 (source : [370])

En 2020, un article fait une revue de littérature sur l’utilisation des ontologies en éducation [370]. La revue comporte 95 articles publiés entre 2015 et 2019. Les ontologies sont utilisées selon différents objectifs illustrés dans la figure B.44 et la modélisation est le quatrième objectif.

(voir énoncé p.216)

**Explication de l’assertion AS17 : Cela fait de nombreuses années que des experts considèrent la formation en informatique appliquée insuffisante.**

David L. Parnas a écrit de nombreux articles sur le sujet, comme en témoignent les titres de ses articles :

- *Education for Computing Professionals* (1990) [289] ;
- *The Professional Responsibilities of Software Engineers* (1994) [292] ;
- *Why software developers should be licensed* (2001) [288] ;
- *Goals for software engineering student education* (2005) [290] ;
- *Software Engineering : A Profession in Waiting* (2021) [295].

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

Mary Shaw, dans son article *Software Engineering Education : A Roadmap* [361], formule des recommandations quant à l'insuffisance de formation. Ces recommandations, réparties à travers quatre sections, ont été énumérées dans le corps du mémoire (voir P78 p.67).

(voir énoncé p.216)

**Explication de l'assertion AS18 : Il y a plus de vingt ans, des connaissances ont été regroupées et ont occupé un rôle important dans l'informatique appliquée.**

Un article décrit le SWEBOK, un corpus de connaissances pour la profession d'informaticien appliqué :

[Traduction libre]

[—] Le projet SWEBOK vise à identifier et à définir le corpus de connaissances pour la pratique du génie logiciel ; il cherche à caractériser les limites de cette discipline et à la délimiter de l'informatique et d'autres disciplines, ainsi qu'à fournir un accès thématique au corpus de connaissances qui la sous-tend.

[—] [...] trois documents clés : une version préliminaire (1998), une version définitive (2001) et, plus récemment, une version finale (2004).

[—] L'élaboration du SWEBOK a été menée par une équipe de l'Université du Québec à Montréal, mais a impliqué la consultation de milliers de professionnels du logiciel dans 42 pays [...]. <sup>464</sup> [4]

Un article de Richard E. Fairley décrit différents rôles joués par le SWEBOK :

[Traduction libre]

---

464.

[—] *The SWEBOK project aims to identify and outline the body of knowledge for SWE practice ; it seeks to characterize “the bounds of the software engineering discipline” and demarcate it from computer science and other disciplines, as well as to “provide a topical access to the Body of Knowledge supporting that discipline” [...].*

[—] *[...] three key documents : a “straw man” version (1998), a “stone man” version (2001), and most recently a final version (2004).*

[—] *The development of the SWEBOK was led by a team at the University of Quebec at Montreal, but involved consultation with thousands of software practitioners in 42 countries [...].*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

[—] SWEBOK Version 3 est un document de base pour les programmes de certification Certified Software Development Associate et Certified Software Development Professional parrainés par l'IEEE Computer Society.

[—] Le comité d'accréditation en informatique d'ABET (CSAB) définit les critères d'accréditation des programmes accrédités par ABET en informatique, génie logiciel, technologies de l'information et sciences de l'information. Le guide SWEBOK constitue un document de référence pour l'établissement et la mise à jour des critères d'accréditation en génie logiciel.

[—] Le Conseil des activités professionnelles de l'IEEE Computer Society développe actuellement un modèle de compétences en génie logiciel (SECOM) basé principalement sur la version 3 du SWEBOK. <sup>465</sup> [85]

Le SWEBOK a aussi servi à l'élaboration des *Curricula Recommendations* de l'ACM que voici <sup>466</sup> :

- *Computing Curricula* (CC2020, CC2005) ;
- *Computer Engineering* (CE2004, CE 2016) ;
- *Computer Science* (CC2001, CC2008, CC2013, CC2023) ;
- *Cybersecurity* (CSEC2017) ;
- *Data Science* (CCDS2021) ;
- *Information Systems* (MSIS2006, MSIS2016, IS2002, IS2010, IS2020) ;

---

465.

[—] *SWEBOK Version 3 is a foundation document for the Certified Software Development Associate and Certified Software Development Professional certification programs sponsored by the IEEE Computer Society.*

[—] *The Computer Science Accreditation Board of ABET (CSAB) specifies the accreditation criteria for ABET accredited programs in computer science, software engineering, information technology, and information science. The SWEBOK Guide is a foundation document for establishing and updating the software engineering accreditation criteria.*

[—] *The Professional Activities Board of the IEEE Computer Society is currently developing a Software Engineering Competency Model (SECOM) that is based primarily on SWEBOK Version 3.*

466. ACM, site web, « Curricula Recommendations », <https://www.acm.org/education/curricula-recommendations> (consulté le 2025-04-12).

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

- *Information Technology* (IT2008, IT2017) ;
- *Software Engineering* (SE2004, GSwE2009, SE2014).

(voir énoncé p.216)

#### **Explication de l’assertion AS19 : Les changements trop nombreux en informatique appliquée mettent à mal l’accumulation de connaissances.**

En 2021, David L. Parnas, à l’intérieur d’un article, affirme l’inutilité de développer un corpus de connaissances et décrit le manque de législation pour encadrer la profession :

[Traduction libre] Un corpus de connaissances est utile pour caractériser une science, car une science est un ensemble organisé de connaissances. L’ingénierie est différente.<sup>467</sup> [295]

Il cite Theodore von Kármán, qui donne les raisons de cet échec :

[Traduction libre]

- L’ensemble des concepts et des faits connus des développeurs de logiciels est immense ; il existe peu de consensus sur l’importance et l’utilité, même des concepts les plus répandus. L’ampleur du corpus de connaissances a incité certains développeurs à tenter de hiérarchiser ces connaissances et d’identifier un sous-ensemble indispensable, mais cette tâche s’avère très complexe. Ce qui paraît important pour certains peut sembler inutile pour d’autres.
- Le corpus de connaissances du logiciel est en constante expansion ; de plus, une grande partie de son contenu devient obsolète quelques années seulement après son ajout.
- Une grande partie de ces connaissances est liée à des outils spécifiques. Les caractéristiques de ces outils résultent de nombreux choix de conception arbitraires, et ces outils peuvent évoluer ou devenir obsolètes ; par

---

467.

*A BoK is useful for characterizing a science because a science is an organized body of knowledge. Engineering is different.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

conséquent, certaines connaissances les concernant perdront rapidement de leur valeur. <sup>468</sup> [295]  
(voir énoncé p.216)

**Explication de l’assertion AS20 : Les compétences importantes pour la formation touchent entre autres aux documents et aux modèles depuis au moins cinquante ans.**

En 1976, Barry W. Boehm publie l’article *Software engineering education : some industry needs* dans lequel il nomme les champs suivants dans lesquels les nouveaux diplômés doivent être mieux formés :

[Traduction libre]

- documentation technique ; - analyse des exigences logicielles et conception ; - développement de logiciels applicatifs ; - aspect économiques du traitement des données ; - techniques de gestions ; - performance des équipes ;
- considérations et méthodes de maintenance logicielles. <sup>469</sup> [397]

Il conclut en mentionnant ce qu’il considère être le besoin le plus important en regard à la formation :

[Traduction libre] Notre besoin le plus important, cependant, est que les universités continuent d’être une source d’esprits curieux et d’idées nou-

---

468.

- *The set of concepts and facts known to software developers is huge ; there is little agreement on the importance and usefulness of even the most popular concepts. The size of the BoK has caused some developers to try to prioritize the knowledge and identify a required subset, but that is very difficult. What seems important to some seems useless to others.*
- *The software BoK is always rapidly growing ; further, much of it becomes irrelevant just a few years after it has been added.*
- *Much of that knowledge is tied to specific tools. The characteristics of those tools are the result of many arbitrary design decisions, and the tools may evolve or become out of date ; consequently, some of the knowledge about those tools will not be of lasting value.*

469.

- *Technical documentation ; - Software requirements analysis and design ; Applications software development ; - Data processing economics ; - Management techniques ; - Group performance ; - Software maintenance considerations and techniques.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

**TABLEAU B.18** – Lacunes fréquentes chez des professionnels (source : *Table 2* [313])

| <i>Most Identified Knowledge Deficiencies</i> |            |
|-----------------------------------------------|------------|
| Knowledge Deficiency                          | Occurances |
| Written Communication                         | 10         |
| Oral Communication                            | 10         |
| Project Management                            | 9          |
| Software Tools                                | 9          |
| Testing                                       | 9          |
| Teamwork                                      | 7          |
| Problem Solving & Critical Thinking           | 7          |
| Programming                                   | 7          |
| Requirements                                  | 7          |
| Personal Skills                               | 6          |
| Ethics                                        | 5          |
| Leadership                                    | 5          |
| Design                                        | 5          |
| Databases                                     | 5          |
| Development & Improvement Processes           | 5          |

velles, ce qui contribue grandement à éviter que les pratiques industrielles ne s'enlisent dans la routine.<sup>470</sup> [397]

La conception, les exigences, les procédés, les modèles sont des lacunes importantes pour les informaticiens appliqués et cela se manifeste fréquemment dans la documentation. En 2013, une revue de littérature ( $n = 30$ ) est menée sur les différences qui subsistent entre la formation en informatique et les besoins de l'industrie [313]. La revue s'étale de 1995 à 2012. Parmi les participants, il y a d'anciens étudiants et des employeurs. L'article énumère les dix compétences présentant les lacunes les plus marquées (tableau B.18). Parmi celles-ci, la communication tant écrite qu'orale revient fréquemment. Or, la communication écrite correspond au domaine de la documentation.

L'article de 2019 intitulé *Aligning software engineering education with industrial needs : A meta-analysis* [96] met en évidence les lacunes compilées à la figure B.45. Celle-ci

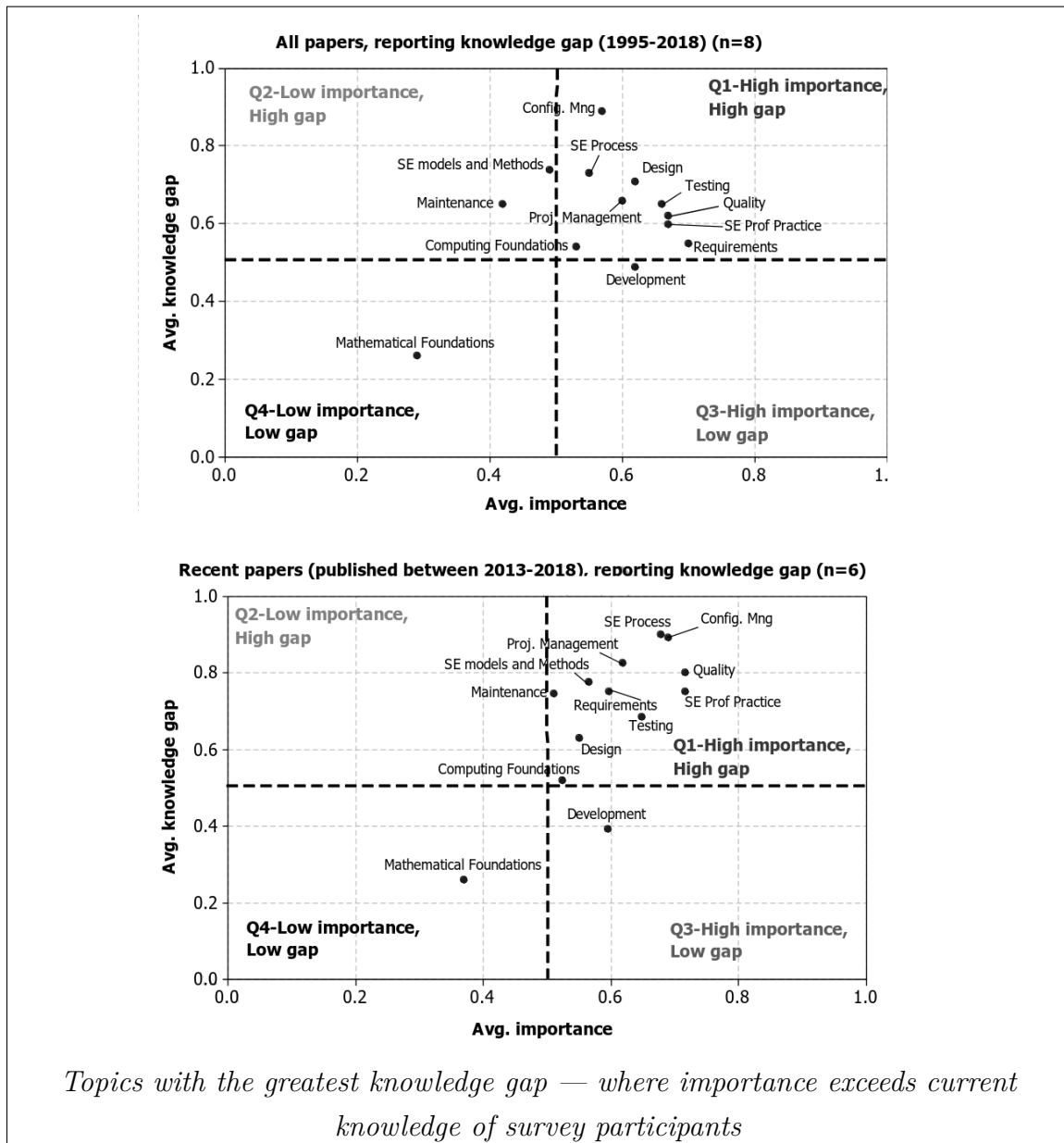
---

470.

*By far our most important need, however, is for Universities to continue as a source of the inquiring minds and fresh ideas which help so much in keeping industrial practice from getting into a rut.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

regroupe des données provenant à la fois d'anciens étudiants et des employeurs<sup>471</sup>.  
(voir énoncé p.216)



**FIGURE B.45** – Lacunes importantes chez des professionnels (source : Figure 9 [96])

471. Il y a : opinion surveys, job advertisements, recent college graduates, feedback from an employee's manager, peers and direct reports, etc..



### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

#### **Explication de l'assertion AS21 : Un critère de réussite dans l'usage des méthodes formelles est l'expertise.**

L'article *Ten Commandments Of Formal Methods* affirme qu'il est autant possible de donner des formations approfondies que de faire appel à des consultants externes pour bien appliquer les méthodes formelles :

[Traduction libre] La plupart des projets de méthodes formelles couronnés de succès ont fait appel à au moins un consultant expert en techniques formelles. Cette expertise externe apparaît quasi indispensable, comme en témoignent les projets IBM CICS et Inmos T800 Transputer. Chez IBM, les experts en méthodes formelles ont dispensé une formation approfondie aux employés afin de les rendre autonomes dans ce domaine. Inmos a adopté une approche différente : consultants et ingénieurs de projet ont travaillé de concert, tout en communiquant constamment pour garantir le succès du projet. Inmos a également recruté des personnes possédant l'expérience mathématique requise pour concevoir et vérifier formellement les conceptions critiques, et a consulté des experts externes. Les deux approches ont fait leurs preuves ; une entreprise doit choisir celle qui correspond à son style d'organisation et à ses objectifs à long terme.<sup>472</sup> [35]

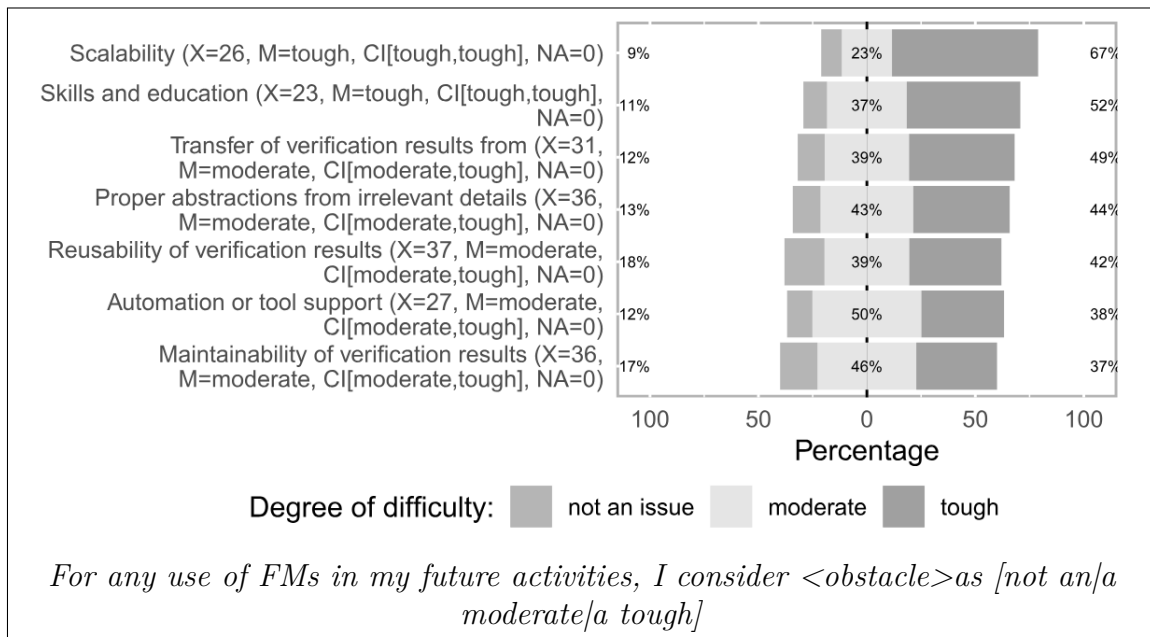
En 1998, l'étude de quatre expérimentations provenant de la NASA [80] a notamment permis de conclure qu'il est plus facile d'intégrer une personne avec l'expertise en méthode formelle que de former une personne appartenant déjà à une équipe de développement :

---

472.

*Most successful formal methods projects have relied on at least one consultant with formal techniques expertise. Such outside expertise appears almost indispensable, as evidenced by the IBM CICS project and the Inmos T800 Transputer project. In IBM's case, formal methods experts extensively trained employees to become self-sufficient in formal techniques. A different approach was adopted at Inmos, where consultants and project engineers worked in tandem but communicated constantly to ensure success. Inmos also hired people with the relevant mathematical experience to formally produce and check critical designs and consulted with outside experts. Both approaches have proved successful-a company must choose an approach that reflects its organizational style as well as its long-term goals.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE



**FIGURE B.46** – Obstacles identifiés par des professionnels à l’usage des méthodes formelles (source : *Figure 16* [103])

[Traduction libre] Pour chacun des cas, des méthodes formelles ont été utilisées pour une partie de la modélisation des exigences. Peu importe les instruments ou les techniques utilisés, il est plus facile d’inclure des experts en méthodes formelles dans une équipe que d’entraîner une équipe aux méthodes formelles.<sup>473</sup> [80]

Un sondage réalisé en 2020 sur l’utilisation des méthodes formelles auprès de 216 participants [103] demande d’identifier les principaux obstacles à leur utilisation (figure B.46). Les compétences et la formation arrivent en deuxième position, juste après la mise à l’échelle.

(voir énoncé p.216)

473.

*An important characteristic of these studies is that in each case the formal modeling was carried out by a small team of experts who were not part of the development team. Results from the formal modeling were fed back into the requirements analysis phase, but formal specification languages were not adopted for baseline specifications.*

### B.3. DES CONNAISSANCES EN INFORMATIQUE APPLIQUÉE

#### **Explication de l’assertion AS22 : L’enseignement des méthodes formelles est sur le déclin.**

En 2020, un article dont les auteurs sont Maurice H. Ter Beek *et al.* décrit la situation actuelle avec les méthodes formelles. Même si elles occupaient une place importante, ce n’est vraisemblablement plus le cas (voir P80 p.68 la citation complète) [378].

En 2024, les auteurs Maurice H. Ter Beek *et al.* écrivent un autre article sur les méthodes formelles et y encouragent leur enseignement en informatique [377] :

[Traduction libre] Les méthodes formelles ne figurent pas dans le programme CS2023, le cursus d’informatique ACM/IEEE-CS/AAAI [197], à la mesure où elles sont essentielles en informatique et aux avantages que leur enseignement peut apporter à l’industrie. Le programme CS2023 couvre 17 domaines de connaissances. Dans [34, 70, 118], il est avancé que huit d’entre eux sont liés aux méthodes formelles. Nous listons ici ces domaines et proposons des pistes de réflexion quant aux contenus à enseigner en lien avec les méthodes formelles : fondements algorithmiques, architecture et organisation, intelligence artificielle, calcul parallèle et distribué, sécurité, principes fondamentaux du développement logiciel, bases de données, génie logiciel.<sup>474</sup> [377]

(voir énoncé p.216)

---

474.

*Formal methods do not appear in CS2023, the ACM/IEEE-CS/AAAI Computer Science Curricula [197], to the extent that relects their pivotal role in Computer Science and the beneits that formal methods education can bring to industry. CS2023 encompasses 17 knowledge areas. In [34, 70, 118], it is argueded that eight of them are related to formal methods. Here we list these areas and provide suggestions for what to teach in relation with formal methods : Algorithmic Foundations, Architecture and Organization, Artificial Intelligence, Parallel and Distributed Computing, Security, Software Development Fundamentals, Databases, Software Engineering.*

# Annexe C

L'annexe C contient les descriptions formelles des trois langages utilisés lors de ce mémoire : le diagramme entité-association, le diagramme de flux de données et le diagramme provenant du modèle et notation des procédés métiers (BPMN, *Business Process Model and Notation* en anglais).

Le diagramme d'entité-association comprend :

[Traduction et description libres]

[entité : ] [...] est une chose ou un objet du monde réel doté d'une existence indépendante.

[type d'association : ] [...] définit un ensemble d'associations — ou un ensemble de relations — entre les entités de ces types d'entités.

[rapport de cardinalité : ] Le ratio de cardinalité d'une relation binaire spécifie le nombre maximal d'instances de relation auxquelles une entité peut participer.

[contrainte de participation : ] La contrainte de participation spécifie si l'existence d'une entité dépend de sa relation avec une autre entité via le type de relation.

[l'expression rectangle représente une entité ;]

[l'expression losange représente une association ;]

[l'expression (minimum, maximum) représente les contraintes structurelles où le minimum est la participation minimale et le maximum est la contrainte de cardinalité maximale.] <sup>475</sup> [82]

La définition du modèle conceptuel de données en utilisant le langage de diagramme EA nécessite des éléments supplémentaires qui sont décrits ci-dessous :

[Traduction et description libres]

[— définition d'attribut] : Chaque entité possède des attributs — les propriétés particulières qui la décrivent.

[— description d'un attribut clé] : Un type d'entité possède généralement un ou plusieurs attributs dont les valeurs sont distinctes pour chaque entité de l'ensemble. Un tel attribut est appelé attribut clé, et ses valeurs permettent d'identifier chaque entité de manière unique.

[— définition d'un type d'entité faible] : Les types d'entités qui ne possèdent pas d'attributs clés propres [...].

[— description d'un type d'entité faible] : Une entité faible présente toujours une contrainte de participation totale (dépendance d'existence) vis-à-vis de sa relation d'identification, car elle ne peut être identifiée sans une entité propriétaire. Cependant, toute dépendance d'existence n'entraîne pas nécessairement la création d'une entité faible.

[— description d'ISA] : L'héritage de comportement est également connu sous le nom d'héritage d'interface ou d'héritage ISA [...].

[— l'expression ovale représente un attribut.]

---

475.

*[entity :] [...] is a thing or object in the real world with an independent existence.*

*[relation type :] [...] defines a set of associations — or a relationship set — among entities from these entity types.*

*[cardinality ratio :] The cardinality ratio for a binary relationship specifies the maximum number of relationship instances that an entity can participate in.*

*[participation constraint :] The participation constraint specifies whether the existence of an entity depends on its being related to another entity via the relationship type.*

- [— l'expression ovale avec le mot souligné représente un attribut clé.]
- [— l'expression rectangle avec deux contours représente une entité faible.]
- [— l'expression losange avec deux contours représente une identification d'association.]
- [— l'expression triangle avec **Is a** représente une relation entre un type et un sous-type. L'utilisation de (d) signifie disjoint et (o) chevauchement (*overlapping*, en anglais). Une ligne double signifie que la spécialisation est totale et une ligne simple qu'elle est partielle.] <sup>476</sup> [82]

Voici la signification du langage du diagramme de contexte et du diagramme de flux de données (DFD) :

- [Traduction et description libre]
- [— définition de processus] : [...] une partie du système qui transforme les intrants en extrants [...].
- [— définition de flux] : [...] décrit le déplacement de blocs, ou paquets d'informations, d'une partie du système à une autre.
- [— définition d'un dépôt] : [...] est utilisé pour modéliser une collection de paquets de données au repos.
- [— définition d'un agent] : [...] représentent des entités externes avec lesquelles le système communique.

---

476.

- [— *attribute definition*] : *Each entity has attributes — the particular properties that describe it.*
- [— *key attribute description*] : *An entity type usually has one or more attributes whose values are distinct for each individual entity in the entity set. Such an attribute is called a key attribute, and its values can be used to identify each entity uniquely.*
- [— *weak entity type definition*] : *Entity types that do not have key attributes of their own [...].*
- [— *weak entity type description*] : *A weak entity type always has a total participation constraint (existence dependency) with respect to its identifying relationship because a weak entity cannot be identified without an owner entity. However, not every existence dependency results in a weak entity type.*
- [— *ISA description*] : *Behavior inheritance is also known as ISA or interface inheritance [...].*

- [— l’expression de cercle représente un processus ;]
- [— l’expression de flèche représente un flux ;]
- [— l’expression de ligne au-dessus et ligne en dessous représente un dépôt ;]
- [— l’expression de rectangle représente un agent et, lorsque la nature de l’agent est contrainte à un processus ou à un dépôt, leurs symboles respectifs (cercle, double ligne) peuvent être substitués au rectangle ;]
- [— l’expression de grande boîte centrale représente la machine qui exécute le système ou le processus.] <sup>477</sup> [408]

Le *Business Process Model and Notation* (BPMN) est utilisé pour décrire des processus, la signification du langage est donnée avec les éléments suivants :

- [Traduction et description libres]
- [— définition d’un événement] : Un événement est quelque chose qui se produit au cours d’un processus [...].
- [— définition d’activité] : Une activité est un terme générique désignant le travail effectué par une entreprise (voir page 149) dans un processus.
- [— définition d’une passerelle] : Une passerelle est utilisée pour contrôler la divergence et la convergence des flux de séquence dans un processus [...].
- [— définition d’un flux séquentiel] : Un flux séquentiel est utilisé pour indiquer l’ordre dans lequel les activités seront exécutées dans un processus [...].
- [— définition d’un *pool*] : Un *pool* est la représentation graphique d’un Participant dans une Collaboration (voir page 113). Il sert également de

---

477.

- [—] *process* : [...] *a part of the system that transforms inputs into outputs* [...].
- [—] *flow* : [...] *describe the movement of chunks, or packets of information from one part of the system to another part.*
- [—] *store* : [...] *is used to model a collection of data packets at rest.*
- [—] *terminator* : [...] *represent external entities with which the system communicates.*

« couloir » et de conteneur graphique pour partitionner un ensemble d'Activités provenant d'autres *pools* [...].

[— l'expression de cercle avec un contour mince représente un événement de départ ;]

[— l'expression de cercle avec un contour épais représente un événement d'arrivée ;]

[— l'expression de flèche représente un flux séquentiel ;]

[— l'expression de rectangle représente une activité ; ]

[— l'expression de losange ayant un symbole X à l'intérieur représente une porte de type exclusif ;]

[— l'expression de losange ayant un symbole + à l'intérieur représente une porte de type parallèle ;]

[— l'expression de boîte représente le *pool*.] <sup>478</sup> [235]

---

478.

[— *event definition*] : An Event is something that “happens” during the course of a Process [...].

[— *activity definition*] : An Activity is a generic term for work that company performs (see page 149) in a Process.

[— *gateway definition*] : A Gateway is used to control the divergence and convergence of Sequence Flows in a Process [...].

[— *sequence flow definition*] : A Sequence Flow is used to show the order that Activities will be performed in a Process [...].

[— *pool definition*] : A Pool is the graphical representation of a Participant in a Collaboration (see page 113). It also acts as a “swimlane” and a graphical container for partitioning a set of Activities from other Pools [...].



# Annexe D

L'annexe D contient des exemples d'intrants et d'extrants pour l'instrument d'aide à la rédaction (IAR). Chaque sous-section correspond à un type d'intrant, d'extrant ou d'un agent se retrouvant dans la diagramme de contexte de l'IAR (figure 3.1) : la configuration, l'utilisateur, la description proposée d'un modèle, la représentation standardisée d'un modèle et le journal.

## D.1 Configuration

Les exemples de la configuration sont :

- une grammaire rationnelle pour épurer la description proposée d'un modèle (code source D.1) ;
- une grammaire algébrique avec règles sémantiques pour épurer la description proposée d'un modèle (code source D.2) ;
- une grammaire rationnelle pour traduire la description proposée épurée d'un modèle (code source D.3) ;
- une grammaire algébrique avec règles sémantiques pour traduire la description proposée épurée d'un modèle (code source D.4) ;
- des règles pour vérifier partiellement le modèle intermédiaire (code source D.5) ;
- des règles pour présenter le modèle vérifié aux modélisateurs (code source D.6) ;
- des règles pour présenter le modèle vérifié aux approbateurs (code source D.7).

## D.1. CONFIGURATION

Voici une grammaire rationnelle (fichier `AsciidocMacroLexer.g4`) se retrouvant dans le code source D.1 pour épurer la description proposée d'un modèle :

---

```
lexer grammar AsciidocMacroLexer;
OC: 'ifdef::';
CC: 'endif::';
OPENBRACKET: '[';
CLOSEBRACKET: ']';
OS: '{';
CS: '}';
OI: 'include::';
// C'est comme une string avec un \ en plus.
URI: ( SLASH? ( ALPHANUMERIC | SYMBOL) )+ ( DOT ALPHANUMERIC+ );
CK: COLON SPACE;
OK: COLON;
ONEG: COLON EXCLAMATION;
CNEG: EXCLAMATION COLON;
COMMENT: '//' ~'/' ~[\r\n]*? EOL+ -> skip;
//https://stackoverflow.com/questions/51576395/antlr4-grammar-
    extraneous-input-with-single-space
WS: ( SPACE | TABULATION )+;
EOL: '\r'? '\n';
WORD: ( ALPHANUMERIC | SYMBOL )+;
SYMBOL:
    (
        '_'
        | '_'
        | '.'
        | '='
        | 'à'
        | 'é'
        | 'ê'
        | 'è'
        | '\'
        | 'â'
```

## D.1. CONFIGURATION

```
    | ','  
    | '\\'  
    | ' :='  
    | ';' '  
    | ')' '  
    | '(' '  
    | '+' '  
    | '\"'  
    | '%'  
)  
;  
fragment ALPHANUMERIC: [a-zA-Z~0-9];  
fragment SPACE: ' '  
fragment TABULATION: '\\t';  
fragment SLASH: '/';  
fragment DOT: '.';  
fragment COLON: ':';  
fragment EXCLAMATION: '!';  
fragment LOWERCASE: [a-z];
```

---

### Code source D.1 – AsciiDocMacroLexer.g4

Voici une grammaire algébrique avec règles sémantiques (fichier `AsciiDocMacroParser.g4`) se retrouvant dans le code source D.2 pour épurer la description proposée d'un modèle :

---

```
parser grammar AsciiDocMacroParser;  
@header {  
import java.util.Arrays;  
import ca.mbjolin.editor.preprocess._2_macro.grammar.MacroSymbol;  
import ca.mbjolin.editor.preprocess._2_macro.grammar.  
    MacroSymbolTable;  
import ca.mbjolin.editor.preprocess._2_macro.grammar.  
    MacroSymbolType;
```

## D.1. CONFIGURATION

```
import ca.mbjolin.editor.preprocess._2_macro.grammar.
    ConditionalSymbol;
import ca.mbjolin.editor.preprocess._2_macro.grammar.
    ContentSymbol;
import ca.mbjolin.editor.preprocess._2_macro.grammar.
    IncludeSymbol;
import ca.mbjolin.editor.preprocess._2_macro.grammar.
    SubstitutionSymbol;
import ca.mbjolin.editor.preprocess._2_macro.grammar.ScopeSymbol;
import ca.mbjolin.editor.preprocess._2_macro.grammar.
    VariableSymbol;
}
@parser::members {
private List<MacroSymbolType> resolutionOrder = Arrays.asList(
    MacroSymbolType.VARIABLE,
    MacroSymbolType.INCLUDE,
    MacroSymbolType.SUBSTITUTION,
    MacroSymbolType.CONDITIONAL
);
public MacroSymbolTable macroSymbolTable = new MacroSymbolTable(
    resolutionOrder);
}
options {
    tokenVocab = AsciiDocMacroLexer;
}
//https://stackoverflow.com/questions/25914154/antlr4-parse-a-
    file-line-by-line
//https://stackoverflow.com/questions/27951798/macro-expansion-in
    -antlr4
doc: content? ( macro content)* EOF;
content:
    (WORD | SYMBOL | WS | EOL | OPENBRACKET | CLOSEBRACKET)+ {
ContentSymbol content = new ContentSymbol(_localctx);
macroSymbolTable.add(content);
```

## D.1. CONFIGURATION

```
    }  
;  
/* Je dois considérer l'intérieur d'une condition comme un  
   contenu,  
   * donc il faut ignorer les symboles de macros. Il doit y avoir  
   une solution  
   * plus élégante.  
   */  
contentsymbolmacro:  
    (  
        WORD  
        | SYMBOL  
        | WS  
        | EOL  
        | OS  
        | CS  
        | OPENBRACKET  
        | CLOSEBRACKET  
        | DOT  
        | OK  
        | CK  
    )+  
;  
contentmacro: ( WORD | SYMBOL | WS | EOL | macro)+;  
macro:  
    {  
        ScopeSymbol scopebegin = new ScopeSymbol(_localctx, true);  
        macroSymbolTable.add(scopebegin);  
    } (substitution | include | variable | conditional) {  
        ScopeSymbol scopeend = new ScopeSymbol(_localctx, false);  
        macroSymbolTable.add(scopeend);  
    }  
;  
substitution:
```

## D.1. CONFIGURATION

```
    OS wordmacro CS {
SubstitutionSymbol substitution = new SubstitutionSymbol(
    _localctx, _localctx.wordmacro().getText());
macroSymbolTable.add(substitution);
    }
;
wordmacro: ( macro | WORD)+;
include:
    OI (macro | URI) OPENBRACKET CLOSEBRACKET {
IncludeSymbol include = new IncludeSymbol(_localctx, _localctx.
    URI().getText());
macroSymbolTable.add(include);
    }
;
//TRIM la valeur pour avoir un valeur de longueur 0.
variable:
    (
        OK key CNEG WS* {
VariableSymbol variable = new VariableSymbol(_localctx,
    _localctx.key().getText(), null);
macroSymbolTable.add(variable);
        }
        | ONEG key OK WS* {
VariableSymbol variable = new VariableSymbol(_localctx,
    _localctx.key().getText(), null);
macroSymbolTable.add(variable);
        }
        | OK key CK value {
VariableSymbol variable = new VariableSymbol(_localctx,
    _localctx.key().getText(), _localctx.value().getText());
macroSymbolTable.add(variable);
        }
    )
;
```

## D.1. CONFIGURATION

```
key: WORD;
value: ( WORD | SYMBOL | WS | macro)+;
conditional:
    OC wordmacro OPENBRACKET CLOSEBRACKET contentsymbolmacro CC
    wordmacro? OPENBRACKET CLOSEBRACKET
    {
ConditionalSymbol condition = new ConditionalSymbol(_localctx,
    _localctx.wordmacro().get(0).getText(),
    _localctx.contentsymbolmacro().getText());
macroSymbolTable.add(condition);
    }
;
```

---

### Code source D.2 – AsciiDocMacroParser.g4

Voici une grammaire rationnelle (fichier `AsciiDocLexer.g4`) se retrouvant dans le **code source D.3** pour traduire la description proposée épurée d'un modèle :

---

```
/*
 * Inspiré de : https://github.com/venkatperi/asciidoc-antlr/blob/master/src/main/antlr/AsciiDocLexer.g4
 */

lexer grammar AsciiDocLexer;
@header {
import ca.mbjolin.editor.process._1_translation.grammar.asciidoc.
    AbstractAsciiDocLexer;
}
options {
    superClass = AbstractAsciiDocLexer;
}
tokens {
    SECTION_END,
    BLOCK_TABLE_START,
    BLOCK_ANON_START,
    BLOCK_COMMENT_START,
```

## D.1. CONFIGURATION

```
BLOCK_FENCED_START,  
BLOCK_LISTING_START,  
BLOCK_LITERAL_START,  
BLOCK_PASS_START,  
BLOCK_SIDEBAR_START,  
BLOCK_VERSE_START,  
BLOCK_EXAMPLE_START  
}  
/////////////////  
// default mode  
H0: {this.isBOL}? EQUAL WS ~[\r\n]+ EOLF -> pushMode(HEADER);  
ATTR_START: { this.isBOL }? COLON -> pushMode(ATTR);  
PPD_START:  
    { this.isBOL }? ('ifdef' | 'ifndef' | 'ifeval') '::' WS_CHAR*  
    -> pushMode(PPMODE)  
;  
COMMENT: COMMENT_F -> channel(HIDDEN);  
EOL: EOLF -> channel(HIDDEN);  
WS: WS_CHAR+ -> channel(HIDDEN);  
fragment COMMENT_F: '//' ~'/' ~[\r\n]*? EOLF+;  
fragment OC: '[';  
fragment CC: ']';  
EOLF: '\r'? '\n';  
fragment LETTER: [\p{L}];  
fragment GT: '>';  
fragment LT: '<';  
fragment SEMI: ';';  
fragment ESC: '\\';  
fragment SINGLE_QUOTE: '\'';  
fragment DOUBLE_QUOTE: '\"';  
fragment COMMA: ',';  
fragment EQUAL: '=';  
fragment HASH: '#';  
fragment PERCENT: '%';
```



## D.1. CONFIGURATION

```
fragment PERIOD: '.';
fragment COLON: ':';
fragment BANG: '!';
fragment WS_CHAR: [ \t];
fragment SPACE: ' ';
fragment DIGIT: [\p{N}];
fragment PUNCT: [\p{P}];
fragment ATTR_ID_F: [_a-zA-Z0-9] [-_a-zA-Z0-9]*;
////////////////////
mode HEADER;
HEADER_ATTR_START:
    { this.isBOL }? COLON -> pushMode(ATTR), type(ATTR_START)
;
HEADER_PPD_START:
    { this.isBOL }? ('ifdef' | 'ifndef' | 'ifeval') '::' WS_CHAR*
    -> pushMode(PPMODE), type(
        PPD_START)
;
AUTHOR:
    AUTHOR_NAME AUTHOR_NAME? AUTHOR_NAME AUTHOR_CONTACT (
        SEMI WS_CHAR* AUTHOR
    ) * EOLF
;
// any non-whitespace char
fragment AUTHOR_NAME: ~[<>|\r\n]+ WS_CHAR+;
fragment AUTHOR_CONTACT: LT .*? GT WS_CHAR*;
END_OF_HEADER: { this.isBOL }? WS_CHAR* EOLF+ -> mode(BLOCK);
REVISION:
    { this.isBOL }? (
        (( ([vV] REV_NUMBER) | (REV_NUMBER COMMA)) REV_REMARK?)
        | ( [vV]? REV_NUMBER COMMA REV_DATE REV_REMARK?)
        | ( ([vV] REV_NUMBER) | (REV_NUMBER COMMA))
    ) EOLF
;
```

## D.1. CONFIGURATION

```
fragment REV_NUMPREFIX: [vV];
fragment REV_DATE: WS_CHAR* ~[:\r\n]+ WS_CHAR*;
fragment REV_REMARK: COLON WS_CHAR* ~[\r\n]+;
fragment REV_NUMBER: DIGIT+ (PERIOD DIGIT+)*;
////////////////////////////////
mode ATTR;
ATTR_ID: ATTR_ID_F;
ATTR_UNSET: BANG;
ATTR_SEP: COLON WS_CHAR* -> mode(ATTR_VALUE_MODE);
////////////////////////////////
mode ATTR_VALUE_MODE;
ATTR_VALUE: (~[\\r\n[] | (ESC EOLF))+;
ATTR_UID: OC ~[\r\n\]]+ CC;
ATTR_EOL: WS_CHAR* EOLF -> popMode;
////////////////////////////////
mode BLOCK;
CUSTOM_END_SECTION: '~~~~' ~[\r\n]*? EOLF+;
BLOCK_ANCHOR: {this.isBOL}? '[' .*? ']' EOLF+;
BLOCK_ATTR_START:
    {this.isBOL}? '[' WS_CHAR* -> pushMode(BLOCK_ATTR)
;
BLOCK_TITLE_START:
    {this.isBOL}? '.' -> pushMode(BLOCK_TITLE_MODE)
;
//BLOCK_SECTION_TITLE
// : SECTION_TITLE_START ~[\r\n]+ EOLF+
// ;
BLOCK_DELIM_START:
    {this.isBOL}? ~[\r\n]+ EOLF {this.isDelimBlockStart()}? {this
        .startDelimBlock();}
;
BLOCK_PARA:
    {this.isBOL}? (
        ( [/|+*\`\-_ =] (LETTER | DIGIT | WS_CHAR))
```

## D.1. CONFIGURATION

```
| ~[/|+*`\-_=[.]  
) .*? BLOCK_EOP  
;  
BLOCK_COMMENT: COMMENT_F -> channel(HIDDEN);  
BLOCK_EOP: EOLF EOLF+;  
SECTION_TITLE_START:  
(  
    '=='  
    | '#####'  
    | '===='  
    | '#####'  
    | '===='  
    | '####'  
    | '=====  
    | '###'  
    | '=====  
    | '##'  
) WS_CHAR+ -> pushMode(SECTION_TITLE_MODE)  
;  
/////////////////  
mode SECTION_TITLE_MODE;  
SECTION_ID: '[' ~[\r\n\]]+ '];  
SECTION_TITLE_CONTENT: ~[\r\n[]+;  
SECTION_TITLE_EOL: EOLF -> popMode;  
/////////////////  
mode BLOCK_ATTR;  
BLOCK_ATTR_ID: ATTR_ID_F;  
BLOCK_ATTR_COMMA: WS_CHAR* COMMA WS_CHAR*;  
BLOCK_ATTR_ASSIGN: WS_CHAR* EQUAL -> pushMode(BLOCK_ATTR_VAL);  
BLOCK_ATTR_TYPE_ROLE: WS_CHAR* PERIOD;  
BLOCK_ATTR_TYPE_OPTION: WS_CHAR* PERCENT;  
BLOCK_ATTR_TYPE_ID: WS_CHAR* HASH;  
BLOCK_ATTR_UNSET: WS_CHAR* BANG;  
BLOCK_ATTR_END: WS_CHAR* '];
```

## D.1. CONFIGURATION

```
BLOCK_ATTR_EOL: WS_CHAR* EOLF -> popMode;
//////////
mode BLOCK_ATTR_VAL;
fragment BLOCK_ATTR_VALUE_QUOTED_SINGLE:
    SINGLE_QUOTE (~['\r\n] | (ESC SINGLE_QUOTE))*? SINGLE_QUOTE
;
fragment BLOCK_ATTR_VALUE_QUOTED_DOUBLE:
    DOUBLE_QUOTE (~["\r\n] | (ESC DOUBLE_QUOTE))*? DOUBLE_QUOTE
;
fragment BLOCK_ATTR_VALUE_UNQUOTED: ~['"\],\r\n ]*?;
BLOCK_ATTR_VALUE: (
    BLOCK_ATTR_VALUE_QUOTED_SINGLE
    | BLOCK_ATTR_VALUE_QUOTED_DOUBLE
    | BLOCK_ATTR_VALUE_UNQUOTED
) -> popMode
;
//////////
mode BLOCK_TITLE_MODE;
BLOCK_TITLE_TEXT: ~[\r\n]+;
BLOCK_TITLE_EOL: EOLF -> popMode;
//////////
mode PPMODE;
PPD_ATTR_ID: ATTR_ID_F;
PPD_ATTR_SEP: WS_CHAR* [+ ,] WS_CHAR*;
PPD_CONTENT_SINGLELINE: '[' ~[\]\r\n]+ ']' EOLF -> popMode;
PPD_CONTENT_START: '[' EOLF -> mode(PPCONTENT);
//////////
mode PPCONTENT;
PPD_CONTENT: .*? EOLF 'endif::[]' EOLF+ -> popMode;
//////////
mode DELIM_CONTENT;
DELIM_BLOCK_LINE:
    {this.isBOL}? ~[\r\n]* EOLF {!this.isDelimBlockEnd()}?
;

```

## D.1. CONFIGURATION

```
DELIM_BLOCK_END:
    {this.isBOL}? ~[\r\n]+ EOLF+ {this.isDelimBlockEnd()}? ->
    popMode
;
```

---

### Code source D.3 – AsciiDocLexer.g4

Voici une grammaire algébrique avec règles sémantiques (fichier `AsciiDocParser.g4`) se retrouvant dans le code source D.4 pour traduire la description proposée épurée d'un modèle :

---

```
/*
 * Inspiré de https://github.com/venkatperi/asciidoc-antlr/blob/
 * master/src/main/antlr/AsciiDocLexer.g4
 */
parser grammar AsciiDocParser;
@header {
import ca.mbjolin.editor.process._1_translation.grammar.Symbol;
import ca.mbjolin.editor.process._1_translation.grammar.
    SymbolTable;
import ca.mbjolin.editor.process._1_translation.grammar.
    SymbolType;
import ca.mbjolin.editor.process._1_translation.grammar.
    SymbolAttribut;
}
@parser::members {
public SymbolTable symbolTable = new SymbolTable();
}
options {
    tokenVocab = AsciiDocLexer;
}
////////////////////
// start rule
asciidoc:
    header section* EOF {
```

## D.1. CONFIGURATION

```
Symbol doc = new Symbol(SymbolType.DOCUMENT, _localctx);
doc.addAttribut(SymbolAttribut.IDENTIFIANT, _localctx.header().
    doc_title_line());
symbolTable.add(_localctx, doc);
}
;
//////////
// doc header
header:
    (header_line | EOL)* doc_title_line author_rev? header_line*
    END_OF_HEADER
;
// You cannot have a revision line without an author line.
author_rev: authors revision?;
header_line: global_attr | pp_directive;
//////////
// doc title
doc_title_line:
    H0 //doc_title_def
;
//////////
// doc author / rev
authors: AUTHOR;
revision: REVISION;
//////////
// global_attrs
global_attr: ATTR_START attr_def ATTR_EOL;
attr_def: attr_unset ATTR_SEP | attr_set;
attr_set:
    attr_id ATTR_SEP attr_value? attr_uid? {
Symbol meta = new Symbol(SymbolType.META, _localctx);
meta.addAttribut(SymbolAttribut.IDENTIFIANT, _localctx.attr_uid()
    );
meta.addAttribut(SymbolAttribut.ETIQUETTE, _localctx.attr_id());
```

## D.1. CONFIGURATION

```
meta.addAttribut(SymbolAttribut.VALEUR, _localctx.attr_value());
symbolTable.add(_localctx, meta);
}
;
attr_unset: ATTR_UNSET attr_id | attr_id ATTR_UNSET;
attr_id: ATTR_ID;
attr_value: ATTR_VALUE;
attr_uid: ATTR_UID;
//////////
// doc preamble
preamble: body_item* SECTION_END;
//////////
// doc sections
section:
(
    section_header section_body {
        Symbol paragraphe = new Symbol(SymbolType.PARAGRAPHE,
            _localctx.section_body());
        symbolTable.add(_localctx.section_body(), paragraphe);
    } section* (CUSTOM_END_SECTION | SECTION_END | EOF)
) {
    Symbol section = new Symbol(SymbolType.SECTION, _localctx);
    section.addAttribut(SymbolAttribut.IDENTIFIANT, _localctx.
        section_header().section_title().section_id());
    section.addAttribut(SymbolAttribut.ETIQUETTE, _localctx.
        section_header().section_title());
    symbolTable.add(_localctx, section);
}
;
section_header: block_attr_line* section_title;
section_title:
    SECTION_TITLE_START SECTION_TITLE_CONTENT section_id?
    SECTION_TITLE_EOL+
;
;
```

## D.1. CONFIGURATION

```
section_id: SECTION_ID;
//////////
// element attributes
block_attr_line:
    BLOCK_ATTR_START block_attr (BLOCK_ATTR_COMMA block_attr)*
    BLOCK_ATTR_END BLOCK_ATTR_EOL
    | anchor
;
block_attr: block_named_attr | block_positional_attr;
block_positional_attr:
    block_pos_prefixed_attr*
    | block_attr_id //style etc
;
block_pos_prefixed_attr:
    block_pos_attr_role
    | block_pos_attr_id
    | block_pos_attr_id
;
block_pos_attr_role: BLOCK_ATTR_TYPE_ROLE block_attr_id;
block_pos_attr_id: BLOCK_ATTR_TYPE_ID block_attr_id;
block_pos_attr_option: BLOCK_ATTR_TYPE_OPTION block_attr_id;
block_named_attr:
    block_attr_id BLOCK_ATTR_ASSIGN block_attr_value
;
block_attr_id: BLOCK_ATTR_ID;
block_attr_value: BLOCK_ATTR_VALUE;
//////////
// section content
section_body: body_item*;
body_item:
    block_attr_line* block_title_line? (paragraph | delim_block)
;
paragraph:
    BLOCK_PARA {
```



## D.1. CONFIGURATION

```
Symbol contenu = new Symbol(SymbolType.CONTENU, _localctx);
//contenu.addAttribut(SymbolAttribut.IDENTIFIANT, "paragraph_id
");
contenu.addAttribut(SymbolAttribut.DESRIPTION, _localctx);
symbolTable.add(_localctx, contenu);
}

;
//////////
// block/element title mode
block_title_line: BLOCK_TITLE_START block_title;
block_title: BLOCK_TITLE_TEXT BLOCK_TITLE_EOL;
//////////
// delimited blocks
delim_block:
(
    sidebar_block
    | comment_block
    | fenced_block
    | example_block
    | listing_block
    | literal_block
    | pass_block
    | verse_block
    | table_block
    | anon_block
)

;
table_block:
    BLOCK_TABLE_START delim_block_content DELIM_BLOCK_END
;
comment_block:
    BLOCK_COMMENT_START delim_block_content DELIM_BLOCK_END
;
fenced_block:
```

## D.1. CONFIGURATION

```
    BLOCK_FENCED_START delim_block_content DELIM_BLOCK_END {
Symbol reglelogique = new Symbol(SymbolType.REGLELOGIQUE,
    _localctx);
reglelogique.addAttribut(SymbolAttribut.DESCRPTION, _localctx.
    delim_block_content());
symbolTable.add(_localctx, reglelogique);
    }
;
example_block:
    BLOCK_EXAMPLE_START delim_block_content DELIM_BLOCK_END
;
listing_block:
    BLOCK_LISTING_START delim_block_content DELIM_BLOCK_END
;
literal_block:
    BLOCK_LITERAL_START delim_block_content DELIM_BLOCK_END
;
pass_block:
    BLOCK_PASS_START delim_block_content DELIM_BLOCK_END {
Symbol reglelogique = new Symbol(SymbolType.REGLELOGIQUE,
    _localctx);
reglelogique.addAttribut(SymbolAttribut.DESCRPTIONF, _localctx
    .delim_block_content());
symbolTable.add(_localctx, reglelogique);
    }
;
verse_block:
    BLOCK_VERSE_START delim_block_content DELIM_BLOCK_END {
Symbol citation = new Symbol(SymbolType.CITATION, _localctx);
citation.addAttribut(SymbolAttribut.DESCRPTION, _localctx.
    delim_block_content());
symbolTable.add(_localctx, citation);
    }
;
```

## D.1. CONFIGURATION

```
sidebar_block:
    BLOCK_SIDEBAR_START delim_block_content DELIM_BLOCK_END
;
anon_block:
    BLOCK_ANON_START delim_block_content DELIM_BLOCK_END
;
delim_block_content: DELIM_BLOCK_LINE*;
////////////////////
// conditional pre-processor directives
pp_directive:
    PPD_START ppd_attr (PPD_ATTR_SEP ppd_attr)* ppd_content
;
ppd_attr: PPD_ATTR_ID;
ppd_content:
    PPD_CONTENT_SINGLELINE
    | PPD_CONTENT_START PPD_CONTENT
;
////////////////////
// anchor
anchor: BLOCK_ANCHOR;
```

---

### Code source D.4 – AsciiDocParser.g4

Voici des règles (fichier CustomRule.yml) se retrouvant dans le code source D.5 pour vérifier partiellement le modèle intermédiaire :

---

```
---
name: "fill uuid"
description: "Remplir le modèle avec des identifiants pour toutes
    les informations."
priority: 1
condition: "true"
actions:
    - "resultdoc.setDoc(RuleUtil.fillDocWithId(document));"
---
```

## D.1. CONFIGURATION

```
name: "clean id"
description: "La grammaire nécessite de mettre les ids de
  sections et de meta-infos entre []."
priority: 2
condition: "true"
actions:
  - "resultdoc.setDoc(RuleUtil.cleanId(resultdoc.getDoc()));"
---
name: "strip content"
description: "Il y a des contenus contenant du saut de ligne."
priority: 3
condition: "true"
actions:
  - "resultdoc.setDoc(RuleUtil.stripContent(resultdoc.getDoc())
    );"
---
name: "meta avec documentid"
description: "Remplir le modèle avec une méta-info sur document
  obligatoire."
priority: 4
condition: "!RuleUtil.validDocumentWithMeta(document, 'documentid
  ')"
actions:
  - "resultdoc.setDoc(RuleUtil.fillDocumentWithMeta(resultdoc.
    getDoc(), 'documentid'));"
---
name: "meta avec projetid"
description: "Remplir le modèle avec une méta-info projet
  obligatoire."
priority: 5
condition: "!RuleUtil.validProjetWithMeta(document, 'projetid')"
actions:
  - "resultdoc.setDoc(RuleUtil.fillProjetWithMeta(resultdoc.
    getDoc(), 'projetid'));"
```

## D.1. CONFIGURATION

---

### Code source D.5 – CustomRule.yml

Voici des règles (fichier PresenteModelisateur.stg) se retrouvant dans le code source D.6 pour présenter le modèle vérifié aux modélisateurs :

---

```
meta(identifiant, etiquette, valeur) ::= <<
:<etiquette>: <valeur> [<identifiant>]

>>

doc(identifiant, projet, metas, sections) ::= <<
<metas>
= <identifiant>

<sections>
>>

section(identifiant, etiquette, profondeur, paragraphes, sections
) ::= <<
<etiquette>[<identifiant>]<paragraphes><sections>
~~~~~

>>

paragraphe(contenus, reglelogiquex, citations) ::= <%
<contenus>
<reglelogiquex>
<citations>
%>

contenu(identifiant, description) ::= <<
<description>
>>
```

## D.1. CONFIGURATION

```
citation(identifiant, description, sources) ::= <<
[quote]
_____
<description>_____

>>

reglelogiqueN(identifiant, descriptionNaturelle, sources) ::= <<
```
<descriptionNaturelle>```

>>

reglelogiqueF(identifiant, descriptionFormelle, sources) ::= <<
++++
<descriptionFormelle>++++

>>

source(identifiant, description) ::= <<
<identifiant>"<description>"

>>
```

---

### **Code source D.6 – PresenteModelisateur.stg**

Voici des règles (fichier `PresenteApprobateur.stg`) se retrouvant dans le code source D.7 pour présenter le modèle vérifié aux approbateurs :

---

```
meta(identifiant, etiquette, valeur) ::= <<
:<etiquette>: <valeur>

>>

doc(identifiant, projet, metas, sections) ::= <<
<metas>
```

## D.1. CONFIGURATION

```
<sections>
>>

section(identifiant, etiquette, profondeur, paragraphes, sections
) ::= <<
<paragraphes><sections>
>>

paragraphe(contenus, reglelogiquex, citations) ::= <%
<reglelogiquex>
%>

contenu(identifiant, description) ::= <<
>>

citation(identifiant, description, sources) ::= <<
>>

reglelogiqueN(identifiant, descriptionNaturelle, sources) ::= <<
<descriptionNaturelle>
>>

reglelogiqueF(identifiant, descriptionFormelle, sources) ::= <<
<descriptionFormelle>
>>

source(identifiant, description) ::= <<
>>
```

---

**Code source D.7 – PresenteApprobateur.stg**

## D.2. UTILISATEUR

### D.2 Utilisateur

Voici un exemple d'un ensemble de règles sélectionnées (fichier `SelectionRules.json`) de la part de l'utilisateur se retrouvant dans le `code source D.8` pour indiquer à l'IAR les descriptions formelles à appliquer :

---

```
[
  {
    "etape": "epure",
    "transformation": "AsciidocMacro",
    "intrant": "description1",
    "extrant": "epure1"
  },
  {
    "etape": "epure",
    "transformation": "AsciidocMacro",
    "intrant": "description2",
    "extrant": "epure2"
  },
  {
    "etape": "traduit",
    "transformation": "Asciidoc",
    "intrant": "epure1",
    "extrant": "mmd"
  },
  {
    "etape": "traduit",
    "transformation": "Asciidoc",
    "intrant": "epure2",
    "extrant": "mmd"
  },
  {
    "etape": "assure",
    "transformation": "CustomRule",
    "intrant": "mmd",
```



### D.3. DESCRIPTION PROPOSÉE D'UN MODÈLE

```
    "extrant": "mmdassure"
  },
  {
    "etape": "atteste",
    "transformation": "Relationnal",
    "intranant": "mmdassure",
    "extrant": "mmdatteste"
  },
  {
    "etape": "presente",
    "transformation": "PresenteModelisateur",
    "intranant": "mmdatteste",
    "extrant": "mmdpresente"
  }
}]
```

---

**Code source D.8** – SelectionRules.json

## D.3 Description proposée d'un modèle

Voici un exemple d'une description proposée d'un modèle (fichier `ModeleDescription.adoc`) se retrouvant dans le **code source D.9**, il s'agit de l'intrant principale :

---

```
:documentid: DocumentModelev1
:documentinfo: "Le document qui suit sert à des fins d'expé-
  rimentation."
:projetid: projetdescription1
:var1: vrai

= DocumentModele_v1

== SectionA
Voici un modèle.
ifdef::var1[]
une utilisation de macro var1 := {var1}
```

### D.3. DESCRIPTION PROPOSÉE D'UN MODÈLE

```
endif::var1[]
```

```
=== SectionAa  
[quote]
```

```
———  
Une citation.  
———
```

Règle informelle

```
```\nRegleA=Le fermier ne mange pas.\n```\n~~~~
```

```
=== SectionAb  
Autres règles formelles
```

```
++++  
open util/ordering[State] as ord  
abstract sig Object { eats: set Object }  
one sig Farmer, Fox, Chicken, Grain extends Object {}  
++++
```

```
++++  
fact eating { eats = Fox->Chicken + Chicken->Grain }  
++++
```

```
++++  
sig State {  
    near: set Object,  
    far: set Object  
}  
++++
```

## D.4. REPRÉSENTATION STANDARDISÉE DU MODÈLE

~~~~

~~~~

---

**Code source D.9** – ModeleDescription.adoc

## D.4 Représentation standardisée du modèle

Voici un exemple d'une représentation standardisée du modèle (fichier **ModelePresentation.adoc**) se retrouvant dans le code source D.10, dont le rôle est d'informer l'utilisateur du modèle extrait par l'IAR :

---

```
:documentinfo: "Le document qui suit sert à des fins d'expé-
    rimentation."
:var1: vrai
:documentid: DocumentModelev1
:projetid: projetdescription1

RegleA=Le fermier ne mange pas.
sig State {
    near: set Object,
    far: set Object
}
fact eating { eats = Fox->Chicken + Chicken->Grain }
open util/ordering[State] as ord
abstract sig Object { eats: set Object }
one sig Farmer, Fox, Chicken, Grain extends Object {}
```

---

**Code source D.10** – ModelePresentation.adoc

## D.5. JOURNAL

### D.5 Journal

Voici un exemple du journal (fichier `Journal.txt`) se retrouvant dans le code source D.11, dont le rôle est d’informer l’utilisateur du fonctionnement de l’IAR :

---

```
2026-08-02T13:30:40.30 - INFO - Début de la lecture de la
    configuration.
2026-08-02T13:30:40.31 - TRACE - Voici les transformations : X
2026-08-02T13:30:40.31 - INFO - Fin de la lecture de la
    configuration.
2026-08-02T13:30:40.31 - INFO - Début du traitement de l’é
    puration.
2026-08-02T13:30:40.31 - INFO - Fin du traitement de l’épuration.
2026-08-02T13:30:40.31 - INFO - Début du traitement de
    formalisation.
2026-08-02T13:30:40.31 - INFO - Début de la traduction.
2026-08-02T13:30:40.31 - TRACE - Transformation (identifiant):
    AsciiDoc
2026-08-02T13:30:40.31 - TRACE - Intrant 1 (nbr de caractère):694
2026-08-02T13:30:40.35 - TRACE - Extrant 1 (identifiant):=
    DocumentModele_v1
2026-08-02T13:30:40.35 - INFO - Fin de la traduction.
2026-08-02T13:30:40.35 - INFO - Début de la vérification
    partielle et ajustement.
2026-08-02T13:30:40.42 - INFO - Fin de la vérification partielle
    et ajustement.
2026-08-02T13:30:40.42 - INFO - Début de la vérification totale.
2026-08-02T13:30:41.06 - INFO - Fin de la vérification totale.
2026-08-02T13:30:41.06 - TRACE - Nombre de règle logique : 4
2026-08-02T13:30:41.06 - TRACE - Nombre de citation : 1
2026-08-02T13:30:41.06 - INFO - Début du traitement de pré
    sentation.
2026-08-02T13:30:41.06 - TRACE - Nom de la transformation :
    PresenteApprobateur
```

## D.5. JOURNAL

```
2026-08-02T13:30:41.08 - TRACE - Intranant (identifiant):  
    DocumentModelev1  
2026-08-02T13:30:41.08 - TRACE - Extranant (nbr de caractère): 413  
2026-08-02T13:30:41.08 - INFO - Fin du traitement de présentation  
.
```

---

**Code source D.11** – Journal.txt

## D.5. JOURNAL